Hardware Reference Manual

for Ezairo® 7100

M-20786-022
November 2016

© SCILLC, 2015
Previous Edition © 2014
"All Rights Reserved"

ON Semiconductor®

Hardware Reference Manual for Ezairo 7100

ON Semiconductor and  are registered trademarks of Semiconductor Components Industries, LLC (SCILLC). ARM, the ARM logo and Cortex are trademarks or registered trademarks of ARM Ltd. Ezairo is either a trademark or registered trademark of SCILLC. SCILLC owns the rights to a number of patents, trademarks, copyrights, trade secrets, and other intellectual property. A listing of SCILLC's product/patent coverage may be accessed at www.onsemi.com/site/pdf/Patent-Marking.pdf. SCILLC reserves the right to make changes without further notice to any products herein. SCILLC makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does SCILLC assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. "Typical" parameters which may be provided in SCILLC data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. SCILLC does not convey any license under its patent rights nor the rights of others. SCILLC products are not designed, intended, or authorized for use as components in systems intended for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the SCILLC product could create a situation where personal injury or death may occur. Should Buyer purchase or use SCILLC products for any such unintended or unauthorized application, Buyer shall indemnify and hold SCILLC and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that SCILLC was negligent regarding the design or manufacture of the part. SCILLC is an Equal Opportunity/ Affirmative Action Employer. This literature is subject to all applicable copyright laws and is not for resale in any manner.

PUBLICATION ORDERING INFORMATION

LITERATURE FULFILLMENT:
Literature Distribution Center for ON Semiconductor
P.O. Box 5163, Denver, Colorado 80217 USA
Phone: 303-675-2175 or 800-344-3860 Toll Free USA/Canada
Fax: 303-675-2176 or 800-344-3867 Toll Free USA/Canada
Email: orderlit@onsemi.com

N. American Technical Support: 800-282-9855 Toll Free USA/Canada
Europe, Middle East and Africa Technical Support: Phone: 421 33 790 2910
Japan Customer Focus Center: Phone: 81-3-5817-1050

ON Semiconductor Website: www.onsemi.com

Order Literature: http://www.onsemi.com/orderlit

For additional information, please contact your local Sales Representative

Table of Contents

Page

1. Introduction 20
   1.1 Purpose 20
   1.2 Intended Audience 20
   1.3 Conventions 20
   1.4 Manual Organization 21
   1.5 Further Reading 22
2. System Overview 23
   2.1 System Architecture Introduction 23
   2.2 CFX Digital Signal Processor 24
      2.2.1 CFX Memory. 24
   2.3 HEAR Configurable Accelerator 24
      2.3.1 HEAR Memory 25
   2.4 ARM Cortex-M3 Processor 25
      2.4.1 ARM Cortex-M3 Processor Memory 25
   2.5 Filter Engine 26
      2.5.1 Filter Engine Memory 26
   2.6 FIFO Controller 26
   2.7 Peripherals 27
      2.7.1 CFX Peripherals 27
      2.7.2 ARM Cortex-M3 Processor Peripherals 27
   2.8 Interfaces 27
      2.8.1 CFX Interfaces 27
      2.8.2 ARM Cortex-M3 Core Interfaces 28
   2.9 Inputs and Outputs 28
      2.9.1 Input/Output Controller (IOC) and Audio Multiplexer 28
      2.9.2 Input Stage 29
      2.9.3 Output Stage 29
   2.10 Analog Support Blocks 29
      2.10.1 Oscillator and Clocking 29
      2.10.2 Power Supply, Power Management and Reset Handling 29
   2.11 Device Identification 30
3. CFX Digital Signal Processor. 31
   3.1 Processor Overview 31
   3.2 Application Overview 31
4. HEAR Configurable Accelerator. 34
   4.1 Accelerator Overview 34
      4.1.1 Function Chains 34
      4.1.2 HEAR Control 34
      4.1.3 Control and Configuration Registers 35
         4.1.3.1 D_SYSTEM_CTRL Settings. 35
   4.2 Function Chain Controller 36
      4.2.1 Control and Configuration Registers 36
         4.2.1.1 D_HEAR_FC_ENABLE Settings 37

4.2.1.2 D_HEAR_FC_PRIORITY Settings . . . 38
4.2.1.3 D_HEAR_FC_STATUS Settings . . . 40
4.2.1.4 D_HEAR_CMDS_0_TO_7 Settings . . . 41
4.2.1.5 D_HEAR_CMDS_8_TO_15 Settings . . . 42
4.3 Block Floating-Point . . . 43
4.3.1 Control and Configuration Registers . . . 43
4.3.1.1 D_HEAR_BFP Settings . . . 43
4.3.1.2 D_HEAR_BFP_BLOCK_EXP Settings . . . 44
5. ARM Cortex-M3 Processor . . . 45
5.1 Processor Overview . . . 45
5.1.1 ARM Cortex-M3 Processor Integration and Control . . . 46
5.2 ARM Cortex-M3 Processor Debugging . . . 47
5.3 ARM Cortex-M3 Control and Configuration Register Conventions . . . 47
5.4 Control and Configuration Registers . . . 47
5.4.1 D_SYSTEM_CTRL_2 Settings . . . 48
5.4.2 CFX_INT_CMD, C_CFX_INT_CMD Settings . . . 48
5.4.3 D_CM3_CMDS Settings . . . 49
5.4.4 D_CM3_NMI Settings . . . 49
6. Filter Engine . . . 50
6.1 Processor Overview . . . 50
6.2 Filter Engine in the Audio Path . . . 50
6.3 Filter Engine Control and Configuration . . . 51
6.3.1 Filter Engine Control . . . 51
6.3.1.1 Master Control . . . 51
6.3.1.2 Single Step . . . 51
6.3.1.3 Pause . . . 51
6.3.2 Filter Engine Reset . . . 51
6.3.3 Filter Engine Manual Execution . . . 52
6.3.4 Filter Engine Configuration . . . 52
6.3.5 Filter Engine Filter Coefficients . . . 53
6.3.6 Multi-Rate Counter . . . 53
6.3.7 Phase Memory . . . 54
6.4 Filter Engine and CFX Interface . . . 54
6.4.1 Filter Engine Interrupt . . . 54
6.4.2 Memory and Register Access Conflicts . . . 55
6.5 Control and Configuration Registers . . . 56
6.5.1 Data Register Groups . . . 56
6.5.1.1 Extended-Precision Data Registers . . . 56
6.5.1.2 Single-Precision Data Registers . . . 56
6.5.1.3 Input Data Registers . . . 56
6.5.1.4 Output Data Registers . . . 57
6.5.2 Filter Engine Registers . . . 57
6.5.2.1 D_FENG_INT_STATUS Settings . . . 59
6.5.2.2 D_FENG_INT_ENABLE Settings . . . 63
6.5.2.3 D_FENG_CFG Settings . . . 65

6.5.2.4 D_FENG_MRCNT Settings . . . 67
6.5.2.5 D_FENG_PGM_SECTOR_EN Settings . . . 67
6.5.2.6 D_FENG_PHM_PH0_PH1 Settings . . . 68
6.5.2.7 D_FENG_PHM_PH2_PH3 Settings . . . 68
6.5.2.8 D_FENG_PHM_PH4_PH5 Settings . . . 68
6.5.2.9 D_FENG_PHM_PH6_PH7 Settings . . . 68
6.5.2.10 D_FENG_PHM_PH8_PH9 Settings . . . 68
6.5.2.11 D_FENG_PHM_PH10_PH11 Settings . . . 69
6.5.2.12 D_FENG_PHM_PH12_PH13 Settings . . . 69
6.5.2.13 D_FENG_PHM_PH14_PH15 Settings . . . 69
6.5.2.14 D_FENG_COEFF_OFFSET_EN_SHADOW Settings . . . 69
6.5.2.15 D_FENG_COEFF_OFFSET_EN_ACTIVE Settings . . . 69
6.5.2.16 D_FENG_STATE_W_EN Settings . . . 69
6.5.2.17 D_FENG_STATE_ACTIVE Settings . . . 70
6.5.2.18 D_FENG_STATE_SHADOW Settings . . . 70
6.5.2.19 D_FENG_CMDS Settings . . . 70
6.5.2.20 D_FENG_PGM_ADDR Settings . . . 70
6.5.2.21 D_FENG_CTRL Settings . . . 70
6.5.2.22 D_FENG_RF_EXTENDED_* Settings . . . 71
6.5.2.23 D_FENG_RF_SINGLE_* Settings . . . 71
6.5.2.24 D_FENG_RF_IN_* Settings . . . 71
6.5.2.25 D_FENG_RF_OUT_* Settings . . . 71
7. Memory . . . 72
7.1 Introduction . . . 72
7.2 Memory Architecture . . . 72
7.2.1 Memory Instances . . . 74
7.2.1.1 CFX Memory Block Instances . . . 75
7.2.1.2 HEAR Memory Block Instances . . . 76
7.2.1.3 ARM Cortex-M3 Core Memory Block Instances . . . 76
7.2.2 Memory Bus Structures . . . 76
7.2.3 Memory Controllers and Arbitration . . . 77
7.2.3.1 Shared Memory Bus Controller . . . 78
7.2.3.2 Control and Configuration Registers . . . 78
7.2.4 Memory Interface Write Buffers . . . 78
7.2.4.1 Buffering and Flushing of Write Operations . . . 78
7.2.4.2 Visible Effects of Write Buffering . . . 79
7.3 Program and P Memory . . . 80
7.3.1 Program Memory Space . . . 80
7.3.2 P Memory Space . . . 80
7.3.3 Program Memory (RAM and ROM) . . . 81
7.3.4 Microcode Memory (RAM) . . . 81
7.3.5 ARM Cortex-M3 Processor Program Memory (RAM) . . . 81
7.3.6 ARM Cortex-M3 Processor Data Memory (RAM) . . . 81
7.3.7 Filter Engine Memory (RAM) . . . 81
7.3.8 Program Memory Cache . . . 82

7.3.9 P Memory Space Memory Map . . . 83
7.3.9.1 Interrupt Vector Mapping . . . 84
7.3.9.2 Configuration Register Mapping . . . 85
7.4 Data Memory . . . 86
7.4.1 CFX Data Memories (RAM and ROM) . . . 86
7.4.2 Shared Memory (RAM and ROM) . . . 87
7.4.3 Data Memory Space Memory Maps . . . 88
7.4.3.1 HEAR and FIFO Configuration Register Mapping . . . 89
7.4.4 Control and Configuration Registers . . . 91
7.4.4.1 D_MEM_CFG_0 Settings . . . 91
7.4.4.2 D_MEM_CFG_1 Settings . . . 91
7.4.4.3 D_MEM_CFG_2 Settings . . . 92
7.4.4.4 D_MEM_CFG_3 Settings . . . 93
7.4.4.5 D_MEM_CFG_4 Settings . . . 94
7.4.4.6 D_MEM_CFG_5 Settings . . . 94
7.5 ARM Cortex-M3 Processor Memory Space . . . 95
7.5.1 ARM Cortex-M3 Processor Program and Data Memories (RAM) . . . 95
7.5.2 ARM Cortex-M3 Processor Access to HEAR FIFO Memories . . . 95
7.5.3 ARM Cortex-M3 Processor Memory Map . . . 96
7.5.3.1 ARM Cortex-M3 Processor Configuration Register Mapping . . . 96
8. FIFO Controller . . . 98
8.1 Introduction to the FIFO Controller . . . 98
8.1.1 FIFO Control and Configuration Register Conventions . . . 98
8.2 Defining Memory for Use as a FIFO . . . 98
8.2.1 Control and Configuration Registers . . . 100
8.2.1.1 D_FIFO_x_y_START Settings . . . 100
8.2.1.2 D_FIFO_x_y_END Settings . . . 100
8.2.1.3 D_FIFO_x_y_SIZE Settings . . . 100
8.3 FIFO Buffer Data and Configuration . . . 101
8.3.1 FIFO Input and Output Block . . . 101
8.3.2 FIFO Processing Block . . . 103
8.3.3 Typical FIFO Configurations . . . 104
8.3.3.1 Input FIFO Configuration . . . 104
8.3.3.2 Output FIFO Configuration . . . 105
8.3.4 Control and Configuration Registers . . . 106
8.3.4.1 D_FIFO_x_y_BASE_PTR . . . 106
8.3.4.2 D_FIFO_x_y_IOBLOCK_PTR . . . 106
8.3.4.3 D_FIFO_x_y_BLOCK_SIZE . . . 107
8.3.4.4 D_FIFO_x_y_BLOCK_CNT . . . 107
8.3.4.5 D_FIFO_x_y_ACCESS . . . 107
8.4 FIFO Interrupt Configuration . . . 107
8.4.1 Control and Configuration Registers . . . 107
8.4.1.1 D_FIFO_INT_CFX_* Settings . . . 108
8.4.1.2 D_FIFO_INT_HEAR_* Settings . . . 109
8.4.1.3 D_FIFO_INT_CM3_0 . . . 110

8.5 Access Priority . . . 110
9. Input/Output Audio Control . . . 111
9.1 Audio Multiplexing . . . 111
9.1.1 Audio Multiplexing with Filter Engine Outputs . . . 113
9.1.2 Audio Multiplexing of Audio Input Sources . . . 114
9.1.3 Control and Configuration Registers . . . 114
9.1.3.1 A_AUDIO_MUX_CFG Settings . . . 115
9.1.3.2 A_BYPASS_DATA_REG_0 . . . 116
9.1.3.3 A_BYPASS_DATA_REG_1 . . . 116
9.1.3.4 A_BYPASS_DATA_REG_2 . . . 117
9.1.3.5 A_BYPASS_DATA_REG_3 . . . 117
9.2 Introduction to the Input/Output Controller . . . 117
9.3 IOC Configuration . . . 118
9.4 IOC Input . . . 118
9.4.1 Control and Configuration Registers . . . 120
9.4.1.1 D_IOC_INPUT_ADC_CFG Settings . . . 120
9.4.1.2 D_IOC_INPUT_PCM_CFG Settings . . . 124
9.4.1.3 D_IOC_FIFO_ACCESS_CFG Settings . . . 126
9.4.1.4 D_IOC_FIFO_SHIFT_CFG Settings . . . 129
9.4.1.5 D_ADC0_DATA Settings . . . 130
9.4.1.6 D_ADC1_DATA Settings . . . 130
9.4.1.7 D_ADC2_DATA Settings . . . 130
9.4.1.8 D_ADC3_DATA Settings . . . 130
9.5 IOC Output . . . 130
9.5.1 Automatic FIFO Mute . . . 131
9.5.2 Control and Configuration Registers . . . 132
9.5.2.1 D_IOC_OUTPUT_CFG Settings . . . 133
9.5.2.2 D_DAC0_DATA Settings . . . 137
9.5.2.3 D_DAC1_DATA Settings . . . 137
9.5.2.4 D_FIFO_x_y_AUTO_MUTE Settings . . . 137
9.6 IOC Data Transfer Timing . . . 138
10. Input Stage . . . 140
10.1 Input Stage Architecture . . . 140
10.2 Input Selection . . . 142
10.3 Preamplifiers . . . 142
10.3.1 Low-Pass Filters . . . 143
10.4 Line-out . . . 144
10.5 Analog-to-Digital Converters (ADCs) . . . 144
10.5.1 DC Offset Configuration . . . 145
10.5.2 Sampling Delay . . . 145
10.5.3 Input Stage Control and Configuration Registers . . . 146
10.5.3.1 A_ADC_CTRL_* Settings . . . 147
10.5.3.2 A_ADC_CFG_* Settings . . . 148
10.5.3.3 A_ADC_SAMPLE_FREQ_CFG Settings . . . 149
10.6 Digital Microphone Inputs . . . 149

10.7 Decimation Filtering . . . 150
10.7.1 DC Removal Filters . . . 152
10.7.2 Multi-Chip Synchronization . . . 152
10.7.3 Frequency-Shift Keying . . . 152
10.7.4 Control and Configuration Registers . . . 153
10.7.4.1 A_ADC_DEC_CTRL_* Settings . . . 153
10.7.4.2 D_MULTI_SYNC_CFG Settings . . . 154
10.8 Input Dynamic Range Extension . . . 155
10.8.1 Description . . . 155
10.8.2 Configuration . . . 156
10.8.3 Control and Configuration Registers . . . 157
10.8.3.1 A_IDRX_CFG_* Settings . . . 157
11. Output Stage . . . 164
11.1 Output Stage Architecture . . . 164
11.2 Interpolation Filtering . . . 164
11.2.1 Control and Configuration Registers . . . 165
11.2.1.1 A_O_DAC_GF_CTRL Settings. . . . 165
11.3 Sigma-Delta Modulator . . . 165
11.3.1 Advanced Sigma-Delta Modulator Configuration . . . 166
11.3.1.1 Control and Configuration Registers . . . 167
11.3.1.1.1 A_O_SDM_CTRL . . . 167
11.4 Direct-Digital Output Driver . . . 168
11.4.1 High-Power Output Mode . . . 168
11.4.2 Control and Configuration Registers . . . 169
11.4.2.1 A_OUTPUT_CTRL Settings . . . 169
11.5 Digital Microphone Output . . . 170
11.5.1 Control and Configuration Registers . . . 171
11.5.1.1 A_DMIC_OUTPUT_CTRL Settings . . . 171
12. CFX Shared PCM and I2S™ Interface . . . 172
12.1 PCM Interface Introduction . . . 172
12.1.1 PCM Interface Configuration . . . 172
12.2 PCM Signal Configuration . . . 173
12.2.1 Frame Signal Configuration and Timing . . . 174
12.2.2 Data Serial Input and Output Configuration . . . 177
12.3 PCM Communications Management . . . 178
12.3.1 PCM Data Management . . . 178
12.3.2 PCM Interrupt Configuration . . . 178
12.3.3 Control and Configuration Registers . . . 179
12.3.3.1 D_PCM_CTRL Settings . . . 179
12.4 I2S Configuration and Usage . . . 179
12.5 NOAHlink Digital Audio Streaming Configuration and Usage . . . 182
12.6 PCM Interface Control and Configuration Registers . . . 183
12.6.1 D_PCM_CFG Settings . . . 183
12.6.2 D_PCM_EXT_CFG . . . 185

13. Debug Ports and CFX General-Purpose I2C . . . 188
13.1 Inter-IC Interface (I2C) Bus® . . . 188
13.2 CFX Debug Port I2C Interface . . . 191
13.2.1 Debug Port Configuration. . . . 191
13.2.2 Breakpoints . . . 191
13.2.3 External Breakpoint Configuration . . . 192
13.2.3.1 Data Modes . . . 192
13.2.3.2 Double-Address Mode . . . 194
13.2.3.3 Memory Conflict Mode . . . 194
13.2.4 CFX Debug Port Specific Control and Configuration Registers . . . 195
13.2.4.1 D_DEBUG_CFG Settings . . . 195
13.2.4.2 D_DEBUG_DATA. . . . 196
13.2.4.3 D_EXT_BREAK_CFG Settings . . . 196
13.2.4.4 D_EXT_BREAK_DATA Settings . . . 197
13.2.4.5 D_I2C_CFX_DEBUG_ADDR1 . . . 197
13.2.4.6 D_I2C_CFX_DEBUG_ADDR2 . . . 198
13.2.4.7 D_I2C_CM3_DEBUG_ADDR . . . 198
13.3 ARM Cortex-M3 Core Debug Port I2C Interface . . . 198
13.3.1 Halting Debug Configuration and Status . . . 199
13.3.2 Debug Monitor Configuration . . . 200
13.3.3 ARM Cortex-M3 Processor Core Register Access . . . 200
13.3.4 Debug Fault Status Register . . . 200
13.3.5 ARM Cortex-M3 Debug Port Specific Control and Configuration Registers . . . 201
13.3.5.1 SCB_DFSR Settings . . . 202
13.3.5.2 CoreDebug_DHCSR Settings . . . 202
13.3.5.3 CoreDebug_DCRSR Settings . . . 202
13.3.5.4 Debug Exception and Monitor Control Register. . . . 203
13.4 General Purpose I2C Interface . . . 203
13.4.1 General Purpose I2C Interface - Slave Mode Specific Configuration . . . 205
13.4.2 General Purpose I2C Interface - Master Mode Specific Configuration . . . 206
13.4.3 General Purpose I2C Interrupts . . . 207
13.4.3.1 Operation Using Manual Acknowledgement. . . . 207
13.4.3.2 Operation Using Auto Acknowledgement . . . 208
13.4.4 General Purpose I2C Interface Specific Control and Configuration Registers . . . 208
13.4.4.1 D_I2C_CFG Settings . . . 209
13.4.4.2 D_I2C_CTRL Settings . . . 210
13.4.4.3 D_I2C_ADDR_START Settings . . . 211
13.5 Control and Configuration Registers . . . 212
13.5.1 D_I2C_PIN_CFG Settings . . . 212
13.5.2 D_I2C_WATCHDOG_CFG Settings . . . 212
13.5.3 D_I2C_ERROR_CNT. . . . 213

14. CFX Peripherals. . . . 218
14.1 Interrupt Controller . . . 218
14.1.1 Control and Configuration Registers . . . 221
14.1.1.1 D_HEAR_INT_SELECT Settings . . . 221

14.1.1.2 D_INT_ENABLE Settings . . . 222
14.1.1.3 D_INT_STATUS Settings . . . 224
14.1.1.4 D_INT_PRIORITY Settings . . . 225
14.2 General Purpose Timers . . . 228
14.2.1 Control and Configuration Registers . . . 229
14.2.1.1 D_TIMER_*_CFG Settings . . . 229
14.3 Watchdog Timer . . . 230
14.3.1 Watchdog Interrupt . . . 231
14.3.2 Control and Configuration Registers . . . 231
14.3.2.1 D_WATCHDOG_CFG Settings . . . 231
14.4 Cyclic Redundancy Code (CRC) Generator . . . 232
14.4.1 Control and Configuration Registers . . . 233
14.4.1.1 D_CRC_VALUE Settings . . . 233
14.4.1.2 D_CRC_ADD_1 Settings . . . 233
14.4.1.3 D_CRC_ADD_8 Settings . . . 233
14.4.1.4 D_CRC_ADD_16 Settings . . . 233
14.4.1.5 D_CRC_ADD_24 Settings . . . 233
14.4.1.6 D_CRC_ADD_32 Settings . . . 233
14.5 Access Mode Security Blocks . . . 234
14.6 Signal Detection Unit . . . 234
14.6.1 Control and Configuration Registers . . . 236
14.6.1.1 D_SDU_OUT_* Settings . . . 236
14.6.1.2 D_SDU_CFG Settings . . . 237
15. ARM Cortex-M3 Processor Peripherals . . . 239
15.1 Nested Vectored Interrupt Controller (NVIC) . . . 239
15.1.1 Interrupt Controller Type Register . . . 242
15.1.2 Interrupt Set Enable and Clear Enable Registers . . . 242
15.1.3 Interrupt Set-Pending Registers and Interrupt Clear-Pending Registers . . . 243
15.1.4 Active Bit Register . . . 244
15.1.5 Interrupt Priority Registers . . . 244
15.1.6 Interrupt Control State Register . . . 245
15.1.7 Vector Table Offset Register . . . 247
15.1.8 Application Interrupt and Reset Control Register . . . 247
15.1.9 System Control Register . . . 248
15.1.10 Configuration Control Register . . . 248
15.1.11 System Handler Priority Registers . . . 249
15.1.12 System Handler Control and State Register . . . 250
15.1.13 Fault Status Registers . . . 251
15.1.14 Software Trigger Interrupt Register . . . 253
15.1.15 NVIC Control and Configuration Registers . . . 253
15.1.15.1 SCnSCB_ICTR . . . 254
15.1.15.2 NVIC_ISER0 . . . 255
15.1.15.3 NVIC_ISER1 . . . 256
15.1.15.4 NVIC_ICER0 . . . 257

15.1.15.5 NVIC_ICER1 . . . 259
15.1.15.6 NVIC_ISPR0 . . . 260
15.1.15.7 NVIC_ISPR1 . . . 262
15.1.15.8 NVIC_ICPR0 . . . 263
15.1.15.9 NVIC_ICPR1 . . . 265
15.1.15.10 NVIC_IABR0 . . . 266
15.1.15.11 NVIC_IABR1 . . . 267
15.1.15.12 NVIC_IP0. . . . 268
15.1.15.13 NVIC_IP1. . . . 268
15.1.15.14 NVIC_IP2. . . . 269
15.1.15.15 NVIC_IP3. . . . 269
15.1.15.16 NVIC_IP4. . . . 269
15.1.15.17 NVIC_IP5. . . . 269
15.1.15.18 NVIC_IP6. . . . 269
15.1.15.19 NVIC_IP7. . . . 270
15.1.15.20 NVIC_IP8. . . . 270
15.1.15.21 NVIC_IP9. . . . 270
15.1.15.22 NVIC_IP10 . . . 270
15.1.15.23 SCB_CPUID . . . 270
15.1.15.24 SCB_ICSR . . . 271
15.1.15.25 SCB_VTOR . . . 271
15.1.15.26 SCB_AIRCR . . . 271
15.1.15.27 SCB_SCR. . . . 272
15.1.15.28 SCB_CCR . . . 272
15.1.15.29 SCB_SHP0 . . . 273
15.1.15.30 SCB_SHP1 . . . 273
15.1.15.31 SCB_SHP2 . . . 274
15.1.15.32 SCB_SHCSR. . . . 274
15.1.15.33 SCB_CFSR . . . 275
15.1.15.34 SCB_HFSR . . . 276
15.1.15.35 SCB_DFSR . . . 276
15.1.15.36 SCB_MMFAR . . . 276
15.1.15.37 SCB_BFAR . . . 276
15.1.15.38 NVIC_STIR . . . 276
15.1.15.39 CoreDebug_DHCSR . . . 276
15.1.15.40 CoreDebug_DCRSR . . . 277
15.1.15.41 CoreDebug_DCRDR . . . 278
15.1.15.42 CoreDebug_DEMCR . . . 278
15.2 SysTick . . . 278
15.2.1 Control and Configuration Registers . . . 279
15.2.1.1 SysTick_CTRL Settings . . . 279
15.2.1.2 SysTick_LOAD Settings . . . 280
15.2.1.3 SysTick_VAL Settings . . . 280
15.2.1.4 SysTick_CALIB Settings . . . 280
15.3 Error-Correction Coding Blocks . . . 280
15.3.1 Hamming and Interleaving Encoder . . . 281

15.3.2 Hamming and Interleaving Decoder . . . 284
15.3.3 Reed-Solomon Encoder . . . 285
15.3.3.1 Using the Reed-Solomon Encoder with Interrupts . . . 287
15.3.3.2 Using the Reed-Solomon Encoder with the DMA Controller . . . 288
15.3.4 Reed-Solomon Decoder . . . 288
15.3.4.1 Using the Reed-Solomon Decoder with Interrupts . . . 291
15.3.4.2 Using the Reed-Solomon Decoder with the DMA Controller . . . 291
15.3.5 Control and Configuration Registers . . . 292
15.3.5.1 ECB_INT_STATUS, C_ECB_INT_STATUS Settings . . . 294
15.3.5.2 ECB_INT_ENABLE, C_ECB_INT_ENABLE Settings . . . 295
15.3.5.3 ECB_CTRL, C_ECB_CTRL Settings . . . 296
15.3.5.4 ECB_RS_BLK_CNT, C_ECB_RS_BLK_CNT Settings . . . 298
15.3.5.5 ECB_CMD, C_ECB_CMD Settings . . . 299
15.3.5.6 ECB_RS_ENC_IN, C_ECB_RS_ENC_IN Settings . . . 299
15.3.5.7 ECB_RS_ENC_OUT, C_ECB_RS_ENC_OUT Settings . . . 299
15.3.5.8 ECB_RS_DEC_IN, C_ECB_RS_DEC_IN Settings . . . 299
15.3.5.9 ECB_RS_DEC_OUT, C_ECB_RS_DEC_OUT Settings . . . 299
15.3.5.10 ECB_RS_PTR, C_ECB_RS_PTR Settings . . . 299
15.3.5.11 ECB_HMI_IN0, C_ECB_HMI_IN0 Settings . . . 299
15.3.5.12 ECB_HMI_IN1, C_ECB_HMI_IN1 Settings . . . 300
15.3.5.13 ECB_HMI_IN2, C_ECB_HMI_IN2 Settings . . . 300
15.3.5.14 ECB_HMI_IN3, C_ECB_HMI_IN3 Settings . . . 300
15.3.5.15 ECB_HMI_OUT0, C_ECB_HMI_OUT0 Settings . . . 300
15.3.5.16 ECB_HMI_OUT1, C_ECB_HMI_OUT1 Settings . . . 300
15.3.5.17 ECB_HMI_OUT2, C_ECB_HMI_OUT2 Settings . . . 300
15.3.5.18 ECB_HMI_OUT3, C_ECB_HMI_OUT3 Settings . . . 300
15.3.5.19 ECB_IHM_IN0, C_ECB_IHM_IN0 Settings . . . 300
15.3.5.20 ECB_IHM_IN1, C_ECB_IHM_IN1 Settings . . . 300
15.3.5.21 ECB_IHM_IN2, C_ECB_IHM_IN2 Settings . . . 301
15.3.5.22 ECB_IHM_IN3, C_ECB_IHM_IN3 Settings . . . 301
15.3.5.23 ECB_IHM_OUT0, C_ECB_IHM_OUT0 Settings . . . 301
15.3.5.24 ECB_IHM_OUT1, C_ECB_IHM_OUT1 Settings . . . 301
15.3.5.25 ECB_IHM_OUT2, C_ECB_IHM_OUT2 Settings . . . 301
15.3.5.26 ECB_IHM_OUT3, C_ECB_IHM_OUT3 Settings . . . 301
15.3.5.27 ECB_STATUS, C_ECB_STATUS Settings . . . 301
15.3.5.28 ECB_IHM_ECP, C_ECB_IHM_ECP Settings . . . 303
15.4 Audio Codec Blocks . . . 303
15.4.1 G.722 Encoder . . . 303
15.4.1.1 G.722 Encoder QMF Filterbank . . . 304
15.4.1.2 G.722 SB-ADPCM Encoders . . . 305
15.4.1.3 G.722 Encoder Input and Output Interfaces . . . 306
15.4.2 G.722 Decoder . . . 307
15.4.2.1 G.722 SB-ADPCM Decoders . . . 308
15.4.2.2 G.722 Decoder QMF Filterbank . . . 309
15.4.2.3 G.722 Decoder Concealment Support . . . 310

15.4.2.4 G.722 Decoder Input and Output Interfaces . . . 310
15.4.3 CVSD Encoder . . . 311
15.4.3.1 CVSD Encoder Pre-Filter . . . 312
15.4.3.2 CVSD Encoder . . . 314
15.4.3.3 CVSD Encoder Input and Output Interfaces . . . 316
15.4.4 CVSD Decoder . . . 317
15.4.4.1 CVSD Decoder . . . 318
15.4.4.2 CVSD Decoder Post-Filter . . . 318
15.4.4.3 CVSD Decoder Input and Output Interfaces . . . 319
15.4.5 Control and Configuration Registers . . . 320
15.4.5.1 ACB_INT_STATUS, C_ACB_INT_STATUS Settings . . . 321
15.4.5.2 ACB_INT_ENABLE, C_ACB_INT_ENABLE Settings . . . 322
15.4.5.3 ACB_CTRL, C_ACB_CTRL Settings . . . 322
15.4.5.4 ACB_CMD, C_ACB_CMD Settings . . . 325
15.4.5.5 ACB_CVSD_ENC_IN, C_ACB_CVSD_ENC_IN Settings . . . 325
15.4.5.6 ACB_CVSD_ENC_OUT, C_ACB_CVSD_ENC_OUT Settings . . . 325
15.4.5.7 ACB_CVSD_DEC_IN, C_ACB_CVSD_DEC_IN Settings . . . 326
15.4.5.8 ACB_CVSD_DEC_OUT, C_ACB_CVSD_DEC_OUT Settings . . . 326
15.4.5.9 ACB_CVSD_DELTA, C_ACB_CVSD_DELTA Settings . . . 326
15.4.5.10 ACB_G722_ENC_IN, C_ACB_G722_ENC_IN Settings . . . 326
15.4.5.11 ACB_G722_ENC_OUT, C_ACB_G722_ENC_OUT Settings . . . 326
15.4.5.12 ACB_G722_DEC_IN, C_ACB_G722_DEC_IN Settings . . . 326
15.4.5.13 ACB_G722_DEC_OUT, C_ACB_G722_DEC_OUT Settings . . . 326
15.4.5.14 ACB_G722_DEC_FBIN, C_ACB_G722_DEC_FBIN Settings . . . 327
15.4.5.15 ACB_G722_DEC_PLT, C_ACB_G722_DEC_PLT Settings . . . 327
15.4.5.16 ACB_G722_DEC_RLT, C_ACB_G722_DEC_RLT Settings . . . 327
15.4.5.17 ACB_G722_DEC_SX, C_ACB_G722_DEC_SX Settings . . . 327
15.4.5.18 ACB_G722_DEC_LSB_QUANT, C_ACB_G722_DEC_LSB_QUANT Settings . . . 327
15.4.5.19 ACB_G722_DEC_HSB_QUANT, C_ACB_G722_DEC_HSB_QUANT Settings . . . 327
15.5 Direct Memory Access (DMA) Controller . . . 327
15.5.1 Introduction . . . 327
15.5.2 DMA Channel Configuration . . . 328
15.5.3 Word Size, Data Packing and Transfer Length . . . 330
15.5.3.1 Word Size . . . 330
15.5.3.2 Data Packing . . . 331
15.5.3.3 Transfer Length . . . 333
15.5.4 DMA Transfer Types . . . 333
15.5.4.1 Memory-to-Memory (MM) . . . 335
15.5.4.2 Memory-to-Peripheral . . . 335
15.5.4.3 Peripheral-to-Memory (PM) . . . 336
15.5.4.4 Peripheral-to-Peripheral (PP) . . . 336
15.5.5 DMA Interrupt Configuration . . . 337
15.5.6 Channel Priority . . . 338
15.5.7 Data Memory Usage by the DMA and ARM Cortex-M3 Processor . . . 338
15.5.8 DMA Registers . . . 339

15.5.8.1 DMA_CHx_CTRLy, C_DMA_CHx_CTRLy Settings . . . 340
15.5.8.2 DMA_CH*_SRC_BASE_ADDR, C_DMA_CH*_SRC_BASE_ADDR Settings . . . 344
15.5.8.3 DMA_CH*_DEST_BASE_ADDR, C_DMA_CH*_DEST_BASE_ADDR Settings . . 344
15.5.8.4 DMA_CH*_NEXT_SRC_ADDR, C_DMA_CH*_NEXT_SRC_ADDR Settings . . . 344
15.5.8.5 DMA_CH*_NEXT_DEST_ADDR, C_DMA_CH*_NEXT_DEST_ADDR Settings . . 344
15.5.8.6 DMA_CH*_WORD_CNT, C_DMA_CH*_WORD_CNT Settings . . . 344
15.5.8.7 DMA_STATUS, C_DMA_STATUS Settings . . . 345
15.6 Cyclic Redundancy Check (CRC) Generator . . . 346
15.6.1 CRC Registers . . . 348
15.6.1.1 CRC_CTRL, C_CRC_CTRL . . . 348
15.6.1.2 CRC_VALUE, C_CRC_VALUE . . . 349
15.6.1.3 CRC_ADD_1, C_CRC_ADD_1 . . . 349
15.6.1.4 CRC_ADD_8, C_CRC_ADD_8 . . . 349
15.6.1.5 CRC_ADD_16, C_CRC_ADD_16 . . . 349
15.6.1.6 CRC_ADD_24, C_CRC_ADD_24 . . . 349
15.6.1.7 CRC_ADD_32, C_CRC_ADD_32 . . . 350
16. CFX External Interfaces . . . 351
16.1 Digital I/O (DIO) . . . 351
16.1.1 Functional Configuration . . . 351
16.1.2 Physical Configuration . . . 353
16.1.3 Control and Configuration Registers . . . 354
16.1.3.1 D_DIO_CFG_* Settings . . . 355
16.1.3.2 D_PCM_SRC Settings . . . 356
16.1.3.3 D_SPI_SRC Settings . . . 359
16.1.3.4 D_UART_SRC Settings . . . 360
16.1.3.5 D_SYNC_SLAVE_SRC Settings . . . 361
16.1.3.6 D_DAC_SCALE_SRC Settings . . . 362
16.1.3.7 D_C_PCM_SRC Settings . . . 363
16.1.3.8 D_C_SPI_SRC Settings . . . 366
16.1.3.9 D_C_I2C_SRC Settings . . . 369
16.1.3.10 D_DMIC_SRC Settings . . . 371
16.1.3.11 D_PLL_SRC Settings . . . 375
16.2 General-Purpose I/O (GPIO) Interface . . . 376
16.2.1 GPIO Interrupt . . . 377
16.2.2 Control and Configuration Registers . . . 378
16.2.2.1 D_DIO_DATA Settings . . . 378
16.2.2.2 D_DIO_DIR Settings . . . 382
16.2.2.3 D_DIO_MODE Settings . . . 386
16.2.2.4 D_DIO_DATA_VDDO_* Settings . . . 388
16.2.2.5 D_CFX_DIO_INT_CFG_* . . . 389
16.2.2.6 D_CFX_DIO_INT_STATUS Settings . . . 390
16.3 Low-Speed Analog-to-Digital (LSAD) Converter Interface . . . 391
16.3.1 Control and Configuration Registers . . . 392
16.3.1.1 A_LSAD_CTRL Settings . . . 392
16.3.1.2 A_LSAD_SRC_CFG Settings . . . 394

16.4 Serial Peripheral Interface (SPI) . . . 395
16.4.1 SPI Interrupt . . . 396
16.4.2 Control and Configuration Registers . . . 396
16.4.2.1 D_SPI_CFG Settings . . . 396
16.4.2.2 D_SPI_CTRL Settings . . . 397
16.5 Universal Asynchronous Receiver-Transmitter (UART) Interface . . . 397
16.5.1 UART Interrupts . . . 398
16.5.2 Control and Configuration Registers . . . 398
16.5.2.1 D_UART_CFG Settings . . . 399
16.5.2.2 D_UART_DATA Settings . . . 399
16.5.2.3 D_UART_DATA_READY Settings . . . 399
17. ARM Cortex-M3 Processor External Interfaces . . . 400
17.1 I2C Interface . . . 400
17.1.1 ARM Cortex-M3 Processor I2C interface Register Mapping . . . 400
17.1.2 I2C Registers . . . 401
17.1.2.1 I2C_CTRL0, C_I2C_CTRL0 Settings . . . 401
17.1.2.2 I2C_CTRL1, C_I2C_CTRL1 Settings . . . 403
17.1.2.3 I2C_DATA, C_I2C_DATA Settings . . . 403
17.1.2.4 I2C_DATA_M, C_I2C_DATA_M Settings . . . 403
17.1.2.5 I2C_ADDR_START, C_I2C_ADDR_START Settings . . . 403
17.1.2.6 I2C_STATUS, C_I2C_STATUS Settings . . . 404
17.2 GPIO Interfaces . . . 405
17.2.1 GPIO Interrupts . . . 406
17.2.2 GPIO Interface Registers . . . 406
17.2.2.1 DIO_DATA, C_DIO_DATA Settings . . . 407
17.2.2.2 DIO_DIR, C_DIO_DIR Settings . . . 411
17.2.2.3 DIO_MODE, C_DIO_MODE Settings . . . 414
17.2.2.4 DIO_INT_CFG_*, C_DIO_INT_CFG_*, D_CM3_DIO_INT_CFG_* Settings . . . 417
17.2.2.5 DIO_INT_STATUS, C_DIO_INT_STATUS and D_CM3_DIO_INT_STATUS Settings . . . 418
17.3 PCM Interface . . . 419
17.3.1 PCM Interface Introduction . . . 419
17.3.2 PCM Signal Configuration . . . 420
17.3.3 PCM Interrupt Configuration . . . 421
17.3.4 PCM Registers . . . 421
17.3.4.1 PCM_CTRL, C_PCM_CTRL Settings . . . 422
17.3.4.2 PCM_TX_DATA, C_PCM_TX_DATA Settings . . . 423
17.3.4.3 PCM_RX_DATA, C_PCM_RX_DATA . . . 423
17.3.4.4 PCM_STATUS, C_PCM_STATUS Settings . . . 423
17.3.5 I2S Configuration and Usage . . . 424
17.4 SPI Interface . . . 426
17.4.1 SPI Interface Configuration . . . 426
17.4.2 SPI Interface Data Transfers . . . 426
17.4.3 SPI Interrupts . . . 428
17.4.4 SPI Registers . . . 428

17.4.4.1 SPI_CTRL0, C_SPI_CTRL0 Settings . . . 429
17.4.4.2 SPI_CTRL1, C_SPI_CTRL1 Settings . . . 430
17.4.4.3 SPI_TX_DATA, C_SPI_TX_DATA Settings . . . 431
17.4.4.4 SPI_RX_DATA, C_SPI_RX_DATA Settings . . . 431
17.4.4.5 SPI_STATUS, C_SPI_STATUS Settings . . . 431
18. Clock Components . . . 432
18.1 Clock Generation . . . 432
18.1.1 Primary Oscillator . . . 432
18.1.1.1 Voltage-Controlled Oscillator . . . 433
18.1.1.1.1 VCO Temperature Compensation . . . 434
18.1.1.2 Phase-Locked Loop . . . 435
18.1.1.2.1 PLL Configuration . . . 435
18.1.1.2.2 PLL Lock Monitoring . . . 436
18.1.1.2.3 PLL Reference Clock Configuration . . . 436
18.1.1.2.4 PLL Performance Configuration . . . 437
18.1.1.2.5 PLL Reference and SDMCLK Synchronization . . . 439
18.1.2 Standby Oscillator . . . 440
18.1.3 Control and Configuration Registers . . . 440
18.1.3.1 D_CLK_SYNC Settings . . . 440
18.1.3.2 A_PLL_CTRL_0 Settings . . . 441
18.1.3.3 A_PLL_CTRL_1 . . . 441
18.1.3.4 A_PLL_CTRL_2 Settings . . . 441
18.1.3.5 A_PLL_CTRL_3 Settings . . . 442
18.1.3.6 A_PLL_CTRL_4 Settings . . . 442
18.1.3.7 A_PLL_CTRL_5 Settings . . . 443
18.1.3.8 A_PLL_CTRL_6 Settings . . . 443
18.1.3.9 A_PLL_CTRL_7 Settings . . . 443
18.1.3.10 A_PLL_REF_CLK_DET_CTRL Settings . . . 444
18.2 Clock Distribution . . . 445
18.2.1 System Clock (SYSCLK) . . . 445
18.2.2 Slow Clock (SLOWCLK) . . . 446
18.2.3 Sigma-Delta Modulator Clock (SDMCLK) . . . 446
18.2.4 HEAR Accelerator Clock (HEARCLK) . . . 447
18.2.5 ARM Cortex-M3 Processor Clock (CM3CLK) . . . 448
18.2.6 External Clock (EXTCLK) . . . 448
18.2.7 User Clock (UCLK) . . . 448
18.3 Clock Throttling . . . 449
18.3.1 Multiplier-Based Clock Throttling . . . 449
18.3.2 Frequency-Based Clock Throttling . . . 450
18.3.2.1 Advanced Frequency-Based Clock Throttling Options . . . 451
18.4 Control and Configuration Registers . . . 452
18.4.1 D_CLK_CFG_0 Settings . . . 453
18.4.2 D_CLK_CFG_1 Settings . . . 456
18.4.3 D_CLK_CFG_CPA Settings . . . 458

18.4.4 D_CLK_CFG_CPD Settings . . . 458
18.4.5 D_CLK_CFG_CPC Settings . . . 459
18.4.6 D_CLK_CFG_CPM Settings . . . 459
18.4.7 D_SLOWCLK_THROTTLING Settings . . . 459
18.4.8 D_CLK_THROTTLING_CTRL Settings . . . 459
18.4.9 D_CLK_THROTTLING_CFG Settings . . . 459
18.4.10 D_CLK_MULT_THROTTLING_CFG Settings . . . 460
18.4.11 A_OSC_CTRL_0 Settings . . . 460
18.4.12 A_OSC_CTRL_1 Settings . . . 461
18.4.13 A_OSC_CTRL_2 Settings . . . 461
18.4.14 A_OSC_CTRL_3 Settings . . . 461
18.4.15 A_OSC_CTRL_THROTTLING Settings . . . 462
18.4.16 A_OSC_CTRL_THROTTLING_2 Settings . . . 462
18.4.17 A_OSC_CTRL_THROTTLING_3 Settings . . . 462
18.4.18 A_O_SDMCLK Settings . . . 462
18.4.19 A_O_SDMCLK_THROTTLING . . . 465
19. Power Components . . . 466
19.1 Power Supply . . . 466
19.1.1 Power Supply Inputs . . . 467
19.1.1.1 Battery Supply Voltage (VBAT). . . . 467
19.1.1.2 Output Receiver Supply Voltage (VBATOD) . . . 467
19.1.1.3 Digital Output Supply Voltages (VDDO1, VDDO2, and VDDO3) . . . 468
19.1.2 Internal Power Supply Voltages . . . 468
19.1.2.1 Internal Band Gap Reference Voltage (VREF) . . . 468
19.1.2.2 Internal Digital Supply Voltages (VDDC and VDDM) . . . 468
19.1.2.3 Regulated Supply Voltage (VREG). . . . 469
19.1.2.4 Analog Supply Voltage (VDDA) . . . 469
19.1.3 Power Supply Outputs . . . 470
19.1.3.1 Regulated Doubled Supply Voltage (VDBL). . . . 470
19.1.3.2 Microphone Supply Voltage (VMIC) and Microphone Ground Reference (GND_MIC) . 470
19.1.4 Control and Configuration Registers . . . 471
19.1.4.1 A_VREG_CTRL Settings . . . 471
19.1.4.2 A_BG_CTRL Settings . . . 471
19.1.4.3 A_VMIC_SW_CTRL Settings . . . 472
19.1.4.4 A_CP_VDDA_CTRL Settings . . . 473
19.1.4.5 A_CP_VDBL_CTRL Settings . . . 473
19.1.4.6 A_CP_VDDC_TRIM . . . 474
19.1.4.7 A_CP_VDDC_CTRL . . . 474
19.1.4.8 A_CP_VDDM_TRIM . . . 475
19.1.4.9 A_CP_VDDM_CTRL . . . 475
19.2 Power Management Unit . . . 476
19.2.1 Power Supervisory Unit . . . 476
19.2.2 Power-On-Reset (POR) Sequence . . . 478
19.3 Resets . . . 478

19.4 Battery Monitor . . . 479
19.4.1 Control and Configuration Registers . . . 479
19.4.1.1 A_SUP_THRSH_DATA Settings . . . 479
19.4.1.2 A_SUP_COUNT_DATA Settings . . . 480
19.5 Soft Power Down Mode Tools . . . 480
19.5.1 Control and Configuration Registers . . . 480
19.5.1.1 A_ANALOG_PWR_CTRL Settings . . . 481
A. Control and Configuration Registers . . . 482
A.1 Error Correction Coding (ECC) Configuration and Control . . . 484
A.2 Audio Codec Block (ACB) Configuration and Control . . . 488
A.3 DMA Controller Configuration and Control . . . 491
A.4 External Interrupt Configuration and Control . . . 498
A.5 DIO Configuration and Control . . . 499
A.6 SPI Interface Configuration and Control . . . 500
A.7 PCM Interface Configuration and Control . . . 501
A.8 I2C Interface Configuration and Control . . . 502
A.9 CRC Generator Control . . . 504
A.10 SYSTICK Timer . . . 505
A.11 Nested Vector Interrupt Controller . . . 506
A.12 Debug Controller . . . 521
A.13 System Control . . . 522
A.14 Watchdog Timer Configuration . . . 523
A.15 General Purpose Timers 1 and 2 . . . 524
A.16 HEAR Function Chain Commands . . . 525
A.17 HEAR Interrupt Multiplexing . . . 526
A.18 CFX/ARM Cortex-M3 Processor Control and Configuration . . . 527
A.19 Memory Configuration . . . 528
A.20 External Breakpoint Configuration . . . 530
A.21 Hardware CRC . . . 531
A.22 Chip Version Information (Read-only) . . . 532
A.23 Debug Port Configuration . . . 533
A.24 Extended Configuration . . . 534
A.25 Interrupt Configuration . . . 535
A.26 Clock Configuration . . . 544
A.27 IOC Input Configuration . . . 546
A.28 IOC FIFO Configuration . . . 547
A.29 IOC Output Configuration . . . 549
A.30 A/D and D/A Converter Access . . . 550
A.31 Oscillator Control . . . 551
A.32 Analog . . . 553
A.33 Charge Pump . . . 554
A.34 A/D Converter Control . . . 555
A.35 Output Stage Control . . . 559
A.36 Audio Mux Configuration . . . 561
A.37 DMIC Output Control . . . 562
A.38 Low-speed A/D . . . 563
A.39 Low-voltage Detection . . . 565
A.40 Low-voltage Count . . . 566

A.41 SPI Interface 567
A.42 I2C Interface 568
A.43 PCM Interface 570
A.44 DIO Interface 571
A.45 UART Interface 578
A.46 Filter Engine Control and Configuration 579
A.47 Filter Engine Register File 587
A.48 Signal Detection Unit 590
A.49 HEAR FIFO A0 Configuration and Control 591
A.50 HEAR FIFO A1 Configuration and Control 594
A.51 HEAR FIFO B0 Configuration and Control 597
A.52 HEAR FIFO B1 Configuration and Control 600
A.53 CFX FIFO Interrupt Selection 603
A.54 HEAR FIFO Interrupt Selection 604
A.55 ARM Cortex-M3 Core FIFO Interrupt Selection 605
A.56 HEAR Interrupt Configuration and Control 606
A.57 Block Floating Point Configuration and Control 610
B. Sampling Frequency Reference 612
C. Glossary 614

CHAPTER 1
Introduction

1.1 PURPOSE

This manual provides a reference to the system hardware for application developers working with Ezairo® 7100. This manual describes all of the components belonging to the Ezairo 7100 chip, including:

- Data processing and control components
- Memory components
- Peripherals
- External interfaces

The data processing and control component information provided by this manual complements:

- The CFX DSP description in the *CFX DSP Architecture Manual*
- The HEAR coprocessor description in the *HEAR Configurable Accelerator Reference Manual for Ezairo 7100*
- The ARM® Cortex™-M3 core description in the *The Definitive Guide to the ARM Cortex-M3* and other third-party documentation for the ARM Cortex-M3 processor
- The Filter Engine coprocessor description in the *Filter Engine Reference Manual for Ezairo 7100*

It further describes how each component of the Ezairo 7100 chip can be used in the implementation of an application, and provides information about the configuration of the various system components.

The *Hardware Reference Manual for Ezairo 7100* is a part of the Ezairo 7100 Evaluation and Development Kit (Ezairo 7100 EDK).

1.2 INTENDED AUDIENCE

This manual is primarily intended for engineers and other individuals who are responsible for developing and/or maintaining signal processing applications that make use of an Ezairo 7100 chip. People who are developing communications between an external device and Ezairo 7100, as well as those interested in the details of the audio components, will find this manual particularly helpful as it focuses on the configuration and use of system components such as the HEAR Accelerator core in the implementation of a signal processing application by the CFX DSP core.

This manual assumes that readers are familiar with assembly-level programming, digital signal processing (DSP) application concepts, the design of basic audio systems and DSP tools.

1.3 CONVENTIONS

The following typographical conventions are used in this manual:

- Control and configuration registers are shown in a `special font`. All control register names are prefixed to identify what type of entity the register controls or configures:

A	An analog entity on the chip.
C	A digital entity in the ARM Cortex-M3 processor portion of the chip.
D	A digital entity in other portions of the chip.

- Angle brackets “<” and “>” identify an optional parameter or indicate a placeholder for specific information. To use an optional parameter or replace a placeholder, specify the information within the brackets; do not include the brackets themselves.
- Default settings for registers and bit fields are marked with an asterisk (*).
- All sample rates specified are the final decimated sample rates, unless stated otherwise.
- In general, numbers are presented in decimal notation. In cases where hexadecimal or binary notation is more convenient, these numbers are identified by the prefixes “0x” and “0b” respectively. For example, the decimal number 123456 can also be represented as 0x1E240 or 0b11110001001000000.

1.4 Manual Organization

The *Hardware Reference Manual for Ezairo 7100* contains the following chapters and appendices:

- **Chapter 1: Introduction**, introduces this manual—explains the purpose, intended audience, conventions used and organization. It also contains a list of further reading materials that provide information on other aspects of application development using Ezairo 7100.
- **Chapter 2: System Overview**, provides a high-level view of all of the hardware components of the Ezairo 7100 system and how these components interact.
- **Chapter 3: CFX Digital Signal Processor**, focuses on the interaction of the CFX DSP with the rest of the system, and provides a model of data flow through the Ezairo 7100 system.
- **Chapter 4: HEAR Configurable Accelerator**, focuses on the configuration of the HEAR Accelerator using microcode, and details of the hardware features of the HEAR that can be configured by the CFX DSP to perform a variety of signal processing tasks.
- **Chapter 5: ARM Cortex-M3 Processor**, provides an overview of the general-purpose ARM Cortex-M3 processor, its use in the Ezairo 7100 system, and its interaction with the CFX DSP. It also describes its control and configuration registers.
- **Chapter 6: Filter Engine**, focuses on the configuration of the Filter Engine programs and details of the hardware features supporting the Filter Engine that can be configured by the CFX DSP to perform a variety of signal processing tasks.
- **Chapter 7: Memory**, provides details about the various memory instances, buses and other memory structures that are associated with the four processing cores.
- **Chapter 8: FIFO Controller**, provides details about the FIFO controller, including FIFO buffer configuration information and how the components of the Ezairo 7100 system use the FIFO buffers.
- **Chapter 9: Input/Output Audio Control**, describes how to configure the audio multiplexing and input/output controller (IOC). This chapter also provides detailed information about the use of the IOC in acquiring input data, and distributing output data.
- **Chapter 10: Input Stage**, provides information about the audio input stage of the Ezairo 7100 system.
- **Chapter 11: Output Stage**, provides information about the audio output stage of the Ezairo 7100 system.
- **Chapter 12: CFX Shared PCM and I2S™ Interface**, provides detailed information about the shared PCM and I^2S interface that either the CFX or the IOC can use to move signal data into and out of the Ezairo 7100 system.
- **Chapter 13: Debug Ports and CFX General-Purpose I2C**, provides information about the debug ports and the general-purpose I^2C interface that shares the same interface bus as the debug ports.
- **Chapter 14: CFX Peripherals**, provides information about the interrupt controller, timers and other peripherals that are provided for the CFX processor.
- **Chapter 15: ARM Cortex-M3 Processor Peripherals**, provides information about the interrupt controller, system tick timer, DMA, and hardware accelerators that are provided for the ARM Cortex-M3 processor.
- **Chapter 16: CFX External Interfaces**, provides information about the external interfaces (other than the PCM/I^2S and I^2C interfaces) available to the CFX processor that can be used to pass data between the Ezairo 7100 chip and external components.

- **Chapter 17: ARM Cortex-M3 Processor External Interfaces**, provides information about the external interfaces available to the ARM Cortex-M3 processor that can be used to pass data between the Ezairo 7100 chip and external components.
- **Chapter 18: Clock Components**, provides descriptions and configuration information for all of the clock generation and clock distribution components used by the Ezairo 7100 system.
- **Chapter 19: Power Components**, provides descriptions and configuration information for all of the power supply, power management and reset related components used by the Ezairo 7100 system.
- **Appendix A: Control and Configuration Registers**, contains a summary table listing all of the hardware control and configuration registers and their settings.
- **Appendix B: Sampling Frequency Reference**, provides a reference lookup table to help you to select a sampling frequency configuration for a given clock configuration, and the desired sampling frequency.
- **Appendix C: Glossary**, provides definitions for many of the terms defined and used throughout this document.

1.5 Further Reading

For more information, refer to the following documents that are included in the Ezairo 7100 EDK:

- *Integrated Development Environment User's Guide for Ezairo 7100*
- *Firmware Reference Manual for Ezairo 7100*
- *SK4 ELF Toolchain Reference for Ezairo 7100*
- *CFX DSP Architecture Manual*
- *HEAR Configurable Accelerator Reference Manual for Ezairo 7100*
- *Filter Engine Reference Manual for Ezairo 7100*
- *Ezairo 7100 Errata*

For more information about the ARM Cortex-M3 processor, refer to the following documents:

- *Cortex-M3 Technical Reference Manual*
- *ARM and Thumb-2 Instruction Set Quick Reference Card*
- ARM *Cortex Microcontroller Software Interface Standard (CMSIS)*
- Yiu, Joseph *The Definitive Guide to the ARM Cortex-M3* (Newnes, December 2009) or a similar book describing the ARM Cortex-M3 core architecture

You can find ARM publications on their website at http://infocenter.arm.com/help/index.jsp.

Other publicly available standards that are relevant to this manual are:

- ITU-T G.722 (09/2012)
- *Specification of the Bluetooth® System*: https://www.bluetooth.org/en-us/specification/adopted-specifications

CHAPTER 2
System Overview

2.1 System Architecture Introduction

The Ezairo 7100 system is a mixed-signal system-on-chip designed specifically for audio processing. It is centered on an asymmetric quad-core architecture, including four different processing cores: the CFX Digital Signal Processor (DSP), the HEAR Configurable Accelerator, the ARM Cortex-M3 processor, and the Filter Engine core.

- The CFX DSP core acts as the system master, controlling the other cores, peripherals, interfaces and analog components. It coordinates the flow of signal data progressing through the system. The CFX DSP can also be used for custom signal processing applications that are not handled by the HEAR or the Filter Engine.
- The HEAR Configurable Accelerator is designed to perform both common signal processing operations and complex standard filterbanks, such as a WOLA filterbank, reducing the load on the CFX DSP core.
- The ARM Cortex-M3 processor acts as a coprocessor to provide additional computation and support for external components with a focus on supporting wireless capability.
- The Filter Engine core is a coprocessor that provides basic filtering and arithmetic operations on audio signals, and is tightly integrated with the audio path to provide a low delay audio path.

The four cores have a number of local and shared memories available to them. The CFX DSP, the HEAR and the ARM Cortex-M3 core process data brought into or read from a set of first-in, first-out (FIFO) buffers by the input/output controller (IOC). The IOC can take inputs from an analog input stage, digital microphones or pulse code modulation (PCM) interface input, and can deliver outputs to a direct digital output stage, digital microphone outputs or a PCM interface output. The FIFO controller simplifies access to data by the CFX DSP, the HEAR, ARM Cortex-M3 core and the IOC by providing configurable hardware-based FIFOs.

A top-level block overview of the components of the Ezairo 7100 system is provided in Figure 1. More information on the various component blocks is provided in the following subsections.

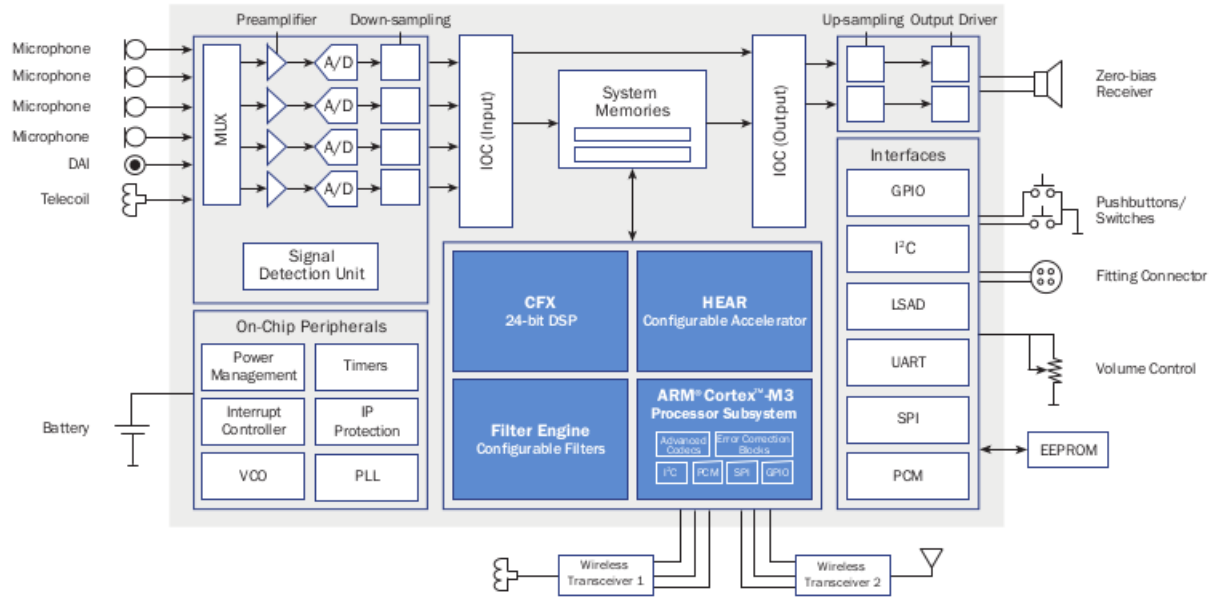


Figure 1. Top-Level System Block Diagram

2.2 CFX Digital Signal Processor

The CFX DSP is the primary digital processor in the system. It is a user-programmable, general-purpose DSP. The CFX serves many roles in the system: it acts as the system master; it interacts closely with all peripherals and external interfaces; and it is used to coordinate and assist in all data processing tasks performed by the four cores.

The CFX DSP is closely tied to its local memories, and also has access to all other memory-mapped components of the system except those items mapped onto the ARM Cortex-M3 core's private peripheral bus.

More detailed information on the CFX processor is available in Chapter 3, "CFX Digital Signal Processor" on page 31 of this manual, and in the *CFX DSP Architecture Manual*.

2.2.1 CFX Memory

The CFX DSP uses a dual-Harvard memory bus architecture to provide separate buses for its program memory and two separate data memories.

The program memory space available to the CFX includes a program ROM as well as program RAM space to store a user application and interrupt vectors. This memory is accessible as instructions to the CFX through the instruction memory bus, and as data for the CFX through the P memory bus. Additionally, the P memory space also provides access for the CFX to:

- The HEAR core's microcode memory
- The ARM Cortex-M3 processor's:
  - Program memory
  - Data memory
  - Peripheral bus-mapped control and configuration registers
- The Filter Engine core's memory and registers
- System control and configuration registers

The data memory areas available to the CFX are divided into two memory spaces that can be accessed either separately through the X memory and Y memory buses as 24-bit data, or together through these buses as 48-bit data in XY memory. These memory buses also provide access for the CFX DSP to the HEAR shared memory, and the HEAR's configuration registers.

More detailed information about the memories and associated memory buses is available in Chapter 7, "Memory" on page 72 of this manual.

2.3 HEAR Configurable Accelerator

The HEAR Configurable Accelerator provides additional digital signal processing functionality within the Ezairo 7100 system. The signal processing functionality of the HEAR is configured through a set of pre-defined microcode modules. The microcode modules provide functions that can be linked together to form function chains. These function chains are triggered through the integrated function chain controller of the HEAR via commands issued by the CFX or events from the FIFO controller.

The HEAR also has a number of other configurable components and communicates with the CFX DSP through its shared memories and CFX interrupts. The HEAR includes integrated block floating-point support for maintaining signal fidelity when the processing steps demand a large dynamic range.

The HEAR Configurable Accelerator is controlled by the CFX, which controls the clock and execution state of the core, as well as the core's memory configuration.

More detailed information about the HEAR Configurable Accelerator is available in Chapter 4, "HEAR Configurable Accelerator" on page 34 of this manual, and in the *HEAR Configurable Accelerator Reference Manual for Ezairo 7100*.

2.3.1 HEAR Memory

The HEAR Accelerator core uses a Harvard memory bus architecture that provides separate memories for the microcode modules and data.

Microcode memory contains the user-configured microcode modules for the HEAR. This memory is accessible from the CFX core through the P memory bus.

The HEAR shared memory is a set of distinct physical memories that are used to store temporary processing data, algorithm coefficients and signal data at all points during its processing (from input, to partially processed signal data, to output). The CFX DSP can access this memory through the X and Y memory buses. Access to shared memory is managed by the shared memory bus controller. Some portions of this shared memory are further controlled by the FIFO controller, which provides FIFO access to other system components including the IOC and ARM Cortex-M3 processor.

More detailed information about the microcode and shared memories is available in Chapter 4, "HEAR Configurable Accelerator" on page 34 of this manual, and in Chapter 7, "Memory" on page 72. Additional information about the FIFO controller and how it affects memory accesses is available in Chapter 8, "FIFO Controller" on page 98.

2.4 ARM Cortex-M3 Processor

The ARM Cortex-M3 general-purpose processor is used as a coprocessor that provides additional processing for the Ezairo 7100 system. This processing can be used to control communications using external wireless radio chips and to process data from those communications, or it can be used for additional signal processing. It connects to a number of peripherals using ARM's proprietary bus system. The controller is a 32-bit core with embedded multiplier and arithmetic logic unit (ALU) to handle typical control functions as well as DSP functions used for implementing a variety of wireless protocols.

The ARM Cortex-M3 processor is controlled by the CFX, which controls the clock and execution state of the core, as well as the core's memory and digital I/O configuration.

More detailed information about the ARM Cortex-M3 processor is available from:

- Chapter 5, "ARM Cortex-M3 Processor" on page 45 of this manual
- The *Cortex-M3 Technical Reference Manual*
- The *The Definitive Guide to the ARM Cortex-M3* and other third-party documentation for the ARM Cortex-M3 processor

2.4.1 ARM Cortex-M3 Processor Memory

The ARM Cortex-M3 processor memory subsystem consists of a program memory area that is connected to the core's instruction and data buses, as well as a data memory area connected to the core's system bus. Additionally, the ARM Cortex-M3 processor can access the HEAR FIFO memory in parallel with memory-mapped components through its peripheral bus.

More detailed information about the ARM Cortex-M3 processor's memories and memory buses is available in Chapter 7, "Memory" on page 72.

2.5 Filter Engine

The Filter Engine is a specialized processing core that is optimized to support pre- and post-processing of audio data. It also supports low-group delay processing of data at faster data rates than the system's sampling rate by bypassing the input stage's decimation filters and output stage's interpolation filters. With flexible multiplexing between the Filter Engine signals and the default audio paths, the input and output stages can use the Filter Engine outputs to bypass processing with the default audio paths.

This processor is primarily designed to perform time-domain filter operations such as infinite-impulse response (IIR) filtering and finite impulse response (FIR) filtering. It supports these processing requirements using second-order sections (biquad) filters and 5-tap FIR filters that can be cascaded to form larger filtering structures.

The Filter Engine is controlled by the CFX, which defines the Filter Engine program flow by controlling the execution and debug states of the core.

More detailed information about the Filter Engine is available in Chapter 6, "Filter Engine" on page 50, and in the *Filter Engine Reference Manual for Ezairo 7100*.

2.5.1 Filter Engine Memory

The Filter Engine architecture includes the following memory elements:

- A 64-word (32 bits per word) program memory
- Configuration registers
- 48 single-precision data registers (24 bits each) and 16 extended-precision data registers (50 bits each)
- A 160-word (24 bits per word) filter coefficient memory
- A 192-word (24 bits per word) filter state variable memory

More detailed information about the Filter Engine's memory structures is available in Chapter 7, "Memory" on page 72.

2.6 FIFO Controller

The FIFO controller controls access to the subset of shared memory that can be configured for use as FIFO buffers. This controller has three purposes in managing access to shared memory:

- It remaps portions of the address space so that a hardware circular buffer is implemented, and the most recent samples in the buffer can always be accessed at the same logical address.
- It resolves access conflicts between the input/output controller (IOC) and the three system cores that access these memory areas through the shared memory bus.
- It tracks incoming data to a buffer or outgoing data from a buffer. Changes to these buffers can trigger interrupts on the CFX DSP, function chains on the HEAR, and interrupts on the ARM Cortex-M3 processor.

More detailed information about configuring FIFOs and using the FIFO controller is available in Chapter 8, "FIFO Controller" on page 98.

2.7 PERIPHERALS

2.7.1 CFX Peripherals

The Ezairo 7100 system has a number of peripherals to support various tasks that the system needs to perform. Each of these peripherals are controlled through the CFX DSP. The CFX system peripherals include:

- An interrupt controller
- Two general-purpose timers
- A watchdog timer
- A CRC calculation unit
- System security blocks that are used to restrict external access to application code and data

More detailed information about configuring and using the peripherals is available in Chapter 14, “CFX Peripherals” on page 218 of this manual.

2.7.2 ARM Cortex-M3 Processor Peripherals

The ARM Cortex-M3 processor has a number of peripherals to support the tasks it needs to perform. The ARM Cortex-M3 processor peripherals include:

- A standard nested-vectored interrupt controller (NVIC)
- A system timer (SYSTICK)
- A CRC calculation unit
- Error-correction coding blocks (ECB) supporting Hamming and Reed-Solomon encoding
- Audio CODEC blocks (ACB) supporting G.722 and CVSD encoding
- A dynamic-memory access (DMA) unit

For more information about the ARM Cortex-M3 processor peripherals, see Chapter 15, “ARM Cortex-M3 Processor Peripherals” on page 239.

2.8 INTERFACES

Interfaces in the Ezairo 7100 system are assigned to 30 digital input/output (DIO) pads that are configured through the DIO block. Each of these pads can be configured as the source for a number of interfaces or as the outputs for a number of interface signals, system clocks and other system signals.

For more information about this DIO configuration, including physical configuration of the DIO pads, see Section 16.1, “Digital I/O (DIO)” on page 351.

2.8.1 CFX Interfaces

The Ezairo 7100 system has a number of external interfaces which are controlled by the CFX core. These interfaces provide support for a variety of types of external communications, as well as signalling through general-purpose input/output (GPIO) pads and low-speed analog-to-digital (LSAD) converters. The external interfaces provided are:

- An I^2C debug interface
- A general-purpose I^2C interface
- An SPI interface
- A PCM/I^2S interface
- A UART interface
- 6 LSAD inputs

- GPIOs

In addition to these interfaces, the CFX has access to all of the ARM Cortex-M3 processor's external interfaces.

The shared I^2S and PCM interface is designed to allow streaming of digitized signal data directly into or out of the system through either the IOC or CFX core. This interface can be configured to communicate with any I^2S device, many PCM devices and the NOAHlink™ (audio streaming) device. For more detailed configuration and use information, see Chapter 12, "CFX Shared PCM and I2S™ Interface" on page 172.

The CFX I^2C debug interface and general-purpose I^2C interface share an interface bus with the ARM Cortex-M3 processor's debug port. More detailed information about the interfaces that share this interface bus are described in Chapter 13, "Debug Ports and CFX General-Purpose I2C" on page 188. Further information about the debug ports can be found in the *Communication Protocols Manual for Ezairo 7100*.

For more detailed information about configuring and using the other external interfaces provided for the CFX, see Chapter 16, "CFX External Interfaces" on page 351 of this manual.

2.8.2 ARM Cortex-M3 Core Interfaces

The Ezairo 7100 system has a number of external interfaces that can be controlled from the ARM Cortex-M3 processor or the CFX DSP. These interfaces support connecting Ezairo 7100 to external components, including a variety of wireless devices. The ARM Cortex-M3 processor's interfaces include:

- An I^2C debug interface
- A general-purpose I^2C interface
- An SPI interface
- A PCM/I^2S interface
- GPIOs

More detailed information about the ARM Cortex-M3 processor's I^2C debug interface, which shares an interface bus with the CFX I^2C debug port and CFX general-purpose I^2C interface, can be found in Chapter 13, "Debug Ports and CFX General-Purpose I2C" on page 188. For more information about the other ARM Cortex-M3 processor interfaces, see Chapter 17, "ARM Cortex-M3 Processor External Interfaces" on page 400.

2.9 Inputs and Outputs

2.9.1 Input/Output Controller (IOC) and Audio Multiplexer

The input/output controller (IOC) and audio multiplexer control audio signal data input and output from the system. The IOC can be directed to acquire input data from any combination of:

- The four input A/D channels belonging to the input stage
- The four digital microphone inputs
- The Filter Engine inputs
- The two shared PCM/I^2S input channels.

The IOC is also used to direct output data, sending the data to a combination of:

- The two output data channels belonging to the output stage
- The two digital microphone outputs

- The two shared PCM/I^2S output channels

The IOC routes this data to and from the shared memory via the FIFO controller. Input data from and output data to the IOC are grouped into some combination of single-channel and multi-channel FIFO buffers.

For more detailed information about the configuration and operation of the IOC, see Chapter 9, “Input/Output Audio Control” on page 111.

2.9.2 Input Stage

The audio input stage on Ezairo 7100 provides four input channels multiplexed from six analog inputs. Each of these input channels uses a number of components including a preamplifier, several filters, and a 16-bit oversampling sigma-delta A/D converter to provide audio input data to the IOC for the rest of the Ezairo 7100 system to use.

For more detailed information about the configuration of the input stage and using the input stage in the acquisition of data, see Chapter 10, “Input Stage” on page 140.

2.9.3 Output Stage

The audio output stage on Ezairo 7100 provides two output channels. Each output channel uses a 16-bit oversampling sigma-delta D/A converter as the source for a direct digital output, which can be used to drive zero-bias hearing aid receivers.

For more detailed information about the configuration of the output stage and the distribution of data through the output stage, see Chapter 11, “Output Stage” on page 164.

2.10 Analog Support Blocks

2.10.1 Oscillator and Clocking

The system provides an on-chip voltage-controlled oscillator (VCO) that supports system clock frequencies up to 10.24 MHz, and a standby oscillator that can also be used to clock the system for low-power profiles. The VCO can be used as part of a phase-locked loop (PLL) to provide the Ezairo 7100 system with a system clock that is in phase with an external reference signal. Additionally, the system allows other clock sources to be selected as the clock for the system, and provides a number of clock dividers that produce a set of derived clock domains needed by the various components of the Ezairo 7100 system.

For detailed information about the operation and configuration of the oscillator and clocking components, see Chapter 18, “Clock Components” on page 432.

2.10.2 Power Supply, Power Management and Reset Handling

The Ezairo 7100 system includes integrated power supply and power management components. These components supply power to the system and cover all configuration aspects of managing power consumption through regular and stand-by power modes. These components also provide a regulated power supply to system components that require voltage regulation, monitor the power supplied to the system as a whole, and provide soft power-down tools to limit power consumption by the chip.

The power supply components include reset handling blocks that provide clean handling of all possible reset sources, including power-on-reset and other power management related resets, input from the NRESET pad, and the expiry of the watchdog timer.

For details about the power supply and power management features, see Chapter 19, “Power Components” on page 466.

2.11 Device Identification

The Ezairo 7100 chip provides a number of registers that contain static values, which identify a chip and provide version information about the chip. These registers can be used to verify compatibility for both user applications and external applications communicating through the debug port. A list of these registers along with a brief description of each are in Table 1.

Table 1. Device Identification Registers

Register	Description	PMEM Address
D_CHIP_FAMILY	The chip technology family to which the chip belongs. Ezairo 7100 belongs to chip family 0x06.	0xF030
D_CHIP_VERSION	Version number for the chip. Used to indicate major updates which might not be backwards compatible. Ezairo 7100 uses a chip version of 0x01.	0xF031
D_CHIP_REVISION	Revision number for the chip. • An update of the minor revision number (bits [7:0]) indicates minor backwards-compatible or non-functional updates to the system that require no update to source code or binary objects. • An update of the major revision number (bits [15:8]) indicates updates that could affect source code or binary objects, and thus could require firmware library or software updates. When the major revision number is updated, the minor revision number is reset.	0xF032
D_CFX_DEBUG_VERSION	Version number of the CFX debug port communications protocol. Used to indicate which communication options are available for the chip.	0xF033
D_CM3_DEBUG_VERSION	Version number of the ARM Cortex-M3 processor debug port communications protocol. Used to indicate which communication options are available for the chip.	0xF034

The chip version is commonly written as X.YY.ZZ (for example, 1.02.03), where X is the chip version number, YY is the major revision number, and ZZ is the minor revision number. Chips are also identified using just the chip version number and major revision number as a chip identifier (CID) value. For a chip version of X.YY.ZZ, the CID would be XYY (for example, 102).

CHAPTER 3
CFX Digital Signal Processor

3.1 Processor Overview

The CFX DSP core is a programmable, pipelined DSP that uses a dual-Harvard, dual-MAC architecture to efficiently process 24-bit signal data. For information about the hardware features of this processor and details of the CFX instruction set, refer to the *CFX DSP Architecture Manual*.

The CFX is the primary controller for the Ezairo 7100 system. As the only core that has access to other system components, it provides an interface for configuration and control of all of the other system components. Following a power-on reset (POR), the Ezairo 7100 system executes the Boot ROM portion of the Program ROM using the CFX DSP. These ROM routines initialize all aspects of a system application, including each of the other cores.

The CFX is closely coupled to many of the memory components. As a result, it has access to all of the signal data that the system is processing.

All applications executing on the CFX DSP core can be debugged through the CFX debug controller. It provides access to all the CFX registers and the memory available through the memory buses attached to the CFX DSP. This transparent view of the system facilitates efficient debugging by enabling you to:

- Review the current state of system components through your associated memory mapped control registers.
- Use this information to troubleshoot most issues in your application.

For further details about the debug controller and its interaction with the CFX DSP, see Section 13.2, “CFX Debug Port I2C Interface” on page 191.

3.2 Application Overview

The CFX DSP typically executes an application that implements a signal processing algorithm and uses external interfaces for communications and/or control.

As part of implementing a signal processing algorithm, the CFX DSP performs a number of tasks. These tasks include:

- Configuration of the Ezairo 7100 system including:
  - The IOC, PCM/I^2S interface and input stage to obtain input signal information
  - The IOC, PCM/I^2S interface and output stage to distribute output signal information
  - FIFO buffers for use in handling input, output and intermediate data
  - The HEAR Configurable Accelerator by setting configuration registers and loading microcode to perform regular signal processing algorithms on the available data
  - The ARM Cortex-M3 processor by setting configuration registers and loading a program to perform additional signal processing or external communications
  - The Filter Engine by setting configuration register and loading a phased program to perform any required input or output filtering in addition to low-delay path processing
- Execution of the signal processing algorithm code
- Communication with and/or signalling external components through the debug port and other external interfaces
- Communication with and coordination of the other cores in the Ezairo 7100 system

Figure 2 provides an overview of possible paths that signal data will typically flow through the Ezairo 7100 system.

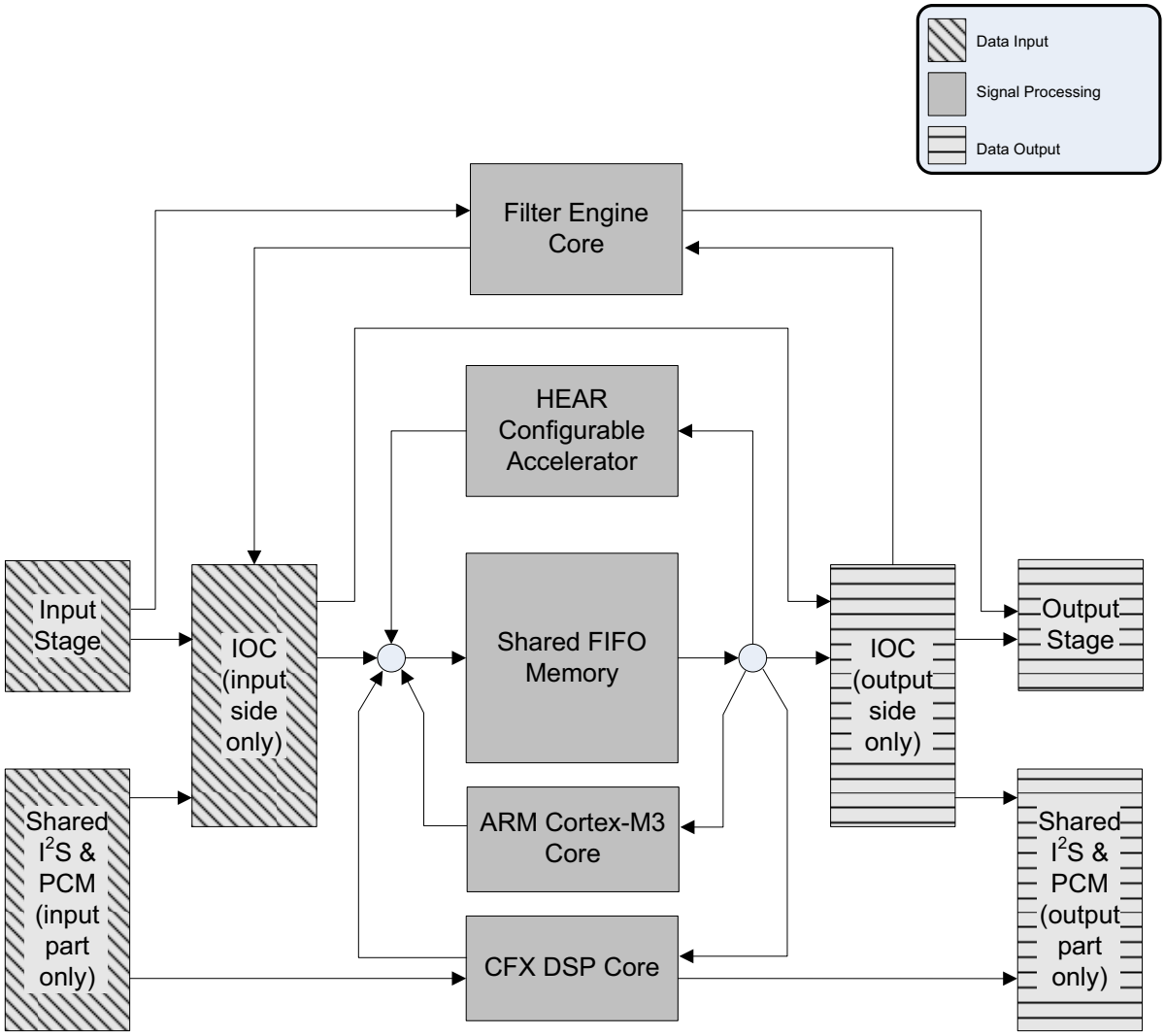


Figure 2. Signal Data Flow in a Typical CFX Digital Signal Processor Controlled Application

Signal data flow can be divided into three steps as shown in Figure 2: data input, signal processing and data output.

Data Input — For input, the signal data to be processed enters the system either through the input stage (either from the analog inputs or digital microphone inputs), or the CFX shared PCM and I^2S interface to the system. This data is processed through the input side of the IOC directly to FIFOs in shared memory. Alternatively, the data from the input stage can be processed by the Filter Engine before, or instead of, being passed to the IOC. Data from the PCM/I^2S interface can alternatively be passed directly to the CFX DSP through the PCM interface data registers.

Signal Processing — The signal data stored in the FIFOs can then be processed through any number of data processing steps by functions implemented for the CFX DSP, function chains available through

the microcode configuration supplied for the HEAR, or functions implemented for the ARM Cortex-M3 processor. After each processing step, the data is typically written back to either an intermediate or output FIFO buffer.

The Filter Engine supports pre-filtering of input data, post-filtering of output data, and high-frequency/low group-delay processing.

Data Output Once all of the desired data processing has been performed, the data is distributed from the system either directly through the shared PCM/I^2S interface, from the output of the Filter Engine, or through the output side of the IOC. As shown, the IOC can then direct this output information to the PCM/I^2S interface or the output stage (including outputs to both the direct-digital outputs and the digital microphone outputs).

Algorithms running on the CFX can adapt their behavior based on external conditions or stimuli. For example, the CFX might have a signal attached to an LSAD pin that is used to control the gain, and hence the output volume, of the output signal provided by the system.

The CFX DSP can configure the other cores, peripherals, and external interfaces to send and receive control, configuration and signal data. These components of an application might be used for a variety of things, from supplying information to an external component to configuring the clock and power components. This portion of an application is usually highly specific based on the overall product needs; therefore, this document does not attempt to provide a generic view of such application components.

For more details about the CFX, see the *CFX DSP Architecture Manual*.

CHAPTER 4
HEAR Configurable Accelerator

4.1 Accelerator Overview

The HEAR Configurable Accelerator is a core that provides additional signal processing resources to the Ezairo 7100 system. Unlike the CFX DSP and the ARM Cortex-M3 processor, the HEAR is not programmable, does not interact directly with any external interfaces, and does not include direct debugging access for applications that make use of the core. To debug algorithms that use the HEAR, the debug controller can view some information on the HEAR through memories available on the memory buses attached to the CFX.

For information about the HEAR's hardware features, available microcode modules, and how to configure the HEAR Configurable Accelerator to work with a given microcode configuration, see the *HEAR Configurable Accelerator Reference Manual for Ezairo 7100*. Further hardware details regarding the function chain controller can be found in Section 4.2, "Function Chain Controller", and further details about the block floating-point feature that can be used with some microcode configurations are in Section 4.3, "Block Floating-Point".

4.1.1 Function Chains

The HEAR is a configurable 24-bit processing unit that runs data-processing functions defined in HEAR microcode configurations. These functions, which are typically used to process signal data within the system, are linked together into sequences of one or more functions known as microcode function chains. These function chains are executed by the HEAR coprocessor through triggered events such as a command sent by the CFX DSP or an interrupt from one of the configured FIFOs. These chains normally operate on data stored in shared memory: usually an H**x** memory instance (see Section 7.2, "Memory Architecture" on page 72 and Chapter 8, "FIFO Controller" on page 98 for more information), an input FIFO, or an intermediate FIFO. Output from these chains is typically stored to another location in shared memory: usually another H**x** memory instance, an output FIFO, or an intermediate FIFO.

When any function completes, or at the end of a complete function chain, the HEAR processor can send the CFX processor an interrupt through one of eight HEAR interrupts. These interrupts are used to notify the CFX when data is ready for additional processing, or to signal that the CFX can issue another HEAR command.

4.1.2 HEAR Control

To pause and resume the HEAR processing, set the `SYSTEM_CTRL_HEAR_PAUSE` and `SYSTEM_CTRL_HEAR_RESUME` bits respectively in the `D_SYSTEM_CTRL` register. When the HEAR is paused, the accelerator clock is stopped, which prevents the execution of any functions and all responses to the function chain controller. All incomplete accesses to shared memory by the HEAR are put into a non-blocking wait state, and these will be completed once processing resumes on the HEAR. While the HEAR is paused, the CFX core and IOC can access shared memory at any address, and the ARM Cortex-M3 processor can access FIFO memory at any address, without the HEAR Accelerator introducing additional delays.

As bits in the `D_SYSTEM_CTRL` register only change state when set, do not worry about the state of the other bits of this register when controlling the HEAR. However, the behavior of the system is undefined if both of these bits are set by the same instruction. Reading the `SYSTEM_CTRL_HEAR_RUNNING` bit (which shares a bit position with the `SYSTEM_CTRL_HEAR_RESUME` bit) in the `D_SYSTEM_CTRL` register provides the current status of the HEAR.

IMPORTANT: While the HEAR is paused, function chain triggers are still collected in the `D_HEAR_FC_STATUS` register. When processing on the HEAR is resumed, these triggers will cause their associated function chains to be executed as described in Section 4.2, "Function Chain Controller".

The HEAR can also be reset by setting the SYSTEM_CTRL_HEAR_RESET bit in the D_SYSTEM_CTRL register. This resets the HEAR core and its peripherals, including the block-floating point unit and the function chain controller.

IMPORTANT: The HEAR reset additionally resets the FIFO controllers, clearing the current FIFO register settings.

4.1.3 Control and Configuration Registers

Register Name	Register Description	Address
D_SYSTEM_CTRL	CFX/HEAR Configuration and Control	P:0xF000

4.1.3.1 D_SYSTEM_CTRL Settings

Bit Field	Field Name	Description
7	PRIORITY_HEAR	Write a 1 to give HEAR priority
6	PRIORITY_STATUS	CFX/HEAR priority status
	PRIORITY_CFX	Write a 1 to give CFX priority
5	HEAR_PAUSE	Write a 1 to pause HEAR
4	HEAR_RUNNING	HEAR running status
	HEAR_RESUME	Write a 1 to run HEAR
3	HEAR_RESET	Write a 1 to reset HEAR
2	TIMER_2_BUSY	Check status of timer 2
	TIMER_2_START	Write a 1 to start timer 2
1	TIMER_1_BUSY	Check status of timer 1
	TIMER_1_START	Write a 1 to start timer 1
0	WATCHDOG_REFRESH	Write a 1 to reset the watchdog

Field Name	Value Symbol	Value Description	Hex Value
PRIORITY_HEAR	SYSTEM_PRIORITY_HEAR	Give HEAR priority over CFX	0x1
PRIORITY_STATUS	SYSTEM_PRIORITY_STATE_HEAR	HEAR has priority over CFX	0x0*
	SYSTEM_PRIORITY_STATE_CFX	CFX has priority over HEAR	0x1
PRIORITY_CFX	SYSTEM_PRIORITY_CFX	Give CFX priority over HEAR	0x1
HEAR_PAUSE	SYSTEM_HEAR_PAUSE	Pause HEAR	0x1
HEAR_RUNNING	SYSTEM_HEAR_STATE_PAUSE	HEAR paused	0x0*
	SYSTEM_HEAR_STATE_RUN	HEAR running	0x1
HEAR_RESUME	SYSTEM_HEAR_RESUME	Resume HEAR	0x1
HEAR_RESET	SYSTEM_HEAR_RESET	HEAR reset	0x1
TIMER_2_BUSY	SYSTEM_TIMER_2_STOPPED	Timer2 stopped	0x0*
	SYSTEM_TIMER_2_RUNNING	Timer2 running	0x1
TIMER_2_START	SYSTEM_TIMER_2_START	Timer2 start	0x1

Field Name	Value Symbol	Value Description	Hex Value
TIMER_1_BUSY	SYSTEM_TIMER_1_STOPPED	Timer1 stopped	0x0*
	SYSTEM_TIMER_1_RUNNING	Timer1 running	0x1
TIMER_1_START	SYSTEM_TIMER_1_START	Timer1 start	0x1
WATCHDOG_REFRESH	SYSTEM_WATCHDOG_REFRESH	Watchdog reset	0x1

4.2 Function Chain Controller

The CFX DSP configures the function chain controller of the HEAR to control the execution of the HEAR's configurable microcode function chains.

Function chains are typically associated with triggers that can be used to execute them. The function chain controller is directed to trigger specific function chains within a microcode configuration by setting bits in the D_HEAR_FC_ENABLE register.

The CFX DSP sends commands to start the execution of one or more of the HEAR function chains by setting the appropriate bit or bits in the D_HEAR_CMDS_0_TO_7 and D_HEAR_CMDS_8_TO_15 registers. The processing of function chains can also be triggered by HEAR FIFO interrupts in the system. In both cases, the microcode functions to be executed in the function chain depend on the microcode configuration used by the user application. Each function chain has an entry in a vector table stored at D_HEAR_FC_VECT_BASE located in shared memory at address 0xBFE0 in the X memory address space. These function chain vector table entries can be modified during an application's execution to change which function chains are mapped to the CFX commands and FIFO interrupts. For more information about the HEAR microcode, function chains and possible microcode configurations, see the *HEAR Configurable Accelerator Reference Manual for Ezairo 7100*.

Setting the corresponding bit in the D_HEAR_FC_PRIORITY register prioritizes enabled function chains as either normal or high priority. This allows an application to select the function chains to process first if more than one function chain is pending when the function chain controller initiates the processing of a function chain. If multiple function chains are pending, all high priority function chains are handled first. If multiple pending function chains share the same priority level, then function chains are executed with the following order of precedence:

1. Function chains triggered by CFX commands, starting with the lowest numbered command
2. Function chains triggered by FIFO events, starting with the lowest numbered FIFO function chain

The processing status of function chains can be observed through the D_HEAR_FC_STATUS register. The bit corresponding to a particular enabled function chain is set whenever the execution of the function chain is pending.

IMPORTANT: Writing to the D_HEAR_FC_STATUS register might result in undefined application behavior.

4.2.1 Control and Configuration Registers

Register Name	Register Description	Address
D_HEAR_FC_ENABLE	HEAR Function Chain Enable register	X:0x7230
D_HEAR_FC_PRIORITY	HEAR Function Chain Priority register	X:0x7231
D_HEAR_FC_STATUS	HEAR Function Chain Status register	X:0x7237

Register Name	Register Description	Address
D_HEAR_CMDS_0_TO_7	CFX issued HEAR commands 0-7	P:0xF004
D_HEAR_CMDS_8_TO_15	CFX issued HEAR commands 8-15	P:0xF005

4.2.1.1 D_HEAR_FC_ENABLE Settings

Bit Field	Field Name	Description
23	FIFO_7	Enable/disable HEAR FIFO function chain 7
22	FIFO_6	Enable/disable HEAR FIFO function chain 6
21	FIFO_5	Enable/disable HEAR FIFO function chain 5
20	FIFO_4	Enable/disable HEAR FIFO function chain 4
19	FIFO_3	Enable/disable HEAR FIFO function chain 3
18	FIFO_2	Enable/disable HEAR FIFO function chain 2
17	FIFO_1	Enable/disable HEAR FIFO function chain 1
16	FIFO_0	Enable/disable HEAR FIFO function chain 0
15	CMD15	Enable/disable CFX function chain 15
14	CMD14	Enable/disable CFX function chain 14
13	CMD13	Enable/disable CFX function chain 13
12	CMD12	Enable/disable CFX function chain 12
11	CMD11	Enable/disable CFX function chain 11
10	CMD10	Enable/disable CFX function chain 10
9	CMD9	Enable/disable CFX function chain 9
8	CMD8	Enable/disable CFX function chain 8
7	CMD7	Enable/disable CFX function chain 7
6	CMD6	Enable/disable CFX function chain 6
5	CMD5	Enable/disable CFX function chain 5
4	CMD4	Enable/disable CFX function chain 4
3	CMD3	Enable/disable CFX function chain 3
2	CMD2	Enable/disable CFX function chain 2
1	CMD1	Enable/disable CFX function chain 1
0	CMD0	Enable/disable CFX function chain 0

Field Name	Value Symbol	Value Description	Hex Value
FIFO_7	HEAR_FC_EBL_FIFO_7	Enable function chain FIFO_7	0x1
FIFO_6	HEAR_FC_EBL_FIFO_6	Enable function chain FIFO_6	0x1
FIFO_5	HEAR_FC_EBL_FIFO_5	Enable function chain FIFO_5	0x1
FIFO_4	HEAR_FC_EBL_FIFO_4	Enable function chain FIFO_4	0x1
FIFO_3	HEAR_FC_EBL_FIFO_3	Enable function chain FIFO_3	0x1
FIFO_2	HEAR_FC_EBL_FIFO_2	Enable function chain FIFO_2	0x1
FIFO_1	HEAR_FC_EBL_FIFO_1	Enable function chain FIFO_1	0x1

Field Name	Value Symbol	Value Description	Hex Value
FIFO_0	HEAR_FC_EBL_FIFO_0	Enable function chain FIFO_0	0x1
CMD15	HEAR_FC_EBL_CMD15	Enable function chain CMD15	0x1
CMD14	HEAR_FC_EBL_CMD14	Enable function chain CMD14	0x1
CMD13	HEAR_FC_EBL_CMD13	Enable function chain CMD13	0x1
CMD12	HEAR_FC_EBL_CMD12	Enable function chain CMD12	0x1
CMD11	HEAR_FC_EBL_CMD11	Enable function chain CMD11	0x1
CMD10	HEAR_FC_EBL_CMD10	Enable function chain CMD10	0x1
CMD9	HEAR_FC_EBL_CMD9	Enable function chain CMD9	0x1
CMD8	HEAR_FC_EBL_CMD8	Enable function chain CMD8	0x1
CMD7	HEAR_FC_EBL_CMD7	Enable function chain CMD7	0x1
CMD6	HEAR_FC_EBL_CMD6	Enable function chain CMD6	0x1
CMD5	HEAR_FC_EBL_CMD5	Enable function chain CMD5	0x1
CMD4	HEAR_FC_EBL_CMD4	Enable function chain CMD4	0x1
CMD3	HEAR_FC_EBL_CMD3	Enable function chain CMD3	0x1
CMD2	HEAR_FC_EBL_CMD2	Enable function chain CMD2	0x1
CMD1	HEAR_FC_EBL_CMD1	Enable function chain CMD1	0x1
CMD0	HEAR_FC_EBL_CMD0	Enable function chain CMD0	0x1

4.2.1.2 D_HEAR_FC_PRIORITY Settings

Bit Field	Field Name	Description
23	FIFO_7	Prioritize the HEAR FIFO function chain 7
22	FIFO_6	Prioritize the HEAR FIFO function chain 6
21	FIFO_5	Prioritize the HEAR FIFO function chain 5
20	FIFO_4	Prioritize the HEAR FIFO function chain 4
19	FIFO_3	Prioritize the HEAR FIFO function chain 3
18	FIFO_2	Prioritize the HEAR FIFO function chain 2
17	FIFO_1	Prioritize the HEAR FIFO function chain 1
16	FIFO_0	Prioritize the HEAR FIFO function chain 0
15	CFX_15	Prioritize the CFX function chain 15
14	CFX_14	Prioritize the CFX function chain 14
13	CMD13	Prioritize the CFX function chain 13
12	CMD12	Prioritize the CFX function chain 12
11	CMD11	Prioritize the CFX function chain 11
10	CMD10	Prioritize the CFX function chain 10
9	CMD9	Prioritize the CFX function chain 9
8	CMD8	Prioritize the CFX function chain 8
7	CMD7	Prioritize the CFX function chain 7
6	CMD6	Prioritize the CFX function chain 6

Bit Field	Field Name	Description
5	CMD5	Prioritize the CFX function chain 5
4	CMD4	Prioritize the CFX function chain 4
3	CMD3	Prioritize the CFX function chain 3
2	CMD2	Prioritize the CFX function chain 2
1	CMD1	Prioritize the CFX function chain 1
0	CMD0	Prioritize the CFX function chain 0

Field Name	Value Symbol	Value Description	Hex Value
FIFO_7	HEAR_FC_NORMALPRIO_FIFO_7	Function chain FIFO_7 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_FIFO_7	Function chain FIFO_7 has high priority	0x1
FIFO_6	HEAR_FC_NORMALPRIO_FIFO_6	Function chain FIFO_6 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_FIFO_6	Function chain FIFO_6 has high priority	0x1
FIFO_5	HEAR_FC_NORMALPRIO_FIFO_5	Function chain FIFO_5 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_FIFO_5	Function chain FIFO_5 has high priority	0x1
FIFO_4	HEAR_FC_NORMALPRIO_FIFO_4	Function chain FIFO_4 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_FIFO_4	Function chain FIFO_4 has high priority	0x1
FIFO_3	HEAR_FC_NORMALPRIO_FIFO_3	Function chain FIFO_3 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_FIFO_3	Function chain FIFO_3 has high priority	0x1
FIFO_2	HEAR_FC_NORMALPRIO_FIFO_2	Function chain FIFO_2 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_FIFO_2	Function chain FIFO_2 has high priority	0x1
FIFO_1	HEAR_FC_NORMALPRIO_FIFO_1	Function chain FIFO_1 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_FIFO_1	Function chain FIFO_1 has high priority	0x1
FIFO_0	HEAR_FC_NORMALPRIO_FIFO_0	Function chain FIFO_0 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_FIFO_0	Function chain FIFO_0 has high priority	0x1
CFX_15	HEAR_FC_NORMALPRIO_CFX_15	Function chain CFX_15 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CFX_15	Function chain CFX_15 has high priority	0x1
CFX_14	HEAR_FC_NORMALPRIO_CFX_14	Function chain CFX_14 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CFX_14	Function chain CFX_14 has high priority	0x1
CMD13	HEAR_FC_NORMALPRIO_CMD13	Function chain CMD13 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CMD13	Function chain CMD13 has high priority	0x1
CMD12	HEAR_FC_NORMALPRIO_CMD12	Function chain CMD12 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CMD12	Function chain CMD12 has high priority	0x1
CMD11	HEAR_FC_NORMALPRIO_CMD11	Function chain CMD11 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CMD11	Function chain CMD11 has high priority	0x1
CMD10	HEAR_FC_NORMALPRIO_CMD10	Function chain CMD10 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CMD10	Function chain CMD10 has high priority	0x1
CMD9	HEAR_FC_NORMALPRIO_CMD9	Function chain CMD9 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CMD9	Function chain CMD9 has high priority	0x1

Field Name	Value Symbol	Value Description	Hex Value
CMD8	HEAR_FC_NORMALPRIO_CMD8	Function chain CMD8 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CMD8	Function chain CMD8 has high priority	0x1
CMD7	HEAR_FC_NORMALPRIO_CMD7	Function chain CMD7 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CMD7	Function chain CMD7 has high priority	0x1
CMD6	HEAR_FC_NORMALPRIO_CMD6	Function chain CMD6 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CMD6	Function chain CMD6 has high priority	0x1
CMD5	HEAR_FC_NORMALPRIO_CMD5	Function chain CMD5 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CMD5	Function chain CMD5 has high priority	0x1
CMD4	HEAR_FC_NORMALPRIO_CMD4	Function chain CMD4 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CMD4	Function chain CMD4 has high priority	0x1
CMD3	HEAR_FC_NORMALPRIO_CMD3	Function chain CMD3 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CMD3	Function chain CMD3 has high priority	0x1
CMD2	HEAR_FC_NORMALPRIO_CMD2	Function chain CMD2 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CMD2	Function chain CMD2 has high priority	0x1
CMD1	HEAR_FC_NORMALPRIO_CMD1	Function chain CMD1 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CMD1	Function chain CMD1 has high priority	0x1
CMD0	HEAR_FC_NORMALPRIO_CMD0	Function chain CMD0 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CMD0	Function chain CMD0 has high priority	0x1

4.2.1.3 D_HEAR_FC_STATUS Settings

Bit Field	Field Name	Description
23	FIFO_HEAR_7_PENDING	Read the HEAR FIFO pending function chain 7 status
23	FIFO_HEAR_7_CLEAR	Clear the HEAR FIFO pending function chain 7 status
22	FIFO_HEAR_6_PENDING	Read the HEAR FIFO pending function chain 6 status
22	FIFO_HEAR_6_CLEAR	Clear the HEAR FIFO pending function chain 6 status
21	FIFO_HEAR_5_PENDING	Read the HEAR FIFO pending function chain 5 status
21	FIFO_HEAR_5_CLEAR	Clear the HEAR FIFO pending function chain 5 status
20	FIFO_HEAR_4_PENDING	Read the HEAR FIFO pending function chain 4 status
20	FIFO_HEAR_4_CLEAR	Clear the HEAR FIFO pending function chain 4 status
19	FIFO_HEAR_3_PENDING	Read the HEAR FIFO pending function chain 3 status
19	FIFO_HEAR_3_CLEAR	Clear the HEAR FIFO pending function chain 3 status
18	FIFO_HEAR_2_PENDING	Read the HEAR FIFO pending function chain 2 status
18	FIFO_HEAR_2_CLEAR	Clear the HEAR FIFO pending function chain 2 status
17	FIFO_HEAR_1_PENDING	Read the HEAR FIFO pending function chain 1 status
17	FIFO_HEAR_1_CLEAR	Clear the HEAR FIFO pending function chain 1 status
16	FIFO_HEAR_0_PENDING	Read the HEAR FIFO pending function chain 0 status
16	FIFO_HEAR_0_CLEAR	Clear the HEAR FIFO pending function chain 0 status
15	CFX_15_PENDING	Read the CFX pending function chain 15 status

Bit Field	Field Name	Description
15	CFX_15_CLEAR	Clear the CFX pending function chain 15 status
14	CFX_14_PENDING	Read the CFX pending function chain 14 status
14	CFX_14_CLEAR	Clear the CFX pending function chain 14 status
13	CFX_13_PENDING	Read the CFX pending function chain 13 status
13	CFX_13_CLEAR	Clear the CFX pending function chain 13 status
12	CFX_12_PENDING	Read the CFX pending function chain 12 status
12	CFX_12_CLEAR	Clear the CFX pending function chain 12 status
11	CFX_11_PENDING	Read the CFX pending function chain 11 status
11	CFX_11_CLEAR	Clear the CFX pending function chain 11 status
10	CFX_10_PENDING	Read the CFX pending function chain 10 status
10	CFX_10_CLEAR	Clear the CFX pending function chain 10 status
9	CFX_9_PENDING	Read the CFX pending function chain 9 status
9	CFX_9_CLEAR	Clear the CFX pending function chain 9 status
8	CFX_8_PENDING	Read the CFX pending function chain 8 status
8	CFX_8_CLEAR	Clear the CFX pending function chain 8 status
7	CFX_7_PENDING	Read the CFX pending function chain 7 status
7	CFX_7_CLEAR	Clear the CFX pending function chain 7 status
6	CFX_6_PENDING	Read the CFX pending function chain 6 status
6	CFX_6_CLEAR	Clear the CFX pending function chain 6 status
5	CFX_5_PENDING	Read the CFX pending function chain 5 status
5	CFX_5_CLEAR	Clear the CFX pending function chain 5 status
4	CFX_4_PENDING	Read the CFX pending function chain 4 status
4	CFX_4_CLEAR	Clear the CFX pending function chain 4 status
3	CFX_3_PENDING	Read the CFX pending function chain 3 status
3	CFX_3_CLEAR	Clear the CFX pending function chain 3 status
2	CFX_2_PENDING	Read the CFX pending function chain 2 status
2	CFX_2_CLEAR	Clear the CFX pending function chain 2 status
1	CFX_1_PENDING	Read the CFX pending function chain 1 status
1	CFX_1_CLEAR	Clear the CFX pending function chain 1 status
0	CFX_0_PENDING	Read the CFX pending function chain 0 status
0	CFX_0_CLEAR	Clear the CFX pending function chain 0 status

4.2.1.4 D_HEAR_CMDS_0_TO_7 Settings

Bit Field	Field Name	Description
7	HEAR_CMD_7	Write a 1 to issue HEAR function chain 7
6	HEAR_CMD_6	Write a 1 to issue HEAR function chain 6
5	HEAR_CMD_5	Write a 1 to issue HEAR function chain 5
4	HEAR_CMD_4	Write a 1 to issue HEAR function chain 4

Bit Field	Field Name	Description
3	HEAR_CMD_3	Write a 1 to issue HEAR function chain 3
2	HEAR_CMD_2	Write a 1 to issue HEAR function chain 2
1	HEAR_CMD_1	Write a 1 to issue HEAR function chain 1
0	HEAR_CMD_0	Write a 1 to issue HEAR function chain 0

Field Name	Value Symbol	Value Description	Hex Value
HEAR_CMD_7	HEAR_CMD_7	Issue HEAR function chain 7	0x1
HEAR_CMD_6	HEAR_CMD_6	Issue HEAR function chain 6	0x1
HEAR_CMD_5	HEAR_CMD_5	Issue HEAR function chain 5	0x1
HEAR_CMD_4	HEAR_CMD_4	Issue HEAR function chain 4	0x1
HEAR_CMD_3	HEAR_CMD_3	Issue HEAR function chain 3	0x1
HEAR_CMD_2	HEAR_CMD_2	Issue HEAR function chain 2	0x1
HEAR_CMD_1	HEAR_CMD_1	Issue HEAR function chain 1	0x1
HEAR_CMD_0	HEAR_CMD_0	Issue HEAR function chain 0	0x1

4.2.1.5 D_HEAR_CMDS_8_TO_15 Settings

Bit Field	Field Name	Description
7	HEAR_CMD_15	Write a 1 to issue HEAR function chain 15
6	HEAR_CMD_14	Write a 1 to issue HEAR function chain 14
5	HEAR_CMD_13	Write a 1 to issue HEAR function chain 13
4	HEAR_CMD_12	Write a 1 to issue HEAR function chain 12
3	HEAR_CMD_11	Write a 1 to issue HEAR function chain 11
2	HEAR_CMD_10	Write a 1 to issue HEAR function chain 10
1	HEAR_CMD_9	Write a 1 to issue HEAR function chain 9
0	HEAR_CMD_8	Write a 1 to issue HEAR function chain 8

Field Name	Value Symbol	Value Description	Hex Value
HEAR_CMD_15	HEAR_CMD_15	Issue HEAR function chain 15	0x1
HEAR_CMD_14	HEAR_CMD_14	Issue HEAR function chain 14	0x1
HEAR_CMD_13	HEAR_CMD_13	Issue HEAR function chain 13	0x1
HEAR_CMD_12	HEAR_CMD_12	Issue HEAR function chain 12	0x1
HEAR_CMD_11	HEAR_CMD_11	Issue HEAR function chain 11	0x1
HEAR_CMD_10	HEAR_CMD_10	Issue HEAR function chain 10	0x1
HEAR_CMD_9	HEAR_CMD_9	Issue HEAR function chain 9	0x1
HEAR_CMD_8	HEAR_CMD_8	Issue HEAR function chain 8	0x1

4.3 BLOCK FLOATING-POINT

The HEAR Configurable Accelerator supports the use of a block floating-point data type for data calculations. Using this feature allows:

1. The detection of the maximum magnitude of a block of data
2. Shifting of data by a previous magnitude factor
3. Storing of the total shift amount (referred to as the block exponent)

The block floating-point configuration is normally controlled by the microcode configuration running on the HEAR. As use of block floating-point is tightly coupled with a microcode configuration, only specific configuration details of the available bit fields and defined symbols for these bit fields are provided here. For more information and further details about using this feature of the HEAR, see the *HEAR Configurable Accelerator Reference Manual for Ezairo 7100*.

This feature of the HEAR can also be configured by the CFX through the `D_HEAR_BFP` register as part of the overall configuration of the HEAR. The block exponent bit field (`HEAR_BFP_BLOCK_EXP`) from the `D_HEAR_BFP` register has been mirrored into the low bits of the `D_HEAR_BFP_BLOCK_EXP` register to allow efficient access to this important field.

4.3.1 Control and Configuration Registers

Register Name	Register Description	Address
`D_HEAR_BFP`	Configure block floating-point functionality of the HEAR	X:0x7260
`D_HEAR_BFP_BLOCK_EXP`	Mirror of the block floating-point total shift/block exponent field from the `D_HEAR_BFP` register	X:0x7261

4.3.1.1 D_HEAR_BFP Settings

Bit Field	Field Name	Description
21:16	`BLOCK_EXP`	Stored block exponent for the data
15	`FMD_MODE`	Enable/disable full magnitude detection mode
12:8	`BLOCK_MAX_MAG`	Maximum magnitude for a block of data
5:4	`SCALE_FACTOR`	Select a scaling factor for BFP
1:0	`FORMAT`	Select a format based on the number of bits

Field Name	Value Symbol	Value Description	Hex Value
`FMD_MODE`	`HEAR_BFP_FMD_DISABLE`	Disable FMD mode	0x0*
	`HEAR_BFP_FMD_ENABLE`	Enable FMD mode	0x1
`FORMAT`	`HEAR_BFP_FORMAT_24BITS`	Allow no shifts	0x0*
	`HEAR_BFP_FORMAT_23BITS`	Allow 0 or 1 shift	0x1
	`HEAR_BFP_FORMAT_22BITS`	Allow 0, 1 or 2 shifts	0x2
	`HEAR_BFP_FORMAT_21BITS`	Allow 0, 1, 2 or 3 shifts	0x3

4.3.1.2 D_HEAR_BFP_BLOCK_EXP Settings

Bit Field	Field Name	Description
5:0	HEAR_BFP_BLOCK_EXP	Mirror of the block exponent for the data

CHAPTER 5
ARM Cortex-M3 Processor

5.1 Processor Overview

The ARM Cortex-M3 processor acts as a coprocessor to provide additional computation and support for external components with a focus on supporting wireless capability for the Ezairo 7100 system. This ARM Cortex-M3 processor is a low-power microcontroller that implements the ARMv7-M architecture, which features low interrupt latency and low-cost debugging.

The purpose of including the ARM Cortex-M3 processor, its peripherals, and interfaces in the Ezairo 7100 system is to provide the functionality required to implement a variety of wireless protocols for the system. Typical protocol elements that can be supported using the ARM Cortex-M3 processor and peripherals include:

- Header generation and decoding
- Error correction encoding and decoding
- Audio compression encoding and decoding
- Combining and extracting header and payload fields

Some key features of the ARM Cortex-M3 processor include:

- The Thumb-2 (ISA) subset consisting of all base Thumb-2 instructions, both 16-bit and 32-bit
- Harvard processor architecture enabling simultaneous instruction fetches with data load or store
- A three-stage pipeline
- Single cycle 32-bit multiply
- Hardware divide
- Thumb and debug states
- Handler and thread modes
- Low latency ISR entry and exit
- Processor state saving and restoration, with no instruction fetch overhead; the exception vector is fetched from memory in parallel with the state saving, enabling faster ISR entry
- Support for late-arriving interrupts
- A tightly coupled interface to an interrupt controller, enabling efficient processing of late-arriving interrupts
- Tail-chaining of interrupts, enabling back-to-back interrupt processing without the overhead of state saving and restoration between interrupts
- Interruptible-continued LDM/STM, and PUSH/POP
- ARMv6 style BE8/LE support
- ARMv6 unaligned
- Registers:
    - 13 general-purpose 32-bit registers
    - Link Register (LR)
    - Program Counter (PC)
    - Program Status Register (xPSR)
    - Two banked SP registers

For power management, the processor can be placed into a sleep mode under firmware control in which the processor clock is disabled. The Nested Vectored Interrupt Controller (NVIC) continues to run to enable exiting sleep on an interrupt. For more information about the NVIC, see Section 15.1, “Nested Vectored Interrupt Controller (NVIC)” on page 239.

For information about the ARM Cortex-M3 processor's hardware features, see the *Cortex-M3 Technical Reference Manual*, or the *The Definitive Guide to the ARM Cortex-M3* and other third-party documentation for the ARM Cortex-M3 processor. Further hardware details regarding peripherals accessible to the ARM Cortex-M3 processor in the Ezairo 7100 system are in Chapter 15, "ARM Cortex-M3 Processor Peripherals" on page 239, and further details about the external interfaces accessible to the ARM Cortex-M3 processor in the Ezairo 7100 system are in Chapter 17, "ARM Cortex-M3 Processor External Interfaces" on page 400.

5.1.1 ARM Cortex-M3 Processor Integration and Control

The ARM Cortex-M3 processor is slave to the CFX which has master control of the Ezairo 7100 system. The inter-processor communication methods between the ARM Cortex-M3 processor and the rest of Ezairo 7100 are:

Shared memory — The CFX can access any memory or peripheral in the ARM Cortex-M3 processor's memory space except for the internal peripherals mapped onto the ARM Cortex-M3 core's private peripheral bus. Shared memory accessible on both the CFX and ARM Cortex-M3 core includes:

- The ARM Cortex-M3 core program memory
- The ARM Cortex-M3 core data memory
- The ARM Cortex-M3 core peripheral bus registers
- The HEAR FIFO memory (through CFX/HEAR shared memory)

HEAR FIFO access — The FIFO in the HEAR subsystem is mapped to a portion of the ARM Cortex-M3 processor data memory space

Interrupts — The ARM Cortex-M3 processor has 8 input interrupts from the CFX processor controlled by the `D_CM3_INT_CMD` register. In addition, the ARM Cortex-M3 processor can generate up to four interrupts to the CFX controlled by the `C_CFX_INT_CMD` register.

For more details on how shared memory and HEAR FIFO memories are mapped into the ARM Cortex-M3 processor memory space, see Section 7.5, "ARM Cortex-M3 Processor Memory Space" on page 95.

The ARM Cortex-M3 processor does not have a boot ROM. Instead it relies on the CFX to initialize its memories and peripherals.

To pause and resume the ARM Cortex-M3 processing, set the `SYSTEM_CTRL_2_CM3_PAUSE` and `SYSTEM_CTRL_2_CM3_RESUME` bits respectively in the `D_SYSTEM_CTRL_2` register. When the ARM Cortex-M3 core is paused, the clock provided to the core is stopped, which prevents the execution of any functions and all responses to the NVIC or other components. All incomplete accesses to any memory by the ARM Cortex-M3 core are put into a non-blocking wait state, and these accesses will be completed once processing resumes on the ARM Cortex-M3 core. While the ARM Cortex-M3 core is paused, the CFX core can access shared memory at any address without the ARM Cortex-M3 core introducing additional delays.

As bits in the `D_SYSTEM_CTRL_2` register only change state when set, do not worry about the state of the other bits of this register when controlling the ARM Cortex-M3 processor. However, the behavior of the system is undefined if both of these bits are set by the same instruction. Reading the `SYSTEM_CTRL_2_CM3_RUNNING` bit (which shares a bit

position with the SYSTEM_CTRL2_CM3_RESUME bit) in the D_SYSTEM_CTRL_2 register provides the current status of the ARM Cortex-M3 processor.

IMPORTANT: While the ARM Cortex-M3 processor is paused, interrupt triggers are still collected by the NVIC. When processing on the ARM Cortex-M3 processor resumes, these triggers will cause their associated interrupt vectors to be executed as described in Section 15.1, "Nested Vectored Interrupt Controller (NVIC)" on page 239.

The ARM Cortex-M3 processor can also be reset by setting the SYSTEM_CTRL_2_CM3_RESET bit in the D_SYSTEM_CTRL_2 register. This resets the ARM Cortex-M3 processor core, including private peripherals, but not external interfaces and peripherals, which the CFX DSP could currently be using.

5.2 ARM Cortex-M3 Processor Debugging

All applications executing on the ARM Cortex-M3 processor can be debugged through a debug port that is integrated into the ARM Debug Access Port (DAP). It provides the interface between the I^2C debug interface module and the ARM DAP in a manner similar to the standard JTAG Debug Port (JTAG-DP) and the standard Serial Wire Debug Port (SW-DP). It provides access to all of the ARM Cortex-M3 processor registers and the memory available through the memory buses attached to the ARM Cortex-M3 processor. This transparent view of the system enables you to review the current state of the system components through your associated memory-mapped control registers, and use this information to troubleshoot most issues in your application.

For more information about debugging the ARM Cortex-M3 processor, see Section 13.3, "ARM Cortex-M3 Core Debug Port I2C Interface" on page 198, the debug components of the NVIC as described in Section 15.1, "Nested Vectored Interrupt Controller (NVIC)" on page 239, and the *Communication Protocols Manual for Ezairo 7100*.

5.3 ARM Cortex-M3 Control and Configuration Register Conventions

Control and configuration registers for the ARM Cortex-M3 core that are mapped onto the peripheral bus are accessible in three forms:

- When accessed from the CFX, these registers are accessed with a "C_" prefix (e.g., ECB_INT_STATUS is accessed as C_ECB_INT_STATUS)
- When accessed from the ARM Cortex-M3 core, these registers are accessed through CMSIS-compatible structures, with the first part of each register broken off as a named structure. To access the registers in this section use the "->" operator to access the register (e.g., ECB_INT_STATUS is accessed as ECB->INT_STATUS)
- When accessed from the ARM Cortex-M3 core in the bit-band aliased areas, the register is defined as a CMSIS compatible structure that provides aliased access to each bit-band accessible field though a structure based on the register and bit names (e.g., the RS_ENC_IN_ERR bit from the ECB_INT_STATUS register is accessed as ECB_INT_STATUS->RS_ENC_IN_ERR_ALIAS).

5.4 Control and Configuration Registers

Register Name	Register Description	Address
D_SYSTEM_CTRL_2	CFX/ARM Cortex-M3 core configuration and control	P:0xF00A
CFX_INT_CMD C_CFX_INT_CMD	ARM Cortex-M3 core to CFX core interface interrupt register	0x40000300 P:0x30C0
D_CM3_CMDS	CFX issued ARM Cortex-M3 core commands 0-7	P:0xF00B
D_CM3_NMI	ARM Cortex-M3 core non-maskable interrupt control	P:0xF00C

5.4.1 D_SYSTEM_CTRL_2 Settings

Bit Field	Field Name	Description
7	PRIORITY_CM3	Write a 1 to give the ARM Cortex-M3 core priority
6	PRIORITY_STATUS	CFX/ARM Cortex-M3 core priority status
	PRIORITY_CFX	Write a 1 to give CFX priority
5	CM3_PAUSE	Write a 1 to pause the ARM Cortex-M3 core
4	CM3_RUNNING	ARM Cortex-M3 core running status
	CM3_RESUME	Write a 1 to run the ARM Cortex-M3 core
3	CM3_RESET	Write a 1 to reset the ARM Cortex-M3 core
2	CM3_SUBSYSTEM_RESET	Write a 1 to reset the ARM Cortex-M3 core and sub-system
1	PRAM0_CM3	Write a 1 to assign PRAM0 to the ARM Cortex-M3 core
0	PRAM0_STATUS	PRAM0 CFX/ARM Cortex-M3 core assignment status
	PRAM0_CFX	Write a 1 to assign PRAM0 to CFX

Field Name	Value Symbol	Value Description	Hex Value
PRIORITY_CM3	SYSTEM2_PRIORITY_CM3	Give the ARM Cortex-M3 core priority over the CFX core	0x1
PRIORITY_STATUS	SYSTEM2_PRIORITY_STATE_CM3	ARM Cortex-M3 core has priority over CFX	0x0*
	SYSTEM2_PRIORITY_STATE_CFX	CFX has priority over ARM Cortex-M3 core	0x1
PRIORITY_CFX	SYSTEM2_PRIORITY_CFX	Give CFX priority over ARM Cortex-M3 core	0x1
CM3_PAUSE	SYSTEM2_CM3_PAUSE	Pause the ARM Cortex-M3 core	0x1
CM3_RUNNING	SYSTEM2_CM3_STATE_PAUSE	ARM Cortex-M3 core paused	0x0*
	SYSTEM2_CM3_STATE_RUN	ARM Cortex-M3 core running	0x1
CM3_RESUME	SYSTEM2_CM3_RESUME	Resume the ARM Cortex-M3 core	0x1
CM3_RESET	SYSTEM2_CM3_RESET	ARM Cortex-M3 core reset	0x1
CM3_SUBSYSTEM_RESET	SYSTEM2_CM3_SUBSYSTEM_RESET	ARM Cortex-M3 core and sub-system reset	0x1
PRAM0_CM3	SYSTEM2_PRAM0_CM3	Assign PRAM0 to the ARM Cortex-M3 core	0x1
PRAM0_STATUS	SYSTEM2_PRAM0_STATE_CFX	PRAM0 assigned to CFX	0x0*
	SYSTEM2_PRAM0_STATE_CM3	PRAM0 assigned to the ARM Cortex-M3 core	0x1
PRAM0_CFX	SYSTEM2_PRAM0_CFX	Assign PRAM0 to CFX	0x1

5.4.2 CFX_INT_CMD, C_CFX_INT_CMD Settings

Bit Field	Field Name	Description
3	CFX_CMD_3	Trigger an ARM Cortex-M3 core command 3 interrupt on the CFX
2	CFX_CMD_2	Trigger an ARM Cortex-M3 core command 2 interrupt on the CFX
1	CFX_CMD_1	Trigger an ARM Cortex-M3 core command 1 interrupt on the CFX
0	CFX_CMD_0	Trigger an ARM Cortex-M3 core command 0 interrupt on the CFX

Field Name	Value Symbol	Value Description	Hex Value
CFX_CMD_3	CFX_CMD_3	Issue an ARM Cortex-M3 core command 3 interrupt on the CFX core	0x1
CFX_CMD_2	CFX_CMD_2	Issue an ARM Cortex-M3 core command 2 interrupt on the CFX core	0x1
CFX_CMD_1	CFX_CMD_1	Issue an ARM Cortex-M3 core command 1 interrupt on the CFX core	0x1
CFX_CMD_0	CFX_CMD_0	Issue an ARM Cortex-M3 core command 0 interrupt on the CFX core	0x1

5.4.3 D_CM3_CMDS Settings

Bit Field	Field Name	Description
7	CM3_CMD_7	Write a 1 to issue ARM Cortex-M3 processor interrupt 7
6	CM3_CMD_6	Write a 1 to issue ARM Cortex-M3 processor interrupt 6
5	CM3_CMD_5	Write a 1 to issue ARM Cortex-M3 processor interrupt 5
4	CM3_CMD_4	Write a 1 to issue ARM Cortex-M3 processor interrupt 4
3	CM3_CMD_3	Write a 1 to issue ARM Cortex-M3 processor interrupt 3
2	CM3_CMD_2	Write a 1 to issue ARM Cortex-M3 processor interrupt 2
1	CM3_CMD_1	Write a 1 to issue ARM Cortex-M3 processor interrupt 1
0	CM3_CMD_0	Write a 1 to issue ARM Cortex-M3 processor interrupt 0

Field Name	Value Symbol	Value Description	Hex Value
CM3_CMD_7	CM3_CMD_7	Issue ARM Cortex-M3 processor interrupt 7	0x1
CM3_CMD_6	CM3_CMD_6	Issue ARM Cortex-M3 processor interrupt 6	0x1
CM3_CMD_5	CM3_CMD_5	Issue ARM Cortex-M3 processor interrupt 5	0x1
CM3_CMD_4	CM3_CMD_4	Issue ARM Cortex-M3 processor interrupt 4	0x1
CM3_CMD_3	CM3_CMD_3	Issue ARM Cortex-M3 processor interrupt 3	0x1
CM3_CMD_2	CM3_CMD_2	Issue ARM Cortex-M3 processor interrupt 2	0x1
CM3_CMD_1	CM3_CMD_1	Issue ARM Cortex-M3 processor interrupt 1	0x1
CM3_CMD_0	CM3_CMD_0	Issue ARM Cortex-M3 processor interrupt 0	0x1

5.4.4 D_CM3_NMI Settings

Bit Field	Field Name	Description
0	CM3_NMI_STATE	ARM Cortex-M3 core non-maskable interrupt

Field Name	Value Symbol	Value Description	Hex Value
CM3_NMI_STATE	CM3_NMI_DISABLE	Disable ARM Cortex-M3 core NMI	0x0*
	CM3_NMI_ENABLE	Enable ARM Cortex-M3 core NMI	0x1

CHAPTER 6
Filter Engine

6.1 Processor Overview

The Filter Engine is a specialized processing core that is optimized for basic audio filtering of both decimated and undecimated data in the Ezairo 7100 system. The Filter Engine can be used to implement a set of FIR and IIR data filters that contain up to 160 coefficients. FIR filters are implemented using a direct-form structure, with 5 taps per section. IIR filters are implemented as cascaded second-order sections (biquads), each implemented as a direct-form I filter.

Some Filter Engine use cases include:

- Pre-filtering of input data before it is handled by the input/output controller (IOC)
- Post-filtering of output data before it is output to the sigma-delta modulator
- Alternate decimation and interpolation of input and output data (replacing the input stage decimation filtering described in Section 10.7, “Decimation Filtering” on page 150 and output stage interpolation filtering described in Section 11.2, “Interpolation Filtering” on page 164)
- Low group-delay processing of undecimated data
- Combining decimated and undecimated data

The Filter Engine is programmable, but does not include direct debugging access. The CFX can monitor the Filter Engine state through control and configuration registers on the IOMEM bus.

Information about the Filter Engine’s features and instruction set can be found in the *Filter Engine Reference Manual for Ezairo 7100*.

6.2 Filter Engine in the Audio Path

The Filter Engine is linked to the audio input and output stage to optionally pre-process the input signal and post-process the output signal. It is also an indispensable part in forming a low group-delay processing path, where:

- The decimation filter at the input stage is bypassed.
- The interpolation filter at the output stage is bypassed.
- The audio data is processed at the undecimated sampling rate.

Figure 12 on page 112 shows how the Filter Engine input and output signals are connected to the input and output stages, and how the audio signals that bypass the Filter Engine are multiplexed with Filter Engine pre- and post-processed signals.

- Each audio input channel includes Filter Engine filters to optionally pre-process the input signal from the decimation filters, before the IOC transfers this data to an input FIFO for use by the CFX, HEAR or ARM Cortex-M3 cores. The filters can be bypassed independently for each channel.
- Each audio output channel includes Filter Engine filters to optionally post-process the output signal before it is sent to the interpolation filters. The filters can be bypassed independently for each channel.
- The Filter Engine can be configured to provide a low-delay processing path by using the undecimated input from a selected input channel's ADC, and providing this data to the selected output channel's sigma-delta modulator. This data can be optionally combined with data from the interpolation filter before being routed to an output channel.

NOTE: The fractional delay observed by the input data can impact the synchronization (i.e., the sampling time) between the different ADC driven inputs to the Filter Engine. By default, data

from the ADCs are sampled in a synchronized manner; however, changes to the fractional delay affect this synchronization. For more information, see Section 10.5.2, “Sampling Delay” on page 145.

For more information about multiplexing the Filter Engine with the audio input and output paths, see Section 9.1.1, “Audio Multiplexing with Filter Engine Outputs” on page 113.

IMPORTANT: Because inputs 4 and 5 to the Filter Engine do not have a natural sampling interval, sampling of these inputs is synchronized to their associated interpolation filters. This ensures that they are sampled at the system sampling frequency. This requires the interpolation filter for output channel 0 to be enabled to enable data input using Filter Engine input 4, and the interpolation filter for output channel 1 to be enabled to enable data input using Filter Engine input 5.

6.3 Filter Engine Control and Configuration

6.3.1 Filter Engine Control

6.3.1.1 Master Control

The CFX can start or stop the Filter Engine by setting or clearing the FENG_CTRL_ENABLE bit field of the D_FENG_CTRL register. When this field is cleared, the clock to the Filter Engine is gated immediately.

NOTE: Unless the CFX updates them, all Filter Engine registers hold their states while the Filter Engine is disabled.

6.3.1.2 Single Step

When the Filter Engine is disabled, the CFX can make the Filter Engine execute its program code for only one clock cycle by setting the FENG_CTRL_SINGLE_STEP bit field of the D_FENG_CTRL register. When this field is set, the FENG_CTRL_ENABLE bit field of the D_FENG_CTRL register is set for one clock cycle, thus allowing you to single step through Filter Engine program execution. After this one cycle, the FENG_CTRL_ENABLE and FENG_CTRL_SINGLE_STEP bits are cleared.

6.3.1.3 Pause

When the Filter Engine is enabled, the CFX can pause the Filter Engine the next time it would begin execution by setting the FENG_CTRL_PAUSE bit field of the D_FENG_CTRL register. When this field is set, the Filter Engine continues running until the next time it would start execution of a new program phase (i.e., the next time the Filter Engine would start execution of the root sector). At this time, the FENG_CTRL_ENABLE and FENG_CTRL_PAUSE bit fields are cleared.

Note that this feature is supported by the firmware macro Sys_FENG_DelayedPause, and the name DelayedPause differentiates it from the firmware macro Sys_FENG_Pause, which pauses the Filter Engine immediately (see also Section 6.3.1.1, “Master Control” on page 51).

6.3.2 Filter Engine Reset

The Filter Engine can be reset by using the two reset bits from the D_FENG_CMDS control register. When the Filter Engine is idle, setting the:

- FENG_CMDS_AUDIO_RESET bit to FENG_AUDIO_RESET resets the audio path registers (excluding the memories)
- FENG_CMDS_DECODER_RESET bit to FENG_DECODER_RESET resets the program decoder

We recommend resetting both the audio path and program decoder during application initialization.

6.3.3 Filter Engine Manual Execution

Besides the input sample ready signal that triggers Filter Engine processing, the CFX can also manually start Filter Engine program execution. This is useful for debugging, or using the Filter Engine for filtering or arithmetic operations on data that is not directly linked to the input or output stage.

To enable manual starting of Filter Engine execution, set the FENG_CFG_SRI_SEL bit field of the D_FENG_CFG register to FENG_SRI_DISABLE. Assuming that the Filter Engine is enabled and idle, the CFX can start Filter Engine processing immediately by writing FENG_EXEC to the FENG_CMDS_EXEC bit field of the D_FENG_CMDS register; this executes the Filter Engine program code once. If the Filter Engine is already active, issuing the FENG_EXEC command results in the FENG_INT_STATUS_OVERLOAD bit field being set in the D_FENG_INT_STATUS register.

6.3.4 Filter Engine Configuration

The D_FENG_CFG register has four fields that define some of the most basic aspects of the Filter Engine behavior:

Sample-ready input that triggers Filter Engine processing

For sample-triggered Filter Engine processing, select which one of the 12 input signals triggers the Filter Engine processing. If the application uses more than one sampling rate, set the FENG_CFG_SRI_SEL bit field to **x** where SR_IN**x** has the highest sampling rate.

For manual execution of the Filter Engine program code, set this field to FENG_SRI_DISABLE.

NOTE: Input data received from the decimation filters for ADC0 and ADC2 will arrive 1/8 of a sample period ahead of data from the decimation filters for ADC1 and ADC3 (inputs to the Filter Engine from the input stage decimation filters are inputs 0 to 3 in Figure 12).

Input sample capture

Disable or enable each of the 12 Filter Engine input sources. If input **x** is enabled, then whenever a new sample arrives at input **x**, it is captured in the register D_FENG_RF_IN_**x** and becomes available for both the Filter Engine and the CFX for processing.

Output shadow register capture modes

Shadow registers at the output ports can ensure that all or a subset of the output samples are synchronized. When data is written to an output via an output data register (by the Filter Engine or the CFX), the data is transferred to the output shadow register, whose update timing is controlled by the FENG_CFG_OSR_EBL bit field in the D_FENG_CFG register. The eight bits in this bit field control the shadow registers at the eight output ports:

- If the corresponding bit is set, the output shadow register is updated at the start of the program execution. Therefore, if some or all outputs have their FENG_CFG_OSR_EBL bit set to 1, these output samples are synchronized.

- If the corresponding bit is cleared, the output shadow register is updated immediately. This option minimizes system delay and is especially useful for low group delay processing.

IMPORTANT: Be careful if you use unsynchronized output sample updates. If you update the output sample through the register file during Filter Engine program execution, the following restrictions apply:

- **Do not use this output data register for intermediate data.**
- **Do not update this register file location through conditional processing by multiple instructions.**
- **In multi-rate processing, if the same register file location is updated in multiple phases, ensure that the delay from the start of the program phase to when this register file location is updated is the same in each program phase.**

Coefficient offset

It specifies an offset to add to the coefficient set index if the corresponding bit is set in the D_FENG_COEFF_OFFSET_EN_ACTIVE register. This is useful in dynamic coefficient switching. For more information, see Section 6.3.5, “Filter Engine Filter Coefficients” on page 53.

6.3.5 Filter Engine Filter Coefficients

If certain coefficient sets are used in the Filter Engine source code, the CFX must initialize them. Up to 32 coefficient sets, each supporting one biquad-IIR or 5-tap FIR filter step can be stored at address locations 0xF900 – 0xF99F in PMEM. Each set of coefficients (C0 to C31) occupies five words. For more information about filter coefficients, see the *Filter Engine Reference Manual for Ezairo 7100*.

To implement dynamic coefficient updates:

1. Specify a coefficient offset in the Filter Engine configuration register D_FENG_CFG (see Section 6.3.4, “Filter Engine Configuration” on page 52 for more information).
2. Set the coefficient offset enable bits that correspond to the coefficient set indexes, in the Filter Engine configuration register D_FENG_COEFF_OFFSET_EN_SHADOW or D_FENG_COEFF_OFFSET_EN_ACTIVE. An update to the shadow register D_FENG_COEFF_OFFSET_EN_SHADOW is transferred to the active register as soon as the Filter Engine becomes idle, or at the start of the next program execution cycle if the Filter Engine is always busy: this is the recommended approach.

   The CFX can also write to the active register to make the new value take effect immediately. However, use this feature with caution to avoid unintended memory access conflicts (see Section 6.4.2, “Memory and Register Access Conflicts” on page 55).

3. Update the inactive coefficients from CFX.

See the dynamic coefficient update section of the *Filter Engine Reference Manual for Ezairo 7100* for more information.

6.3.6 Multi-Rate Counter

The Filter Engine has two multi-rate counters and can support up to three sampling rates in the system: fs, fs/N1, and fs/N2.

If the Filter Engine only processes data at the same rate that samples become available, do not use any multi-rate counters. If an application needs to process data at a decimated rate, fs/N1, in addition to the base sampling rate, set the FENG_MRCNT_MRCNT0_DR bit field of the D_FENG_MRCNT register to (N1-1). Similarly, to configure a third sampling

rate, fs/N2, use the second multi-rate counter by setting the FENG_MRCNT_MRCNT1_DR bit field of the D_FENG_MRCNT register to (N2-1). Current counter values can be read from the bit fields FENG_MRCNT_MRCNT0 and FENG_MRCNT_MRCNT1 of the D_FENG_MRCNT register.

For information about multi-rate processing on the Filter Engine, refer to the *Filter Engine Reference Manual for Ezairo 7100*.

6.3.7 Phase Memory

The Filter Engine uses a bank of eight registers, D_FENG_PHM_PH0_PH1 to D_FENG_PHM_PH14_PH15, to store the starting addresses of program memory for each enabled program phase sector. The program phase sector enable is stored in the D_FENG_PGM_SECTOR_EN register.

Initialization data for the phase memory and program sector enable registers are generated automatically by the compilation tools. You do not need to configure these registers manually.

For more information about multi-rate processing and the program phase concept on the Filter Engine, refer to the *Filter Engine Reference Manual for Ezairo 7100*.

6.4 Filter Engine and CFX Interface

6.4.1 Filter Engine Interrupt

The Filter Engine can send an interrupt to the CFX. This interrupt is multiplexed with HEAR interrupt 6. When the HEAR_INT_SELECT_HEAR_6_FENG bit field of the register D_HEAR_INT_SELECT has the value INT_SEL_FENG, the Filter Engine interrupt is selected; otherwise, the HEAR_6 interrupt is selected.

The Filter Engine can raise an interrupt to the CFX when a new sample has arrived, if a particular condition in its processing has occurred, or a CMP instruction has met certain criteria. The Filter Engine interrupt can be set by any of the sources listed in Table 2. To enable an interrupt source, set the corresponding bit in the D_FENG_INT_ENABLE register. To check the status of an interrupt source, read the corresponding bit value in the D_FENG_INT_STATUS register.

Table 2. Filter Engine Interrupt Sources

Index	Interrupt Source	Description
0	FENG_SRIN0	A new data sample has been received at IN0; a sample ready event is triggered if SRI0 has been selected as the SRI_SRC.
1	FENG_SRIN1	A new data sample has been received at IN1; a sample ready event is triggered if SRI1 has been selected as the SRI_SRC.
2	FENG_SRIN2	A new data sample has been received at IN2; a sample ready event is triggered if SRI2 has been selected as the SRI_SRC.
3	FENG_SRIN3	A new data sample has been received at IN3; a sample ready event is triggered if SRI3 has been selected as the SRI_SRC.
4	FENG_SRIN4	A new data sample has been received at IN4; a sample ready event is triggered if SRI4 has been selected as the SRI_SRC.
5	FENG_SRIN5	A new data sample has been received at IN5; a sample ready event is triggered if SRI5 has been selected as the SRI_SRC.
6	FENG_SRIN6	A new data sample has been received at IN6; a sample ready event is triggered if SRI6 has been selected as the SRI_SRC.
7	FENG_SRIN7	A new data sample has been received at IN7; a sample ready event is triggered if SRI7 has been selected as the SRI_SRC.

Table 2. Filter Engine Interrupt Sources (Continued)

Index	Interrupt Source	Description
8	FENG_SRIN8	A new data sample has been received at IN8; a sample ready event is triggered if SRI8 as been selected as the SRI_SRC.
9	FENG_SRIN9	A new data sample has been received at IN9; a sample ready event is triggered if SRI9 has been selected as the SRI_SRC.
10	FENG_SRIN10	A new data sample has been received at IN10; a sample ready event is triggered if SRI10 has been selected as the SRI_SRC.
11	FENG_SRIN11	A new data sample has been received at IN11; a sample ready event is triggered if SRI11 has been selected as the SRI_SRC.
12	FENG_ISHC	The new input shadow register data values have been transferred into the register file (one system clock cycle after a sample ready event).
13	FENG_OUT0	New data is captured at D_FENG_RF_OUT_0
14	FENG_OUT1	New data is captured at D_FENG_RF_OUT_1
15	FENG_OUT2	New data is captured at D_FENG_RF_OUT_2
16	FENG_OUT3	New data is captured at D_FENG_RF_OUT_3
17	FENG_OUT4	New data is captured at D_FENG_RF_OUT_4
18	FENG_OUT5	New data is captured at D_FENG_RF_OUT_5
19	FENG_OUT6	New data is captured at D_FENG_RF_OUT_6
20	FENG_OUT7	New data is captured at D_FENG_RF_OUT_7
21	FENG_PGE	End of the program memory execution. The Filter Engine is idle until the next sample ready event.
22	FENG_CMP	A `CMP` instruction has met its interrupt condition
23	FENG_REG_ERR	A write conflict has occurred on the configuration registers (including the Register File) between the CFX and Filter Engine.
24	FENG_MEM_ERR	A memory access (either read or write) conflict has occurred when the CFX attempts to access Filter Engine memories (state variables, coefficients, and program memory)
25	FENG_CORE_SAT	Data saturation was detected under the core block
26	FENG_ALU_SAT	Data saturation was detected under the ALU block
27	FENG_OVERLOAD	A sample ready event or manual execution command attempted to start Filter Engine execution while the Filter Engine was still active.

6.4.2 Memory and Register Access Conflicts

The CFX can access Filter Engine memory over the P memory bus. If a memory access conflict (either read or write) occurs, the Filter Engine is granted memory access, and the CFX access is corrupted. When any access conflict occurs, any data read by the CFX is invalid, and the data to be written by the CFX is lost.

A memory access conflict occurs if the CFX attempts to access an address in the Filter Engine's IOMEM areas. When a memory access conflict occurs, the `FENG_INT_STATUS_MEM_ERR` bit field is set in the `D_FENG_INT_STATUS` register. This occurs if the CFX attempts to:

- Access the Filter Engine coefficient memory while the Filter Engine is executing the first cycle of a `BIQ` or `FIR` instruction.
- Access the state variable memory while the Filter Engine is executing a `BIQ` or `FIR` instruction.
- Access the Filter Engine program memory while the Filter Engine is active.

A register write conflict occurs if the CFX attempts to write to a configuration register (e.g., D_FENG_MRCNT) or a data register location (e.g., D_FENG_RF_SINGLE_0) when the Filter Engine is also writing to it. These registers are addressed in the filter Engine's IOREG areas. When a register write conflict occurs, the FENG_INT_STATUS_REG_ERR bit field is set in the D_FENG_INT_STATUS register.

Unlike other Filter Engine registers, the D_FENG_STATE_ACTIVE register allows individual bit write-access from the CFX, and this is controlled by the D_FENG_STATE_W_EN register. This allows both the Filter Engine and CFX to write to this register at the same time without a register write conflict, provided they are not writing to the same bits.

For more information about dividing the Filter Engine's memory-mapped components into IOMEM and IOREG memory areas, refer to Table 10 on page 82.

To avoid register write access conflicts on the offset enable register and state register, a set of shadow registers is provided. These registers can be written safely by the CFX at any time, and are copied to their equivalent active registers at the start of Filter Engine program execution. These register pairs are listed in Table 3.

Table 3. Filter Engine Shadow/Active Register Pairs

Shadow Register	Active Register
D_FENG_COEFF_OFFSET_EN_SHADOW	D_FENG_COEFF_OFFSET_EN_ACTIVE
D_FENG_STATE_SHADOW	D_FENG_STATE_ACTIVE

6.5 Control and Configuration Registers

6.5.1 Data Register Groups

There are 64 data registers with 24-bit or 50-bit word lengths to store input, output and intermediate data. Reading and writing to the same register under the same instruction is permitted. In this case, the read occurs first; the read data is processed, and at the end of the instruction execution, the result is written to the same register.

All data registers can be accessed by both the Filter Engine and the CFX. If a register access conflict occurs, it is resolved as described in Section 6.4.2, "Memory and Register Access Conflicts" on page 55.

NOTE: For an explanation of the numeric formats used to describe each register group, see the section about numeric types in the *Firmware Reference Manual for Ezairo 7100*.

6.5.1.1 Extended-Precision Data Registers

Registers D_FENG_RF_EXTENDED_0 to D_FENG_RF_EXTENDED_15 are used to store intermediate data with higher resolution. These registers are not linked to the I/O interface. These 50-bit extended-precision registers have an m4p46 format when used to store linear audio data, or an m11p39 format when used to store log2-format data.

6.5.1.2 Single-Precision Data Registers

Registers D_FENG_RF_SINGLE_0 to D_FENG_RF_SINGLE_27 are used to store intermediate data with lower resolution. These registers are not linked to the I/O interface. These 24-bit single-precision registers have a frac24 format when used to store linear audio data, or an m8p16 format when used to store log2-format data.

6.5.1.3 Input Data Registers

Registers D_FENG_RF_IN_0 to D_FENG_RF_IN_11 hold input data and are linked to the input stage. Input data registers can also be used to store single-precision intermediate data if no more intermediate data registers are available.

These 24-bit input data registers have a frac24 format when used to store linear audio data, or an m8p16 format when used to store log2-format data.

6.5.1.4 Output Data Registers

Registers D_FENG_RF_OUT_0 to D_FENG_RF_OUT_7 hold output data and are linked to the output stage. Output data registers can also be used to store single-precision intermediate data if no more intermediate data registers are available. These 24-bit output data registers have a frac24 format when used to store linear audio data, or an m8p16 format when used to store log2-format data.

6.5.2 Filter Engine Registers

Register Name	Register Description	Address
D_FENG_INT_STATUS	Filter Engine interrupt status register	P:0xF300
D_FENG_INT_ENABLE	Filter Engine interrupt enable register	P:0xF301
D_FENG_CFG	Filter Engine configuration register	P:0xF302
D_FENG_MRCNT	Filter Engine multi-rate counter register control	P:0xF303
D_FENG_PGM_SECTOR_EN	Filter Engine program memory sector enables	P:0xF304
D_FENG_PHM_PH0_PH1	Filter Engine phase memory phase 0 and phase 1	P:0xF305
D_FENG_PHM_PH2_PH3	Filter Engine phase memory phase 2 and phase 3	P:0xF306
D_FENG_PHM_PH4_PH5	Filter Engine phase memory phase 4 and phase 5	P:0xF307
D_FENG_PHM_PH6_PH7	Filter Engine phase memory phase 6 and phase 7	P:0xF308
D_FENG_PHM_PH8_PH9	Filter Engine phase memory phase 8 and phase 9	P:0xF309
D_FENG_PHM_PH10_PH11	Filter Engine phase memory phase 10 and phase 11	P:0xF30A
D_FENG_PHM_PH12_PH13	Filter Engine phase memory phase 12 and phase 13	P:0xF30B
D_FENG_PHM_PH14_PH15	Filter Engine phase memory phase 14 and phase 15	P:0xF30C
D_FENG_COEFF_OFFSET_EN_SHADOW	Filter Engine coefficient read address offset enable shadow register	P:0xF30D
D_FENG_COEFF_OFFSET_EN_ACTIVE	Filter Engine coefficient read address offset enable active register	P:0xF30E
D_FENG_STATE_W_EN	Filter Engine conditional processing state register write enable	P:0xF313
D_FENG_STATE_ACTIVE	Filter Engine conditional processing state active register	P:0xF314
D_FENG_STATE_SHADOW	Filter Engine conditional processing state shadow register	P:0xF30F
D_FENG_CMDS	Filter Engine commands register	P:0xF310
D_FENG_PGM_ADDR	Filter Engine program memory decoder address	P:0xF311
D_FENG_CTRL	Filter Engine master clock control	P:0xF312
D_FENG_RF_EXTENDED_0	Filter Engine extended-precision data register 0	P:0xF340
D_FENG_RF_EXTENDED_1	Filter Engine extended-precision data register 1	P:0xF341
D_FENG_RF_EXTENDED_2	Filter Engine extended-precision data register 2	P:0xF342
D_FENG_RF_EXTENDED_3	Filter Engine extended-precision data register 3	P:0xF343
D_FENG_RF_EXTENDED_4	Filter Engine extended-precision data register 4	P:0xF344
D_FENG_RF_EXTENDED_5	Filter Engine extended-precision data register 5	P:0xF345
D_FENG_RF_EXTENDED_6	Filter Engine extended-precision data register 6	P:0xF346
D_FENG_RF_EXTENDED_7	Filter Engine extended-precision data register 7	P:0xF347

Register Name	Register Description	Address
D_FENG_RF_EXTENDED_8	Filter Engine extended-precision data register 8	P:0xF348
D_FENG_RF_EXTENDED_9	Filter Engine extended-precision data register 9	P:0xF349
D_FENG_RF_EXTENDED_10	Filter Engine extended-precision data register 10	P:0xF34A
D_FENG_RF_EXTENDED_11	Filter Engine extended-precision data register 11	P:0xF34B
D_FENG_RF_EXTENDED_12	Filter Engine extended-precision data register 12	P:0xF34C
D_FENG_RF_EXTENDED_13	Filter Engine extended-precision data register 13	P:0xF34D
D_FENG_RF_EXTENDED_14	Filter Engine extended-precision data register 14	P:0xF34E
D_FENG_RF_EXTENDED_15	Filter Engine extended-precision data register 15	P:0xF34F
D_FENG_RF_SINGLE_0	Filter Engine single-precision data register 0	P:0xF350
D_FENG_RF_SINGLE_1	Filter Engine single-precision data register 1	P:0xF351
D_FENG_RF_SINGLE_2	Filter Engine single-precision data register 2	P:0xF352
D_FENG_RF_SINGLE_3	Filter Engine single-precision data register 3	P:0xF353
D_FENG_RF_SINGLE_4	Filter Engine single-precision data register 4	P:0xF354
D_FENG_RF_SINGLE_5	Filter Engine single-precision data register 5	P:0xF355
D_FENG_RF_SINGLE_6	Filter Engine single-precision data register 6	P:0xF356
D_FENG_RF_SINGLE_7	Filter Engine single-precision data register 7	P:0xF357
D_FENG_RF_SINGLE_8	Filter Engine single-precision data register 8	P:0xF358
D_FENG_RF_SINGLE_9	Filter Engine single-precision data register 9	P:0xF359
D_FENG_RF_SINGLE_10	Filter Engine single-precision data register 10	P:0xF35A
D_FENG_RF_SINGLE_11	Filter Engine single-precision data register 11	P:0xF35B
D_FENG_RF_SINGLE_12	Filter Engine single-precision data register 12	P:0xF35C
D_FENG_RF_SINGLE_13	Filter Engine single-precision data register 13	P:0xF35D
D_FENG_RF_SINGLE_14	Filter Engine single-precision data register 14	P:0xF35E
D_FENG_RF_SINGLE_15	Filter Engine single-precision data register 15	P:0xF35F
D_FENG_RF_SINGLE_16	Filter Engine single-precision data register 16	P:0xF360
D_FENG_RF_SINGLE_17	Filter Engine single-precision data register 17	P:0xF361
D_FENG_RF_SINGLE_18	Filter Engine single-precision data register 18	P:0xF362
D_FENG_RF_SINGLE_19	Filter Engine single-precision data register 19	P:0xF363
D_FENG_RF_SINGLE_20	Filter Engine single-precision data register 20	P:0xF364
D_FENG_RF_SINGLE_21	Filter Engine single-precision data register 21	P:0xF365
D_FENG_RF_SINGLE_22	Filter Engine single-precision data register 22	P:0xF366
D_FENG_RF_SINGLE_23	Filter Engine single-precision data register 23	P:0xF367
D_FENG_RF_SINGLE_24	Filter Engine single-precision data register 24	P:0xF368
D_FENG_RF_SINGLE_25	Filter Engine single-precision data register 25	P:0xF369
D_FENG_RF_SINGLE_26	Filter Engine single-precision data register 26	P:0xF36A
D_FENG_RF_SINGLE_27	Filter Engine single-precision data register 27	P:0xF36B
D_FENG_RF_IN_0	Filter Engine single-precision input data register 0	P:0xF36C
D_FENG_RF_IN_1	Filter Engine single-precision input data register 1	P:0xF36D
D_FENG_RF_IN_2	Filter Engine single-precision input data register 2	P:0xF36E

Register Name	Register Description	Address
D_FENG_RF_IN_3	Filter Engine single-precision input data register 3	P:0xF36F
D_FENG_RF_IN_4	Filter Engine single-precision input data register 4	P:0xF370
D_FENG_RF_IN_5	Filter Engine single-precision input data register 5	P:0xF371
D_FENG_RF_IN_6	Filter Engine single-precision input data register 6	P:0xF372
D_FENG_RF_IN_7	Filter Engine single-precision input data register 7	P:0xF373
D_FENG_RF_IN_8	Filter Engine single-precision input data register 8	P:0xF374
D_FENG_RF_IN_9	Filter Engine single-precision input data register 9	P:0xF375
D_FENG_RF_IN_10	Filter Engine single-precision input data register 10	P:0xF376
D_FENG_RF_IN_11	Filter Engine single-precision input data register 11	P:0xF377
D_FENG_RF_OUT_0	Filter Engine single-precision output data register 0	P:0xF378
D_FENG_RF_OUT_1	Filter Engine single-precision output data register 1	P:0xF379
D_FENG_RF_OUT_2	Filter Engine single-precision output data register 2	P:0xF37A
D_FENG_RF_OUT_3	Filter Engine single-precision output data register 3	P:0xF37B
D_FENG_RF_OUT_4	Filter Engine single-precision output data register 4	P:0xF37C
D_FENG_RF_OUT_5	Filter Engine single-precision output data register 5	P:0xF37D
D_FENG_RF_OUT_6	Filter Engine single-precision output data register 6	P:0xF37E
D_FENG_RF_OUT_7	Filter Engine single-precision output data register 7	P:0xF37F

6.5.2.1 D_FENG_INT_STATUS Settings

Bit Field	Field Name	Description
27	OVERLOAD	Processor overload interrupt status
26	ALU_SAT	ALU saturation interrupt status
25	CORE_SAT	Core saturation interrupt status
24	MEM_ERR	Illegal memory access interrupt status
23	REG_ERR	Register write access collision interrupt status
22	CMP	Compare instruction interrupt status
21	PGE	End of program execution interrupt status
20	OUT7	New data capture at Filter Engine output 7 interrupt status
19	OUT6	New data capture at Filter Engine output 6 interrupt status
18	OUT5	New data capture at Filter Engine output 5 interrupt status
17	OUT4	New data capture at Filter Engine output 4 interrupt status
16	OUT3	New data capture at Filter Engine output 3 interrupt status
15	OUT2	New data capture at Filter Engine output 2 interrupt status
14	OUT1	New data capture at Filter Engine output 1 interrupt status
13	OUT0	New data capture at Filter Engine output 0 interrupt status
12	ISHC	Input shadow registers capture in RF interrupt status
11	SRIN11	New Filter Engine input 11 and signal ready arrival interrupt status
10	SRIN10	New Filter Engine input 10 and signal ready arrival interrupt status

Bit Field	Field Name	Description
9	SRIN9	New Filter Engine input 9 and signal ready arrival interrupt status
8	SRIN8	New Filter Engine input 8 and signal ready arrival interrupt status
7	SRIN7	New Filter Engine input 7 and signal ready arrival interrupt status
6	SRIN6	New Filter Engine input 6 and signal ready arrival interrupt status
5	SRIN5	New Filter Engine input 5 and signal ready arrival interrupt status
4	SRIN4	New Filter Engine input 4 and signal ready arrival interrupt status
3	SRIN3	New Filter Engine input 3 and signal ready arrival interrupt status
2	SRIN2	New Filter Engine input 2 and signal ready arrival interrupt status
1	SRIN1	New Filter Engine input 1 and signal ready arrival interrupt status
0	SRIN0	New Filter Engine input 0 and signal ready arrival interrupt status

Field Name	Value Symbol	Value Description	Hex Value
OVERLOAD	INT_ACK_FENG_OVERLOAD	Acknowledge the interrupt	0x1
	INT_IDLE_FENG_OVERLOAD	Interrupt is not pending	0x0*
	INT_PENDING_FENG_OVERLOAD	Configured sample ready input is active before execution completion interrupt status	0x1
ALU_SAT	INT_ACK_FENG_ALU_SAT	Acknowledge the interrupt	0x1
	INT_IDLE_FENG_ALU_SAT	Interrupt is not pending	0x0*
	INT_PENDING_FENG_ALU_SAT	A data saturation is detected under the ALU block	0x1
CORE_SAT	INT_ACK_FENG_CORE_SAT	Acknowledge the interrupt	0x1
	INT_IDLE_FENG_CORE_SAT	Interrupt is not pending	0x0*
	INT_PENDING_FENG_CORE_SAT	A data saturation is detected under the Filter Engine core block	0x1
MEM_ERR	INT_ACK_FENG_MEM_ERR	Acknowledge the interrupt	0x1
	INT_IDLE_FENG_MEM_ERR	Interrupt is not pending	0x0*
	INT_PENDING_FENG_MEM_ERR	CFX executed an illegal memory access	0x1
REG_ERR	INT_ACK_FENG_REG_ERR	Acknowledge the interrupt	0x1
	INT_IDLE_FENG_REG_ERR	Interrupt is not pending	0x0*
	INT_PENDING_FENG_REG_ERR	CFX tried to write into registers at the same time as Filter Engine	0x1
CMP	INT_ACK_FENG_CMP	Acknowledge the interrupt	0x1
	INT_IDLE_FENG_CMP	Interrupt is not pending	0x0*
	INT_PENDING_FENG_CMP	A CMP instruction with its interrupt field enabled modified the FE_STATE register	0x1
PGE	INT_ACK_FENG_PGE	Acknowledge the interrupt	0x1
	INT_IDLE_FENG_PGE	Interrupt is not pending	0x0*
	INT_PENDING_FENG_PGE	Filter Engine finished executing the program	0x1

Field Name	Value Symbol	Value Description	Hex Value
OUT7	INT_ACK_FENG_OUT7	Acknowledge the interrupt	0x1
	INT_IDLE_FENG_OUT7	Interrupt is not pending	0x0*
	INT_PENDING_FENG_OUT7	New output data captured for transfer into output shadow register	0x1
OUT6	INT_ACK_FENG_OUT6	Acknowledge the interrupt	0x1
	INT_IDLE_FENG_OUT6	Interrupt is not pending	0x0*
	INT_PENDING_FENG_OUT6	New output data captured for transfer into output shadow register	0x1
OUT5	INT_ACK_FENG_OUT5	Acknowledge the interrupt	0x1
	INT_IDLE_FENG_OUT5	Interrupt is not pending	0x0*
	INT_PENDING_FENG_OUT5	New output data captured for transfer into output shadow register	0x1
OUT4	INT_ACK_FENG_OUT4	Acknowledge the interrupt	0x1
	INT_IDLE_FENG_OUT4	Interrupt is not pending	0x0*
	INT_PENDING_FENG_OUT4	New output data captured for transfer into output shadow register	0x1
OUT3	INT_ACK_FENG_OUT3	Acknowledge the interrupt	0x1
	INT_IDLE_FENG_OUT3	Interrupt is not pending	0x0*
	INT_PENDING_FENG_OUT3	New output data captured for transfer into output shadow register	0x1
OUT2	INT_ACK_FENG_OUT2	Acknowledge the interrupt	0x1
	INT_IDLE_FENG_OUT2	Interrupt is not pending	0x0*
	INT_PENDING_FENG_OUT2	New output data captured for transfer into output shadow register	0x1
OUT1	INT_ACK_FENG_OUT1	Acknowledge the interrupt	0x1
	INT_IDLE_FENG_OUT1	Interrupt is not pending	0x0*
	INT_PENDING_FENG_OUT1	New output data captured for transfer into output shadow register	0x1
OUT0	INT_ACK_FENG_OUT0	Acknowledge the interrupt	0x1
	INT_IDLE_FENG_OUT0	Interrupt is not pending	0x0*
	INT_PENDING_FENG_OUT0	New output data captured for transfer into output shadow register	0x1
ISHC	INT_ACK_FENG_ISHC	Acknowledge the interrupt	0x1
	INT_IDLE_FENG_ISHC	Interrupt is not pending	0x0*
	INT_PENDING_FENG_ISHC	There are updated input shadow registers that have been transferred to the input registers	0x1
SRIN11	INT_ACK_FENG_SRIN11	Acknowledge the interrupt	0x1
	INT_IDLE_FENG_SRIN11	Interrupt is not pending	0x0*
	INT_PENDING_FENG_SRIN11	A new input sample and sample ready signal have arrived	0x1

Field Name	Value Symbol	Value Description	Hex Value
SRIN10	INT_ACK_FENG_SRIN10	Acknowledge the interrupt	0x1
	INT_IDLE_FENG_SRIN10	Interrupt is not pending	0x0*
	INT_PENDING_FENG_SRIN10	A new input sample and sample ready signal have arrived	0x1
SRIN9	INT_ACK_FENG_SRIN9	Acknowledge the interrupt	0x1
	INT_IDLE_FENG_SRIN9	Interrupt is not pending	0x0*
	INT_PENDING_FENG_SRIN9	A new input sample and sample ready signal have arrived	0x1
SRIN8	INT_ACK_FENG_SRIN8	Acknowledge the interrupt	0x1
	INT_IDLE_FENG_SRIN8	Interrupt is not pending	0x0*
	INT_PENDING_FENG_SRIN8	A new input sample and sample ready signal have arrived	0x1
SRIN7	INT_ACK_FENG_SRIN7	Acknowledge the interrupt	0x1
	INT_IDLE_FENG_SRIN7	Interrupt is not pending	0x0*
	INT_PENDING_FENG_SRIN7	A new input sample and sample ready signal have arrived	0x1
SRIN6	INT_ACK_FENG_SRIN6	Acknowledge the interrupt	0x1
	INT_IDLE_FENG_SRIN6	Interrupt is not pending	0x0*
	INT_PENDING_FENG_SRIN6	A new input sample and sample ready signal have arrived	0x1
SRIN5	INT_ACK_FENG_SRIN5	Acknowledge the interrupt	0x1
	INT_IDLE_FENG_SRIN5	Interrupt is not pending	0x0*
	INT_PENDING_FENG_SRIN5	A new input sample and sample ready signal have arrived	0x1
SRIN4	INT_ACK_FENG_SRIN4	Acknowledge the interrupt	0x1
	INT_IDLE_FENG_SRIN4	Interrupt is not pending	0x0*
	INT_PENDING_FENG_SRIN4	A new input sample and sample ready signal have arrived	0x1
SRIN3	INT_ACK_FENG_SRIN3	Acknowledge the interrupt	0x1
	INT_IDLE_FENG_SRIN3	Interrupt is not pending	0x0*
	INT_PENDING_FENG_SRIN3	A new input sample and sample ready signal have arrived	0x1
SRIN2	INT_ACK_FENG_SRIN2	Acknowledge the interrupt	0x1
	INT_IDLE_FENG_SRIN2	Interrupt is not pending	0x0*
	INT_PENDING_FENG_SRIN2	A new input sample and sample ready signal have arrived	0x1
SRIN1	INT_ACK_FENG_SRIN1	Acknowledge the interrupt	0x1
	INT_IDLE_FENG_SRIN1	Interrupt is not pending	0x0*
	INT_PENDING_FENG_SRIN1	A new input sample and sample ready signal have arrived	0x1

Field Name	Value Symbol	Value Description	Hex Value
SRIN0	INT_ACK_FENG_SRIN0	Acknowledge the interrupt	0x1
	INT_IDLE_FENG_SRIN0	Interrupt is not pending	0x0*
	INT_PENDING_FENG_SRIN0	A new input sample and sample ready signal have arrived	0x1

6.5.2.2 D_FENG_INT_ENABLE Settings

Bit Field	Field Name	Description
27	OVERLOAD	Processor overload interrupt enable
26	ALU_SAT	ALU saturation interrupt enable
25	CORE_SAT	Core saturation interrupt enable
24	MEM_ERR	Illegal memory access interrupt enable
23	REG_ERR	Register write access collision interrupt enable
22	CMP	Compare instruction interrupt enable
21	PGE	End of program execution interrupt enable
20	OUT7	New data capture at Filter Engine output 7 interrupt enable
19	OUT6	New data capture at Filter Engine output 6 interrupt enable
18	OUT5	New data capture at Filter Engine output 5 interrupt enable
17	OUT4	New data capture at Filter Engine output 4 interrupt enable
16	OUT3	New data capture at Filter Engine output 3 interrupt enable
15	OUT2	New data capture at Filter Engine output 2 interrupt enable
14	OUT1	New data capture at Filter Engine output 1 interrupt enable
13	OUT0	New data capture at Filter Engine output 0 interrupt enable
12	ISHC	Input shadow registers captured in RF interrupt enable
11	SRIN11	New Filter Engine input 11 and signal ready arrival interrupt enable
10	SRIN10	New Filter Engine input 10 and signal ready arrival interrupt enable
9	SRIN9	New Filter Engine input 9 and signal ready arrival interrupt enable
8	SRIN8	New Filter Engine input 8 and signal ready arrival interrupt enable
7	SRIN7	New Filter Engine input 7 and signal ready arrival interrupt enable
6	SRIN6	New Filter Engine input 6 and signal ready arrival interrupt enable
5	SRIN5	New Filter Engine input 5 and signal ready arrival interrupt enable
4	SRIN4	New Filter Engine input 4 and signal ready arrival interrupt enable
3	SRIN3	New Filter Engine input 3 and signal ready arrival interrupt enable
2	SRIN2	New Filter Engine input 2 and signal ready arrival interrupt enable
1	SRIN1	New Filter Engine input 1 and signal ready arrival interrupt enable
0	SRIN0	New Filter Engine input 0 and signal ready arrival interrupt enable

Field Name	Value Symbol	Value Description	Hex Value
OVERLOAD	INT_DIS_FENG_OVERLOAD	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_OVERLOAD	This source can set the Filter Engine interrupt	0x1
ALU_SAT	INT_DIS_FENG_ALU_SAT	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_ALU_SAT	This source can set the Filter Engine interrupt	0x1
CORE_SAT	INT_DIS_FENG_CORE_SAT	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_CORE_SAT	This source can set the Filter Engine interrupt	0x1
MEM_ERR	INT_DIS_FENG_MEM_ERR	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_MEM_ERR	This source can set the Filter Engine interrupt	0x1
REG_ERR	INT_DIS_FENG_REG_ERR	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_REG_ERR	This source can set the Filter Engine interrupt	0x1
CMP	INT_DIS_FENG_CMP	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_CMP	This source can set the Filter Engine interrupt	0x1
PGE	INT_DIS_FENG_PGE	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_PGE	This source can set the Filter Engine interrupt	0x1
OUT7	INT_DIS_FENG_OUT7	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_OUT7	This source can set the Filter Engine interrupt	0x1
OUT6	INT_DIS_FENG_OUT6	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_OUT6	This source can set the Filter Engine interrupt	0x1
OUT5	INT_DIS_FENG_OUT5	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_OUT5	This source can set the Filter Engine interrupt	0x1
OUT4	INT_DIS_FENG_OUT4	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_OUT4	This source can set the Filter Engine interrupt	0x1
OUT3	INT_DIS_FENG_OUT3	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_OUT3	This source can set the Filter Engine interrupt	0x1
OUT2	INT_DIS_FENG_OUT2	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_OUT2	This source can set the Filter Engine interrupt	0x1
OUT1	INT_DIS_FENG_OUT1	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_OUT1	This source can set the Filter Engine interrupt	0x1
OUT0	INT_DIS_FENG_OUT0	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_OUT0	This source can set the Filter Engine interrupt	0x1
ISHC	INT_DIS_FENG_ISHC	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_ISHC	This source can set the Filter Engine interrupt	0x1
SRIN11	INT_DIS_FENG_SRIN11	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_SRIN11	This source can set the Filter Engine interrupt	0x1
SRIN10	INT_DIS_FENG_SRIN10	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_SRIN10	This source can set the Filter Engine interrupt	0x1
SRIN9	INT_DIS_FENG_SRIN9	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_SRIN9	This source can set the Filter Engine interrupt	0x1

Field Name	Value Symbol	Value Description	Hex Value
SRIN8	INT_DIS_FENG_SRIN8	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_SRIN8	This source can set the Filter Engine interrupt	0x1
SRIN7	INT_DIS_FENG_SRIN7	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_SRIN7	This source can set the Filter Engine interrupt	0x1
SRIN6	INT_DIS_FENG_SRIN6	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_SRIN6	This source can set the Filter Engine interrupt	0x1
SRIN5	INT_DIS_FENG_SRIN5	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_SRIN5	This source can set the Filter Engine interrupt	0x1
SRIN4	INT_DIS_FENG_SRIN4	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_SRIN4	This source can set the Filter Engine interrupt	0x1
SRIN3	INT_DIS_FENG_SRIN3	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_SRIN3	This source can set the Filter Engine interrupt	0x1
SRIN2	INT_DIS_FENG_SRIN2	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_SRIN2	This source can set the Filter Engine interrupt	0x1
SRIN1	INT_DIS_FENG_SRIN1	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_SRIN1	This source can set the Filter Engine interrupt	0x1
SRIN0	INT_DIS_FENG_SRIN0	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_SRIN0	This source can set the Filter Engine interrupt	0x1

6.5.2.3 D_FENG_CFG Settings

Bit Field	Field Name	Description
28:24	COEFF_OFFSET	The coefficient decoder offset
23	OSR7_EBL	Shadow register capture mode for output 7
22	OSR6_EBL	Shadow register capture mode for output 6
21	OSR5_EBL	Shadow register capture mode for output 5
20	OSR4_EBL	Shadow register capture mode for output 4
19	OSR3_EBL	Shadow register capture mode for output 3
18	OSR2_EBL	Shadow register capture mode for output 2
17	OSR1_EBL	Shadow register capture mode for output 1
16	OSR0_EBL	Shadow register capture mode for output 0
15	FIN11_EBL	Enable/Disable sample capture on input 11
14	FIN10_EBL	Enable/Disable sample capture on input 10
13	FIN9_EBL	Enable/Disable sample capture on input 9
12	FIN8_EBL	Enable/Disable sample capture on input 8
11	FIN7_EBL	Enable/Disable sample capture on input 7
10	FIN6_EBL	Enable/Disable sample capture on input 6
9	FIN5_EBL	Enable/Disable sample capture on input 5
8	FIN4_EBL	Enable/Disable sample capture on input 4

Bit Field	Field Name	Description
7	FIN3_EBL	Enable/Disable sample capture on input 3
6	FIN2_EBL	Enable/Disable sample capture on input 2
5	FIN1_EBL	Enable/Disable sample capture on input 1
4	FIN0_EBL	Enable/Disable sample capture on input 0
3:0	SRI_SEL	Sample ready input starting the Filter Engine processing

Field Name	Value Symbol	Value Description	Hex Value
OSR7_EBL	FENG_OSR7_DISABLE	Disable the shadow register for Filter Engine output 7	0x0*
	FENG_OSR7_ENABLE	Enable the shadow register for Filter Engine output 7	0x1
OSR6_EBL	FENG_OSR6_DISABLE	Disable the shadow register for Filter Engine output 6	0x0*
	FENG_OSR6_ENABLE	Enable the shadow register for Filter Engine output 6	0x1
OSR5_EBL	FENG_OSR5_DISABLE	Disable the shadow register for Filter Engine output 5	0x0
	FENG_OSR5_ENABLE	Enable the shadow register for Filter Engine output 5	0x1*
OSR4_EBL	FENG_OSR4_DISABLE	Disable the shadow register for Filter Engine output 4	0x0
	FENG_OSR4_ENABLE	Enable the shadow register for Filter Engine output 4	0x1*
OSR3_EBL	FENG_OSR3_DISABLE	Disable the shadow register for Filter Engine output 3	0x0
	FENG_OSR3_ENABLE	Enable the shadow register for Filter Engine output 3	0x1*
OSR2_EBL	FENG_OSR2_DISABLE	Disable the shadow register for Filter Engine output 2	0x0
	FENG_OSR2_ENABLE	Enable the shadow register for Filter Engine output 2	0x1*
OSR1_EBL	FENG_OSR1_DISABLE	Disable the shadow register for Filter Engine output 1	0x0
	FENG_OSR1_ENABLE	Enable the shadow register for Filter Engine output 1	0x1*
OSR0_EBL	FENG_OSR0_DISABLE	Disable the shadow register for Filter Engine output 0	0x0
	FENG_OSR0_ENABLE	Enable the shadow register for Filter Engine output 0	0x1*
FIN11_EBL	FENG_FIN11_DISABLE	Disable Filter Engine input 11	0x0*
	FENG_FIN11_ENABLE	Enable Filter Engine input 11	0x1
FIN10_EBL	FENG_FIN10_DISABLE	Disable Filter Engine input 10	0x0*
	FENG_FIN10_ENABLE	Enable Filter Engine input 10	0x1
FIN9_EBL	FENG_FIN9_DISABLE	Disable Filter Engine input 9	0x0*
	FENG_FIN9_ENABLE	Enable Filter Engine input 9	0x1
FIN8_EBL	FENG_FIN8_DISABLE	Disable Filter Engine input 8	0x0*
	FENG_FIN8_ENABLE	Enable Filter Engine input 8	0x1
FIN7_EBL	FENG_FIN7_DISABLE	Disable Filter Engine input 7	0x0*
	FENG_FIN7_ENABLE	Enable Filter Engine input 7	0x1
FIN6_EBL	FENG_FIN6_DISABLE	Disable Filter Engine input 6	0x0*
	FENG_FIN6_ENABLE	Enable Filter Engine input 6	0x1
FIN5_EBL	FENG_FIN5_DISABLE	Disable Filter Engine input 5	0x0*
	FENG_FIN5_ENABLE	Enable Filter Engine input 5	0x1

Field Name	Value Symbol	Value Description	Hex Value
FIN4_EBL	FENG_FIN4_DISABLE	Disable Filter Engine input 4	0x0*
	FENG_FIN4_ENABLE	Enable Filter Engine input 4	0x1
FIN3_EBL	FENG_FIN3_DISABLE	Disable Filter Engine input 3	0x0*
	FENG_FIN3_ENABLE	Enable Filter Engine input 3	0x1
FIN2_EBL	FENG_FIN2_DISABLE	Disable Filter Engine input 2	0x0*
	FENG_FIN2_ENABLE	Enable Filter Engine input 2	0x1
FIN1_EBL	FENG_FIN1_DISABLE	Disable Filter Engine input 1	0x0*
	FENG_FIN1_ENABLE	Enable Filter Engine input 1	0x1
FIN0_EBL	FENG_FIN0_DISABLE	Disable Filter Engine input 0	0x0*
	FENG_FIN0_ENABLE	Enable Filter Engine input 0	0x1
SRI_SEL	FENG_SRI_SELECT_IN0	Filter Engine executes on input sample ready 0	0x0
	FENG_SRI_SELECT_IN1	Filter Engine executes on input sample ready 1	0x1
	FENG_SRI_SELECT_IN2	Filter Engine executes on input sample ready 2	0x2
	FENG_SRI_SELECT_IN3	Filter Engine executes on input sample ready 3	0x3
	FENG_SRI_SELECT_IN4	Filter Engine executes on input sample ready 4	0x4
	FENG_SRI_SELECT_IN5	Filter Engine executes on input sample ready 5	0x5
	FENG_SRI_SELECT_IN6	Filter Engine executes on input sample ready 6	0x6
	FENG_SRI_SELECT_IN7	Filter Engine executes on input sample ready 7	0x7
	FENG_SRI_SELECT_IN8	Filter Engine executes on input sample ready 8	0x8
	FENG_SRI_SELECT_IN9	Filter Engine executes on input sample ready 9	0x9
	FENG_SRI_SELECT_IN10	Filter Engine executes on input sample ready 10	0xA
	FENG_SRI_SELECT_IN11	Filter Engine executes on input sample ready 11	0xB
	FENG_SRI_DISABLE	Filter Engine is disabled	0xF*

6.5.2.4 D_FENG_MRCNT Settings

Bit Field	Field Name	Description
15:12	MRCNT1	Multi-rate counter 1 value
11:8	MRCNT0	Multi-rate counter 0 value
7:4	MRCNT1_DR	Multi-rate counter 1 divide ratio (0 to15)
3:0	MRCNT0_DR	Multi-rate counter 0 divide ratio (0 to15)

6.5.2.5 D_FENG_PGM_SECTOR_EN Settings

Bit Field	Field Name	Description
31:16	CNT1	FENG_MRCNT1 counter phase controlled sectors enable
15:0	CNT0	FENG_MRCNT0 counter phase controlled sectors enable

6.5.2.6 D_FENG_PHM_PH0_PH1 Settings

Bit Field	Field Name	Description
23:18	PHM_CNT1PH1	FENG_MRCNT1 phase 1 controlled program memory sector start
17:12	PHM_CNT0PH1	FENG_MRCNT0 phase 1 controlled program memory sector start
11:6	PHM_CNT1PH0	FENG_MRCNT1 phase 0 controlled program memory sector start
5:0	PHM_CNT0PH0	FENG_MRCNT0 phase 0 controlled program memory sector start

6.5.2.7 D_FENG_PHM_PH2_PH3 Settings

Bit Field	Field Name	Description
23:18	PHM_CNT1PH3	FENG_MRCNT1 phase 3 controlled program memory sector start
17:12	PHM_CNT0PH3	FENG_MRCNT0 phase 3 controlled program memory sector start
11:6	PHM_CNT1PH2	FENG_MRCNT1 phase 2 controlled program memory sector start
5:0	PHM_CNT0PH2	FENG_MRCNT0 phase 2 controlled program memory sector start

6.5.2.8 D_FENG_PHM_PH4_PH5 Settings

Bit Field	Field Name	Description
23:18	PHM_CNT1PH5	FE_MRCNT1 phase 5 controlled program memory sector start
17:12	PHM_CNT0PH5	FE_MRCNT0 phase 5 controlled program memory sector start
11:6	PHM_CNT1PH4	FE_MRCNT1 phase 4 controlled program memory sector start
5:0	PHM_CNT0PH4	FE_MRCNT0 phase 4 controlled program memory sector start

6.5.2.9 D_FENG_PHM_PH6_PH7 Settings

Bit Field	Field Name	Description
23:18	PHM_CNT1PH7	FENG_MRCNT1 phase 7 controlled program memory sector start
17:12	PHM_CNT0PH7	FENG_MRCNT0 phase 7 controlled program memory sector start
11:6	PHM_CNT1PH6	FENG_MRCNT1 phase 6 controlled program memory sector start
5:0	PHM_CNT0PH6	FENG_MRCNT0 phase 6 controlled program memory sector start

6.5.2.10 D_FENG_PHM_PH8_PH9 Settings

Bit Field	Field Name	Description
23:18	PHM_CNT1PH9	FENG_MRCNT1 phase 9 controlled program memory sector start
17:12	PHM_CNT0PH9	FENG_MRCNT0 phase 9 controlled program memory sector start
11:6	PHM_CNT1PH8	FENG_MRCNT1 phase 8 controlled program memory sector start
5:0	PHM_CNT0PH8	FENG_MRCNT0 phase 8 controlled program memory sector start

6.5.2.11 D_FENG_PHM_PH10_PH11 Settings

Bit Field	Field Name	Description
23:18	PHM_CNT1PH11	FENG_MRCNT1 phase 11 controlled program memory sector start
17:12	PHM_CNT0PH11	FENG_MRCNT0 phase 11 controlled program memory sector start
11:6	PHM_CNT1PH10	FENG_MRCNT1 phase 10 controlled program memory sector start
5:0	PHM_CNT0PH10	FENG_MRCNT0 phase 10 controlled program memory sector start

6.5.2.12 D_FENG_PHM_PH12_PH13 Settings

Bit Field	Field Name	Description
23:18	PHM_CNT1PH13	FENG_MRCNT1 phase 13 controlled program memory sector start
17:12	PHM_CNT0PH13	FENG_MRCNT0 phase 13 controlled program memory sector start
11:6	PHM_CNT1PH12	FENG_MRCNT1 phase 12 controlled program memory sector start
5:0	PHM_CNT0PH12	FENG_MRCNT0 phase 12 controlled program memory sector start

6.5.2.13 D_FENG_PHM_PH14_PH15 Settings

Bit Field	Field Name	Description
29:24	PH15_END	FENG_MRCNT1 phase 15 program memory sector end
23:18	PHM_CNT1PH15	FENG_MRCNT1 phase 15 controlled program memory sector start
17:12	PHM_CNT0PH15	FENG_MRCNT0 phase 15 controlled program memory sector start
11:6	PHM_CNT1PH14	FENG_MRCNT1 phase 14 controlled program memory sector start
5:0	PHM_CNT0PH14	FENG_MRCNT0 phase 14 controlled program memory sector start

6.5.2.14 D_FENG_COEFF_OFFSET_EN_SHADOW Settings

Bit Field	Field Name	Description
31:0	COEFF_OFFSET_EN_SHADOW	The coefficient offset enable shadow register

6.5.2.15 D_FENG_COEFF_OFFSET_EN_ACTIVE Settings

Bit Field	Field Name	Description
31:0	COEFF_OFFSET_EN_ACTIVE	The coefficient offset enable active register

6.5.2.16 D_FENG_STATE_W_EN Settings

Bit Field	Field Name	Description
15:0	STATE_W_EN	The conditional processing state registers write enable

6.5.2.17 D_FENG_STATE_ACTIVE Settings

Bit Field	Field Name	Description
15:0	STATE_ACTIVE	The conditional processing state active register

6.5.2.18 D_FENG_STATE_SHADOW Settings

Bit Field	Field Name	Description
15:0	STATE_SHADOW	The conditional processing state shadow register

6.5.2.19 D_FENG_CMDS Settings

Bit Field	Field Name	Description
2	EXEC	Filter Engine program execution start
1	DECODER_RESET	Filter Engine decoder reset
0	AUDIO_RESET	Filter Engine audio reset

Field Name	Value Symbol	Value Description	Hex Value
EXEC	FENG_EXEC	Start a single Filter Engine execution sequence	0x1
DECODER_RESET	FENG_DECODER_RESET	Reset the Filter Engine decoder registers	0x1
AUDIO_RESET	FENG_AUDIO_RESET	Reset the Filter Engine audio registers	0x1

6.5.2.20 D_FENG_PGM_ADDR Settings

Bit Field	Field Name	Description
5:0	DECODER	Program memory decoder address

6.5.2.21 D_FENG_CTRL Settings

Bit Field	Field Name	Description
2	PAUSE	If set, the Filter Engine is paused at the next execution start
1	SINGLE_STEP	If set, the CTRL_ENABLE is set for one clock cycle
0	ENABLE	If cleared, all the clocks are shut down (CFX can still access Filter Engine)

Field Name	Value Symbol	Value Description	Hex Value
PAUSE	FENG_PAUSE_DISABLE	Filter Engine can execute	0x0*
	FENG_PAUSE_ENABLE	Filter Engine is paused at the start of the next program execution cycle	0x1

Field Name	Value Symbol	Value Description	Hex Value
SINGLE_STEP	FENG_STEP_DISABLE	No change in Filter Engine status	0x0*
	FENG_STEP_ENABLE	Filter Engine executes 1 clock cycle of its program (bit is automatically cleared)	0x1
ENABLE	FENG_CTRL_DISABLE	Filter Engine is disabled; CFX accesses provide a clock edge to the region accessed	0x0*
	FENG_CTRL_ENABLE	Filter Engine is enabled	0x1

6.5.2.22 D_FENG_RF_EXTENDED_* Settings

D_FENG_RF_EXTENDED_0 to D_FENG_RF_EXTENDED_15 all have the same bit fields.

Bit Field	Field Name	Description
26:0	RF_EXTENDED	Data read in m5p23 format, written in m4p23 format (sign extended to 32 bits)

6.5.2.23 D_FENG_RF_SINGLE_* Settings

D_FENG_RF_SINGLE_0 to D_FENG_RF_SINGLE_27 all have the same bit fields.

Bit Field	Field Name	Description
23:0	RF_SINGLE	Data in frac24 format (sign extended to 32 bits)

6.5.2.24 D_FENG_RF_IN_* Settings

D_FENG_RF_IN_0 to D_FENG_RF_IN_11 all have the same bit fields.

Bit Field	Field Name	Description
23:0	RF_IN	Data in frac24 format (sign extended to 32 bits)

6.5.2.25 D_FENG_RF_OUT_* Settings

D_FENG_RF_OUT_0 to D_FENG_RF_OUT_7 all have the same bit fields.

Bit Field	Field Name	Description
23:0	RF_OUT	Data in frac24 format (sign extended to 32 bits)

CHAPTER 7
Memory

7.1 Introduction

The memory systems provided by the Ezairo 7100 system are constructed using a number of memories (memory instances), memory buses and controllers. These memories and other memory-mapped elements are addressable from the CFX DSP and the ARM Cortex-M3 processor through a set of memory spaces (also known as address spaces).

This chapter describes:

- The objects that form the memory architecture in Section 7.2, “Memory Architecture”
- The CFX Program and P memory spaces in Section 7.3, “Program and P Memory” on page 80
- The CFX X and Y (data) memory spaces in Section 7.4, “Data Memory” on page 86
- The ARM Cortex-M3 processor unified memory space in Section 7.5, “ARM Cortex-M3 Processor Memory Space” on page 95

7.2 Memory Architecture

The Ezairo 7100 memory architecture consists of a number of single-port memories interconnected with the CFX DSP, HEAR Accelerator, ARM Cortex-M3 processor, and the IOC through a series of buses and data controllers. Memory is divided into memory instances attached to the:

- CFX DSP
- HEAR Configurable Accelerator
- ARM Cortex-M3 processor

All memory instances connected to the HEAR or connected to the ARM Cortex-M3 processor are also accessible by the CFX DSP through either the P memory bus or X memory bus. An overview of the layout of the memories and buses is illustrated in Figure 3.

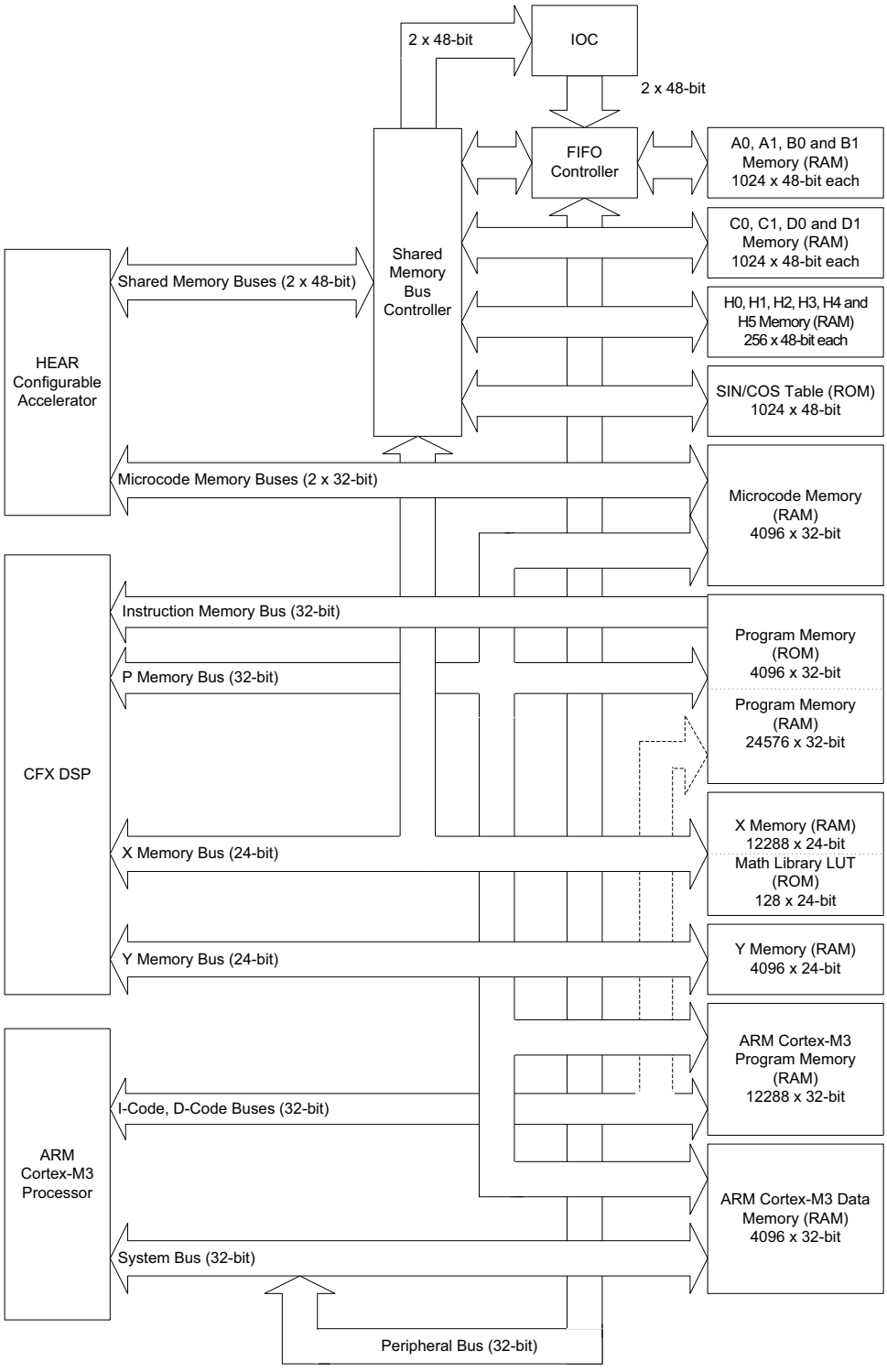


Figure 3. System Memory Architecture

7.2.1 Memory Instances

Table 4 lists the memory structures attached to the CFX, the size and width of each memory structure, and the bus or buses that the CFX DSP uses to access the memory.

Table 4. CFX Memory Structures

Memory Structure	Data Width	Memory Length	CFX Access Using
Program memory (ROM)	32	4096	P, Instruction memory bus
Program memory (RAM)	32	24576 (16384[1])	P, Instruction memory bus
X memory (RAM)	24	12288	X, XY memory buses
Math Library look up table (ROM)	24	128	X memory bus
Y memory (RAM)	24	4096	Y, XY memory buses

1. One section of CFX program memory can be re-assigned to the ARM Cortex-M3 processor as described in Section 7.2.1.1, “CFX Memory Block Instances”

Table 5 lists the memory structures attached to the HEAR, the size and width of each memory structure, and the bus that the CFX DSP uses to access the memory.

Table 5. HEAR Memory Structures

Memory Structure	Data Width	Memory Length	CFX Access Using
Microcode memory (RAM)	32	4096	P memory bus
H0 memory (RAM)	48	256	X memory bus
H1 memory (RAM)	48	256	X memory bus
H2 memory (RAM)	48	256	X memory bus
H3 memory (RAM)	48	256	X memory bus
H4 memory (RAM)	48	256	X memory bus
H5 memory (RAM)	48	256	X memory bus
A0 memory (RAM)	48	1024	X memory bus
A1 memory (RAM)	48	1024	X memory bus
B0 memory (RAM)	48	1024	X memory bus
B1 memory (RAM)	48	1024	X memory bus
C0 memory (RAM)	48	1024	X memory bus
C1 memory (RAM)	48	1024	X memory bus
D0 memory (RAM)	48	1024	X memory bus
D1 memory (RAM)	48	1024	X memory bus
SIN/COS Table (ROM)	48	1024	X memory bus

Table 6 lists the memory structures attached to the ARM Cortex-M3 processor, the size and width of each memory structure, and the bus that the CFX DSP uses to access the memory.

Table 6. ARM Cortex-M3 Core Memory Structures

Memory Structure	Data Width	Memory Length	CFX Access Using
Program memory (RAM)	32	12288 (20480[1])	P memory bus
Data memory (RAM)	32	4096	P memory bus

1. One section of CFX program memory can be re-assigned to the ARM Cortex-M3 processor as described in Section 7.2.1.1, “CFX Memory Block Instances”

Since memory conflicts only occur when accesses are made to the same physical memory instance, some memory structures are broken down into blocks. This breakdown into memory instance blocks is described in the following subsections.

7.2.1.1 CFX Memory Block Instances

The program memory (RAM) structure that is attached to the CFX is broken down further into memory blocks, as shown in Table 7.

Table 7. Program Memory Blocks

Block	Start Address	End Address
P memory 0[1]	0x8000	0x9FFF
P memory 1	0xA000	0xBFFF
P memory 2	0xC000	0xDFFF

1. This memory instance can be reassigned to the ARM Cortex-M3 processor using the `SYSTEM_CTRL_2_SYSTEM2_PRAM0_*` bits in the `D_SYSTEM_CTRL_2` register.

P memory 0 (PRAM0) is a special case RAM structure that can be reassigned to the ARM Cortex-M3 processor. To chose which processor controls this memory set the `SYSTEM_CTRL_2_PRAM0_CFX` or the `SYSTEM_CTRL_2_PRAM0_CM3` bit in the `D_SYSTEM_CTRL_2` register. The `SYSTEM_CTRL_2_PRAM0_STATUS` bit from the `D_SYSTEM_CTRL_2` register is used to indicate which core this memory block is currently assigned to. For more information, see Section 5.4.1, “D_SYSTEM_CTRL_2 Settings” on page 48.

NOTE: If both `SYSTEM_CTRL_2_PRAM0_CFX` and `SYSTEM_CTRL_2_PRAM0_CM3` bits are set when writing the `D_SYSTEM_CTRL_2` register, the `SYSTEM_CTRL_2_PRAM0_CM3` bit is ignored.

The X memory RAM structure can be broken down further into memory blocks, as shown in Table 8.

Table 8. X Memory Blocks

Block	Start Address	End Address
X memory 0	0x0000	0x0FFF
X memory 1	0x1000	0x1FFF
X memory 2	0x2000	0x2FFF

7.2.1.2 HEAR Memory Block Instances

The microcode memory RAM structure that is attached to the HEAR can be broken down further into memory blocks, as shown in Table 9.

Table 9. Microcode Memory Blocks

Block	Start Address	End Address
Microcode memory 0	0xE000	0xE7FF
Microcode memory 1	0xE800	0xEFFF

7.2.1.3 ARM Cortex-M3 Core Memory Block Instances

The memory structures that are attached to the ARM Cortex-M3 processor cannot be broken down further into memory blocks.

7.2.2 Memory Bus Structures

Four memory buses connect the various memory instances to the CFX DSP. The primary connections and uses for these buses, which can be seen in Figure 3 on page 73, are as follows:

Instruction Bus Used by the CFX DSP to fetch program instructions for execution. Additionally, this bus gives the CFX processor access to the interrupt vector table.

X Memory Bus Used by the CFX DSP to access data in X memory, XY memory and shared memory. This bus also provides the CFX with access to the memory-mapped control and configuration registers for the HEAR and for the FIFOs. This bus provides the least significant word of any accesses to the compound XY memory bus.

Y Memory Bus Used by the CFX DSP to access data in Y memory and XY memory. This bus provides the most significant word of any accesses to the compound XY memory bus.

P Memory Bus Used by the CFX DSP to access program and microcode memory as data. This bus also provides the CFX with access to the memory-mapped control and configuration registers used to configure analog and digital system components.

An additional four buses attached to the HEAR Accelerator provide access to microcode memory and shared memory for the HEAR. These buses, which can also be seen in Figure 3 on page 73, are set up in matched pairs to increase the data throughput of the HEAR:

Microcode Memory Buses
Two buses that allow the HEAR to fetch configuration information from the microcode memory instance.

Shared Memory Buses
Two buses that allow the HEAR to access all of shared memory through the shared memory bus controller.

Buses connected to the ARM Cortex-M3 processor implement the standard ARM Cortex-M3 core memory map. These buses, which can also be seen in Figure 3 on page 73, are as follows:

I-Code Bus

Allows the ARM Cortex-M3 processor to fetch instruction information from the ARM Cortex-M3 core program memory instance or PRAM0 memory instance.

D-Code Bus

Allows the ARM Cortex-M3 processor to fetch data information from the ARM Cortex-M3 core program memory instance or PRAM0 memory instance.

System Bus

Allows the ARM Cortex-M3 processor to fetch instructions and data information from the ARM Cortex-M3 core data memory instance. This bus also provides access to the peripheral bus.

Peripheral Bus

Allows the ARM Cortex-M3 processor to access memory-mapped peripherals and external memory instances (including, in the Ezairo 7100 implementation, the FIFO controller).

Private Peripheral Bus

Allows the ARM Cortex-M3 processor to access standard memory-mapped peripherals. This includes the NVIC, SysTick timer and the ARM Cortex-M3 processor debug port. This bus and memory-mapped peripherals mapped onto this bus are accessible by only the ARM Cortex-M3 processor.

7.2.3 Memory Controllers and Arbitration

As shown in Figure 3, the memory architecture contains a pair of controllers that operate between memory buses attached to the cores, and part or all of shared memory.

The shared memory bus controller coordinates access to the shared memories, separately directing data between the shared memory instances and:

- The X and Y memory buses that are attached to the CFX
- The shared memory buses attached to the HEAR

For more information about this controller, see Section 7.2.3.1, “Shared Memory Bus Controller”.

The FIFO controller is connected to the IOC, the shared memory bus controller, and the ARM Cortex-M3 core peripheral bus. This controller is used to provide FIFO access/addressing into A0, A1, B0 and B1 memories while coordinating and controlling memory access requests to these memories. More information about this controller is available in Chapter 8, “FIFO Controller” on page 98.

Outside of these two controllers, the Ezairo 7100 system has several bus arbitration components. These include the arbiters controlling access to the following memories:

- Microcode memory access from the CFX DSP versus the HEAR Configurable Accelerator. This arbiter is described further in Section 7.3.2, “P Memory Space”.
- ARM Cortex-M3 processor program and data memory access from the CFX DSP versus the ARM Cortex-M3 processor. This arbiter is described further in Section 7.3.2, “P Memory Space”.
- ARM Cortex-M3 processor data memory access from either the CFX DSP or the ARM Cortex-M3 processor versus the DMA controller. This arbiter is described further in Section 15.5.6, “Channel Priority” on page 338.

External interrupts on the CFX debug port can be used in memory conflict mode to detect when any memory conflicts between any two of the CFX DSP, HEAR Configurable Accelerator, or the ARM Cortex-M3 processor occur. For more information, see Section 13.2.3.3, “Memory Conflict Mode” on page 194.

Memory conflicts between the CFX DSP and the Filter Engine follow a strict priority scheme with no arbitration. In general, conflicting accesses between the CFX DSP and the Filter Engine result in a failure of the CFX’s memory access. For more information about the CFX DSP and the Filter Engine memory accesses conflicts see in Section 6.4.2, “Memory and Register Access Conflicts” on page 55.

7.2.3.1 Shared Memory Bus Controller

The shared memory bus controller coordinates memory accesses and addressing into the underlying HEAR shared memories. For more information about the shared memory itself, see Section 7.4.2, “Shared Memory (RAM and ROM)” on page 87.

If a memory conflict is detected by this controller, the `SYSTEM_CTRL_PRIORITY_HEAR` and `SYSTEM_CTRL_PRIORITY_CFX` bits of the `D_SYSTEM_CTRL` register control the memory access (see Section 4.1.3.1, “D_SYSTEM_CTRL Settings” on page 35). As only one of these bits can be set at any time, the core with priority retains access to the conflicting memory structure until the conflict is removed or resolved, while the other core is held in a wait state. Reading the value of the `SYSTEM_CTRL_PRIORITY_CFX` bit provides the current configuration status and whether the HEAR or CFX has priority.

NOTE: Although A and B memory are also accessible to the IOC, resolution of memory access conflicts with the IOC are handled separately by the FIFO controller, as detailed in Section 8.5, “Access Priority” on page 110.

IMPORTANT: You can change the access priority at any time, provided the CFX core does not execute a read or write to the shared memory in the clock cycle immediately preceding the change of priority. (It is not possible to access shared memory in the same cycle in which the access priority is updated.)

7.2.3.2 Control and Configuration Registers

For more information about memory architecture control and configuration registers, see Section 4.1.3.1, “D_SYSTEM_CTRL Settings” on page 35.

7.2.4 Memory Interface Write Buffers

The CFX DSP core contains a write buffer for each of the P memory, X memory and Y memory buses. The buffers allow the CFX to perform any combination of memory-read and memory-write operations without stalling the instruction pipeline. A buffered memory-write operation generally does not affect program behavior and is transparent to the user application, but the CFX’s write buffering is visible in some situations, which are explained further in the following subsections.

7.2.4.1 Buffering and Flushing of Write Operations

A memory-write operation is buffered by the CFX when a read operation from the same memory space as the write occurs in the clock cycle following the write. This is necessary when a write is performed in the **Execute** stage of the instruction pipeline while a read is performed in the **Decode** stage (see the *CFX DSP Architecture Manual* for details about the Program Control Unit’s execution pipeline). Thus, when a write is immediately followed by a read from the same memory, both operations need the bus at the same time. The read operation takes priority and the write operation is buffered.

A buffered write is flushed (stored to memory) as soon as the bus has a cycle during which it is not busy, which occurs for any instruction that is not a read from the same memory space. This includes:

- Another write operation (during its **Decode** stage the memory bus is free)
- Memory operations to a different memory space
- Control-flow instructions (including interrupt handling)
- Register-based operations

In summary, when a write operation is followed by a consecutive sequence of read operations from the same memory, the write is only stored to memory after all of the read operations have occurred. If any of the read operations read from the same address to which the buffered write will store its value, the read operation returns the write operation's buffered value.

7.2.4.2 Visible Effects of Write Buffering

Write buffering can be visible to a user application for two reasons:

1. The delay in actually storing a value to memory.
2. Reading a buffered value instead of memory. This is visible where one or more memory bits are not instantiated, or they use different read and write definitions.

Several control and configuration registers can behave unexpectedly for the second reason if you write to the register's P memory location, and then immediately read it back so that you read the buffered value. Examples of special register behaviors that are not implemented by a buffered value include:

- Some bits might not be defined (for example, `D_WATCHDOG_CFG`).
- Writing a one to a bit clears the bit that was set, or otherwise changes the register values or initiates an action (for example, `D_SYSTEM_CTRL`).
- The bits in a register have different meanings when read than when written (for example, `D_I2C_CTRL`).
- Reading a register changes its value or initiates an action (for example, `D_SPI_DATA` when the SPI interface is configured for auto mode).

Visible write buffering due to a delayed memory store is less common. It is usually combined with writing a control register that automatically modifies another memory location (typically another control register). Since the store is delayed, the side-effect modification has not yet occurred if the other memory location is read immediately after the write. For example:

- Blocks such as the CRC Generator, where a read of `D_CRC_VALUE` would not include the value written to any `D_CRC_ADD_x` registers in the previous cycle
- Reading back the interrupt enable or priority bits in `D_INT_ENABLE` or `D_INT_PRIORITY` immediately after writing them through `D_INT_EBL_x` or `D_INT_PRIO_x`

Although all of the preceding examples involve control registers in P memory, similar write buffering affects registers mapped into X memory. Examples include the FIFO registers, HEAR function chain controller registers, and HEAR block floating point registers. The value written to a register such as `D_FIFO_xx_BLOCK_CNT` might not match

the value returned by that register, and the value of other registers in the same block could be updated by the value written to the D_FIFO_xx_BLOCK_CNT register.

IMPORTANT: In all of the above examples, you can avoid the undesirable side effects of write buffering by inserting (after the write operation) any instruction that does not read the same memory space. If you cannot move an existing instruction into place after the write, a NOP instruction will suffice.

Since the Y memory space has no control registers with special behavior, use of the write buffer for the Y memory bus is always transparent to the user application.

7.3 Program and P Memory

This section provides detailed information about the components (e.g., program memory) that are accessible through the dedicated instruction bus and the P memory bus via the program and P memory spaces.

7.3.1 Program Memory Space

The program memory space allows the CFX to access the RAM and ROM instances of CFX program memory through the instruction bus. This memory space contains only the memories listed in Section 7.3.3, “Program Memory (RAM and ROM)”.

7.3.2 P Memory Space

The P memory space (P memory) allows the CFX to access a proper superset of the program memory space for use as data. The P memory bus provides access to both the program memory instances for the CFX and the microcode memory instances for the HEAR. It also provides access to the memory-mapped control and configuration registers for the majority of analog and digital components, and access to interrupt vectors for use by the interrupt controller and CFX.

When the P memory bus is used to read or write, a bus conflict occurs if the instruction bus or microcode buses are currently using the memory block to be accessed. When a bus conflict occurs, the CFX DSP is held in a wait state, which incurs a delay in the completion of the instruction accessing P memory. The wait state holds the CFX’s clock, so the delay is not visible to the CFX in terms of instruction cycle counts. Thus a P memory access is valid in a single delay slot, for example, regardless of whether the memory access might cause a bus conflict and thus be delayed.

Note that a bus conflict does *not* occur, and P memory access proceeds without introducing a delay, if any of the following conditions hold:

- The CFX instruction and data are located in different memory blocks (see Section 7.2.1, “Memory Instances” on page 74 for a list of memory blocks).
- The instruction for the CFX is loaded from the program cache (see Section 7.3.8, “Program Memory Cache” on page 82).
- The CFX is currently in the first execution cycle of a multi-cycle instruction, including *Move-Move* instructions, so that it is not pre-fetching the next instruction.

Similar bus conflicts can also occur between the CFX DSP core and HEAR Accelerator core when both are accessing the microcode memory. Handling this possible conflict uses the SYSTEM_CTRL_PRIORITY_HEAR and SYSTEM_CTRL_PRIORITY_CFX bits of the D_SYSTEM_CTRL register. The SYSTEM_CTRL_PRIORITY_STATUS bit field in the D_SYSTEM_CTRL register indicates which core has priority. As only one of these bits can be set at any time, the processor with priority retains access to the conflicting memory structure until the conflict is removed or resolved, while

the other core is held in a wait state. For configuration information about these bits, see Section 4.1.3.1, “D_SYSTEM_CTRL Settings” on page 35 on control and configuration registers.

Furthermore, bus conflicts can also occur between the CFX DSP core and the ARM Cortex-M3 core when both are accessing an ARM Cortex-M3 processor program or data memory instance. Handling this possible conflict uses the SYSTEM_CTRL_2_PRIORITY_CM3 and SYSTEM_CTRL_2_PRIORITY_CFX bits of the D_SYSTEM_CTRL_2 register. The SYSTEM_CTRL_2_PRIORITY_STATUS bit field in D_SYSTEM_CTRL_2 register indicates which core has priority. As only one of these bits can be set at any time, the processor with priority retains access to the conflicting memory structure until the conflict is removed or resolved, while the other core is held in a wait state. For configuration information about these bits, see Section 5.4.1, “D_SYSTEM_CTRL_2 Settings” on page 48 on control and configuration registers.

7.3.3 Program Memory (RAM and ROM)

Instructions to execute on the CFX are fetched from the program memory through the dedicated instruction bus. Access on this bus is restricted to this use; therefore all data access to the program memory areas occurs over the P memory bus.

The program ROM is located from 0x0000 to 0x2FFF of the addressable program memory space. This ROM contains code for booting the chip as well as system-level routines for performing common tasks. For detailed information about the contents of the program ROM, see the *Firmware Reference Manual for Ezairo 7100*.

The program RAM segments are located from 0x8000 to 0xDFFF of the addressable program memory space. This memory is used for user applications and for any 32-bit application data that a user application needs. The block of memory from 0xDC00 to 0xDFFF is also mirrored at the top of the address space located at 0xFC00 to 0xFFFF.

7.3.4 Microcode Memory (RAM)

The microcode memory is located in the addressable P memory space from 0xE000 to 0xEFFF. This memory is loaded by the CFX with a microcode configuration for the HEAR. Each function chain’s microcode is loaded by the HEAR Configurable Accelerator using the microcode memory bus when that function chain is triggered.

7.3.5 ARM Cortex-M3 Processor Program Memory (RAM)

ARM Cortex-M3 processor program memory is mapped into the CFX P memory space from 0x5000 to 0x7FFF. This memory can optionally be extended to include the PRAM0 instance located between 0x8000 and 0x9FFF.

These memory areas map to addresses 0x00000000 to 0x0000BFFF and 0x0000C000 to 0x0013FFF respectively on the ARM Cortex-M3 core’s instruction and data buses.

7.3.6 ARM Cortex-M3 Processor Data Memory (RAM)

ARM Cortex-M3 processor data memory is mapped into the CFX P memory space from 0x4000 to 0x4FFF.

This memory area maps to addresses 0x20000000 to 0x20003FFF on the ARM Cortex-M3 core’s system bus.

7.3.7 Filter Engine Memory (RAM)

The Filter Engine memory map is provided in Table 10. It is partitioned into two sections:

- 0xF300-0xF37F: This partition holds the configuration and data registers (IOREG).
- 0xF800-0xF9FF: This partition holds all the program, coefficient and state variable memories (IOMEM).

Table 10. Filter Engine Memory Map

DSS Address Range	Description	Memory Area	Reset Value	Size (words)	Data
0xF300-0xF312	Configuration registers	IOREG	See Appendix A, “Control and Configuration Registers” on page 482	19	N/A
0xF340-0xF34F	Extended-precision data registers	IOREG	0	16	Read: sign extension [31:28] data [27:0] Write: sign extension [31:27] data [26:0]
0xF350-0xF37F	Single-precision data registers including input and output data registers	IOREG	0	48	Read/Write: sign extension [31:24] data [23:0]
0xF800+6***x** – (0xF805+6*(**x**+1))-1	Filter stage-**x** state variables **x**= 0,1,2,...31 S**x**.**y** = Stage **x** State **y** States at stage **x**: S**x**.1, S**x**.2, S**x**.3, S**x**.4, S**x**.5, S**x**.5tmp	IOMEM	N/A	192	Read/Write: sign extension [31:24] data [23:0]
0xF900+5***x** – (0xF904+5*(**x**+1))-1	Filter stage-**x** coefficients **x**= 0,1,2,...31 C**x**.**y** = Stage **x** Coefficient **y** Coefficients at stage **x**: C**x**.0, C**x**.1, C**x**.2, C**x**.3, C**x**.4	IOMEM	N/A	160	Read/Write: sign extension [31:24] data [23:0]
0xF9C0-0xF9FF	Program Memory	IOMEM	N/A	64	Read/Write: data [31:0]

7.3.8 Program Memory Cache

To increase the power-efficiency of the CFX DSP, a 16-word read-only instruction cache has been included on the instruction bus between the CFX DSP and the program memory. This cache is designed to reduce the power consumed by constantly accessing program memory to fetch subsequent instructions for the CFX DSP. There is no detectable difference in functional code execution when using the cache, except for a slight change in timing due to the special case discussed in the note below.

The program memory cache automatically reconfigures whenever a backwards branch of less than 256 words indicates that a loop has been detected. At this point, the cache aligns itself to ensure that the largest possible number of instructions from the loop are stored in the cache.

NOTE: Access to P memory while the CFX is fetching cached instructions from a loop does not result in a memory access conflict. As a result, when the program memory cache is being used for a loop that uses program memory as data, the core does not incur delays as described in Section 7.3.2, “P Memory Space” on page 80.

7.3.9 P Memory Space Memory Map

The specific organization of the P memory space is shown in Figure 4. For reference by a program, many of the start and end addresses for memory areas in P memory are associated with `#define` statements that can be found in the *sk4_map.cfi* file.

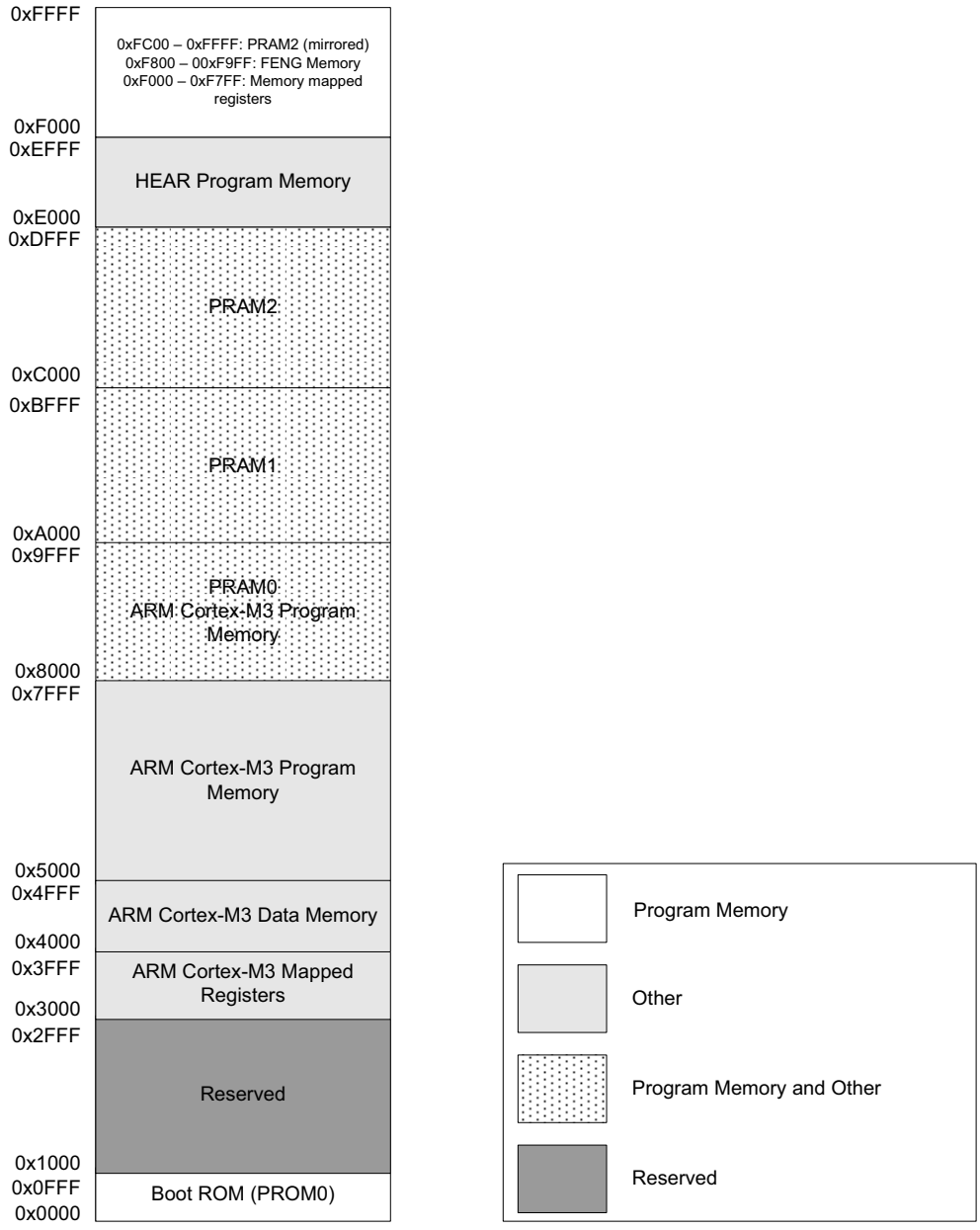


Figure 4. P Memory Space Memory Map

7.3.9.1 Interrupt Vector Mapping

The interrupt vector table, which is shared by the interrupt controller and the CFX DSP, is mapped into the program memory between addresses 0xFFE0 and 0xFFFF (mirrored from between addresses 0xDFE0 and 0xDFFF). The specific interrupt vectors and their locations in P memory are shown in Table 11.

Table 11. Interrupt Vector Map

Interrupt	Vector	Address
System Reset	D_VECT_SYS_RESET	0x0000
Watchdog	D_VECT_WATCHDOG	0xFFE1
Timer 1	D_VECT_TIMER_1	0xFFE2
Timer 2	D_VECT_TIMER_2	0xFFE3
SPI	D_VECT_SPI	0xFFE4
I^2C	D_VECT_I2C	0xFFE5
Shared PCM and I^2S	D_VECT_PCM	0xFFE6
GPIO	D_VECT_GPIO	0xFFE7
HEAR Event 0	D_VECT_HEAR_0	0xFFE8
HEAR Event 1	D_VECT_HEAR_1	0xFFE9
HEAR Event 2	D_VECT_HEAR_2	0xFFEA
HEAR Event 3	D_VECT_HEAR_3	0xFFEB
HEAR Event 4	D_VECT_HEAR_4	0xFFEC
HEAR Event 5 or UART	D_VECT_HEAR_5 D_VECT_UART	0xFFED
HEAR Event 6 or Filter Engine	D_VECT_HEAR_6 D_VECT_FENG	0xFFEE
HEAR Event 7 or Signal Detection Unit	D_VECT_HEAR_7 D_VECT_SDU	0xFFEF
FIFO 0	D_VECT_FIFO_0	0xFFF0
FIFO 1	D_VECT_FIFO_1	0xFFF1
FIFO 2	D_VECT_FIFO_2	0xFFF2
FIFO 3	D_VECT_FIFO_3	0xFFF3
FIFO 4	D_VECT_FIFO_4	0xFFF4
FIFO 5	D_VECT_FIFO_5	0xFFF5
FIFO 6	D_VECT_FIFO_6	0xFFF6
FIFO 7	D_VECT_FIFO_7	0xFFF7
Clock	D_VECT_CLK	0xFFF8
ARM Cortex-M3 Core SPI	D_VECT_C_SPI	0xFFF9
ARM Cortex-M3 Core PCM	D_VECT_C_PCM	0xFFFA
ARM Cortex-M3 Core I^2C	D_VECT_C_I2C	0xFFFB
ARM Cortex-M3 Core Event 0	D_VECT_CM3_0	0xFFFC

Table 11. Interrupt Vector Map (Continued)

Interrupt	Vector	Address
ARM Cortex-M3 Core Event 1	D_VECT_CM3_1	0xFFFD
ARM Cortex-M3 Core Event 2	D_VECT_CM3_2	0xFFFE
ARM Cortex-M3 Core Event 3	D_VECT_CM3_3	0xFFFF

7.3.9.2 Configuration Register Mapping

The analog and digital control and configuration registers are mapped into the P memory space between addresses 0xF000 and 0xF3FF. These registers are organized by their area of control and mapped as blocks into the P memory space as shown in Table 12.

Table 12. Analog and Digital Control and Configuration Register Block Mapping

Controlled/Configured Area	Area Type	Block Start Address
General System Configuration	Digital	0xF000
Debug Port Related Configuration (External Breakpoints)	Digital	0xF020
CRC	Digital	0xF022
Debug Port Related Configuration	Digital	0xF030
Interrupt Controller	Digital	0xF040
Clock Configuration	Digital	0xF080
I/O Controller (IOC)	Digital	0xF0C0
Digital A/D and D/A Converter Data	Digital	0xF0D0
Clock Generation (Oscillator)	Analog	0xF100
Power Components	Analog	0xF110
Input Stage	Analog	0xF140
Output Stage	Analog	0xF150
Audio Multiplexing	Analog	0xF156
Output Stage (DMIC)	Digital	0xF15B
Low Speed A/D Converters (LSAD)	Analog	0xF180
Power Components (Battery Monitor)	Analog	0xF190
SPI Interface	Digital	0xE1C0
I^2C Interfaces	Digital	0xE200
Shared PCM/I^2S Interface	Digital	0xE240
DIO and GPIO	Digital	0xE280
UART Interface	Digital	0xE2C0
Filter Engine	Digital	0xE300
Signal Detection Unit (SDU)	Analog	0xE380

Digital control registers allowing the CFX DSP to access components in the ARM Cortex-M3 processor's subsystem are also mapped into the P memory space. These registers are organized by their area of control and mapped as blocks into the P memory space as shown in Table 13.

Table 13. ARM Cortex-M3 Processor Configuration Register Block Mapping into the CFX Memory Space

Controlled/Configured Area	Block Start Address
Error Correction Block (ECB)	0x3000
Audio Codec Block (ACB)	0x3040
DMA Controller	0x3080
External Interrupts	0x30C0
DIO and GPIO	0x3140
SPI	0x31C0
PCM	0x3200
I^2C	0x3240
CRC	0x3280

7.4 DATA MEMORY

Data memory is used to store all types of signal and control data. The two kinds of memory accessible as data memory are memory attached to the CFX DSP and memory attached to the HEAR. The CFX has direct access to a number of data memories that a user application can use as needed. All data memory attached to the HEAR Accelerator core is also available to the CFX as shared memory, although both cores access the memory through the shared memory bus controller. This section provides detailed information about these memory components; all of the related components that are accessible through the X, Y and shared memory buses; and memory components and related components that are located in the X, Y and XY memory maps.

7.4.1 CFX Data Memories (RAM and ROM)

Data memory attached to the CFX DSP primarily consists of two banks of 24-bit memory that can be accessed either individually as 24-bit memory or in combination as a single bank of 48-bit memory. These two individual memories are referred to as X memory and Y memory respectively. X memory is located from 0x0000 to 0x2FFF in the X memory space and is accessed through the X memory bus. Y memory is located from 0x0000 to 0x0FFF in the Y memory space and is accessed through the Y memory bus.

XY memory is a combined memory space that is addressable from 0x0000 to 0x0FFF. Each addressable element is a double-word 48-bit value using the data at the corresponding X memory address as the low word, and at the corresponding Y memory address as the high word. XY memory is accessed through both the X and Y memory buses together as the XY memory bus.

NOTE: As XY memory uses physical memory containing both the X memory instance and Y memory instance located between 0x0000 and 0x0FFF, we generally recommend that any space reserved for XY memory also be reserved in both X memory and Y memory.

X memory also contains a set of four look up tables (LUT) used by the math library firmware. These LUTs are collected into one 128-word math LUT ROM that is used in math library calculations that make use of the functions: 2^x, $\log_2 x$, $\frac{1}{x}$, and $\sqrt{x}$.

7.4.2 Shared Memory (RAM and ROM)

All of the shared memories are double-word structured 48-bit memories that allow 24-bit word addressing by both the CFX and HEAR. These memories and their locations in the X memory map are outlined in Table 14. All accesses to these memories are controlled by the shared memory bus controller. The controller provides access for:

- The CFX DSP through the X and Y memory spaces
- The HEAR through the core's 48-bit shared memory buses

Table 14. Memory Instances in Shared Memory

Memory Name	Start Address	End Address
H0 memory	0x4C00	0x4DFF
H1 memory	0x4E00	0x4FFF
H2 memory	0x5000	0x51FF
H3 memory	0x5200	0x53FF
H4 memory	0x5400	0x55FF
H5 memory	0x5600	0x57FF
SIN/COS Data Table (ROM)	0x6800	0x6FFF
A0 memory	0xC000	0xC7FF
A1 memory	0xC800	0xCFFF
B0 memory	0xD000	0xD7FF
B1 memory	0xD800	0xDFFF
C0 memory	0xE000	0xE7FF
C1 memory	0xE800	0xEFFF
D0 memory	0xF000	0xF7FF
D1 memory	0xF800	0xFFFF

The shared memory instances are both accessible independently, as listed in Table 14, and interleaved with other shared memory instances. All interleaved combinations of shared memory instances are listed in Table 15. Both independent instances and interleaved memory instances are shown as appropriate in the X or Y memory map (Figure 5 on page 89).

Table 15. Interleaved Memory Instances in Shared Memory

Memory	Memory Space	First Memory	Second Memory	Start Address	End Address
H01 memory	X	H0 memory	H1 memory	0x4000	0x43FF
H23 memory	X	H2 memory	H3 memory	0x4400	0x47FF
H45 memory	X	H4 memory	H5 memory	0x4800	0x4BFF
H02 memory	X	H0 memory	H2 memory	0x5800	0x5BFF
H13 memory	X	H1 memory	H3 memory	0x5C00	0x5FFF
H03 memory	X	H0 memory	H3 memory	0x6000	0x63FF
H12 memory	X	H1 memory	H2 memory	0x6400	0x67FF
AB0 memory	X	A0 memory	B0 memory	0x8000	0x8FFF
AB1 memory	X	A1 memory	B1 memory	0x9000	0x9FFF
CD0 memory	X	C0 memory	D0 memory	0xA000	0xAFFF

Table 15. Interleaved Memory Instances in Shared Memory

Memory	Memory Space	First Memory	Second Memory	Start Address	End Address
CD1 memory	X	C1 memory	D1 memory	0xB000	0xBFFF
AC0 memory	Y	A0 memory	C0 memory	0xC000	0xCFFF
AC1 memory	Y	A1 memory	A0 memory	0xD000	0xDFFF
BD0 memory	Y	B0 memory	D0 memory	0xE000	0xEFFF
BD1 memory	Y	B1 memory	D1 memory	0xF000	0xFFFF

The interleaving of memory is in 48-bit double-words, resulting in two addressable locations into each memory at each step. For example, the memory addressed between 0x8000 and 0x8FFF is an interleaved memory mapping that provides access to data in both A0 and B0 memory. Reading from 0x8000-0x8001 (the first double-word of the interleaved A0B0 Memory) provides the first double-word of A0 memory. Reading from 0x8002-0x8003 (the second double-word of the interleaved A0B0 Memory) provides the first double-word of B0 memory.

When using shared memory with both the CFX and HEAR cores, bus conflicts occur if both cores attempt to access the same memory instance in the same cycle. If a bus conflict occurs, access is granted to the CFX or HEAR based on the settings of the `D_SYSTEM_CTRL` register, as described in Section 7.2.3.1, “Shared Memory Bus Controller” on page 78, while the other core is held in a wait state. When the CFX DSP is held in a wait state, the delay is not visible to instructions running on the DSP, as explained in Section 7.3.2, “P Memory Space” on page 80.

When using shared FIFO memory with the ARM Cortex-M3 processor, all ARM Cortex-M3 core accesses have the lowest priority. If a bus conflict occurs, the ARM Cortex-M3 processor is held in a wait state until the memory instance is available. When the ARM Cortex-M3 processor is held in a wait state, the delay is not visible to instructions running on the microcontroller.

7.4.3 Data Memory Space Memory Maps

The specific organization of the various data memory spaces is shown in Figure 5. For program references, many of the start and end addresses for memory areas in these memory spaces are associated with `#define` statements that are located in the *sk4_map.cfi* file.

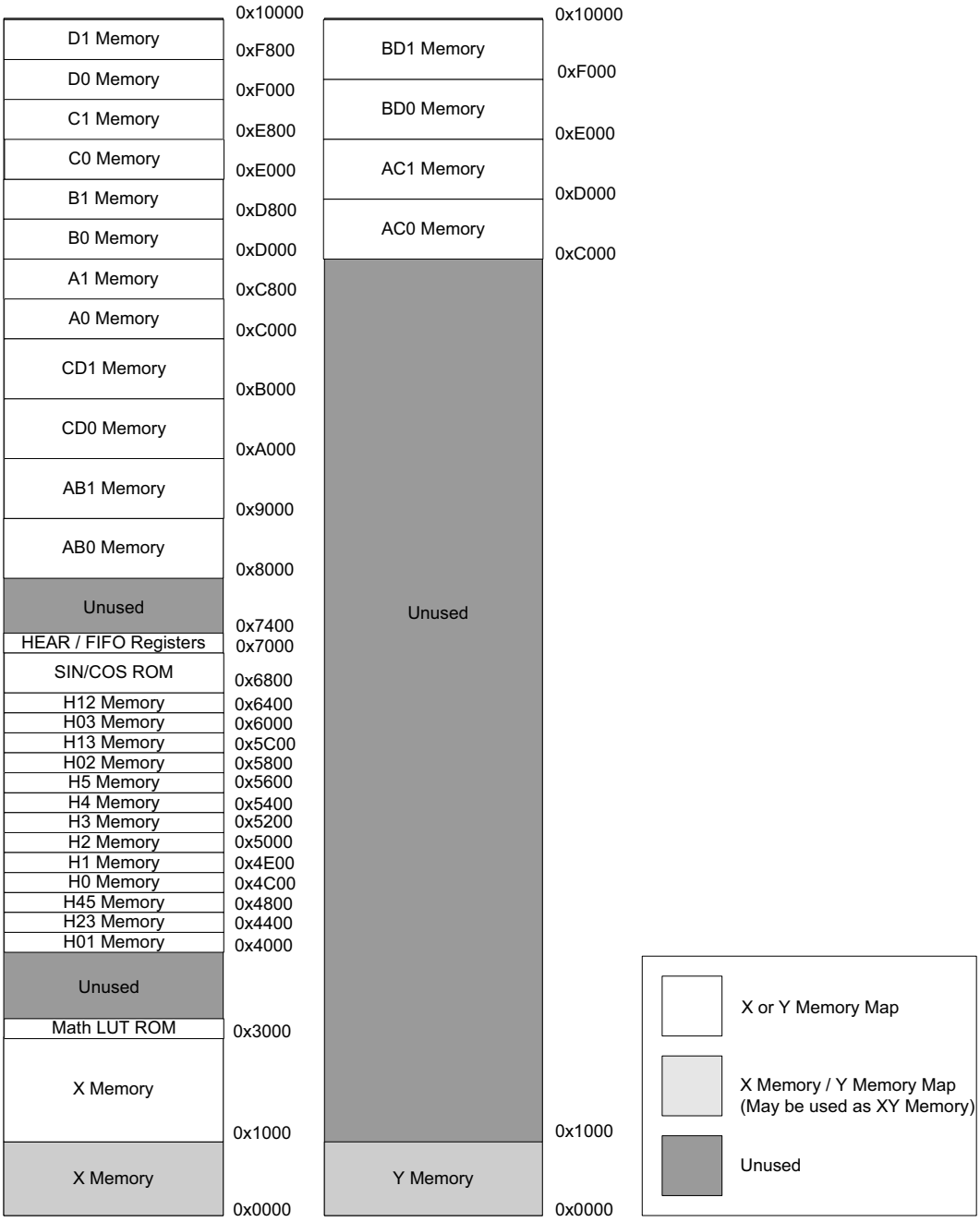


Figure 5. Data Memory Space Memory Maps

7.4.3.1 HEAR and FIFO Configuration Register Mapping

The HEAR and FIFO control and configuration registers are mapped into the X memory space between addresses 0x7000 and 0x72FF. These registers are organized by their area of control and mapped as blocks into the X memory space as shown in Table 16.

Table 16. HEAR and FIFO Control and Configuration Register Block Mapping

Controlled/Configured Area	Block Start Address
FIFO A0_0	0x7000
FIFO A0_1	0x7010
FIFO A0_2	0x7020
FIFO A0_3	0x7030
FIFO A0_4	0x7040
FIFO A0_5	0x7050
FIFO A0_6	0x7060
FIFO A0_7	0x7070
FIFO A1_0	0x7080
FIFO A1_1	0x7090
FIFO A1_2	0x70A0
FIFO A1_3	0x70B0
FIFO A1_4	0x70C0
FIFO A1_5	0x70D0
FIFO A1_6	0x70E0
FIFO A1_7	0x70F0
FIFO B00	0x7100
FIFO B01	0x7110
FIFO B02	0x7120
FIFO B03	0x7130
FIFO B04	0x7140
FIFO B05	0x7150
FIFO B06	0x7160
FIFO B07	0x7170
FIFO B10	0x7180
FIFO B11	0x7190
FIFO B12	0x71A0
FIFO B13	0x71B0
FIFO B14	0x71C0
FIFO B15	0x71D0
FIFO B16	0x71E0
FIFO B17	0x71F0
FIFO Interrupt Configuration	0x7200
Function Chain Controller	0x7230
Block Floating-Point Configuration	0x7260

7.4.4 Control and Configuration Registers

Register Name	Register Description	Address
D_MEM_CFG_0	Memory configuration 0	P:0xF010
D_MEM_CFG_1	Memory configuration 1	P:0xF011
D_MEM_CFG_2	Memory configuration 2	P:0xF012
D_MEM_CFG_3	Memory configuration 3	P:0xF013
D_MEM_CFG_4	Memory configuration 4	P:0xF014
D_MEM_CFG_5	Memory configuration 5	P:0xF015

7.4.4.1 D_MEM_CFG_0 Settings

Bit Field	Field Name	Description
3	ME_FORCE_PROM1	Force the memory enable for PROM1
2	ME_FORCE_PROM0	Force the memory enable for PROM0
1	PROM1_PD	PROM1 power-down control
0	PROM0_PD	PROM0 power-down control

Field Name	Value Symbol	Value Description	Hex Value
ME_FORCE_PROM1	ME_PROM1_AUTO	ME of PROM1 is only enabled when accessed	0x0*
	ME_PROM1_ENABLE	ME of PROM1 is forced on	0x1
ME_FORCE_PROM0	ME_PROM0_AUTO	ME of PROM0 is only enabled when accessed	0x0*
	ME_PROM0_ENABLE	ME of PROM0 is forced on	0x1
PROM1_PD	PROM1_PD_DISABLE	PROM1 powered-up	0x0*
	PROM1_PD_ENABLE	PROM1 powered-down	0x1
PROM0_PD	PROM0_PD_DISABLE	PROM0 powered-up	0x0*
	PROM0_PD_ENABLE	PROM0 powered-down	0x1

7.4.4.2 D_MEM_CFG_1 Settings

Bit Field	Field Name	Description
6	ME_FORCE_CFX_YRAM0	Force ME of CFX YRAM0
5	ME_FORCE_CFX_XRAM2	Force ME of CFX XRAM2
4	ME_FORCE_CFX_XRAM1	Force ME of CFX XRAM1
3	ME_FORCE_CFX_XRAM0	Force ME of CFX XRAM0
2	ME_FORCE_CFX_PRAM2	Force ME of CFX PRAM2
1	ME_FORCE_CFX_PRAM1	Force ME of CFX PRAM1
0	ME_FORCE_CFX_PRAM0	Force ME of CFX PRAM0

Field Name	Value Symbol	Value Description	Hex Value
ME_FORCE_CFX_YRAM0	ME_CFX_YRAM0_AUTO	ME of CFX YRAM0 is only enabled when accessed	0x0*
	ME_CFX_YRAM0_ENABLE	ME of CFX YRAM0 is forced on	0x1
ME_FORCE_CFX_XRAM2	ME_CFX_XRAM2_AUTO	ME of CFX XRAM2 is only enabled when accessed	0x0*
	ME_CFX_XRAM2_ENABLE	ME of CFX XRAM2 is forced on	0x1
ME_FORCE_CFX_XRAM1	ME_CFX_XRAM1_AUTO	ME of CFX XRAM1 is only enabled when accessed	0x0*
	ME_CFX_XRAM1_ENABLE	ME of CFX XRAM1 is forced on	0x1
ME_FORCE_CFX_XRAM0	ME_CFX_XRAM0_AUTO	ME of CFX XRAM0 is only enabled when accessed	0x0*
	ME_CFX_XRAM0_ENABLE	ME of CFX XRAM0 is forced on	0x1
ME_FORCE_CFX_PRAM2	ME_CFX_PRAM2_AUTO	ME of CFX PRAM2 is only enabled when accessed	0x0*
	ME_CFX_PRAM2_ENABLE	ME of CFX PRAM2 is forced on	0x1
ME_FORCE_CFX_PRAM1	ME_CFX_PRAM1_AUTO	ME of CFX PRAM1 is only enabled when accessed	0x0*
	ME_CFX_PRAM1_ENABLE	ME of CFX PRAM1 is forced on	0x1
ME_FORCE_CFX_PRAM0	ME_CFX_PRAM0_AUTO	ME of CFX PRAM0 is only enabled when accessed	0x0*
	ME_CFX_PRAM0_ENABLE	ME of CFX PRAM0 is forced on	0x1

7.4.4.3 D_MEM_CFG_2 Settings

Bit Field	Field Name	Description
6	ME_FORCE_HEAR_PMEM	Force ME of HEAR PMEM
5	ME_FORCE_HEAR_MEMT5	Force ME of HEAR MEMT5
4	ME_FORCE_HEAR_MEMT4	Force ME of HEAR MEMT4
3	ME_FORCE_HEAR_MEMT3	Force ME of HEAR MEMT3
2	ME_FORCE_HEAR_MEMT2	Force ME of HEAR MEMT2
1	ME_FORCE_HEAR_MEMT1	Force ME of HEAR MEMT1
0	ME_FORCE_HEAR_MEMT0	Force ME of HEAR MEMT0

Field Name	Value Symbol	Value Description	Hex Value
ME_FORCE_HEAR_PMEM	ME_HEAR_PMEM_AUTO	ME of HEAR PMEM is only enabled when accessed	0x0*
	ME_HEAR_PMEM_ENABLE	ME of HEAR PMEM is forced on	0x1
ME_FORCE_HEAR_MEMT5	ME_HEAR_MEMT5_AUTO	ME of HEAR MEMT5 is only enabled when accessed	0x0*
	ME_HEAR_MEMT5_ENABLE	ME of HEAR MEMT5 is forced on	0x1
ME_FORCE_HEAR_MEMT4	ME_HEAR_MEMT4_AUTO	ME of HEAR MEMT4 is only enabled when accessed	0x0*
	ME_HEAR_MEMT4_ENABLE	ME of HEAR MEMT4 is forced on	0x1
ME_FORCE_HEAR_MEMT3	ME_HEAR_MEMT3_AUTO	ME of HEAR MEMT3 is only enabled when accessed	0x0*
	ME_HEAR_MEMT3_ENABLE	ME of HEAR MEMT3 is forced on	0x1
ME_FORCE_HEAR_MEMT2	ME_HEAR_MEMT2_AUTO	ME of HEAR MEMT2 is only enabled when accessed	0x0*
	ME_HEAR_MEMT2_ENABLE	ME of HEAR MEMT2 is forced on	0x1

Field Name	Value Symbol	Value Description	Hex Value
ME_FORCE_HEAR_MEMT1	ME_HEAR_MEMT1_AUTO	ME of HEAR MEMT1 is only enabled when accessed	0x0*
	ME_HEAR_MEMT1_ENABLE	ME of HEAR MEMT1 is forced on	0x1
ME_FORCE_HEAR_MEMT0	ME_HEAR_MEMT0_AUTO	ME of HEAR MEMT0 is only enabled when accessed	0x0*
	ME_HEAR_MEMT0_ENABLE	ME of HEAR MEMT0 is forced on	0x1

7.4.4.4 D_MEM_CFG_3 Settings

Bit Field	Field Name	Description
7	ME_FORCE_HEAR_D1	Force ME of HEAR D1
6	ME_FORCE_HEAR_D0	Force ME of HEAR D0
5	ME_FORCE_HEAR_C1	Force ME of HEAR C1
4	ME_FORCE_HEAR_C0	Force ME of HEAR C0
3	ME_FORCE_HEAR_B1	Force ME of HEAR B1
2	ME_FORCE_HEAR_B0	Force ME of HEAR B0
1	ME_FORCE_HEAR_A1	Force ME of HEAR A1
0	ME_FORCE_HEAR_A0	Force ME of HEAR A0

Field Name	Value Symbol	Value Description	Hex Value
ME_FORCE_HEAR_D1	ME_HEAR_D1_AUTO	ME of HEAR D1 is only enabled when accessed	0x0*
	ME_HEAR_D1_ENABLE	ME of HEAR D1 is forced on	0x1
ME_FORCE_HEAR_D0	ME_HEAR_D0_AUTO	ME of HEAR D0 is only enabled when accessed	0x0*
	ME_HEAR_D0_ENABLE	ME of HEAR D0 is forced on	0x1
ME_FORCE_HEAR_C1	ME_HEAR_C1_AUTO	ME of HEAR C1 is only enabled when accessed	0x0*
	ME_HEAR_C1_ENABLE	ME of HEAR C1 is forced on	0x1
ME_FORCE_HEAR_C0	ME_HEAR_C0_AUTO	ME of HEAR C0 is only enabled when accessed	0x0*
	ME_HEAR_C0_ENABLE	ME of HEAR C0 is forced on	0x1
ME_FORCE_HEAR_B1	ME_HEAR_B1_AUTO	ME of HEAR B1 is only enabled when accessed	0x0*
	ME_HEAR_B1_ENABLE	ME of HEAR B1 is forced on	0x1
ME_FORCE_HEAR_B0	ME_HEAR_B0_AUTO	ME of HEAR B0 is only enabled when accessed	0x0*
	ME_HEAR_B0_ENABLE	ME of HEAR B0 is forced on	0x1
ME_FORCE_HEAR_A1	ME_HEAR_A1_AUTO	ME of HEAR A1 is only enabled when accessed	0x0*
	ME_HEAR_A1_ENABLE	ME of HEAR A1 is forced on	0x1
ME_FORCE_HEAR_A0	ME_HEAR_A0_AUTO	ME of HEAR A0 is only enabled when accessed	0x0*
	ME_HEAR_A0_ENABLE	ME of HEAR A0 is forced on	0x1

7.4.4.5 D_MEM_CFG_4 Settings

Bit Field	Field Name	Description
3	ME_FORCE_CM3_SRAM	Force ME of ARM Cortex-M3 processor SRAM
2	ME_FORCE_CM3_PRAM2	Force ME of ARM Cortex-M3 processor PRAM2
1	ME_FORCE_CM3_PRAM1	Force ME of ARM Cortex-M3 processor PRAM1
0	ME_FORCE_CM3_PRAM0	Force ME of ARM Cortex-M3 processor PRAM0

Field Name	Value Symbol	Value Description	Hex Value
ME_FORCE_CM3_SRAM	ME_CM3_SRAM_AUTO	ME of ARM Cortex-M3 processor SRAM is only enabled when accessed	0x0*
	ME_CM3_SRAM_ENABLE	ME of ARM Cortex-M3 processor SRAM is forced on	0x1
ME_FORCE_CM3_PRAM2	ME_CM3_PRAM2_AUTO	ME of ARM Cortex-M3 processor PRAM2 is only enabled when accessed	0x0*
	ME_CM3_PRAM2_ENABLE	ME of ARM Cortex-M3 processor PRAM2 is forced on	0x1
ME_FORCE_CM3_PRAM1	ME_CM3_PRAM1_AUTO	ME of ARM Cortex-M3 processor PRAM1 is only enabled when accessed	0x0*
	ME_CM3_PRAM1_ENABLE	ME of ARM Cortex-M3 processor PRAM1 is forced on	0x1
ME_FORCE_CM3_PRAM0	ME_CM3_PRAM0_AUTO	ME of ARM Cortex-M3 processor PRAM0 is only enabled when accessed	0x0*
	ME_CM3_PRAM0_ENABLE	ME of ARM Cortex-M3 processor PRAM0 is forced on	0x1

7.4.4.6 D_MEM_CFG_5 Settings

Bit Field	Field Name	Description
3	ME_FORCE_FENG_PMEM	Force ME of Filter Engine PMEM
2	ME_FORCE_FENG_CMEM	Force ME of Filter Engine CMEM
1	ME_FORCE_FENG_SMEM1	Force ME of Filter Engine SMEM1
0	ME_FORCE_FENG_SMEM0	Force ME of Filter Engine SMEM0

Field Name	Value Symbol	Value Description	Hex Value
ME_FORCE_FENG_PMEM	ME_FENG_PMEM_AUTO	ME of Filter Engine PMEM is only enabled when accessed	0x0*
	ME_FENG_PMEM_ENABLE	ME of Filter Engine PMEM is forced on	0x1
ME_FORCE_FENG_CMEM	ME_FENG_CMEM_AUTO	ME of Filter Engine CMEM is only enabled when accessed	0x0*
	ME_FENG_CMEM_ENABLE	ME of Filter Engine CMEM is forced on	0x1
ME_FORCE_FENG_SMEM1	ME_FENG_SMEM1_AUTO	ME of Filter Engine SMEM1 is only enabled when accessed	0x0*
	ME_FENG_SMEM1_ENABLE	ME of Filter Engine SMEM1 is forced on	0x1
ME_FORCE_FENG_SMEM0	ME_FENG_SMEM0_AUTO	ME of Filter Engine SMEM0 is only enabled when accessed	0x0*
	ME_FENG_SMEM0_ENABLE	ME of Filter Engine SMEM0 is forced on	0x1

7.5 ARM Cortex-M3 Processor Memory Space

7.5.1 ARM Cortex-M3 Processor Program and Data Memories (RAM)

The ARM Cortex-M3 processor memory consists of two memory instances as shown in Table 6. These memory instances can be augmented by the PRAM0 instance, as noted in Section 7.2.1.1, “CFX Memory Block Instances”.

7.5.2 ARM Cortex-M3 Processor Access to HEAR FIFO Memories

In addition to these memories, the ARM Cortex-M3 processor can access the HEAR FIFO memory as a part of its data memory space. The details of this mapping are listed in Table 17 and described below.

Table 17. HEAR FIFO Memory Mapping

Alignment	HEAR FIFO Name	Type	ARM Cortex-M3 Processor Address
LSB aligned, sign extended	memA0 || memB0	Interleaved	0x20100000-0x20103FFF
	memA1 || memB1	Interleaved	0x20104000-0x20107FFF
	memA0	Linear	0x20108000-0x20109FFF
	memA1	Linear	0x2010A000-0x2010BFFF
	memB0	Linear	0x2010C000-0x2010DFFF
	memB1	Linear	0x2010E000-0x2010FFFF
MSB aligned, zero padded	memA0 || memB0	Interleaved	0x20110000-0x20113FFF
	memA1 || memB1	Interleaved	0x20114000-0x20117FFF
	memA0	Linear	0x20118000-0x20119FFF
	memA1	Linear	0x2011A000-0x2011BFFF
	memB0	Linear	0x2011C000-0x2011DFFF
	memB1	Linear	0x2011E000-0x2011FFFF
LSB aligned, zero extended	memA0 || memB0	Interleaved	0x20120000-0x20123FFF
	memA1 || memB1	Interleaved	0x20124000-0x20127FFF
	memA0	Linear	0x20128000-0x20129FFF
	memA1	Linear	0x2012A000-0x2012BFFF
	memB0	Linear	0x2012C000-0x2012DFFF
	memB1	Linear	0x2012E000-0x2012FFFF
LSB aligned, 16-bit access (MSB data)	memA0 || memB0	Interleaved	0x20130000-0x20133FFF
	memA1 || memB1	Interleaved	0x20134000-0x20137FFF
	memA0	Linear	0x20138000-0x20139FFF
	memA1	Linear	0x2013A000-0x2013BFFF
	memB0	Linear	0x2013C000-0x2013DFFF
	memB1	Linear	0x2013E000-0x2013FFFF

The HEAR FIFO memories are implemented as 4096x48 bits, but they are mapped to the ARM Cortex-M3 core memory space into two separate 8192x24 spaces. The ARM Cortex-M3 processor has the lowest priority when accessing the FIFOs. When the requested FIFO is not available, the access is stalled. Memory accesses to and from the FIFOs are going through an access buffer, which decouples the ARM Cortex-M3 core timing from the other cores. This means that a read access takes at least two cycles. Write accesses are stalled when there is DMA activity or when other conflicts occur. Sequential accesses always have a wait state to allow previous access buffer activity to be completed.

7.5.3 ARM Cortex-M3 Processor Memory Map

The ARM Cortex-M3 processor memory mapping is shown in Figure 6. The address ranges are in hexadecimal format.

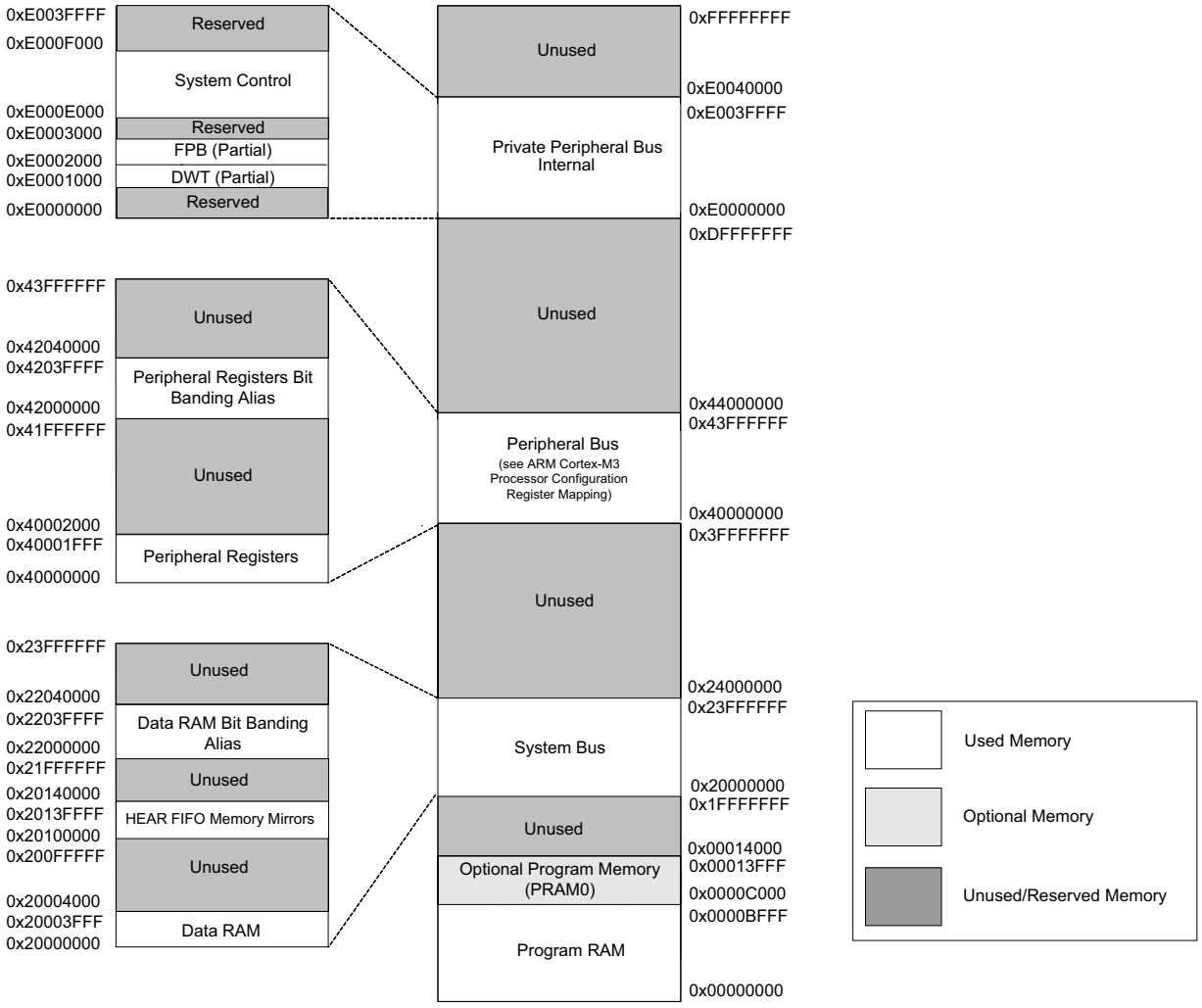


Figure 6. ARM Cortex-M3 Core System Memory Map

7.5.3.1 ARM Cortex-M3 Processor Configuration Register Mapping

The ARM Cortex-M3 core configuration registers are mapped into the P memory space between addresses 0x3000 to 0x37FF and into the ARM Cortex-M3's peripheral memory space between addresses 0x40000000 to 0x4001FFFF. These digital registers are organized by their area of control and mapped as blocks into the P memory space as shown in Table 13. The corresponding ARM Cortex-M3 core system bus memory mapping for these digital registers is shown in Table 18.

Table 18. ARM Cortex-M3 Processor Configuration Register Block Mapping

Controlled/Configured Area	Block Start Address
Error Correction Block (ECB)	0x40000000
Audio Codec Block (ACB)	0x40000100
DMA Controller	0x40000200
External Interrupts	0x40000300
GPIO	0x40000500
SPI	0x40000700
PCM	0x40000800
I^2C	0x40000900
CRC	0x40000A00

CHAPTER 8
FIFO Controller

8.1 Introduction to the FIFO Controller

The Ezairo 7100 system's FIFO controller provides the ability to define up to 32 FIFO buffers. These FIFOs are defined as up to 8 FIFOs in each of the A0, A1, B0 and B1 memories. To simplify discussion of FIFOs, the FIFOs are denoted as FIFO A0.*, FIFO A1.*, FIFO B0.*, and FIFO B1.* in documentation and defines. These FIFOs can be used as:

- Input FIFOs as used by the IOC
- Output FIFOs as used by the IOC
- Intermediate FIFOs for transferring data between the CFX, HEAR and ARM Cortex-M3 cores
- Software FIFOs for the CFX DSP through the use of access registers

Each individual FIFO is associated with a set of FIFO configuration registers that are mapped into X memory starting at 0x7000 (as defined in Table 16 on page 90). Each FIFO can also be associated with one or more of the eight FIFO interrupts that are available for the CFX DSP, eight FIFO events that are available for the HEAR, and eight FIFO interrupts that are available for the ARM Cortex-M3 core.

IMPORTANT: Using the HEAR reset by setting the `SYSTEM_CTRL_HEAR_RESET` bit in the `D_SYSTEM_CTRL` register resets the FIFO controllers, clearing the current FIFO register settings.

8.1.1 FIFO Control and Configuration Register Conventions

Each memory has an identical set of eight FIFOs. As a result, all registers described in this chapter define x, y in `D_FIFO_x_y_*` to allow us to represent all of the FIFO registers as a group by replacing:

x with the name of the memory space: A0, A1, B0 or B1

y with the FIFO number: 0 to 7

The registers for FIFO A0_0 are based at X:0x7000, and the register blocks are spaced 0x10 apart.

8.2 Defining Memory for Use as a FIFO

A FIFO is a section of memory that is treated as a circular buffer in memory where components that use the memory use pointers to indicate the next location to be read or written. Each of the FIFO memories listed in Table 19 can have eight FIFOs defined. All start and end addresses are defined relative to the address offsets listed in this table.

Table 19. FIFO Memories

Memory	Address Offset
A0	0xC000
A1	0xC800
B0	0xD000
B1	0xD800

For each FIFO in memory, define a start and end point for the FIFO, as well as specify the size of the FIFO. Figure 7 shows how the FIFO start and end pointers, along with the FIFO size indicator, are used to define an area of memory for use as a FIFO.

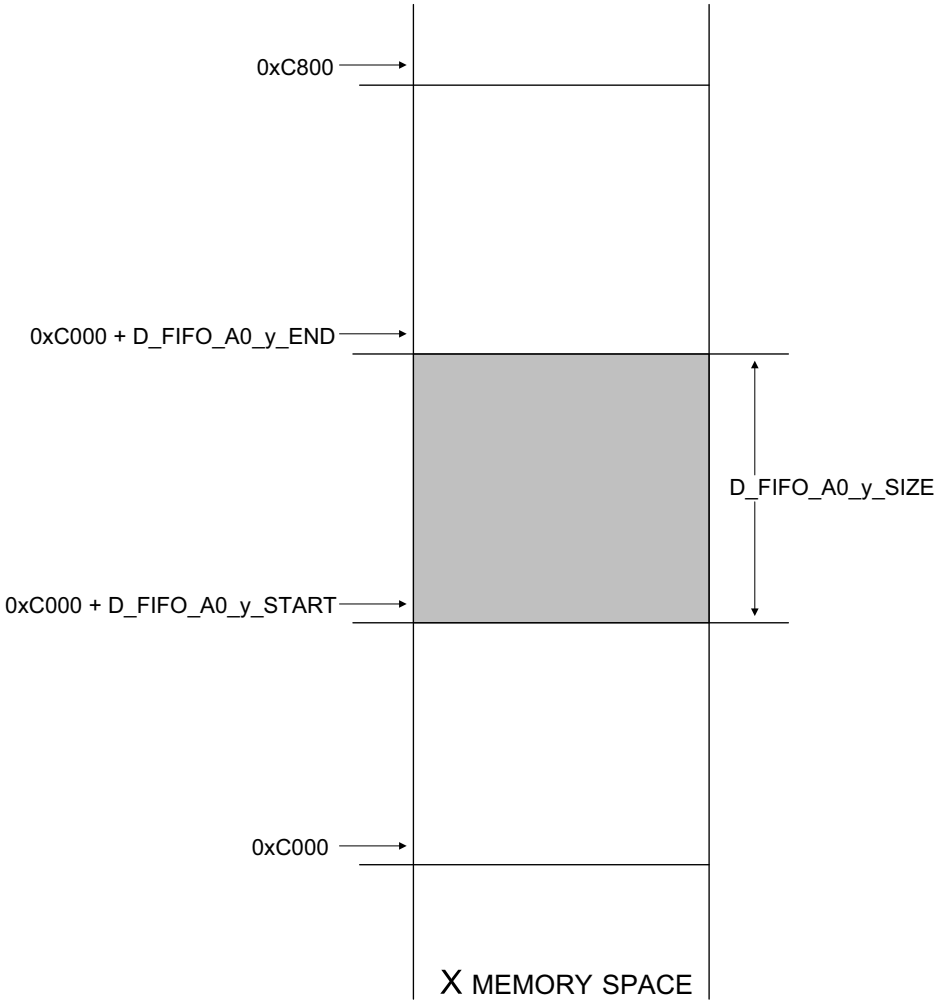


Figure 7. Defining an Area of A0 Memory for Use as a FIFO

The start address for a FIFO is defined by the associated `D_FIFO_x_y_START` register. This register defines the offset from the start of the memory to the FIFO. This start address is limited to steps of 32 words; therefore, the associated bit field of this register is aligned to truncate values that do not meet this requirement. The end address for a FIFO is similarly defined by an offset specified in the associated `D_FIFO_x_y_END` register, and this address is also limited to steps of 32 words by truncation. If a FIFO is enabled, the size of a FIFO must be defined by setting the `D_FIFO_x_y_SIZE` register to the value of (D_FIFO_x_y_END – D_FIFO_x_y_START). FIFOs of up to 2048 words are supported. For all non-zero values, the `FIFO_x_y_SIZE_FIFO_ENABLE` bit from the `D_FIFO_x_y_SIZE` register is automatically set to indicate that the FIFO is active. The address that the `D_FIFO_x_y_END` register points to is not considered part of the area of memory allocated to the FIFO.

NOTE: Setting the D_FIFO_x_y_SIZE register for a FIFO to zero disables a FIFO. All accesses from the IOC or through the associated D_FIFO_x_y_ACCESS register are ignored.

CAUTION: Defining two or more FIFOs that overlap in memory results in undefined behavior. Further, defining an invalid FIFO configuration also results in undefined behavior. Some examples of invalid FIFO configurations include specifying a start address and end address that reserve less memory than the defined size, defining a FIFO that exceeds 2048 words, or defining an invalid area (as would be the case if the end address offset is lower than the start address offset).

All accesses to areas of the A0, A1, B0 or B1 memories that are included in defined FIFOs are processed by the FIFO controller as described throughout the remainder of this chapter. For areas of the A0, A1, B0 or B1 memories that are not included in the defined FIFO ranges, the FIFO controller provides no address manipulation. User applications will access those areas of memory linearly.

8.2.1 Control and Configuration Registers

The following registers are the same for **x** and **y**, where **x** represents A0, A1, B0 or B1 memories, and **y** is 0 to 7.

Register Name	Register Description	Address
D_FIFO_x_y_START	Start offset in x memory for FIFO **x_y**	X:(Base)
D_FIFO_x_y_END	End offset (+1) in x memory for FIFO **x_y**	X:(Base + 1)
D_FIFO_x_y_SIZE	Total size of x memory used in FIFO **x_y**	X:(Base + 2)

8.2.1.1 D_FIFO_x_y_START Settings

Bit Field	Field Name	Description
10:5	OFFSET	Start address for FIFO **x y**

8.2.1.2 D_FIFO_x_y_END Settings

Bit Field	Field Name	Description
10:5	OFFSET	End address (+1) for FIFO **x y**

8.2.1.3 D_FIFO_x_y_SIZE Settings

Bit Field	Field Name	Description
11:5	FIFO_SIZE	Size (End-Start) for FIFO **x y**
11	FIFO_ENABLE	FIFO enable for FIFO **x y**

Field Name	Value Symbol	Value Description	Hex Value
FIFO_ENABLE	FIFO_x_y_DISABLE	Disable the FIFO	0x0*
	FIFO_x_y_ENABLE	Enable the FIFO	0x1

8.3 FIFO Buffer Data and Configuration

There are four primary uses for a FIFO in the Ezairo 7100 system:

1. To collect blocks of input data from the IOC
2. To provide blocks of output data to the IOC
3. In software, to supply an application with a hardware FIFO buffer
4. To transfer data between two cores, including:
   - Transferring data between the CFX DSP and the HEAR
   - Transferring data between the CFX DSP and the ARM Cortex-M3 processor
   - Transferring data between the HEAR and the ARM Cortex-M3 processor

To understand how a FIFO buffer is used for input from the IOC, it is useful to think of the FIFO buffer as divided into two areas: an area to store the input block of data that is currently being received, and an area containing previously received blocks of data. Received data is collected into a block until a complete block of samples has been received.

To understand how a FIFO buffer is used for output from the IOC, it is useful to think of the FIFO buffer as divided into two areas: an area from which the IOC is currently reading output data, and an area to store future output blocks of samples. This second area contains previously read blocks of output data until this data is overwritten by new (future) output samples.

Finally, a FIFO can be used in software as an input FIFO, an output FIFO, or both—except that data is stored or loaded through the appropriate access register (`D_FIFO_x_y_ACCESS`) rather than storing or loading data through the IOC. Alternatively, if a user application written for the CFX is not using the access register, it can read or write directly to the appropriate part of the FIFO and then force a block update by writing the block counter directly to advance the FIFO pointers.

CAUTION: The FIFOs are located in memory that has been organized into a 48-bit double word configuration; therefore, the access register is actually a 48-bit register that allows reading or writing of up to two adjacent words of a FIFO within one cycle. Since the CFX processor and related components do not have 48-bit access to this register, do not read or write the address immediately following an access register (i.e., the second half of the access register) because this is registered as a data access without reading or writing any useful data.

NOTE: All pointer updates, IOC accesses, and accesses through the access register use the FIFO buffer as a circular buffer. As a result, any update calculation that would result in a pointer or access outside of the FIFO itself is reduced modulo the size of the FIFO.

8.3.1 FIFO Input and Output Block

In each use case, the input/output block pointer and block size defines the area of input/output that the IOC or access register is currently accessing. The current block of input or output samples is commonly referred to as the *I/O block* or *IOBLOCK*, and is pointed to by the I/O block pointer. The elements related to the currently used I/O block are shown in Figure 8.

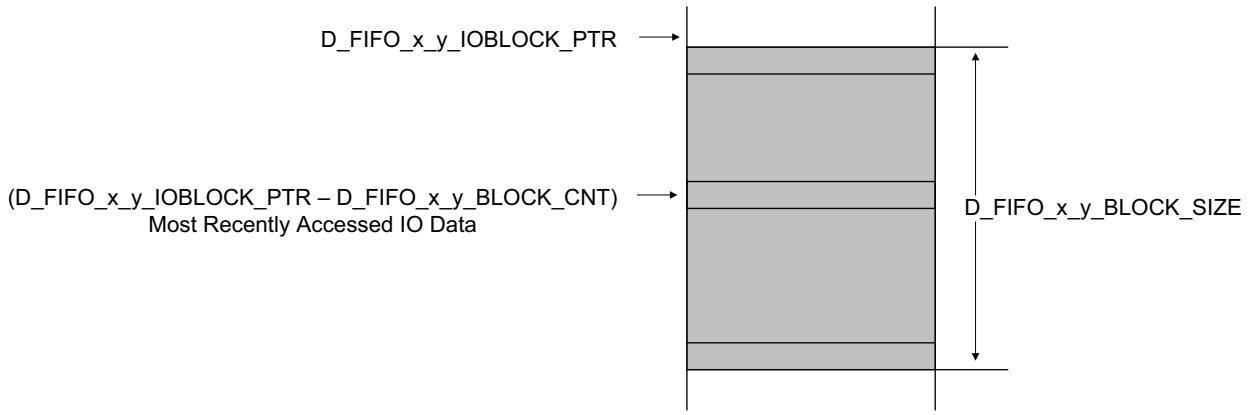


Figure 8. Current Input or Output Data Block for a FIFO

The block size for each I/O block is defined by the `D_FIFO_x_y_BLOCK_SIZE` register. The block size for a FIFO must not exceed the defined size of the FIFO. The block size is further limited to 1 to 1024 words.

NOTE: A setting of 0 in the `D_FIFO_x_y_BLOCK_SIZE` register configures the FIFO for a block size of 1024. For all other block sizes, the register setting matches the block size configured.

The I/O block (shaded area of Figure 8) is an area of the block size that is filled starting at the top of the block. The I/O block pointer, which is accessible through the appropriate `D_FIFO_x_y_IOBLOCK_PTR` register, provides an offset within the FIFO to the first address above the top of this block, indicating that the first data item stored in the block is located one address below the pointer.

Finally, a block counter is maintained by the FIFO controller to indicate to the FIFO controller where to locate the next data access within the I/O block. This counter, which you can see through the appropriate `D_FIFO_x_y_BLOCK_CNT` register, indicates how many words have already been read or written in the current data block. This value is incremented following an IOC access to the FIFO data, or through the access register. The most recently accessed input or output data in the FIFO is located at (D_FIFO_x_y_IOBLOCK_PTR – D_FIFO_x_y_BLOCK_CNT). The next location that will be accessed to store data written to the FIFO, or provide data read from the FIFO, is (D_FIFO_x_y_IOBLOCK_PTR – (D_FIFO_x_y_BLOCK_CNT + 1)).

If at any point the block count is equal to or exceeds the block size, the FIFO configuration is updated to indicate that use of the current input/output data block is complete. This can occur when the block count is updated through one of the following:

- An IOC FIFO access
- An access register read/write
- By explicitly writing a value to the block count register from the CFX or the HEAR

When this happens, the following updates occur:

- The block count resets to 0.

- Both the I/O block pointer and base pointer are decremented by the block size. Both pointers must point to an even address, so if the block size was odd, the I/O block pointer and base pointer are decremented by the (block size + 1).
- The interrupts associated with this FIFO is triggered. For more information about configuring CFX interrupts and using interrupts within an application, see Section 14.1, “Interrupt Controller” on page 218. For more information about configuring and using ARM Cortex-M3 processor interrupts within an application, see Section 15.1, “Nested Vectored Interrupt Controller (NVIC)” on page 239.
- The HEAR receives a function chain controller event triggering the function chain linked to the FIFO. For more information about the function chain controller, see Section 4.2, “Function Chain Controller” on page 36. For more information indicating which FIFO configurations to use with a HEAR microcode configuration, see the *HEAR Configurable Accelerator Reference Manual for Ezairo 7100*.

8.3.2 FIFO Processing Block

The data not currently being used for input or output from the FIFO is available for signal processing operations. A section of this area is normally reserved for use by these operations and is referred to as the processing block or processing area. The beginning of the processing block is defined by the base pointer, which you access through the appropriate `D_FIFO_x_y_BASE_PTR` register. The base pointer provides an offset into the FIFO that is updated along with the I/O block pointer. The base pointer is used to specify what actual hardware data is mapped to the base of the FIFO from a software application perspective. Normally the mapping is used so that the base pointer indicates where in the physical memory the most recent input blocks or output buffer area is located. The FIFO controller in turn uses this pointer to remap the data when addressed by software so that when a software application accesses memory within a FIFO, the memory addressed at the `D_FIFO_x_y_START` register matches the physical memory pointed to by the base pointer.

Figure 9 shows the area of one FIFO (shaded area) as it is bounded by the FIFO start and end points. The hardware memory mapping on the left shows where in memory the `D_FIFO_x_y_BASE_PTR` register actually points and how memory would be aligned if the FIFO were disabled by setting its size to zero (see Section 8.2, “Defining Memory for Use as a FIFO” on page 98 above). The software memory mapping on the right shows how memory appears to be aligned to software accessing the FIFO. This aligns the memory pointed to by the `D_FIFO_x_y_BASE_PTR` register to the base of the FIFO (located at the offset of the `D_FIFO_x_y_START` register).

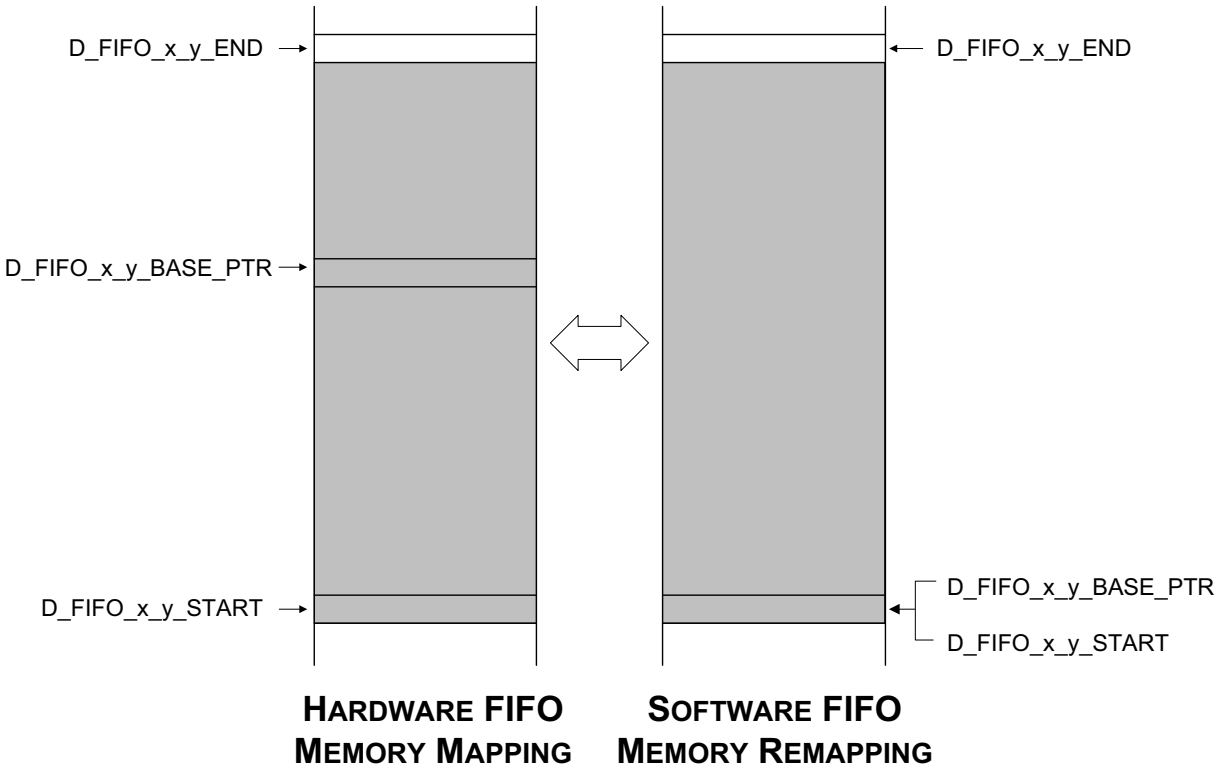


Figure 9. Hardware Memory Mapping and Software View of FIFO Memory Remapping

8.3.3 Typical FIFO Configurations

8.3.3.1 Input FIFO Configuration

When using a FIFO for input, a typical initial configuration might be:

- Base pointer set to zero
- I/O block pointer set to zero

This results in the latest full block of input samples always being located at the base of memory, and provides the maximum amount of historical FIFO data to draw upon for algorithm calculations. Figure 10 illustrates a typical input FIFO configuration.

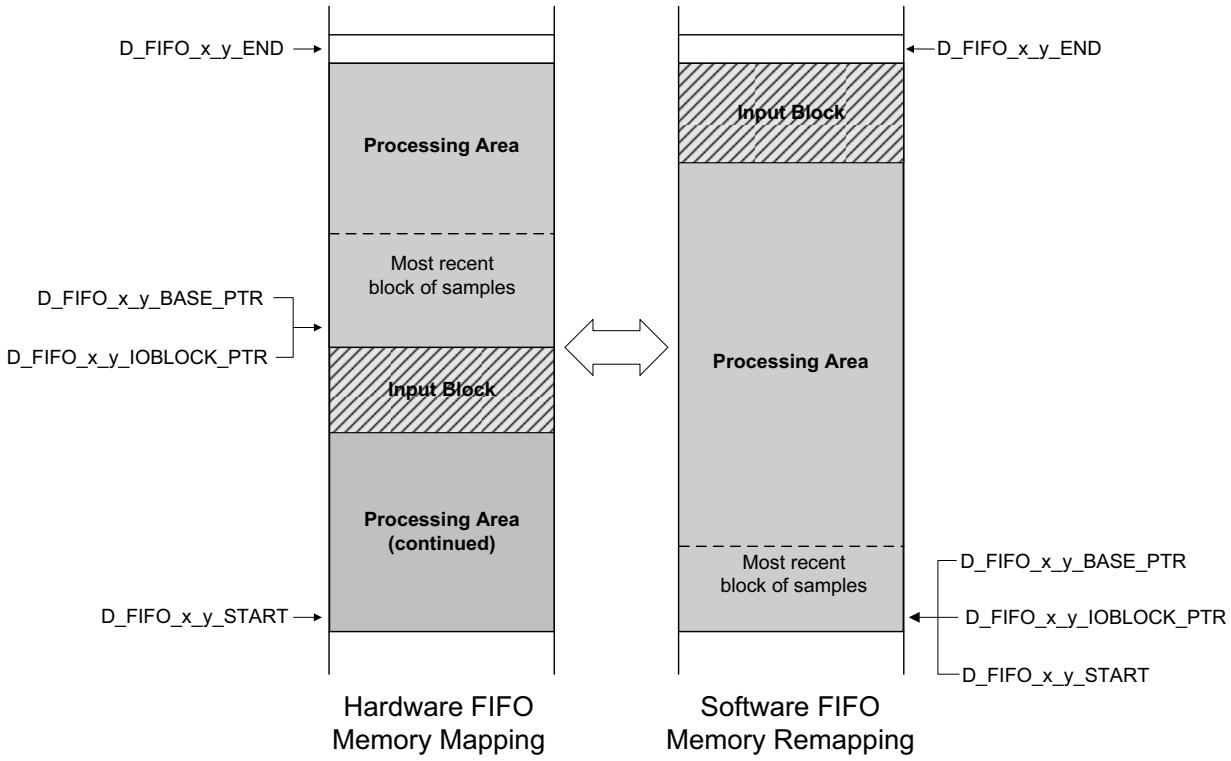


Figure 10. Input FIFO Hardware Memory Mapping and Software View of Memory Remapping

8.3.3.2 Output FIFO Configuration

When using a FIFO for output, a typical initial configuration might be:

- Base pointer set to zero
- I/O block pointer set to (processing area + block size)

This results in the top block of data from the processing area becoming the next output block each time the pointers are updated. It also ensures that the processing area is located at the start of the FIFO for convenient application access. Figure 11 illustrates a typical output FIFO configuration.

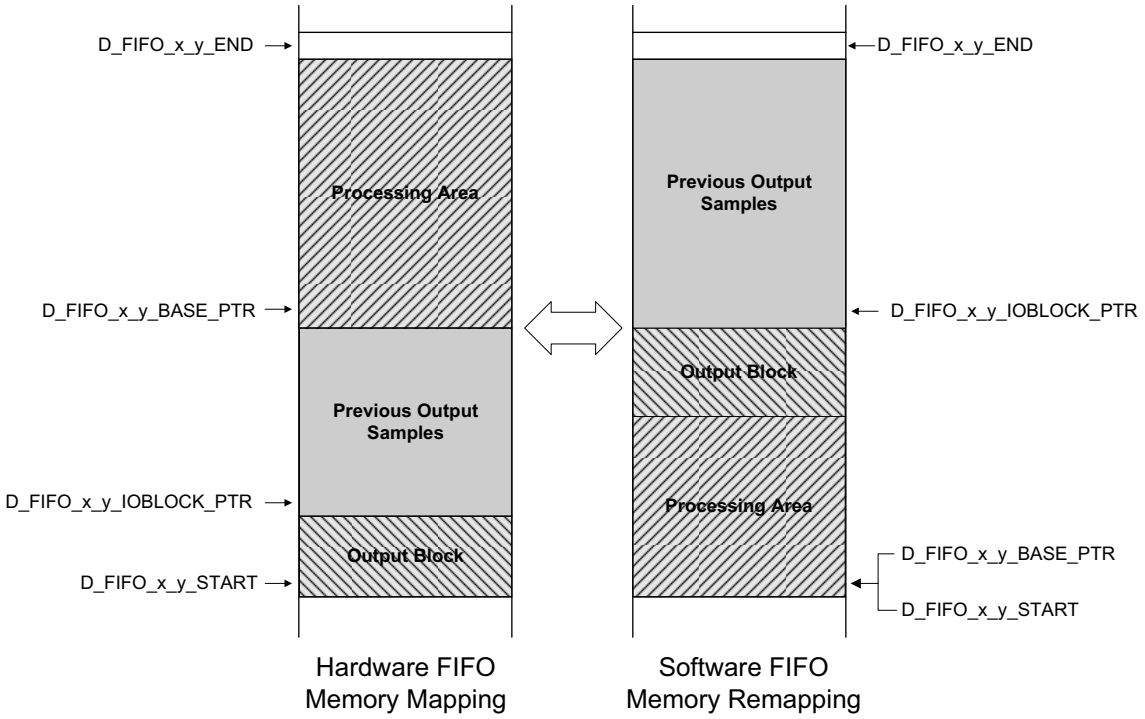


Figure 11. Output FIFO Hardware Memory Mapping and Software View of Memory Remapping

8.3.4 Control and Configuration Registers

These registers are the same for **x** and **y**, where **x** represents A0, A1, B0 or B1 memories, and **y** is 0 to 7.

Register Name	Register Description	Address
D_FIFO_x_y_BASE_PTR	Pointer to the base of FIFO **x_y**	X:(Base + 4)
D_FIFO_x_y_IOBLOCK_PTR	Pointer to the latest I/O block in FIFO **x_y**	X:(Base + 5)
D_FIFO_x_y_BLOCK_SIZE	Size of one block of samples for FIFO **x_y**	X:(Base + 8)
D_FIFO_x_y_BLOCK_CNT	Next read/write location in current sample block	X:(Base + C)
D_FIFO_x_y_ACCESS	Smart FIFO access for FIFO **x_y**	X:(Base + E)

8.3.4.1 D_FIFO_x_y_BASE_PTR

Bit Field	Field Name	Description
10:1	FIFO_PTR	FIFO pointer for FIFO **x y**

8.3.4.2 D_FIFO_x_y_IOBLOCK_PTR

Bit Field	Field Name	Description
10:1	IOC_PTR	IOC pointer for FIFO **x y**

8.3.4.3 D_FIFO_x_y_BLOCK_SIZE

Bit Field	Field Name	Description
9:0	BLOCK_SIZE	Block size for FIFO **x y**

8.3.4.4 D_FIFO_x_y_BLOCK_CNT

Bit Field	Field Name	Description
9:0	BLOCK_CNT	Block counter for FIFO **x y**

8.3.4.5 D_FIFO_x_y_ACCESS

Bit Field	Field Name	Description
23:0	FIFO_DATA	Smart FIFO access for FIFO **x y**

8.4 FIFO Interrupt Configuration

The Ezairo 7100 system allows each of the 32 FIFOs to be defined as the source for one or more of the 24 FIFO interrupts and events that can be used to trigger processing on the CFX, the HEAR, or the ARM Cortex-M3 processor.

To configure a FIFO as the source for an interrupt or event, set the D_FIFO_INT_CFX_*, D_FIFO_INT_HEAR_*, or D_FIFO_INT_CM3_* registers using the FIFO_INT_x_y bit settings to indicate which FIFO will trigger which interrupts and events. For example, to trigger ARM Cortex-M3 processor FIFO interrupt 2 on FIFO A1.3, write FIFO_INT_A1_3 to D_FIFO_INT_CM3_2.

8.4.1 Control and Configuration Registers

Register Name	Register Description	Address
D_FIFO_INT_CFX_0	CFX FIFO interrupt selection	X:0x7200
D_FIFO_INT_CFX_1	CFX FIFO interrupt selection	X:0x7201
D_FIFO_INT_CFX_2	CFX FIFO interrupt selection	X:0x7202
D_FIFO_INT_CFX_3	CFX FIFO interrupt selection	X:0x7203
D_FIFO_INT_CFX_4	CFX FIFO interrupt selection	X:0x7204
D_FIFO_INT_CFX_5	CFX FIFO interrupt selection	X:0x7205
D_FIFO_INT_CFX_6	CFX FIFO interrupt selection	X:0x7206
D_FIFO_INT_CFX_7	CFX FIFO interrupt selection	X:0x7207
D_FIFO_INT_HEAR_0	HEAR FIFO event selection	X:0x7210
D_FIFO_INT_HEAR_1	HEAR FIFO event selection	X:0x7211
D_FIFO_INT_HEAR_2	HEAR FIFO event selection	X:0x7212
D_FIFO_INT_HEAR_3	HEAR FIFO event selection	X:0x7213
D_FIFO_INT_HEAR_4	HEAR FIFO event selection	X:0x7214
D_FIFO_INT_HEAR_5	HEAR FIFO event selection	X:0x7215
D_FIFO_INT_HEAR_6	HEAR FIFO event selection	X:0x7216
D_FIFO_INT_HEAR_7	HEAR FIFO event selection	X:0x7217

Register Name	Register Description	Address
D_FIFO_INT_CM3_0	ARM Cortex-M3 core FIFO interrupt selection	X:0x7220
D_FIFO_INT_CM3_1	ARM Cortex-M3 core FIFO interrupt selection	X:0x7221
D_FIFO_INT_CM3_2	ARM Cortex-M3 core FIFO interrupt selection	X:0x7222
D_FIFO_INT_CM3_3	ARM Cortex-M3 core FIFO interrupt selection	X:0x7223
D_FIFO_INT_CM3_4	ARM Cortex-M3 core FIFO interrupt selection	X:0x7224
D_FIFO_INT_CM3_5	ARM Cortex-M3 core FIFO interrupt selection	X:0x7225
D_FIFO_INT_CM3_6	ARM Cortex-M3 core FIFO interrupt selection	X:0x7226
D_FIFO_INT_CM3_7	ARM Cortex-M3 core FIFO interrupt selection	X:0x7227

8.4.1.1 D_FIFO_INT_CFX_* Settings

The following bit fields and field names apply equally to registers D_FIFO_INT_CFX_0 to D_FIFO_INT_CFX_7.

Bit Field	Field Name	Description
4:0	SELECT_INT	Select the source for a CFX FIFO interrupt

Field Name	Value Symbol	Value Description	Hex Value
SELECT_INT	FIFO_INT_A0_0	Select FIFO controller A0 interrupt 0 (interrupt 0 default)	0x0*
	FIFO_INT_A0_1	Select FIFO controller A0 interrupt 1 (interrupt 1 default)	0x1
	FIFO_INT_A0_2	Select FIFO controller A0 interrupt 2	0x2
	FIFO_INT_A0_3	Select FIFO controller A0 interrupt 3	0x3
	FIFO_INT_A0_4	Select FIFO controller A0 interrupt 4	0x4
	FIFO_INT_A0_5	Select FIFO controller A0 interrupt 5	0x5
	FIFO_INT_A0_6	Select FIFO controller A0 interrupt 6	0x6
	FIFO_INT_A0_7	Select FIFO controller A0 interrupt 7	0x7
	FIFO_INT_A1_0	Select FIFO controller A1 interrupt 0 (interrupt 2 default)	0x8
	FIFO_INT_A1_1	Select FIFO controller A1 interrupt 1 (interrupt 3 default)	0x9
	FIFO_INT_A1_2	Select FIFO controller A1 interrupt 2	0xA
	FIFO_INT_A1_3	Select FIFO controller A1 interrupt 3	0xB
	FIFO_INT_A1_4	Select FIFO controller A1 interrupt 4	0xC
	FIFO_INT_A1_5	Select FIFO controller A1 interrupt 5	0xD
	FIFO_INT_A1_6	Select FIFO controller A1 interrupt 6	0xE
	FIFO_INT_A1_7	Select FIFO controller A1 interrupt 7	0xF
	FIFO_INT_B0_0	Select FIFO controller B0 interrupt 0 (interrupt 4 default)	0x10
	FIFO_INT_B0_1	Select FIFO controller B0 interrupt 1 (interrupt 5 default)	0x11
	FIFO_INT_B0_2	Select FIFO controller B0 interrupt 2	0x12
	FIFO_INT_B0_3	Select FIFO controller B0 interrupt 3	0x13
	FIFO_INT_B0_4	Select FIFO controller B0 interrupt 4	0x14
	FIFO_INT_B0_5	Select FIFO controller B0 interrupt 5	0x15
	FIFO_INT_B0_6	Select FIFO controller B0 interrupt 6	0x16
	FIFO_INT_B0_7	Select FIFO controller B0 interrupt 7	0x17
	FIFO_INT_B1_0	Select FIFO controller B1 interrupt 0 (interrupt 6 default)	0x18
	FIFO_INT_B1_1	Select FIFO controller B1 interrupt 1 (interrupt 7 default)	0x19
	FIFO_INT_B1_2	Select FIFO controller B1 interrupt 2	0x1A
	FIFO_INT_B1_3	Select FIFO controller B1 interrupt 3	0x1B
	FIFO_INT_B1_4	Select FIFO controller B1 interrupt 4	0x1C
	FIFO_INT_B1_5	Select FIFO controller B1 interrupt 5	0x1D
	FIFO_INT_B1_6	Select FIFO controller B1 interrupt 6	0x1E
	FIFO_INT_B1_7	Select FIFO controller B1 interrupt 7	0x1F

8.4.1.2 D_FIFO_INT_HEAR_* Settings

The following bit fields and field names apply equally to registers D_FIFO_INT_HEAR_0 to D_FIFO_INT_HEAR_7.

Bit Field	Field Name	Description
4:0	SELECT_INT	Select the source for a HEAR FIFO event

8.4.1.3 D_FIFO_INT_CM3_0

The following bit fields and field names apply equally to registers D_FIFO_INT_CM3_0 to D_FIFO_INT_CM3_7.

Bit Field	Field Name	Description
4:0	SELECT_INT	Select the source for an ARM Cortex-M3 core FIFO interrupt

8.5 Access Priority

All access to A0, A1, B0 and B1 memories as well as the FIFO configuration registers goes through the FIFO controller. To control which access has priority and to set the sequence in which the FIFOs receive accesses, the FIFO controller uses a fixed access priority scheme. The fixed access priority that is enforced by the FIFO controller is:

1. Direct read/write access by the IOC.
2. Read/write access using an access register (D_FIFO_xx_ACCESS) for a FIFO through the shared memory bus controller.
3. Write access to a block counter register (D_FIFO_xx_BLOCK_CNT) for a FIFO through the shared memory bus controller.
4. Read/write access to any remaining FIFO register through the shared memory bus controller.
5. Read/write access to any area outside of a defined FIFO buffer through the shared memory bus controller (linear access).
6. Read/write access to an area within a FIFO buffer through the shared memory bus controller.
7. Read/write access to an area outside of a FIFO buffer through the ARM Cortex-M3 core access to the FIFO controller.
8. Read/write access to an area within a FIFO buffer through the ARM Cortex-M3 core access to the FIFO controller.

CHAPTER 9
Input/Output Audio Control

9.1 AUDIO MULTIPLEXING

Input/output data can be routed along a number of different paths using the multiplexing options available in the Ezairo 7100 system. Figure 12 shows how the input and output stages are multiplexed with the Filter Engine, digital microphones, and bypass registers. This multiplexing ensures that all relevant data is handled appropriately outside of the main processing paths that use the CFX DSP, HEAR Accelerator, and the ARM Cortex-M3 processor.

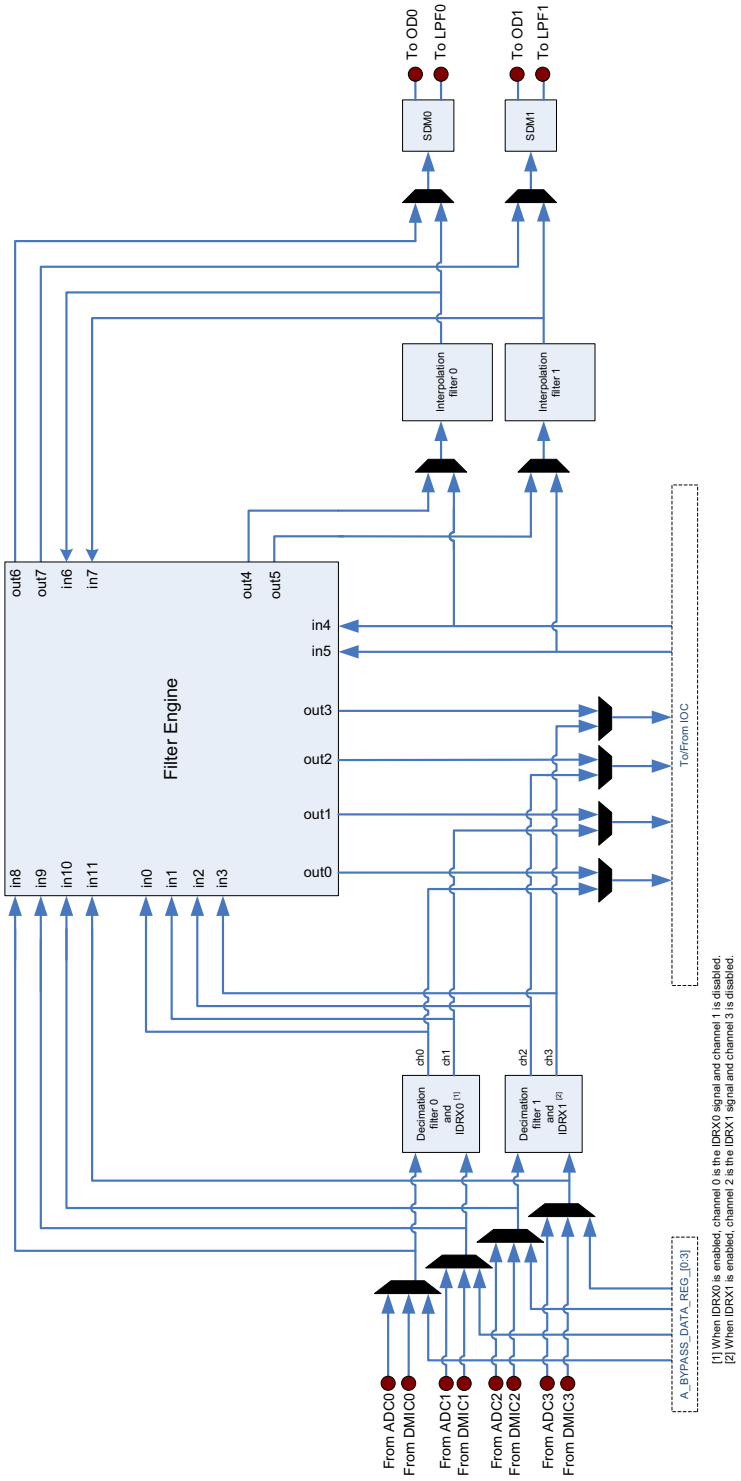


Figure 12. Audio Filtering and Multiplexing

9.1.1 Audio Multiplexing with Filter Engine Outputs

The Filter Engine is linked to the audio input and output stage to optionally pre-process the input signal and post-process the output signal. It also provides a low group-delay processing path that can bypass or replace the input decimation filters and output interpolation filters. Figure 12 on page 112 shows how:

- The Filter Engine input and output signals are connected to the input and output stages
- The audio signals that bypass the Filter Engine are multiplexed with Filter Engine pre-processed signals and post-processed signals.

The group-delay in the low group-delay processing path provided by the Filter Engine consists of delays introduced by:

- The high-pass filter formed by the input decoupling capacitor and input bias resistor
- The preamplifier including input low pass filters
- The ADC
- The Filter Engine processing itself
- The sigma-delta modulator
- The H-bridge

In addition to these delays, the filtering used on outputs from the Filter Engine affect the overall system group delay. Figure 13 shows the cumulative effects of the delays that are encountered internal to the Ezairo 7100 system (i.e., not including the group delay introduced by the output filtering) for a simple passthrough from an analog input to a receiver output with no Filter Engine processing. As shown, the group delay observed is frequency dependent, with the total group delay settling to approximately 32 µs for frequencies above 1 kHz.

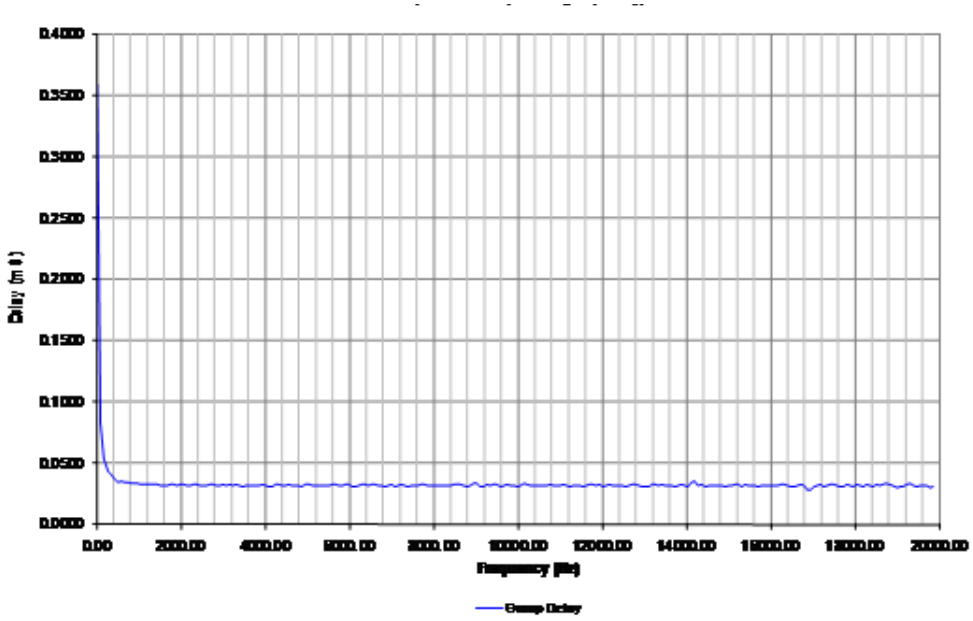


Figure 13. Group Delay from Analog Input to Receiver Output Using the Filter Engine

To select the source that the IOC uses for each input channel, set the AUDIO_MUX_CFG_IOC_SRC* bit fields from the A_AUDIO_MUX_CFG register. Each source can be configured to use the output of the input stage's decimation filters or outputs 0, 1, 2, 3 (for channels 0, 1, 2 and 3 respectively) from the Filter Engine.

To select the source for the interpolation filter inputs between the IOC and outputs 4 or 5 from the Filter Engine, set the AUDIO_MUX_CFG_INT_FILTER0 and AUDIO_MUX_CFG_INT_FILTER1 bit fields from the A_AUDIO_MUX_CFG register.

To select the source for the sigma-delta modulators between the interpolation filter outputs and outputs 6 or 7 from the Filter Engine, set the AUDIO_MUX_CFG_SDM0 and AUDIO_MUX_CFG_SDM1 bit fields from the A_AUDIO_MUX_CFG register.

The audio multiplexing circuitry allows inputs to the interpolation filters and sigma-delta modulators to be truncated or rounded. To configure whether rounding or truncation is used to convert the input data to these blocks from 24 bits to 18 bits of precision, use the AUDIO_MUX_CFG_INT_FILTER0_ROUND, AUDIO_MUX_CFG_INT_FILTER1_ROUND, AUDIO_MUX_CFG_SDM0_ROUND and AUDIO_MUX_CFG_SDM1_ROUND bits from the A_AUDIO_MUX_CFG register. If the input will result in a value of zero, use the AUDIO_MUX_CFG_ROUND_MODE bit from the A_AUDIO_MUX_CFG register to add an offset of ±1 instead of strictly rounding to the nearest integer.

For more information, see Section 6.2, "Filter Engine in the Audio Path" on page 50.

9.1.2 Audio Multiplexing of Audio Input Sources

Audio input data for each of the input channels can be configured to come from one of the following sources:

- The analog inputs, preamplifiers and ADCs of the input stages as described in Chapter 10, "Input Stage" on page 140
- The digital microphone inputs, which are described in Section 10.6, "Digital Microphone Inputs" on page 149. Each input channel can be configured to be triggered from the rising edge or falling edge of the specified DMIC input. For more information about configuring DIOs as digital microphone inputs or outputs, see Section 16.1, "Digital I/O (DIO)" on page 351.
- Instead of data from the ADCs or digital microphones, each input channel can source data from CFX-controlled data registers. The core provides this input data through the A_BYPASS_DATA_REG_* registers. These bypass registers can be used to form a feedback path or to provide debug data at the inputs to the decimation filter and Filter Engine.

The data from each of these sources is used as the input to the input stage's decimation filters for each channel; the Filter Engine can use it directly as inputs 8, 9, 10 and 11. To select the source for each channel, set the AUDIO_MUX_CFG_INPUT_CH*_SRC bit fields from the A_AUDIO_MUX_CFG register.

9.1.3 Control and Configuration Registers

Register Name	Register Description	Address
A_AUDIO_MUX_CFG	Configuration register for the audio filtering and multiplexing block	P:0xF156
A_BYPASS_DATA_REG_0	Audio input registers for the audio filtering and multiplexing block	P:0xF157
A_BYPASS_DATA_REG_1	Audio input registers for the audio filtering and multiplexing block	P:0xF158
A_BYPASS_DATA_REG_2	Audio input registers for the audio filtering and multiplexing block	P:0xF159
A_BYPASS_DATA_REG_3	Audio input registers for the audio filtering and multiplexing block	P:0xF15A

9.1.3.1 A_AUDIO_MUX_CFG Settings

Bit Field	Field Name	Description
21	DMIC_IN_CLK_SRC	Selects the DMIC IN clock source
20	ROUND_MODE	Selects the rounding method
19	SDM1_ROUND	Applies rounding at the Sigma-Delta 1 input
18	SDM0_ROUND	Applies rounding at the Sigma-Delta 0 input
17	INT_FILTER1_ROUND	Applies rounding at the interpolation filter 1 input
16	INT_FILTER0_ROUND	Applies rounding at the interpolation filter 0 input
15:14	INPUT_CH3_SRC	Selects the source of the audio input channel 3
13:12	INPUT_CH2_SRC	Selects the source of the audio input channel 2
11:10	INPUT_CH1_SRC	Selects the source of the audio input channel 1
9:8	INPUT_CH0_SRC	Selects the source of the audio input channel 0
7	IOC_SRC3	Configures the IOC audio input channel 3
6	IOC_SRC2	Configures the IOC audio input channel 2
5	IOC_SRC1	Configures the IOC audio input channel 1
4	IOC_SRC0	Configures the IOC audio input channel 0
3	INT_FILTER1	Configures the interpolation filter input channel 1
2	INT_FILTER0	Configures the interpolation filter input channel 0
1	SDM1	Configures the Sigma-Delta input channel 1
0	SDM0	Configures the Sigma-Delta input channel 0

Field Name	Value Symbol	Value Description	Hex Value
DMIC_IN_CLK_SRC	DMIC_CLK_SRC_EXTCLK	EXTCLK is used as DMIC clock source	0x0*
	DMIC_CLK_SRC_SLOWCLK	SLOWCLK is used as DMIC clock source	0x1
ROUND_MODE	ROUND_NORMAL	Normal rounding is applied	0x0*
	ROUND_AVOID_ALL_ZEROS	Avoid a signal containing all zeros	0x1
SDM1_ROUND	SDM1_TRUNCATE	SDM1 input data are truncated (24 to 18 bits)	0x0*
	SDM1_ROUND	SDM1 input data are rounded	0x1
SDM0_ROUND	SDM0_TRUNCATE	SDM0 input data are truncated (24 to 18 bits)	0x0*
	SDM0_ROUND	SDM0 input data are rounded	0x1
INT_FILTER1_ROUND	INT_FILTER1_TRUNCATE	Interpolation filter 1 data are truncated (24 to 18 bits)	0x0*
	INT_FILTER1_ROUND	Interpolation filter 1 data are rounded	0x1
INT_FILTER0_ROUND	INT_FILTER0_TRUNCATE	Interpolation filter 0 data are truncated (24 to 18 bits)	0x0*
	INT_FILTER0_ROUND	Interpolation filter 0 data are rounded	0x1
INPUT_CH3_SRC	ADC3_OUT	Data is provided by the ADC	0x0*
	BYPASS_DATA_REG3	Data is provided by the bypass register	0x1
	DMIC3_DATA_RE	Data is provided from DMIC (rising-edge)	0x2
	DMIC3_DATA_FE	Data is provided from DMIC (falling-edge)	0x3

Field Name	Value Symbol	Value Description	Hex Value
INPUT_CH2_SRC	ADC2_OUT	Data is provided by the ADC	0x0*
	BYPASS_DATA_REG2	Data is provided by the bypass register	0x1
	DMIC2_DATA_RE	Data is provided from DMIC (rising-edge)	0x2
	DMIC2_DATA_FE	Data is provided from DMIC (falling-edge)	0x3
INPUT_CH1_SRC	ADC1_OUT	Data is provided by the ADC	0x0*
	BYPASS_DATA_REG1	Data is provided by the bypass register	0x1
	DMIC1_DATA_RE	Data is provided from DMIC (rising-edge)	0x2
	DMIC1_DATA_FE	Data is provided from DMIC (falling-edge)	0x3
INPUT_CH0_SRC	ADC0_OUT	Data is provided by the ADC	0x0*
	BYPASS_DATA_REG0	Data is provided by the bypass register	0x1
	DMIC0_DATA_RE	Data is provided from DMIC (rising-edge)	0x2
	DMIC0_DATA_FE	Data is provided from DMIC (falling-edge)	0x3
IOC_SRC3	IOC_SRC3_DEC_FILTER	Decimation filter 3 output	0x0*
	IOC_SRC3_FENG	Filter Engine output f_out3	0x1
IOC_SRC2	IOC_SRC2_DEC_FILTER	Decimation filter 2 output	0x0*
	IOC_SRC2_FENG	Filter Engine output f_out2	0x1
IOC_SRC1	IOC_SRC1_DEC_FILTER	Decimation filter 1 output	0x0*
	IOC_SRC1_FENG	Filter Engine output f_out1	0x1
IOC_SRC0	IOC_SRC0_DEC_FILTER	Decimation filter 0 output	0x0*
	IOC_SRC0_FENG	Filter Engine output f_out0	0x1
INT_FILTER1	INT_FILTER1_IOC	IOC audio output 1	0x0*
	INT_FILTER1_FENG	Filter Engine output f_out5	0x1
INT_FILTER0	INT_FILTER0_IOC	IOC audio output 0	0x0*
	INT_FILTER0_FENG	Filter Engine output f_out4	0x1
SDM1	SDM1_INT_FILTER	Interpolation filter 1 output	0x0*
	SDM1_FENG	Filter Engine output f_out7	0x1
SDM0	SDM0_INT_FILTER	Interpolation filter 0 output	0x0*
	SDM0_FENG	Filter Engine output f_out6	0x1

9.1.3.2 A_BYPASS_DATA_REG_0

Bit Field	Field Name	Description
23:7	BYPASS_DATA	17-bit signed data for the input data bypass (sign extended to 32 bits)

9.1.3.3 A_BYPASS_DATA_REG_1

Bit Field	Field Name	Description
23:7	BYPASS_DATA	17-bit signed data for the input data bypass (sign extended to 32 bits)

9.1.3.4 A_BYPASS_DATA_REG_2

Bit Field	Field Name	Description
23:7	BYPASS_DATA	17-bit signed data for the input data bypass (sign extended to 32 bits)

9.1.3.5 A_BYPASS_DATA_REG_3

Bit Field	Field Name	Description
23:7	BYPASS_DATA	17-bit signed data for the input data bypass (sign extended to 32 bits)

9.2 Introduction to the Input/Output Controller

The input/output controller (IOC) moves signal data from the Ezairo 7100 system's input sources into system memory to be processed, and out of system memory to the system's output sinks. To do this, the IOC is connected appropriately between each of the interfaces (supported by a number of audio multiplexers as described in Section 9.1, "Audio Multiplexing"), and enjoys privileged access to the FIFOs through the FIFO controller. The IOC uses the FIFOs to provide:

- Data for processing by the CFX DSP, HEAR Accelerator, and the ARM Cortex-M3 processor
- Source data to the system outputs and the Filter Engine

The IOC interacts directly with each of the following components, as illustrated in Figure 14:

- The input stage, where inputs come from the ADCs or the digital microphone inputs
- The output stage, where the outputs go to the DACs or the digital microphone outputs
- The PCM interface
- The FIFOs and the FIFO controller
- The Filter Engine

As a result, these components must be referenced when discussing the IOC. For more details about any of these other system components, refer to their respective chapters in this document.

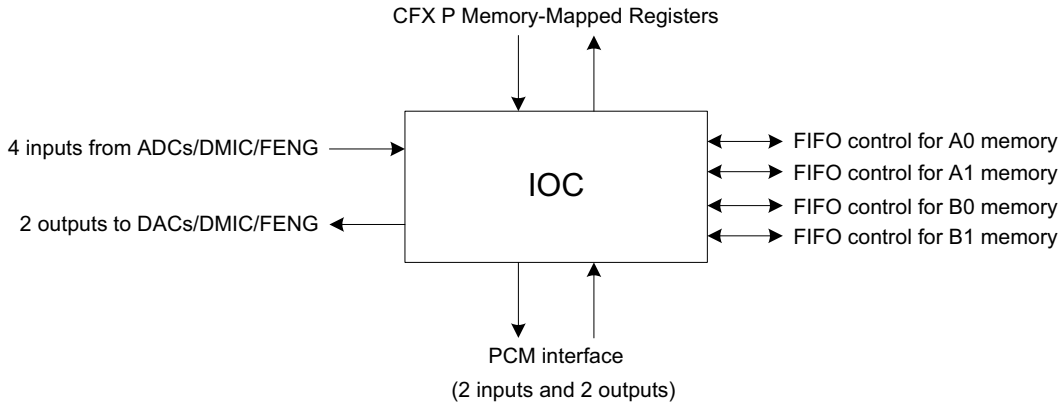


Figure 14. Components Interfacing with the IOC

NOTE: Writing to any of the IOC configuration registers, even if no register settings are updated, resets the internal state machines of the IOC. This forces the IOC to resynchronize to the input sources it is reading data from and output sinks that it is providing data to. Until the next time the IOC would read from AI0 (even if AI0 is disabled), when the IOC is resynchronized, input samples are not read by the IOC and output samples are not provided by the IOC.

IMPORTANT: To synchronize all of the input and output signals, the IOC operates at the sampling frequency determined by the ADC sample frequency configuration. For information about configuring the sampling frequency, see Section 10.5, "Analog-to-Digital Converters (ADCs)" on page 144.

9.3 IOC CONFIGURATION

Data being written or read by the IOC from a single FIFO can be accessed one word (single access mode) or two words (double access mode) at a time. Using double access mode reduces the number of times the IOC accesses the FIFOs, thus reducing the possibility of FIFO access conflicts with the CFX, HEAR and ARM Cortex-M3 cores.

If one input source or one output sink uses a FIFO, the `IOC_FIFO_ACCESS_CFG_FIFO_x_y_DBL_ACC` bit in the `D_IOC_FIFO_ACCESS_CFG` register configures whether the IOC uses single or double access mode.

If more than one input source or output sink uses a FIFO, the `IOC_FIFO_ACCESS_CFG_FIFO_x_y_DBL_ACC` bit must be cleared. In this configuration, the IOC always uses single access mode for an odd number of sources or sinks. The IOC always uses double access mode for an even number of sources or sinks.

NOTE: Writing to the `D_IOC_INPUT_ADC_CFG`, `D_IOC_INPUT_PCM_CFG` or the `D_IOC_FIFO_ACCESS_CFG` register resets the input state machine of the IOC.

9.4 IOC INPUT

On the input side, the IOC is attached to the four input channels from the input stage and Filter Engine, denoted as ADC0, ADC1, ADC2 and ADC3 in constants and documentation.

- The decimation filter outputs for each of the input stage channels provide 18-bit input data. This 18-bit data can be aligned to the lower or upper 18 bits of the 24-bit input word by setting or clearing the appropriate `IOC_FIFO_SHIFT_CFG_ADC*_SHIFT` bit in the `D_IOC_FIFO_SHIFT_CFG` register. If aligned to the lower 18 bits, the data is sign extended to 24 bits. If aligned to the upper 18 bits, the data is zero padded to 24 bits.
- Data provided to the IOC by the Filter Engine is treated as 24-bit data. As a result, if an input channel is configured to use the Filter Engine as its input source for a channel, this data is used directly by the IOC.

For each of these input channels the IOC provides an access register (`D_ADC*_DATA`) that can be used to supply the most recent data transferred for that input channel.

In addition to the input channels, the IOC can be attached to up to two channels of input from the PCM interface. Denoted as PCM0 and PCM1 in constants and documentation, these channels are nominally considered as providing 24-bit input data. The PCM interface configuration can be used to align data that is shorter than 24 bits.

The IOC (if configured correctly) can direct each of the six input sources into individual FIFOs that have been configured. You can select which FIFO each channel is connected to by writing the desired configuration to the

IOC_INPUT_CFG_xxx fields in the D_IOC_INPUT_ADC_CFG and D_IOC_INPUT_PCM_CFG registers. Each of these bit fields allows you to select from one of the 32 FIFOs, or to not store the input at all.

IMPORTANT: Connecting IOC input sources to FIFOs that have not been configured results in undefined behavior.

Additionally, the input channels that are attached to the IOC can be pair-wise configured to provide input dynamic range extended (IDRX) channels of 23-bit data. In constants and documentation, the input channel formed by combining ADC0 with ADC1 is denoted as IDRX0, and the input channel formed by combining ADC2 with ADC3 is denoted as IDRX1.

Input and system configuration requirements related to the use of one or both dynamic range extended input channels are described in Section 10.8, “Input Dynamic Range Extension” on page 155. The input channel used by the IOC to transfer data from the combined IDRX input can be observed using the IDRX_CFG_*_CHANNEL_SEL bit field from the A_IDRX_CFG_* register.

Data from more than one input channel can be stored in the same FIFO. In the case that more than one channel of the input stage is directed into a FIFO, the data received is guaranteed to be written to the FIFO ordered (in time sequence) ADC0, ADC2, ADC1 and ADC3. Similarly, data from more than one PCM input channel can be stored into the same FIFO. The data received from the PCM interface is guaranteed to be written to the FIFO ordered PCM0, PCM1. For details about how input FIFO data is written to memory, see Section 8.3.1, “FIFO Input and Output Block” on page 101.

Data that is received from dynamic range extended inputs are aligned in memory where the later of their two component input channel sources would be. As a result, data received from IDRX0 will be placed where the data would have been received by ADC1. Similarly, data received from IDRX1 will be placed where the data would have been received by ADC3.

IMPORTANT: The Ezairo 7100 system does not support inputs from both the input channels and the PCM interface being stored to the same FIFO. Because of this, users should always store data from the input channels and the PCM interface to different FIFOs.

The relative timing of the input channels and the input PCM data is undefined if the IOC_OUTPUT_CFG_PCM_SYNC_ADCDAC bit from the D_IOC_OUTPUT_CFG register is not set. The relative timing of reads from these input sources can be guaranteed if:

- The IOC_OUTPUT_CFG_PCM_SYNC_ADCDAC bit is set
- The PCM and ADC data rates are synchronized (typically by using SLOWCLK, USRCLK or SYSCLK as the clock source for the PCM interface), or the PCM clock is used as the input reference clock to the PLL

For this configuration data is written to FIFOs from (in order) ADC0, ADC2, ADC1/IDRX0, ADC3/IDRX1, PCM0 and PCM1.

IMPORTANT: Data synchronization between the ADCs and the PCM interface is only possible if these interfaces are using synchronous clock sources. Therefore, if clocking the PCM interface externally, the ADCs and system should be clocked externally by this same clock source. For all other configurations processing of ADC and PCM input data should be triggered on separate FIFO interrupts or events.

9.4.1 Control and Configuration Registers

Register Name	Register Description	Address
D_IOC_INPUT_ADC_CFG	Configuration register for ADC input	P:0xF0C0
D_IOC_INPUT_PCM_CFG	Configuration register for PCM input	P:0xF0C1
D_IOC_FIFO_ACCESS_CFG	Configuration register for IOC FIFO access types	P:0xF0C2
D_IOC_FIFO_SHIFT_CFG	Configuration register for data alignment when moving data between the FIFOs and audio inputs/outputs.	P:0xF0C3
D_ADC0_DATA	Latest audio sample from input channel 0	P:0xF0D0
D_ADC1_DATA	Latest audio sample from input channel 1	P:0xF0D1
D_ADC2_DATA	Latest audio sample from input channel 2	P:0xF0D2
D_ADC3_DATA	Latest audio sample from input channel 3	P:0xF0D3

9.4.1.1 D_IOC_INPUT_ADC_CFG Settings

Bit Field	Field Name	Description
29:24	ADC3_STORE	Select where to store the ADC3 input
21:16	ADC2_STORE	Select where to store the ADC2 input
13:8	ADC1_STORE	Select where to store the ADC1 input
5:0	ADC0_STORE	Select where to store the ADC0 input

Field Name	Value Symbol	Value Description	Hex Value
ADC3_STORE	IOC_INPUT_CFG_ADC3_FA0_0	Store ADC3 in FIFO A0_0	0x0
	IOC_INPUT_CFG_ADC3_FA0_1	Store ADC3 in FIFO A0_1	0x1
	IOC_INPUT_CFG_ADC3_FA0_2	Store ADC3 in FIFO A0_2	0x2
	IOC_INPUT_CFG_ADC3_FA0_3	Store ADC3 in FIFO A0_3	0x3
	IOC_INPUT_CFG_ADC3_FA0_4	Store ADC3 in FIFO A0_4	0x4
	IOC_INPUT_CFG_ADC3_FA0_5	Store ADC3 in FIFO A0_5	0x5
	IOC_INPUT_CFG_ADC3_FA0_6	Store ADC3 in FIFO A0_6	0x6
	IOC_INPUT_CFG_ADC3_FA0_7	Store ADC3 in FIFO A0_7	0x7
	IOC_INPUT_CFG_ADC3_FA1_0	Store ADC3 in FIFO A1_0	0x8
	IOC_INPUT_CFG_ADC3_FA1_1	Store ADC3 in FIFO A1_1	0x9
	IOC_INPUT_CFG_ADC3_FA1_2	Store ADC3 in FIFO A1_2	0xA
	IOC_INPUT_CFG_ADC3_FA1_3	Store ADC3 in FIFO A1_3	0xB
	IOC_INPUT_CFG_ADC3_FA1_4	Store ADC3 in FIFO A1_4	0xC
	IOC_INPUT_CFG_ADC3_FA1_5	Store ADC3 in FIFO A1_5	0xD
	IOC_INPUT_CFG_ADC3_FA1_6	Store ADC3 in FIFO A1_6	0xE
	IOC_INPUT_CFG_ADC3_FA1_7	Store ADC3 in FIFO A1_7	0xF
	IOC_INPUT_CFG_ADC3_FB0_0	Store ADC3 in FIFO B0_0	0x10
	IOC_INPUT_CFG_ADC3_FB0_1	Store ADC3 in FIFO B0_1	0x11
	IOC_INPUT_CFG_ADC3_FB0_2	Store ADC3 in FIFO B0_2	0x12
	IOC_INPUT_CFG_ADC3_FB0_3	Store ADC3 in FIFO B0_3	0x13
	IOC_INPUT_CFG_ADC3_FB0_4	Store ADC3 in FIFO B0_4	0x14
	IOC_INPUT_CFG_ADC3_FB0_5	Store ADC3 in FIFO B0_5	0x15
	IOC_INPUT_CFG_ADC3_FB0_6	Store ADC3 in FIFO B0_6	0x16
	IOC_INPUT_CFG_ADC3_FB0_7	Store ADC3 in FIFO B0_7	0x17
	IOC_INPUT_CFG_ADC3_FB1_0	Store ADC3 in FIFO B1_0	0x18
	IOC_INPUT_CFG_ADC3_FB1_1	Store ADC3 in FIFO B1_1	0x19
	IOC_INPUT_CFG_ADC3_FB1_2	Store ADC3 in FIFO B1_2	0x1A
	IOC_INPUT_CFG_ADC3_FB1_3	Store ADC3 in FIFO B1_3	0x1B
	IOC_INPUT_CFG_ADC3_FB1_4	Store ADC3 in FIFO B1_4	0x1C
	IOC_INPUT_CFG_ADC3_FB1_5	Store ADC3 in FIFO B1_5	0x1D
	IOC_INPUT_CFG_ADC3_FB1_6	Store ADC3 in FIFO B1_6	0x1E
	IOC_INPUT_CFG_ADC3_FB1_7	Store ADC3 in FIFO B1_7	0x1F
	IOC_INPUT_CFG_ADC3_NONE	Not stored	0x3F*

Field Name	Value Symbol	Value Description	Hex Value
ADC2_STORE	IOC_INPUT_CFG_ADC2_FA0_0	Store ADC2 in FIFO A0_0	0x0
	IOC_INPUT_CFG_ADC2_FA0_1	Store ADC2 in FIFO A0_1	0x1
	IOC_INPUT_CFG_ADC2_FA0_2	Store ADC2 in FIFO A0_2	0x2
	IOC_INPUT_CFG_ADC2_FA0_3	Store ADC2 in FIFO A0_3	0x3
	IOC_INPUT_CFG_ADC2_FA0_4	Store ADC2 in FIFO A0_4	0x4
	IOC_INPUT_CFG_ADC2_FA0_5	Store ADC2 in FIFO A0_5	0x5
	IOC_INPUT_CFG_ADC2_FA0_6	Store ADC2 in FIFO A0_6	0x6
	IOC_INPUT_CFG_ADC2_FA0_7	Store ADC2 in FIFO A0_7	0x7
	IOC_INPUT_CFG_ADC2_FA1_0	Store ADC2 in FIFO A1_0	0x8
	IOC_INPUT_CFG_ADC2_FA1_1	Store ADC2 in FIFO A1_1	0x9
	IOC_INPUT_CFG_ADC2_FA1_2	Store ADC2 in FIFO A1_2	0xA
	IOC_INPUT_CFG_ADC2_FA1_3	Store ADC2 in FIFO A1_3	0xB
	IOC_INPUT_CFG_ADC2_FA1_4	Store ADC2 in FIFO A1_4	0xC
	IOC_INPUT_CFG_ADC2_FA1_5	Store ADC2 in FIFO A1_5	0xD
	IOC_INPUT_CFG_ADC2_FA1_6	Store ADC2 in FIFO A1_6	0xE
	IOC_INPUT_CFG_ADC2_FA1_7	Store ADC2 in FIFO A1_7	0xF
	IOC_INPUT_CFG_ADC2_FB0_0	Store ADC2 in FIFO B0_0	0x10
	IOC_INPUT_CFG_ADC2_FB0_1	Store ADC2 in FIFO B0_1	0x11
	IOC_INPUT_CFG_ADC2_FB0_2	Store ADC2 in FIFO B0_2	0x12
	IOC_INPUT_CFG_ADC2_FB0_3	Store ADC2 in FIFO B0_3	0x13
	IOC_INPUT_CFG_ADC2_FB0_4	Store ADC2 in FIFO B0_4	0x14
	IOC_INPUT_CFG_ADC2_FB0_5	Store ADC2 in FIFO B0_5	0x15
	IOC_INPUT_CFG_ADC2_FB0_6	Store ADC2 in FIFO B0_6	0x16
	IOC_INPUT_CFG_ADC2_FB0_7	Store ADC2 in FIFO B0_7	0x17
	IOC_INPUT_CFG_ADC2_FB1_0	Store ADC2 in FIFO B1_0	0x18
	IOC_INPUT_CFG_ADC2_FB1_1	Store ADC2 in FIFO B1_1	0x19
	IOC_INPUT_CFG_ADC2_FB1_2	Store ADC2 in FIFO B1_2	0x1A
	IOC_INPUT_CFG_ADC2_FB1_3	Store ADC2 in FIFO B1_3	0x1B
	IOC_INPUT_CFG_ADC2_FB1_4	Store ADC2 in FIFO B1_4	0x1C
	IOC_INPUT_CFG_ADC2_FB1_5	Store ADC2 in FIFO B1_5	0x1D
	IOC_INPUT_CFG_ADC2_FB1_6	Store ADC2 in FIFO B1_6	0x1E
	IOC_INPUT_CFG_ADC2_FB1_7	Store ADC2 in FIFO B1_7	0x1F
	IOC_INPUT_CFG_ADC2_NONE	Not stored	0x3F*

Field Name	Value Symbol	Value Description	Hex Value
ADC1_STORE	IOC_INPUT_CFG_ADC1_FA0_0	Store ADC1 in FIFO A0_0	0x0
	IOC_INPUT_CFG_ADC1_FA0_1	Store ADC1 in FIFO A0_1	0x1
	IOC_INPUT_CFG_ADC1_FA0_2	Store ADC1 in FIFO A0_2	0x2
	IOC_INPUT_CFG_ADC1_FA0_3	Store ADC1 in FIFO A0_3	0x3
	IOC_INPUT_CFG_ADC1_FA0_4	Store ADC1 in FIFO A0_4	0x4
	IOC_INPUT_CFG_ADC1_FA0_5	Store ADC1 in FIFO A0_5	0x5
	IOC_INPUT_CFG_ADC1_FA0_6	Store ADC1 in FIFO A0_6	0x6
	IOC_INPUT_CFG_ADC1_FA0_7	Store ADC1 in FIFO A0_7	0x7
	IOC_INPUT_CFG_ADC1_FA1_0	Store ADC1 in FIFO A1_0	0x8
	IOC_INPUT_CFG_ADC1_FA1_1	Store ADC1 in FIFO A1_1	0x9
	IOC_INPUT_CFG_ADC1_FA1_2	Store ADC1 in FIFO A1_2	0xA
	IOC_INPUT_CFG_ADC1_FA1_3	Store ADC1 in FIFO A1_3	0xB
	IOC_INPUT_CFG_ADC1_FA1_4	Store ADC1 in FIFO A1_4	0xC
	IOC_INPUT_CFG_ADC1_FA1_5	Store ADC1 in FIFO A1_5	0xD
	IOC_INPUT_CFG_ADC1_FA1_6	Store ADC1 in FIFO A1_6	0xE
	IOC_INPUT_CFG_ADC1_FA1_7	Store ADC1 in FIFO A1_7	0xF
	IOC_INPUT_CFG_ADC1_FB0_0	Store ADC1 in FIFO B0_0	0x10
	IOC_INPUT_CFG_ADC1_FB0_1	Store ADC1 in FIFO B0_1	0x11
	IOC_INPUT_CFG_ADC1_FB0_2	Store ADC1 in FIFO B0_2	0x12
	IOC_INPUT_CFG_ADC1_FB0_3	Store ADC1 in FIFO B0_3	0x13
	IOC_INPUT_CFG_ADC1_FB0_4	Store ADC1 in FIFO B0_4	0x14
	IOC_INPUT_CFG_ADC1_FB0_5	Store ADC1 in FIFO B0_5	0x15
	IOC_INPUT_CFG_ADC1_FB0_6	Store ADC1 in FIFO B0_6	0x16
	IOC_INPUT_CFG_ADC1_FB0_7	Store ADC1 in FIFO B0_7	0x17
	IOC_INPUT_CFG_ADC1_FB1_0	Store ADC1 in FIFO B1_0	0x18
	IOC_INPUT_CFG_ADC1_FB1_1	Store ADC1 in FIFO B1_1	0x19
	IOC_INPUT_CFG_ADC1_FB1_2	Store ADC1 in FIFO B1_2	0x1A
	IOC_INPUT_CFG_ADC1_FB1_3	Store ADC1 in FIFO B1_3	0x1B
	IOC_INPUT_CFG_ADC1_FB1_4	Store ADC1 in FIFO B1_4	0x1C
	IOC_INPUT_CFG_ADC1_FB1_5	Store ADC1 in FIFO B1_5	0x1D
	IOC_INPUT_CFG_ADC1_FB1_6	Store ADC1 in FIFO B1_6	0x1E
	IOC_INPUT_CFG_ADC1_FB1_7	Store ADC1 in FIFO B1_7	0x1F
	IOC_INPUT_CFG_ADC1_NONE	Not stored	0x3F*

Field Name	Value Symbol	Value Description	Hex Value
ADC0_STORE	IOC_INPUT_CFG_ADC0_FA0_0	Store ADC0 in FIFO A0_0	0x0
	IOC_INPUT_CFG_ADC0_FA0_1	Store ADC0 in FIFO A0_1	0x1
	IOC_INPUT_CFG_ADC0_FA0_2	Store ADC0 in FIFO A0_2	0x2
	IOC_INPUT_CFG_ADC0_FA0_3	Store ADC0 in FIFO A0_3	0x3
	IOC_INPUT_CFG_ADC0_FA0_4	Store ADC0 in FIFO A0_4	0x4
	IOC_INPUT_CFG_ADC0_FA0_5	Store ADC0 in FIFO A0_5	0x5
	IOC_INPUT_CFG_ADC0_FA0_6	Store ADC0 in FIFO A0_6	0x6
	IOC_INPUT_CFG_ADC0_FA0_7	Store ADC0 in FIFO A0_7	0x7
	IOC_INPUT_CFG_ADC0_FA1_0	Store ADC0 in FIFO A1_0	0x8
	IOC_INPUT_CFG_ADC0_FA1_1	Store ADC0 in FIFO A1_1	0x9
	IOC_INPUT_CFG_ADC0_FA1_2	Store ADC0 in FIFO A1_2	0xA
	IOC_INPUT_CFG_ADC0_FA1_3	Store ADC0 in FIFO A1_3	0xB
	IOC_INPUT_CFG_ADC0_FA1_4	Store ADC0 in FIFO A1_4	0xC
	IOC_INPUT_CFG_ADC0_FA1_5	Store ADC0 in FIFO A1_5	0xD
	IOC_INPUT_CFG_ADC0_FA1_6	Store ADC0 in FIFO A1_6	0xE
	IOC_INPUT_CFG_ADC0_FA1_7	Store ADC0 in FIFO A1_7	0xF
	IOC_INPUT_CFG_ADC0_FB0_0	Store ADC0 in FIFO B0_0	0x10
	IOC_INPUT_CFG_ADC0_FB0_1	Store ADC0 in FIFO B0_1	0x11
	IOC_INPUT_CFG_ADC0_FB0_2	Store ADC0 in FIFO B0_2	0x12
	IOC_INPUT_CFG_ADC0_FB0_3	Store ADC0 in FIFO B0_3	0x13
	IOC_INPUT_CFG_ADC0_FB0_4	Store ADC0 in FIFO B0_4	0x14
	IOC_INPUT_CFG_ADC0_FB0_5	Store ADC0 in FIFO B0_5	0x15
	IOC_INPUT_CFG_ADC0_FB0_6	Store ADC0 in FIFO B0_6	0x16
	IOC_INPUT_CFG_ADC0_FB0_7	Store ADC0 in FIFO B0_7	0x17
	IOC_INPUT_CFG_ADC0_FB1_0	Store ADC0 in FIFO B1_0	0x18
	IOC_INPUT_CFG_ADC0_FB1_1	Store ADC0 in FIFO B1_1	0x19
	IOC_INPUT_CFG_ADC0_FB1_2	Store ADC0 in FIFO B1_2	0x1A
	IOC_INPUT_CFG_ADC0_FB1_3	Store ADC0 in FIFO B1_3	0x1B
	IOC_INPUT_CFG_ADC0_FB1_4	Store ADC0 in FIFO B1_4	0x1C
	IOC_INPUT_CFG_ADC0_FB1_5	Store ADC0 in FIFO B1_5	0x1D
	IOC_INPUT_CFG_ADC0_FB1_6	Store ADC0 in FIFO B1_6	0x1E
	IOC_INPUT_CFG_ADC0_FB1_7	Store ADC0 in FIFO B1_7	0x1F
	IOC_INPUT_CFG_ADC0_NONE	Not stored	0x3F*

9.4.1.2 D_IOC_INPUT_PCM_CFG Settings

Bit Field	Field Name	Description
13:8	PCM1_STORE	Select where to store PCM 1 input
5:0	PCM0_STORE	Select where to store PCM 0 input

Field Name	Value Symbol	Value Description	Hex Value
PCM1_STORE	IOC_INPUT_CFG_PCM1_FA0_0	Store PCM1 in FIFO A0_0	0x0
	IOC_INPUT_CFG_PCM1_FA0_1	Store PCM1 in FIFO A0_1	0x1
	IOC_INPUT_CFG_PCM1_FA0_2	Store PCM1 in FIFO A0_2	0x2
	IOC_INPUT_CFG_PCM1_FA0_3	Store PCM1 in FIFO A0_3	0x3
	IOC_INPUT_CFG_PCM1_FA0_4	Store PCM1 in FIFO A0_4	0x4
	IOC_INPUT_CFG_PCM1_FA0_5	Store PCM1 in FIFO A0_5	0x5
	IOC_INPUT_CFG_PCM1_FA0_6	Store PCM1 in FIFO A0_6	0x6
	IOC_INPUT_CFG_PCM1_FA0_7	Store PCM1 in FIFO A0_7	0x7
	IOC_INPUT_CFG_PCM1_FA1_0	Store PCM1 in FIFO A1_0	0x8
	IOC_INPUT_CFG_PCM1_FA1_1	Store PCM1 in FIFO A1_1	0x9
	IOC_INPUT_CFG_PCM1_FA1_2	Store PCM1 in FIFO A1_2	0xA
	IOC_INPUT_CFG_PCM1_FA1_3	Store PCM1 in FIFO A1_3	0xB
	IOC_INPUT_CFG_PCM1_FA1_4	Store PCM1 in FIFO A1_4	0xC
	IOC_INPUT_CFG_PCM1_FA1_5	Store PCM1 in FIFO A1_5	0xD
	IOC_INPUT_CFG_PCM1_FA1_6	Store PCM1 in FIFO A1_6	0xE
	IOC_INPUT_CFG_PCM1_FA1_7	Store PCM1 in FIFO A1_7	0xF
	IOC_INPUT_CFG_PCM1_FB0_0	Store PCM1 in FIFO B0_0	0x10
	IOC_INPUT_CFG_PCM1_FB0_1	Store PCM1 in FIFO B0_1	0x11
	IOC_INPUT_CFG_PCM1_FB0_2	Store PCM1 in FIFO B0_2	0x12
	IOC_INPUT_CFG_PCM1_FB0_3	Store PCM1 in FIFO B0_3	0x13
	IOC_INPUT_CFG_PCM1_FB0_4	Store PCM1 in FIFO B0_4	0x14
	IOC_INPUT_CFG_PCM1_FB0_5	Store PCM1 in FIFO B0_5	0x15
	IOC_INPUT_CFG_PCM1_FB0_6	Store PCM1 in FIFO B0_6	0x16
	IOC_INPUT_CFG_PCM1_FB0_7	Store PCM1 in FIFO B0_7	0x17
	IOC_INPUT_CFG_PCM1_FB1_0	Store PCM1 in FIFO B1_0	0x18
	IOC_INPUT_CFG_PCM1_FB1_1	Store PCM1 in FIFO B1_1	0x19
	IOC_INPUT_CFG_PCM1_FB1_2	Store PCM1 in FIFO B1_2	0x1A
	IOC_INPUT_CFG_PCM1_FB1_3	Store PCM1 in FIFO B1_3	0x1B
	IOC_INPUT_CFG_PCM1_FB1_4	Store PCM1 in FIFO B1_4	0x1C
	IOC_INPUT_CFG_PCM1_FB1_5	Store PCM1 in FIFO B1_5	0x1D
	IOC_INPUT_CFG_PCM1_FB1_6	Store PCM1 in FIFO B1_6	0x1E
	IOC_INPUT_CFG_PCM1_FB1_7	Store PCM1 in FIFO B1_7	0x1F
	IOC_INPUT_CFG_PCM1_NONE	Not stored	0x3F*

Field Name	Value Symbol	Value Description	Hex Value
PCM0_STORE	IOC_INPUT_CFG_PCM0_FA0_0	Store PCM0 in FIFO A0_0	0x0
	IOC_INPUT_CFG_PCM0_FA0_1	Store PCM0 in FIFO A0_1	0x1
	IOC_INPUT_CFG_PCM0_FA0_2	Store PCM0 in FIFO A0_2	0x2
	IOC_INPUT_CFG_PCM0_FA0_3	Store PCM0 in FIFO A0_3	0x3
	IOC_INPUT_CFG_PCM0_FA0_4	Store PCM0 in FIFO A0_4	0x4
	IOC_INPUT_CFG_PCM0_FA0_5	Store PCM0 in FIFO A0_5	0x5
	IOC_INPUT_CFG_PCM0_FA0_6	Store PCM0 in FIFO A0_6	0x6
	IOC_INPUT_CFG_PCM0_FA0_7	Store PCM0 in FIFO A0_7	0x7
	IOC_INPUT_CFG_PCM0_FA1_0	Store PCM0 in FIFO A1_0	0x8
	IOC_INPUT_CFG_PCM0_FA1_1	Store PCM0 in FIFO A1_1	0x9
	IOC_INPUT_CFG_PCM0_FA1_2	Store PCM0 in FIFO A1_2	0xA
	IOC_INPUT_CFG_PCM0_FA1_3	Store PCM0 in FIFO A1_3	0xB
	IOC_INPUT_CFG_PCM0_FA1_4	Store PCM0 in FIFO A1_4	0xC
	IOC_INPUT_CFG_PCM0_FA1_5	Store PCM0 in FIFO A1_5	0xD
	IOC_INPUT_CFG_PCM0_FA1_6	Store PCM0 in FIFO A1_6	0xE
	IOC_INPUT_CFG_PCM0_FA1_7	Store PCM0 in FIFO A1_7	0xF
	IOC_INPUT_CFG_PCM0_FB0_0	Store PCM0 in FIFO B0_0	0x10
	IOC_INPUT_CFG_PCM0_FB0_1	Store PCM0 in FIFO B0_1	0x11
	IOC_INPUT_CFG_PCM0_FB0_2	Store PCM0 in FIFO B0_2	0x12
	IOC_INPUT_CFG_PCM0_FB0_3	Store PCM0 in FIFO B0_3	0x13
	IOC_INPUT_CFG_PCM0_FB0_4	Store PCM0 in FIFO B0_4	0x14
	IOC_INPUT_CFG_PCM0_FB0_5	Store PCM0 in FIFO B0_5	0x15
	IOC_INPUT_CFG_PCM0_FB0_6	Store PCM0 in FIFO B0_6	0x16
	IOC_INPUT_CFG_PCM0_FB0_7	Store PCM0 in FIFO B0_7	0x17
	IOC_INPUT_CFG_PCM0_FB1_0	Store PCM0 in FIFO B1_0	0x18
	IOC_INPUT_CFG_PCM0_FB1_1	Store PCM0 in FIFO B1_1	0x19
	IOC_INPUT_CFG_PCM0_FB1_2	Store PCM0 in FIFO B1_2	0x1A
	IOC_INPUT_CFG_PCM0_FB1_3	Store PCM0 in FIFO B1_3	0x1B
	IOC_INPUT_CFG_PCM0_FB1_4	Store PCM0 in FIFO B1_4	0x1C
	IOC_INPUT_CFG_PCM0_FB1_5	Store PCM0 in FIFO B1_5	0x1D
	IOC_INPUT_CFG_PCM0_FB1_6	Store PCM0 in FIFO B1_6	0x1E
	IOC_INPUT_CFG_PCM0_FB1_7	Store PCM0 in FIFO B1_7	0x1F
	IOC_INPUT_CFG_PCM0_NONE	Not stored	0x3F*

9.4.1.3 D_IOC_FIFO_ACCESS_CFG Settings

Bit Field	Field Name	Description
31	FIFO_B1_7_DBL_ACC	Defines FIFO B1_7 access type
30	FIFO_B1_6_DBL_ACC	Defines FIFO B1_6 access type

Bit Field	Field Name	Description
29	FIFO_B1_5_DBL_ACC	Defines FIFO B1_5 access type
28	FIFO_B1_4_DBL_ACC	Defines FIFO B1_4 access type
27	FIFO_B1_3_DBL_ACC	Defines FIFO B1_3 access type
26	FIFO_B1_2_DBL_ACC	Defines FIFO B1_2 access type
25	FIFO_B1_1_DBL_ACC	Defines FIFO B1_1 access type
24	FIFO_B1_0_DBL_ACC	Defines FIFO B1_0 access type
23	FIFO_B0_7_DBL_ACC	Defines FIFO B0_7 access type
22	FIFO_B0_6_DBL_ACC	Defines FIFO B0_6 access type
21	FIFO_B0_5_DBL_ACC	Defines FIFO B0_5 access type
20	FIFO_B0_4_DBL_ACC	Defines FIFO B0_4 access type
19	FIFO_B0_3_DBL_ACC	Defines FIFO B0_3 access type
18	FIFO_B0_2_DBL_ACC	Defines FIFO B0_2 access type
17	FIFO_B0_1_DBL_ACC	Defines FIFO B0_1 access type
16	FIFO_B0_0_DBL_ACC	Defines FIFO B0_0 access type
15	FIFO_A1_7_DBL_ACC	Defines FIFO A1_7 access type
14	FIFO_A1_6_DBL_ACC	Defines FIFO A1_6 access type
13	FIFO_A1_5_DBL_ACC	Defines FIFO A1_5 access type
12	FIFO_A1_4_DBL_ACC	Defines FIFO A1_4 access type
11	FIFO_A1_3_DBL_ACC	Defines FIFO A1_3 access type
10	FIFO_A1_2_DBL_ACC	Defines FIFO A1_2 access type
9	FIFO_A1_1_DBL_ACC	Defines FIFO A1_1 access type
8	FIFO_A1_0_DBL_ACC	Defines FIFO A1_0 access type
7	FIFO_A0_7_DBL_ACC	Defines FIFO A0_7 access type
6	FIFO_A0_6_DBL_ACC	Defines FIFO A0_6 access type
5	FIFO_A0_5_DBL_ACC	Defines FIFO A0_5 access type
4	FIFO_A0_4_DBL_ACC	Defines FIFO A0_4 access type
3	FIFO_A0_3_DBL_ACC	Defines FIFO A0_3 access type
2	FIFO_A0_2_DBL_ACC	Defines FIFO A0_2 access type
1	FIFO_A0_1_DBL_ACC	Defines FIFO A0_1 access type
0	FIFO_A0_0_DBL_ACC	Defines FIFO A0_0 access type

Field Name	Value Symbol	Value Description	Hex Value
FIFO_B1_7_DBL_ACC	FIFO_B1_7_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B1_7_DBL_ACC_EN	Double access	0x1
FIFO_B1_6_DBL_ACC	FIFO_B1_6_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B1_6_DBL_ACC_EN	Double access	0x1
FIFO_B1_5_DBL_ACC	FIFO_B1_5_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B1_5_DBL_ACC_EN	Double access	0x1

Field Name	Value Symbol	Value Description	Hex Value
FIFO_B1_4_DBL_ACC	FIFO_B1_4_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B1_4_DBL_ACC_EN	Double access	0x1
FIFO_B1_3_DBL_ACC	FIFO_B1_3_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B1_3_DBL_ACC_EN	Double access	0x1
FIFO_B1_2_DBL_ACC	FIFO_B1_2_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B1_2_DBL_ACC_EN	Double access	0x1
FIFO_B1_1_DBL_ACC	FIFO_B1_1_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B1_1_DBL_ACC_EN	Double access	0x1
FIFO_B1_0_DBL_ACC	FIFO_B1_0_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B1_0_DBL_ACC_EN	Double access	0x1
FIFO_B0_7_DBL_ACC	FIFO_B0_7_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B0_7_DBL_ACC_EN	Double access	0x1
FIFO_B0_6_DBL_ACC	FIFO_B0_6_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B0_6_DBL_ACC_EN	Double access	0x1
FIFO_B0_5_DBL_ACC	FIFO_B0_5_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B0_5_DBL_ACC_EN	Double access	0x1
FIFO_B0_4_DBL_ACC	FIFO_B0_4_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B0_4_DBL_ACC_EN	Double access	0x1
FIFO_B0_3_DBL_ACC	FIFO_B0_3_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B0_3_DBL_ACC_EN	Double access	0x1
FIFO_B0_2_DBL_ACC	FIFO_B0_2_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B0_2_DBL_ACC_EN	Double access	0x1
FIFO_B0_1_DBL_ACC	FIFO_B0_1_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B0_1_DBL_ACC_EN	Double access	0x1
FIFO_B0_0_DBL_ACC	FIFO_B0_0_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B0_0_DBL_ACC_EN	Double access	0x1
FIFO_A1_7_DBL_ACC	FIFO_A1_7_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A1_7_DBL_ACC_EN	Double access	0x1
FIFO_A1_6_DBL_ACC	FIFO_A1_6_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A1_6_DBL_ACC_EN	Double access	0x1
FIFO_A1_5_DBL_ACC	FIFO_A1_5_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A1_5_DBL_ACC_EN	Double access	0x1
FIFO_A1_4_DBL_ACC	FIFO_A1_4_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A1_4_DBL_ACC_EN	Double access	0x1
FIFO_A1_3_DBL_ACC	FIFO_A1_3_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A1_3_DBL_ACC_EN	Double access	0x1
FIFO_A1_2_DBL_ACC	FIFO_A1_2_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A1_2_DBL_ACC_EN	Double access	0x1

Field Name	Value Symbol	Value Description	Hex Value
FIFO_A1_1_DBL_ACC	FIFO_A1_1_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A1_1_DBL_ACC_EN	Double access	0x1
FIFO_A1_0_DBL_ACC	FIFO_A1_0_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A1_0_DBL_ACC_EN	Double access	0x1
FIFO_A0_7_DBL_ACC	FIFO_A0_7_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A0_7_DBL_ACC_EN	Double access	0x1
FIFO_A0_6_DBL_ACC	FIFO_A0_6_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A0_6_DBL_ACC_EN	Double access	0x1
FIFO_A0_5_DBL_ACC	FIFO_A0_5_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A0_5_DBL_ACC_EN	Double access	0x1
FIFO_A0_4_DBL_ACC	FIFO_A0_4_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A0_4_DBL_ACC_EN	Double access	0x1
FIFO_A0_3_DBL_ACC	FIFO_A0_3_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A0_3_DBL_ACC_EN	Double access	0x1
FIFO_A0_2_DBL_ACC	FIFO_A0_2_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A0_2_DBL_ACC_EN	Double access	0x1
FIFO_A0_1_DBL_ACC	FIFO_A0_1_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A0_1_DBL_ACC_EN	Double access	0x1
FIFO_A0_0_DBL_ACC	FIFO_A0_0_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A0_0_DBL_ACC_EN	Double access	0x1

9.4.1.4 D_IOC_FIFO_SHIFT_CFG Settings

Bit Field	Field Name	Description
5	DAC1_SHIFT	Configure the data alignment for DAC1
4	DAC0_SHIFT	Configure the data alignment for DAC0
3	ADC3_SHIFT	Configure the data alignment for ADC3
2	ADC2_SHIFT	Configure the data alignment for ADC2
1	ADC1_SHIFT	Configure the data alignment for ADC1
0	ADC0_SHIFT	Configure the data alignment for ADC0

Field Name	Value Symbol	Value Description	Hex Value
DAC1_SHIFT	IOC_FIFO_DAC1_SHIFT_LEFT	MSB aligned	0x0*
	IOC_FIFO_DAC1_SHIFT_RIGHT	LSB aligned for Interpolation filter + saturation	0x1
DAC0_SHIFT	IOC_FIFO_DAC0_SHIFT_LEFT	MSB aligned	0x0*
	IOC_FIFO_DAC0_SHIFT_RIGHT	LSB aligned for Interpolation filter + saturation	0x1
ADC3_SHIFT	IOC_FIFO_ADC3_SHIFT_LEFT	MSB aligned	0x0*
	IOC_FIFO_ADC3_SHIFT_RIGHT	ADC without DRX LSB aligned + sign extension	0x1

Field Name	Value Symbol	Value Description	Hex Value
ADC2_SHIFT	IOC_FIFO_ADC2_SHIFT_LEFT	MSB aligned	0x0*
	IOC_FIFO_ADC2_SHIFT_RIGHT	ADC without DRX LSB aligned + sign extension	0x1
ADC1_SHIFT	IOC_FIFO_ADC1_SHIFT_LEFT	MSB aligned	0x0*
	IOC_FIFO_ADC1_SHIFT_RIGHT	ADC without DRX LSB aligned + sign extension	0x1
ADC0_SHIFT	IOC_FIFO_ADC0_SHIFT_LEFT	MSB aligned	0x0*
	IOC_FIFO_ADC0_SHIFT_RIGHT	ADC without DRX LSB aligned + sign extension	0x1

9.4.1.5 D_ADC0_DATA Settings

Bit Field	Field Name	Description
31:0	ADC0_DATA	Latest audio sample from ADC0 (m1p23 format, sign extended to 32 bits)

9.4.1.6 D_ADC1_DATA Settings

Bit Field	Field Name	Description
31:0	ADC1_DATA	Latest audio sample from ADC1 (m1p23 format, sign extended to 32 bits)

9.4.1.7 D_ADC2_DATA Settings

Bit Field	Field Name	Description
31:0	ADC2_DATA	Latest audio sample from ADC2 (m1p23 format, sign extended to 32 bits)

9.4.1.8 D_ADC3_DATA Settings

Bit Field	Field Name	Description
31:0	ADC3_DATA	Latest audio sample from ADC3 (m1p23 format, sign extended to 32 bits)

9.5 IOC Output

On the output side, the IOC is attached to the two output channels from the output stage and the Filter Engine, denoted as DAC0 and DAC1 in constants and documentation.

- The interpolation filter inputs for each of the output stage channels are interpreted as 18-bit input data. This 18-bit data can be aligned to the lower or upper 18 bits of the 24-bit input word by setting or clearing the appropriate IOC_FIFO_SHIFT_CFG_DAC*_SHIFT bit in the D_IOC_FIFO_SHIFT_CFG register. If aligned to the lower 18 bits, the data is saturated if it exceeds the 18-bit range.
- Data provided to the Filter Engine by the IOC is processed as 24-bit data.

For each of these output channels, the IOC provides an access register (D_DAC*_DATA) that can be used to supply data to be transferred to that output channel.

In addition to the output channels, the IOC can be attached to up to two output channels from the PCM interface. Denoted as PCM0 and PCM1 in constants and documentation, these channels are nominally considered as 24-bit output data. The PCM interface configuration can be used to align data that is shorter than 24 bits.

The IOC (if configured correctly) can direct data from the FIFOs to each of the four output sources. Select the source FIFO for each output by configuring the IOC_OUTPUT_CFG_DAC0_LOAD, IOC_OUTPUT_CFG_DAC1_LOAD, IOC_OUTPUT_CFG_PCM0_LOAD and IOC_OUTPUT_CFG_PCM1_LOAD bit fields of the D_IOC_OUTPUT_CFG register. Each of these bit fields allows you to select the source of your output data from one of the 32 FIFOs, any of the six input sources, or no source. Additionally, the DAC0 and DAC1 configurations can be used to select the D_DAC*_DATA bypass data registers as their source.

IMPORTANT: Connecting IOC output sinks to FIFOs that have not been configured results in undefined behavior.

Data for more than one output channel can be read from the same FIFO. In this case the data written to the outputs is guaranteed to be read from the FIFO in the time sequence order DAC0, DAC1. Similarly, data for more than one PCM output channel can be loaded from the same FIFO. The data provided to the PCM interface is guaranteed to be in the order PCM0, PCM1. For details about how output FIFO data is read from memory, see Section 8.3.1, “FIFO Input and Output Block” on page 101.

IMPORTANT: The Ezairo 7100 system does not support storing outputs to both the output channels and the PCM interface to the same FIFO. Because of this, always use different FIFOs to provide data to the output channels and the PCM interface.

The relative timing of the output channels and the input PCM data is undefined if the IOC_OUTPUT_CFG_PCM_SYNC_ADCDAC bit from the D_IOC_OUTPUT_CFG register is not set. The relative timing of reads from the FIFOs for these output sources can be guaranteed if:

- The IOC_OUTPUT_CFG_PCM_SYNC_ADCDAC bit is set
- The PCM and DAC data rates are synchronized (typically by using SDMCLK, USRCLK or SYSCLK as the clock source for the PCM interface), or the PCM clock is used as the input reference clock to the PLL

For this configuration data is read from FIFOs to (in order) DAC0, DAC1, PCM0 and PCM1.

IMPORTANT: Data synchronization between the DACs and the PCM interface is only possible if these interfaces are using synchronous clock sources. Therefore, if the PCM interface is clocked externally, use the same clock to supply the DACs and system through the EXTCLK input. For all other configurations, handle processing of DAC and PCM output data separately.

NOTE: Writing the D_IOC_OUTPUT_CFG or the D_IOC_FIFO_ACCESS_CFG register resets the output state machine of the IOC.

9.5.1 Automatic FIFO Mute

To prevent the IOC from repeatedly reading the same set of data from a FIFO, use the automatic mute feature. This feature can be enabled for a given FIFO by setting the AUTO_MUTE_ENABLE bit in the appropriate D_FIFO_x_y_AUTO_MUTE register.

When the automatic mute feature is enabled for a FIFO and the IOC has read all of the data in the defined FIFO memory since the FIFO was last written, data read by the IOC is forced to a value of 0x000000. The IOC output data returns to normal operation whenever a write is detected or the auto mute feature is disabled.

9.5.2 Control and Configuration Registers

Register Name	Register Description	Address
D_IOC_OUTPUT_CFG	Configuration registers for IOC to output stage	P:0xF0C4
D_DAC0_DATA	Input sample for output channel 0	P:0xF0D4
D_DAC1_DATA	Input sample for output channel 1	P:0xF0D5
D_FIFO_A0_0_AUTO_MUTE	Auto mute control of FIFO A0_0	X:0x7003
D_FIFO_A0_1_AUTO_MUTE	Auto mute control of FIFO A0_1	X:0x7013
D_FIFO_A0_2_AUTO_MUTE	Auto mute control of FIFO A0_2	X:0x7023
D_FIFO_A0_3_AUTO_MUTE	Auto mute control of FIFO A0_3	X:0x7033
D_FIFO_A0_4_AUTO_MUTE	Auto mute control of FIFO A0_4	X:0x7043
D_FIFO_A0_5_AUTO_MUTE	Auto mute control of FIFO A0_5	X:0x7053
D_FIFO_A0_6_AUTO_MUTE	Auto mute control of FIFO A0_6	X:0x7063
D_FIFO_A0_7_AUTO_MUTE	Auto mute control of FIFO A0_7	X:0x7073
D_FIFO_A1_0_AUTO_MUTE	Auto mute control of FIFO A1_0	X:0x7083
D_FIFO_A1_1_AUTO_MUTE	Auto mute control of FIFO A1_1	X:0x7093
D_FIFO_A1_2_AUTO_MUTE	Auto mute control of FIFO A1_2	X:0x70A3
D_FIFO_A1_3_AUTO_MUTE	Auto mute control of FIFO A1_3	X:0x70B3
D_FIFO_A1_4_AUTO_MUTE	Auto mute control of FIFO A1_4	X:0x70C3
D_FIFO_A1_5_AUTO_MUTE	Auto mute control of FIFO A1_5	X:0x70D3
D_FIFO_A1_6_AUTO_MUTE	Auto mute control of FIFO A1_6	X:0x70E3
D_FIFO_A1_7_AUTO_MUTE	Auto mute control of FIFO A1_7	X:0x70F3
D_FIFO_B0_0_AUTO_MUTE	Auto mute control of FIFO B0_0	X:0x7103
D_FIFO_B0_1_AUTO_MUTE	Auto mute control of FIFO B0_1	X:0x7113
D_FIFO_B0_2_AUTO_MUTE	Auto mute control of FIFO B0_2	X:0x7123
D_FIFO_B0_3_AUTO_MUTE	Auto mute control of FIFO B0_3	X:0x7133
D_FIFO_B0_4_AUTO_MUTE	Auto mute control of FIFO B0_4	X:0x7143
D_FIFO_B0_5_AUTO_MUTE	Auto mute control of FIFO B0_5	X:0x7153
D_FIFO_B0_6_AUTO_MUTE	Auto mute control of FIFO B0_6	X:0x7163
D_FIFO_B0_7_AUTO_MUTE	Auto mute control of FIFO B0_7	X:0x7173
D_FIFO_B1_0_AUTO_MUTE	Auto mute control of FIFO B1_0	X:0x7183
D_FIFO_B1_1_AUTO_MUTE	Auto mute control of FIFO B1_1	X:0x7193
D_FIFO_B1_2_AUTO_MUTE	Auto mute control of FIFO B1_2	X:0x71A3
D_FIFO_B1_3_AUTO_MUTE	Auto mute control of FIFO B1_3	X:0x71B3
D_FIFO_B1_4_AUTO_MUTE	Auto mute control of FIFO B1_4	X:0x71C3
D_FIFO_B1_5_AUTO_MUTE	Auto mute control of FIFO B1_5	X:0x71D3
D_FIFO_B1_6_AUTO_MUTE	Auto mute control of FIFO B1_6	X:0x71E3
D_FIFO_B1_7_AUTO_MUTE	Auto mute control of FIFO B1_7	X:0x71F3

9.5.2.1 D_IOC_OUTPUT_CFG Settings

Bit Field	Field Name	Description
30	PCM_SYNC	Indicate that the PCM and ADC/DAC data share the same rate
29:24	PCM1_LOAD	Defines the origin of the audio sample for PCM channel 1
21:16	PCM0_LOAD	Defines the origin of the audio sample for PCM channel 0
13:8	DAC1_LOAD	Defines the origin of the audio sample for the DAC1 channel
5:0	DAC0_LOAD	Defines the origin of the audio sample for the DAC0 channel

Field Name	Value Symbol	Value Description	Hex Value
PCM_SYNC	IOC_OUTPUT_CFG_PCM_SYNC_ADCDAC_DIS	Disabled	0x0*
	IOC_OUTPUT_CFG_PCM_SYNC_ADCDAC_EN	Enabled	0x1
PCM1_LOAD	IOC_OUTPUT_CFG_PCM1_FA0_0	Load PCM1 from FIFO A0_0	0x0
	IOC_OUTPUT_CFG_PCM1_FA0_1	Load PCM1 from FIFO A0_1	0x1
	IOC_OUTPUT_CFG_PCM1_FA0_2	Load PCM1 from FIFO A0_2	0x2
	IOC_OUTPUT_CFG_PCM1_FA0_3	Load PCM1 from FIFO A0_3	0x3
	IOC_OUTPUT_CFG_PCM1_FA0_4	Load PCM1 from FIFO A0_4	0x4
	IOC_OUTPUT_CFG_PCM1_FA0_5	Load PCM1 from FIFO A0_5	0x5
	IOC_OUTPUT_CFG_PCM1_FA0_6	Load PCM1 from FIFO A0_6	0x6
	IOC_OUTPUT_CFG_PCM1_FA0_7	Load PCM1 from FIFO A0_7	0x7
	IOC_OUTPUT_CFG_PCM1_FA1_0	Load PCM1 from FIFO A1_0	0x8
	IOC_OUTPUT_CFG_PCM1_FA1_1	Load PCM1 from FIFO A1_1	0x9
	IOC_OUTPUT_CFG_PCM1_FA1_2	Load PCM1 from FIFO A1_2	0xA
	IOC_OUTPUT_CFG_PCM1_FA1_3	Load PCM1 from FIFO A1_3	0xB
	IOC_OUTPUT_CFG_PCM1_FA1_4	Load PCM1 from FIFO A1_4	0xC
	IOC_OUTPUT_CFG_PCM1_FA1_5	Load PCM1 from FIFO A1_5	0xD
	IOC_OUTPUT_CFG_PCM1_FA1_6	Load PCM1 from FIFO A1_6	0xE
	IOC_OUTPUT_CFG_PCM1_FA1_7	Load PCM1 from FIFO A1_7	0xF
	IOC_OUTPUT_CFG_PCM1_FB0_0	Load PCM1 from FIFO B0_0	0x10
	IOC_OUTPUT_CFG_PCM1_FB0_1	Load PCM1 from FIFO B0_1	0x11
	IOC_OUTPUT_CFG_PCM1_FB0_2	Load PCM1 from FIFO B0_2	0x12
	IOC_OUTPUT_CFG_PCM1_FB0_3	Load PCM1 from FIFO B0_3	0x13

Field Name	Value Symbol	Value Description	Hex Value
PCM1_LOAD (continued)	IOC_OUTPUT_CFG_PCM1_FB0_4	Load PCM1 from FIFO B0_4	0x14
	IOC_OUTPUT_CFG_PCM1_FB0_5	Load PCM1 from FIFO B0_5	0x15
	IOC_OUTPUT_CFG_PCM1_FB0_6	Load PCM1 from FIFO B0_6	0x16
	IOC_OUTPUT_CFG_PCM1_FB0_7	Load PCM1 from FIFO B0_7	0x17
	IOC_OUTPUT_CFG_PCM1_FB1_0	Load PCM1 from FIFO B1_0	0x18
	IOC_OUTPUT_CFG_PCM1_FB1_1	Load PCM1 from FIFO B1_1	0x19
	IOC_OUTPUT_CFG_PCM1_FB1_2	Load PCM1 from FIFO B1_2	0x1A
	IOC_OUTPUT_CFG_PCM1_FB1_3	Load PCM1 from FIFO B1_3	0x1B
	IOC_OUTPUT_CFG_PCM1_FB1_4	Load PCM1 from FIFO B1_4	0x1C
	IOC_OUTPUT_CFG_PCM1_FB1_5	Load PCM1 from FIFO B1_5	0x1D
	IOC_OUTPUT_CFG_PCM1_FB1_6	Load PCM1 from FIFO B1_6	0x1E
	IOC_OUTPUT_CFG_PCM1_FB1_7	Load PCM1 from FIFO B1_7	0x1F
	IOC_OUTPUT_CFG_PCM1_PCMRX0	Load PCM1 from PCM RX0	0x20
	IOC_OUTPUT_CFG_PCM1_PCMRX1	Load PCM1 from PCM RX1	0x21
	IOC_OUTPUT_CFG_PCM1_ADC0	Load PCM1 from ADC0	0x22
	IOC_OUTPUT_CFG_PCM1_ADC1	Load PCM1 from ADC1	0x23
	IOC_OUTPUT_CFG_PCM1_ADC2	Load PCM1 from ADC2	0x24
	IOC_OUTPUT_CFG_PCM1_ADC3	Load PCM1 from ADC3	0x25
	IOC_OUTPUT_CFG_PCM1_NONE	Do not load (load 0 instead)	0x3F*
PCM0_LOAD	IOC_OUTPUT_CFG_PCM0_FA0_0	Load PCM0 from FIFO A0_0	0x0
	IOC_OUTPUT_CFG_PCM0_FA0_1	Load PCM0 from FIFO A0_1	0x1
	IOC_OUTPUT_CFG_PCM0_FA0_2	Load PCM0 from FIFO A0_2	0x2
	IOC_OUTPUT_CFG_PCM0_FA0_3	Load PCM0 from FIFO A0_3	0x3
	IOC_OUTPUT_CFG_PCM0_FA0_4	Load PCM0 from FIFO A0_4	0x4
	IOC_OUTPUT_CFG_PCM0_FA0_5	Load PCM0 from FIFO A0_5	0x5
	IOC_OUTPUT_CFG_PCM0_FA0_6	Load PCM0 from FIFO A0_6	0x6
	IOC_OUTPUT_CFG_PCM0_FA0_7	Load PCM0 from FIFO A0_7	0x7
	IOC_OUTPUT_CFG_PCM0_FA1_0	Load PCM0 from FIFO A1_0	0x8
	IOC_OUTPUT_CFG_PCM0_FA1_1	Load PCM0 from FIFO A1_1	0x9
	IOC_OUTPUT_CFG_PCM0_FA1_2	Load PCM0 from FIFO A1_2	0xA
	IOC_OUTPUT_CFG_PCM0_FA1_3	Load PCM0 from FIFO A1_3	0xB
	IOC_OUTPUT_CFG_PCM0_FA1_4	Load PCM0 from FIFO A1_4	0xC
	IOC_OUTPUT_CFG_PCM0_FA1_5	Load PCM0 from FIFO A1_5	0xD
	IOC_OUTPUT_CFG_PCM0_FA1_6	Load PCM0 from FIFO A1_6	0xE
	IOC_OUTPUT_CFG_PCM0_FA1_7	Load PCM0 from FIFO A1_7	0xF
	IOC_OUTPUT_CFG_PCM0_FB0_0	Load PCM0 from FIFO B0_0	0x10
	IOC_OUTPUT_CFG_PCM0_FB0_1	Load PCM0 from FIFO B0_1	0x11
	IOC_OUTPUT_CFG_PCM0_FB0_2	Load PCM0 from FIFO B0_2	0x12

Field Name	Value Symbol	Value Description	Hex Value
PCM0_LOAD (continued)	IOC_OUTPUT_CFG_PCM0_FB0_3	Load PCM0 from FIFO B0_3	0x13
	IOC_OUTPUT_CFG_PCM0_FB0_4	Load PCM0 from FIFO B0_4	0x14
	IOC_OUTPUT_CFG_PCM0_FB0_5	Load PCM0 from FIFO B0_5	0x15
	IOC_OUTPUT_CFG_PCM0_FB0_6	Load PCM0 from FIFO B0_6	0x16
	IOC_OUTPUT_CFG_PCM0_FB0_7	Load PCM0 from FIFO B0_7	0x17
	IOC_OUTPUT_CFG_PCM0_FB1_0	Load PCM0 from FIFO B1_0	0x18
	IOC_OUTPUT_CFG_PCM0_FB1_1	Load PCM0 from FIFO B1_1	0x19
	IOC_OUTPUT_CFG_PCM0_FB1_2	Load PCM0 from FIFO B1_2	0x1A
	IOC_OUTPUT_CFG_PCM0_FB1_3	Load PCM0 from FIFO B1_3	0x1B
	IOC_OUTPUT_CFG_PCM0_FB1_4	Load PCM0 from FIFO B1_4	0x1C
	IOC_OUTPUT_CFG_PCM0_FB1_5	Load PCM0 from FIFO B1_5	0x1D
	IOC_OUTPUT_CFG_PCM0_FB1_6	Load PCM0 from FIFO B1_6	0x1E
	IOC_OUTPUT_CFG_PCM0_FB1_7	Load PCM0 from FIFO B1_7	0x1F
	IOC_OUTPUT_CFG_PCM0_PCMRX0	Load PCM0 from PCM RX0	0x20
	IOC_OUTPUT_CFG_PCM0_PCMRX1	Load PCM0 from PCM RX1	0x21
	IOC_OUTPUT_CFG_PCM0_ADC0	Load PCM0 from ADC0	0x22
	IOC_OUTPUT_CFG_PCM0_ADC1	Load PCM0 from ADC1	0x23
	IOC_OUTPUT_CFG_PCM0_ADC2	Load PCM0 from ADC2	0x24
	IOC_OUTPUT_CFG_PCM0_ADC3	Load PCM0 from ADC3	0x25
	IOC_OUTPUT_CFG_PCM0_NONE	Do not load (load 0 instead)	0x3F*
DAC1_LOAD	IOC_OUTPUT_CFG_DAC1_FA0_0	Load DAC1 from FIFO A0_0	0x0
	IOC_OUTPUT_CFG_DAC1_FA0_1	Load DAC1 from FIFO A0_1	0x1
	IOC_OUTPUT_CFG_DAC1_FA0_2	Load DAC1 from FIFO A0_2	0x2
	IOC_OUTPUT_CFG_DAC1_FA0_3	Load DAC1 from FIFO A0_3	0x3
	IOC_OUTPUT_CFG_DAC1_FA0_4	Load DAC1 from FIFO A0_4	0x4
	IOC_OUTPUT_CFG_DAC1_FA0_5	Load DAC1 from FIFO A0_5	0x5
	IOC_OUTPUT_CFG_DAC1_FA0_6	Load DAC1 from FIFO A0_6	0x6
	IOC_OUTPUT_CFG_DAC1_FA0_7	Load DAC1 from FIFO A0_7	0x7
	IOC_OUTPUT_CFG_DAC1_FA1_0	Load DAC1 from FIFO A1_0	0x8
	IOC_OUTPUT_CFG_DAC1_FA1_1	Load DAC1 from FIFO A1_1	0x9
	IOC_OUTPUT_CFG_DAC1_FA1_2	Load DAC1 from FIFO A1_2	0xA
	IOC_OUTPUT_CFG_DAC1_FA1_3	Load DAC1 from FIFO A1_3	0xB
	IOC_OUTPUT_CFG_DAC1_FA1_4	Load DAC1 from FIFO A1_4	0xC
	IOC_OUTPUT_CFG_DAC1_FA1_5	Load DAC1 from FIFO A1_5	0xD
	IOC_OUTPUT_CFG_DAC1_FA1_6	Load DAC1 from FIFO A1_6	0xE
	IOC_OUTPUT_CFG_DAC1_FA1_7	Load DAC1 from FIFO A1_7	0xF
	IOC_OUTPUT_CFG_DAC1_FB0_0	Load DAC1 from FIFO B0_0	0x10
	IOC_OUTPUT_CFG_DAC1_FB0_1	Load DAC1 from FIFO B0_1	0x11

Field Name	Value Symbol	Value Description	Hex Value
DAC1_LOAD (continued)	IOC_OUTPUT_CFG_DAC1_FB0_2	Load DAC1 from FIFO B0_2	0x12
	IOC_OUTPUT_CFG_DAC1_FB0_3	Load DAC1 from FIFO B0_3	0x13
	IOC_OUTPUT_CFG_DAC1_FB0_4	Load DAC1 from FIFO B0_4	0x14
	IOC_OUTPUT_CFG_DAC1_FB0_5	Load DAC1 from FIFO B0_5	0x15
	IOC_OUTPUT_CFG_DAC1_FB0_6	Load DAC1 from FIFO B0_6	0x16
	IOC_OUTPUT_CFG_DAC1_FB0_7	Load DAC1 from FIFO B0_7	0x17
	IOC_OUTPUT_CFG_DAC1_FB1_0	Load DAC1 from FIFO B1_0	0x18
	IOC_OUTPUT_CFG_DAC1_FB1_1	Load DAC1 from FIFO B1_1	0x19
	IOC_OUTPUT_CFG_DAC1_FB1_2	Load DAC1 from FIFO B1_2	0x1A
	IOC_OUTPUT_CFG_DAC1_FB1_3	Load DAC1 from FIFO B1_3	0x1B
	IOC_OUTPUT_CFG_DAC1_FB1_4	Load DAC1 from FIFO B1_4	0x1C
	IOC_OUTPUT_CFG_DAC1_FB1_5	Load DAC1 from FIFO B1_5	0x1D
	IOC_OUTPUT_CFG_DAC1_FB1_6	Load DAC1 from FIFO B1_6	0x1E
	IOC_OUTPUT_CFG_DAC1_FB1_7	Load DAC1 from FIFO B1_7	0x1F
	IOC_OUTPUT_CFG_DAC1_PCMRX0	Load DAC1 from PCM RX0	0x20
	IOC_OUTPUT_CFG_DAC1_PCMRX1	Load DAC1 from PCM RX1	0x21
	IOC_OUTPUT_CFG_DAC1_ADC0	Load DAC1 from ADC0	0x22
	IOC_OUTPUT_CFG_DAC1_ADC1	Load DAC1 from ADC1	0x23
	IOC_OUTPUT_CFG_DAC1_ADC2	Load DAC1 from ADC2	0x24
	IOC_OUTPUT_CFG_DAC1_ADC3	Load DAC1 from ADC3	0x25
	IOC_OUTPUT_CFG_DAC1_LOAD	Load DAC1 from the D_DAC1_DATA register	0x26
	IOC_OUTPUT_CFG_DAC1_NONE	Do not load (load 0 instead)	0x3F*
DAC0_LOAD	IOC_OUTPUT_CFG_DAC0_FA0_0	Load DAC0 from FIFO A0_0	0x0
	IOC_OUTPUT_CFG_DAC0_FA0_1	Load DAC0 from FIFO A0_1	0x1
	IOC_OUTPUT_CFG_DAC0_FA0_2	Load DAC0 from FIFO A0_2	0x2
	IOC_OUTPUT_CFG_DAC0_FA0_3	Load DAC0 from FIFO A0_3	0x3
	IOC_OUTPUT_CFG_DAC0_FA0_4	Load DAC0 from FIFO A0_4	0x4
	IOC_OUTPUT_CFG_DAC0_FA0_5	Load DAC0 from FIFO A0_5	0x5
	IOC_OUTPUT_CFG_DAC0_FA0_6	Load DAC0 from FIFO A0_6	0x6
	IOC_OUTPUT_CFG_DAC0_FA0_7	Load DAC0 from FIFO A0_7	0x7
	IOC_OUTPUT_CFG_DAC0_FA1_0	Load DAC0 from FIFO A1_0	0x8
	IOC_OUTPUT_CFG_DAC0_FA1_1	Load DAC0 from FIFO A1_1	0x9
	IOC_OUTPUT_CFG_DAC0_FA1_2	Load DAC0 from FIFO A1_2	0xA
	IOC_OUTPUT_CFG_DAC0_FA1_3	Load DAC0 from FIFO A1_3	0xB
	IOC_OUTPUT_CFG_DAC0_FA1_4	Load DAC0 from FIFO A1_4	0xC
	IOC_OUTPUT_CFG_DAC0_FA1_5	Load DAC0 from FIFO A1_5	0xD
	IOC_OUTPUT_CFG_DAC0_FA1_6	Load DAC0 from FIFO A1_6	0xE

Field Name	Value Symbol	Value Description	Hex Value
DAC0_LOAD (continued)	IOC_OUTPUT_CFG_DAC0_FA1_7	Load DAC0 from FIFO A1_7	0xF
	IOC_OUTPUT_CFG_DAC0_FB0_0	Load DAC0 from FIFO B0_0	0x10
	IOC_OUTPUT_CFG_DAC0_FB0_1	Load DAC0 from FIFO B0_1	0x11
	IOC_OUTPUT_CFG_DAC0_FB0_2	Load DAC0 from FIFO B0_2	0x12
	IOC_OUTPUT_CFG_DAC0_FB0_3	Load DAC0 from FIFO B0_3	0x13
	IOC_OUTPUT_CFG_DAC0_FB0_4	Load DAC0 from FIFO B0_4	0x14
	IOC_OUTPUT_CFG_DAC0_FB0_5	Load DAC0 from FIFO B0_5	0x15
	IOC_OUTPUT_CFG_DAC0_FB0_6	Load DAC0 from FIFO B0_6	0x16
	IOC_OUTPUT_CFG_DAC0_FB0_7	Load DAC0 from FIFO B0_7	0x17
	IOC_OUTPUT_CFG_DAC0_FB1_0	Load DAC0 from FIFO B1_0	0x18
	IOC_OUTPUT_CFG_DAC0_FB1_1	Load DAC0 from FIFO B1_1	0x19
	IOC_OUTPUT_CFG_DAC0_FB1_2	Load DAC0 from FIFO B1_2	0x1A
	IOC_OUTPUT_CFG_DAC0_FB1_3	Load DAC0 from FIFO B1_3	0x1B
	IOC_OUTPUT_CFG_DAC0_FB1_4	Load DAC0 from FIFO B1_4	0x1C
	IOC_OUTPUT_CFG_DAC0_FB1_5	Load DAC0 from FIFO B1_5	0x1D
	IOC_OUTPUT_CFG_DAC0_FB1_6	Load DAC0 from FIFO B1_6	0x1E
	IOC_OUTPUT_CFG_DAC0_FB1_7	Load DAC0 from FIFO B1_7	0x1F
	IOC_OUTPUT_CFG_DAC0_PCMRX0	Load DAC0 from PCM RX0	0x20
	IOC_OUTPUT_CFG_DAC0_PCMRX1	Load DAC0 from PCM RX1	0x21
	IOC_OUTPUT_CFG_DAC0_ADC0	Load DAC0 from ADC0	0x22
	IOC_OUTPUT_CFG_DAC0_ADC1	Load DAC0 from ADC1	0x23
	IOC_OUTPUT_CFG_DAC0_ADC2	Load DAC0 from ADC2	0x24
	IOC_OUTPUT_CFG_DAC0_ADC3	Load DAC0 from ADC3	0x25
	IOC_OUTPUT_CFG_DAC0_LOAD	Load DAC0 from the D_DAC0_DATA register	0x26
	IOC_OUTPUT_CFG_DAC0_NONE	Do not load (load 0 instead)	0x3F*

9.5.2.2 D_DAC0_DATA Settings

Bit Field	Field Name	Description
23:0	DAC0_DATA	Input sample for output channel 0 (m1p23 format)

9.5.2.3 D_DAC1_DATA Settings

Bit Field	Field Name	Description
23:0	DAC1_DATA	Input sample for output channel 1 (m1p23 format)

9.5.2.4 D_FIFO_x_y_AUTO_MUTE Settings

These bit fields are the same for **x** and **y**, where **x** represents A0, A1, B0 or B1 memories, and **y** is 0 to 7.

Bit Field	Field Name	Description
0	AUTO_MUTE_ENABLE	FIFO auto mute enable for FIFO **x y**

Field Name	Value Symbol	Value Description	Hex Value
AUTO_MUTE_ENABLE	FIFO_x_y_AUTO_MUTE_DISABLE	Disable the auto mute feature	0x0*
	FIFO_x_y_AUTO_MUTE_ENABLE	Enable the auto mute feature	0x1

9.6 IOC Data Transfer Timing

The IOC coordinates the sequential movement of data to and from the input and output components. The IOC creates a fixed ordering of data transfers to and from the FIFOs that results in the associated FIFO interrupts being generated in the same fixed order.

IMPORTANT: To provide a fixed ordering of data reception in the FIFOs (and consequently a fixed ordering of FIFO interrupts), the IOC must not be providing data to or requesting data from a FIFO while that FIFO is being configured.

If the PCM interface has been synchronized with the ADC and DAC components, interrupts generated by IOC data transfers between the PCM interface and the FIFOs have a guaranteed timing relative to the other FIFO interrupts (as described in the previous sections). When unsynchronized, the relative timing of interrupts generated for PCM interface data to other FIFO interrupts cannot be guaranteed.

IMPORTANT: The relative timing of PCM interface inputs relative to the input stage data is only guaranteed if the PCM interface is synchronized with the input stage. Similarly the relative timing of PCM interface outputs and output stage data is only guaranteed if the PCM interface is synchronized with the output stage.

The timing of software FIFO interrupts depends on the application using the software FIFO and is not controlled by the IOC.

The ordering and relative timing of the IOC data transfers to and from the FIFOs (when the PCM interface is synchronized with the input and output stages) is found in Table 20. The timing of the FIFO interrupts is based on this ordering and relative timing, as well as the configuration of each FIFO's block size and pointers.

Table 20. Relative IOC Data Transfer Timing Between FIFOs and System Components

Ordering	Components	Relative Timing
1	ADC0, ADC2	Data is transferred from ADC0 and ADC2 at the beginning of each sample period.
2	DAC0, DAC1	Data is transferred after a delay of approximately 1/4 of the sampling period from the beginning of the sampling period.

Table 20. Relative IOC Data Transfer Timing Between FIFOs and System Components

Ordering	Components	Relative Timing
3	ADC1, ADC3, IDRX0, IDRX1	Data is transferred after a delay of approximately 1/2 of the sampling period from the beginning of the sampling period.
4	PCM0, PCM1 (Receive) and PCM0, PCM1 (Transmit)	Data transfers due to PCM input data occur at the same time within a sample period as interrupts due to PCM output data requests at either of the following times, whichever happens later: • Immediately following the last enabled ADC or DAC • When data for the PCM interface arrives or is needed by the interface for transmission.

This relative timing sequence is important for applications that process data synchronously, triggering their input and output processing on a single interrupt (typically a FIFO interrupt). In these situations, the application normally waits for the last interrupt to arrive before beginning to process the data. For example, consider timing when a FIFO interrupt is generated for a block of data transferred from ADC0. A block of data from ADC1 that should be received and processed synchronously with this data would not have been received completely, and a previous output result block for the DAC would not have been transmitted completely.

To ensure that the IOC has completed transferring the current block of data for all of the relevant components, synchronize with the interrupt associated with the last scheduled data transfer. As a result, a user application using ADC0 for input and DAC0 for output would use the FIFO interrupt associated with DAC0 to synchronize processing. Similarly, if the same application instead used ADC1 and DAC0, the FIFO interrupt associated with ADC1 would be used.

IMPORTANT: To ensure proper system behavior, if the PCM interface is not synchronized with the input stage, use separate interrupts or events to trigger processing of input channel data and PCM data. Similarly, if the PCM interface is not synchronized with the output stage, handle output channel data and PCM data separately.

CHAPTER 10
Input Stage

10.1 Input Stage Architecture

The input stage of an Ezairo 7100 provides four audio input channels that supply signal data to the rest of the Ezairo 7100 system through the IOC. Each input channel includes:

- Input selection using an analog multiplexer to select between 4 input sources for each channel from the available 6 input sources
- Programmable input preamplification to adjust the selected raw input signal to a suitable level for the analog-to-digital converters (ADCs)
- An anti-aliasing low-pass filter with a selectable 20 kHz or 50 kHz (bypass) -3 dB cut-off frequency
- Available line-out for the preamplified and filtered input signal
- An over-sampling analog-to-digital converter (ADC) which uses a programmable sampling frequency and provides a configurable sampling delay
- High-quality decimation filtering at all selectable sampling rates
- Selectable input muting
- Selectable digital microphone inputs that can source data in place of the ADCs

Each of these components are shown in Figure 15, which shows the overall architecture of the input stage.

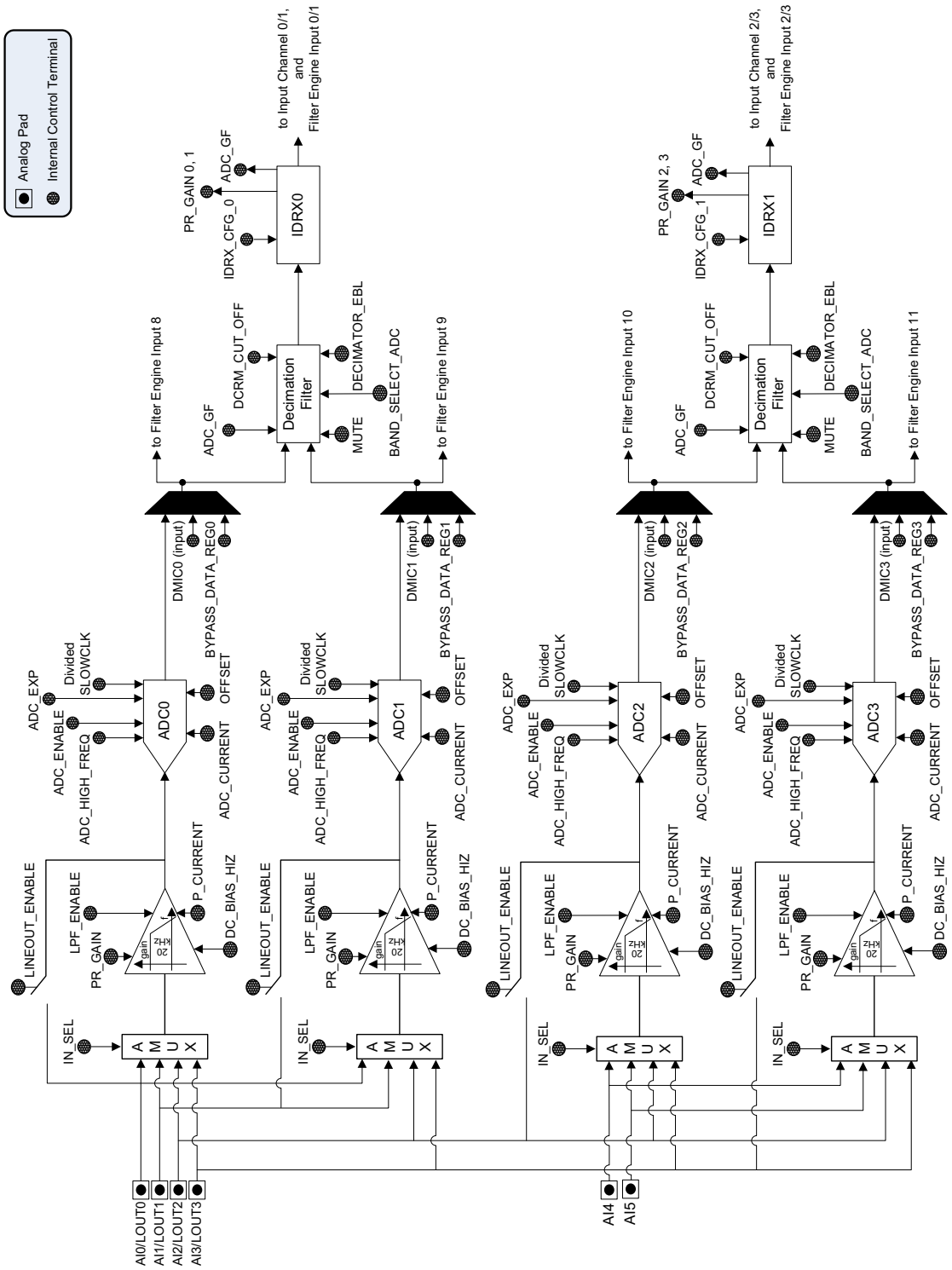


Figure 15. Input Stage Block Diagram

The input data provided by the ADCs are multiplexed with the digital microphone inputs and bypass data registers, as described in Section 9.1.2, “Audio Multiplexing of Audio Input Sources” on page 114. Signals from the input stage are then routed to the Filter Engine directly or through the IOC to the rest of the system, as described in Chapter 6, “Filter Engine” on page 50 and Chapter 9, “Input/Output Audio Control” on page 111 respectively.

10.2 Input Selection

The four input channels select their signal source from four of the six input pads through the A_ADC_CTRL_* registers. Channels 0 and 1 use the ADC_CTRL_IN_SEL bit field to select one of the analog input pins 0, 1, 2 or 3 as the input source for the channel. Channels 2 and 3 use the ADC_CTRL_IN_SEL bit field to select one of analog input pins 2, 3, 4 or 5 as the input source for the channel.

The system also has the ADC_CTRL_ADC_ENABLE bits from the A_ADC_CTRL_* registers that allow you to selectively enable or disable the receipt of data and components for each of the four input stage channels.

CAUTION: Do not use the same input source for multiple input channels or the signal quality for all channels is reduced due to the associated reduction of input impedance. Depending on the preamplifier configuration, this signal degradation might also result in preamplifier saturation that will effectively mute the input. This is an issue even if only one of the input channels sharing an input source is active.

External de-coupling capacitors are required for all inputs that are used in the system. These de-coupling capacitors, when combined with the input biasing resistors in the preamplifiers, form a single-pole high-pass input filter that reduces the amount of low-frequency and DC coupling that could saturate the input preamplifiers. The 500 kΩ input biasing resister can be disconnected from the inputs using the ADC_CTRL_DC_BIAS_HIZ bit from the A_ADC_CTRL_* registers.

Ground unused input pins.

10.3 Preamplifiers

The input stage provides a preamplifier for each of the four input channels. When enabled these preamplifiers can amplify the input signal between 0 and 36 dB in 3 dB steps. Use the ADC_CTRL_PR_GAIN bit field from the A_ADC_CTRL_* registers to select a preamplifier gain for each channel.

The preamplifiers used as part of the input stage introduce a delay in sampling the input signal by the system. This delay varies depending on the configuration of the ADC_CTRL_PR_GAIN bit field from the A_ADC_CTRL_* registers, whether the input low-pass filter (LPF) is enabled, and the selection of input coupling capacitor. The approximate amount of delay introduced by each configuration using a 10 nF coupling capacitor and operating from a standard 2.56 MHz slow clock is given in Table 21.

Table 21. Approximate Sampling Delay Introduced by the Preamplifier Component

Preamplifier Setting	Added Sampling Delay with Input LPF Disabled (µs)	Added Sampling Delay with Input LPF Enabled (µs)
PGAIN_BYPASS	1.50	2.26
PGAIN_3DB	1.58	3.10
PGAIN_6DB	1.62	4.70
PGAIN_9DB	1.80	6.40
PGAIN_12DB	1.97	8.10

Table 21. Approximate Sampling Delay Introduced by the Preamplifier Component (Continued)

Preamplifier Setting	Added Sampling Delay with Input LPF Disabled (µs)	Added Sampling Delay with Input LPF Enabled (µs)
PGAIN_15DB	2.20	6.80
PGAIN_18DB	2.37	7.40
PGAIN_21DB	2.50	8.00
PGAIN_24DB	2.79	8.80
PGAIN_27DB	3.04	9.40
PGAIN_30DB	3.58	10.1
PGAIN_33DB	4.09	11.2
PGAIN_36DB	5.01	12.8

The input stage uses an external reference signal for each input channel through the associated AIRxy pads which each provide a clean ground reference signal for the attached channels. We recommend that input microphones be connected between VREG (through the supplied VMIC pad) and microphone GND. This results in a typical voltage drop of 0.97 V across the microphone.

The current supplied to the preamplifiers is configured by setting the ADC_CTRL_P_CURRENT bit field in the A_ADC_CFG_* control registers. In general, when using sampling frequencies below 20 kHz, use the nominal value for these bit fields. For higher sampling frequencies, apply a higher current to prevent any reduction in fidelity. The nominal current (Inom) can be found as part of the chip's product data sheet.

For information about preamplifier register configuration, see Section 10.5.3, "Input Stage Control and Configuration Registers" on page 146.

10.3.1 Low-Pass Filters

At the input, the signal can be passed through one of the anti-aliasing low-pass filters that are combined with the preamplifiers. These filters can be used to attenuate high-frequency aliasing components of the input signals, partially eliminating this side-effect of digital sampling.

Each filter is a 2nd-order, active low-pass filter with an approximate -3 dB cut-off frequency of:

- 70 kHz for preamplifier configurations that apply a gain of less than 9 dB
- 37 kHz for preamplifier configurations that apply a gain between 12 dB and 30 dB
- 29 kHz for preamplifier configurations that apply a gain of more than 30 dB

To enable the filter for a channel, set the appropriate ADC_CTRL_LPF_ENABLE bits in the A_ADC_CTRL_* registers. If disabled, a first-order low-pass filter with an approximate -3 dB cut-off frequency of 130 kHz is applied to the channel.

The ADCs oversample the input signal by a factor of eight. As a result, if a 16 kHz sampling frequency is selected with the slow clock set to 1.28 MHz, no aliased images appear for input frequencies below 120 kHz. As a result, if you consider the combined roll-off of the low-pass filters at a cut-off frequency of 20 kHz, and the roll-off present in a typical microphone, this configuration provides sufficient aliasing protection.

The low-pass filters add a delay component to the overall sampling delay for a channel. For further information about the approximate effects of enabling the low-pass filter on sampling delay, see Section 10.3, “Preamplifiers” on page 142.

For information about low-pass filter registers, see Section 10.5.3, “Input Stage Control and Configuration Registers” on page 146.

10.4 Line-out

The preamplified and low-pass-filtered input signals from channels 0 to 3 are available as output on the input pads AI0/LOUT0 to AI3/LOUT3. To configure an input pad for use as a line-out signal, set the appropriate `ADC_CTRL_LINEOUT_ENABLE` bit in the `A_ADC_CTRL_*` registers.

For information about line-out registers, see Section 10.5.3, “Input Stage Control and Configuration Registers” on page 146.

10.5 Analog-to-Digital Converters (ADCs)

The ADCs supplied on the Ezairo 7100 chip can be used to supply the system with data obtained at a configurable sampling rate. These ADCs oversample the input signal by a factor of 8 to obtain a higher quality signal, which is then filtered by the decimation filter to obtain the desired sampling frequency.

NOTE: The high oversampling of the ADCs shifts aliased signal components into a significantly higher frequency band. For example, if a sampling frequency of 16 kHz is selected, no aliased images appears for input frequencies below 120 kHz.

Select the sampling rate for the ADCs by setting the `ADC_CTRL_SAMPLE_FREQ` bit field from the `A_ADC_SAMPLE_FREQ_CFG` register. The ADC oversamples the incoming signal at eight times this requested data rate. The following equations calculates the sampling frequency:

$$f_{sampling} = \frac{f_{SLOWCLK}}{(ADC_SAMPLE_FREQ_CFG_SAMPLE_FREQ + 8) \cdot 8}$$

A table providing the sampling frequency obtained for each setting and slow clock configuration is in Appendix B, “Sampling Frequency Reference” on page 612. As specifying the sampling frequency largely depends on the slow clock configuration, refer to Section 18.2.2, “Slow Clock (SLOWCLK)” on page 446 for other related information.

The input and output stages of the Ezairo 7100 system both use the same sampling rate (more information about the output stage is in Chapter 11, “Output Stage” on page 164).

IMPORTANT: When using the ADCs with a slow clock configuration that results in the supplied clock frequency exceeding 1.5 MHz, set the `HIGH_FREQ` bit in the `A_ADC_SAMPLE_FREQ_CFG` register.

The ADC can also be used to apply a digital gain between 0 and 18 dB in 6 dB steps to augment the gain applied by the preamplifier. This digital gain is configured by setting the `ADC_CTRL_ADC_EXP` bit field in the `A_ADC_CTRL_*` registers.

The current supplied to the ADCs is configured by setting the `ADC_CFG_ADC_CURRENT` bit field in the `A_ADC_CFG_*` control registers. In general, when using sampling frequencies below 20 kHz, use the nominal value for

these bit fields. For higher sampling frequencies, apply a higher current to prevent any reduction in fidelity. The nominal current (I_{nom}) expected can be found as part of the chip's product data sheet.

10.5.1 DC Offset Configuration

For optimal noise performance, we recommend configuring the DC offset observed by the ADCs to include a calibrated DC offset for the input signals. This calibration is required as the ADCs will produce a periodic output with a frequency directly correlated to the DC level observed at the ADC input. This level can be controlled by setting the amount of DC added to the preamplifier's output signal to shift the ADC's periodic output outside of the bandwidth of the audio signal.

The observed level of the DC offset changes with the input preamplifier configuration and the sampling frequency, and thus needs to be adjusted whenever either of these parameters are adjusted. The DC offset can be configured using the `ADC_CFG_OFFSET_SIGN` and `ADC_CFG_OFFSET_MAGN` bit fields from the `A_ADC_CFG_*` registers. The resulting DC signal can be removed using the decimation filter's DC removal filter as described in Section 10.7.1, "DC Removal Filters".

Appropriate DC offset settings can be calculated using the ADC calibration components of the calibration library. An application can load a previously calibrated DC offset settings by using the `Sys_Set_ADC_CTRL`, `Sys_SetReg_ADC_CTRL` and `Sys_CalibrateADCs` macros. For more information about these calibration support macros, see the *Firmware Reference Manual for Ezairo 7100*.

10.5.2 Sampling Delay

The ADCs are designed so that you can precisely set the timing of when a signal is sampled. The specification of a sampling delay between channels is defined in terms of two components: an integer sampling delay component and a fractional sampling delay component.

NOTE: The default time at which the signal on ADC0 would be sampled is the point of reference for all delays.

The integer component of the sampling delay specifies the delay in terms of the time between samples in the oversample domain. Figure 16 shows the relationship between the sampling frequency of the ADC, and the integer component and fractional component of the sampling delay.

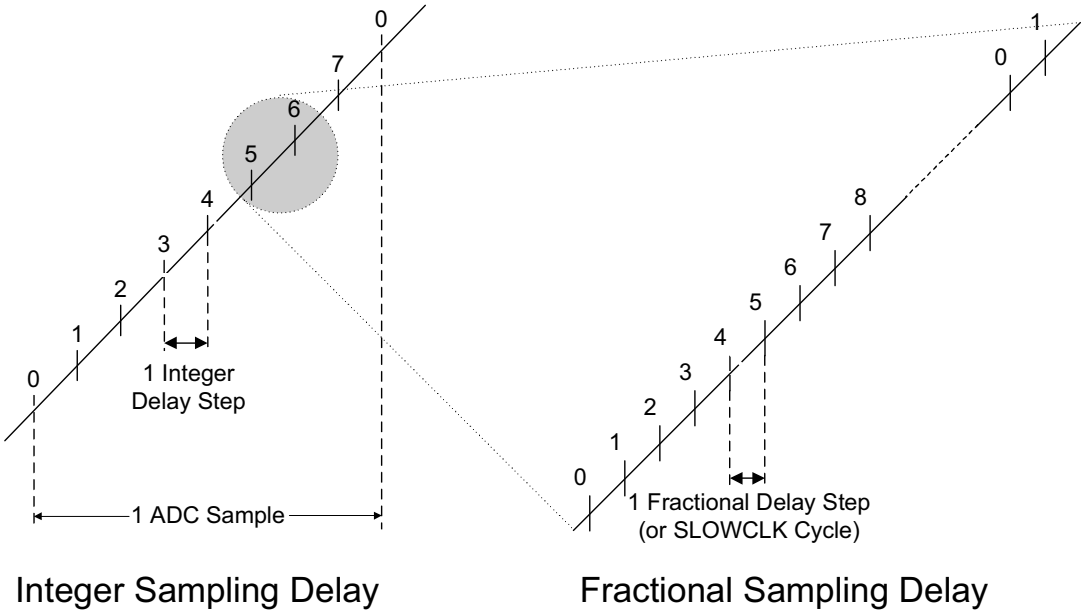


Figure 16. Sampling Delay Components

To set the integer delay for each channel, set the ADC_DEC_CTRL_DELAY_INTEGER bit fields of the A_ADC_DEC_CTRL_* registers. The integer delay allows for a delay of up to 7 additional oversampling periods, or a delay of just under one full sample.

The fractional component of the sampling delay allows you to specify an additional delay in the timing of obtaining samples. This delay is measured in terms of additional slow clock cycles before the system takes a sample measurement in the oversample domain. You can see this in Figure 16 and specify it by setting the ADC_DEC_CTRL_DELAY_FRACTIONAL bit fields in the A_ADC_DEC_CTRL_* registers. This fine tuning of the sampling delay allows you to add to the integer delay component from 0 to (ADC_SAMPLE_FREQ_CFG_SAMPLE_FREQ + 8) slow clock cycles of fractional delay.

CAUTION: The preamplifier stage and preamplifier configuration settings also contribute to the sampling delay. Particularly if you are using different configurations for preamplifiers in your application, refer to Section 10.3, “Preamplifiers” on page 142 for more information.

10.5.3 Input Stage Control and Configuration Registers

Register Name	Register Description	Address
A_ADC_CTRL_0	Input control registers	P:0xF140
A_ADC_CTRL_1	Input control registers	P:0xF141
A_ADC_CTRL_2	Input control registers	P:0xF142
A_ADC_CTRL_3	Input control registers	P:0xF143
A_ADC_CFG_0	Input configuration registers	P:0xF144
A_ADC_CFG_1	Input configuration registers	P:0xF145

Register Name	Register Description	Address
A_ADC_CFG_2	Input configuration registers	P:0xF146
A_ADC_CFG_3	Input configuration registers	P:0xF147
A_ADC_SAMPLE_FREQ_CFG	Sampling frequency configuration	P:0xF14E

10.5.3.1 A_ADC_CTRL_* Settings

The following bit fields and field names apply equally to registers A_ADC_CTRL_0 to A_ADC_CTRL_3.

Bit Field	Field Name	Description
13	LINEOUT_ENABLE	Line output control
12	LPF_ENABLE	Enables low-pass filter in preamplifier
9:8	ADC_EXP	Gain exponent for ADC
7:4	PR_GAIN	Select the gain of the preamplifier
3:1	IN_SEL	Select the input applied to ADC or preamplifier
0	ADC_ENABLE	Enables ADC

Field Name	Value Symbol	Value Description	Hex Value
LINEOUT_ENABLE	LINEOUT_DISABLE	Line out disabled	0x0*
	LINEOUT_ENABLE	Line out enabled	0x1
LPF_ENABLE	LPF_IN_BYPASS	Low pass filter disabled	0x0
	LPF_IN_ACTIVE_20KHZ	Low pass filter enabled	0x1*
ADC_EXP	ADC_GAIN_0DB	0 dB digital gain (no shift)	0x0*
	ADC_GAIN_6DB	6 dB digital gain (1 bit left shift with saturation)	0x1
	ADC_GAIN_12DB	12 dB digital gain (2 bit left shift with saturation)	0x2
	ADC_GAIN_18DB	18 dB digital gain (3 bit left shift with saturation)	0x3
PR_GAIN	PGAIN_BYPASS	0 dB (Preamplifier bypassed)	0x0*
	PGAIN_3DB	3 dB	0x1
	PGAIN_6DB	6 dB	0x2
	PGAIN_9DB	9 dB	0x3
	PGAIN_12DB	12 dB	0x4
	PGAIN_15DB	15 dB	0x5
	PGAIN_18DB	18 dB	0x6
	PGAIN_21DB	21 dB	0x7
	PGAIN_24DB	24 dB	0x8
	PGAIN_27DB	27 dB	0x9
	PGAIN_30DB	30 dB	0xA
	PGAIN_33DB	33 dB	0xB
	PGAIN_36DB	36 dB	0xC

Field Name	Value Symbol	Value Description	Hex Value
IN_SEL	ADC_SEL_AI0_AI4	AI0 or AI4	0x0
	ADC_SEL_AI1_AI5	AI1 or AI5	0x1
	ADC_SEL_AI2	AI2 (LOUT2)	0x2*
	ADC_SEL_AI3	AI3 (LOUT3)	0x3
	ADC_SEL_MUTE	Mute	0x4
ADC_ENABLE	ADC_DISABLE	ADC disabled	0x0*
	ADC_ENABLE	ADC enabled	0x1

10.5.3.2 A_ADC_CFG_* Settings

The following bit fields and field names apply equally to registers A_ADC_CFG_0 to A_ADC_CFG_3.

Bit Field	Field Name	Description
16	DC_BIAS_HIZ	Disconnect the 500 kOhm DC bias resistor
14:12	P_CURRENT	Trimming of the preamplifier current
10:8	ADC_CURRENT	Trimming the current of the ADC
7	OFFSET_SIGN	Set sign of the offset in ADC
6:0	OFFSET_MAGN	Set the offset magnitude in ADC

Field Name	Value Symbol	Value Description	Hex Value
DC_BIAS_HIZ	ADC_INPUT_BIASED	DC bias resistor connected	0x0*
	ADC_INPUT_HIZ	Input is floating (HiZ)	0x1
P_CURRENT	P_CURRENT_4_0X	4 times the nominal current	0x0
	P_CURRENT_2_0X	2 times the nominal current	0x1
	P_CURRENT_1_3X	1.3 times the nominal current	0x2
	P_CURRENT_1_0X	The nominal current	0x3*
	P_CURRENT_0_8X	0.8 times the nominal current	0x4
	P_CURRENT_0_7X	0.7 times the nominal current	0x5
	P_CURRENT_0_6X	0.6 times the nominal current	0x6
	P_CURRENT_0_5X	0.5 times the nominal current	0x7
ADC_CURRENT	ADC_CURRENT_3_0X	3 times the nominal current	0x0
	ADC_CURRENT_1_8X	1.8 times the nominal current	0x1
	ADC_CURRENT_1_3X	1.3 times the nominal current	0x2
	ADC_CURRENT_1_0X	The nominal current	0x3*
	ADC_CURRENT_0_8X	0.8 times the nominal current	0x4
	ADC_CURRENT_0_7X	0.7 times the nominal current	0x5
	ADC_CURRENT_0_6X	0.6 times the nominal current	0x6
	ADC_CURRENT_0_5X	0.5 times the nominal current	0x7

Field Name	Value Symbol	Value Description	Hex Value
OFFSET_SIGN	ADC_OFFSET_POS	ADC offset is positive	0x0*
	ADC_OFFSET_NEG	ADC offset is negative	0x1
OFFSET_MAGN	ADC_OFFSET_16MV	16 mV offset	0x18*

10.5.3.3 A_ADC_SAMPLE_FREQ_CFG Settings

Bit Field	Field Name	Description
9	COMPBUF_EBL	ADC comparator buffer enable
8	HIGH_FREQ	Input stage high frequency mode (use for slow clock >1.92 MHz)
5:0	SAMPLE_FREQ	Sampling frequency setting

Field Name	Value Symbol	Value Description	Hex Value
COMPBUF_EBL	ADC_COMP_BUFFER_BYPASS	Comparison buffer bypassed	0x0*
	ADC_COMP_BUFFER_ENABLE	Comparison buffer enabled	0x1
HIGH_FREQ	ADC_HIGH_FREQ_DISABLE	High frequency mode disabled	0x0*
	ADC_HIGH_FREQ_ENABLE	High frequency mode enabled	0x1

10.6 Digital Microphone Inputs

Each of the input channels from the Ezairo 7100 system can optionally source their input data from a digital microphone instead of the channel's ADC. The source for each digital microphone is selected from the digital I/Os and configured as described in Section 16.1, "Digital I/O (DIO)" on page 351.

The clock used to sample the DMIC input data can be selected from SLOWCLK or EXTCLK using the AUDIO_MUX_CFG_DMIC_IN_CLK_SRC bit from the A_AUDIO_MUX_CFG register. If the Ezairo 7100 system is providing the clock for DMIC inputs, select SLOWCLK here and provide SLOWCLK to the digital microphone using either the DIO pads or EXTCLK pad. If you are using EXTCLK as the source for the DMIC clock input, this clock signal must:

- Have the same nominal frequency as SLOWCLK
- Be in phase with SLOWCLK so that the rising edges of EXTCLK occur no less then 15 ns before or 15 ns after a SLOWCLK falling edge

The DMIC input signals from each of the DMIC inputs are configured to clock in the digital input data by setting the AUDIO_MUX_CFG_INPUT_CH*_SRC bit field from the A_AUDIO_MUX_CFG register to either:

- The DMIC*_DATA_RE bit setting for the rising edge of this clock
- The DMIC*_DATA_FE bit setting for the falling edge of this clock

If the DMIC input is a mono signal, the data can be captured on either the rising or the falling edges of the DMIC input clock. If the DMIC input is a stereo signal, assign both DMIC inputs to the same DIO pad. The low data channel will be captured on the rising DMIC clock edge and the high data channel will be captured on the falling DMIC clock edge, with the mapping of DMIC input channels to input channels being assigned as appropriate to the user application. DMIC input timing is shown in Figure 17.

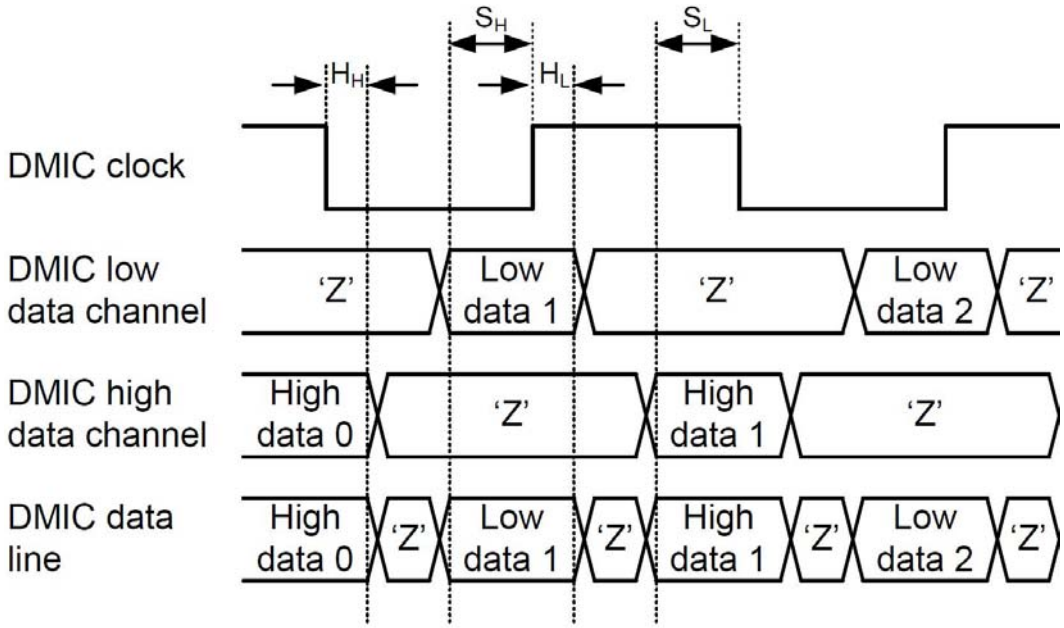


Figure 17. Relative DMIC input timing

10.7 Decimation Filtering

To obtain a final non-oversampled stream of input data, the decimation filters downsample and filter the oversampled signal from one of the following:

- ADCs
- Digital microphones
- Bypass data registers

This low-pass filtering is done using 11th-order wave digital filters. The decimation filters are enabled by setting the appropriate `ADC_DEC_CTRL_ADC_DECIMATOR_EBL` bits in the `A_ADC_DEC_CTRL_*` registers.

During the decimation filtering step, a configurable gain factor can be added to the signal from each of the system channels. To set these gains, select the appropriate value for the `ADC_DEC_CTRL_ADC_GF` bit field of the `A_ADC_DEC_CTRL_*` registers. Each of these four fields contains a 12-bit two's-complement signed value which can be used to configure a gain for the corresponding decimation filter output. The actual range of the gain depends on the value of `ADC_SAMPLE_FREQ_CFG_SAMPLE_FREQ`. The smallest range that can be configured is from -1 to $(1 - 2^{-11})$ when `ADC_SAMPLE_FREQ_CFG_SAMPLE_FREQ` is 0. Selecting a negative gain inverts the phase of the signal. A gain of 0 mutes the channel.

The gain that is added through this gain value is scaled by a factor based on the range of `ADC_SAMPLE_FREQ_CFG_SAMPLE_FREQ` as described in Table 22.

Table 22. Sample Frequency Gain Scaling Factor

Range for ADC_SAMPLE_FREQ_CFG_SAMPLE_FREQ	Value of SAMPLE_FREQ_SCALING_FACTOR
0 to 7	1
8 to 23	2
Greater than or equal to 24	4

We recommend that the gain value for each filter is set in such a way that the signal level, after downsampling and filtering, is the same as it was before downsampling and filtering (i.e., no gain or attenuation performed). These gain factors are useful to fine tune the gain realized in the input stage and are commonly used to compensate for changes in sampling frequency, or to match the gain between microphones in a multiple-microphone system. To obtain a signal that has no gain or attenuation added during this step, set the gain in the following equality to 1:

$$\text{GAIN} = \frac{\text{ADC_DEC_CTRL_ADC_GF} \times (\text{ADC_SAMPLE_FREQ_CFG_SAMPLE_FREQ} + 8)}{(16384 \times \text{SAMPLE_FREQ_SCALING_FACTOR})}$$

NOTE: Setting any of the ADC_DEC_CTRL_ADC_GF bit field values to zero effectively mutes the associated input channel.

The default value for the decimation filter gain factor, 0x665, results in approximately unity gain for the default ADC_SAMPLE_FREQ_CFG_SAMPLE_FREQ bit field configuration.

Although each ADC is only a 16-bit converter, the output generated by the decimation filter when downsampling the provided 16-bit oversampled data is 18-bit output data. As a result, all input data provided by the input stage to the system is handled as 18-bit data.

The decimation filter output can also be temporarily muted by setting the ADC_DEC_CTRL_MUTE bit from the A_ADC_DEC_CTRL_* registers. Muting the decimation filter in this way is useful in an application where an input channel is temporarily disabled, but you want to maintain the data ordering of the data in the input FIFOs. To use this feature:

1. Disable the decimation filter and mute the decimation filter output using the ADC_DEC_CTRL_MUTE and ADC_DEC_CTRL_DECIMATOR_EBL bits of the appropriate A_ADC_DEC_CTRL_* register.
2. Disable the associated ADC.

To re-enable the input channel:

1. Enable the ADC.
2. Enable the decimation filter (continuing to mute the output).
3. Delay at least 500 μs to allow the decimation filter to stabilize.
4. Unmute the decimation filter output.

If the analog input associated with the input channel was also disconnected when disabling the input channel, a user application should allow the input high pass filter to stabilize. In this case, we recommend implementing a delay of up to five times the RC constant, where RC is 500 kΩ times the capacitance of the input decoupling capacitor.

10.7.1 DC Removal Filters

The decimation filtering component of the input stage includes a high-pass filter for each channel designed to remove any DC signal component from the input signal belonging to each channel. The DC removal filters are based on first-order digital filter structures with programmable cut-off frequencies. To configure these filters, use the ADC_DEC_CTRL_DCRM_CUT_OFF bit field of the A_ADC_DEC_CTRL_* registers to select between cut-off frequencies of 5 Hz, 10 Hz, and 20 Hz, or to bypass the DC removal filter.

The filter cut-off frequency scales linearly with the sampling frequency, and cut-off frequencies for the filters are defined in terms of a 16 kHz sampling frequency. For sampling frequencies other than 16 kHz, scale the cut-off frequency along with any increase or decrease in sampling frequency. For example, the cut-off frequencies for a sampling frequency of 32 kHz are 10 Hz, 20 Hz and 40 Hz respectively.

10.7.2 Multi-Chip Synchronization

The decimation filters provide multi-chip synchronization by generating a synchronization signal that the remote chip can use to synchronize the filter of one Ezairo 7100 chip with other Ezairo chips. This signal, which can be transmitted or received on a multiplexed DIO pin, synchronizes the slow clock divider and decimation filters on the rising edge of the signal, allowing an application to ensure that all signals received are sampled at the same time.

Selecting the configuration DIO_MODE_SYNC_MASTER for the DIO_CFG_IO_MODE bit field for a specific DIO pad allows a chip to act as the master chip in a multi-chip synchronization configuration. In this configuration, the chip is producing an output signal to which other chips can be synchronized.

Configuring the D_SYNC_SLAVE_SRC register to select a DIO that has been configured to DIO_MODE_INPUT mode allows a chip to act as a slave chip in a multi-chip synchronization configuration. In this configuration, the chip is receiving and synchronizing to the signal provided by the chip using the “out” configuration.

For information on configuring the DIOs for multi-chip synchronization functionality, refer to Section 16.1, “Digital I/O (DIO)” on page 351.

IMPORTANT: When using multi-chip synchronization, it is essential that all chips that are being synchronized operate from the same system clock source. Typical configurations include using the system clock generated by one of the chips in the multi-chip configuration, or using an external clock source from all the chips as the source to SYSCLK through the external clock of the remaining chips. For more information about configurations that provide or receive an external clock, see Section 18.2.6, “External Clock (EXTCLK)” on page 448.

10.7.3 Frequency-Shift Keying

The decimation filters provide support for receiving data encoded with frequency-shift keying. This allows an interface to implement a receive-only remote control interface.

Frequency-shift keying is a form of signal modulation that typically uses signals at two discrete frequencies to transmit binary data. This feature, which is available only on input channels 2 and 3, allows you to define a signal band that you can use to limit the received signal to a specific transmission band without filtering out the frequency-shift keying carrier frequency.

The pass band for the frequency-shift keying feature is configurable in steps of one half of the configured sampling frequency up to four times the sampling frequency. The pass band for each of the band-pass filters is configurable using the ADC_DEC_CTRL_BAND_SELECT_ADC bit field from the A_ADC_DEC_CTRL_* registers.

NOTE: The supplied defines for the ADC_DEC_CTRL_BAND_SELECT_ADC bit fields have been specified in terms of a 16 kHz sampling frequency. If your application uses a different sampling frequency, appropriately scale the values used to define the pass-band to obtain the correct input band for the filter.

Support for handling frequency-shift keyed input data is not provided directly by the Ezairo 7100 chip. As a result either the CFX DSP or the HEAR Accelerator core has to perform any analysis of the frequency-shift keyed signal.

CAUTION: The band-pass filter of the Frequency Shift Keying feature does not function correctly unless the DC removal filter is configured for bypass operation.

10.7.4 Control and Configuration Registers

Register Name	Register Description	Address
A_ADC_DEC_CTRL_0	Decimation configuration registers	P:0xF148
A_ADC_DEC_CTRL_1	Decimation configuration registers	P:0xF149
A_ADC_DEC_CTRL_2	Decimation configuration registers	P:0xF14A
A_ADC_DEC_CTRL_3	Decimation configuration registers	P:0xF14B
D_MULTI_SYNC_CFG	Multi-chip synchronization configuration	P:0xF08D

For more information about decimation filtering registers, see Section 10.5.3, “Input Stage Control and Configuration Registers” on page 146.

10.7.4.1 A_ADC_DEC_CTRL_* Settings

The following bit fields and field names apply equally to registers A_ADC_DEC_CTRL_0 to A_ADC_DEC_CTRL_3.

Bit Field	Field Name	Description
31	MUTE	Mute output and enable data ready signal
30:28	BAND_SELECT_ADC	Indicate which band is filtered in the decimation filter
26:24	DELAY_INTEGER	Integer sample delay
21:16	DELAY_FRACTIONAL	Fractional sample delay
15	DECIMATOR_EBL	Enables ADC decimation
14:12	DCRM_CUT_OFF	Control the DC removal filter cut-off frequency of channel
11:0	ADC_GF	Gain factor for ADC

Field Name	Value Symbol	Value Description	Hex Value
MUTE	ADC_UNMUTE	ADC output is not muted	0x0*
	ADC_MUTE	ADC output is muted and data ready enabled	0x1

Field Name	Value Symbol	Value Description	Hex Value
BAND_SELECT_ADC	BAND_SELECT_ADC_0K_8K	Select the 0 to 8 kHz band	0x0*
	BAND_SELECT_ADC_8K_16K	Select the 8 to 16 kHz band	0x1
	BAND_SELECT_ADC_16K_24K	Select the 16 to 24 kHz band	0x2
	BAND_SELECT_ADC_24K_32K	Select the 24 to 32 kHz band	0x3
	BAND_SELECT_ADC_32K_40K	Select the 32 to 40 kHz band	0x4
	BAND_SELECT_ADC_40K_48K	Select the 40 to 48 kHz band	0x5
	BAND_SELECT_ADC_48K_56K	Select the 48 to 56 kHz band	0x6
	BAND_SELECT_ADC_56K_64K	Select the 56 to 64 kHz band	0x7
DELAY_INTEGER	ADC_INTEGER_DELAY_0	No integer sampling delay	0x0*
	ADC_INTEGER_DELAY_1	Delay the sampling by 1/8 of the sampling frequency	0x1
	ADC_INTEGER_DELAY_2	Delay the sampling by 2/8 of the sampling frequency	0x2
	ADC_INTEGER_DELAY_3	Delay the sampling by 3/8 of the sampling frequency	0x3
	ADC_INTEGER_DELAY_4	Delay the sampling by 4/8 of the sampling frequency	0x4
	ADC_INTEGER_DELAY_5	Delay the sampling by 5/8 of the sampling frequency	0x5
	ADC_INTEGER_DELAY_6	Delay the sampling by 6/8 of the sampling frequency	0x6
	ADC_INTEGER_DELAY_7	Delay the sampling by 7/8 of the sampling frequency	0x7
DECIMATOR_EBL	ADC_DEC_DISABLE	ADC decimation disabled	0x0*
	ADC_DEC_ENABLE	ADC decimation enabled	0x1
DCRM_CUT_OFF	ADC_DC_REMOVE_CUTOFF_5HZ	Cut-off frequency at 5 Hz	0x0
	ADC_DC_REMOVE_CUTOFF_10HZ	Cut-off frequency at 10 Hz	0x1
	ADC_DC_REMOVE_CUTOFF_20HZ	Cut-off frequency at 20 Hz	0x2*
	ADC_DC_REMOVE_CUTOFF_40HZ	Cut-off frequency at 40 Hz	0x3
	ADC_DC_REMOVE_CUTOFF_80HZ	Cut-off frequency at 80 Hz	0x4
	ADC_DC_REMOVE_CUTOFF_160HZ	Cut-off frequency at 160 Hz	0x5
	ADC_DC_REMOVE_CUTOFF_BYPASS	No DC remove	0x6

10.7.4.2 D_MULTI_SYNC_CFG Settings

Bit Field	Field Name	Description
1:0	SLAVE_CFG	Multi-chip slave configuration

Field Name	Value Symbol	Value Description	Hex Value
SLAVE_CFG	MULT_SYNC_SLAVE_OFF	Multi-sync slave feature disabled	0x0*
	MULT_SYNC_SLAVE_SYSCLK	Multi-sync slave enabled; master chip runs on SYSCLK	0x1
	MULT_SYNC_SLAVE_SLOWCLK	Multi-sync slave enabled; master chip runs on SLOWCLK	0x2
	MULT_SYNC_SLAVE_SDMCLK	Multi-sync slave enabled; master chip runs on SDMCLK	0x3

10.8 Input Dynamic Range Extension

10.8.1 Description

The four input channels on Ezairo 7100 can be combined into pairs to increase the input dynamic range available to an application. This can be used to avoid design trade-offs between the sufficient amplification of low-level input signals and the saturation of high-level signals. In principle, the input dynamic range extension (IDRX) feature selects between the two ADCs based on variations in the energy of an incoming signal that is provided to both ADCs. The IDRX block switches the source between the two ADCs, selecting each ADC based on the variation of energy of the incoming signal. In parallel, the preamplifiers of both ADCs are also changed based on the energy of this input signal to continually keep the signal in an optimal range for the ADCs to sense (whenever possible).

The extension of the input dynamic range provides a single channel of 24-bit input data from each pair of inputs by combining channel 0 with 1 and channel 2 with 3. This is achieved by shifting the levels of the input signals to the ADCs to center the signal to their 16-bit range by:

- Lowering the preamplifier gain applied to the signal as the signal level rises
- Raising the preamplifier gain applied to the signal as the signal level falls

The IDRX block then combines the 18-bit input data provided from the ADCs at the output of the decimation filters into the IDRX 24-bit signed input data that is provided to the IOC or Filter Engine for processing in the rest of the Ezairo 7100 system.

When configured to provide an extended input dynamic range, the following settings are controlled internally and overwritten by the hardware:

- The gain setting for the preamplifier belonging to each component input channel
- The decimation filter gain factor setting belonging to each component input channel
- The sampling delay associated with channel 0 for IDRX0, and channel 2 for IDRX1

To use a dynamic range extended input channel, the inputs to that channel must be connected together externally to the Ezairo 7100 chip. However, a separate external coupling capacitor is still needed for each of the inputs connected this way, as the input impedances will differ between channels based on the preamplifier gain configuration selected by the system for each component input channel.

IMPORTANT: If the DC removal filters provided by the decimation filters for either component channel of a dynamic range extended input channel are disabled, the combined input signal channel will be corrupted.

10.8.2 Configuration

The IDRX_CFG_ENABLE bit from the A_IDRX_CFG_* registers allows you to selectively enable or disable the use of these channel pairs as dynamic range extended input stage channels.

Configuration of the extended dynamic range input channels is performed through the A_IDRX_CFG_* configuration registers. These registers provide bit fields that allow an application to configure the timers, signal limits and available preamp gain settings belonging to the underlying input controller for each extended dynamic range input channel.

The IDRX block only uses gains in a specified range. This range can be limited to providing a minimum gain of 0 dB to 18 dB (configurable in 6 dB steps), and a maximum gain of 18 dB to 36 dB (configurable in 6 dB steps). The minimum preamplifier gain can be set using the IDRX_CFG_MIN_PG bit field in the A_IDRX_CFG_* registers. Similarly, the maximum preamplifier gain can be set using the IDRX_CFG_MAX_PG bit field in the A_IDRX_CFG_* registers.

The IDRX_CFG_CHANNEL_SEL bit from the IDRX configuration registers indicates the underlying channel that is currently providing the data to the combined input channel. To configure the combined input channel to disable an ADC from an input channel when it is not providing data to the combined channel, set the IDRX_CFG_ADC_OFF_EN bit in the configuration register.

The channel switching can be further configured with two threshold limits and two timers. The threshold limits are:

- IDRX_CFG_HIGH_LIMIT defines the high limit threshold that the signal must reach before beginning the channel switching process to a channel with a lower preamplifier gain.
- IDRX_CFG_LOW_LIMIT defines the low limit threshold that the signal must reach before beginning the channel switching process to a channel with a higher preamplifier gain.

These settings control the gain setting and switching between channels. To set the high limit, set the IDRX_CFG_HIGH_LIMIT bit field from the A_IDRX_CFG_* registers. To set the low limit, set the IDRX_CFG_LOW_LIMIT bit field from the A_IDRX_CFG_* registers. The limit values are always relative to the active channel which has a maximum range of 2^{17} (based on the 18-bit data provided from the decimation filter). Relative to this full scale, the high limit is set according to the following equation:

$$\text{HIGH_LIMIT in dB} = 20 \times \log\left(\frac{(\text{IDRX_CFG_HIGH_LIMIT} \times 4096) + 2048}{2^{17}}\right)$$

Similarly, relative to this full scale, the low limit is set according to the following equation:

$$\text{LOW_LIMIT in dB} = 20 \times \log\left(\frac{\text{IDRX_CFG_LOW_LIMIT} \times 1024}{2^{17}}\right)$$

To prevent endless switching, the difference between the high and low thresholds has to be at least 12 dB if a preamplifier gain of 0 dB is allowed. If the 0 dB preamplifier gain setting is not used, the difference between the thresholds must be at least 6 dB.

The timers used for IDRX are:

- IDRX_CFG_TIMER_1 defines the numbers of samples a signal needs to be above the high threshold limit or below the low threshold limit before the gain of the inactive channel is switched.
- IDRX_CFG_TIMER_2 defines the numbers of samples to be used as a delay after the gain of the inactive channel is switched and before the newly configured channels is selected as the source for the IDRX channel.

IMPORTANT: We recommend that the duration of timer 2 be longer than the preamplifier stabilization time. If this is not the case, the automatic channel switch can cause undesired audio artifacts. As a result, this delay should typically be set to exceed 125 ms.

Each of these timers can be configured for a delay that is a multiple of 256 times the sampling rate. To set the delay for timer 1, set the `IDRX_CFG_TIMER_1` bit field in the `A_IDRX_CFG_*` registers. To set the delay for timer 2, set the `IDRX_CFG_TIMER_2` bit field in the `A_IDRX_CFG_*` registers.

NOTE: If the preamplifier value does not have to be switched because it is already configured to provide the desired gain, timer 2 will not start, and the active channel is switched immediately when timer 1 expires. This situation occurs, for example, when the input signal decreases in intensity, producing a gain and channel switch, before immediately increasing in intensity.

To assist in situations that could result in saturation of the input signals, the IDRX block can be configured to always set the unused channel for a lower gain to reduce the time needed to switch from higher gains to lower gains (as timer 2 is not run). To select this configuration, set the `IDRX_CFG_LOW_GAIN_EN` bit from the `A_IDRX_CFG_*` registers.

10.8.3 Control and Configuration Registers

Register Name	Register Description	Address
`A_IDRX_CFG_0`	IDRX configuration registers	P:0xF14C
`A_IDRX_CFG_1`	IDRX configuration registers	P:0xF14D

10.8.3.1 A_IDRX_CFG_* Settings

The following bit fields and field names apply equally to registers `A_IDRX_CFG_0` and `A_IDRX_CFG_1`.

Bit Field	Field Name	Description
31	`CHANNEL_SEL`	Indicates which ADC channel is currently used by the IDRX
30	`ADC_OFF_EN`	Disables the unused AD to save power
29	`LOW_GAIN_EN`	Prepares the unused channel with lower gain
28	`ENABLE`	Enables DRX on channels 0 and 1 or 2 and 3
27:26	`MAX_PG`	max_pg indicates range used by the DRX
25:24	`MIN_PG`	min_pg indicates range used by the DRX
23:20	`HIGH_LIMIT`	Gain switching limit for up gain changed
16:12	`LOW_LIMIT`	Gain switching limit for down gain changed
11:6	`TIMER2`	Stabilization time of preamplifier (delay before the modified channel is ready)
5:0	`TIMER1`	Delay time before PG switches (delay before PG changes)

Field Name	Value Symbol	Value Description	Hex Value
CHANNEL_SEL	IDRX_CHANNEL_LOW	IDRX is using ADC0 or ADC2	0x0
	IDRX_CHANNEL_HIGH	IDRX is using ADC1 or ADC3	0x1*
ADC_OFF_EN	IDRX_INACTIVE_ADC_ENABLED	Does not disable any ADC	0x0*
	IDRX_INACTIVE_ADC_DISABLED	Inactive ADC is disabled	0x1
LOW_GAIN_EN	IDRX_LOW_GAIN_DISABLED	Uses the default gain settings	0x0*
	IDRX_LOW_GAIN_ENABLED	Prepares the unused channel with the lower gain for faster high to low gain switching	0x1
ENABLE	IDRX_DISABLE	IDRX disabled	0x0*
	IDRX_ENABLE	IDRX enabled	0x1
MAX_PG	IDRX_MAX_PREAMP_GAIN_36DB	Maximum preamplifier setting available is 36 dB	0x3
	IDRX_MAX_PREAMP_GAIN_30DB	Maximum preamplifier setting available is 30 dB	0x2
	IDRX_MAX_PREAMP_GAIN_24DB	Maximum preamplifier setting available is 24 dB	0x1
	IDRX_MAX_PREAMP_GAIN_18DB	Maximum preamplifier setting available is 18 dB	0x0*
MIN_PG	IDRX_MIN_PREAMP_GAIN_18DB	Minimum preamplifier setting available is 18 dB	0x3
	IDRX_MIN_PREAMP_GAIN_12DB	Minimum preamplifier setting available is 12 dB	0x2
	IDRX_MIN_PREAMP_GAIN_6DB	Minimum preamplifier setting available is 6 dB	0x1
	IDRX_MIN_PREAMP_GAIN_0DB	Minimum preamplifier setting available is 0 dB	0x0*
HIGH_LIMIT	IDRX_HIGH_LIMIT_6_3DB	High limit at -6.3 dB	0xF
	IDRX_HIGH_LIMIT_6_9DB	High limit at -6.9 dB	0xE
	IDRX_HIGH_LIMIT_7_5DB	High limit at -7.5 dB	0xD
	IDRX_HIGH_LIMIT_8_2DB	High limit at -8.2 dB	0xC
	IDRX_HIGH_LIMIT_8_9DB	High limit at -8.9 dB	0xB
	IDRX_HIGH_LIMIT_9_7DB	High limit at -9.7 dB	0xA
	IDRX_HIGH_LIMIT_10_5DB	High limit at -10.5 dB	0x9
	IDRX_HIGH_LIMIT_11_5DB	High limit at -11.5 dB	0x8
	IDRX_HIGH_LIMIT_12_6DB	High limit at -12.6 dB	0x7*
	IDRX_HIGH_LIMIT_13_8DB	High limit at -13.8 dB	0x6
	IDRX_HIGH_LIMIT_15_3DB	High limit at -15.3 dB	0x5
	IDRX_HIGH_LIMIT_17_0DB	High limit at -17.0 dB	0x4
	IDRX_HIGH_LIMIT_19_2DB	High limit at -19.2 dB	0x3
	IDRX_HIGH_LIMIT_22_1DB	High limit at -22.1 dB	0x2
	IDRX_HIGH_LIMIT_26_6DB	High limit at -26.6 dB	0x1
	IDRX_HIGH_LIMIT_36_1DB	High limit at -36.1 dB	0x0

Field Name	Value Symbol	Value Description	Hex Value
LOW_LIMIT	IDRX_LOW_LIMIT_12_3DB	Low limit at -12.3 dB	0x1F
	IDRX_LOW_LIMIT_12_6DB	Low limit at -12.6 dB	0x1E
	IDRX_LOW_LIMIT_12_9DB	Low limit at -12.9 dB	0x1D
	IDRX_LOW_LIMIT_13_2DB	Low limit at -13.2 dB	0x1C
	IDRX_LOW_LIMIT_13_5DB	Low limit at -13.5 dB	0x1B
	IDRX_LOW_LIMIT_13_8DB	Low limit at -13.8 dB	0x1A
	IDRX_LOW_LIMIT_14_2DB	Low limit at -14.2 dB	0x19
	IDRX_LOW_LIMIT_14_5DB	Low limit at -14.5 dB	0x18
	IDRX_LOW_LIMIT_14_9DB	Low limit at -14.9 dB	0x17
	IDRX_LOW_LIMIT_15_3DB	Low limit at -15.3 dB	0x16
	IDRX_LOW_LIMIT_15_7DB	Low limit at -15.7 dB	0x15
	IDRX_LOW_LIMIT_16_1DB	Low limit at -16.1 dB	0x14
	IDRX_LOW_LIMIT_16_6DB	Low limit at -16.6 dB	0x13
	IDRX_LOW_LIMIT_17_0DB	Low limit at -17.0 dB	0x12
	IDRX_LOW_LIMIT_17_5DB	Low limit at -17.5 dB	0x11
	IDRX_LOW_LIMIT_18_1DB	Low limit at -18.1 dB	0x10
	IDRX_LOW_LIMIT_18_6DB	Low limit at -18.6 dB	0xF
	IDRX_LOW_LIMIT_19_2DB	Low limit at -19.2 dB	0xE*
	IDRX_LOW_LIMIT_19_9DB	Low limit at -19.9 dB	0xD
	IDRX_LOW_LIMIT_20_6DB	Low limit at -20.6 dB	0xC
	IDRX_LOW_LIMIT_21_3DB	Low limit at -21.3 dB	0xB
	IDRX_LOW_LIMIT_22_1DB	Low limit at -22.1 dB	0xA
	IDRX_LOW_LIMIT_23_1DB	Low limit at -23.1 dB	0x9
	IDRX_LOW_LIMIT_24_1DB	Low limit at -24.1 dB	0x8
	IDRX_LOW_LIMIT_25_2DB	Low limit at -25.2 dB	0x7
	IDRX_LOW_LIMIT_26_6DB	Low limit at -26.6 dB	0x6
	IDRX_LOW_LIMIT_28_2DB	Low limit at -28.2 dB	0x5
	IDRX_LOW_LIMIT_30_1DB	Low limit at -30.1 dB	0x4
	IDRX_LOW_LIMIT_32_6DB	Low limit at -32.6 dB	0x3
	IDRX_LOW_LIMIT_36_1DB	Low limit at -36.1 dB	0x2
	IDRX_LOW_LIMIT_42_1DB	Low limit at -42.1 dB	0x1
TIMER2	IDRX_TIMER2_16384	16384 samples	0x3F
	IDRX_TIMER2_16128	16128 samples	0x3E
	IDRX_TIMER2_15872	15872 samples	0x3D
	IDRX_TIMER2_15616	15616 samples	0x3C
	IDRX_TIMER2_15360	15360 samples	0x3B
	IDRX_TIMER2_15104	15104 samples	0x3A
	IDRX_TIMER2_14848	14848 samples	0x39

Field Name	Value Symbol	Value Description	Hex Value
TIMER2 (continued)	IDRX_TIMER2_14592	14592 samples	0x38
	IDRX_TIMER2_14336	14336 samples	0x37
	IDRX_TIMER2_14080	14080 samples	0x36
	IDRX_TIMER2_13824	13824 samples	0x35
	IDRX_TIMER2_13568	13568 samples	0x34
	IDRX_TIMER2_13312	13312 samples	0x33
	IDRX_TIMER2_13056	13056 samples	0x32
	IDRX_TIMER2_12800	12800 samples	0x31
	IDRX_TIMER2_12544	12544 samples	0x30
	IDRX_TIMER2_12288	12288 samples	0x2F
	IDRX_TIMER2_12032	12032 samples	0x2E
	IDRX_TIMER2_11776	11776 samples	0x2D
	IDRX_TIMER2_11520	11520 samples	0x2C
	IDRX_TIMER2_11264	11264 samples	0x2B
	IDRX_TIMER2_11008	11008 samples	0x2A
	IDRX_TIMER2_10752	10752 samples	0x29
	IDRX_TIMER2_10496	10496 samples	0x28
	IDRX_TIMER2_10240	10240 samples	0x27
	IDRX_TIMER2_9984	9984 samples	0x26
	IDRX_TIMER2_9728	9728 samples	0x25
	IDRX_TIMER2_9472	9472 samples	0x24
	IDRX_TIMER2_9216	9216 samples	0x23
	IDRX_TIMER2_8960	8960 samples	0x22
	IDRX_TIMER2_8704	8704 samples	0x21
	IDRX_TIMER2_8448	8448 samples	0x20
	IDRX_TIMER2_8192	8192 samples	0x1F
	IDRX_TIMER2_7936	7936 samples	0x1E
	IDRX_TIMER2_7680	7680 samples	0x1D
	IDRX_TIMER2_7424	7424 samples	0x1C
	IDRX_TIMER2_7168	7168 samples	0x1B
	IDRX_TIMER2_6912	6912 samples	0x1A
	IDRX_TIMER2_6656	6656 samples	0x19
	IDRX_TIMER2_6400	6400 samples	0x18
	IDRX_TIMER2_6144	6144 samples	0x17
	IDRX_TIMER2_5888	5888 samples	0x16
	IDRX_TIMER2_5632	5632 samples	0x15
	IDRX_TIMER2_5376	5376 samples	0x14
	IDRX_TIMER2_5120	5120 samples	0x13

Field Name	Value Symbol	Value Description	Hex Value
TIMER2 (continued)	IDRX_TIMER2_4864	4864 samples	0x12
	IDRX_TIMER2_4608	4608 samples	0x11
	IDRX_TIMER2_4352	4352 samples	0x10
	IDRX_TIMER2_4096	4096 samples	0xF
	IDRX_TIMER2_3840	3840 samples	0xE
	IDRX_TIMER2_3584	3584 samples	0xD
	IDRX_TIMER2_3328	3328 samples	0xC
	IDRX_TIMER2_3072	3072 samples	0xB
	IDRX_TIMER2_2816	2816 samples	0xA
	IDRX_TIMER2_2560	2560 samples	0x9
	IDRX_TIMER2_2304	2304 samples	0x8*
	IDRX_TIMER2_2048	2048 samples	0x7
	IDRX_TIMER2_1792	1792 samples	0x6
	IDRX_TIMER2_1536	1536 samples	0x5
	IDRX_TIMER2_1280	1280 samples	0x4
	IDRX_TIMER2_1024	1024 samples	0x3
	IDRX_TIMER2_768	768 samples	0x2
	IDRX_TIMER2_512	512 samples	0x1
	IDRX_TIMER2_256	256 samples	0x0
TIMER1	IDRX_TIMER1_16384	16384 samples	0x3F
	IDRX_TIMER1_16128	16128 samples	0x3E
	IDRX_TIMER1_15872	15872 samples	0x3D
	IDRX_TIMER1_15616	15616 samples	0x3C
	IDRX_TIMER1_15360	15360 samples	0x3B
	IDRX_TIMER1_15104	15104 samples	0x3A
	IDRX_TIMER1_14848	14848 samples	0x39
	IDRX_TIMER1_14592	14592 samples	0x38
	IDRX_TIMER1_14336	14336 samples	0x37
	IDRX_TIMER1_14080	14080 samples	0x36
	IDRX_TIMER1_13824	13824 samples	0x35
	IDRX_TIMER1_13568	13568 samples	0x34
	IDRX_TIMER1_13312	13312 samples	0x33
	IDRX_TIMER1_13056	13056 samples	0x32
	IDRX_TIMER1_12800	12800 samples	0x31
	IDRX_TIMER1_12544	12544 samples	0x30
	IDRX_TIMER1_12288	12288 samples	0x2F
	IDRX_TIMER1_12032	12032 samples	0x2E
	IDRX_TIMER1_11776	11776 samples	0x2D

Field Name	Value Symbol	Value Description	Hex Value
TIMER1 (continued)	IDRX_TIMER1_11520	11520 samples	0x2C
	IDRX_TIMER1_11264	11264 samples	0x2B
	IDRX_TIMER1_11008	11008 samples	0x2A
	IDRX_TIMER1_10752	10752 samples	0x29
	IDRX_TIMER1_10496	10496 samples	0x28
	IDRX_TIMER1_10240	10240 samples	0x27
	IDRX_TIMER1_9984	9984 samples	0x26
	IDRX_TIMER1_9728	9728 samples	0x25
	IDRX_TIMER1_9472	9472 samples	0x24
	IDRX_TIMER1_9216	9216 samples	0x23
	IDRX_TIMER1_8960	8960 samples	0x22
	IDRX_TIMER1_8704	8704 samples	0x21
	IDRX_TIMER1_8448	8448 samples	0x20
	IDRX_TIMER1_8192	8192 samples	0x1F
	IDRX_TIMER1_7936	7936 samples	0x1E
	IDRX_TIMER1_7680	7680 samples	0x1D
	IDRX_TIMER1_7424	7424 samples	0x1C
	IDRX_TIMER1_7168	7168 samples	0x1B
	IDRX_TIMER1_6912	6912 samples	0x1A
	IDRX_TIMER1_6656	6656 samples	0x19
	IDRX_TIMER1_6400	6400 samples	0x18
	IDRX_TIMER1_6144	6144 samples	0x17
	IDRX_TIMER1_5888	5888 samples	0x16
	IDRX_TIMER1_5632	5632 samples	0x15
	IDRX_TIMER1_5376	5376 samples	0x14
	IDRX_TIMER1_5120	5120 samples	0x13
	IDRX_TIMER1_4864	4864 samples	0x12
	IDRX_TIMER1_4608	4608 samples	0x11
	IDRX_TIMER1_4352	4352 samples	0x10
	IDRX_TIMER1_4096	4096 samples	0xF
	IDRX_TIMER1_3840	3840 samples	0xE
	IDRX_TIMER1_3584	3584 samples	0xD
	IDRX_TIMER1_3328	3328 samples	0xC
	IDRX_TIMER1_3072	3072 samples	0xB
	IDRX_TIMER1_2816	2816 samples	0xA
	IDRX_TIMER1_2560	2560 samples	0x9
	IDRX_TIMER1_2304	2304 samples	0x8
	IDRX_TIMER1_2048	2048 samples	0x7

Field Name	Value Symbol	Value Description	Hex Value
TIMER1 (continued)	IDRX_TIMER1_1792	1792 samples	0x6
	IDRX_TIMER1_1536	1536 samples	0x5
	IDRX_TIMER1_1280	1280 samples	0x4
	IDRX_TIMER1_1024	1024 samples	0x3*
	IDRX_TIMER1_768	768 samples	0x2
	IDRX_TIMER1_512	512 samples	0x1
	IDRX_TIMER1_256	256 samples	0x0

CHAPTER 11
Output Stage

11.1 Output Stage Architecture

The output stage of Ezairo 7100 provides two audio output channels that post-process signal data from the rest of the Ezairo 7100 system, and provide it to external receivers or speakers. The output channels include:

- High-quality interpolation filtering that automatically tracks the selected sample rate of the sigma-delta modulator
- Ultra low-power, high fidelity, over-sampled sigma-delta D/A conversion with programmable sample rates
- A direct digital output driver for zero-bias hearing aid receivers and standard loudspeakers with a selectable high-power mode that combines multiple output pads for maximum output signal power
- Selectable digital microphone outputs that can sink data in parallel with either output channel

Each of these components of the output channel is shown in Figure 18, which shows the overall architecture of the output stage.

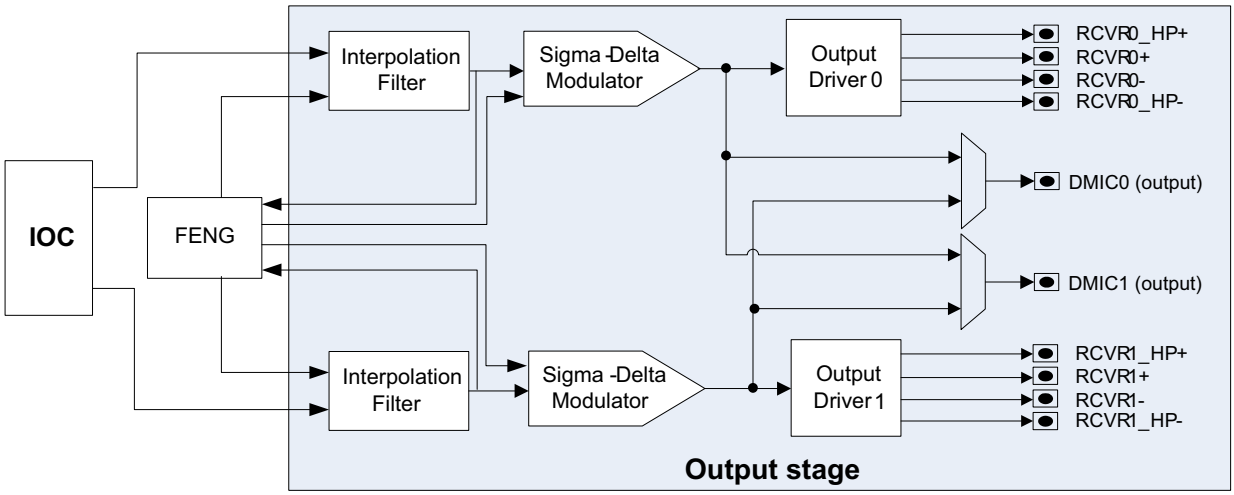


Figure 18. Output Stage Block Diagram

11.2 Interpolation Filtering

The output stage contains an interpolation filter for each output channel which upsamples and filters the signal that is being passed to the channel's sigma-delta modulator. This filter is part of the decimation filtering block of the input stage and is based on an 11th-order low-pass wave digital filter structure. As a result of this link to the decimation filters, the interpolation filters automatically track the selected sample rate and upsampled frequency of the ADCs. For more information about configuring the sampling rate of the ADCs (and hence the decimation filters upsampled frequency), see Section 10.5, "Analog-to-Digital Converters (ADCs)" on page 144.

The interpolation filter component of each output channel provides the ability to apply a gain to the output signals. By default, this gain is disabled; however a user application can configure the `O_DAC_GF_CTRL_DA*_CTRL` bit fields in the `A_O_DAC_GF_CTRL` register to change this setting. These bit fields allow you to select whether the gain is enabled/disabled, or whether the application uses the setting of a DIO to enable/disable the gain. If the system is configured to use a DIO to control the application of gain, configure the `D_DAC_SCALE_SRC` register to select a

DIO that is configured as an input, using an I/O mode of DIO_MODE_INPUT for its DIO_CFG_IO_MODE bit field. For more information about configuring DIOs for alternate functionality, see Section 16.1, “Digital I/O (DIO)” on page 351.

The amount of gain applied by the interpolation filters is configured using the O_DAC_GF_CTRL_DA*_GF bit fields in the A_O_DAC_GF_CTRL register. These bit fields use 12-bit signed values, with 2 integer bits and 10 fractional bits. This can be used to configure a gain in the range -2 to $(2 - 2^{-10})$. The default setting of 0x400 for this bit field corresponds to the application of no gain across the interpolation filter.

Similar to the input stage, the output stage uses 18-bit data supplied to each interpolation filter to produce the 16-bit data provided to each channel’s sigma-delta converter.

The output data directed to each interpolation filter is controlled by the IOC and its audio multiplexers. For more information, see Section 9.5, “IOC Output” on page 130.

11.2.1 Control and Configuration Registers

Register Name	Register Description	Address
A_O_DAC_GF_CTRL	DAC gain factor settings	P:0xF152

11.2.1.1 A_O_DAC_GF_CTRL Settings

Bit Field	Field Name	Description
29:28	DA1_CTRL	Gain factor control for DAC1
27:16	DA1_GF	Gain factor for DAC1
13:12	DA0_CTRL	Gain factor control for DAC0
11:0	DA0_GF	Gain factor for DAC0

Field Name	Value Symbol	Value Description	Hex Value
DA1_CTRL	O_DAC1_GF_CTRL_DISABLE	DAC gain factor disabled	0x0*
	O_DAC1_GF_CTRL_ENABLE	DAC gain factor enabled, no control	0x2
	O_DAC1_GF_CTRL_BY_DIO	DAC gain factor enabled, controlled by DIO through DAC_SCALE signal (see Section 16.1, “Digital I/O (DIO)” on page 351)	0x3
DA0_CTRL	O_DAC0_GF_CTRL_DISABLE	DAC gain factor disabled	0x0*
	O_DAC0_GF_CTRL_ENABLE	DAC gain factor enabled, no control	0x2
	O_DAC0_GF_CTRL_BY_DIO	DAC gain factor enabled, controlled by DIO through DAC_SCALE signal (see Section 16.1, “Digital I/O (DIO)” on page 351)	0x3

11.3 Sigma-Delta Modulator

Ezairo 7100 includes a third-order three-level sigma-delta modulator for each of the output channels to produce a pulse density modulated (PDM) output signal from the oversampled signal data. The oversampled signal data is provided by either the interpolation filter or the Filter Engine. For information about selecting the data source used for the sigma-delta modulators, see Section 9.1.1, “Audio Multiplexing with Filter Engine Outputs” on page 113.

The sigma-delta modulators are clocked using SDMCLK. For more information about this prescaled clock source, see Section 18.2.3, “Sigma-Delta Modulator Clock (SDMCLK)” on page 446.

When the oversampled signal is received from the interpolation filters by each sigma-delta modulator, it is quantized to 1 bit by the modulator. This process shapes the noise of the output signal. As a result, the noise floor is no longer flat; instead, it is flat for a pass-band before the noise climbs at higher frequencies. The frequency band where the noise floor is flat depends on the frequency of SDMCLK. The useful range for the modulator, where the noise floor remains flat, ranges from a lower bound of 20 Hz to an upper bound defined by:

$$F_{\text{UPPER BOUND}} = \frac{F_{\text{SDMCLK}}}{128}$$

The outputs of this modulator OD*- and OD*+ can be configured to toggle on either the rising or falling edge of this SDMCLK using the OUTPUT_CTRL_OD*_NEG_CLK_EDGE and OUTPUT_CTRL_OD*_POS_CLK_EDGE bits from the A_OUTPUT_CTRL register. For example, to configure the ODO- output to toggle on the falling edge of the clock, set the OUTPUT_CTRL_OD0_NEG_CLK_EDGE bit to OD0_NEG_EDGE_FALLING.

11.3.1 Advanced Sigma-Delta Modulator Configuration

The sigma-delta modulator includes several advanced configuration options that can be used to adjust the behavior of the output modulator.

IMPORTANT: We recommend using the default configurations for this register in most cases. However if your algorithm mutes the output by overwriting the output FIFO with a specific value, or otherwise creates an imbalanced output, adjusting the modulator configuration might improve the performance of your device.

Advanced configuration features of the sigma-delta modulators include:

Intermediate Data Truncation

The third-order sigma delta modulators have three internal stages that are used to create the output signal. The intermediate data between these stages can be truncated to the input data width, or the intermediate data can be passed through without truncation. To configure the intermediate data truncation, set or clear the O_SDM_CTRL_TRUNCATE_EN bit from the A_O_SDM_CTRL register.

Output Delay

The output from the sigma-delta modulator can be configured to provide an output to both the high and low sides of the digital output driver at the same time, or to delay one side by a cycle. Delaying one half of the output modulator signal relative to the other typically improves the noise performance of the system, and reduces the power consumption in the output stage. To configure the output delay, set or clear the O_SDM_CTRL_OD_DELAY bit from the A_O_SDM_CTRL register.

NOTE: Using the output state control advanced frequency-based clock throttling feature requires a delay between the outputs of the sigma-delta modulator. If the O_SDM_CTRL_OD_DELAY bit is not set, this version of clock throttling will never update the clock frequency. For more information, see Section 18.3.2.1, “Advanced Frequency-Based Clock Throttling Options” on page 451.

Dithering

The output signals from the sigma-delta modulator can create a periodic signal due to the quantization of the signal. To eliminate this possible issue, a dithering stage can be added to the output of the sigma-delta modulator to randomize the quantization error. To enable dithering, set the O_SDM_CTRL_DITHER_ENABLE bit from the A_O_SDM_CTRL register.

DC Removal

The sigma-delta modulator includes a DC removal filter at the input to the modulator. This filter is used to remove the effects of DC offsets due to quantizations introduced in user applications and in the interpolation filter. To enable the DC removal filter, set the O_SDM_CTRL_RCRM_ENABLE bit from the A_O_SDM_CTRL register.

Output Offsets

The sigma-delta modulator can introduce a DC offset to the input signal after DC removal. This offset is divided into two parts: a small offset and a large offset.

The small offset can be used to correct for a known DC offset by adding an additional offset. This can be used with the DC removal filter, but is more commonly used in place of the DC removal filter. The small offset is configured using the O_SDM_CTRL_S_OFFSET bit field from the A_O_SDM_CTRL register. A DC offset of (O_SDM_CTRL_S_OFFSET/8) LSBs is added to the input signal prior to processing through the sigma-delta modulator.

The large offset can be used to shift the frequency of undesired periodic signals generated in the sigma-delta modulator out of the audio band. The large offset is configured using the O_SDM_CTRL_L_OFFSET bit field from the A_O_SDM_CTRL register. A DC offset of (O_SDM_CTRL_L_OFFSET × 8) LSBs is added to the input signal prior to processing through the sigma-delta modulator.

11.3.1.1 Control and Configuration Registers

Register Name	Register Description	Address
A_O_SDM_CTRL	Additional sigma-delta modulator configuration	P:0xF153

11.3.1.1.1 A_O_SDM_CTRL

Bit Field	Field Name	Description
17	TRUNCATE_EN	Truncates data between the successive SDM stages
16	OD_DELAY	Delay on the RCVR
14:8	L_OFFSET	Larger DC offset added to the sigma-delta modulator input
7	DITHER_ENABLE	Dither enable
6	DCRM_ENABLE	DC remove enable
5:0	S_OFFSET	Small DC offset added to the sigma-delta modulator input

Field Name	Value Symbol	Value Description	Hex Value
TRUNCATE_EN	SDM_TRUNCATE_DISABLE	Full data resolution is used	0x0*
	SDM_TRUNCATE_ENABLE	Data is truncated between stages	0x1

Field Name	Value Symbol	Value Description	Hex Value
OD_DELAY	OD_DELAY_DISABLE	Two-state output mode	0x0
	OD_DELAY_ENABLE	Three-state output mode	0x1*
L_OFFSET	L_OFFSET_NONE	No large offset added	0x0*
	L_OFFSET_64	Offset of 64/8 LSBs added	0x1
	L_OFFSET_128	Offset of 128/8 LSBs added	0x2
	L_OFFSET_8128	Offset of 8128/8 LSBs added	0x7F
DITHER_ENABLE	DITHER_DISABLE	Output dithering disabled	0x0
	DITHER_ENABLE	Output dithering enabled	0x1*
DCRM_ENABLE	DC_REMOVE_DISABLE	DC not removed from the output	0x0
	DC_REMOVE_ENABLE	DC removed from the output	0x1*
S_OFFSET	S_OFFSET_NONE	No small offset added	0x0*
	S_OFFSET_1_8	Offset of 1/8 LSB added	0x1
	S_OFFSET_2_8	Offset of 2/8 LSBs added	0x2
	S_OFFSET_63_8	Offset of 63/8 LSBs added	0x3F

11.4 DIRECT-DIGITAL OUTPUT DRIVER

The direct-digital output drivers provided by the Ezairo 7100 chip are designed to drive a typical zero-bias receiver without the need for an external amplifier.

The sigma-delta modulators supply the pulse density modulated (PDM) output data to an output H-bridge structure that can be used to directly drive a receiver with the digital output signal. To enable or disable the digital output drivers, configure the OUTPUT_CTRL_OD*_ENABLE bits in the A_OUTPUT_CTRL register.

The positive terminal of the H-bridge structure (RCVR*+) and negative terminal of the H-bridge structure (RCVR*-) can be independently configured to synchronize the outputs with the SDMCLK's rising or falling clock edges. To configure the positive terminal to update on the rising or falling edge of the SDMCLK, set the OUTPUT_CTRL_OD*_POS_CLK_EDGE bits from the A_OUTPUT_CTRL register. Similarly, to configure the negative terminal to update on the rising or falling edge of the SDMCLK, set the OUTPUT_CTRL_OD*_NEG_CLK_EDGE bits from the A_OUTPUT_CTRL register.

NOTE: For proper behavior of some clock throttling configurations, the output driver positive and negative terminals must not be updated on the same clock edge. For more information, see Section 18.3.2.1, "Advanced Frequency-Based Clock Throttling Options" on page 451.

11.4.1 High-Power Output Mode

The output drivers provided on the Ezairo 7100 chip include a mode for high-power operation. To configure the output driver to operate in either normal or high-power mode, use the OUTPUT_CTRL_OD*_HIGHPOWER bits in the A_OUTPUT_CTRL register.

If high-power operation mode is selected, the output signal that is normally provided to the RCVR*+ and RCVR*- pads of the output driver is also provided to the RCVR*_HP+ and RCVR*_HP- pads respectively. This configuration reduces the impedance of the output stage, allowing for a higher overall output power, particularly for low impedance loads.

When an output driver is configured to operate in high-power mode, you must combine the synchronized output signals externally to the Ezairo 7100 chip. Connect both RCVR*+ and RCVR*_HP+ to a single terminal on an output transducer, and connect both RCVR*- and RCVR*_HP- to the other terminal. Figure 19 shows this typical signal routing of connections for the output driver for output channel 0 in high-power mode. Connections for output channel 1 are identical.

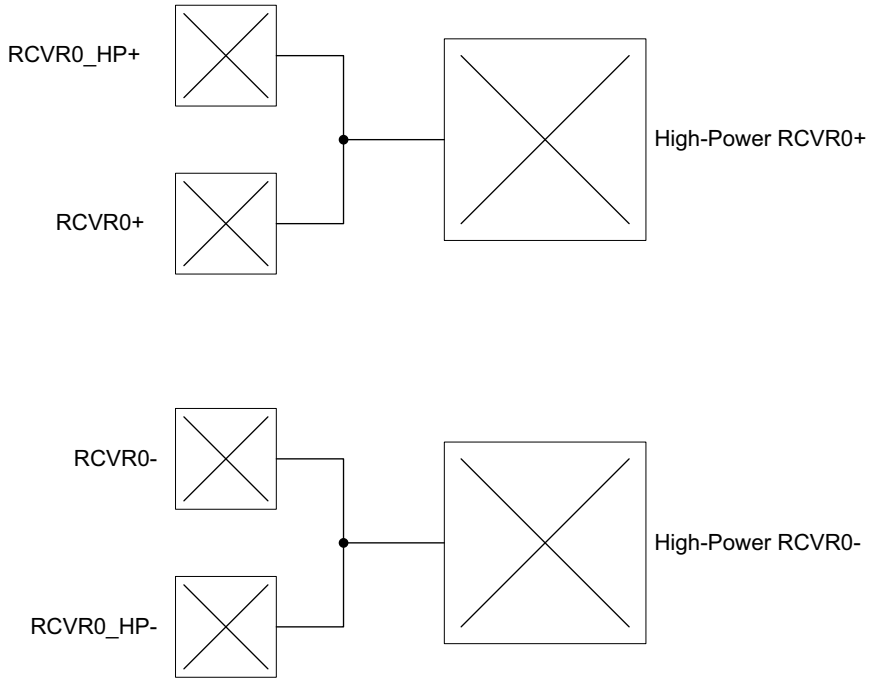


Figure 19. External Signal Routing of Connections for High-Power Output Mode

CAUTION: The audio fidelity when in high-power mode can be slightly degraded compared to normal mode depending on the application. Prior to building a prototype with the configuration shown in Figure 19, you are invited to contact ON Semiconductor for further implementation details.

11.4.2 Control and Configuration Registers

Register Name	Register Description	Address
A_OUTPUT_CTRL	Output driver configuration registers	P:0xF150

11.4.2.1 A_OUTPUT_CTRL Settings

Bit Field	Field Name	Description
7	OD1_NEG_CLK_EDGE	Configure the active clock edge for RCVR1-
6	OD1_POS_CLK_EDGE	Configure the active clock edge for RCVR1+
5	OD1_HIGHPOWER	Enable high power mode for RCVR+/- for channel 1
4	OD1_ENABLE	Enable RCVR1

Bit Field	Field Name	Description
3	OD0_NEG_CLK_EDGE	Configure the active clock edge for RCVR0-
2	OD0_POS_CLK_EDGE	Configure the active clock edge for RCVR0+
1	OD0_HIGHPOWER	Enable high power mode for RCVR+/- for channel 0
0	OD0_ENABLE	Enable RCVR0

Field Name	Value Symbol	Value Description	Hex Value
OD1_NEG_CLK_EDGE	OD1_NEG_EDGE_FALLING	Output drivers toggle on the falling DACCLK edge	0x0*
	OD1_NEG_EDGE_RISING	Output drivers toggle on the rising DACCLK edge	0x1
OD1_POS_CLK_EDGE	OD1_POS_EDGE_FALLING	Output drivers toggle on the falling DACCLK edge	0x0*
	OD1_POS_EDGE_RISING	Output drivers toggle on the rising DACCLK edge	0x1
OD1_HIGHPOWER	OD1_HIGHPOWER_DISABLE	Output not routed to high-power pads	0x0*
	OD1_HIGHPOWER_ENABLE	Output routed to high-power pads	0x1
OD1_ENABLE	OD1_DISABLE	Output driver disabled	0x0*
	OD1_ENABLE	Output driver enabled	0x1
OD0_NEG_CLK_EDGE	OD0_NEG_EDGE_FALLING	Output drivers toggle on the falling DACCLK edge	0x0*
	OD0_NEG_EDGE_RISING	Output drivers toggle on the rising DACCLK edge	0x1
OD0_POS_CLK_EDGE	OD0_POS_EDGE_FALLING	Output drivers toggle on the falling DACCLK edge	0x0*
	OD0_POS_EDGE_RISING	Output drivers toggle on the rising DACCLK edge	0x1
OD0_HIGHPOWER	OD0_HIGHPOWER_DISABLE	Output not routed to high-power pads	0x0*
	OD0_HIGHPOWER_ENABLE	Output routed to high-power pads	0x1
OD0_ENABLE	OD0_DISABLE	Output driver disabled	0x0*
	OD0_ENABLE	Output driver enabled	0x1

11.5 Digital Microphone Output

The Ezairo 7100 system provides the ability to sink data from each of the output channels to a pair of digital microphone (DMIC) outputs. The pad used for each DMIC output is selected from the DIOs and configured as described in Section 16.1, “Digital I/O (DIO)” on page 351.

The DMIC outputs are enabled using the DMIC_OUTPUT_CTRL_DMIC*_ENABLE bits of the A_DMIC_OUTPUT_CTRL register. These outputs take input data from the sigma-delta modulators of the output channels and can be used either in parallel with or independently from the direct-digital outputs.

The DMIC output is synchronized using a clock provided to the Ezairo 7100 system through the EXTCLK pad. This clock input has the following requirements for the DMIC outputs to function correctly. EXTCLK, which is treated as the DMIC output clock, must:

- Have the same nominal frequency as SDMCLK
- Be in phase with SDMCLK so that the rising edges of SDMCLK occur no less then 15 ns before or 15 ns after a EXTCLK falling edge

For each DMIC output, the DMIC data can be configured for the high and low portions of the EXTCLK clock cycle. While the clock is high, the DMIC_OUTPUT_CTRL_DMIC*_HIGH_SRC bits from the A_DMIC_OUTPUT_CTRL register choose between output channels 0 and 1 as the source for the DMIC output. While the clock is low, the DMIC_OUTPUT_CTRL_DMIC*_LOW_SRC bits from the A_DMIC_OUTPUT_CTRL register choose between output channels 0 and 1 as the source for the DMIC output.

NOTE: To treat a DMIC output as a mono output, we recommend configuring both the high and low sources to use the same output channel because this is less error prone. To treat a DMIC output as a stereo output, the high and low sources can be derived independently from the output channels as needed by the remote system.

11.5.1 Control and Configuration Registers

Register Name	Register Description	Address
A_DMIC_OUTPUT_CTRL	Configuration registers for the DMIC outputs	P:0xF15B

11.5.1.1 A_DMIC_OUTPUT_CTRL Settings

Bit Field	Field Name	Description
5	DMIC1_HIGH_SRC	Selects the source of the DMIC1 high data channel
4	DMIC1_LOW_SRC	Selects the source of the DMIC1 low data channel
3	DMIC0_HIGH_SRC	Selects the source of the DMIC0 high data channel
2	DMIC0_LOW_SRC	Selects the source of the DMIC0 low data channel
1	DMIC1_ENABLE	Enables/disables the DMIC1 output
0	DMIC0_ENABLE	Enables/disables the DMIC0 output

Field Name	Value Symbol	Value Description	Hex Value
DMIC1_HIGH_SRC	DMIC1_HIGH_SRC_CH0	Output channel 0	0x0*
	DMIC1_HIGH_SRC_CH1	Output channel 1	0x1
DMIC1_LOW_SRC	DMIC1_LOW_SRC_CH0	Output channel 0	0x0*
	DMIC1_LOW_SRC_CH1	Output channel 1	0x1
DMIC0_HIGH_SRC	DMIC0_HIGH_SRC_CH0	Output channel 0	0x0*
	DMIC0_HIGH_SRC_CH1	Output channel 1	0x1
DMIC0_LOW_SRC	DMIC0_LOW_SRC_CH0	Output channel 0	0x0*
	DMIC0_LOW_SRC_CH1	Output channel 1	0x1
DMIC1_ENABLE	DMIC1_OUT_DISABLE	DMIC1 output disabled	0x0*
	DMIC1_OUT_ENABLE	DMIC1 output enabled	0x1
DMIC0_ENABLE	DMIC0_OUT_DISABLE	DMIC0 output disabled	0x0*
	DMIC0_OUT_ENABLE	DMIC0 output enabled	0x1

CHAPTER 12

CFX Shared PCM and I²S™ Interface

12.1 PCM Interface Introduction

The Ezairo 7100 system includes a highly configurable pulse code modulation (PCM) interface connected to the CFX and IOC that can be used to stream control, configuration or signal data into and out of the chip. This chapter discusses the configuration and control of this interface using the CFX. For information about using the PCM interface as an input source or output sink for audio data, see Chapter 9, “Input/Output Audio Control” on page 111.

One of the advantages of this interface is the high communication speed that allows the interface to operate at the system clock frequency. Additionally, you can configure the interface to operate in either full-duplex mode or half-duplex mode (either transmitting or receiving) because of the separate serial data lines.

The Ezairo 7100 system’s CFX PCM interface can be configured to be compatible with communication over the inter-IC sound (I^2S) protocol. For information about configuring and using the PCM interface for I^2S communications, see Section 12.4, “I2S Configuration and Usage” on page 179. It also supports a configuration that is compatible with digital audio streaming from a NOAHlink device. For detailed information about configuring and using the PCM interface in NOAHlink digital audio streaming mode, see Section 12.5, “NOAHlink Digital Audio Streaming Configuration and Usage” on page 182.

12.1.1 PCM Interface Configuration

The PCM interface uses the `PCM_CFG_ENABLE` bit in the `D_PCM_CFG` register to enable and disable the PCM interface entirely. When using the PCM interface in any mode, enable this bit.

The PCM interface makes use of four DIO pads in communications. These pins are:

- An interface clock input (PCM_CLK)
- A frame signal (PCM_FRAME)
- A serial output data (PCM_SERO)
- A serial input data (PCM_SERI)

The PCM interface can be configured as either a master device (controlling the frame signal and usually with Ezairo 7100 providing the interface clocking), or as a slave device (receiving the frame signal and usually with the interface being clocked from an external device). To select between these configurations, set or clear the `PCM_CFG_PCM_SLAVE` bit as appropriate in the `D_PCM_CFG` register.

In master mode, configure the PCM frame signal as an output; in slave mode, configure it as an input. The PCM clock signal is always an input to the PCM interface; this clock signal can be sourced:

- Internally, using the DIO output mode for the pad used for the PCM clock to route an internal clock signal within the Ezairo 7100 system to the same pad
- Externally, using an externally generated clock signal

The PCM interface is sensitive to only one edge of the PCM clock. All settings are clocked in and signal updates are captured on the rising or falling edge as specified by the `PCM_EXT_CFG_CFX_PCM_CLK_POL` bit from the `D_PCM_EXT_CFG` register. By default, the PCM interface is only sensitive to falling edges of the PCM clock. For I^2S standard compatibility, we recommend configuring the interface to sample on the rising edges of this clock when using this interface in I^2S mode.

All physical configuration for the PCM pads is controlled by the DIO block. This includes drive strength of outputs, and pull-up resistors for input signals.

CAUTION: Disabling the bus pull-up resistors is not recommended for the PCM frame or PCM clock inputs. If these pads are used as outputs, you can disable the pull-up resistors. For all other configurations, attempting to use this interface without the pull-up resistors can result in unintended interface behavior that would result in the PCM interface transmitting and/or receiving undefined data.

The PCM interface can also be configured so that the PCM frame signal is driven from a reference clock source, such as the user clock (UCLK). To do this:

- Configure the PCM interface for slave mode operation using long width frame signals (by setting the `PCM_CFG_FRAME_WIDTH` bit to `PCM_FRAME_WIDTH_LONG`)
- Assign the PCM frame input source (`PCM_SRC_FRAME` bit from the `D_PCM_SRC` register) to read from a DIO where the desired reference clock is being provided to the system, or output by the system.

For information about configuring UCLK, see Section 18.2.7, “User Clock (UCLK)” on page 448.

CAUTION: It is important that the PCM interface is configured as a slave device so that the interface does not try to drive the PCM frame signal at the same time that the reference clock source is driving it.

For more information about configuring the DIOs for PCM functionality, see Section 16.1, “Digital I/O (DIO)” on page 351.

12.2 PCM Signal Configuration

The shared PCM and I^2S interface uses a wide number of configuration options to help to define a set of signals that the system can interpret. For the purpose of clarifying the following explanations, the signals generated by the system in the default PCM configuration are shown in Figure 20.

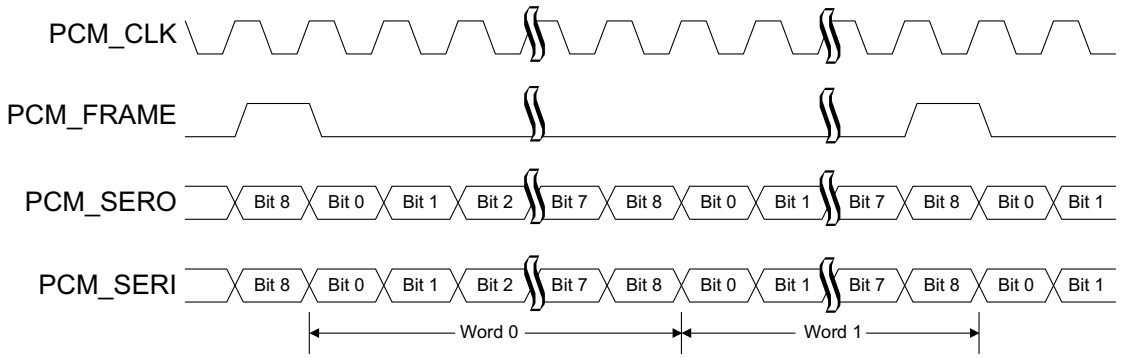


Figure 20. Signal Timing Diagram: Default PCM Configuration

Further information about the required signal configuration and other interface configuration that is needed to allow the interface to be compatible with I^2S, including an example timing diagram, is presented in Section 12.4, “I2S Configuration and Usage” on page 179.

Further information about the required signal configuration and hardware configuration that allows this interface to be compatible with NOAHlink digital PCM signals, including an example timing diagram, is presented in Section 12.5, “NOAHlink Digital Audio Streaming Configuration and Usage” on page 182. This section also briefly discusses the hardware limitations of this interface configuration.

IMPORTANT: If the PCM interface is not idle when changing any configuration settings, changing the PCM configuration during a transaction results in undefined behavior on the associated PCM signal lines.

12.2.1 Frame Signal Configuration and Timing

The PCM interface uses a signal, called a frame signal, to realign transmission over the PCM interface between two devices with every data frame transmitted.

The frame signal divides the communications on the PCM interface into data frames (or sub-frames, if the PCM_CFG_SUBFRAME bit of the D_PCM_CFG register is enabled). Each aspect of the PCM frame signal can be configured as follows (all bits are in the D_PCM_CFG register):

Interval — In the default configuration of the PCM_CFG_MULTIWORD_CTRL bit field (configured for two words per frame), and the PCM_CFG_SUBFRAME bit (configured to disable a frame signal being generated for each word within a frame), the frame signal is generated once for every two-word frame. Changing the PCM_CFG_MULTIWORD_CTRL bit field in PCM master mode allows you to select the number of words per frame, incrementing in multiples of two words, and hence the number of words per data frame. The PCM_CFG_MULTIWORD_CTRL bit field is not used for PCM slave mode; it should be left in the default configuration (configured for two words per frame), regardless of the actual frame length used, to get the expected behavior. Alternately, if the PCM_CFG_SUBFRAME bit is configured to enable subframes, a frame signal is generated with every word of each data frame.

IMPORTANT: Configuring the PCM_CFG_MULTIWORD_CTRL bit field to something other than two words per frame while setting the PCM_CFG_SUBFRAME bit results in a configuration where you will receive a PCM frame signal for every subframe word. Despite the well-defined behavior of the frame signal generated for this configuration, the handling of data and interrupts by the PCM interface might not appear sensible and will likely result in undefined or unexpected data.

Shape — The frame signal can be configured to take one of two shapes by setting the PCM_CFG_FRAME_WIDTH bit. By default, the frame signal is configured to produce a short width frame signal which produces a single cycle pulse in the frame signal at the beginning of each frame. Alternately, you can configure the signal to produce a long width frame signal which uses a 50% duty cycle square wave and is spaced to align with the specified width between signal events.

Alignment — You can configure the PCM frame signal by using the PCM_CFG_FRAME_ALIGN bit from the D_PCM_CFG register so that the first bit of data sampled is sampled at the same time as the frame signal, or one cycle after the frame signal. In this way, the frame signal is aligned with the first bit of the new data frame, or with the last bit of the previous data frame.

NOTE: Regardless of the PCM frame signal alignment configuration, the PCM frame signal occurs at the beginning of each frame and only one frame signal occurs for each frame received. The PCM frame signal, if aligned

with the last bit, occurs one bit before the first data bit transmitted at the start of a transaction, and does not occur on the last bit of the last frame of a transaction. Similarly, if aligned to the first bit, the first frame signal is aligned with the first data bit transmitted at the start of a transaction, and no trailing frame signal occurs for a transaction.

NOTE: To properly terminate a transmission, an expected frame signal pulse must be missed. When the frame signal is configured to be aligned with the first bit of a transmission, this requires one additional PCM clock pulse following the transmission of the last bit of the last frame.

For clarification of the above PCM frame signal timing configuration explanations, several examples of different PCM frame signal traces are shown in Figure 21. In all cases the `PCM_CFG_FRAME_ALIGN` bit has been set to `PCM_FRAME_ALIGN_FIRST_BIT` (which differs from the default configuration shown in Figure 20 on page 173) to prevent any confusion as to what are the effects of the other frame signal configuration bits. The frame signal configurations shown are:

1. Configured to indicate the subframe words of a frame with a short pulse frame signal.
2. Configured to indicate the subframe words of a frame with a long width frame signal.
3. Configured to use a 2-word frame with a short pulse frame signal.
4. Configured to use a 2-word frame with a long width frame signal.
5. Configured to use a 4-word frame with a short pulse frame signal.
6. Configured to use a 4-word frame with a long width frame signal.

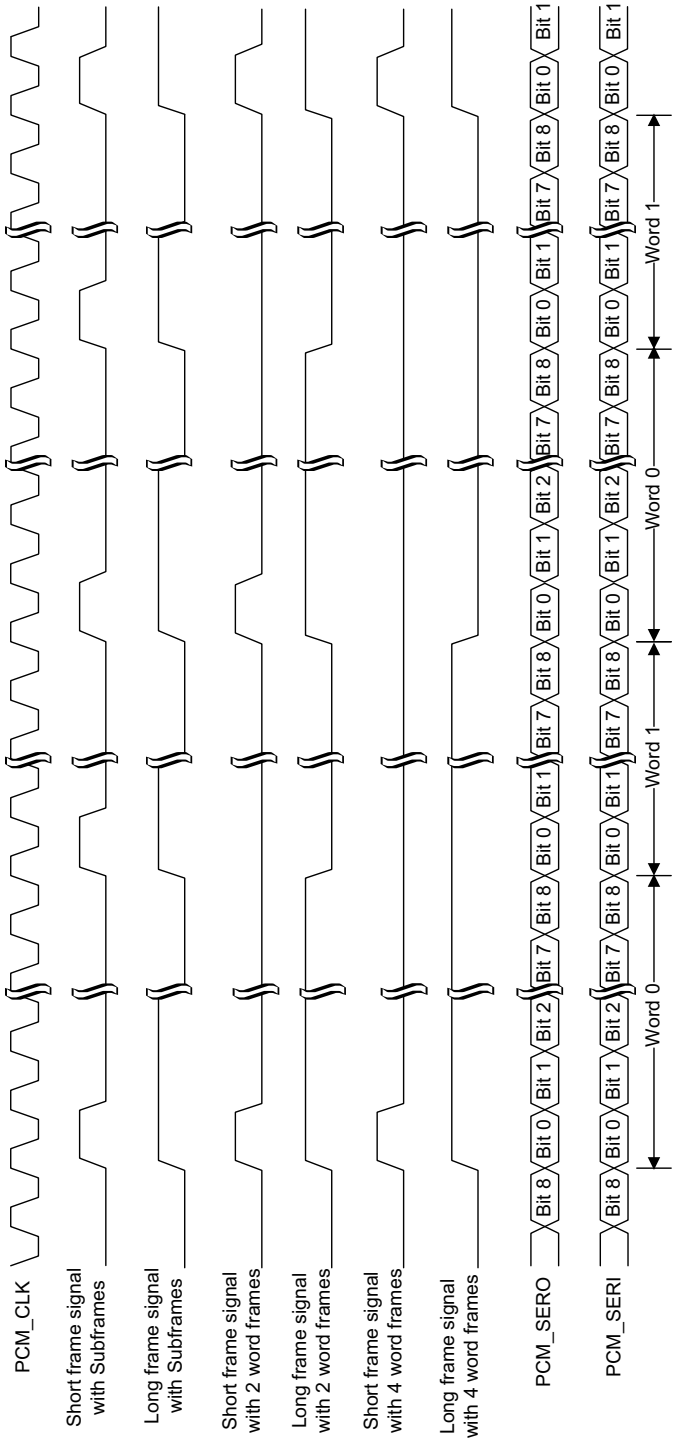


Figure 21. Frame Signal Timing Examples

12.2.2 Data Serial Input and Output Configuration

The PCM interface allows data bits to be configured for transmission and reception in either an MSB to LSB ordering or an LSB to MSB ordering. To select between these two configurations, set the PCM_CFG_BIT_ORDER bit in the D_PCM_CFG register. You can also set the length of each word of a PCM transaction to be between 9 and 24 bits per word by setting the PCM_CFG_WORD_SIZE bit field in the D_PCM_CFG register.

IMPORTANT: In all configurations, the PCM data serial input and output lines are sampled on only one edge of the PCM clock signal. At the start of a new transmission, at least two clock edges must occur on the PCM clock line before the first bit can be transmitted or received on either of the PCM data lines.

Additionally, the PCM interface also allows for a number of different data configurations for both data being received and data being transmitted over the PCM interface.

The data received is stored into a 24-bit data type. The data bits that are actually received can be left-shifted to place them in the MSB positions, or right-shifted into the LSB positions of the data stored. To select between these two configurations, use the PCM_CFG_SHIFT_RX_DATA bit in the D_PCM_CFG register. An example of the resulting input data for each configuration is shown in Table 23.

Table 23. Receive Data Configuration Example for a 16-Bit Transmission

PCM Data Received (Using PCM_CFG_WORD_SIZE_16)	PCM Receive Data Configuration	PCM Data Input
0x1234	PCM_SHIFT_RX_DATA_LEFT	0x123400
	PCM_SHIFT_RX_DATA_RIGHT	0x001234

Similarly the output data transmitted by the PCM interface is loaded from 24-bit data. As the word size might not be 24 bits in length, we provide some possible configurations of the transmit data.

First of all, setting the PCM_CFG_FIELD_TX_DATA bit in the D_PCM_CFG register selects the bit field within the data to be transmitted. This bit allows you to specify that the data to be sent is loaded from the most significant portion or least significant portion of the data.

If you selected the least significant portion of the data for transmission, you can also configure the interface to either saturate or truncate the data that is to be sent by setting the PCM_CFG_SATURATE_TX_DATA bit in the D_PCM_CFG register. Truncating the data to be transmitted results in the exact value of the lower data bits. Saturating the data to be transmitted takes the original data; but if that data exceeds the size that can fit in the bit field data selected, the data that is sent is set to the maximum or minimum value that can be stored in the bit field (as appropriate).

IMPORTANT: Selecting data saturation by setting the PCM_CFG_SATURATE_TX_DATA bit when the PCM_CFG_FIELD_TX_DATA bit has been configured to select the most significant data bits of the data for transmission results in undefined behavior.

An example of the effects that these configuration bits have on data supplied to the PCM interface for transmission of 16-bit words is provided in Table 24.

Table 24. Transmit Data Configuration Example for a 16-Bit Transmission

PCM Data Supplied	PCM Transmit Data Configuration	PCM Data Output (Using PCM_CFG_WORD_SIZE_16)
0x123456	PCM_FIELD_TX_DATA_MSB (with PCM_TX_DATA_UNSATURATED or PCM_TX_DATA_SATURATED)	0x1234
	PCM_FIELD_TX_DATA_LSB, PCM_TX_DATA_UNSATURATED	0x3456
	PCM_FIELD_TX_DATA_LSB, PCM_TX_DATA_SATURATED	0x7FFF

12.3 PCM Communications Management

12.3.1 PCM Data Management

Either the CFX DSP or the IOC can control the data received or transmitted by the PCM interface. To select between these two configurations, use the PCM_CFG_DATA_MGT bit in the D_PCM_CFG register.

In both configurations any data received is interpreted as being received on two separate channels. In all underlying signal configurations, the interface treats all data that is to be sent or that has been received in pairs of words—the first being assigned to PCM channel 0 and the second being assigned to PCM channel 1. The most recent pair of words received is always stored in the D_PCM_DATA_0 (storing data for channel 0) and D_PCM_DATA_1 (storing data for channel 1) registers.

IMPORTANT: In all of the timing diagrams included in this chapter, the data assigned to channel 0 is marked as “Word 0”, and the data assigned to channel 1 is marked as “Word 1”.

When you configure the interface to allow the IOC to manage PCM interface data, data input using the interface is stored into a FIFO, and data output using the interface is read from a FIFO as per the IOC configuration. For more information about configuring the IOC to use the PCM interface for data communications, see Chapter 9, “Input/Output Audio Control” on page 111.

When the interface is configured to allow the CFX to manage PCM interface data, we recommend that the core use the PCM transmit and PCM receive interrupts to control data flow. All data sent and received by the PCM interface is only accessible through the D_PCM_DATA_0 and D_PCM_DATA_1 registers.

12.3.2 PCM Interrupt Configuration

The PCM interface provides an interrupt that can be configured to trigger on both the transmission and reception of data. The triggers for this interrupt are independently configurable, and interrupts due to the transmission of data or the reception of data can be received separately or together, depending on the interface configuration. See Section 14.1, “Interrupt Controller” on page 218 for information regarding interrupt configuration and handling.

Enable or disable the PCM transmit interrupt with the PCM_CFG_TX_INT bit in the D_PCM_CFG register. When transmit interrupts are enabled, interrupts are generated for every two words of data transmitted regardless of the length of each PCM frame. These interrupts occur when the data has been loaded from the D_PCM_DATA_x registers to the internal interface registers. As a result, after a transmit interrupt, you can reload the PCM data registers with the next words to be transmitted without incurring a delay between words transmitted over the PCM interface.

When a PCM interrupt has been generated due to the transmission of data, the PCM_CTRL_TX_INT_STATUS bit in the D_PCM_CTRL register is set. When one or both of the D_PCM_DATA_x registers is written, providing the interface with the requested data, this bit is cleared.

Enable or disable the PCM receive interrupt with the PCM_CFG_RX_INT bit in the D_PCM_CFG register. When receive interrupts are enabled, interrupts are generated for every two words of data received regardless of the length of each PCM frame.

When a PCM interrupt has been generated due to the reception of data, the PCM_CTRL_RX_INT_STATUS bit in the D_PCM_CTRL register is set. When one or both of the D_PCM_DATA_x registers is used to read the received data that caused this interrupt, this bit is cleared.

CAUTION: Take extra care when configuring the PCM_TX_INT and PCM_RX_INT bits without using defines since both of these bits enable their respective interrupt triggers when cleared (and disable those interrupt triggers when set).

12.3.3 Control and Configuration Registers

Register Name	Register Description	Address
D_PCM_CTRL	PCM Control Register	P:0xF243
D_PCM_DATA_0	PCM Channel 0	P:0xF241
D_PCM_DATA_1	PCM Channel 1	P:0xF242

For information about other PCM communications management registers, see Section 12.6, “PCM Interface Control and Configuration Registers” on page 183.

12.3.3.1 D_PCM_CTRL Settings

Bit Field	Field Name	Description
1	TX_INT_STATUS	Indicate if a PCM transmit interrupt has been generated
0	RX_INT_STATUS	Indicate if a PCM receive interrupt has been generated

12.4 I^2S Configuration and Usage

The I^2S protocol is designed to allow a device to share data digitally between devices. To use the Ezairo 7100 PCM interface for I^2S communications, configure the interface to conform to the I^2S communications standard. Table 25 lists the required signal and data management settings for the PCM interface to enable I^2S communication. As a side effect of the configuration requiring that the device operate in PCM slave mode, the frame signal must be used as an input for I^2S communications.

Table 25. Required Configuration Settings for I^2S Configuration

Configuration Bit or Bit Field	Setting
PCM_CFG_WORD_SIZE	PCM_WORD_SIZE_16
PCM_CFG_MULTIWORD_CTRL	PCM_MULTIWORD_2
PCM_CFG_FRAME_WIDTH	PCM_FRAME_WIDTH_LONG

Table 25. Required Configuration Settings for I²S Configuration

Configuration Bit or Bit Field	Setting
PCM_CFG_FRAME_ALIGN	PCM_FRAME_ALIGN_LAST_BIT
PCM_CFG_BIT_ORDER	PCM_BIT_ORDER_MSB_FIRST
PCM_CFG_SUBFRAME	PCM_SUBFRAME_DISABLE
PCM_CFG_SLAVE	PCM_SLAVE
PCM_EXT_CFG_CFX_PCM_CLK_POL	CFX_PCM_SAMPLE_RISING_EDGE[1]

1. The PCM interface produces a data stream that complies with the I²S standard with either sampling edge. However, to comply completely with the standard, the interface must be configured to sample data on the rising edge of the clock.

When operating in this configuration, the behavior of the PCM interface is slightly different from any other configuration. As shown in the I²S signal timing diagram (Figure 22), the device allows bit 15 to be repeatedly transmitted until a rising or falling edge on the frame signal is detected. When configured for slave mode with a word size of 16 and a long frame width, the PCM interface synchronizes communications with both the rising edge and falling edge of the frame signal (instead of synchronizing with only the rising edge as it does in all other configurations). In this way, the interface is made compatible with I²S signals that use a width greater than 16 bits of data.

This I²S implementation is compatible with other I²S implementations for data that is exactly 16 bits in length, and where the frame signal has a duty cycle of exactly 50%.

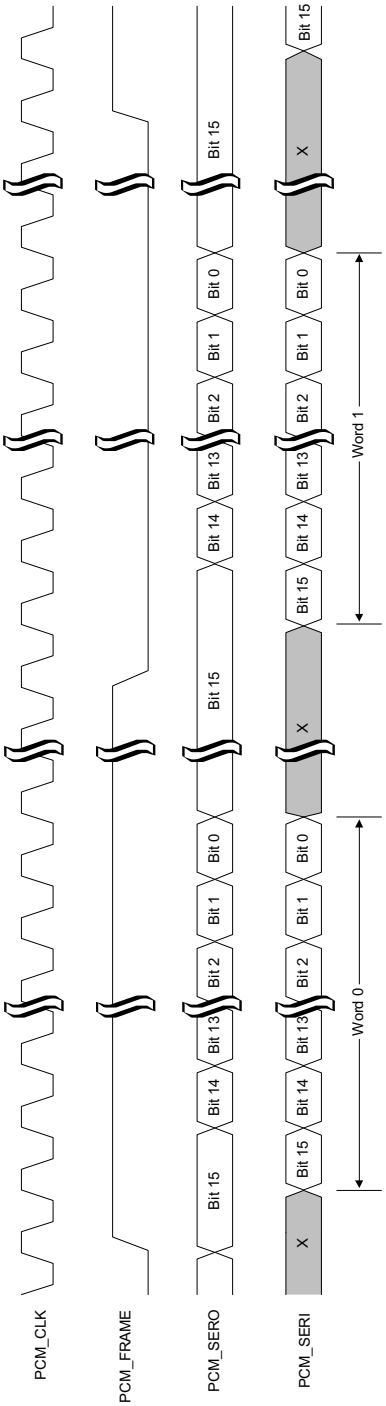


Figure 22. Signal Timing Diagram: I^2S Configuration

12.5 NOAHlink Digital Audio Streaming Configuration and Usage

The PCM interface can be configured for compatibility with using the digital audio streaming feature of a NOAHlink device to input data. This configuration, however, requires a number of signal configuration settings as shown in Table 26. This configuration also requires several modifications to the hardware connections as described in Table 27.

Table 26. Required Configuration Settings for NOAHlink Digital Audio Streaming Configuration

Configuration Bit or Bit Field	Setting
PCM_CFG_WORD_SIZE	PCM_WORD_SIZE_16
PCM_CFG_MULTIWORD_CTRL	PCM_MULTIWORD_2
PCM_CFG_FRAME_WIDTH	PCM_FRAME_WIDTH_LONG
PCM_CFG_FRAME_ALIGN	PCM_FRAME_ALIGN_FIRST_BIT
PCM_CFG_BIT_ORDER	PCM_BIT_ORDER_LSB_FIRST
PCM_CFG_SUBFRAME	PCM_SUBFRAME_DISABLE

Table 27. Hardware Connection Configuration for NOAHlink Digital Audio Streaming

PCM Signal	Connected Signal	Connection	Comment
PCM_CLK	UCLK	Both signals routed to a shared DIO pad	Signal output or input at PCM clock is ignored
PCM_FRAME	CFX I^2C SDA	External; PCM_FRAME configured as an output	Disconnect the pull-ups on the I^2C SDA line to limit glitches in the connection to a NOAHlink audio streaming device
PCM_SERO	-	Unconnected	NOAHlink digital audio streaming is input only
PCM_SERI	CFX I^2C SCL	External; PCM_SERI configured as an input	Disconnect the pull-ups on I^2C SCL line to limit glitches in the connection to a NOAHlink device

To enable audio streaming when connected to a NOAHlink device, configure the debug port to operate in audio streaming mode.

NOTE: Support for configuring the debug port for use with audio streaming can be obtained by contacting ON Semiconductor Customer Support.

When in audio streaming mode with the required PCM interface configuration, the device will stream audio data from a NOAHlink device. After configuring the PCM interface in this way and enabling audio streaming, the resulting signals will match the timing diagram shown in Figure 23.

IMPORTANT: If you configured the PCM interface for NOAHlink digital audio streaming by connecting the relevant PCM and I^2C pads, analog audio streaming using the NOAHlink device and the I^2C interface is not possible.

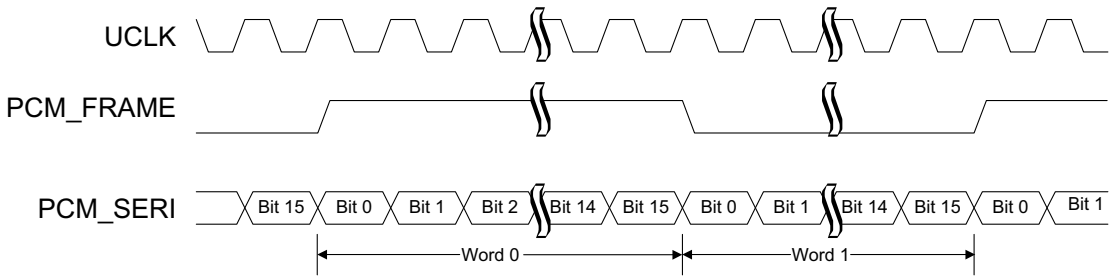


Figure 23. Signal Timing Diagram: NOAHlink Digital Audio Streaming PCM Configuration

12.6 PCM Interface Control and Configuration Registers

Register Name	Register Description	Address
D_PCM_CFG	PCM Configuration Register	P:0xF240
D_PCM_EXT_CFG	Extended configuration for CFX and ARM Cortex-M3 processor PCM interfaces	P:0xF03E

12.6.1 D_PCM_CFG Settings

Bit Field	Field Name	Description
21	SATURATE_TX_DATA	Data transmitted is saturated if it can't be stored within the configured word size
20	FIELD_TX_DATA	Indicate which bits from PCM_DATA or IOC are transmitted
19	SHIFT_RX_DATA	Left or right shift the data received within the 24-bit data value
18:15	WORD_SIZE	Select the number of bits per PCM word
14:12	MULTIWORD_CTRL	Configure the number of words per PCM frame
11	DATA_MGT	Select whether the CFX or the IOC will manage PCM data
7	TX_INT	Enable/disable transmit interrupts
6	RX_INT	Enable/disable receive interrupts
5	FRAME_WIDTH	Select between a short (pulse) or a long PCM frame signal
4	FRAME_ALIGN	Select the relative alignment of the PCM frame and data signals
3	BIT_ORDER	Select a data bit ordering for data that is sent and received, selecting whether data starts with the MSB or the LSB
2	SUBFRAME	Select either a frame signal with every word, or a frame signal with every frame
1	ENABLE	Enable the PCM interface
0	SLAVE	Select master or slave mode operation for the PCM interface

Field Name	Value Symbol	Value Description	Hex Value
SATURATE_TX_DATA	PCM_TX_DATA_UNSATURATED	Do not saturate TX data	0x0*
	PCM_TX_DATA_SATURATED	Saturate TX data	0x1

Field Name	Value Symbol	Value Description	Hex Value
FIELD_TX_DATA	PCM_FIELD_TX_DATA_MSB	Select the most significant bit field for transmission	0x0*
	PCM_FIELD_TX_DATA_LSB	Select the least significant bit field for transmission	0x1
SHIFT_RX_DATA	PCM_SHIFT_RX_DATA_LEFT	Shift the received data left	0x0*
	PCM_SHIFT_RX_DATA_RIGHT	Shift the received data right	0x1
WORD_SIZE	PCM_WORD_SIZE_9	Each PCM word contains 9 bits	0x0*
	PCM_WORD_SIZE_10	Each PCM word contains 10 bits	0x1
	PCM_WORD_SIZE_11	Each PCM word contains 11 bits	0x2
	PCM_WORD_SIZE_12	Each PCM word contains 12 bits	0x3
	PCM_WORD_SIZE_13	Each PCM word contains 13 bits	0x4
	PCM_WORD_SIZE_14	Each PCM word contains 14 bits	0x5
	PCM_WORD_SIZE_15	Each PCM word contains 15 bits	0x6
	PCM_WORD_SIZE_16	Each PCM word contains 16 bits	0x7
	PCM_WORD_SIZE_17	Each PCM word contains 17 bits	0x8
	PCM_WORD_SIZE_18	Each PCM word contains 18 bits	0x9
	PCM_WORD_SIZE_19	Each PCM word contains 19 bits	0xA
	PCM_WORD_SIZE_20	Each PCM word contains 20 bits	0xB
	PCM_WORD_SIZE_21	Each PCM word contains 21 bits	0xC
	PCM_WORD_SIZE_22	Each PCM word contains 22 bits	0xD
	PCM_WORD_SIZE_23	Each PCM word contains 23 bits	0xE
	PCM_WORD_SIZE_24	Each PCM word contains 24 bits	0xF
MULTIWORD_CTRL	PCM_MULTIWORD_2	PCM frames contain 2 words	0x0*
	PCM_MULTIWORD_4	PCM frames contain 4 words	0x1
	PCM_MULTIWORD_6	PCM frames contain 6 words	0x2
	PCM_MULTIWORD_8	PCM frames contain 8 words	0x3
	PCM_MULTIWORD_10	PCM frames contain 10 words	0x4
	PCM_MULTIWORD_12	PCM frames contain 12 words	0x5
	PCM_MULTIWORD_14	PCM frames contain 14 words	0x6
	PCM_MULTIWORD_16	PCM frames contain 16 words	0x7
DATA_MGT	PCM_DATA_MGT_IOC	The IOC will handle data received/transmitted	0x0*
	PCM_DATA_MGT_CFX	The CFX will handle data received/transmitted	0x1
TX_INT	PCM_TX_INT_ENABLE	TX interrupt enabled	0x0*
	PCM_TX_INT_DISABLE	TX interrupt disabled	0x1
RX_INT	PCM_RX_INT_ENABLE	RX interrupt enabled	0x0*
	PCM_RX_INT_DISABLE	RX interrupt disabled	0x1
FRAME_WIDTH	PCM_FRAME_WIDTH_SHORT	The frame signal is one cycle (short)	0x0*
	PCM_FRAME_WIDTH_LONG	The frame signal is 1/2 frame (long)	0x1
FRAME_ALIGN	PCM_FRAME_ALIGN_LAST_BIT	Align the frame signal to the last bit	0x0*
	PCM_FRAME_ALIGN_FIRST_BIT	Align the frame signal to the first bit	0x1

Field Name	Value Symbol	Value Description	Hex Value
BIT_ORDER	PCM_BIT_ORDER_LSB_FIRST	Transmit LSB first	0x0*
	PCM_BIT_ORDER_MSB_FIRST	Transmit MSB first	0x1
SUBFRAME	PCM_SUBFRAME_DISABLE	Generate a frame signal every frame	0x0*
	PCM_SUBFRAME_ENABLE	Generate a frame signal every word	0x1
ENABLE	PCM_DISABLE	Disable the PCM interface	0x0*
	PCM_ENABLE	Enable the PCM interface	0x1
SLAVE	PCM_MASTER	Configure the PCM interface as master	0x0
	PCM_SLAVE	Configure the PCM interface as slave	0x1*

12.6.2 D_PCM_EXT_CFG

Bit Field	Field Name	Description
1	CM3_PCM_CLK_POL	ARM Cortex-M3 processor PCM clock polarity
0	CFX_PCM_CLK_POL	CFX PCM clock polarity

Field Name	Value Symbol	Value Description	Hex Value
CM3_PCM_CLK_POL	CM3_PCM_SAMPLE_FALLING_EDGE	Sample on the falling edge of PCM clock	0x0*
	CM3_PCM_SAMPLE_RISING_EDGE	Sample on the rising edge of PCM clock	0x1
CFX_PCM_CLK_POL	CFX_PCM_SAMPLE_FALLING_EDGE	Sample on the falling edge of PCM clock	0x0*
	CFX_PCM_SAMPLE_RISING_EDGE	Sample on the rising edge of PCM clock	0x1

CHAPTER 13
Debug Ports and CFX General-Purpose I²C

This chapter does not address the ARM Cortex-M3 processor's I²C interface. For more information about that interface, refer to Section 17.1, "I2C Interface" on page 400.

13.1 INTER-IC INTERFACE (I²C) BUS®

The Ezairo 7100 system contains three I²C interfaces that share the same I²C bus within the system. All of these interfaces implement a subset of the Philips I²C protocol as described in the *Philips I²C Bus Specification* document.

The I²C interface uses the rising edges of a serial clock signal (SCL) to clock in data from a serial data signal (SDA) to communicate between devices. The I²C interface is designed to handle bus traffic operating at up to 1 MHz; however, for communications to proceed on the I²C interface, the following relation between the system clock (which is used internally to clock the I²C interface) and the SCL must hold:

$$F_{SYSCLK} \geq 3.125 \times F_{SCL}$$

The example timing diagram shown in Figure 24 provides some information about the important elements in an I²C transaction, which are described in further detail in the previously mentioned bus specification. These elements are:

Start Condition

The SDA transitions from the idle high state to the low state while the SCL remains high. This can also happen during a transmission as a repeated start condition, which indicates that the transaction will start again without an intermediate stop condition.

Address Bits

During the first byte transmitted, the first seven bits on the SDA (clocked in on a rising edge of SCL) provide the address of the device with which the master device wants to communicate. A device that is properly addressed should acknowledge this transaction if communications are to conform to the I²C standard.

Read/Write Direction Bit

During the first byte transmitted, the eighth bit on the SDA (clocked in on a rising edge of SCL) indicates the direction of the transaction. A zero for this bit indicates a write by the master; a one indicates a read by the master. If the master device has requested a read, the slave device controls the SDA line on subsequent bytes to transmit data.

Data Byte

All other bytes, excluding the addressing and read/write byte, that are transmitted on the SDA are considered data bytes for the transaction.

Acknowledge or Not Acknowledge Bit

The acknowledge bit is used to tell the sender that the byte has been received. The device receiving data should acknowledge each data byte, including the address byte. During this time, the bus device that is sending data on the data bus stops driving the SDA and allows the line to be pulled high. To not acknowledge a byte, the receiving device does not need to do anything. To acknowledge a byte, the receiving device needs to pull the SDA low.

A receiving slave device should not acknowledge a byte if the slave device is not the device that was addressed, or if the device cannot handle the byte received. A master device should not acknowledge a byte if the master is receiving and wants to end the transaction. If a not acknowledge is encountered, the device transmitting data should generate a stop condition.

Stop Condition

The SDA transitions from a low state to high state while the SCL remains high. This ends an I^{2}C transaction.

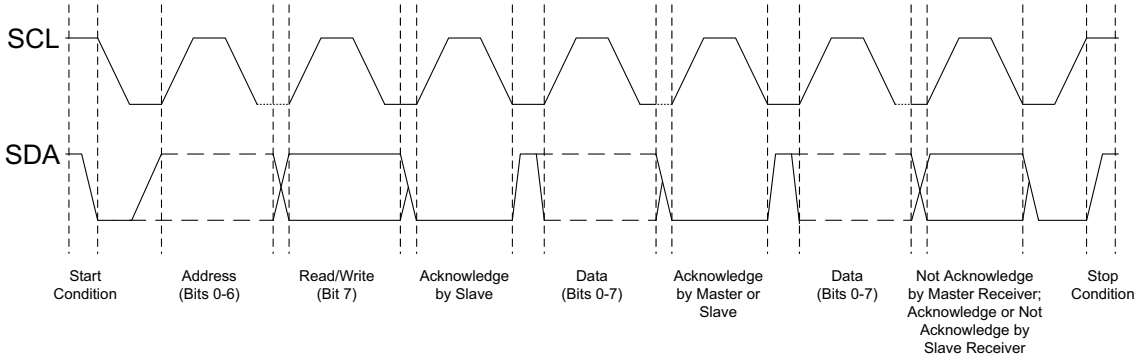


Figure 24. I^{2}C Signal Timing Diagram

Since three interfaces share the same bus, addressing these interfaces becomes an important concern. Be careful not to address the CFX debug port I^{2}C interface, ARM Cortex-M3 core debug port I^{2}C interface, and general-purpose I^{2}C interface at the same address because if they share an address, the order of precedence for handling data using the shared address is:

1. The CFX debug port I^{2}C interface
2. The ARM Cortex-M3 core debug port I^{2}C interface
3. The general-purpose I^{2}C interface

The addressing for these interfaces is defined in Table 28.

Table 28. I^{2}C Bus Addressing for the Debug Ports and CFX General-Purpose I^{2}C Interface

Interface	Response Location	Default Address Setting
CFX Debug Port	Internally-programmable address	0x60
	Externally-programmable address	0x60
ARM Cortex-M3 Debug Port	Externally-programmable address	0x40
General-Purpose I^{2}C	I^{2}C general call address	0x00 (fixed)
	Programmable address	None

CAUTION: If you are using the ON Semiconductor Communications Accelerator Adaptor (CAA) to communicate with the Ezairo 7100 chip over either I2C interface, do not use the address 0x51 for I2C communications because it conflicts with the non-volatile memory storage of the CAA.

For further information about addressing the CFX debug port interface, ARM Cortex-M3 core debug port interface, and the CFX general-purpose I2C interfaces, see their respective subsections below.

The Ezairo 7100 system includes four configuration parameters for the I2C bus itself:

- The I2C_PIN_CFG_PULLUP bit in the D_I2C_PIN_CFG register selects the value of the internal pull-up registers used for the I2C interface lines.
- The I2C_PIN_CFG_PULLUP_DIS bit in the D_I2C_PIN_CFG register allows you to disconnect the pull-ups on the I2C interface lines.
- The I2C_PIN_CFG_FILTER_EN bit in the D_I2C_PIN_CFG register enables a pair of low-pass filters for the bus lines that can be used to clean up the I2C bus signals.
- The I2C_PIN_CFG_DRIVE bit in the D_I2C_PIN_CFG register allows you to select the drive strength for the I2C interface lines. The drive strength of the I2C interface can be augmented by combining the dedicated I2C bus lines with DIOs, as described in Section 16.1, “Digital I/O (DIO)”.

CAUTION: Using the I2C_PIN_CFG_PULLUP_DIS bit to disable the bus pull-up resistors is not recommended unless external pull-up resistors are used. We recommend that you consult ON Semiconductor before using configurations that do not use pull-up resistors, because attempting to use this interface without the pull-up resistors will likely result in unintended behavior.

As multiple interfaces can communicate on this bus with one or more external devices, the I2C_CTRL_LINE_FREE bit in the D_I2C_CTRL register provides information on current bus traffic. This bit is only set when the internal I2C bus state machine is currently idle, and both bus lines are high.

For this same reason, a watchdog timer is provided for the I2C bus lines. This timer watches the bus lines to ensure that the Ezairo 7100 system does not hold them low for an extended period of time; otherwise, if noise or another I2C driver causes an error on the bus, the chip’s debug ports would be locked out. To configure the timeout period, set the I2C_WATCHDOG_CFG_TIMEOUT bit field in the D_I2C_WATCHDOG_CFG register. The timer is cleared, preventing a timeout, when the interface is not holding the bus lines and no start condition is pending. If a timeout does occur, the I2C interface is reset, the I2C_CTRL_ERROR and I2C_CTRL_ERROR_STICKY bits are set, and the count in the D_I2C_ERROR_CNT register is incremented.

The interface can detect a bus error which allows the use of multi-master bus configurations and prevents issues from arising when an I2C bus master reads from multiple slaves at the same address. A bus error is detected whenever the value written to the I2C data line (SDA) does not match the physical value of this line. When a bus error is detected, the I2C_CTRL_ERROR, I2C_CTRL_BUS_ERROR, I2C_CTRL_ERROR_STICKY and I2C_CTRL_BUS_ERROR_STICKY bits are set.

Reading the D_I2C_ERROR_CNT register resets the error count register to 0, and clears the I2C_CTRL_ERROR and I2C_CTRL_ERROR_STICKY error status bits if the I2C_CTRL_BUS_ERROR status bit is not set to 1.

NOTE: The SCL and SDA pads used for this bus are disconnected by default from the I2C interface when SYSCLK is sourced from the standby clock because this could cause the Ezairo 7100 debug ports to respond to I2C traffic on the bus even when they are not being addressed. To enable these pads

when clocking the system using the standby clock, set the I2C_CFG_CONNECT_IN_STANDBY bit from the D_I2C_CFG register.

13.2 CFX Debug Port I²C Interface

The CFX debug port I²C interface is a hardware debugger for the Ezairo 7100 system that is always enabled regardless of the configuration of the general-purpose I²C interface.

The debug port I²C interface always responds to communications at two configurable addresses that default to an address of 0x60. These addresses are:

- The internally-programmable address that can be configured by writing the desired 7-bit address into the D_DEBUG_ADDRESS register.
- The externally-programmable address that can be configured using the **Write Address** and **Acquire Address** debug port commands (for more information, see the *Communication Protocols Manual for Ezairo 7100*).

The debug port implements the debug port protocol command set and is tightly coupled with the CFX DSP and the memory components attached to the CFX. As a result, the debug port only provides a peripheral view (through the contents of shared and microcode memory) of the HEAR processing of microcode configurations, ARM Cortex-M3 core application processing, and Filter Engine configuration processing.

13.2.1 Debug Port Configuration

While the DEBUG_CFG_ACCESS_MODE bit in the D_DEBUG_CFG register is set, the functionality of the debug port is limited. This is designed as part of the system security mechanism and a user application can bypass this security by clearing this bit. For more information about security in the Ezairo 7100 system, see Section 14.5, “Access Mode Security Blocks” on page 234, and security related information on the Ezairo 7100 file system in the *Firmware Reference Manual for Ezairo 7100*.

The debug port uses the D_DEBUG_DATA register as a memory-mapped register for direct communication with the CFX DSP over the debug port while the system is running. When the debug port has written to this register, the DEBUG_CFG_REG_WRITTEN bit in the D_DEBUG_CFG register remains set until the register is read. Similarly, when the debug port has read this register, the DEBUG_CFG_REG_READ bit in the D_DEBUG_CFG register remains set until the register is written.

The debug port interface can be used with the PCM interface to provide digital audio streaming functionality. When operating in this mode, the I²C interface is connected to the PCM interface and the debug port is flipped into audio streaming mode. The debug port indicates this mode to the system by setting the DEBUG_CFG_AUDIO_MODE in the D_DEBUG_CFG register. More information about digital audio streaming is in Section 12.5, “NOAHlink Digital Audio Streaming Configuration and Usage” on page 182, and in the *Communication Protocols Manual for Ezairo 7100*.

NOTE: For additional support to configure the debug port for use with audio streaming, contact ON Semiconductor Customer Support.

13.2.2 Breakpoints

The primary troubleshooting functionality of the debug port is provided by the different types of breakpoints that can be configured through the debug port protocol. These breakpoints include the following:

Software breakpoints

Included in code for the CFX processor as `breakpoint` instructions. See the *CFX DSP Architecture Manual* for details about the `breakpoint` instruction, and the *Communication Protocols Manual for Ezairo 7100* for information about configuring the debug port to stop at software breakpoints.

Hardware breakpoints

Configured as pointers to P Memory that are triggered when the CFX DSP issues the instruction that is pointed to by the hardware breakpoint for execution. See the *Communication Protocols Manual for Ezairo 7100* for details about configuring hardware breakpoints, and configuring the debug port to stop at hardware breakpoints.

External breakpoints

Configured to trigger a breakpoint event based on accessed data at addresses corresponding to specified conditions and/or the value of the written data. See Section 13.2.3, “External Breakpoint Configuration” in this manual for details about configuring external breakpoints.

13.2.3 External Breakpoint Configuration

External breakpoints can be used to detect a variety of different memory access conditions. The different types of memory access conditions that can be tested for are grouped into comparison modes as follows:

- Six data modes that monitor for data accesses where the address and data match specific conditions; these modes are described in Section 13.2.3.1, “Data Modes”
- A double-address mode that monitors for data accesses relative to a pair of addresses; this mode is described in Section 13.2.3.2, “Double-Address Mode” on page 194
- A memory conflict mode that detects if a memory conflict has occurred; this mode is described in Section 13.2.3.3, “Memory Conflict Mode” on page 194

The external breakpoint mode is selected using the `EXT_BREAK_CFG_COMP_MODE` bit field from the `D_EXT_BREAK_CFG` register. Each of these modes interprets the other external breakpoint configuration registers differently.

IMPORTANT: When an external breakpoint condition is observed, the instructions currently loaded to the CFX DSP instruction pipeline complete before the core is stopped. This typically results in a perceived delay of 3 cycles between an external breakpoint event and the CFX DSP stopping.

13.2.3.1 Data Modes

In the external breakpoint data modes, an external breakpoint is triggered when a data access on the specified memory bus occurs that fits the data condition at an address that meets the address condition.

- The bus of interest for the external breakpoint is selected by setting the `EXT_BREAK_CFG_BUS_SELECT` bit field from the `D_EXT_BREAK_CFG` register. If no bus is selected, the external breakpoint is disabled.
- Data conditions are used to restrict external breakpoint events based on the value of data being written or read. A description of data conditions that can be used to trigger an external breakpoint are listed in Table 29. Data conditions are selected using the `EXT_BREAK_CFG_ACCESS_COND` bit field from the `D_EXT_BREAK_CFG` register.
- Address conditions are used to restrict external breakpoint triggers based on the memory address where the access occurred. A description of address conditions that can be used to trigger an external breakpoint are listed in Table 30. Address conditions are selected using the `EXT_BREAK_CFG_ADDR_COND` bit field in the

D_EXT_BREAK_CFG register.

Table 29. Data Conditions for Data Modes

Condition	Description
Equal	Data is nominally equal to the data component
Not Equal	Data is nominally not equal to the data component
Greater Than or Equal	Data is greater than or equal to the data component
Less Than	Data is nominally less than the data component

Table 30. Address Conditions for Data Modes

Condition	Description
Greater Than or Equal	Address is greater than or equal to the address component
Less Than	Address is less than the address component
Nominally Equal	Address is equal to the specified bits of the address component; bits of the address component that are padded with zeros are ignored
Strictly Equal	Address is equal to the address component including padded zeros

In these modes, the 32-bit D_EXT_BREAK_DATA register is divided into address and data elements of various lengths. Interpretation details for this register for each of the data modes can be found in Table 31.

NOTE: In 32-bit data mode, no address component is specified, and the address condition is ignored.

Table 31. D_EXT_BREAK_DATA Usage for External Breakpoint Data Modes

Mode	Address Component	Data Component	Notes
32-bit data	-	31:0	Address condition is ignored Data is fully specified
High 24-bit data	31:24	23:0	Address is padded to 16 bits with 8 zeros to the right Data is padded to 32 bits with 8 zeros to the right
Low 24-bit data	31:24	23:0	Address is padded to 16 bits with 8 zeros to the right Data is sign-extended to 32 bits
High 16-bit data	31:16	15:0	Address is fully specified Data is padded to 32 bits with 16 zeros to the right
Mid 16-bit data	31:16	15:0	Address is fully specified Data is padded to 24 bits with 8 zeros to the right, and then sign-extended to 32 bits
Low 16-bit data	31:16	15:0	Address is fully specified Data is sign-extended to 32 bits

IMPORTANT: Data and address conditions that use zero-padding generally use “nominal equality” where specified. A value is considered to be nominally equal to a specified condition if the value is equal to all bits in the condition that are either explicitly specified or are created by sign extension.

13.2.3.2 Double-Address Mode

In external breakpoint double-address mode, an external breakpoint is triggered when a data access on the specified memory bus occurs that fits the access condition at an address that meets the address condition.

- The bus of interest for the external breakpoint is selected by setting the EXT_BREAK_CFG_BUS_SELECT bit field from the D_EXT_BREAK_CFG register. If no bus is selected, the external breakpoint is disabled.
- Access conditions are used to restrict external breakpoint events based on the type of data access that occurred. A description of access conditions that can be used to trigger an external breakpoint are listed in Table 32. Access conditions are selected using the EXT_BREAK_CFG_ACCESS_COND bit field from the D_EXT_BREAK_CFG register.
- Address conditions for double address mode are also used to restrict external breakpoint triggers based on the memory address where the access occurred. A description of address conditions that can be used to trigger an external breakpoint when in double address mode are listed in Table 33. Address conditions are selected using the EXT_BREAK_CFG_ADDR_COND bit field in the D_EXT_BREAK_CFG register.

Table 32. Access Conditions for Double-Address Mode

Condition	Description
Disabled	External breakpoints are not generated
Read or Write Access	Address is considered when data is either read or written
Read Access Only	Address is only considered when data is read
Write Access Only	Address is only considered when data is written

Table 33. Address Conditions for Double Address Mode

Condition	Description
Equal	Address is equal to address 1 or address 2
Bounded Range	Address is between address 1 and address 2
Greater Than or Equal	Address is greater than or equal to address 1, or is equal to address 2
Less Than or Equal	Address is equal to address 1, or is less than or equal to address 2

The D_EXT_BREAK_DATA register is interpreted as two 16-bit addresses, with the high 16 bits being used as the first address and the low 16 bits used as the second address.

13.2.3.3 Memory Conflict Mode

In external breakpoint memory conflict mode, an external breakpoint is triggered when a selected memory conflict occurs. The D_EXT_BREAK_DATA register is used to select the monitored memory conflicts detected in this mode. Possible conflicts that can be selected for detection are:

- The CFX DSP core and HEAR Accelerator core
- The CFX DSP core and the ARM Cortex-M3 processor
- The ARM Cortex-M3 processor and any accesses to the FIFO-controlled portions of shared memory

In this mode, the address condition, access condition and bus selection are ignored.

13.2.4 CFX Debug Port Specific Control and Configuration Registers

Register Name	Register Description	Address
D_DEBUG_CFG	Debug port mode	P:0xF035
D_DEBUG_DATA	Debug port data register	P:0xF036
D_EXT_BREAK_CFG	External break point configuration	P:0xF020
D_EXT_BREAK_DATA	External break point configuration	P:0xF021
D_I2C_CFX_DEBUG_ADDR1	First CFX processor debug slave ID	P:0xF204
D_I2C_CFX_DEBUG_ADDR2	Second CFX processor debug slave ID	P:0xF209
D_I2C_CM3_DEBUG_ADDR	ARM Cortex-M3 processor debug slave ID	P:0xF20A

For control and configuration registers that apply to the debug port I^{2}C interfaces for the CFX and ARM Cortex-M3 processor, as well as the general-purpose I^{2}C interface, see Section 13.5, “Control and Configuration Registers” on page 212.

13.2.4.1 D_DEBUG_CFG Settings

Bit Field	Field Name	Description
3	DBG_REG_WRITTEN	Data register write status
2	DBG_REG_READ	Data register read status
1	AUDIO_MODE	Enable/disable the debug port audio streaming mode
0	ACCESS_MODE	Restrict/unrestrict access through the debug port

Field Name	Value Symbol	Value Description	Hex Value
DBG_REG_WRITTEN	DEBUG_PORT_NOT_WRITTEN	The CFX debug port register has not been written	0x0*
	DEBUG_PORT_WRITTEN	The CFX debug port register has been written	0x1
DBG_REG_READ	DEBUG_PORT_NOT_READ	The CFX debug port register has not been read	0x0*
	DEBUG_PORT_READ	The CFX debug port register has been read	0x1
AUDIO_MODE	AUDIO_STREAM_DISABLED	The CFX debug port is not in audio streaming mode	0x0*
	AUDIO_STREAM_ENABLED	The CFX debug port is in audio streaming mode	0x1
ACCESS_MODE	ACCESS_UNRESTRICTED	Access through the debug ports is not restricted	0x0
	ACCESS_RESTRICTED	Access through the debug ports is restricted	0x1*

13.2.4.2 D_DEBUG_DATA

Bit Field	Field Name	Description
31:0	DEBUG_DATA	Debug port data register

13.2.4.3 D_EXT_BREAK_CFG Settings

Bit Field	Field Name	Description
8:6	COMP_MODE	Select the comparison mode
5:4	ADDR_COND	Select the address condition
3:2	ACCESS_COND	Select the access or data condition
1:0	BUS_SELECT	Select the data bus to monitor for external breakpoints

Field Name	Value Symbol	Value Description	Hex Value
COMP_MODE	EXTBREAK_DOUBLE_ADDR	Double address mode	0x0*
	EXTBREAK_32BIT_DATA	32-bit data mode	0x1
	EXTBREAK_24BIT_DATA_HIGH	8-bit address, 24-bit high data mode	0x2
	EXTBREAK_24BIT_DATA_LOW	8-bit address, 24-bit low data mode	0x3
	EXTBREAK_16BIT_DATA_HIGH	16-bit address, 16-bit high data mode	0x4
	EXTBREAK_16BIT_DATA_MID	16-bit address, 16-bit mid data mode	0x5
	EXTBREAK_16BIT_DATA_LOW	16-bit address, 16-bit low data mode	0x6
	EXTBREAK_MEM_CONFLICT	Shared memory conflict mode	0x7
ADDR_COND	EXTBREAK_ADDR_EQUAL	Double address mode: equal to address 1 or 2	0x0*
	EXTBREAK_ADDR_RANGE	Double address mode: greater than or equal to address 1 and less than or equal to address 2	0x1
	EXTBREAK_ADDR_GT	Double address mode: greater than or equal to address 1 or equal to address 2	0x2
	EXTBREAK_ADDR_LT	Double address mode: equal to address 1 or less than or equal to address 2	0x3
	EXTBREAK_ADDRDATA_GTE	Data mode: address greater than or equal (zone)	0x0*
	EXTBREAK_ADDRDATA_LT	Data mode: address less than (zone)	0x1
	EXTBREAK_ADDRDATA_EQU	Data mode: address equal (zone)	0x2
	EXTBREAK_ADDRDATA_SEQU	Data mode: address strictly equal (including padded zeros)	0x3
ACCESS_COND	EXTBREAK_ACCESSDATA_EQU	Data mode: equal (zone)	0x0*
	EXTBREAK_ACCESSDATA_NOTEQU	Data mode: not equal (zone)	0x1
	EXTBREAK_ACCESSDATA_GTE	Data mode: greater than or equal (zone)	0x2
	EXTBREAK_ACCESSDATA_LT	Data mode: less than (zone)	0x3
	EXTBREAK_ACCESS_FALSE	Double address mode: false (disabled)	0x0*
	EXTBREAK_ACCESS_RW	Double address mode: read or write access	0x1
	EXTBREAK_ACCESS_READ	Double address mode: read access	0x2
	EXTBREAK_ACCESS_WRITE	Double address mode: write access	0x3

Field Name	Value Symbol	Value Description	Hex Value
BUS_SELECT	EXTBREAK_BUS_DISABLED	No bus (disabled)	0x0*
	EXTBREAK_BUS_XMEM	XMEM	0x1
	EXTBREAK_BUS_YMEM	YMEM	0x2
	EXTBREAK_BUS_PMEM	PMEM	0x3

13.2.4.4 D_EXT_BREAK_DATA Settings

Bit Field	Field Name	Description
31:0	EXT_BREAK_DATA	External breakpoint data
2	CM3_FIFO_CONFLICT	Configured detection of FIFO memory conflicts between the ARM Cortex-M3 core or DMA, and the HEAR or CFX
1	CFX_CM3_CONFLICT	Configured detection of memory conflicts between the CFX and ARM Cortex-M3 core
0	CFX_HEAR_CONFLICT	Configured detection of memory conflicts between the CFX and HEAR

Field Name	Value Symbol	Value Description	Hex Value
CM3_FIFO_CONFLICT	EXTBREAK_CM3_FIFO_CONFLICT_DISABLE	Don't trigger on memory conflicts between the ARM Cortex-M3 processor/DMA and HEAR/CFX when accessing FIFOs	0x0*
	EXTBREAK_CM3_FIFO_CONFLICT_ENABLE	Trigger on conflicts between the ARM Cortex-M3 processor/DMA and HEAR/CFX when accessing the FIFOs	0x1
CFX_CM3_CONFLICT	EXTBREAK_CFX_CM3_CONFLICT_DISABLE	Don't trigger on conflicts between the CFX and ARM Cortex-M3 processor/DMA	0x0*
	EXTBREAK_CFX_CM3_CONFLICT_ENABLE	Trigger on conflicts between the CFX and ARM Cortex-M3 processor/DMA	0x1
CFX_HEAR_CONFLICT	EXTBREAK_CFX_HEAR_CONFLICT_DISABLE	Don't trigger conflicts between the CFX and HEAR	0x0*
	EXTBREAK_CFX_HEAR_CONFLICT_ENABLE	Trigger on conflicts between the CFX and HEAR	0x1

13.2.4.5 D_I2C_CFX_DEBUG_ADDR1

Bit Field	Field Name	Description
6:0	DEBUG_ADDR	Defines the first CFX processor debug I^2C slave address

Field Name	Value Symbol	Value Description	Hex Value
DEBUG_ADDR	DEFAULT_CFX_DBG_I2C_SLAVE_ADDR	Default CFX I^2C debug slave address	0x60*

13.2.4.6 D_I2C_CFX_DEBUG_ADDR2

Bit Field	Field Name	Description
6:0	DEBUG_ADDR	Defines the second CFX processor debug I2C slave address

13.2.4.7 D_I2C_CM3_DEBUG_ADDR

Bit Field	Field Name	Description
6:0	DEBUG_ADDR	Defines the ARM Cortex-M3 processor debug I2C slave address

Field Name	Value Symbol	Value Description	Hex Value
DEBUG_ADDR	DEFAULT_CM3_DBG_I2C_SLAVE_ADDR	Default ARM Cortex-M3 core I2C debug slave address	0x40*

13.3 ARM Cortex-M3 Core Debug Port I2C Interface

The ARM Cortex-M3 core debug port I2C interface implements a hardware debug connection for the ARM Cortex-M3 processor in the Ezairo 7100 system. The debug port implements an ARM Cortex-M3 core debug port protocol command set that is tightly coupled with the ARM Cortex-M3 core and the memory components attached to this core. This includes all of the memory mapped registers that are used to control the various components of the ARM Cortex-M3 processor subsystem. This interface is always enabled regardless of the configuration of the general-purpose I2C interface; however, it can be blocked if the CFX debug port shares the same I2C address.

This debug port implements an interface that is used to replace the standard SWJ-DP provided by ARM for use with their processors. Compared to the SWJ-DP, the I2C debug port implementation provided by the Ezairo 7100 system changes only the physical interface layer from the standard implementations from ARM. Underlying this physical interface layer, the debug port implements the AHB_AP to support full debugging with data matching as required for watchpoint generation. Because this interface uses an underlying set of standard blocks for all aspects of this debug port beyond the physical I2C interface implementation, information on the ARM Cortex-M3 debug port can be found in the *Cortex-M3 Technical Reference Manual*. The underlying standard debug components implemented as part of this interface include:

- The Flash Patch and Breakpoint (FPB) unit, which supports six hardware breakpoints and two literal breakpoints
- The Data Watchpoint and Trace (DWT) unit, which supports only the data watch component of this standard component

The ARM Cortex-M3 debug port does not include any of the optional trace support blocks or the memory protection unit for the ARM Cortex-M3 processor. The blocks that are not included are the:

- Memory Protection Unit (MPU)
- Trace Port Interface Unit (TPIU)
- Embedded Trace Macrocell (ETM)
- The trace component of Data Watchpoint and Trace (DWT)
- Instruction Trace Macrocell (ITM)
- AHB Trace Macrocell

The debug port I^{2}C interface always responds to communications at one configurable address that defaults to an address of 0x40. This address is an externally-programmable address that can be configured using the **Write Cortex-M3 Address** CFX debug port command. For more information about this command, see the *Communication Protocols Manual for Ezairo 7100*.

CAUTION: Since the CFX's debug port and the ARM Cortex-M3 processor's debug port share the same physical bus, if both interfaces are configured to use the same address, the CFX debug port takes precedence by limiting or blocking communications with the ARM Cortex-M3 debug port.

13.3.1 Halting Debug Configuration and Status

The Debug Halting Control and Status Register (`DHCSR`) provides status information on the processor state, enables core debugging, and allows an external system to halt and single step the core. To write to this register, `DEBUG_HALT_KEY` must be written to the `CoreDebug_DHCSR_DBGKEY` bit field.

The `DHCSR` is used to configure the ARM Cortex-M3 processor for halting debug. To enable halting debug, set the `CoreDebug_DHCSR_C_DEBUGEN` bit. If halting debug is enabled:

- To halt the core, set the `CoreDebug_DHCSR_C_HALT` bit
- To single-step the core, set the `CoreDebug_DHCSR_C_STEP` bit
- To mask interrupts while single-stepping, set the `CoreDebug_DHCSR_C_MASKINTS` bit (the core must be halted to write this bit)
- To break a stalled memory access, where the memory access might be stalled due to a memory conflict with another component of the Ezairo 7100 system, set the `CoreDebug_DHCSR_C_SNAPALL` bit

The Debug Exception and Monitor Control Register (`DEMCR`) contains a number of possible exception conditions that the debug tools might want to monitor during debug. When enabled, each of these vector catch configuration bits monitors for the specified fault or reset event. When a fault or event that is being monitored is detected, a core halt request is used to halt the core as soon as the currently executing instruction completes. Supported vector catch events include debug traps that will trigger on:

- A core reset
- A memory management fault
- Usage faults for:
  - No coprocessor errors
  - Unaligned accesses or division by 0
  - State errors

- A bus fault
- Errors when handling an interrupt or exception
- A hard fault

To enable watchpoints (triggered using the DWT block), set the `CoreDebug_DEMCR_TRCENA` bit from the `DEMCR` register.

The `DHCSR` also provides a variety of debug related status information, including:

- If the core has been reset or is resetting (`CoreDebug_DHCSR_S_RESET_ST`); this bit is cleared when read
- If an instruction has completed execution since this register has been last read; this bit is cleared when read
- If the core is in a locked state

- If the core is in sleep mode
- If the core has been halted
- If the most recent register read/write has completed; for more information, see Section 13.3.3, “ARM Cortex-M3 Processor Core Register Access”

NOTE: The DHCSR and all DEMCR bits that are not related to the debug monitor are only reset if a POR or similar occurs. These registers are not reset for a core reset. For more information about resets, see Section 19.3, “Resets” on page 478.

CAUTION: We strongly recommend that only the debugger use the DHCSR because accesses to this register from application code can interfere with the debug behavior of the ARM Cortex-M3 processor debug port.

13.3.2 Debug Monitor Configuration

The NVIC from the ARM Cortex-M3 processor contains a debug monitor that can be used to control debug activities. The debug monitor is tied to the debug monitor system interrupt (vector number 12) and is configured using the DEMCR. To enable the debug monitor and debug monitor exception, set the CoreDebug_DEMCR_MON_EN bit from the DEMCR. To manually pend the debug monitor exception, set the CoreDebug_DEMCR_MON_PEND bit from the DEMCR. To single-step the core using the debug monitor (if the debug monitor is enabled), set the CoreDebug_DEMCR_MON_STEP bit from the DEMCR.

The CoreDebug_DEMCR_MON_REQ bit from the DEMCR indicates if a debug monitor event was caused by a manual request or if it was caused by a debug event (including a debug trap).

13.3.3 ARM Cortex-M3 Processor Core Register Access

The ARM Cortex-M3 debug port includes a pair of registers that the debug port uses to provide read and write access to the ARM Cortex-M3’s core registers: the Debug Core Register Selector Register (DCRSR) and the Debug Core Register Data Register (DCRDR). The DCRSR contains the selection of the register to be read or written, and the type of access used. To define the read/write direction, write REGWNR_READ or REGWNR_WRITE to the CoreDebug_DCRSR_REGWnR bit field. To set the register to be read, use the REGSEL_* bit settings for the CoreDebug_DCRSR_REGSEL bit field from the DCRSR.

Data written using the DCRSR is copied from the DCRDR to the specified core register. Similarly data read using the DCRSR is written to the DCRDR, where it can be accessed using debug port memory reads. If the selector register selects the core special registers, the data read or written is interpreted using the bit fields described in Table 34.

Table 34. Debug Core Register Data Register Special Register Mapping

Bit Field	ARM Cortex-M3 Processor Register
31:24	CONTROL
23:16	FAULTMASK
15:8	BASEPRI
7:0	PRIMASK

13.3.4 Debug Fault Status Register

The Debug Fault Status register (DFSR) is used to monitor for debug events including:

- External debug requests

- Vector catches
- Data watchpoint matches
- BKPT instruction execution
- Halt requests

Each flag in the Debug Fault Status register is set independently when its debug condition occurs. The bits in this register are not set unless the event is caught. One of four things occurs if an event is detected:

If halting debug is enabled

Debug events stop the processor by going into debug mode

If halting debug is disabled and the debug monitor is enabled

Debug events trigger a debug monitor handler call, if priority permits.

If halting debug and the debug monitor are both disabled

Some debug events are interpreted as Hard Faults, setting the SCB_HFSR_DBGEVT bit in the Hard Fault status register. This always includes BKPT events, and includes VCATCH and DWTTRAP events (see Section 13.3.5.1, “SCB_DFSR Settings” on page 202 for descriptions of the events) if those events are enabled in the Debug Exception and Monitor Control Register (see Section 13.3.5.4, “Debug Exception and Monitor Control Register” on page 203).

If not interpreted as a hard fault, the debug event is ignored.

This register is part of the ARM Cortex-M3 processor’s SCB register block.

13.3.5 ARM Cortex-M3 Debug Port Specific Control and Configuration Registers

Register Name	Register Description	Address
SCB_DFSR	NVIC Debug Fault Status Register	0xE000ED30
CoreDebug_DHCSR	Debug Halting Control and Status Register	0xE000EDF0
CoreDebug_DCRSR	Debug Core Register Selector Register	0xE000EDF4
CoreDebug_DCRDR	Debug Core Register Data Register	0xE000EDF8
CoreDebug_DEMCR	Debug Exception and Monitor Control Register	0xE000EDFC

IMPORTANT: The ARM Cortex-M3 debug port components are part of a standard component provided with the ARM Cortex-M3 processor. The registers for this peripheral are defined in *core_cm3.h* and augmented by defines for the bit fields, bit settings and subregisters in *sk4_hw.h*. These registers are only accessible through the ARM Cortex-M3’s private peripheral bus (see Section 7.2.2, “Memory Bus Structures” on page 76), and are not accessible by the CFX DSP.

For control and configuration registers that apply to the debug port I^2C interfaces for the CFX and ARM Cortex-M3 processor, as well as the general-purpose I^2C interface, see Section 13.5, “Control and Configuration Registers” on page 212.

13.3.5.1 SCB_DFSR Settings

Bit Field	Field Name	Description
4	EXTERNAL	Indicate if an external debug request has been asserted. The processor stops on the next instruction boundary.
3	VCATCH	Indicate if a vector catch occurred. When the VCATCH flag is set, a flag in one of the local fault status registers is also set to indicate the type of fault. For information about configuring the system to catch specific vectors, see Section 13.3.5.4, “Debug Exception and Monitor Control Register”.
2	DWTTRAP	Indicate if a data watchpoint has been matched. The processor stops at the current instruction or at the next instruction.
1	BKPT	Indicate if a `BKPT` instruction executed. The `BKPT` flag is set by a `BKPT` instruction in code. The PC’s return value points to the instruction containing the breakpoint.
0	HALTED	Indicate if a halt was requested by the NVIC (including a halt due to a single-step debug operation).

13.3.5.2 CoreDebug_DHCSR Settings

Bit Field	Field Name	Description
31:16	KEY	Must be written with the DEBUG_HALT_KEY (0xA05F) to write to other parts of this register
25	S_RESET_ST	Status bit indicating if the core was reset or is being reset since this register was last read
24	S_RETIRE_ST	Status bit indicating an instruction completed since this register was last read
19	S_LOCKUP	Status bit indicating if the ARM Cortex-M3 processor is in a locked state
18	S_SLEEP	Status bit indicating if the ARM Cortex-M3 processor is in sleep mode
17	S_HALT	Status bit indicating if the ARM Cortex-M3 processor is halted
16	S_REGRDY	Status bit indicating if a register read or write has completed since this register was last read
5	C_SNAPSTALL	Control bit used to break a stalled memory access
3	C_MASKINTS	Control bit used to mask interrupts while single-stepping; this bit can only be modified if the ARM Cortex-M3 processor is halted
2	C_STEP	Control bit used to single-step the ARM Cortex-M3 processor if halt mode debugging is enabled
1	C_HALT	Control bit used to halt the ARM Cortex-M3 processor if halt mode debugging is enabled
0	C_DEBUGEN	Control bit used to enable halt mode debugging

13.3.5.3 CoreDebug_DCRSR Settings

Bit Field	Field Name	Description
16	REGWnR	Write/not-read bit controlling the direction of the register transfer
4:0	REGSEL	Select the core register to read from or write to

13.3.5.4 Debug Exception and Monitor Control Register

Bit Field	Field Name	Description
24	TRCENA	Trace system enable bit; needed to enable the data watchpoint and trace (DWT) block
19	MON_REQ	Indicate if a debug monitor exception has been triggered by a pending manual request or a hardware debug event
18	MON_STEP	Single step the core; this bit can only be written if the debug monitor exception is enabled
17	MON_PEND	Manually pend a debug monitor exception
16	MON_EN	Enable the debug monitor exception
10	VC_HARDERR	Capture hard fault events
9	VC_INTERR	Capture errors encountered when serving an interrupt
8	VC_BUSERR	Capture bus fault events
7	VC_STATERR	Capture usage faults due to a state error
6	VC_CHKERR	Capture usage faults for the optionally enabled usage errors (i.e., due to an unaligned access or division by zero); for information on configuring these optional error sources, see Section 15.1.10, “Configuration Control Register” on page 248
5	VC_NOCPERR	Capture usage faults due to a no-coprocessor error
4	VC_MMERR	Capture memory management faults
0	VC_CORERESET	Capture core reset events

13.4 General Purpose I²C Interface

The general-purpose I²C interface provides a half-duplex communications channel that operates at speeds of up to 1 MHz using the I²C bus it shares with the debug ports. During any transaction, the `I2C_CTRL_RW` bit from the `D_I2C_CTRL` register indicates whether the current I²C transaction is a read or write transaction.

The `D_I2C_CTRL` register is the interface control register. Within `D_I2C_CTRL`, the bit fields are defined as either event bits, state bits, or write-only bits. The event bits are used to indicate that an event has occurred, the state bits are used to indicate an interface operating state, and the write-only bits are used to trigger an event. All of the bit fields and their corresponding types are summarized in Table 35.

Table 35. I²C Control Bits

Bit Field	Field Name	Bit Type	Field Description
14	`I2C_CTRL_ERROR_STICKY`	Event	Indicate if an error (I²C bus or I²C watchdog) has occurred. `I2C_CTRL2_ERROR_STICKY` is 0 when `D_I2C_ERROR_CNT` contains 0 and `I2C_CTRL2_BUS_ERROR_STICKY` is not 1.
13	`I2C_CTRL_BUS_ERROR_STICKY`	Event	Indicate if an I²C bus error has occurred. Automatically cleared when the `D_I2C_CTRL2` register is read, unless the bus error is still present. The bus error disappears when the I²C interface detects a stop condition.
12	`I2C_CTRL_START_PENDING`	State	Indicate if an I²C master transfer is pending. Transition out of this state occurs when the pending I²C master transfer has started.

Table 35. I²C Control Bits

Bit Field	Field Name	Bit Type	Field Description
11	I2C_CTRL_MASTER_MODE	State	Indicate if the I²C interface is currently operating in master mode. Transition out of this state occurs when the I²C master transfer has finished, which happens when a stop condition or bus error occurs.
10	I2C_CTRL_STOP_DETECTED	Event	Indicate if a stop condition has been detected. Cleared after a new transaction starts, or after reading the D_I2C_CTRL register.
9	I2C_CTRL_DATA_EVENT	Event	Indicate if data has been received or is needed for transmission. Cleared after reading data from or writing data to the D_I2C_DATA register.
8	I2C_CTRL_ERROR	Event	Indicate if an I²C bus error has occurred since the last stop condition or if an I²C watchdog error has occurred. Cleared after reading the D_I2C_ERROR_CNT register. D_I2C_CTRL_ERROR is 0 when D_I2C_ERROR_CNT contains 0 and I2C_CTRL_BUS_ERROR is not 1.
7	I2C_CTRL_BUS_ERROR	Event	Indicate if an I²C bus error has occurred since the last stop condition. Automatically cleared when the bus error disappears, which happens when the I²C interface detects a stop condition.
6	I2C_CTRL_BUFFER_FULL	State	Indicate that the I²C buffer is full. Transition out of this state occurs after reading the D_I2C_DATA register or transmitting the next byte.
5	I2C_CTRL_CLK_STRETCH	State	Indicate that the SCL line of the I²C bus is being held due to an empty transmit buffer. Transition out of this state occurs after writing the D_I2C_DATA register.
5	I2C_CTRL_RESET	Write-Only	Abort any current transaction and reset the I²C interface.
4	I2C_CTRL_LINE_FREE	State	Indicate that both I²C bus lines are currently high. Transition out of this state occurs when either SDA or SCL are pulled or driven low.
4	I2C_CTRL_LAST_DATA	Write-Only	Indicate that a byte is the last byte in a transaction, and will automatically terminate an I²C transfer.
3	I2C_CTRL_ADDR_DATA	State	Indicate that an address has been sent or received. Transition out of this state occurs after a data byte has been sent or received.
3	I2C_CTRL_STOP_CMD	Write-Only	Issue a stop condition (master mode only).
2	I2C_CTRL_RW	State	Indicate the state of the Read/Write direction bit last sent or received. Transition out of this state occurs after a new Read/Write direction bit is sent or received.
1	I2C_CTRL_GEN_CALL	State	Indicate that the last address sent or received was the general call address (0x0). Transition out of this state occurs after a non-general call address is sent or received.
1	I2C_CTRL_NACK	Write-Only	Negative acknowledgement of a sent/received byte.
0	I2C_CTRL_ACK	Write-Only	Acknowledgement of a sent/received byte.

IMPORTANT: We recommend using the I2C_CTRL_ERROR_STICKY and I2C_CTRL_BUS_ERROR_STICKY bits when testing for I²C errors. Only use the ERROR and BUS_ERROR bits to check if an error condition still exists on the I²C interface.

NOTE: The interface supports both master operation and slave operation. A user application can use the interface in both modes provided the application ensures that the I2C bus is not currently in use. Check the I2C_CTRL_LINE_FREE bit in the D_I2C_CTRL register to verify that the I2C bus is not currently in use when attempting to initialize a transfer as a master device.

The general-purpose I2C interface uses the D_I2C_DATA register to transmit and receive data through a 1-byte internal buffer. As use of the D_I2C_DATA register has side effects that can advance the I2C interface state machine, this register is mirrored without side effects in the D_I2C_DATA_M register. If the internal buffer is full, the I2C_CTRL_BUFFER_FULL bit from the D_I2C_CTRL register is set.

Generally each byte of an I2C transaction must be acknowledged or not-acknowledged by a user application. Use the I2C_CFG_AUTO_ACK bit from the D_I2C_CFG register to select whether the application itself handles this acknowledgement manually or the interface handles it automatically. In either case, if the interface does not have the needed data or a place to put data, the transaction is paused by stretching the clock signal until the interface can continue. If the clock is being stretched, the I2C_CTRL_CLK_STRETCH bit from the D_I2C_CTRL register is set as an indicator.

Use of the above mentioned data registers and the general behavior of the I2C state machine is described in Section 13.4.3.1, “Operation Using Manual Acknowledgement” on page 207 for manual acknowledgement mode, and in Section 13.4.3.2, “Operation Using Auto Acknowledgement” on page 208 for automatic acknowledgement mode.

NOTE: If at any point, when operating in either master or slave mode, a user application needs to reset the I2C bus lines (for example, after encountering a bus error condition), the application should write the I2C_CTRL_RESET bit from the D_I2C_CTRL register. This resets both the I2C bus and the internal state machines of the I2C interface to allow the system to cleanly start again from a known state.

13.4.1 General Purpose I2C Interface - Slave Mode Specific Configuration

In slave mode, the device is receiving the serial clock signal from an external master device, which also controls addressing, direction and the start/stop conditions for each I2C transfer.

To enable a device in slave mode, set the I2C_CFG_I2C_SLAVE_EN bit from the D_I2C_CFG register. This allows the device to respond to communications on both the I2C general call address (0x00) and at a programmable slave address that can be selected using the I2C_CFG_SLAVE_ID bit field in the D_I2C_CFG register. During a transaction, the I2C_CTRL_GEN_CALL bit in the D_I2C_CTRL register can be queried to identify which of the addresses was used to address the device.

IMPORTANT: If at any point the Ezairo 7100 system is addressed using the I2C bus at an address configured for use by both a debug port interface and the general-purpose interface, the debug port interface will take precedence.

A device receiving data in a slave mode configuration can use the I2C_CTRL_LAST_DATA bit from the D_I2C_CTRL register to automatically NACK the last data byte to indicate to a master device that it should send a stop condition.

NOTE: The I2C_CTRL_LAST_DATA bit is cleared when a new transfer is initiated on the I2C bus.

13.4.2 General Purpose I2C Interface - Master Mode Specific Configuration

In master mode, the device provides the serial clock signal and controls all transfer information. To configure the clock signal, select a prescaling division by 3 using the I2C_CFG_MASTER_SPEED bit field in the D_I2C_CFG register. The interface clock frequency for a given configuration can be calculated using:

$$F_{I2C\ MASTER} = \frac{SYSCLK}{3 \cdot (I2C_CFG_MASTER_SPEED + 1)}$$

As noted in Section 13.1, “Inter-IC Interface (I2C) Bus®” on page 188, the I2C interface is designed to operate at a maximum speed of 400 kHz.

A user application must manually control all components of a master mode transaction. To transmit or receive using master mode, a device must:

1. Check that the I2C bus lines are not currently in use by reading the I2C_CTRL_LINE_FREE bit in the D_I2C_CTRL register. It can only start a transaction if the lines are free.
2. Start a transaction, sending the start condition, address and direction, by loading the appropriate address and direction to the D_I2C_ADDR_START register.

NOTE: Using a slave address that matches one of the debug port addresses is technically allowed for a master device; however, the local debug port I2C interface will not respond to communications originating from the general-purpose I2C interface. Due to possible confusion related to this unusual configuration (since, for example, another master producing the same I2C bus traffic will observe different behavior), we recommend that users avoid using debug port assigned I2C addresses for I2C master mode transactions.

3. Handle data and interrupts as appropriate for the transaction (see Section 13.4.3, “General Purpose I2C Interrupts”).
4. Complete the transaction by sending a stop condition.

   A stop condition can be generated using either the I2C_CTRL_STOP_CMD bit or the I2C_CTRL_LAST_DATA bit in the D_I2C_CTRL register.

   When using the I2C_CTRL_LAST_DATA bit, a stop condition is automatically generated after the current data byte transfer is completed. If the interface is receiving data, this bit will also automatically generate the required NACK of the last data byte that is received.

NOTE: If the I2C interface is used for transmitting, the last byte of data to be transmitted must be written to the D_I2C_DATA register before setting the I2C_CTRL_LAST_DATA bit, as this bit triggers a stop condition at the end of the current transfer.

NOTE: As previously noted, the I2C_CTRL_LAST_DATA bit is cleared when a new transfer is initiated on the I2C bus.

When using the I2C_CTRL_STOP_CMD bit, a stop condition is issued immediately. This bit is normally set when the I2C_CTRL_ACK or I2C_CTRL_NACK bits in the D_I2C_CTRL register are set.

CAUTION: Using the stop event (STOP) immediately transmits a stop condition as soon as possible. This can result in a terminated transfer, and remote devices on the bus might detect I2C errors.

13.4.3 General Purpose I2C Interrupts

The general-purpose I2C interface uses an associated interrupt which, when enabled, signals the receipt of a correct address byte and the completion of each data byte in the transaction.

When enabled, the I2C interrupt signals the stop condition following a transaction for a master transfer. The system can also be configured to receive this interrupt when operating in slave mode by setting the I2C_CFG_STOP_INT bit from the D_I2C_CFG register.

See Section 14.1, “Interrupt Controller” on page 218 for information regarding interrupt configuration and handling.

Several indicators can be used with the interrupt to identify the state of the general-purpose I2C interface:

- If the I2C_CTRL_ADDR_DATA bit from the D_I2C_CTRL register is set, the interrupt was generated in response to a recognized sequence including a start condition and address on the general-purpose I2C interface.
- If the I2C_CTRL_STOP_DETECTED bit from the D_I2C_CTRL register is set, the interrupt was generated in response to a stop interrupt being detected on the general-purpose I2C interface. This bit is cleared immediately when a new transaction starts. No interrupt is generated for I2C transactions that use an address that was not recognized by the general-purpose I2C interface.
- If the I2C_CTRL_DATA_EVENT bit from the D_I2C_CTRL register is set, the interrupt indicates that data has been received and can be read, or that data is needed to continue with the transmission of data.
- If the I2C_CTRL_BUS_ERROR_STICKY bit from the D_I2C_CTRL register is set, then a bus error has occurred. If the I2C_CTRL_BUS_ERROR bit from the D_I2C_CTRL register is set, the bus error has not yet been cleared.
- If the I2C_CTRL_ERROR_STICKY bit from the D_I2C_CTRL register is set, then either a bus error or an I2C watchdog timeout has occurred. If the I2C_CTRL_ERROR bit from the D_I2C_CTRL register is set, the error has not yet been cleared.

13.4.3.1 Operation Using Manual Acknowledgement

If a user application is using manual acknowledgement (i.e., the I2C_CFG_AUTO_ACK bit from the D_I2C_CFG register is disabled), it must use the I2C_CTRL_NACK and I2C_CTRL_ACK bits from the D_I2C_CTRL register to acknowledge (ACK) or not-acknowledge (NACK) transaction bytes.

When handling a transaction, an interrupt is issued whenever data or an acknowledgement is required (in addition to interrupts triggered by stop conditions).

NOTE: An interrupt is not generated following a NACK for any data condition. However, an interrupt is generated (if operating in master mode or stop interrupts are enabled) on the stop condition that is expected to follow the NACK.

An interrupt is also generated if the interface cannot send data that has been provided because the interface is idle or has already received a NACK from the slave device during the current transaction. As noted above, when the interface is waiting for data, the clock line is automatically stretched until the required data is provided.

A special interrupt is also generated for a master read once the acknowledge bit from the address byte has been received. This scenario is a special case because it does not require data nor acknowledgement to trigger an interrupt, but instead is designed to allow the interface to check whether or not a slave device has acknowledged the transfer at that point. Provided the slave device responded, the interface can issue either an ACK or a NACK command to continue the transfer.

NOTE: Use of the `I2C_CTRL_NACK`, `I2C_CTRL_ACK` and `I2C_CTRL_STOP_CMD` bits from the `D_I2C_CTRL` register is only defined when an acknowledgement is needed during a data transfer. Interface behavior in response to these control bits being set at other times is undefined.

13.4.3.2 Operation Using Auto Acknowledgement

When operating using auto acknowledgement (i.e., the `I2C_CFG_AUTO_ACK` bit from the `D_I2C_CFG` register is enabled), transfers are controlled by the availability of bytes for transmission, or a place to put a byte when receiving data. This allows transfers to occur without clock stretching, provided the firmware is able to handle each data byte fast enough.

When the interface handles a transaction, interrupts are issued whenever new data can be written or new data has been received. This allows a user application to maintain a transaction without introducing delay.

NOTE: An interrupt is not generated following a NACK for any data condition. However, an interrupt is generated (if operating in master mode or stop interrupts are enabled) on the stop condition that is expected to follow the NACK.

Additionally, interrupts are generated if data was supplied but cannot be sent because the interface received a NACK from the slave device, or is currently idle (as the interface would be following a stop condition). If data is required but is not currently available because the buffer is empty, the clock is stretched and an additional interrupt is generated. If new data was received, but the previous data has not been read (resulting in the buffer not being available), the clock is stretched until the buffer becomes free.

IMPORTANT: When receiving data in master mode, the timing of interrupts for auto acknowledgement of data prevents a user application from cleanly terminating a transfer using the LAST_DATA event. As a result, if the interface is running in this mode, a dummy byte must be transferred to cleanly terminate the current transfer with a stop condition following this byte.

13.4.4 General Purpose I²C Interface Specific Control and Configuration Registers

Register Name	Register Description	Address
`D_I2C_CFG`	I²C configuration registers	P:0xF200
`D_I2C_CTRL`	I²C control registers	P:0xF201
`D_I2C_ADDR_START`	Initiate a transaction sending a start condition and address/direction	P:0xF206
`D_I2C_DATA`	I²C transmission data	P:0xF202
`D_I2C_DATA_M`	I²C transmission data read (same as data received but a read does NOT empty the buffer)	P:0xF203

For control and configuration registers that apply to both the debug port I²C interface and the general-purpose I²C interface, see Section 13.5, “Control and Configuration Registers” on page 212.

13.4.4.1 D_I2C_CFG Settings

Bit Field	Field Name	Description
19:12	MASTER_SPEED	Define the I2C_SCL speed in master mode
10:4	SLAVE_ID	Define the I²C slave address ID
3	STOP_INT	Enable/disable interrupts on stop conditions
2	AUTO_ACK	Auto acknowledge mode
1	CONNECT_IN_STANDBY	Control if the I²C lines are connected when running on the standby clock
0	SLAVE_EN	I²C slave mode enable

Field Name	Value Symbol	Value Description	Hex Value
MASTER_SPEED	I2C_MASTER_PRESCALE_3	I²C master speed is SYSCLK/3	0x0*
	I2C_MASTER_PRESCALE_6	I²C master speed is SYSCLK/6	0x1
	I2C_MASTER_PRESCALE_9	I²C master speed is SYSCLK/9	0x2
	I2C_MASTER_PRESCALE_12	I²C master speed is SYSCLK/12	0x3
	I2C_MASTER_PRESCALE_15	I²C master speed is SYSCLK/15	0x4
	I2C_MASTER_PRESCALE_18	I²C master speed is SYSCLK/18	0x5
	I2C_MASTER_PRESCALE_21	I²C master speed is SYSCLK/21	0x6
	I2C_MASTER_PRESCALE_24	I²C master speed is SYSCLK/24	0x7
	I2C_MASTER_PRESCALE_27	I²C master speed is SYSCLK/27	0x8
	I2C_MASTER_PRESCALE_30	I²C master speed is SYSCLK/30	0x9
	I2C_MASTER_PRESCALE_33	I²C master speed is SYSCLK/33	0xA
	I2C_MASTER_PRESCALE_36	I²C master speed is SYSCLK/36	0xB
	I2C_MASTER_PRESCALE_39	I²C master speed is SYSCLK/39	0xC
	I2C_MASTER_PRESCALE_42	I²C master speed is SYSCLK/42	0xD
	I2C_MASTER_PRESCALE_45	I²C master speed is SYSCLK/45	0xE
	I2C_MASTER_PRESCALE_48	I²C master speed is SYSCLK/48	0xF
	I2C_MASTER_PRESCALE_51	I²C master speed is SYSCLK/51	0x10
STOP_INT	I2C_CFG_STOP_INT_DISABLE	No interrupt on a stop condition	0x0*
	I2C_CFG_STOP_INT_ENABLE	Interrupt on a stop condition	0x1
AUTO_ACK	I2C_CFG_AUTO_ACK_DISABLE	Disable auto acknowledge mode	0x0*
	I2C_CFG_AUTO_ACK_ENABLE	Enable auto acknowledge mode	0x1
CONNECT_IN_STANDBY	I2C_CFG_CONNECT_IN_STANDBY_DISABLE	Do not connect the I²C lines when running on the standby clock	0x0*
	I2C_CFG_CONNECT_IN_STANDBY_ENABLE	Connect the I²C lines when running on the standby clock	0x1
SLAVE_EN	I2C_CFG_SLAVE_DISABLE	Disable I²C slave mode	0x0*
	I2C_CFG_SLAVE_ENABLE	Enable I²C slave mode	0x1

13.4.4.2 D_I2C_CTRL Settings

Bit Field	Field Name	Description
14	ERROR_STICKY	Error status bit (sticky)
13	BUS_ERROR_STICKY	Bus error status bit (sticky) - this status bit is automatically reset when the D_I2C_CTRL register is read.
12	START_PENDING	Master frame start pending status bit
11	MASTER_MODE	Master mode status bit
10	STOP_DETECTED	Indicate if an I2C stop bit has been detected
9	DATA_EVENT	Indicate that I2C interface either needs data to transmit or has received data
8	ERROR	Error status bit (non-sticky)
7	BUS_ERROR	Bus error status bit (non-sticky) - this status bit is automatically reset when the D_I2C_CTRL register is read.
6	BUFFER_FULL	Indicate if the I2C data buffer is full
5	CLK_STRETCH	Clock stretching flag
5	RESET	Reset interface
4	LINE_FREE	Line free flag
4	LAST_DATA	Last data in master mode
3	ADDR_DATA	Address/Data byte
3	STOP_CMD	Send stop command
2	RW	Read/Write frame
1	GEN_CALL	General call flag
1	NACK	Do not acknowledge event
0	ACK_STATUS	Acknowledge status
0	ACK	Acknowledge event

Field Name	Value Symbol	Value Description	Hex Value
ERROR_STICKY	I2C_NO_ERROR_S	BUS_ERROR_STICKY is 0 and I2C_ERROR_CNT equals 0	0x0*
	I2C_ERROR_S	BUS_ERROR_STICKY is 1 or I2C_ERROR_CNT is not equal to 0	0x1
BUS_ERROR_STICKY	I2C_NO_BUS_ERROR_S	I2C interface is not and has not been in the bus error state	0x0*
	I2C_BUS_ERROR_S	I2C interface is or has been in the bus error state	0x1
START_PENDING	I2C_START_NOT_PENDING	No pending master start frame	0x0*
	I2C_START_PENDING	A master frame is pending to start (bit is set when the D_I2C_ADDR_START register is written)	0x1
MASTER_MODE	I2C_MASTER_INACTIVE	I2C interface is not operating in master mode	0x0*
	I2C_MASTER_ACTIVE	I2C interface is operating in master mode	0x1
STOP_DETECTED	I2C_NO_STOP_DETECTED	No stop condition has been detected on the bus	0x0*
	I2C_STOP_DETECTED	A stop condition has been detected on the bus	0x1

Field Name	Value Symbol	Value Description	Hex Value
DATA_EVENT	I2C_NON_DATA_EVENT	No data is needed or available	0x0*
	I2C_DATA_EVENT	I²C data is needed or is available	0x1
ERROR	I2C_NO_ERROR	BUS_ERROR is 0 and I2C_ERROR_CNT equals 0	0x0*
	I2C_ERROR	BUS_ERROR is 1 or I2C_ERROR_CNT is not equal to 0	0x1
BUS_ERROR	I2C_NO_BUS_ERROR	I²C interface is not in the bus error state	0x0*
	I2C_BUS_ERROR	I²C interface is in the bus error state	0x1
BUFFER_FULL	I2C_BUFFER_EMPTY	The I²C interface buffer is empty	0x0*
	I2C_BUFFER_FULL	The I²C interface buffer is full	0x1
CLK_STRETCH	I2C_CLK_NOT_STRETCHED	I²C interface is not in the bus error state	0x0*
	I2C_CLK_STRETCHED	I²C interface is in the bus error state	0x1
RESET	I2C_RESET	Reset the I²C interface	0x1
LINE_FREE	I2C_BUS_BUSY	I²C interface is not in the bus error state	0x0*
	I2C_BUS_FREE	I²C interface is in the bus error state	0x1
LAST_DATA	I2C_LAST_DATA	Indicate that the current data is the last byte of a data transfer	0x1
ADDR_DATA	I2C_DATA_IS_DATA	The I²C data register holds data	0x0*
	I2C_DATA_IS_ADDR	The I²C data register holds an address	0x1
STOP_CMD	I2C_STOP	Issue a stop condition the I²C interface bus	0x1
RW	I2C_IS_WRITE	The current I²C transfer is a write	0x0*
	I2C_IS_READ	The current I²C transfer is a read	0x1
GEN_CALL	I2C_ADDR_OTHER	The address used for the current I²C transfer is not the general call address	0x0*
	I2C_ADDR_GEN_CALL	The address used for the current I²C transfer is the general call address	0x1
NACK	I2C_NACK	Issue a not acknowledge on the I²C interface bus	0x1
ACK_STATUS	I2C_HAS_ACK	Indicate that the last I²C byte was acknowledged	0x0*
	I2C_HAS_NACK	Indicate that the last I²C byte was not acknowledged	0x1
ACK	I2C_ACK	Issue an acknowledge on the I²C interface bus	0x1

13.4.4.3 D_I2C_ADDR_START Settings

Bit Field	Field Name	Description
7:1	ADDR	Address to send with the start condition over the I²C bus
0	RW	Direction of the initiated I²C transfer

Field Name	Value Symbol	Value Description	Hex Value
RW	I2C_START_WRITE	Initiate a write transfer	0x0
	I2C_START_READ	Initiate a read transfer	0x1

13.5 Control and Configuration Registers

Register Name	Register Description	Address
D_I2C_PIN_CFG	I²C interface configuration register	P:0xF207
D_I2C_WATCHDOG_CFG	I²C watchdog timeout register	P:0xF208
D_I2C_ERROR_CNT	I²C timeout counter	P:0xF205

13.5.1 D_I2C_PIN_CFG Settings

Bit Field	Field Name	Description
3	DRIVE	Drive control
2	PULLUP_DIS	Pull-up control
1	PULLUP	Pull-up value selection
0	FILTER_EN	Low Pass Filter control

Field Name	Value Symbol	Value Description	Hex Value
DRIVE	I2C_PIN_CFG_LOW_DRIVE	Low drive strength	0x0
	I2C_PIN_CFG_HIGH_DRIVE	High drive strength	0x1*
PULLUP_DIS	I2C_PIN_CFG_PULLUP_ENABLE	Enable pull-up resistors	0x0*
	I2C_PIN_CFG_PULLUP_DISABLE	Disable pull-up resistors	0x1
PULLUP	I2C_PIN_CFG_PULLUP_250K	250 kOhm pull-up resistors	0x0
	I2C_PIN_CFG_PULLUP_1K	1 kOhm pull-up resistors	0x1*
FILTER_EN	I2C_PIN_CFG_FILTER_DISABLE	Disable low-pass filter	0x0
	I2C_PIN_CFG_FILTER_ENABLE	Enable low-pass filter	0x1*

13.5.2 D_I2C_WATCHDOG_CFG Settings

Bit Field	Field Name	Description
2:0	TIMEOUT	Watchdog timeout configuration

Field Name	Value Symbol	Value Description	Hex Value
TIMEOUT	I2C_WATCHDOG_TIMEOUT_250M	Timeout is 0x0	0x0
	I2C_WATCHDOG_TIMEOUT_500M	Timeout is 0x1	0x1
	I2C_WATCHDOG_TIMEOUT_1000M	Timeout is 0x2	0x2*
	I2C_WATCHDOG_TIMEOUT_2000M	Timeout is 0x3	0x3
	I2C_WATCHDOG_TIMEOUT_4000M	Timeout is 0x4	0x4
	I2C_WATCHDOG_TIMEOUT_DISABLED	Disable the I²C watchdog timer	0x5

13.5.3 D_I2C_ERROR_CNT

Bit Field	Field Name	Description
7:0	I2C_ERROR_CNT	Count the number of I^2C errors since this register was last read

CHAPTER 14
CFX Peripherals

14.1 INTERRUPT CONTROLLER

The Ezairo 7100 system contains an interrupt controller linked to the CFX DSP that services 31 interrupt sources. The controller primarily uses an external interrupt channel which is directly connected to the various interfaces, peripherals and other components that can trigger an interrupt. The controller also uses a software interrupt channel that is triggered by the `interrupt` instruction of the CFX DSP. For more information about the `interrupt` instruction, see the *CFX DSP Architecture Manual*.

Each interrupt source is associated with an interrupt vector that is mapped into program memory starting at 0xFFE0 (mirrored from 0xDFE0). These vectors must be filled with an unconditional branch to the desired interrupt handler for the interrupt source associated with the vector. The possible interrupt sources along with their interrupt vectors and the ordering of interrupts are listed in Table 36. You can find details specifying what events or conditions trigger an interrupt condition for a specific interrupt source in the section of this manual that describes the interrupt source.

Table 36. Interrupt Sources and Vectors

Interrupt Source	Interrupt Vector	Number
Watchdog timer	D_VECT_WATCHDOG	1
General purpose timer 1	D_VECT_TIMER_1	2
General purpose timer 2	D_VECT_TIMER_2	3
SPI interface	D_VECT_SPI	4
I^2C interface	D_VECT_I2C	5
PCM interface	D_VECT_PCM	6
GPIO interface	D_VECT_GPIO	7
HEAR Accelerator (interrupt #0)	D_VECT_HEAR_0	8
HEAR Accelerator (interrupt #1)	D_VECT_HEAR_1	9
HEAR Accelerator (interrupt #2)	D_VECT_HEAR_2	10
HEAR Accelerator (interrupt #3)	D_VECT_HEAR_3	11
HEAR Accelerator (interrupt #4)	D_VECT_HEAR_4	12
HEAR Accelerator (interrupt #5) or UART	D_VECT_HEAR_5, D_VECT_UART	13
HEAR Accelerator (interrupt #6) or Filter Engine	D_VECT_HEAR_6, D_VECT_FENG	14
HEAR Accelerator (interrupt #7) or Signal Detection Unit	D_VECT_HEAR_7, D_VECT_SDU	15
FIFO 0	D_VECT_FIFO_0	16
FIFO 1	D_VECT_FIFO_1	17
FIFO 2	D_VECT_FIFO_2	18
FIFO 3	D_VECT_FIFO_3	19
FIFO 4	D_VECT_FIFO_4	20
FIFO 5	D_VECT_FIFO_5	21
FIFO 6	D_VECT_FIFO_6	22

Table 36. Interrupt Sources and Vectors (Continued)

Interrupt Source	Interrupt Vector	Number
FIFO 7	D_VECT_FIFO_7	23
Clock	D_VECT_CLK	24
ARM Cortex-M3 core SPI	D_VECT_C_SPI	25
ARM Cortex-M3 core PCM	D_VECT_C_PCM	26
ARM Cortex-M3 core I^2C	D_VECT_C_I2C	27
ARM Cortex-M3 core Event 0	D_VECT_CM3_0	28
ARM Cortex-M3 core Event 1	D_VECT_CM3_1	29
ARM Cortex-M3 core Event 2	D_VECT_CM3_2	30
ARM Cortex-M3 core Event 3	D_VECT_CM3_3	31

Even when more than one interrupt is enabled, only one interrupt can be issued for handling at a time. As a result, an application might find it valuable to prioritize certain interrupts over others. Assign either normal or high priority to an interrupt by setting or clearing the appropriate bits in the D_INT_PRIORITY register. This setting is also mapped into the D_INT_PRIO_x registers in 8-bit blocks for efficient configuration of a subset of the interrupt sources. If multiple interrupts are pending, all high priority interrupts are handled before any normal priority interrupts are handled. As well, if multiple pending interrupts share a priority level, the lowest numbered interrupt (as numbered in Table 36) is handled first.

The interrupt enable register, D_INT_ENABLE, provides interrupt enable flags for each of the interrupt vectors. This register also contains the INT_EBL_MASTER bit, which acts as a master enable/disable for all interrupts regardless of their individual settings. These bits have also been mapped into the D_INT_EBL_x and D_INT_DIS_x registers. The D_INT_EBL_x registers provide an interface that allows you to only enable interrupts, which makes it unnecessary to read the interrupt enable register before enabling a set of interrupts. Similarly, the D_INT_DIS_x registers provide an interface that allows you to only disable interrupts. Finally, an interrupt can also be enabled through the D_INT_ACK_EBL_x registers. These registers act like the D_INT_EBL_x registers except that they also acknowledge the interrupts that are being enabled to clear out any interrupt conditions that might have been pending when the interrupt source was enabled.

When an interrupt source generates an interrupt event, the interrupt status bit that corresponds to the interrupt source is set in the D_INT_STATUS register. The bits in this register can be manually cleared to acknowledge their linked interrupt by writing a 1 to either this bit in the D_INT_STATUS register, or to the corresponding bit in one of the D_INT_ACK_x registers. The define INT_ACK_ALL_INTS is available as a short form for use when writing to the D_INT_STATUS register to clear all interrupts. The bit corresponding to an interrupt is also cleared when the CFX DSP handles the associated interrupt. When the CFX DSP is acknowledging an interrupt, no new interrupt signals for that interrupt will be accepted until the following clock cycle.

NOTE: An interrupt that is not enabled is still marked in the status register because the interrupt enable register only masks interrupt notification and not generation. As a result, if you want to ensure that the interrupt triggers only for new interrupt conditions, acknowledge interrupts when the interrupt is enabled. In general, this behavior can also be used to support event polling in the Ezairo 7100 system.

IMPORTANT: Unlike the case where the CFX DSP is automatically acknowledging an interrupt, when an interrupt is acknowledged by a user application, an interrupt signal for it is accepted immediately.

In summary, the interrupt controller selects an interrupt to issue based on the following criteria:

1. The interrupt controller starts with those interrupts that are pending as marked in the interrupt status register (D_INT_STATUS).
2. This list of pending interrupts is culled by the interrupt enable settings in the D_INT_ENABLE register, with all pending interrupts being culled from the list if the INT_EBL_MASTER bit is not set.
3. The remaining list is split into high priority and normal priority interrupts.
4. The first interrupt from the remaining list with a high priority is issued. If no high priority interrupts are pending, the first normal priority interrupt is issued.

This selection criteria is illustrated in Figure 25.

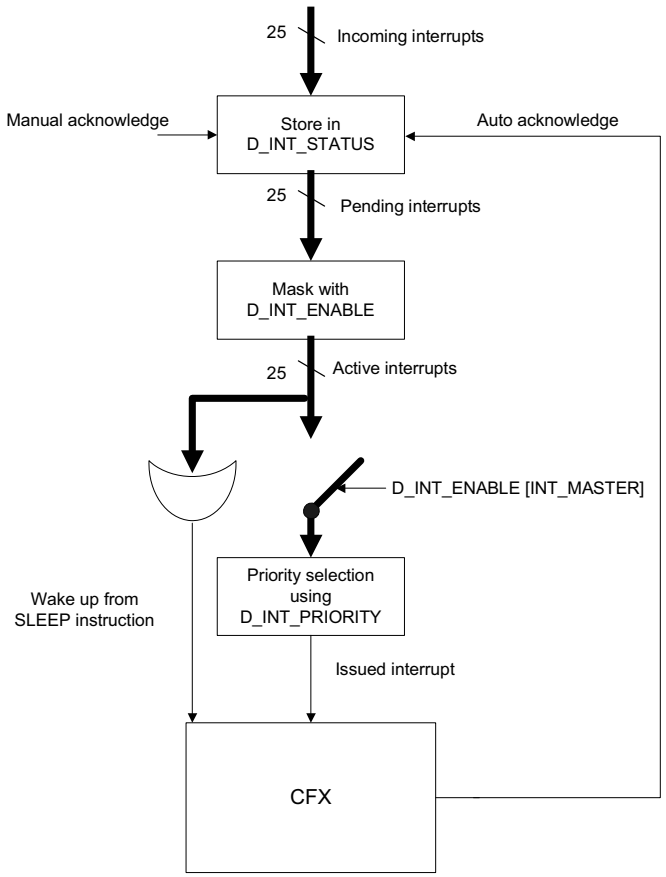


Figure 25. Criteria the Interrupt Controller Uses to Select an Interrupt to Issue

Three of the HEAR Accelerator triggered interrupts to the CFX are multiplexed with other interrupt sources, as shown in Table 36. The multiplexed interrupts and their associated control bits from the D_HEAR_INT_SELECT register are as follows:

- The UART interrupt is multiplexed with the HEAR 5 interrupt, and interrupt source selection is controlled by the HEAR_INT_SELECT_HEAR_5_UART bit.

- The Filter Engine interrupt is multiplexed with the HEAR 6 interrupt, and interrupt source selection is controlled by the HEAR_INT_SELECT_HEAR_6_FENG bit.
- The signal detection unit interrupt is multiplexed with the HEAR 7 interrupt, and interrupt source selection is controlled by the HEAR_INT_SELECT_HEAR_7_SDU bit.

14.1.1 Control and Configuration Registers

Register Name	Register Description	Address
D_HEAR_INT_SELECT	Interrupt multiplexing control	P:0xF009
D_INT_ENABLE	Interrupt Enable/Mask registers	P:0xF040
D_INT_STATUS	Interrupt ACK_Status registers	P:0xF041
D_INT_PRIORITY	Interrupt priority setting	P:0xF042
D_INT_EBL_0	Enable (no disable) of interrupts 1 to 7 and master interrupt enable	P:0xF050
D_INT_EBL_1	Enable (no disable) of interrupts 8 to 15 (HEAR)	P:0xF051
D_INT_EBL_2	Enable (no disable) of interrupts 16 to 23 (FIFO)	P:0xF052
D_INT_EBL_3	Enable (no disable) of interrupts 24 to 31	P:0xF053
D_INT_ACK_EBL_0	Acknowledge and enable interrupts 1 to 7 and master interrupt enable	P:0xF054
D_INT_ACK_EBL_1	Acknowledge and enable interrupts 8 to 15 (HEAR)	P:0xF055
D_INT_ACK_EBL_2	Acknowledge and enable interrupts 16 to 23 (FIFO)	P:0xF056
D_INT_ACK_EBL_3	Acknowledge and enable interrupts 24 to 31	P:0xF057
D_INT_DIS_0	Disable (no enable) of interrupts 1 to 7 and master interrupt enable	P:0xF058
D_INT_DIS_1	Disable (no enable) of interrupts 8 to 15 (HEAR)	P:0xF059
D_INT_DIS_2	Disable (no enable) of interrupts 16 to 23 (FIFO)	P:0xF05A
D_INT_DIS_3	Disable (no enable) of interrupts 24 to 31	P:0xF05B
D_INT_ACK_0	Acknowledge/status of interrupts 1 to 7 and master interrupt enable	P:0xF05C
D_INT_ACK_1	Acknowledge/status of interrupts 8 to 15 (HEAR)	P:0xF05D
D_INT_ACK_2	Acknowledge/status of interrupts 16 to 23 (FIFO)	P:0xF05E
D_INT_ACK_3	Acknowledge/status of interrupts 24 to 31	P:0xF05F
D_INT_PRIO_0	Priority level of interrupts 7-1	P:0xF060
D_INT_PRIO_1	Priority level of interrupts 15-8 (HEAR)	P:0xF061
D_INT_PRIO_2	Priority level of interrupts 23-16 (FIFO)	P:0xF062
D_INT_PRIO_3	Priority level of interrupts 31-24	P:0xF063

14.1.1.1 D_HEAR_INT_SELECT Settings

Bit Field	Field Name	Description
2	HEAR_5_UART	HEAR 5/UART interrupt selection
1	HEAR_7_SDU	HEAR 7/Signal detection unit interrupt selection
0	HEAR_6_FENG	HEAR 6/Filter Engine interrupt selection

Field Name	Value Symbol	Value Description	Hex Value
HEAR_5_UART	INT_SEL_HEAR_5	Select HEAR 5 interrupt	0x0*
	INT_SEL_UART	Select UART interrupt	0x1
HEAR_7_SDU	INT_SEL_HEAR_7	Select HEAR 7 interrupt	0x0*
	INT_SEL_SDU	Select signal detection unit interrupt	0x1
HEAR_6_FENG	INT_SEL_HEAR_6	Select HEAR 6 interrupt	0x0*
	INT_SEL_FENG	Select Filter Engine interrupt	0x1

14.1.1.2 D_INT_ENABLE Settings

Bit Field	Field Name	Description
31	CM3_3	ARM Cortex-M3 core interrupt 3 enable/disable
30	CM3_2	ARM Cortex-M3 core interrupt 2 enable/disable
29	CM3_1	ARM Cortex-M3 core interrupt 1 enable/disable
28	CM3_0	ARM Cortex-M3 core interrupt 0 enable/disable
27	C_I2C	ARM Cortex-M3 core I^2C interrupt enable/disable
26	C_PCM	ARM Cortex-M3 core PCM interrupt enable/disable
25	C_SPI	ARM Cortex-M3 core SPI interrupt enable/disable
24	CLK	CLK interrupt enable/disable
23	FIFO_7	FIFO interrupt 7 enable/disable
22	FIFO_6	FIFO interrupt 6 enable/disable
21	FIFO_5	FIFO interrupt 5 enable/disable
20	FIFO_4	FIFO interrupt 4 enable/disable
19	FIFO_3	FIFO interrupt 3 enable/disable
18	FIFO_2	FIFO interrupt 2 enable/disable
17	FIFO_1	FIFO interrupt 1 enable/disable
16	FIFO_0	FIFO interrupt 0 enable/disable
15	HEAR_7_SDU	HEAR 7 / SDU interrupt enable/disable
14	HEAR_6_FENG	HEAR 6 / Filter Engine interrupt enable/disable
13	HEAR_5_UART	HEAR 5 / UART interrupt enable/disable
12	HEAR_4	HEAR 4 interrupt enable/disable
11	HEAR_3	HEAR 3 interrupt enable/disable
10	HEAR_2	HEAR 2 interrupt enable/disable
9	HEAR_1	HEAR 1 interrupt enable/disable
8	HEAR_0	HEAR 0 interrupt enable/disable
7	GPIO	GPIO interrupt enable/disable
6	PCM	PCM interrupt enable/disable
5	I2C	I^2C interrupt enable/disable
4	SPI	SPI interrupt enable/disable
3	TIMER_2	Timer 2 interrupt enable/disable

Bit Field	Field Name	Description
2	TIMER_1	Timer 1 interrupt enable/disable
1	WATCHDOG	Watchdog timer interrupt enable/disable
0	MASTER	Master enable/disable for all CFX interrupts

Field Name	Value Symbol	Value Description	Hex Value
CM3_3	INT_EBL_CM3_3	Enable the CM3_3 interrupt	0x1
CM3_2	INT_EBL_CM3_2	Enable the CM3_2 interrupt	0x1
CM3_1	INT_EBL_CM3_1	Enable the CM3_1 interrupt	0x1
CM3_0	INT_EBL_CM3_0	Enable the CM3_0 interrupt	0x1
C_I2C	INT_EBL_C_I2C	Enable the C_I2C interrupt	0x1
C_PCM	INT_EBL_C_PCM	Enable the C_PCM interrupt	0x1
C_SPI	INT_EBL_C_SPI	Enable the C_SPI interrupt	0x1
CLK	INT_EBL_CLK	Enable the CLK interrupt	0x1
FIFO_7	INT_EBL_FIFO_7	Enable the FIFO_7 interrupt	0x1
FIFO_6	INT_EBL_FIFO_6	Enable the FIFO_6 interrupt	0x1
FIFO_5	INT_EBL_FIFO_5	Enable the FIFO_5 interrupt	0x1
FIFO_4	INT_EBL_FIFO_4	Enable the FIFO_4 interrupt	0x1
FIFO_3	INT_EBL_FIFO_3	Enable the FIFO_3 interrupt	0x1
FIFO_2	INT_EBL_FIFO_2	Enable the FIFO_2 interrupt	0x1
FIFO_1	INT_EBL_FIFO_1	Enable the FIFO_1 interrupt	0x1
FIFO_0	INT_EBL_FIFO_0	Enable the FIFO_0 interrupt	0x1
HEAR_7_SDU	INT_EBL_HEAR_7_SDU	Enable the HEAR_7_SDU interrupt	0x1
HEAR_6_FENG	INT_EBL_HEAR_6_FENG	Enable the HEAR_6_FENG interrupt	0x1
HEAR_5_UART	INT_EBL_HEAR_5_UART	Enable the HEAR_5_UART interrupt	0x1
HEAR_4	INT_EBL_HEAR_4	Enable the HEAR_4 interrupt	0x1
HEAR_3	INT_EBL_HEAR_3	Enable the HEAR_3 interrupt	0x1
HEAR_2	INT_EBL_HEAR_2	Enable the HEAR_2 interrupt	0x1
HEAR_1	INT_EBL_HEAR_1	Enable the HEAR_1 interrupt	0x1
HEAR_0	INT_EBL_HEAR_0	Enable the HEAR_0 interrupt	0x1
GPIO	INT_EBL_GPIO	Enable the GPIO interrupt	0x1
PCM	INT_EBL_PCM	Enable the PCM interrupt	0x1
I2C	INT_EBL_I2C	Enable the I2C interrupt	0x1
SPI	INT_EBL_SPI	Enable the SPI interrupt	0x1
TIMER_2	INT_EBL_TIMER_2	Enable the TIMER_2 interrupt	0x1
TIMER_1	INT_EBL_TIMER_1	Enable the TIMER_1 interrupt	0x1
WATCHDOG	INT_EBL_WATCHDOG	Enable the WATCHDOG interrupt	0x1
MASTER	INT_EBL_MASTER	Enable interrupts for the CFX	0x1

14.1.1.3 D_INT_STATUS Settings

Bit Field	Field Name	Description
31	CM3_3	ARM Cortex-M3 core interrupt 3
30	CM3_2	ARM Cortex-M3 core interrupt 2
29	CM3_1	ARM Cortex-M3 core interrupt 1
28	CM3_0	ARM Cortex-M3 core interrupt 0
27	C_I2C	ARM Cortex-M3 core I^2C interrupt
26	C_PCM	ARM Cortex-M3 core PCM interrupt
25	C_SPI	ARM Cortex-M3 core SPI interrupt
24	CLK	CLK interrupt
23	FIFO_7	FIFO interrupt 7
22	FIFO_6	FIFO interrupt 6
21	FIFO_5	FIFO interrupt 5
20	FIFO_4	FIFO interrupt 4
19	FIFO_3	FIFO interrupt 3
18	FIFO_2	FIFO interrupt 2
17	FIFO_1	FIFO interrupt 1
16	FIFO_0	FIFO interrupt 0
15	HEAR_7_SDU	HEAR 7 / SDU interrupt
14	HEAR_6_FENG	HEAR 6 / Filter Engine interrupt
13	HEAR_5_UART	HEAR 5 / UART interrupt
12	HEAR_4	HEAR 4 interrupt
11	HEAR_3	HEAR 3 interrupt
10	HEAR_2	HEAR 2 interrupt
9	HEAR_1	HEAR 1 interrupt
8	HEAR_0	HEAR 0 interrupt
7	GPIO	GPIO interrupt
6	PCM	PCM interrupt
5	I2C	I^2C interrupt
4	SPI	SPI interrupt
3	TIMER_2	Timer 2 interrupt
2	TIMER_1	Timer 1 interrupt
1	WATCHDOG	Watchdog timer interrupt

Field Name	Value Symbol	Value Description	Hex Value
CM3_3	INT_ACK_CM3_3	Acknowledge the CM3_3 interrupt	0x1
CM3_2	INT_ACK_CM3_2	Acknowledge the CM3_2 interrupt	0x1
CM3_1	INT_ACK_CM3_1	Acknowledge the CM3_1 interrupt	0x1
CM3_0	INT_ACK_CM3_0	Acknowledge the CM3_0 interrupt	0x1

Field Name	Value Symbol	Value Description	Hex Value
C_I2C	INT_ACK_C_I2C	Acknowledge the C_I2C interrupt	0x1
C_PCM	INT_ACK_C_PCM	Acknowledge the C_PCM interrupt	0x1
C_SPI	INT_ACK_C_SPI	Acknowledge the C_SPI interrupt	0x1
CLK	INT_ACK_CLK	Acknowledge the CLK interrupt	0x1
FIFO_7	INT_ACK_FIFO_7	Acknowledge the FIFO_7 interrupt	0x1
FIFO_6	INT_ACK_FIFO_6	Acknowledge the FIFO_6 interrupt	0x1
FIFO_5	INT_ACK_FIFO_5	Acknowledge the FIFO_5 interrupt	0x1
FIFO_4	INT_ACK_FIFO_4	Acknowledge the FIFO_4 interrupt	0x1
FIFO_3	INT_ACK_FIFO_3	Acknowledge the FIFO_3 interrupt	0x1
FIFO_2	INT_ACK_FIFO_2	Acknowledge the FIFO_2 interrupt	0x1
FIFO_1	INT_ACK_FIFO_1	Acknowledge the FIFO_1 interrupt	0x1
FIFO_0	INT_ACK_FIFO_0	Acknowledge the FIFO_0 interrupt	0x1
HEAR_7_SDU	INT_ACK_HEAR_7_SDU	Acknowledge the HEAR_7_SDU interrupt	0x1
HEAR_6_FENG	INT_ACK_HEAR_6_FENG	Acknowledge the HEAR_6_FENG interrupt	0x1
HEAR_5_UART	INT_ACK_HEAR_5_UART	Acknowledge the HEAR_5_UART interrupt	0x1
HEAR_4	INT_ACK_HEAR_4	Acknowledge the HEAR_4 interrupt	0x1
HEAR_3	INT_ACK_HEAR_3	Acknowledge the HEAR_3 interrupt	0x1
HEAR_2	INT_ACK_HEAR_2	Acknowledge the HEAR_2 interrupt	0x1
HEAR_1	INT_ACK_HEAR_1	Acknowledge the HEAR_1 interrupt	0x1
HEAR_0	INT_ACK_HEAR_0	Acknowledge the HEAR_0 interrupt	0x1
GPIO	INT_ACK_GPIO	Acknowledge the GPIO interrupt	0x1
PCM	INT_ACK_PCM	Acknowledge the PCM interrupt	0x1
I2C	INT_ACK_I2C	Acknowledge the I2C interrupt	0x1
SPI	INT_ACK_SPI	Acknowledge the SPI interrupt	0x1
TIMER_2	INT_ACK_TIMER_2	Acknowledge the TIMER_2 interrupt	0x1
TIMER_1	INT_ACK_TIMER_1	Acknowledge the TIMER_1 interrupt	0x1
WATCHDOG	INT_ACK_WATCHDOG	Acknowledge the WATCHDOG interrupt	0x1
	INT_ACK_ALL_INTS	Acknowledge all interrupts	0x7FFFFFFF

14.1.1.4 D_INT_PRIORITY Settings

Bit Field	Field Name	Description
31	CM3_3	ARM Cortex-M3 core interrupt 3 priority
30	CM3_2	ARM Cortex-M3 core interrupt 2 priority
29	CM3_1	ARM Cortex-M3 core interrupt 1 priority
28	CM3_0	ARM Cortex-M3 core interrupt 0 priority
27	C_I2C	ARM Cortex-M3 core I^2C interrupt priority
26	C_PCM	ARM Cortex-M3 core PCM interrupt priority
25	C_SPI	ARM Cortex-M3 core SPI interrupt priority

Bit Field	Field Name	Description
24	CLK	CLK interrupt priority
23	FIFO_7	FIFO interrupt 7 priority
22	FIFO_6	FIFO interrupt 6 priority
21	FIFO_5	FIFO interrupt 5 priority
20	FIFO_4	FIFO interrupt 4 priority
19	FIFO_3	FIFO interrupt 3 priority
18	FIFO_2	FIFO interrupt 2 priority
17	FIFO_1	FIFO interrupt 1 priority
16	FIFO_0	FIFO interrupt 0 priority
15	HEAR_7_SDU	HEAR 7 / SDU interrupt priority
14	HEAR_6_FENG	HEAR 6 / Filter Engine interrupt priority
13	HEAR_5_UART	HEAR 5 / UART interrupt priority
12	HEAR_4	HEAR 4 interrupt priority
11	HEAR_3	HEAR 3 interrupt priority
10	HEAR_2	HEAR 2 interrupt priority
9	HEAR_1	HEAR 1 interrupt priority
8	HEAR_0	HEAR 0 interrupt priority
7	GPIO	GPIO interrupt priority
6	PCM	PCM interrupt priority
5	I2C	I^2C interrupt priority
4	SPI	SPI interrupt priority
3	TIMER_2	Timer 2 interrupt priority
2	TIMER_1	Timer 1 interrupt priority
1	WATCHDOG	Watchdog timer interrupt priority

Field Name	Value Symbol	Value Description	Hex Value
CM3_3	INT_NORMALPRIO_CM3_3	CM3_3 has normal priority	0x0*
	INT_HIGHPRIO_CM3_3	CM3_3 has high priority	0x1
CM3_2	INT_NORMALPRIO_CM3_2	CM3_2 has normal priority	0x0*
	INT_HIGHPRIO_CM3_2	CM3_2 has high priority	0x1
CM3_1	INT_NORMALPRIO_CM3_1	CM3_1 has normal priority	0x0*
	INT_HIGHPRIO_CM3_1	CM3_1 has high priority	0x1
CM3_0	INT_NORMALPRIO_CM3_0	CM3_0 has normal priority	0x0*
	INT_HIGHPRIO_CM3_0	CM3_0 has high priority	0x1
C_I2C	INT_NORMALPRIO_C_I2C	C_I2C has normal priority	0x0*
	INT_HIGHPRIO_C_I2C	C_I2C has high priority	0x1
C_PCM	INT_NORMALPRIO_C_PCM	C_PCM has normal priority	0x0*
	INT_HIGHPRIO_C_PCM	C_PCM has high priority	0x1

Field Name	Value Symbol	Value Description	Hex Value
C_SPI	INT_NORMALPRIO_C_SPI	C_SPI has normal priority	0x0*
	INT_HIGHPRIO_C_SPI	C_SPI has high priority	0x1
CLK	INT_NORMALPRIO_CLK	CLK has normal priority	0x0*
	INT_HIGHPRIO_CLK	CLK has high priority	0x1
FIFO_7	INT_NORMALPRIO_FIFO_7	FIFO_7 has normal priority	0x0*
	INT_HIGHPRIO_FIFO_7	FIFO_7 has high priority	0x1
FIFO_6	INT_NORMALPRIO_FIFO_6	FIFO_6 has normal priority	0x0*
	INT_HIGHPRIO_FIFO_6	FIFO_6 has high priority	0x1
FIFO_5	INT_NORMALPRIO_FIFO_5	FIFO_5 has normal priority	0x0*
	INT_HIGHPRIO_FIFO_5	FIFO_5 has high priority	0x1
FIFO_4	INT_NORMALPRIO_FIFO_4	FIFO_4 has normal priority	0x0*
	INT_HIGHPRIO_FIFO_4	FIFO_4 has high priority	0x1
FIFO_3	INT_NORMALPRIO_FIFO_3	FIFO_3 has normal priority	0x0*
	INT_HIGHPRIO_FIFO_3	FIFO_3 has high priority	0x1
FIFO_2	INT_NORMALPRIO_FIFO_2	FIFO_2 has normal priority	0x0*
	INT_HIGHPRIO_FIFO_2	FIFO_2 has high priority	0x1
FIFO_1	INT_NORMALPRIO_FIFO_1	FIFO_1 has normal priority	0x0*
	INT_HIGHPRIO_FIFO_1	FIFO_1 has high priority	0x1
FIFO_0	INT_NORMALPRIO_FIFO_0	FIFO_0 has normal priority	0x0*
	INT_HIGHPRIO_FIFO_0	FIFO_0 has high priority	0x1
HEAR_7_SDU	INT_NORMALPRIO_HEAR_7_SDU	HEAR_7_SDU has normal priority	0x0*
	INT_HIGHPRIO_HEAR_7_SDU	HEAR_7_SDU has high priority	0x1
HEAR_6_FENG	INT_NORMALPRIO_HEAR_6_FENG	HEAR_6_FENG has normal priority	0x0*
	INT_HIGHPRIO_HEAR_6_FENG	HEAR_6_FENG has high priority	0x1
HEAR_5_UART	INT_NORMALPRIO_HEAR_5_UART	HEAR_5_UART has normal priority	0x0*
	INT_HIGHPRIO_HEAR_5_UART	HEAR_5_UART has high priority	0x1
HEAR_4	INT_NORMALPRIO_HEAR_4	HEAR_4 has normal priority	0x0*
	INT_HIGHPRIO_HEAR_4	HEAR_4 has high priority	0x1
HEAR_3	INT_NORMALPRIO_HEAR_3	HEAR_3 has normal priority	0x0*
	INT_HIGHPRIO_HEAR_3	HEAR_3 has high priority	0x1
HEAR_2	INT_NORMALPRIO_HEAR_2	HEAR_2 has normal priority	0x0*
	INT_HIGHPRIO_HEAR_2	HEAR_2 has high priority	0x1
HEAR_1	INT_NORMALPRIO_HEAR_1	HEAR_1 has normal priority	0x0*
	INT_HIGHPRIO_HEAR_1	HEAR_1 has high priority	0x1
HEAR_0	INT_NORMALPRIO_HEAR_0	HEAR_0 has normal priority	0x0*
	INT_HIGHPRIO_HEAR_0	HEAR_0 has high priority	0x1
GPIO	INT_NORMALPRIO_GPIO	GPIO has normal priority	0x0*
	INT_HIGHPRIO_GPIO	GPIO has high priority	0x1

Field Name	Value Symbol	Value Description	Hex Value
PCM	INT_NORMALPRIO_PCM	PCM has normal priority	0x0*
	INT_HIGHPRIO_PCM	PCM has high priority	0x1
I2C	INT_NORMALPRIO_I2C	I2C has normal priority	0x0*
	INT_HIGHPRIO_I2C	I2C has high priority	0x1
SPI	INT_NORMALPRIO_SPI	SPI has normal priority	0x0*
	INT_HIGHPRIO_SPI	SPI has high priority	0x1
TIMER_2	INT_NORMALPRIO_TIMER_2	TIMER_2 has normal priority	0x0*
	INT_HIGHPRIO_TIMER_2	TIMER_2 has high priority	0x1
TIMER_1	INT_NORMALPRIO_TIMER_1	TIMER_1 has normal priority	0x0*
	INT_HIGHPRIO_TIMER_1	TIMER_1 has high priority	0x1
WATCHDOG	INT_NORMALPRIO_WATCHDOG	WATCHDOG has normal priority	0x0*
	INT_HIGHPRIO_WATCHDOG	WATCHDOG has high priority	0x1

14.2 General Purpose Timers

The Ezairo 7100 system contains two general-purpose timers for use in a user's application. Each of these timers operates independently of the other and can be configured to trigger at a wide range of frequencies.

By default the general-purpose timers operate at a division of 32 from the slow clock. To increase this prescaling division for a timer, set the TIMER_CFG_PRESCALE bit field in the appropriate D_TIMER_x_CFG register. Prescalers increase in multiples of two allowing the selection of a clock that is divided by as much as 4096 for each timer. As a result, the range of the timers can be increased; however the granularity of the steps at which the timer can be configured to trigger at increases in parallel.

After prescaling the slow clock, each timer can be configured to trigger after 1 to 4096 cycles of the prescaled clock by setting the TIMER_CFG_DELAY bit field in the appropriate D_TIMER_x_CFG register. The resulting timer delay is equal to:

$$\text{DELAY} = \frac{2^{(5 + \text{TIMER_CFG_PRESCALE})} \times (\text{TIMER_CFG_DELAY} + 1)}{f_{\text{SLOWCLK}}}$$

A summary of possible timer delays for 1.28 MHz is in Table 37, and timer delays for 1.92 MHz are in Table 38. For other slow clock frequencies, the timer delays are scaled appropriately.

Table 37. General Purpose Timer Configuration (1.28 MHz)

Prescaler Divisor (Bit Field setting)	Timer Granularity	Counter Range	Minimum Timeout	Maximum Timeout
32 (0)	0.025 ms	1 to 4096	0.025 ms	102.4 ms
64 (1)	0.05 ms	1 to 4096	0.05 ms	204.8 ms
128 (2)	0.1 ms	1 to 4096	0.1 ms	409.6 ms
256 (3)	0.2 ms	1 to 4096	0.2 ms	819.2 ms
512 (4)	0.4 ms	1 to 4096	0.4 ms	1.6384 s

Table 37. General Purpose Timer Configuration (1.28 MHz)

Prescaler Divisor (Bit Field setting)	Timer Granularity	Counter Range	Minimum Timeout	Maximum Timeout
1024 (5)	0.8 ms	1 to 4096	0.8 ms	3.2768 s
2056 (6)	1.6 ms	1 to 4096	1.6 ms	6.5536 s
4096 (7)	3.2 ms	1 to 4096	3.2 ms	13.1072 s

Table 38. General Purpose Timer Configuration (1.92 MHz)

Prescaler Divisor (Bit Field setting)	Timer Granularity	Counter Range	Minimum Timeout	Maximum Timeout
32 (0)	0.017 ms	1 to 4096	0.017 ms	68.27 ms
64 (1)	0.033 ms	1 to 4096	0.033 ms	136.53 ms
128 (2)	0.067 ms	1 to 4096	0.067 ms	273.07 ms
256 (3)	0.133 ms	1 to 4096	0.133 ms	546.13 ms
512 (4)	0.267 ms	1 to 4096	0.267 ms	1.092 s
1024 (5)	0.533 ms	1 to 4096	0.533 ms	2.185 s
2056 (6)	1.067 ms	1 to 4096	1.067 ms	4.369 s
4096 (7)	2.133 ms	1 to 4096	2.133 ms	8.738 s

The general-purpose timers can be configured to either run continuously (free running mode), or to trigger a single time (one-shot mode). To select the timer mode, use the `TIMER_CFG_MODE` bit in the `D_TIMER_x_CFG` registers.

Start or stop each timer by writing to the `TIMER_CFG_START_STOP` bit in the `D_TIMER_x_CFG` registers. This method of starting/stopping the timer is typically used when configuring the timer. Alternatively, the timers can be started (but not stopped) by setting the appropriate `SYSTEM_CTRL_TIMER_x_START` bit in the `D_SYSTEM_CTRL` register. As with all bits in the `D_SYSTEM_CTRL` register, setting a `SYSTEM_CTRL_TIMER_x_START` bit does not require knowledge of the configuration of the other register bits (as bit settings in the `D_SYSTEM_CTRL` register only change when explicitly set and not when cleared). Reading the `SYSTEM_CTRL_TIMER_x_START` bits indicates whether the timers are currently running or are stopped.

For information on slow clock frequency configuration, see Section 18.2.2, “Slow Clock (SLOWCLK)” on page 446.

14.2.1 Control and Configuration Registers

Register Name	Register Description	Address
`D_TIMER_1_CFG`	General-purpose timer configuration	P:0xF002
`D_TIMER_2_CFG`	General-purpose timer configuration	P:0xF003

For information about other general-purpose timer registers, see Section 4.1.3.1, “D_SYSTEM_CTRL Settings” on page 35.

14.2.1.1 D_TIMER_*_CFG Settings

The following bit fields and field names apply equally to registers `D_TIMER_1_CFG` and `D_TIMER_2_CFG`.

Bit Field	Field Name	Description
16	START_STOP	State of the timer
15	MODE	Mode of operation of the timer
14:12	PRESCALE	Prescale value of the timer
11:0	DELAY	Expiration delay of the timer

Field Name	Value Symbol	Value Description	Hex Value
START_STOP	TIMER_STOP	Timer stop	0x0*
	TIMER_START	Timer start	0x1
MODE	TIMER_ONESHOT	Timer oneshot	0x0*
	TIMER_FREERUN	Timer freerun	0x1
PRESCALE	TIMER_PRESCALE_32	Timer prescale 32	0x0*
	TIMER_PRESCALE_64	Timer prescale 64	0x1
	TIMER_PRESCALE_128	Timer prescale 128	0x2
	TIMER_PRESCALE_256	Timer prescale 256	0x3
	TIMER_PRESCALE_512	Timer prescale 512	0x4
	TIMER_PRESCALE_1024	Timer prescale 1024	0x5
	TIMER_PRESCALE_2048	Timer prescale 2048	0x6
	TIMER_PRESCALE_4096	Timer prescale 4096	0x7

14.3 WATCHDOG TIMER

The watchdog timer is a simple safety system that, in most cases, resets a system that has malfunctioned. It is a countdown timer that must be periodically acknowledged by writing SYSTEM_WATCHDOG_REFRESH to the D_SYSTEM_CTRL register before it reaches zero. Setting this bit does not affect the other bits in this register as these bits only change when they are explicitly set. The system assumes that the application's failure to acknowledge this countdown timer before it reaches zero indicates that the system is malfunctioning and must be reset. The countdown timer value for the watchdog timer is not visible to the CFX DSP. The watchdog timer cannot be disabled.

The watchdog timer runs on a prescaled clock that has been derived from the slow clock. This clock is used to decrement the value in a 13-bit counter. The delay before this timer will time out can be configured using the WATCHDOG_CFG_TIMEOUT field of the D_WATCHDOG_CFG register. However, since this field is defined in terms of SLOWCLK, the delay is different for different slow clocks (although the configuration defines for this timer are specified in terms of a 1.28 MHz SLOWCLK by historical convention). Table 39 shows the delays associated with SLOWCLK settings of 1.28 MHz and 1.92 MHz. For other slow clock frequencies, the watchdog timeout intervals are scaled appropriately.

Table 39. Watchdog Timeout Intervals for Different SLOWCLK Settings

Watchdog Scaler Setting	Approximate Timeout: SLOWCLK=1.28 MHz (ms)	Approximate Timeout: SLOWCLK=1.92 MHz (ms)
WATCHDOG_TIMEOUT_1M6	1.6	1.1
WATCHDOG_TIMEOUT_3M2	3.2	2.1

Table 39. Watchdog Timeout Intervals for Different SLOWCLK Settings (Continued)

Watchdog Scaler Setting	Approximate Timeout: SLOWCLK=1.28 MHz (ms)	Approximate Timeout: SLOWCLK=1.92 MHz (ms)
WATCHDOG_TIMEOUT_6M4	6.4	4.3
WATCHDOG_TIMEOUT_12M8	12.8	8.5
WATCHDOG_TIMEOUT_25M6	25.6	17.1
WATCHDOG_TIMEOUT_51M2	51.2	34.1
WATCHDOG_TIMEOUT_102M4	102.4	68.3
WATCHDOG_TIMEOUT_204M8	204.8	136.5
WATCHDOG_TIMEOUT_409M6	409.6	273.1
WATCHDOG_TIMEOUT_819M2	819.2	546.1
WATCHDOG_TIMEOUT_1638M4	1638.4	1092.3
WATCHDOG_TIMEOUT_3276M8	3276.8	2184.5

For information on slow clock frequency configuration, see Section 18.2.2, “Slow Clock (SLOWCLK)” on page 446.

14.3.1 Watchdog Interrupt

The watchdog has an associated interrupt that triggers whenever the watchdog times out. If the watchdog timer has not been reset and a second watchdog timeout occurs, the entire system resets. See Section 14.1, “Interrupt Controller” on page 218 for information regarding interrupt configuration and handling.

14.3.2 Control and Configuration Registers

Register Name	Register Description	Address
D_WATCHDOG_CFG	Watchdog timeout configuration	P:0xF001

For information about other general-purpose timer registers, see Section 4.1.3.1, “D_SYSTEM_CTRL Settings” on page 35.

14.3.2.1 D_WATCHDOG_CFG Settings

Bit Field	Field Name	Description
3:0	TIMEOUT	Watchdog timeout period. Values 0xC to 0xF result in the same timeout period as the value 0xB.

Field Name	Value Symbol	Value Description	Hex Value
TIMEOUT	WATCHDOG_TIMEOUT_1M6	1.6 ms	0x0
	WATCHDOG_TIMEOUT_3M2	3.2 ms	0x1
	WATCHDOG_TIMEOUT_6M4	6.4 ms	0x2
	WATCHDOG_TIMEOUT_12M8	12.8 ms	0x3
	WATCHDOG_TIMEOUT_25M6	25.6 ms	0x4
	WATCHDOG_TIMEOUT_51M2	51.2 ms	0x5
	WATCHDOG_TIMEOUT_102M4	102.4 ms	0x6
	WATCHDOG_TIMEOUT_204M8	204.8 ms	0x7
	WATCHDOG_TIMEOUT_409M6	409.6 ms	0x8
	WATCHDOG_TIMEOUT_819M2	819.2 ms	0x9
	WATCHDOG_TIMEOUT_1638M4	1.64 s	0xA
	WATCHDOG_TIMEOUT_3276M8	3.28 s	0xB*

14.4 Cyclic Redundancy Code (CRC) Generator

The Ezairo 7100 system uses a standard cyclic redundancy code (CRC) algorithm to ensure data integrity for the file system and all debug port communications. CRC was selected to ensure data integrity as it will catch most data errors; in particular, it is guaranteed to catch all single-bit errors, two-bit errors, burst errors and any error containing an odd number of bits. Because calculating a CRC is a common but moderately expensive software calculation, a hardware CRC generator is included as a peripheral accessible by the CFX DSP.

The selected implementation of the CRC algorithm used for Ezairo 7100 is the standard CRC-CCITT configuration. The parameters associated with this algorithm implementation are available in Table 40.

Table 40. CRC-CCITT Algorithm Parameters

CRC Parameter	Parameter Value
Order	16
Polynomial	$x^{16} + x^{12} + x^5 + 1$
Polynomial (hex)	0x1021
Initial Value (hex)	0xFFFF
Final XOR Value (hex)	0x0000
Notes	No data manipulation is required (i.e. no data byte reversal, reversal of the final result or other finalization)

To use the CRC generator:

1. Write an initial value of 0xFFFF to the D_CRC_VALUE register.
2. Write any data that needs to be included in the final CRC read from the generator to the CRC input registers.
3. At any time, you can read the D_CRC_VALUE register to obtain the CRC for the data that has been added using the CRC input since the last time the register was initialized.

As part of the ARM Cortex-M3 processor's subsystem, a second CRC generator is provided that can be used to support both CRC-CCITT and CRC32 CRCs. For more information about this generator, see Section 15.6, "Cyclic

Redundancy Check (CRC) Generator” on page 346.

14.4.1 Control and Configuration Registers

Register Name	Register Description	Address
D_CRC_VALUE	CRC value (CRC16-CCITT)	P:0xF022
D_CRC_ADD_1	CRC 1-bit data input	P:0xF023
D_CRC_ADD_8	CRC 8-bit data input	P:0xF024
D_CRC_ADD_16	CRC 16-bit data input	P:0xF025
D_CRC_ADD_24	CRC 24-bit data input	P:0xF026
D_CRC_ADD_32	CRC 32-bit data input	P:0xF027

14.4.1.1 D_CRC_VALUE Settings

Bit Field	Field Name	Description
15:0	CRC_VALUE	Value of CRC-CCITT for supplied data

14.4.1.2 D_CRC_ADD_1 Settings

Bit Field	Field Name	Description
0	CRC_ADD_1	Add 1-bit data to the CRC

14.4.1.3 D_CRC_ADD_8 Settings

Bit Field	Field Name	Description
7:0	CRC_ADD_8	Add 8-bit data to the CRC

14.4.1.4 D_CRC_ADD_16 Settings

Bit Field	Field Name	Description
15:0	CRC_ADD_16	Add 16-bit data to the CRC

14.4.1.5 D_CRC_ADD_24 Settings

Bit Field	Field Name	Description
23:0	CRC_ADD_24	Add 24-bit data to the CRC

14.4.1.6 D_CRC_ADD_32 Settings

Bit Field	Field Name	Description
31:0	CRC_ADD_32	Add 32-bit data to the CRC

14.5 Access Mode Security Blocks

The Ezairo 7100 system includes a few components that can be used to secure intellectual property stored on the chip and attached EEPROM. To activate these protective measures, enable access mode protection on the debug port as described in Section 13.2.1, “Debug Port Configuration” on page 191. A description of the restrictions placed on debug port commands when restricted access is enabled can be found in the debug port reference materials provided by the *Communication Protocols Manual for Ezairo 7100*. Generally, when the Ezairo 7100 system is in restricted mode, the value of memories and registers on the CFX DSP and ARM Cortex-M3 processor are not accessible for read or write from their respective debug port interfaces.

A list of protected memory resources that are cleared by both the program ROM and the CFX debug port’s JumpROM wipe operation prior to switching the debug access mode to unrestricted mode is provided in Table 41. The addresses for these memories are defined with respect to their CFX addresses. Where relevant, ARM Cortex-M3 processor addresses are provided in brackets. For more information about memories provided by the Ezairo 7100 system, see Chapter 7, “Memory” on page 72.

Table 41. Protected Memory Sections

Memory Space	Start Address	End Address	Protected Length (in Words)	Data Cleared Indicator Value
Program Memory	0x8000	0xDFFF	0x5000	0x00000000
Microcode Memory	0xE000	0xEFFF	0x1000	0x00000000
X Memory	0x20	0x2F9B	0x2F7C	0x000000
Y Memory	0x20	0x07FF	0x07E0	0x000000
ARM Cortex-M3 Processor Program Memory	0x5000 (0x00000000)	0x7FFF (0x0000BFFF)	0x3000	0x00000000
ARM Cortex-M3 Processor Data Memory	0x4000 (0x20000000)	0x4FFF (0x20003FFF)	0x1000	0x00000000

Sensitive blocks of program code or data to be loaded to any part of the P memory space (Section 7.3.2, “P Memory Space” on page 80) can be stored as encrypted data in non-volatile memory to protect it from direct attacks. The Ezairo 7100 chip can decrypt this data using non-externally accessible keys at startup, but to obtain access to potentially sensitive parts of the system, either the decoded application must unlock the chip or you must use a “Jump ROM” command sequence, as described in the *Communication Protocols Manual for Ezairo 7100*, to unlock the chip. In the case that the system is using the “Jump ROM” command sequence to unlock the chip, the system is rendered incapable of further deciphering any information that is stored in protected areas of EEPROM, and all data stored in protected areas of memory is cleared.

CAUTION: Although the debug ports can be used to protect data, none of the other interfaces explicitly have any access restrictions placed on them. To guarantee IP protection, ensure that the application included on the chip does not do something to make the system insecure. Examples of this are providing support for unlocking the chip through another interface or providing access to memory through another interface—both of which could allow a user to read back the contents of protected memory areas.

14.6 Signal Detection Unit

The signal detection unit is used to monitor analog input signals (with the goal of detecting changes to the signals), without incurring the cost (in terms of power consumption or resource usage) of having a full input channel active and monitoring a full speed signal.

The signal detection unit consists of the following components:

- An input selection block that is used to select which analog input to monitor
- A flexible preamplifier, similar to that used for the regular audio input channels
- A bypass for the preamplifier
- A 12-bit ADC
- A register bank, D_SDU_OUT_*, containing a circular buffer of eight registers

The signal detection unit reads input data from the selected analog input to an input register bank. This data is optionally passed through the preamplifier block. The analog data is then sampled by the 12-bit ADC, and each sample is placed into the register bank circular buffer. A block diagram, showing this data flow and interconnections with the rest of the Ezairo 7100 system, is provided in Figure 26.

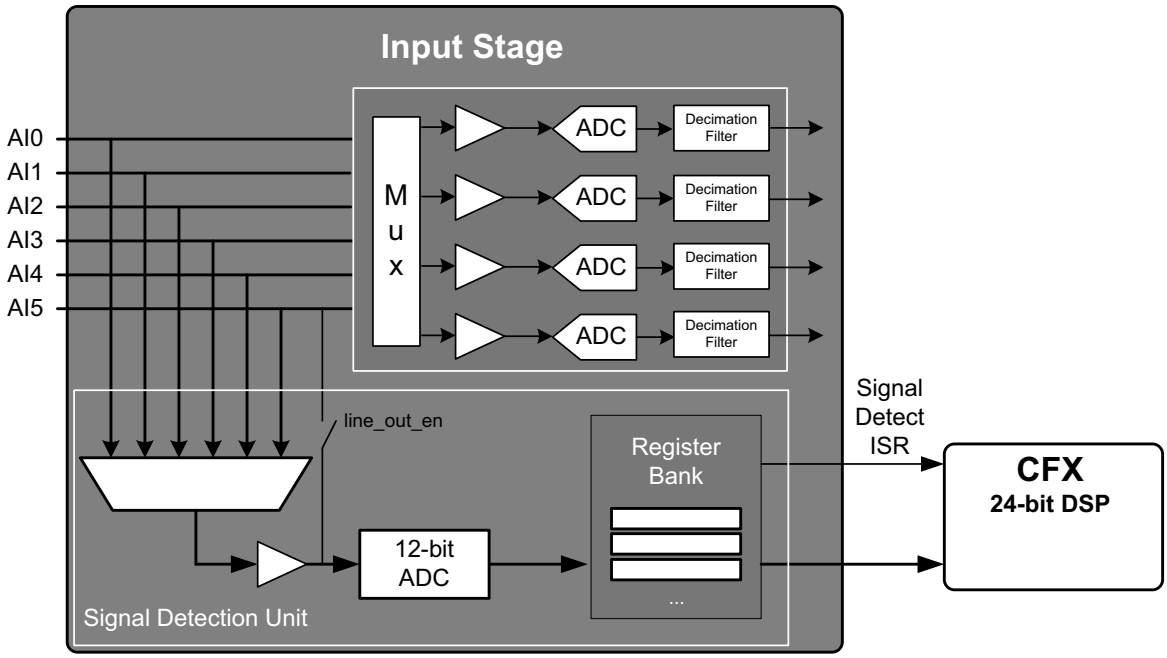


Figure 26. Signal Detection Unit Block Diagram

The ADC for the SDU can be enabled or disabled using the SDU_CFG_ADC_ENABLE bit from the D_SDU_CFG register. When the SDU's ADC is disabled:

- The ADC is not powered
- The internal state machine for the SDU is reset, so the next data register to be updated is D_SDU_OUT_0

While enabled, data is read from the ADC, selected by the SDU_CFG_IN_SEL bit from the D_SDU_CFG register, and is written to the register bank, D_SDU_OUT_*. These registers are written sequentially from index 0 to 7 (wrapping back to 0). The SDU interrupt on the CFX is generated when the data in register D_SDU_OUT_7 is updated. The ADC sampling rate for the SDU is controlled using the SDU_CFG_SFCR bit field from the D_SDU_CFG register. This bit field allows a period between 20 and 1280 SLOWCLK cycles to be specified.

NOTE: For optimal ADC performance, set the SDU_CFG_HIGH_FREQ bit from the D_SDU_CFG register when SLOWCLK exceeds 1.92 MHz.

The preamplifier for the SDU is a replica of the preamplifier implemented for the input channels, which is described in Section 10.3, “Preamplifiers” on page 142. To select the gain for the preamplifier, use the SDU_CFG_PG_GAIN bit field from the D_SDU_CFG register. If no preamplification is required, the preamplifier can be bypassed; in this configuration the preamplifier is not powered.

Similar to the preamplifiers from the input stage, the settling time for the preamplifier depends only on the high pass filter, which is defined by the internal resistor of 500 kΩ and the external capacitor. The programmable gain preamplifier includes a low-pass filtering function with a cut-off frequency around 50 kHz (similar to the regular audio channel in high pass-band configuration). The out-of-band signal attenuation is adequate for the signal detection function when taking into account the ADC averaging function and the input passive GSM filters.

IMPORTANT: The SDU must not select an input that is already connected to another regular ADC channel. Connecting an input channel and the SDU to the same analog input corrupts the samples received by both the regular ADC and the SDU ADC.

The current supplied to the SDU A/D converter is configured by setting the SDU_CFG_ADC_CURRENT bit field in the D_SDU_CFG control register. Similarly, the current supplied to the SDU preamplifier is configured using the SDU_CFG_P_CURRENT bit field from the D_SDU_CFG register. We recommend that currents above the nominal settings only be used for sampling frequencies that exceed 2 kHz. At frequencies above 2 kHz, these currents can be increased to apply a higher current to prevent any reduction in fidelity.

NOTE: The lineout from the SDU is routed to analog input 5. This configuration, which can be selected using the SDU_CFG_LINE_OUT_ENABLE bit from the D_SDU_CFG register, should only be used for debugging purposes.

14.6.1 Control and Configuration Registers

Register Name	Register Description	Address
D_SDU_OUT_0	Signal detection unit samples	P:0xF380
D_SDU_OUT_1	Signal detection unit samples	P:0xF381
D_SDU_OUT_2	Signal detection unit samples	P:0xF382
D_SDU_OUT_3	Signal detection unit samples	P:0xF383
D_SDU_OUT_4	Signal detection unit samples	P:0xF384
D_SDU_OUT_5	Signal detection unit samples	P:0xF385
D_SDU_OUT_6	Signal detection unit samples	P:0xF386
D_SDU_OUT_7	Signal detection unit samples	P:0xF387
D_SDU_CFG	Signal detection unit configuration	P:0xF388

14.6.1.1 D_SDU_OUT_* Settings

D_SDU_OUT_0 to D_SDU_OUT_7 all contain the same bit fields.

Bit Field	Field Name	Description
31:0	SDU_OUT	10-bit AD conversion result, sign extended to 32 bits

14.6.1.2 D_SDU_CFG Settings

Bit Field	Field Name	Description
24	HIGH_FREQ	SDU high frequency mode (use for slow clock >1.92 MHz)
22	LINE_OUT_ENABLE	Lineout pad control
18:16	SFCR	Sampling frequency configuration
14:12	P_CURRENT	Trimming of the preamplifier current
10:8	ADC_CURRENT	Trimming of the ADC current
7:4	PR_GAIN	Select the gain of the preamplifier
3:1	IN_SEL	Select the input applied to preamplifier
0	ADC_ENABLE	Enable/disable the ADC

Field Name	Value Symbol	Value Description	Hex Value
HIGH_FREQ	SDU_HIGH_FREQ_DISABLE	High frequency mode disabled	0x0*
	SDU_HIGH_FREQ_ENABLE	High frequency mode enabled	0x1
LINE_OUT_ENABLE	SDU_LINEOUT_DISABLE	Line out disabled	0x0*
	SDU_LINEOUT_ENABLE	Line out enabled (debugging only)	0x1
SFCR	SDU_SFCR_20	Sampling frequency is SLOWCLK / 20	0x0
	SDU_SFCR_40	Sampling frequency is SLOWCLK / 40	0x1
	SDU_SFCR_60	Sampling frequency is SLOWCLK / 60	0x2
	SDU_SFCR_80	Sampling frequency is SLOWCLK / 80	0x3
	SDU_SFCR_160	Sampling frequency is SLOWCLK / 160	0x4
	SDU_SFCR_320	Sampling frequency is SLOWCLK / 320	0x5
	SDU_SFCR_640	Sampling frequency is SLOWCLK / 640	0x6
	SDU_SFCR_1280	Sampling frequency is SLOWCLK / 1280	0x7*
P_CURRENT	SDU_P_CURRENT_4_0X	4 times the nominal current	0x0
	SDU_P_CURRENT_2_0X	2 times the nominal current	0x1
	SDU_P_CURRENT_1_3X	1.3 times the nominal current	0x2
	SDU_P_CURRENT_1_0X	The nominal current	0x3*
	SDU_P_CURRENT_0_8X	0.8 times the nominal current	0x4
	SDU_P_CURRENT_0_7X	0.7 times the nominal current	0x5
	SDU_P_CURRENT_0_6X	0.6 times the nominal current	0x6
	SDU_P_CURRENT_0_5X	0.5 times the nominal current	0x7

Field Name	Value Symbol	Value Description	Hex Value
ADC_CURRENT	SDU_ADC_CURRENT_3_0X	3 times the nominal current	0x0
	SDU_ADC_CURRENT_1_8X	1.8 times the nominal current	0x1
	SDU_ADC_CURRENT_1_3X	1,3 times the nominal current	0x2
	SDU_ADC_CURRENT_1_0X	The nominal current	0x3*
	SDU_ADC_CURRENT_0_8X	0.8 times the nominal current	0x4
	SDU_ADC_CURRENT_0_7X	0.7 times the nominal current	0x5
	SDU_ADC_CURRENT_0_6X	0.6 times the nominal current	0x6
	SDU_ADC_CURRENT_0_5X	0.5 times the nominal current	0x7
PR_GAIN	SDU_PGAIN_BYPASS	0 dB (Preamplifier bypassed and disabled)	0x0*
	SDU_PGAIN_3DB	3 dB	0x1
	SDU_PGAIN_6DB	6 dB	0x2
	SDU_PGAIN_9DB	9 dB	0x3
	SDU_PGAIN_12DB	12 dB	0x4
	SDU_PGAIN_15DB	15 dB	0x5
	SDU_PGAIN_18DB	18 dB	0x6
	SDU_PGAIN_21DB	21 dB	0x7
	SDU_PGAIN_24DB	24 dB	0x8
	SDU_PGAIN_27DB	27 dB	0x9
	SDU_PGAIN_30DB	30 dB	0xA
	SDU_PGAIN_33DB	33 dB	0xB
	SDU_PGAIN_36DB	36 dB	0xC
IN_SEL	SDU_AI0	Input is Analog Channel 0	0x0
	SDU_AI1	Input is Analog Channel 1	0x1
	SDU_AI2	Input is Analog Channel 2	0x2
	SDU_AI3	Input is Analog Channel 3	0x3
	SDU_AI4	Input is Analog Channel 4	0x4
	SDU_AI5	Input is Analog Channel 5	0x5
	SDU_NO_INPUT	No input connected	0x7*
ADC_ENABLE	SDU_DISABLE	ADC disabled	0x0*
	SDU_ENABLE	ADC enabled	0x1

CHAPTER 15
ARM Cortex-M3 Processor Peripherals

15.1 NESTED VECTORED INTERRUPT CONTROLLER (NVIC)

The ARM Cortex-M3 processor is closely tied to a nested vectored interrupt controller (NVIC) which is an integral part of the processor and provides the interrupt handling functionality. This block is implemented with the ARM Cortex-M3 processor and is described in the *Cortex-M3 Technical Reference Manual*.

The NVIC handles a non-maskable interrupt (NMI), several faults, predefined interrupts, and a set of general-purpose interrupts that are external to the ARM Cortex-M3 processor, and linked to its interfaces and peripherals. A list of the interrupts supported by the NVIC for the ARM Cortex-M3 processor is provided in Table 42.

Table 42. Interrupts in the Ezairo 7100 Microcontroller

Interrupt Enumeration Define	Enumeration Value	Vector Number	Description
Reset_IRQn	-15	1	Reset vector
NonMaskableInt_IRQn	-14	2	Non-maskable interrupt (NMI)
HardFault_IRQn	-13	3	Hard fault interrupt
MemoryManagement_IRQn	-12	4	Memory management interrupt
BusFault_IRQn	-11	5	Bus fault interrupt
UsageFault_IRQn	-10	6	Usage fault interrupt
SVCall_IRQn	-5	11	SVCall interrupt
DebugMonitor_IRQn	-4	12	Debug monitor interrupt
PendSV_IRQn	-2	14	PendSV interrupt
SysTick_IRQn	-1	15	System Tick interrupt
FIFO_0_IRQn	0	16	FIFO interrupt 0
FIFO_1_IRQn	1	17	FIFO interrupt 1
FIFO_2_IRQn	2	18	FIFO interrupt 2
FIFO_3_IRQn	3	19	FIFO interrupt 3
FIFO_4_IRQn	4	20	FIFO interrupt 4
FIFO_5_IRQn	5	21	FIFO interrupt 5
FIFO_6_IRQn	6	22	FIFO interrupt 6
FIFO_7_IRQn	7	23	FIFO interrupt 7
CFX_0_IRQn	8	24	CFX interrupt 0
CFX_1_IRQn	9	25	CFX interrupt 1
CFX_2_IRQn	10	26	CFX interrupt 2
CFX_3_IRQn	11	27	CFX interrupt 3
CFX_4_IRQn	12	28	CFX interrupt 4
CFX_5_IRQn	13	29	CFX interrupt 5
CFX_6_IRQn	14	30	CFX interrupt 6
CFX_7_IRQn	15	31	CFX interrupt 7
DMA0_IRQn	16	32	DMA channel 0 interrupt
DMA1_IRQn	17	33	DMA channel 1 interrupt
DMA2_IRQn	18	34	DMA channel 2 interrupt

Table 42. Interrupts in the Ezairo 7100 Microcontroller (Continued)

Interrupt Enumeration Define	Enumeration Value	Vector Number	Description
DMA3_IRQn	19	35	DMA channel 3 interrupt
GPIO_IRQn	20	36	GPIO interrupt
SPI_RX_IRQn	21	37	SPI receive interrupt
SPI_TX_IRQn	22	38	SPI transmit interrupt
SPI_ERROR_IRQn	23	39	SPI error interrupt
PCM_RX_IRQn	24	40	PCM receive interrupt
PCM_TX_IRQn	25	41	PCM transmit interrupt
PCM_ERROR_IRQn	26	42	PCM error interrupt
I2C_IRQn	27	43	I^2C interrupt
CVSD_ENC_IN_RDY_IRQn	28	44	CVSD encoder input ready interrupt
CVSD_ENC_OUT_RDY_IRQn	29	45	CVSD encoder output ready interrupt
CVSD_DEC_IN_RDY_IRQn	30	46	CVSD decoder input ready interrupt
CVSD_DEC_OUT_RDY_IRQn	31	47	CVSD decoder output ready interrupt
G722_ENC_IN_RDY_IRQn	32	48	G.722 encoder input ready interrupt
G722_ENC_OUT_RDY_IRQn	33	49	G.722 encoder output ready interrupt
G722_DEC_IN_RDY_IRQn	34	50	G.722 decoder input ready interrupt
G722_DEC_OUT_RDY_IRQn	35	51	G.722 decoder output ready interrupt
RS_ENC_IN_RDY_IRQn	36	52	Reed-Solomon encoder input ready interrupt
RS_ENC_OUT_RDY_IRQn	37	53	Reed-Solomon encoder output ready interrupt
RS_DEC_IN_RDY_IRQn	38	54	Reed-Solomon decoder input ready interrupt
RS_DEC_OUT_RDY_IRQn	39	55	Reed-Solomon decoder output ready interrupt
ECB_STATUS_IRQn	40	56	Error Correction Coding blocks status interrupt
ACB_STATUS_IRQn	41	57	Audio CODEC blocks status interrupt

Table 43 lists the NVIC registers and the following subsections describe their bit fields and use by the Ezairo 7100 microcontroller.

Table 43. NVIC Control Registers

Register Name	Register Description	Address
SCnSCB_ICTR	NVIC Interrupt Controller Type Register	0xE000E004
NVIC_ISER0	NVIC External Interrupt Set Enable Register 0	0xE000E100
NVIC_ISER1	NVIC External Interrupt Set Enable Register 1	0xE000E104
NVIC_ICER0	NVIC External Interrupt Clear Enable Register 0	0xE000E180
NVIC_ICER1	NVIC External Interrupt Clear Enable Register 1	0xE000E184
NVIC_ISPR0	NVIC External Interrupt Set Pending Register 0	0xE000E200
NVIC_ISPR1	NVIC External Interrupt Set Pending Register 1	0xE000E204
NVIC_ICPR0	NVIC External Interrupt Clear Pending Register 0	0xE000E280

Table 43. NVIC Control Registers

Register Name	Register Description	Address
NVIC_ICPR1	NVIC External Interrupt Clear Pending Register 1	0xE000E284
NVIC_IABR0	NVIC External Interrupt Active Register 0	0xE000E300
NVIC_IABR1	NVIC External Interrupt Active Register 1	0xE000E304
NVIC_IP0	NVIC External Interrupt Priority Register 0	0xE000E400
NVIC_IP1	NVIC External Interrupt Priority Register 1	0xE000E404
NVIC_IP2	NVIC External Interrupt Priority Register 2	0xE000E408
NVIC_IP3	NVIC External Interrupt Priority Register 3	0xE000E40C
NVIC_IP4	NVIC External Interrupt Priority Register 4	0xE000E410
NVIC_IP5	NVIC External Interrupt Priority Register 5	0xE000E414
NVIC_IP6	NVIC External Interrupt Priority Register 6	0xE000E418
NVIC_IP7	NVIC External Interrupt Priority Register 7	0xE000E41C
NVIC_IP8	NVIC External Interrupt Priority Register 8	0xE000E420
NVIC_IP9	NVIC External Interrupt Priority Register 9	0xE000E424
NVIC_IP10	NVIC External Interrupt Priority Register 10	0xE000E428
SCB_CPUID	NVIC CPU ID Base Register	0xE000ED00
SCB_ICSR	NVIC Interrupt Control and State Register	0xE000ED04
SCB_VTOR	NVIC Vector Table Offset Register	0xE000ED08
SCB_AIRCR	NVIC Application Interrupt and Reset Control Register	0xE000ED0C
SCB_SCR	NVIC System Control Register	0xE000ED10
SCB_CCR	NVIC Configuration Control Register	0xE000ED14
SCB_SHP0	NVIC System Interrupt Priority Register 0	0xE000ED18
SCB_SHP1	NVIC System Interrupt Priority Register 1	0xE000ED1C
SCB_SHP2	NVIC System Interrupt Priority Register 2	0xE000ED20
SCB_SHCSR	NVIC System Handler Control and State Register	0xE000ED24
SCB_CFSR	NVIC Fault Status Register	0xE000ED28
SCB_HFSR	NVIC Hard Fault Status Register	0xE000ED2C
SCB_DFSR	NVIC Debug Fault Status Register	0xE000ED30
SCB_MMFAR	NVIC Memory Management Fault Address Register	0xE000ED34
SCB_BFAR	NVIC Bus Fault Address Register	0xE000ED38
NVIC_STIR	NVIC Software Trigger Interrupt Register	0xE000EF00

IMPORTANT: The NVIC is a standard component provided with the ARM Cortex-M3 processor. The registers for this peripheral are defined in *core_cm3.h* and augmented by defines for the bit fields, bit settings and subregisters in *sk4_hw.h*. Only the ARM Cortex-M3 private peripheral bus (see Section 7.2.2, "Memory Bus Structures" on page 76) can access these registers, and not the CFX DSP.

15.1.1 Interrupt Controller Type Register

The Interrupt Controller Type register (ICTR) indicates the number of interrupts supported by the NVIC. This register is from the ARM Cortex-M3 processor's SCnSCB register block. Table 44 describes this register.

Table 44. Interrupt Controller Type Register Bit Assignments

Bit Field	Field Name	Description
4:0	INTLINESNUM	Total number of interrupt line groups: Ezairo 7100 uses 42 external interrupts (NVIC_INTLINESNUM_33_64, hex value 0x1)

15.1.2 Interrupt Set Enable and Clear Enable Registers

Use the Interrupt Set Enable registers (ISER[0], ISER[1]) to:

- Enable interrupts
- Determine which interrupts are currently enabled

Use the Interrupt Clear-Enable registers (ICER[0], ICER[1]) to:

- Disable interrupts
- Determine which interrupts are currently disabled

These registers contain a bit for each of the external interrupts (vectors 16 to 57) listed in Table 42 on page 239. Setting a bit in an Interrupt Set-Enable register enables the corresponding interrupt. Setting a bit in an Interrupt Clear-Enable register disables the corresponding interrupt.

When the enable bit of a pending interrupt is set, the processor activates the interrupt based on its priority. When the enable bit is cleared, asserting its interrupt signal pends the interrupt, but it is not possible to activate the interrupt, regardless of its priority. Therefore, a disabled interrupt can serve as a latched general-purpose I/O bit that can be read and cleared without invoking an interrupt.

NOTE: Clearing an Interrupt Set-Enable Register bit does not affect currently active interrupts. It only prevents new activations.

These registers are part of the ARM Cortex-M3 processor's NVIC register block. Table 45 describes the field of the Interrupt Set-Enable registers. Table 46 describes the field of the Interrupt Clear-Enable registers.

Table 45. Interrupt Set-Enable Register Bit Assignments

Bit Field	Field Name	Description
31:0	SETENA	Interrupt set enable bits. For write operation: • 1 = Enable interrupt • 0 = No effect. For read operation: • 1 = Interrupt enabled • 0 = Interrupt disabled Writing 0 to a SETENA bit has no effect. Reading the bit returns its current enable state. Reset clears the SETENA fields.

Table 46. Interrupt Clear-Enable Register Bit Assignments

Bit Field	Field Name	Description
31:0	CLRENA	Interrupt clear-enable bits. For write operation: • 1 = Disable interrupt • 0 = No effect. For read operation: • 1 = Enable interrupt • 0 = Disable interrupt. Writing 0 to a CLRENA bit has no effect. Reading the bit returns its current enable state.

15.1.3 Interrupt Set-Pending Registers and Interrupt Clear-Pending Registers

Use the Interrupt Set-Pending Registers (`ISPR[0]`, `ISPR[1]`) to:

- Force interrupts into the pending state
- Determine which interrupts are currently pending

Use the Interrupt Clear-Pending Registers (`ICPR[0]`, `ICPR[1]`) to:

- Clear pending interrupts
- Determine which interrupts are currently pending

These registers contain a bit for each of the external interrupts (vectors 16 to 57) listed in Table 42 on page 239. Setting a bit in an Interrupt Set-Pending register causes the corresponding interrupt to be pending. Setting a bit in an Interrupt Clear-Pending register puts the interrupt into the non-pending state.

NOTE: Writing to an Interrupt Set-Pending register has no effect on an interrupt that is already pending or is disabled. Similarly, writing to an Interrupt Clear-Pending register has no effect on an interrupt that is active unless it is also pending.

These registers are part of the ARM Cortex-M3 processor's `NVIC` register block. Table 47 describes the Interrupt Set-Pending registers. Table 48 describes the Interrupt Clear-Pending registers.

Table 47. Interrupt Set-Pending Register Bit Assignments

Bit Field	Field Name	Description
31:0	SETPEND	Interrupt set-pending bits: • 1 = Pend the corresponding interrupt • 0 = Corresponding interrupt not pending Writing 0 to a SETPEND bit has no effect. Reading the bit returns its current state.

Table 48. Interrupt Clear-Pending Registers Bit Assignments

Bit Field	Field Name	Description
31:0	CLRPEND	Interrupt clear-pending bits: • 1 = Clear pending interrupt • 0 = Do not clear pending interrupt Writing 0 to a CLRPEND bit has no effect. Reading the bit returns its current state.

15.1.4 Active Bit Register

Read the Active Bit registers (`IABR[0], IABR[1]`) to determine which interrupts are active. This register contains a flag for each of the external interrupts (vectors 16 to 57) listed in Table 42 on page 239. These registers are part of the ARM Cortex-M3 processor's `NVIC` register block. Table 49 describes the Active Bit register.

Table 49. Active Bit Register Bit Assignments

Bit Field	Field Name	Description
31:0	ACTIVE	Interrupt active flags: • 1 = Interrupt active or pre-empted and stacked • 0 = Interrupt not active nor stacked.

15.1.5 Interrupt Priority Registers

Use the Interrupt Priority Registers (`IP[0]` to `IP[10]`) to assign a priority to each of the available interrupts. Each byte in an Interrupt Priority can be used to set the priority for one of the external interrupts (vectors 16 to 57) listed in Table 42 on page 239.

NOTE: Configuration of the interrupt priorities for standard ARM Cortex-M3 processor exceptions (vectors 4 to 15) are set using the System Handler Priority registers. For more information, see Section 15.1.11, "System Handler Priority Registers".

The NVIC for the ARM Cortex-M3 processor in the Ezairo 7100 system has been implemented with four interrupt priority bits per interrupt. These four priority bits are MSB aligned to an eight-bit priority bit field as required by ARM. Generally, the lower the priority value, the higher priority that interrupt is given.

The `SCB_AIRCR_PRIGROUP` bit field from the Application Interrupt and Reset Control register (see Section 15.1.8, "Application Interrupt and Reset Control Register" on page 247) is used to divide the interrupt priority settings into interrupt groups, and to prioritize interrupts within those groups. The possible configurations for the division of the priority bit field into pre-emption priority and subgroup priority is shown in Table 50.

Table 50. Division of Priority into Pre-empt Priority and Subgroup Priority

PRIGROUP Setting	Pre-empt Priority Field	Subgroup Priority Field
0x0 to 0x3	7:4	-
0x4	7:5	4
0x5	7:6	5:4

Table 50. Division of Priority into Pre-empt Priority and Subgroup Priority

PRIGROUP Setting	Pre-empt Priority Field	Subgroup Priority Field
0x6	7	6:4
0x7	-	7:4

When choosing which interrupt to activate, the priority settings are applied as follows:

- When multiple interrupts are pending, but no interrupts are active, the interrupt with the lowest priority setting is activated. If more than one pending interrupt shares the lowest priority setting, the interrupt with the lower vector number is activated.
- If an interrupt is currently active, it can be pre-empted by any interrupt that is pended in a lower numbered group. If multiple interrupts that could pre-empt the active interrupt are pending, the interrupt with the lowest priority setting is activated. As before, if more than one pending interrupt shares the lowest priority setting, the interrupt with the lower vector number is activated.

For example, setting the SCB_AIRCR_PRIGROUP bit field to 0x5 divides the four interrupt priority bits to use bits [7:6] to assign the interrupt to a group, and bits [5:4] to assign the interrupt a priority relative to the other interrupts in that group. If the following interrupts are pending:

- FIFO_0 with priority set to 0x60
- CFX_0 with priority set to 0x50
- DMA0 with priority set to 0x50

The CFX_0 and DMA0 interrupts have the lowest priority of the pending interrupts, and the CFX_0 interrupt is activated because it has a lower vector number (24 verses 32). If the FIFO_1 interrupt is then pended with a priority of 0x40, the CFX_0 interrupt is not pre-empted because the FIFO_1, CFX_0 and DMA0 interrupts all share the same priority group. If the GPIO interrupt is then pended with a priority level of 0x10, the CFX_0 interrupt is pre-empted because the GPIO interrupt belongs to a higher priority interrupt group.

IMPORTANT: The reset, NMI and fault vectors have priority levels of -3, -2, and -1 respectively. As such, these events can always pre-empt interrupts with lower priorities.

These registers are part of the ARM Cortex-M3 processor's NVIC register block. Table 51 describes the bit assignments for a single interrupt within the Interrupt Priority registers.

Table 51. Interrupt Priority Register Bit Assignments

Bit Field	Field Name	Description
7:4	PRI_**n**	Priority of interrupt *n*

15.1.6 Interrupt Control State Register

Use the Interrupt Control State register (ICSR) to:

- Pend the *Non-Maskable Interrupt* (NMI)
- Set or clear a pending SVC interrupt
- Set or clear a pending SysTick interrupt
- Check for pending exceptions

- Check the vector number of the highest priority pended exception
- Check the vector number of the active exception

This register is part of the ARM Cortex-M3 processor's SCB register block. Table 52 describes the Interrupt Control State register.

Table 52. Interrupt Control State Register Bit Assignments

Bit Field	Field Name	Type	Description
31	NMIPENDSET	Read/write	Set pending NMI bit: • 1 = Set the NMI to be pending • 0 = Do not set the NMI to be pending NMIPENDSET pends and activates an NMI. Because NMI is the highest-priority interrupt, it takes effect as soon as it registers.
28	PENDSVSET	Read/write	Set pending pendSV bit: • 1 = Set pendSV to be pending • 0 = Do not set pendSV to be pending
27	PENDSVCLR	Write-only	Clear pending pendSV bit: • 1 = Clear a pending pendSV interrupt (if a pendSV interrupt is pending) • 0 = Do not clear the pendSV interrupt pending state
26	PENDSTSET	Read/write	Set pending SysTick bit • 1 = Set the SysTick interrupt to be pending • 0 = Do not set the SysTick interrupt to be pending
25	PENDSTCLR	Write-only	Clear pending SysTick bit: • 1 = Clear a pending SysTick interrupt (if a SysTick interrupt is pending) • 0 = Do not clear the SysTick interrupt pending state
23	ISRPREEMPT	Read-only	Indicates that a pending interrupt becomes active in the next running cycle. If CoreDebug_DHCSR_C_MASKINTS is clear in the Debug Halting Control and Status Register, the interrupt is serviced. NOTE: This bit is only useful for debug purposes and should not be used to check for pending interrupts by an application (it will never be set outside of debug mode)
22	ISRPENDING	Read-only	Interrupt pending flag. Excludes NMI and all faults: • 1 = Interrupt pending • 0 = Interrupt not pending.
21:12	VECTPENDING	Read-only	Pending ISR vector number. This field contains the vector number of the highest priority pending ISR.
11	RETTOBASE	Read-only	This bit is 1 when no exceptions are active or the current active exception is the only active exception (i.e., no pre-empted exception handlers currently exist).
8:0	VECTACTIVE	Read-only	Active ISR number field. This field contains the vector number of the currently running ISR, including NMI and Hard Fault. A shared handler can use SCB_ICSR_VECTACTIVE to determine which interrupt invoked it. An application can subtract 16 from the SCB_ICSR_VECTACTIVE field to index into the Interrupt Clear/Set Enable, Interrupt Clear Pending/Set Pending and Interrupt Priority Registers for all external interrupt sources. Reset clears the SCB_ICSR_VECTACTIVE field.

15.1.7 Vector Table Offset Register

Use the Vector Table Offset register (VTOR) to determine or set where the interrupt vector table is located. This register is part of the ARM Cortex-M3 processor's SCB register block. Table 53 describes the Vector Table Offset register.

Table 53. Vector Table Offset Register Bit Assignments

Bit Field	Field Name	Description
29	TBLBASE	Table base is in ARM Cortex-M3 processor program memory (0) or ARM Cortex-M3 processor data memory (1). See Section 7.5, "ARM Cortex-M3 Processor Memory Space" on page 95.
28:7	TBLOFF	Vector table base offset field. Contains the offset of the table base from the bottom of the SRAM or CODE space.

IMPORTANT: The vector table offset position must be aligned to a 64-word or 256-byte boundary due to the number of interrupts in the Ezairo 7100 microcontroller.

15.1.8 Application Interrupt and Reset Control Register

Use the Application Interrupt and Reset Control register (AIRCR) to:

- Determine data endianness
- Clear all active state information for debugging or to recover from a hard failure
- Request a system reset or trigger a reset of the ARM Cortex-M3 processor
- Alter the priority grouping position (binary point); for more information about setting interrupt priorities, see Section 15.1.5, "Interrupt Priority Registers"

IMPORTANT: Because this register can be used to reset key system configurations (including requesting a full system reset), writes to this register are protected by a vector key. When writing this register, if the VECTKEY is not written to 0x05FA, the write operation is ignored. Reading this key returns 0xFA05.

This register is part of the ARM Cortex-M3 processor's SCB register block. Table 54 describes the bit assignments of the Application Interrupt and Reset Control Register.

Table 54. Application Interrupt and Reset Control Register Bit Assignments

Bit Field	Field Name	Description	
31:16	VECTKEY	Register key; set to 0x05FA when writing this register	
15	ENDIANESS	The ARM Cortex-M3 processor in the Ezairo 7100 system is configured to use data in little endian mode (NVIC_LITTLE_ENDIAN)	
10:8	PRIGROUP	0 - 3	4.0 indicates four bits of pre-emption priority, zero bits of subpriority
		4	3.1 indicates three bits of pre-emption priority, one bit of subpriority
		5	2.2 indicates two bits of pre-emption priority, two bits of subpriority
		6	1.3 indicates one bit of pre-emption priority, seven bits of subpriority
		7	0.4 indicates no pre-emption priority, eight bits of subpriority.

Table 54. Application Interrupt and Reset Control Register Bit Assignments (Continued)

Bit Field	Field Name	Description
2	SYSRESETREQ	Request a POR. Intended to force a system reset of all major digital components except for debug. Setting this bit does not prevent Halting Debug from running. For more information about the POR, see Section 19.3, “Resets” on page 478.
1	VECTCLRACTIVE	Clear active vector bit: 1 = Clear all state information for active NMI, fault, and interrupts 0 = Do not clear the active vector state
0	VECTRESET	Reset bit. Resets the ARM Cortex-M3 core, NVIC and SysTick timer. Does not reset the ARM Cortex-M3 interfaces (including the debug port components) or other peripherals: 1 = Reset the ARM Cortex-M3 core 0 = Do not reset the ARM Cortex-M3 core. A reset clears this bit. NOTE: When debugging, this bit should only be written when the ARM Cortex-M3 core is halted.

15.1.9 System Control Register

Use the System Control register (SCR) for power-management functions related to the ARM Cortex-M3 processor execution. This register is part of the ARM Cortex-M3 processor’s SCB register block. Table 55 describes the System Control register.

Table 55. System Control Register Bit Assignments

Bit Field	Field Name	Description
4	SEVONPEND	Wake up from the ARM Cortex-M3 core wait for event (WFE) instruction if a new interrupt is pended, regardless of whether the interrupt has priority higher than the current interrupt priority level.
2	SLEEPDEEP	Sleep deep bit: 1 = Indicate to the Ezairo 7100 system that the ARM Cortex-M3 core clock can be stopped 0 = Indicate that the ARM Cortex-M3 core clock should not be stopped For more information about the use of **SLEEPDEEP**, see the *ARM Cortex-M3 Technical Reference Manual*.
1	SLEEPONEXIT	Sleep on exit when returning from the last active interrupt to the base application: 1 = Sleep on ISR exit 0 = Do not sleep when returning to the base application code This bit enables interrupt driven applications to avoid returning to an empty main application.

15.1.10 Configuration Control Register

Use the Configuration Control register (CCR) to:

- Ignore bus faults during NMI and hard fault handling
- Enable usage faults for divide by zero and unaligned accesses
- Enable user access to the Software Trigger Exception Register
- Control entry to Thread Mode

This register is part of the ARM Cortex-M3 processor's SCB register block. Table 56 describes the Configuration Control register.

Table 56. Configuration Control Register Bit Assignments

Bit Field	Field Name	Description
9	STKALIGN	If set on exception entry, the stack pointer used prior to the exception is adjusted to be 8-byte aligned, and the context to restore it is saved. The stack pointer is restored on the associated exception return. If not set, only 4-byte alignment is guaranteed for the stack pointer used prior to the exception on exception entry.
8	BFHFNMIGN	When this bit is set to 1, this causes handlers running at priority -2 and -1 (NMI, hard fault, and faults promoted to hard faults) to ignore data bus faults caused by load and store instructions. When this bit is set to 0, these bus faults cause a lock-up. NOTE: Use this enable with extreme caution as all data bus faults are ignored. Only use this when the exception handler and its data are in absolutely safe memory. Its normal use is to probe system devices and bridges to detect control path problems and fix them.
4	DIV_0_TRP	Trap on divide by 0. When enabled (set to 1), this causes a usage fault to occur when an attempt is made to divide by 0.
3	UNALIGN_TRP	Trap for unaligned access. When enabled (set to 1), this causes a usage fault on any unaligned half word or full word access. Unaligned load-store multiples always cause a fault.
1	USERSETMPEND	If set to 1, allows user level code (in addition to privileged level code) to write the Software Trigger Interrupt register to trigger (pend) an exception.
0	NONBASETHRDENA	If set to 0, it is only possible to return to user mode when returning from the last exception. If set to 1, user mode can be entered from any level in Handler mode by a controlled return value.

15.1.11 System Handler Priority Registers

Use the three System Handler Priority registers (SHP[*]) to prioritize the following system handlers:

- Memory management fault (SHP[0])
- Bus fault (SHP[1])
- Usage fault (SHP[2])
- System Service Call (SVC) interrupt (SHP[7])
- Debug monitor interrupt (SHP[8])
- PendSV interrupt (SHP[10])
- SysTick interrupt (SHP[11])

System handlers are a special class of exception handler that can have their priority set to any of the priority levels in the same way that these priority levels are applied to external interrupts. For more information about setting priority levels, see Section 15.1.5, "Interrupt Priority Registers" on page 244.

These registers are part of the ARM Cortex-M3 processor's SCB register block.

IMPORTANT: If a memory management fault, bus fault, or usage fault occurs and the exception handler cannot be activated immediately due to its handler's priority relative to the active exception, the fault will be promoted to a hard fault. An exception to this rule is an imprecise bus fault (see Section 15.1.13, "Fault Status Registers"), which will be pended and not promoted to a hard fault.

15.1.12 System Handler Control and State Register

Use the System Handler Control and State register (SHCSR) to:

- Enable or disable the system fault handlers
- Determine the pending status of bus fault, memory management fault, and SVC
- Determine the active status of the system handlers

If a fault condition occurs while its fault handler is disabled, the fault is promoted to a Hard Fault.

This register is part of the ARM Cortex-M3 processor's SCB register block. Table 57 describes the bit assignments of the System Handler Control Register.

Table 57. System Handler Control and State Register Bit Assignments

Bit Field	Field Name	Description
18	USGFAULTENA	Enable or disable usage faults
17	BUSFAULTENA	Enable or disable bus faults
16	MEMFAULTENA	Enable or disable memory management faults
15	SVCALLPENDED	If 1, SVC is pending; write 0 to clear the SVC status
14	BUSFAULTPENDED	If 1, a bus fault is pending; write 0 to clear the bus fault status
13	MEMFAULTPENDED	If 1, a memory management fault is pending; write 0 to clear the memory management fault status
12	USGFAULTPENDED	If 1, a usage fault is pending; write 0 to clear the usage fault status
11	SYSTICKACT	Set to 1 if the SysTick interrupt handler is active.
10	PENDSVACT	Set to 1 if the PendSV interrupt handler is active.
8	MONITORACT	Set to 1 if the debug monitor interrupt handler is active.
7	SVCALLACT	Set to 1 if the SVC interrupt handler is active.
3	USGFAULTACT	Set to 1 if the usage fault handler is active.
1	BUSFAULTACT	Set to 1 if the bus fault handler is active.
0	MEMFAULTACT	Set to 1 if the memory management fault handler is active.

The pending bits are only set when an exception has been deferred because of the arrival of a higher priority interrupt. This enables higher priority event handlers to clear pended faults before actually running the handler.

The active bits indicate whether any of the system handlers are active, running now, or stacked because of pre-emption. This information is used for debugging and is also used by the application handlers.

CAUTION: Take extra care when writing this register; if an active bit is cleared, and the corresponding exception handler attempts to use an exception exit, a fault will occur.

15.1.13 Fault Status Registers

The Configurable Fault Status register (CFSR) and Hard Fault Status Register (HFSR) are used to obtain information about local faults. The flags in these registers indicate the causes of local faults. Multiple flags can be set if more than one fault occurs.

The Configurable Fault Status register is divided into three areas:

- Bits 31:16 describe the usage fault status
- Bits 15:8 describe the bus fault status
- Bits 7:0 describe the memory management fault status

Table 58 describes the Fault Status register.

Table 58. Fault Status Register Bit Assignments

Bit Field	Field Name	Description
25	DIVBYZERO	When the SCB_CCR_DIV_0_TRP bit is set in the Configuration Control register (CCR) and an SDIV or UDIV instruction is used with a divisor of 0, a usage fault occurs and this flag is set. The instruction is executed and the return PC points to it. If SCB_CCR_DIV_0_TRP is not set, then the divide returns a quotient of 0.
24	UNALIGNED	When the SCB_CCR_UNALIGN_TRP bit is set in the Configuration Control register (CCR), and an unaligned memory access is attempted, this flag is set and a usage fault occurs. Unaligned LDM/STM/LDRD/STRD instructions always fault and set this flag irrespective of the setting of SCB_CCR_UNALIGN_TRP bit setting.
19	NOCP	An attempt was made to use an ARM coprocessor support functionality that is not available for the ARM Cortex-M3 core.
18	INVPC	An attempt was made to set the PC to a value that is invalid (possibly always invalid, invalid due to context or because the new PC value would point to an invalid instruction). The return PC points to the instruction that tried but failed to set the PC.
17	INVSTATE	Invalid combination of the EPSR state and an instruction, for reasons other than an undefined instruction. Return PC points to faulting instruction, with the invalid state.
16	UNDEFINSTR	The UNDEFINSTR flag is set when the processor attempts to execute an undefined instruction. This is an instruction that the processor cannot decode. The return PC points to the undefined instruction.
15	BFARVALID	Bus Fault Address register (BFAR) valid flag; indicates if the value in the BFAR points to the address of a failure. If a fault occurs that is escalated to a Hard Fault because of priority, the Hard Fault handler must clear this bit. This prevents problems on return to a stacked active memory management handler whose BFAR value has been overwritten.
12	STKERR	Stacking when entering an exception handler has caused one or more bus faults. The stack pointer is still adjusted and the values in the context area on the stack might be incorrect. The Bus Fault Address register is not written.

Table 58. Fault Status Register Bit Assignments

Bit Field	Field Name	Description
11	UNSTKERR	Unstacking when an exception handler returns has caused one or more bus faults. This is chained to the handler, so that the original return stack is still present. The stack pointer is not adjusted from the failing return and a new save is not performed. The Bus Fault Address register is not written.
10	IMPREISERR	Imprecise data bus error which causes a bus fault, but the return PC is not related to the instruction causing the fault. This is not a synchronous fault. If detected when the priority of the current activation is higher than the Bus Fault event, it is only pended. The bus fault handler activates when returning to a lower priority activation. If a precise fault occurs before returning to a lower priority exception, both PRECISERR and IMPRECISERR will be set. The Bus Fault Address register is not written.
9	PRECISERR	Precise data bus error which causes a bus fault, and the return PC indicates the instruction causing the fault. The Bus Fault Address register is written.
8	IBUSERR	Prefetch error on the instruction bus. The fault stops on the instruction, so if the error occurs on a prefetched instruction that is not executed (for example an instruction that is fetched but is not reached due to a branch that precedes it), no fault occurs.
7	MMARVALID	Memory Management Address register (MMAR) valid flag; indicates if the value in the MMAR points to the address of a failure. If a fault occurs that is escalated to a Hard Fault because of priority, the Hard Fault handler must clear this bit. This prevents problems on return to a stacked active memory management handler whose MMAR value has been overwritten.
4	MSTKERR	Stacking when entering an exception handler has caused one or more access violations. The stack pointer is still adjusted and the values in the context area on the stack might be incorrect. The Memory Management Address register is not written.
3	MUNSTKERR	Unstacking when an exception handler returns has caused one or more access violations. This is chained to the handler, so that the original return stack is still present. The stack pointer is not adjusted from failing to return and a new save is not performed. The Memory Management Address register is not written.
1	DACCVIOL	Data access violation flag. Attempting to load or store at a location that does not permit the operation sets the DACCVIOL flag. The return PC points to the faulting instruction. This error loads the Memory Management Address register with the address of the attempted access.
0	IACCVIOL	Instruction access violation flag. Attempting to fetch an instruction from a location that does not permit execution sets the IACCVIOL flag. The return PC points to the faulting instruction. The Memory Management Address register is not written.

The Bus Fault Address register (`BFAR`) can be used to obtain the address of the location that generated a bus fault when this address is known. The Memory Management Address register (`MMAR`) can be used to obtain the address affected by a memory management fault when this address is known.

The Hard Fault status register (`HFSR`) is used to provide information about the event that activated the Hard Fault handler. Table 59 describes the Hard Fault Status register.

Table 59. Hard Fault Status Register Bit Assignments

Bit Field	Field Name	Description
31	DEBUGEVT	Set if a fault related to debug occurs. This is only possible when halting debug is not enabled. For monitor enabled debug, this only happens for BKPT when the current priority is higher than the monitor. When both halting and monitor debug are disabled, this only happens for debug events that are not ignored (minimally, BKPT). The Debug Fault Status Register is updated.
30	FORCED	Hard Fault activated because a fault was received that cannot be activated because of relative priority or the original fault was disabled. The Hard Fault handler then has to read the other fault status registers to determine the cause.
1	VECTTBL	Set if there is a fault because of vector table read on exception processing (bus fault). This case is always considered a hard fault. The return PC points to the pre-empted instruction.

The fault status registers and fault address registers are all are part of the ARM Cortex-M3 processor's SCB register block.

15.1.14 Software Trigger Interrupt Register

Use the Software Trigger Interrupt register (STIR) to pend an external interrupt. System exceptions cannot be pended using this register. This register can only be written when running in user mode when the USERSETMPEND bit from the CCR register (see Section 15.1.10, "Configuration Control Register") is set.

Table 60 describes the Software Trigger Interrupt register. This register is part of the ARM Cortex-M3 processor's NVIC register block.

Table 60. Software Trigger Interrupt Register Bit Assignments

Bit Field	Field Name	Description
8:0	INTID	Select which interrupt vector to pend; write to the interrupt's vector number minus 16. Writing a value to this register is equivalent to manually pending an interrupt by setting the corresponding interrupt bit in an Interrupt Set Pending register (which is never possible when running in user mode).

15.1.15 NVIC Control and Configuration Registers

See the ARM *Cortex Microcontroller Software Interface Standard (CMSIS)* for a complete list and description of the NVIC registers.

Register Name	Register Description	Address
SCnSCB_ICTR	NVIC Interrupt Controller Type Register	0xE000E004
NVIC_ISER0	NVIC External Interrupt Set Enable Register 0	0xE000E100
NVIC_ISER1	NVIC External Interrupt Set Enable Register 1	0xE000E104
NVIC_ICER0	NVIC External Interrupt Clear Enable Register 0	0xE000E180
NVIC_ICER1	NVIC External Interrupt Clear Enable Register 1	0xE000E184
NVIC_ISPR0	NVIC External Interrupt Set Pending Register 0	0xE000E200

Register Name	Register Description	Address
NVIC_ISPR1	NVIC External Interrupt Set Pending Register 1	0xE000E204
NVIC_ICPR0	NVIC External Interrupt Clear Pending Register 0	0xE000E280
NVIC_ICPR1	NVIC External Interrupt Clear Pending Register 1	0xE000E284
NVIC_IABR0	NVIC External Interrupt Active Register 0	0xE000E300
NVIC_IABR1	NVIC External Interrupt Active Register 1	0xE000E304
NVIC_IP0	NVIC External Interrupt Priority Register 0	0xE000E400
NVIC_IP1	NVIC External Interrupt Priority Register 1	0xE000E404
NVIC_IP2	NVIC External Interrupt Priority Register 2	0xE000E408
NVIC_IP3	NVIC External Interrupt Priority Register 3	0xE000E40C
NVIC_IP4	NVIC External Interrupt Priority Register 4	0xE000E410
NVIC_IP5	NVIC External Interrupt Priority Register 5	0xE000E414
NVIC_IP6	NVIC External Interrupt Priority Register 6	0xE000E418
NVIC_IP7	NVIC External Interrupt Priority Register 7	0xE000E41C
NVIC_IP8	NVIC External Interrupt Priority Register 8	0xE000E420
NVIC_IP9	NVIC External Interrupt Priority Register 9	0xE000E424
NVIC_IP10	NVIC External Interrupt Priority Register 10	0xE000E428
SCB_CPUID	NVIC CPU ID Base Register	0xE000ED00
SCB_ICSR	NVIC Interrupt Control and State Register	0xE000ED04
SCB_VTOR	NVIC Vector Table Offset Register	0xE000ED08
SCB_AIRCR	NVIC Application Interrupt and Reset Control Register	0xE000ED0C
SCB_SCR	NVIC System Control Register	0xE000ED10
SCB_CCR	NVIC Configuration Control Register	0xE000ED14
SCB_SHP0	NVIC System Interrupt Priority Register 0	0xE000ED18
SCB_SHP1	NVIC System Interrupt Priority Register 1	0xE000ED1C
SCB_SHP2	NVIC System Interrupt Priority Register 2	0xE000ED20
SCB_SHCSR	NVIC System Handler Control and State Register	0xE000ED24
SCB_CFSR	NVIC Fault Status Register	0xE000ED28
SCB_HFSR	NVIC Hard Fault Status Register	0xE000ED2C
SCB_MMFAR	NVIC Memory Management Fault Address Register	0xE000ED34
SCB_BFAR	NVIC Bus Fault Address Register	0xE000ED38
NVIC_STIR	NVIC Software Trigger Interrupt Register	0xE000EF00

15.1.15.1 SCnSCB_ICTR

Bit Field	Field Name	Description
4:0	INTLINESNUM	Number of interrupt inputs in steps of 32

Field Name	Value Symbol	Value Description	Hex Value
INTLINESNUM	NVIC_INTLINESNUM_33_64	NVIC_INTLINESNUM_33_64	0x1*

15.1.15.2 NVIC_ISER0

Bit Field	Field Name	Description
31	CVSD_DEC_OUT_RDY	CVSD CODEC decoder output interface interrupt set enable
30	CVSD_DEC_IN_RDY	CVSD CODEC decoder input interface interrupt set enable
29	CVSD_ENC_OUT_RDY	CVSD CODEC encoder output interface interrupt set enable
28	CVSD_ENC_IN_RDY	CVSD CODEC encoder input interface interrupt set enable
27	I2C	I^2C interrupt set enable
26	PCM_ERROR	PCM error interrupt set enable
25	PCM_TX	PCM transmit buffer empty interrupt set enable
24	PCM_RX	PCM receive buffer full interrupt set enable
23	SPI_ERROR	SPI error interrupt set enable
22	SPI_TX	SPI transmit buffer empty interrupt set enable
21	SPI_RX	SPI receive buffer full interrupt set enable
20	GPIO	GPIO interrupt set enable
19	DMA3	DMA channel 3 interrupt set enable
18	DMA2	DMA channel 2 interrupt set enable
17	DMA1	DMA channel 1 interrupt set enable
16	DMA0	DMA channel 0 interrupt set enable
15	CFX_7	CFX event interrupt 7 set enable
14	CFX_6	CFX event interrupt 6 set enable
13	CFX_5	CFX event interrupt 5 set enable
12	CFX_4	CFX event interrupt 4 set enable
11	CFX_3	CFX event interrupt 3 set enable
10	CFX_2	CFX event interrupt 2 set enable
9	CFX_1	CFX event interrupt 1 set enable
8	CFX_0	CFX event interrupt 0 set enable
7	FIFO_7	FIFO interrupt 7 set enable
6	FIFO_6	FIFO interrupt 6 set enable
5	FIFO_5	FIFO interrupt 5 set enable
4	FIFO_4	FIFO interrupt 4 set enable
3	FIFO_3	FIFO interrupt 3 set enable
2	FIFO_2	FIFO interrupt 2 set enable
1	FIFO_1	FIFO interrupt 1 set enable
0	FIFO_0	FIFO interrupt 0 set enable

Field Name	Value Symbol	Value Description	Hex Value
CVSD_DEC_OUT_RDY	NVIC_CVSD_DEC_OUT_RDY_INT_ENABLE	Enable the CVSD CODEC decoder output interface interrupt	0x1
CVSD_DEC_IN_RDY	NVIC_CVSD_DEC_IN_RDY_INT_ENABLE	Enable the CVSD CODEC decoder input interface interrupt	0x1

Field Name	Value Symbol	Value Description	Hex Value
CVSD_ENC_OUT_RDY	NVIC_CVSD_ENC_OUT_RDY_INT_ENABLE	Enable the CVSD CODEC encoder output interface interrupt	0x1
CVSD_ENC_IN_RDY	NVIC_CVSD_ENC_IN_RDY_INT_ENABLE	Enable the CVSD CODEC encoder input interface interrupt	0x1
I2C	NVIC_I2C_INT_ENABLE	Enable the I^2C interrupt	0x1
PCM_ERROR	NVIC_PCM_ERROR_INT_ENABLE	Enable the PCM error interrupt	0x1
PCM_TX	NVIC_PCM_TX_INT_ENABLE	Enable the PCM transmit buffer empty interrupt	0x1
PCM_RX	NVIC_PCM_RX_INT_ENABLE	Enable the PCM receive buffer full interrupt	0x1
SPI_ERROR	NVIC_SPI_ERROR_INT_ENABLE	Enable the SPI error interrupt	0x1
SPI_TX	NVIC_SPI_TX_INT_ENABLE	Enable the SPI transmit buffer empty interrupt	0x1
SPI_RX	NVIC_SPI_RX_INT_ENABLE	Enable the SPI receive buffer full	0x1
GPIO	NVIC_GPIO_INT_ENABLE	Enable the GPIO interrupt	0x1
DMA3	NVIC_DMA3_INT_ENABLE	Enable the DMA channel 3 interrupt	0x1
DMA2	NVIC_DMA2_INT_ENABLE	Enable the DMA channel 2 interrupt	0x1
DMA1	NVIC_DMA1_INT_ENABLE	Enable the DMA channel 1 interrupt	0x1
DMA0	NVIC_DMA0_INT_ENABLE	Enable the DMA channel 0 interrupt	0x1
CFX_7	NVIC_CFX_7_INT_ENABLE	Enable CFX event interrupt 7	0x1
CFX_6	NVIC_CFX_6_INT_ENABLE	Enable CFX event interrupt 6	0x1
CFX_5	NVIC_CFX_5_INT_ENABLE	Enable CFX event interrupt 5	0x1
CFX_4	NVIC_CFX_4_INT_ENABLE	Enable CFX event interrupt 4	0x1
CFX_3	NVIC_CFX_3_INT_ENABLE	Enable CFX event interrupt 3	0x1
CFX_2	NVIC_CFX_2_INT_ENABLE	Enable CFX event interrupt 2	0x1
CFX_1	NVIC_CFX_1_INT_ENABLE	Enable CFX event interrupt 1	0x1
CFX_0	NVIC_CFX_0_INT_ENABLE	Enable CFX event interrupt 0	0x1
FIFO_7	NVIC_FIFO_7_INT_ENABLE	Enable FIFO interrupt 7	0x1
FIFO_6	NVIC_FIFO_6_INT_ENABLE	Enable FIFO interrupt 6	0x1
FIFO_5	NVIC_FIFO_5_INT_ENABLE	Enable FIFO interrupt 5	0x1
FIFO_4	NVIC_FIFO_4_INT_ENABLE	Enable FIFO interrupt 4	0x1
FIFO_3	NVIC_FIFO_3_INT_ENABLE	Enable FIFO interrupt 3	0x1
FIFO_2	NVIC_FIFO_2_INT_ENABLE	Enable FIFO interrupt 2	0x1
FIFO_1	NVIC_FIFO_1_INT_ENABLE	Enable FIFO interrupt 1	0x1
FIFO_0	NVIC_FIFO_0_INT_ENABLE	Enable FIFO interrupt 0	0x1*

15.1.15.3 NVIC_ISER1

Bit Field	Field Name	Description
9	ACB_COM	ACB combined interrupt set enable
8	ECB_COM	ECB combined interrupt set enable

Bit Field	Field Name	Description
7	RS_DEC_OUT_RDY	RS decoder output registers interrupt set enable
6	RS_DEC_IN_RDY	RS decoder input registers interrupt set enable
5	RS_ENC_OUT_RDY	RS encoder output registers interrupt set enable
4	RS_ENC_IN_RDY	RS encoder input registers interrupt set enable
3	G722_DEC_OUT_RDY	G.722 CODEC decoder output interface registers interrupt set enable
2	G722_DEC_IN_RDY	G.722 CODEC decoder input interface registers interrupt set enable
1	G722_ENC_OUT_RDY	G.722 CODEC encoder output interface registers interrupt set enable
0	G722_ENC_IN_RDY	G.722 CODEC encoder input interface registers interrupt set enable

Field Name	Value Symbol	Value Description	Hex Value
ACB_COM	NVIC_ACB_COM_INT_ENABLE	Enable the ACB combined interrupt	0x1
ECB_COM	NVIC_ECB_COM_INT_ENABLE	Enable the ECB combined interrupt	0x1
RS_DEC_OUT_RDY	NVIC_RS_DEC_OUT_RDY_INT_ENABLE	Enable the RS decoder output registers interrupt	0x1
RS_DEC_IN_RDY	NVIC_RS_DEC_IN_RDY_INT_ENABLE	Enable the RS decoder input registers interrupt	0x1
RS_ENC_OUT_RDY	NVIC_RS_ENC_OUT_RDY_INT_ENABLE	Enable the RS encoder output registers interrupt	0x1
RS_ENC_IN_RDY	NVIC_RS_ENC_IN_RDY_INT_ENABLE	Enable the RS encoder input registers interrupt	0x1
G722_DEC_OUT_RDY	NVIC_G722_DEC_OUT_RDY_INT_ENABLE	Enable the G.722 CODEC decoder output interface registers interrupt	0x1
G722_DEC_IN_RDY	NVIC_G722_DEC_IN_RDY_INT_ENABLE	Enable the G.722 CODEC decoder input interface registers interrupt	0x1
G722_ENC_OUT_RDY	NVIC_G722_ENC_OUT_RDY_INT_ENABLE	Enable the G.722 CODEC encoder output interface registers interrupt	0x1
G722_ENC_IN_RDY	NVIC_G722_ENC_IN_RDY_INT_ENABLE	Enable the G.722 CODEC encoder input interface registers interrupt	0x1

15.1.15.4 NVIC_ICER0

Bit Field	Field Name	Description
31	CVSD_DEC_OUT_RDY	CVSD CODEC decoder output interface interrupt clear enable
30	CVSD_DEC_IN_RDY	CVSD CODEC decoder input interface interrupt clear enable
29	CVSD_ENC_OUT_RDY	CVSD CODEC encoder output interface interrupt clear enable
28	CVSD_ENC_IN_RDY	CVSD CODEC encoder input interface interrupt clear enable
27	I2C	I^2C interrupt clear enable
26	PCM_ERROR	PCM error interrupt clear enable
25	PCM_TX	PCM transmit buffer empty interrupt clear enable
24	PCM_RX	PCM receive buffer full interrupt clear enable
23	SPI_ERROR	SPI error interrupt clear enable
22	SPI_TX	SPI transmit buffer empty interrupt clear enable

Bit Field	Field Name	Description
21	SPI_RX	SPI receive buffer full interrupt clear enable
20	GPIO	GPIO interrupt clear enable
19	DMA3	DMA channel 3 interrupt clear enable
18	DMA2	DMA channel 2 interrupt clear enable
17	DMA1	DMA channel 1 interrupt clear enable
16	DMA0	DMA channel 0 interrupt clear enable
15	CFX_7	CFX event interrupt 7 clear enable
14	CFX_6	CFX event interrupt 6 clear enable
13	CFX_5	CFX event interrupt 5 clear enable
12	CFX_4	CFX event interrupt 4 clear enable
11	CFX_3	CFX event interrupt 3 clear enable
10	CFX_2	CFX event interrupt 2 clear enable
9	CFX_1	CFX event interrupt 1 clear enable
8	CFX_0	CFX event interrupt 0 clear enable
7	FIFO_7	FIFO interrupt 7 clear enable
6	FIFO_6	FIFO interrupt 6 clear enable
5	FIFO_5	FIFO interrupt 5 clear enable
4	FIFO_4	FIFO interrupt 4 clear enable
3	FIFO_3	FIFO interrupt 3 clear enable
2	FIFO_2	FIFO interrupt 2 clear enable
1	FIFO_1	FIFO interrupt 1 clear enable
0	FIFO_0	FIFO interrupt 0 clear enable

Field Name	Value Symbol	Value Description	Hex Value
CVSD_DEC_OUT_RDY	NVIC_CVSD_DEC_OUT_RDY_INT_DISABLE	Disable the CVSD CODEC decoder output interface interrupt	0x1
CVSD_DEC_IN_RDY	NVIC_CVSD_DEC_IN_RDY_INT_DISABLE	Disable the CVSD CODEC decoder input interface interrupt	0x1
CVSD_ENC_OUT_RDY	NVIC_CVSD_ENC_OUT_RDY_INT_DISABLE	Disable the CVSD CODEC encoder output interface interrupt	0x1
CVSD_ENC_IN_RDY	NVIC_CVSD_ENC_IN_RDY_INT_DISABLE	Disable the CVSD CODEC encoder input interface interrupt	0x1
I2C	NVIC_I2C_INT_DISABLE	Disable the I^2C interrupt	0x1
PCM_ERROR	NVIC_PCM_ERROR_INT_DISABLE	Disable the PCM error interrupt	0x1
PCM_TX	NVIC_PCM_TX_INT_DISABLE	Disable the PCM transmit buffer empty interrupt	0x1
PCM_RX	NVIC_PCM_RX_INT_DISABLE	Disable the PCM receive buffer full interrupt	0x1
SPI_ERROR	NVIC_SPI_ERROR_INT_DISABLE	Disable the SPI error interrupt	0x1
SPI_TX	NVIC_SPI_TX_INT_DISABLE	Disable the SPI transmit buffer empty interrupt	0x1

Field Name	Value Symbol	Value Description	Hex Value
SPI_RX	NVIC_SPI_RX_INT_DISABLE	Disable the SPI receive buffer full interrupt	0x1
GPIO	NVIC_GPIO_INT_DISABLE	Disable the GPIO interrupt	0x1
DMA3	NVIC_DMA3_INT_DISABLE	Disable the DMA channel 3 interrupt	0x1
DMA2	NVIC_DMA2_INT_DISABLE	Disable the DMA channel 2 interrupt	0x1
DMA1	NVIC_DMA1_INT_DISABLE	Disable the DMA channel 1 interrupt	0x1
DMA0	NVIC_DMA0_INT_DISABLE	Disable the DMA channel 0 interrupt	0x1
CFX_7	NVIC_CFX_7_INT_DISABLE	Disable CFX event interrupt 7	0x1
CFX_6	NVIC_CFX_6_INT_DISABLE	Disable CFX event interrupt 6	0x1
CFX_5	NVIC_CFX_5_INT_DISABLE	Disable CFX event interrupt 5	0x1
CFX_4	NVIC_CFX_4_INT_DISABLE	Disable CFX event interrupt 4	0x1
CFX_3	NVIC_CFX_3_INT_DISABLE	Disable CFX event interrupt 3	0x1
CFX_2	NVIC_CFX_2_INT_DISABLE	Disable CFX event interrupt 2	0x1
CFX_1	NVIC_CFX_1_INT_DISABLE	Disable CFX event interrupt 1	0x1
CFX_0	NVIC_CFX_0_INT_DISABLE	Disable CFX event interrupt 0	0x1
FIFO_7	NVIC_FIFO_7_INT_DISABLE	Disable FIFO interrupt 7	0x1
FIFO_6	NVIC_FIFO_6_INT_DISABLE	Disable FIFO interrupt 6	0x1
FIFO_5	NVIC_FIFO_5_INT_DISABLE	Disable FIFO interrupt 5	0x1
FIFO_4	NVIC_FIFO_4_INT_DISABLE	Disable FIFO interrupt 4	0x1
FIFO_3	NVIC_FIFO_3_INT_DISABLE	Disable FIFO interrupt 3	0x1
FIFO_2	NVIC_FIFO_2_INT_DISABLE	Disable FIFO interrupt 2	0x1
FIFO_1	NVIC_FIFO_1_INT_DISABLE	Disable FIFO interrupt 1	0x1
FIFO_0	NVIC_FIFO_0_INT_DISABLE	Disable FIFO interrupt 0	0x1*

15.1.15.5 NVIC_ICER1

Bit Field	Field Name	Description
9	ACB_COM	ACB combined interrupt clear enable
8	ECB_COM	ECB combined interrupt clear enable
7	RS_DEC_OUT_RDY	RS decoder output registers interrupt clear enable
6	RS_DEC_IN_RDY	RS decoder input registers interrupt clear enable
5	RS_ENC_OUT_RDY	RS encoder output registers interrupt clear enable
4	RS_ENC_IN_RDY	RS encoder input registers interrupt clear enable
3	G722_DEC_OUT_RDY	G.722 CODEC decoder output interface registers interrupt clear enable
2	G722_DEC_IN_RDY	G.722 CODEC decoder input interface registers interrupt clear enable
1	G722_ENC_OUT_RDY	G.722 CODEC encoder output interface registers interrupt clear enable
0	G722_ENC_IN_RDY	G.722 CODEC encoder input interface registers interrupt clear enable

Field Name	Value Symbol	Value Description	Hex Value
ACB_COM	NVIC_ACB_COM_INT_DISABLE	Disable the ACB combined interrupt	0x1
ECB_COM	NVIC_ECB_COM_INT_DISABLE	Disable the ECB combined interrupt	0x1
RS_DEC_OUT_RDY	NVIC_RS_DEC_OUT_RDY_INT_DISABLE	Disable the RS decoder output registers interrupt	0x1
RS_DEC_IN_RDY	NVIC_RS_DEC_IN_RDY_INT_DISABLE	Disable the RS decoder input registers interrupt	0x1
RS_ENC_OUT_RDY	NVIC_RS_ENC_OUT_RDY_INT_DISABLE	Disable the RS encoder output registers interrupt	0x1
RS_ENC_IN_RDY	NVIC_RS_ENC_IN_RDY_INT_DISABLE	Disable the RS encoder input registers interrupt	0x1
G722_DEC_OUT_RDY	NVIC_G722_DEC_OUT_RDY_INT_DISABLE	Disable the G.722 CODEC decoder output interface registers interrupt	0x1
G722_DEC_IN_RDY	NVIC_G722_DEC_IN_RDY_INT_DISABLE	Disable the G.722 CODEC decoder input interface registers interrupt	0x1
G722_ENC_OUT_RDY	NVIC_G722_ENC_OUT_RDY_INT_DISABLE	Disable the G.722 CODEC encoder output interface registers interrupt	0x1
G722_ENC_IN_RDY	NVIC_G722_ENC_IN_RDY_INT_DISABLE	Disable the G.722 CODEC encoder input interface registers interrupt	0x1

15.1.15.6 NVIC_ISPR0

Bit Field	Field Name	Description
31	CVSD_DEC_OUT_RDY	CVSD CODEC decoder output interface interrupt set pending
30	CVSD_DEC_IN_RDY	CVSD CODEC decoder input interface interrupt set pending
29	CVSD_ENC_OUT_RDY	CVSD CODEC encoder output interface interrupt set pending
28	CVSD_ENC_IN_RDY	CVSD CODEC encoder input interface interrupt set pending
27	I2C	I^2C interrupt set pending
26	PCM_ERROR	PCM error interrupt set pending
25	PCM_TX	PCM transmit buffer empty interrupt set pending
24	PCM_RX	PCM receive buffer full interrupt set pending
23	SPI_ERROR	SPI error interrupt set pending
22	SPI_TX	SPI transmit buffer empty interrupt set pending
21	SPI_RX	SPI receive buffer full interrupt set pending
20	GPIO	GPIO interrupt set pending
19	DMA3	DMA channel 3 interrupt set pending
18	DMA2	DMA channel 2 interrupt set pending
17	DMA1	DMA channel 1 interrupt set pending
16	DMA0	DMA channel 0 interrupt set pending
15	CFX_7	CFX event interrupt 7 set pending
14	CFX_6	CFX event interrupt 6 set pending
13	CFX_5	CFX event interrupt 5 set pending
12	CFX_4	CFX event interrupt 4 set pending

Bit Field	Field Name	Description
11	CFX_3	CFX event interrupt 3 set pending
10	CFX_2	CFX event interrupt 2 set pending
9	CFX_1	CFX event interrupt 1 set pending
8	CFX_0	CFX event interrupt 0 set pending
7	FIFO_7	FIFO interrupt 7 set pending
6	FIFO_6	FIFO interrupt 6 set pending
5	FIFO_5	FIFO interrupt 5 set pending
4	FIFO_4	FIFO interrupt 4 set pending
3	FIFO_3	FIFO interrupt 3 set pending
2	FIFO_2	FIFO interrupt 2 set pending
1	FIFO_1	FIFO interrupt 1 set pending
0	FIFO_0	FIFO interrupt 0 set pending

Field Name	Value Symbol	Value Description	Hex Value
CVSD_DEC_OUT_RDY	NVIC_CVSD_DEC_OUT_RDY_SETPEND	Set the CVSD CODEC decoder output interface interrupt as pending	0x1
CVSD_DEC_IN_RDY	NVIC_CVSD_DEC_IN_RDY_SETPEND	Set the CVSD CODEC decoder input interface interrupt as pending	0x1
CVSD_ENC_OUT_RDY	NVIC_CVSD_ENC_OUT_RDY_SETPEND	Set the CVSD CODEC encoder output interface interrupt as pending	0x1
CVSD_ENC_IN_RDY	NVIC_CVSD_ENC_IN_RDY_SETPEND	Set the CVSD CODEC encoder input interface interrupt as pending	0x1
I2C	NVIC_I2C_SETPEND	Set the I^2C interrupt as pending	0x1
PCM_ERROR	NVIC_PCM_ERROR_SETPEND	Set the PCM error interrupt as pending	0x1
PCM_TX	NVIC_PCM_TX_SETPEND	Set the PCM transmit buffer empty interrupt as pending	0x1
PCM_RX	NVIC_PCM_RX_SETPEND	Set the PCM receive buffer full interrupt as pending	0x1
SPI_ERROR	NVIC_SPI_ERROR_SETPEND	Set the SPI error interrupt as pending	0x1
SPI_TX	NVIC_SPI_TX_SETPEND	Set the SPI transmit buffer empty interrupt as pending	0x1
SPI_RX	NVIC_SPI_RX_SETPEND	Set the SPI receive buffer full interrupt as pending	0x1
GPIO	NVIC_GPIO_SETPEND	Set the GPIO interrupt as pending	0x1
DMA3	NVIC_DMA3_SETPEND	Set the DMA channel 3 interrupt as pending	0x1
DMA2	NVIC_DMA2_SETPEND	Set the DMA channel 2 interrupt as pending	0x1
DMA1	NVIC_DMA1_SETPEND	Set the DMA channel 1 interrupt as pending	0x1
DMA0	NVIC_DMA0_SETPEND	Set the DMA channel 0 interrupt as pending	0x1
CFX_7	NVIC_CFX_7_SETPEND	Set the CFX event interrupt 7 as pending	0x1
CFX_6	NVIC_CFX_6_SETPEND	Set the CFX event interrupt 6 as pending	0x1
CFX_5	NVIC_CFX_5_SETPEND	Set the CFX event interrupt 5 as pending	0x1

Field Name	Value Symbol	Value Description	Hex Value
CFX_4	NVIC_CFX_4_SETPEND	Set the CFX event interrupt 4 as pending	0x1
CFX_3	NVIC_CFX_3_SETPEND	Set the CFX event interrupt 3 as pending	0x1
CFX_2	NVIC_CFX_2_SETPEND	Set the CFX event interrupt 2 as pending	0x1
CFX_1	NVIC_CFX_1_SETPEND	Set the CFX event interrupt 1 as pending	0x1
CFX_0	NVIC_CFX_0_SETPEND	Set the CFX event interrupt 0 as pending	0x1
FIFO_7	NVIC_FIFO_7_SETPEND	Set the FIFO interrupt 7 as pending	0x1
FIFO_6	NVIC_FIFO_6_SETPEND	Set the FIFO interrupt 6 as pending	0x1
FIFO_5	NVIC_FIFO_5_SETPEND	Set the FIFO interrupt 5 as pending	0x1
FIFO_4	NVIC_FIFO_4_SETPEND	Set the FIFO interrupt 4 as pending	0x1
FIFO_3	NVIC_FIFO_3_SETPEND	Set the FIFO interrupt 3 as pending	0x1
FIFO_2	NVIC_FIFO_2_SETPEND	Set the FIFO interrupt 2 as pending	0x1
FIFO_1	NVIC_FIFO_1_SETPEND	Set the FIFO interrupt 1 as pending	0x1
FIFO_0	NVIC_FIFO_0_SETPEND	Set the FIFO interrupt 0 as pending	0x1*

15.1.15.7 NVIC_ISPR1

Bit Field	Field Name	Description
9	ACB_COM	ACB combined interrupt set pending
8	ECB_COM	ECB combined interrupt set pending
7	RS_DEC_OUT_RDY	RS decoder output registers interrupt set pending
6	RS_DEC_IN_RDY	RS decoder input registers interrupt set pending
5	RS_ENC_OUT_RDY	RS encoder output registers interrupt set pending
4	RS_ENC_IN_RDY	RS encoder input registers interrupt set pending
3	G722_DEC_OUT_RDY	G.722 CODEC decoder output interface registers interrupt set pending
2	G722_DEC_IN_RDY	G.722 CODEC decoder input interface registers interrupt set pending
1	G722_ENC_OUT_RDY	G.722 CODEC encoder output interface registers interrupt set pending
0	G722_ENC_IN_RDY	G.722 CODEC encoder input interface registers interrupt set pending

Field Name	Value Symbol	Value Description	Hex Value
ACB_COM	NVIC_ACB_COM_SETPEND	Set the ACB combined interrupt as pending	0x1
ECB_COM	NVIC_ECB_COM_SETPEND	Set the ECB combined interrupt as pending	0x1
RS_DEC_OUT_RDY	NVIC_RS_DEC_OUT_RDY_SETPEND	Set the RS decoder output registers interrupt as pending	0x1
RS_DEC_IN_RDY	NVIC_RS_DEC_IN_RDY_SETPEND	Set the RS decoder input registers interrupt as pending	0x1
RS_ENC_OUT_RDY	NVIC_RS_ENC_OUT_RDY_SETPEND	Set the RS encoder output registers interrupt as pending	0x1
RS_ENC_IN_RDY	NVIC_RS_ENC_IN_RDY_SETPEND	Set the RS encoder input registers interrupt as pending	0x1

Field Name	Value Symbol	Value Description	Hex Value
G722_DEC_OUT_RDY	NVIC_G722_DEC_OUT_RDY_SETPEND	Set the G.722 CODEC decoder output interface registers interrupt as pending	0x1
G722_DEC_IN_RDY	NVIC_G722_DEC_IN_RDY_SETPEND	Set the G.722 CODEC decoder input interface registers interrupt as pending	0x1
G722_ENC_OUT_RDY	NVIC_G722_ENC_OUT_RDY_SETPEND	Set the G.722 CODEC encoder output interface registers interrupt as pending	0x1
G722_ENC_IN_RDY	NVIC_G722_ENC_IN_RDY_SETPEND	Set the G.722 CODEC encoder input interface registers interrupt as pending	0x1

15.1.15.8 NVIC_ICPR0

Bit Field	Field Name	Description
31	CVSD_DEC_OUT_RDY	CVSD CODEC decoder output interface interrupt clear pending
30	CVSD_DEC_IN_RDY	CVSD CODEC decoder input interface interrupt clear pending
29	CVSD_ENC_OUT_RDY	CVSD CODEC encoder output interface interrupt clear pending
28	CVSD_ENC_IN_RDY	CVSD CODEC encoder input interface interrupt clear pending
27	I2C	I^2C interrupt clear pending
26	PCM_ERROR	PCM error interrupt clear pending
25	PCM_TX	PCM transmit buffer empty interrupt clear pending
24	PCM_RX	PCM receive buffer full interrupt clear pending
23	SPI_ERROR	SPI error interrupt clear pending
22	SPI_TX	SPI transmit buffer empty interrupt clear pending
21	SPI_RX	SPI receive buffer full interrupt clear pending
20	GPIO	GPIO interrupt clear pending
19	DMA3	DMA channel 3 interrupt clear pending
18	DMA2	DMA channel 2 interrupt clear pending
17	DMA1	DMA channel 1 interrupt clear pending
16	DMA0	DMA channel 0 interrupt clear pending
15	CFX_7	CFX event interrupt 7 clear pending
14	CFX_6	CFX event interrupt 6 clear pending
13	CFX_5	CFX event interrupt 5 clear pending
12	CFX_4	CFX event interrupt 4 clear pending
11	CFX_3	CFX event interrupt 3 clear pending
10	CFX_2	CFX event interrupt 2 clear pending
9	CFX_1	CFX event interrupt 1 clear pending
8	CFX_0	CFX event interrupt 0 clear pending
7	FIFO_7	FIFO interrupt 7 clear pending
6	FIFO_6	FIFO interrupt 6 clear pending
5	FIFO_5	FIFO interrupt 5 clear pending
4	FIFO_4	FIFO interrupt 4 clear pending

Bit Field	Field Name	Description
3	FIFO_3	FIFO interrupt 3 clear pending
2	FIFO_2	FIFO interrupt 2 clear pending
1	FIFO_1	FIFO interrupt 1 clear pending
0	FIFO_0	FIFO interrupt 0 clear pending

Field Name	Value Symbol	Value Description	Hex Value
CVSD_DEC_OUT_RDY	NVIC_CVSD_DEC_OUT_RDY_CLRPEND	Clear pending CVSD CODEC decoder output interface interrupt	0x1
CVSD_DEC_IN_RDY	NVIC_CVSD_DEC_IN_RDY_CLRPEND	Clear pending CVSD CODEC decoder input interface interrupt	0x1
CVSD_ENC_OUT_RDY	NVIC_CVSD_ENC_OUT_RDY_CLRPEND	Clear pending CVSD CODEC encoder output interface interrupt	0x1
CVSD_ENC_IN_RDY	NVIC_CVSD_ENC_IN_RDY_CLRPEND	Clear pending CVSD CODEC encoder input interface interrupt	0x1
I2C	NVIC_I2C_CLRPEND	Clear pending I^2C interrupt	0x1
PCM_ERROR	NVIC_PCM_ERROR_CLRPEND	Clear pending PCM error interrupt	0x1
PCM_TX	NVIC_PCM_TX_CLRPEND	Clear pending PCM transmit buffer empty interrupt	0x1
PCM_RX	NVIC_PCM_RX_CLRPEND	Clear pending PCM receive buffer full interrupt	0x1
SPI_ERROR	NVIC_SPI_ERROR_CLRPEND	Clear pending SPI error interrupt	0x1
SPI_TX	NVIC_SPI_TX_CLRPEND	Clear pending SPI transmit buffer empty interrupt	0x1
SPI_RX	NVIC_SPI_RX_CLRPEND	Clear pending SPI receive buffer full interrupt	0x1
GPIO	NVIC_GPIO_CLRPEND	Clear pending GPIO interrupt	0x1
DMA3	NVIC_DMA3_CLRPEND	Clear pending DMA channel 3 interrupt	0x1
DMA2	NVIC_DMA2_CLRPEND	Clear pending DMA channel 2 interrupt	0x1
DMA1	NVIC_DMA1_CLRPEND	Clear pending DMA channel 1 interrupt	0x1
DMA0	NVIC_DMA0_CLRPEND	Clear pending DMA channel 0 interrupt	0x1
CFX_7	NVIC_CFX_7_CLRPEND	Clear pending CFX event interrupt 7	0x1
CFX_6	NVIC_CFX_6_CLRPEND	Clear pending CFX event interrupt 6	0x1
CFX_5	NVIC_CFX_5_CLRPEND	Clear pending CFX event interrupt 5	0x1
CFX_4	NVIC_CFX_4_CLRPEND	Clear pending CFX event interrupt 4	0x1
CFX_3	NVIC_CFX_3_CLRPEND	Clear pending CFX event interrupt 3	0x1
CFX_2	NVIC_CFX_2_CLRPEND	Clear pending CFX event interrupt 2	0x1
CFX_1	NVIC_CFX_1_CLRPEND	Clear pending CFX event interrupt 1	0x1
CFX_0	NVIC_CFX_0_CLRPEND	Clear pending CFX event interrupt 0	0x1
FIFO_7	NVIC_FIFO_7_CLRPEND	Clear pending FIFO interrupt 7	0x1
FIFO_6	NVIC_FIFO_6_CLRPEND	Clear pending FIFO interrupt 6	0x1
FIFO_5	NVIC_FIFO_5_CLRPEND	Clear pending FIFO interrupt 5	0x1

Field Name	Value Symbol	Value Description	Hex Value
FIFO_4	NVIC_FIFO_4_CLRPEND	Clear pending FIFO interrupt 4	0x1
FIFO_3	NVIC_FIFO_3_CLRPEND	Clear pending FIFO interrupt 3	0x1
FIFO_2	NVIC_FIFO_2_CLRPEND	Clear pending FIFO interrupt 2	0x1
FIFO_1	NVIC_FIFO_1_CLRPEND	Clear pending FIFO interrupt 1	0x1
FIFO_0	NVIC_FIFO_0_CLRPEND	Clear pending FIFO interrupt 0	0x1*

15.1.15.9 NVIC_ICPR1

Bit Field	Field Name	Description
9	ACB_COM	ACB combined interrupt clear pending
8	ECB_COM	ECB combined interrupt clear pending
7	RS_DEC_OUT_RDY	RS decoder output registers interrupt clear pending
6	RS_DEC_IN_RDY	RS decoder input registers interrupt clear pending
5	RS_ENC_OUT_RDY	RS encoder output registers interrupt clear pending
4	RS_ENC_IN_RDY	RS encoder input registers interrupt clear pending
3	G722_DEC_OUT_RDY	G.722 CODEC decoder output interface registers interrupt clear pending
2	G722_DEC_IN_RDY	G.722 CODEC decoder input interface registers interrupt clear pending
1	G722_ENC_OUT_RDY	G.722 CODEC encoder output interface registers interrupt clear pending
0	G722_ENC_IN_RDY	G.722 CODEC encoder input interface registers interrupt clear pending

Field Name	Value Symbol	Value Description	Hex Value
ACB_COM	NVIC_ACB_COM_CLRPEND	Clear pending ACB combined interrupt	0x1
ECB_COM	NVIC_ECB_COM_CLRPEND	Clear pending ECB combined interrupt	0x1
RS_DEC_OUT_RDY	NVIC_RS_DEC_OUT_RDY_CLRPEND	Clear pending RS decoder output registers interrupt	0x1
RS_DEC_IN_RDY	NVIC_RS_DEC_IN_RDY_CLRPEND	Clear pending RS decoder input registers interrupt	0x1
RS_ENC_OUT_RDY	NVIC_RS_ENC_OUT_RDY_CLRPEND	Clear pending RS encoder output registers interrupt	0x1
RS_ENC_IN_RDY	NVIC_RS_ENC_IN_RDY_CLRPEND	Clear pending RS encoder input registers interrupt	0x1
G722_DEC_OUT_RDY	NVIC_G722_DEC_OUT_RDY_CLRPEND	Clear pending G.722 CODEC decoder output interface registers interrupt	0x1
G722_DEC_IN_RDY	NVIC_G722_DEC_IN_RDY_CLRPEND	Clear pending G.722 CODEC decoder input interface registers interrupt	0x1
G722_ENC_OUT_RDY	NVIC_G722_ENC_OUT_RDY_CLRPEND	Clear pending G.722 CODEC encoder output interface registers interrupt	0x1
G722_ENC_IN_RDY	NVIC_G722_ENC_IN_RDY_CLRPEND	Clear pending G.722 CODEC encoder input interface registers interrupt	0x1

15.1.15.10 NVIC_IABR0

Bit Field	Field Name	Description
31	CVSD_DEC_OUT_RDY	CVSD CODEC decoder output interface interrupt active status
30	CVSD_DEC_IN_RDY	CVSD CODEC decoder input interface interrupt active status
29	CVSD_ENC_OUT_RDY	CVSD CODEC encoder output interface interrupt active status
28	CVSD_ENC_IN_RDY	CVSD CODEC encoder input interface interrupt active status
27	I2C	I^2C interrupt active status
26	PCM_ERROR	PCM error interrupt active status
25	PCM_TX	PCM transmit buffer empty interrupt active status
24	PCM_RX	PCM receive buffer full interrupt active status
23	SPI_ERROR	SPI error interrupt active status
22	SPI_TX	SPI transmit buffer empty interrupt active status
21	SPI_RX	SPI receive buffer full interrupt active status
20	GPIO	GPIO interrupt active status
19	DMA3	DMA channel 3 interrupt active status
18	DMA2	DMA channel 2 interrupt active status
17	DMA1	DMA channel 1 interrupt active status
16	DMA0	DMA channel 0 interrupt active status
15	CFX_7	CFX event interrupt 7 active status
14	CFX_6	CFX event interrupt 6 active status
13	CFX_5	CFX event interrupt 5 active status
12	CFX_4	CFX event interrupt 4 active status
11	CFX_3	CFX event interrupt 3 active status
10	CFX_2	CFX event interrupt 2 active status
9	CFX_1	CFX event interrupt 1 active status
8	CFX_0	CFX event interrupt 0 active status
7	FIFO_7	FIFO interrupt 7 active status
6	FIFO_6	FIFO interrupt 6 active status
5	FIFO_5	FIFO interrupt 5 active status
4	FIFO_4	FIFO interrupt 4 active status
3	FIFO_3	FIFO interrupt 3 active status
2	FIFO_2	FIFO interrupt 2 active status
1	FIFO_1	FIFO interrupt 1 active status
0	FIFO_0	FIFO interrupt 0 active status

Field Name	Value Symbol	Value Description	Hex Value
CVSD_DEC_OUT_RDY	NVIC_CVSD_DEC_OUT_RDY_ACTIVE	CVSD CODEC decoder output interface interrupt is active	0x1
CVSD_DEC_IN_RDY	NVIC_CVSD_DEC_IN_RDY_ACTIVE	CVSD CODEC decoder input interface interrupt is active	0x1

Field Name	Value Symbol	Value Description	Hex Value
CVSD_ENC_OUT_RDY	NVIC_CVSD_ENC_OUT_RDY_ACTIVE	CVSD CODEC encoder output interface interrupt is active	0x1
CVSD_ENC_IN_RDY	NVIC_CVSD_ENC_IN_RDY_ACTIVE	CVSD CODEC encoder input interface interrupt is active	0x1
I2C	NVIC_I2C_ACTIVE	I^2C interrupt is active	0x1
PCM_ERROR	NVIC_PCM_ERROR_ACTIVE	PCM error interrupt is active	0x1
PCM_TX	NVIC_PCM_TX_ACTIVE	PCM transmit buffer empty interrupt is active	0x1
PCM_RX	NVIC_PCM_RX_ACTIVE	PCM receive buffer full interrupt is active	0x1
SPI_ERROR	NVIC_SPI_ERROR_ACTIVE	SPI error interrupt is active	0x1
SPI_TX	NVIC_SPI_TX_ACTIVE	SPI transmit buffer empty interrupt is active	0x1
SPI_RX	NVIC_SPI_RX_ACTIVE	SPI receive buffer full interrupt is active	0x1
GPIO	NVIC_GPIO_ACTIVE	GPIO interrupt is active	0x1
DMA3	NVIC_DMA3_ACTIVE	DMA channel 3 interrupt is active	0x1
DMA2	NVIC_DMA2_ACTIVE	DMA channel 2 interrupt is active	0x1
DMA1	NVIC_DMA1_ACTIVE	DMA channel 1 interrupt is active	0x1
DMA0	NVIC_DMA0_ACTIVE	DMA channel 0 interrupt is active	0x1
CFX_7	NVIC_CFX_7_ACTIVE	CFX event interrupt 7 is active	0x1
CFX_6	NVIC_CFX_6_ACTIVE	CFX event interrupt 6 is active	0x1
CFX_5	NVIC_CFX_5_ACTIVE	CFX event interrupt 5 is active	0x1
CFX_4	NVIC_CFX_4_ACTIVE	CFX event interrupt 4 is active	0x1
CFX_3	NVIC_CFX_3_ACTIVE	CFX event interrupt 3 is active	0x1
CFX_2	NVIC_CFX_2_ACTIVE	CFX event interrupt 2 is active	0x1
CFX_1	NVIC_CFX_1_ACTIVE	CFX event interrupt 1 is active	0x1
CFX_0	NVIC_CFX_0_ACTIVE	CFX event interrupt 0is active	0x1
FIFO_7	NVIC_FIFO_7_ACTIVE	FIFO interrupt 7 is active	0x1
FIFO_6	NVIC_FIFO_6_ACTIVE	FIFO interrupt 6 is active	0x1
FIFO_5	NVIC_FIFO_5_ACTIVE	FIFO interrupt 5 is active	0x1
FIFO_4	NVIC_FIFO_4_ACTIVE	FIFO interrupt 4 is active	0x1
FIFO_3	NVIC_FIFO_3_ACTIVE	FIFO interrupt 3 is active	0x1
FIFO_2	NVIC_FIFO_2_ACTIVE	FIFO interrupt 2 is active	0x1
FIFO_1	NVIC_FIFO_1_ACTIVE	FIFO interrupt 1 is active	0x1
FIFO_0	NVIC_FIFO_0_ACTIVE	FIFO interrupt 0 is active	0x1*

15.1.15.11 NVIC_IABR1

Bit Field	Field Name	Description
9	ACB_COM	ACB combined interrupt active status
8	ECB_COM	ECB combined interrupt active status
7	RS_DEC_OUT_RDY	RS decoder output registers interrupt active status

Bit Field	Field Name	Description
6	RS_DEC_IN_RDY	RS decoder input registers interrupt active status
5	RS_ENC_OUT_RDY	RS encoder output registers interrupt active status
4	RS_ENC_IN_RDY	RS encoder input registers interrupt active status
3	G722_DEC_OUT_RDY	G.722 CODEC decoder output interface registers interrupt active status
2	G722_DEC_IN_RDY	G.722 CODEC decoder input interface registers interrupt active status
1	G722_ENC_OUT_RDY	G.722 CODEC encoder output interface registers interrupt active status
0	G722_ENC_IN_RDY	G.722 CODEC encoder input interface registers interrupt active status

Field Name	Value Symbol	Value Description	Hex Value
ACB_COM	NVIC_ACB_COM_ACTIVE	The ACB combined interrupt is active	0x1
ECB_COM	NVIC_ECB_COM_ACTIVE	The ECB combined interrupt is active	0x1
RS_DEC_OUT_RDY	NVIC_RS_DEC_OUT_RDY_ACTIVE	The RS decoder output registers interrupt is active	0x1
RS_DEC_IN_RDY	NVIC_RS_DEC_IN_RDY_ACTIVE	The RS decoder input registers interrupt is active	0x1
RS_ENC_OUT_RDY	NVIC_RS_ENC_OUT_RDY_ACTIVE	The RS encoder output registers interrupt is active	0x1
RS_ENC_IN_RDY	NVIC_RS_ENC_IN_RDY_ACTIVE	The RS encoder input registers interrupt is active	0x1
G722_DEC_OUT_RDY	NVIC_G722_DEC_OUT_RDY_ACTIVE	The G.722 CODEC decoder output interface registers interrupt is active	0x1
G722_DEC_IN_RDY	NVIC_G722_DEC_IN_RDY_ACTIVE	The G.722 CODEC decoder input interface registers interrupt is active	0x1
G722_ENC_OUT_RDY	NVIC_G722_ENC_OUT_RDY_ACTIVE	The G.722 CODEC encoder output interface registers interrupt is active	0x1
G722_ENC_IN_RDY	NVIC_G722_ENC_IN_RDY_ACTIVE	The G.722 CODEC encoder input interface registers interrupt is active	0x1

15.1.15.12 NVIC_IP0

Bit Field	Field Name	Description
31:24	FIFO_3_PRIORITY	Configure the FIFO interrupt 3 priority
23:16	FIFO_2_PRIORITY	Configure the FIFO interrupt 2 priority
15:8	FIFO_1_PRIORITY	Configure the FIFO interrupt 1 priority
7:0	FIFO_0_PRIORITY	Configure the FIFO interrupt 0 priority

15.1.15.13 NVIC_IP1

Bit Field	Field Name	Description
31:24	FIFO_7_PRIORITY	Configure the FIFO interrupt 7 priority
23:16	FIFO_6_PRIORITY	Configure the FIFO interrupt 6 priority
15:8	FIFO_5_PRIORITY	Configure the FIFO interrupt 5 priority
7:0	FIFO_4_PRIORITY	Configure the FIFO interrupt 4 priority

15.1.15.14 NVIC_IP2

Bit Field	Field Name	Description
31:24	CFX_3_PRIORITY	Configure the CFX event interrupt 3 priority
23:16	CFX_2_PRIORITY	Configure the CFX event interrupt 2 priority
15:8	CFX_1_PRIORITY	Configure the CFX event interrupt 1 priority
7:0	CFX_0_PRIORITY	Configure the CFX event interrupt 0 priority

15.1.15.15 NVIC_IP3

Bit Field	Field Name	Description
31:24	CFX_7_PRIORITY	Configure the CFX event interrupt 7 priority
23:16	CFX_6_PRIORITY	Configure the CFX event interrupt 6 priority
15:8	CFX_5_PRIORITY	Configure the CFX event interrupt 5 priority
7:0	CFX_4_PRIORITY	Configure the CFX event interrupt 4 priority

15.1.15.16 NVIC_IP4

Bit Field	Field Name	Description
31:24	DMA3_PRIORITY	Configure the DMA channel 3 interrupt priority
23:16	DMA2_PRIORITY	Configure the DMA channel 2 interrupt priority
15:8	DMA1_PRIORITY	Configure the DMA channel 1 interrupt priority
7:0	DMA0_PRIORITY	Configure the DMA channel 0 interrupt priority

15.1.15.17 NVIC_IP5

Bit Field	Field Name	Description
31:24	SPI_ERROR_PRIORITY	Configure the SPI error interrupt priority
23:16	SPI_TX_PRIORITY	Configure the SPI TX interrupt priority
15:8	SPI_RX_PRIORITY	Configure the SPI RX interrupt priority
7:0	GPIO_PRIORITY	Configure the GPIO interrupt priority

15.1.15.18 NVIC_IP6

Bit Field	Field Name	Description
31:24	I2C_PRIORITY	Configure the I^2C interrupt priority
23:16	PCM_ERROR_PRIORITY	Configure the PCM error interrupt priority
15:8	PCM_TX_PRIORITY	Configure the PCM TX interrupt priority
7:0	PCM_RX_PRIORITY	Configure the PCM RX interrupt priority

15.1.15.19 NVIC_IP7

Bit Field	Field Name	Description
31:24	CVSD_DEC_OUT_RDY_PRIORITY	Configure the CVSD decoder out ready interrupt priority
23:16	CVSD_DEC_IN_RDY_PRIORITY	Configure the CVSD decoder in ready interrupt priority
15:8	CVSD_ENC_OUT_RDY_PRIORITY	Configure the CVSD encoder out ready interrupt priority
7:0	CVSD_ENC_IN_RDY_PRIORITY	Configure the CVSD encoder in ready interrupt priority

15.1.15.20 NVIC_IP8

Bit Field	Field Name	Description
31:24	G722_DEC_OUT_RDY_PRIORITY	Configure the G.722 decoder out ready interrupt priority
23:16	G722_DEC_IN_RDY_PRIORITY	Configure the G.722 decoder in ready interrupt priority
15:8	G722_ENC_OUT_RDY_PRIORITY	Configure the G.722 encoder out ready interrupt priority
7:0	G722_ENC_IN_RDY_PRIORITY	Configure the G.722 encoder in ready interrupt priority

15.1.15.21 NVIC_IP9

Bit Field	Field Name	Description
31:24	RS_DEC_OUT_RDY_PRIORITY	Configure the Reed-Solomon decoder out ready interrupt priority
23:16	RS_DEC_IN_RDY_PRIORITY	Configure the Reed-Solomon decoder in ready interrupt priority
15:8	RS_ENC_OUT_RDY_PRIORITY	Configure the Reed-Solomon encoder out ready interrupt priority
7:0	RS_ENC_IN_RDY_PRIORITY	Configure the Reed-Solomon encoder in ready interrupt priority

15.1.15.22 NVIC_IP10

Bit Field	Field Name	Description
15:8	ACB_COM_PRIORITY	Configure the usage fault priority
7:0	ECB_COM_PRIORITY	Configure the ECB combined interrupt priority

15.1.15.23 SCB_CPUID

Bit Field	Field Name	Description
31:24	IMPLEMENTER	Implementer code
23:20	VARIANT	Implementation variant
19:16	ARCHITECTURE	Architecture number
15:4	PARTNO	Part number
3:0	REVISION	Revision code

15.1.15.24 SCB_ICSR

Bit Field	Field Name	Description
31	NMIPENDSET	Indicates the NMI exception is pending
28	PENDSVSET	Write 1 to set pending status for PendSV exception. Read indicates PendSV pending status
27	PENDSVCLR	Write 1 to clear PendSV exception pending status
26	PENDSTSET	Write 1 to set pending status for SYSTICK exception. Read value indicates SYSTICK pending status
25	PENDSTCLR	Write 1 to clear SYSTICK exception pending status
23	ISRPREEMPT	Indicates that a pending interrupt is going to be active in the next step
22	ISRPENDING	Indicates that an external interrupt is pending
21:12	VECTPENDING	Pending external interrupt number
11	RETTOBASE	Set when an exception handler is being run, and no other exception handlers have been pre-empted.
9:0	VECTACTIVE	Number of the current running interrupt service routine

Field Name	Value Symbol	Value Description	Hex Value
NMIPENDSET	NVIC_NMI_SETPEND	Set NMI exception to pending	0x1
PENDSVSET	NVIC_PENDSV_SETPEND	Set PendSV exception to pending	0x1
PENDSVCLR	NVIC_PENDSV_CLRPEND	Clear PendSV exception pending status	0x1
PENDSTSET	NVIC_SYSTICK_SETPEND	Set SYSTICK exception to pending	0x1
PENDSTCLR	NVIC_SYSTICK_CLRPEND	Clear SYSTICK exception pending status	0x1

15.1.15.25 SCB_VTOR

Bit Field	Field Name	Description
31:7	TBLOFF	Table offset value in code or RAM region. Must be a multiple of 128.

15.1.15.26 SCB_AIRCR

Bit Field	Field Name	Description
31:16	VECTKEY	Access key for writing this register. Must be set to 0x05FA to write the other register fields.
15	ENDIANESS	Indicates endianness for data.
10:8	PRIGROUP	Priority group setting. Controls how many bits in the priority register are used for pre-empty priority vs. sub-priority.
2	SYSRESETREQ	Requests a chip-level system reset
1	VECTCLRACTIVE	Clears all active exception state information
0	VECTRESET	Reset the ARM Cortex-M3 processor

Field Name	Value Symbol	Value Description	Hex Value
VECTKEY	NVIC_VECTKEY_UNLOCK	Unlocks the NVIC_APP_INT_CTRL register	0x5FA
	NVIC_VECTKEY_UNLOCK_READBACK	Readback value when reading the VECTKEY field after unlocking	0xFA05*
ENDIANESS	NVIC_LITTLE_ENDIAN	Indicates core is configured for little endian data	0x0*
PRIGROUP	NVIC_PRIGROUP_4_0	Indicates 4 bits for pre-emption priority and 0 bits for sub-priority	0x3
	NVIC_PRIGROUP_3_1	Indicates 3 bits for pre-emption priority and 1 bit for sub-priority	0x4
	NVIC_PRIGROUP_2_2	Indicates 2 bits for pre-emption priority and 2 bits for sub-priority	0x5
	NVIC_PRIGROUP_1_3	Indicates 1 bit for pre-emption priority and 3 bits for sub-priority	0x6
	NVIC_PRIGROUP_0_4	Indicates 0 bits for pre-emption priority and 4 bits for sub-priority	0x7*
SYSRESETREQ	NVIC_SYSRESET	Requests a chip-level system reset	0x1
VECTCLRACTIVE	NVIC_CLEAR_ACTIVE_VECTORS	Clears all active exception state information	0x1
VECTRESET	NVIC_VECTRESET	Reset the ARM Cortex-M3 processor	0x1

15.1.15.27 SCB_SCR

Bit Field	Field Name	Description
4	SEVONPEND	Set to 1 to cause the WFE to wake up if a new interrupt is pended
2	SLEEPDEEP	Enable SLEEPDEEP output signal when entering sleep mode
1	SLEEPONEXIT	Enable sleep on exit feature when returning from handler to thread mode

Field Name	Value Symbol	Value Description	Hex Value
SLEEPDEEP	NVIC_SLEEPDEEP_DISABLE	Disable SLEEPDEEP output signal when entering sleep mode	0x0*
	NVIC_SLEEPDEEP_ENABLE	Enable SLEEPDEEP output signal when entering sleep mode	0x1
SLEEPONEXIT	NVIC_SLEEPONEXIT_DISABLE	Disable sleep on exit feature	0x0*
	NVIC_SLEEPONEXIT_ENABLE	Enable sleep on exit feature	0x1

15.1.15.28 SCB_CCR

Bit Field	Field Name	Description
9	STKALIGN	Force stack to be double-word aligned
8	BFHFNMIGN	Ignore data bus faults in hard fault and NMI exceptions
4	DIV_0_TRP	Trap on divide by zero
3	UNALIGN_TRP	Trap on unaligned data access

Bit Field	Field Name	Description
1	USERSETMPEND	Allow user code to write to software trigger interrupt register
0	NONBASETHRDENA	Enable non-base thread mode

Field Name	Value Symbol	Value Description	Hex Value
STKALIGN	NVIC_STKALIGN_DISABLE	Disable double-word stack alignment	0x0*
	NVIC_STKALIGN_ENABLE	Enable double-word stack alignment	0x1
BFHFNMIGN	NVIC_BFHFNMIGN_DISABLE	Disable ignoring of data bus faults in hard fault and NMI exception handlers. Bus faults in these handlers will cause lock-up	0x0*
	NVIC_BFHFNMIGN_ENABLE	Enable ignoring of data bus faults in hard fault and NMI exception handlers	0x1
DIV_0_TRP	NVIC_DIV_0_TRP_DISABLE	Disable trapping on divide by zero	0x0*
	NVIC_DIV_0_TRP_ENABLE	Enable trapping on divide by zero	0x1
UNALIGN_TRP	NVIC_UNALIGN_TRP_DISABLE	Disable trapping on unaligned data access	0x0*
	NVIC_UNALIGN_TRP_ENABLE	Enable trapping on unaligned data access	0x1
USERSETMPEND	NVIC_USERSETMPEND_DISABLE	Disable user code access to software trigger interrupt register	0x0*
	NVIC_USERSETMPEND_ENABLE	Enable user code access to software trigger interrupt register	0x1
NONBASETHRDENA	NVIC_NONBASETHRDENA_DISABLE	Only allow thread mode on return from last exception	0x0*
	NVIC_NONBASETHRDENA_ENABLE	Allow thread mode from any handler level by controlled return value	0x1

15.1.15.29 SCB_SHP0

Bit Field	Field Name	Description
23:16	NVIC_USAGE_FAULT_PRIORITY	Configure the usage fault priority
15:8	NVIC_BUS_FAULT_PRIORITY	Configure the bus fault priority
7:0	NVIC_MEM_FAULT_PRIORITY	Configure the memory fault priority

15.1.15.30 SCB_SHP1

Bit Field	Field Name	Description
31:24	NVIC_SVC_PRIORITY	Configure the SVC priority

15.1.15.31 SCB_SHP2

Bit Field	Field Name	Description
31:24	NVIC_SYSTICK_PRIORITY	Configure the SysTick interrupt priority
23:16	NVIC_PENDSV_PRIORITY	Configure the PendSV priority
7:0	NVIC_MONITOR_PRIORITY	Configure the monitor priority

15.1.15.32 SCB_SHCSR

Bit Field	Field Name	Description
18	USGFAULTENA	Usage fault handler enable
17	BUSFAULTENA	Bus fault handler enable
16	MEMFAULTENA	Memory management fault handler enable
15	SVCALLPENDED	SVCall is pending or was started and replaced by a higher priority exception
14	BUSFAULTPENDED	Bus fault is pending or was started and replaced by a higher priority exception
13	MEMFAULTPENDED	Memory management fault is pending or was started and replaced by a higher priority exception
12	USGFAULTPENDED	Usage fault is pending or was started and replaced by a higher priority exception
11	SYSTICKACT	SYSTICK exception handler is active
10	PENDSVACT	PendSV exception handler is active
8	MONITORACT	Debug monitor exception handler is active
7	SVCALLACT	SVCall exception handler is active
3	USGFAULTACT	Usage fault exception handler is active
1	BUSFAULTACT	Bus fault exception handler is active
0	MEMFAULTACT	Memory management fault exception handler is active

Field Name	Value Symbol	Value Description	Hex Value
USGFAULTENA	NVIC_USGFAULT_DISABLE	Disable usage fault handler	0x0*
	NVIC_USGFAULT_ENABLE	Enable usage fault handler	0x1
BUSFAULTENA	NVIC_BUSFAULT_DISABLE	Disable bus fault handler	0x0*
	NVIC_BUSFAULT_ENABLE	Enable bus fault handler	0x1
MEMFAULTENA	NVIC_MEMFAULT_DISABLE	Disable memory management fault handler	0x0*
	NVIC_MEMFAULT_ENABLE	Enable memory management fault handler	0x1
SVCALLPENDED	NVIC_SVCALL_CLRPEND	SVCall handler not pending	0x0*
	NVIC_SVCALL_SETPEND	SVCall handler is pending	0x1
BUSFAULTPENDED	NVIC_BUSFAULT_CLRPEND	Bus fault handler not pending	0x0*
	NVIC_BUSFAULT_SETPEND	Bus fault handler is pending	0x1

Field Name	Value Symbol	Value Description	Hex Value
MEMFAULTPENDED	NVIC_MEMFAULT_CLRPEND	Memory management fault handler not pending	0x0*
	NVIC_MEMFAULT_SETPEND	Memory management fault handler is pending	0x1
USGFAULTPENDED	NVIC_USGFAULT_CLRPEND	Usage fault handler not pending	0x0*
	NVIC_USGFAULT_SETPEND	Usage fault handler is pending	0x1
SYSTICKACT	NVIC_SYSTICK_ACTIVE	SysTick exception handler is active	0x1
PENDSVACT	NVIC_PENDSV_ACTIVE	PendSV exception handler is active	0x1
MONITORACT	NVIC_MONITOR_ACTIVE	Debug monitor exception handler is active	0x1
SVCALLACT	NVIC_SVCALL_ACTIVE	SVCall exception is active	0x1
USGFAULTACT	NVIC_USGFAULT_ACTIVE	Usage fault exception handler is active	0x1
BUSFAULTACT	NVIC_BUSFAULT_ACTIVE	Bus fault exception handler is active	0x1
MEMFAULTACT	NVIC_MEMFAULT_ACTIVE	Memory management fault exception handler is active	0x1

15.1.15.33 SCB_CFSR

Bit Field	Field Name	Description
25	DIVBYZERO	Indicates divide by zero will take place
24	UNALIGNED	Indicates unaligned access will take place
19	NOCP	Attempt to execute a coprocessor instruction
18	INVPC	Attempt to do exception with bad value in EXC_RETURN number
17	INVSTATE	Attempt to switch to invalid (e.g. ARM) state
16	UNDEFINSTR	Attempt to execute an undefined instruction
15	BFARVALID	Indicates if bus fault address register is valid
12	STKERR	Indicates stacking error
11	UNSTKERR	Indicates unstacking error
10	IMPRECISERR	Indicates imprecise data access violation
9	PRECISERR	Indicates precise data access violation
8	IBUSERR	Indicates instruction access violation
7	MMARVALID	Indicates if memory management fault address register is valid
4	MSTKERR	Indicates stacking error
3	MUNSTKERR	Indicates unstacking error
1	DACCVIOL	Indicates data access violation
0	IACCVIOL	Indicates instruction access violation

15.1.15.34 SCB_HFSR

Bit Field	Field Name	Description
31	DEBUGEVT	Indicates hard fault is triggered by debug event
30	FORCED	Indicates hard fault is taken because of a lower priority (e.g., bus, memory management or usage) fault
1	VECTBL	Indicates hard fault is taken due to failed vector fetch

15.1.15.35 SCB_DFSR

Bit Field	Field Name	Description
4	EXTERNAL	Indicates external debug request signal asserted
3	VCATCH	Indicates vector fetch occurred
2	DWTTRAP	Indicates DWT match occurred
1	BKPT	Indicates BKPT instruction executed
0	HALTED	Indicates halt requested by NVIC

15.1.15.36 SCB_MMFAR

Bit Field	Field Name	Description
31:0	NVIC_MMAR	Memory monitor fault address register

15.1.15.37 SCB_BFAR

Bit Field	Field Name	Description
31:0	NVIC_BFAR	Bus fault address register

15.1.15.38 NVIC_STIR

Bit Field	Field Name	Description
7:0	INTID	The interrupt number to trigger

15.1.15.39 CoreDebug_DHCSR

Bit Field	Field Name	Description
31:16	DBGKEY	Debug key must be written to this field to write the rest of the register
25	S_RESET_ST	Core has been reset or is being rest. Bit is cleared on read.
24	S_RETIRE_ST	An instruction has completed since this bit was last cleared. Bit is cleared on read.
19	S_LOCKUP	Indicates if core is in lockup state
18	S_SLEEP	Indicates if core is in sleep mode
17	S_HALT	Indicates is core is halted
16	S_REGRDY	Indicates register read/write operation is completed
5	C_SNAPSTALL	Set to break a stalled memory access

Bit Field	Field Name	Description
3	C_MASKINTS	Mask interrupts while stepping
2	C_STEP	Single step the processor
1	C_HALT	Halt the processor
0	C_DEBUGEN	Enable halt mode debugging

Field Name	Value Symbol	Value Description	Hex Value
DBGKEY	DEBUG_HALT_KEY	DEBUG_HALT_KEY	0xA05F

15.1.15.40 CoreDebug_DCRSR

Bit Field	Field Name	Description
16	REGWnR	Indicates direction of register transfer
4:0	REGSEL	Indicates register to be accessed

Field Name	Value Symbol	Value Description	Hex Value
REGWnR	REGWNR_READ	Indicates register read	0x0*
	REGWNR_WRITE	Indicates register write	0x1
REGSEL	REGSEL_R0	Select R0	0x0*
	REGSEL_R1	Select R1	0x1
	REGSEL_R2	Select R2	0x2
	REGSEL_R3	Select R3	0x3
	REGSEL_R4	Select R4	0x4
	REGSEL_R5	Select R5	0x5
	REGSEL_R6	Select R6	0x6
	REGSEL_R7	Select R7	0x7
	REGSEL_R8	Select R8	0x8
	REGSEL_R9	Select R9	0x9
	REGSEL_R10	Select R10	0xA
	REGSEL_R11	Select R11	0xB
	REGSEL_R12	Select R12	0xC
	REGSEL_R13	Select R13	0xD
	REGSEL_R14	Select R14	0xE
	REGSEL_R15	Select R15	0xF
	REGSEL_PSRFLGS	Select xPSR/flags	0x10
	REGSEL_MSP	Select main stack pointer	0x11
	REGSEL_PSP	Select process stack pointer	0x12
	REGSEL_SPECREG	Access other registers including control, FAULTMASK, BASEPRI and PRIMASK	0x14

15.1.15.41 CoreDebug_DCRDR

Bit Field	Field Name	Description
31:0	DEBUG_REGDATA	Register read/write data for debugging

15.1.15.42 CoreDebug_DEMCR

Bit Field	Field Name	Description
24	TRCENA	Trace system enable
19	MON_REQ	Indicates that the debug monitor is caused by a manual pending request rather than a hardware event
18	MON_STEP	Single step the processor
17	MON_PEND	Pend the monitor exception request
16	MON_EN	Enable the debug monitor exception
10	VC_HARDERR	Debug trap on hard faults
9	VC_INTERR	Debug trap on interrupt service errors
8	VC_BUSERR	Debug trap on bus faults
7	VC_STATERR	Debug trap on usage fault state errors
6	VC_CHKERR	Debug trap on fault-enabled checking errors (e.g. unaligned access, divide by zero, etc.)
5	VC_NOCPERR	Debug trap on usage fault no coprocessor errors
4	VC_MMERR	Debug trap on memory management fault
0	VC_CORERESET	Debug trap on core reset

Field Name	Value Symbol	Value Description	Hex Value
MON_PEND	DEBUG_MON_CLRPEND	Clear the pending status of the debug monitor exception	0x0*
	DEBUG_MON_SETPEND	Set pending status of the debug monitor exception	0x1
MON_EN	DEBUG_MON_DISABLE	Disable debug monitor exceptions	0x0*
	DEBUG_MON_ENABLE	Enable debug monitor exceptions	0x1

15.2 SysTick

The ARM Cortex-M3 core peripherals include the system tick (SysTick) count-down timer from the ARM Cortex-M3 processor implementation. This block is implemented as part of the NVIC, and is described in the *Cortex-M3 Technical Reference Manual*.

The clock used by the SysTick timer is selected using the SysTick_CTRL_CLKSOURCE bit from the SysTick_CTRL register. This timer can be clocked from the ARM Cortex-M3 core clock or from the SysTick-specific reference clock (STCLK) that is divided from SLOWCLK by 32. For more information about CM3CLK, see Section 18.2.5, “ARM Cortex-M3 Processor Clock (CM3CLK)” on page 448. The SysTick_CALIB register is configured to define a 10 ms timer period based on STCLK.

The delay provided by the SysTick timer is defined using the selected clock and a reload value loaded to the SysTick_LOAD register, as follows:

$$\text{DELAY} = \frac{(\text{SysTick_LOAD} + 1)}{f_{\text{CM3CLK or STCLK}}}$$

The current value of the SysTick counter can be read at any time from the SysTick_VAL register.

The SysTick timer is enabled by setting the SysTick_CTRL_ENABLE bit in the SysTick_CTRL register. SysTick interrupts are enabled by setting the SysTick_CTRL_TICKINT bit in the SysTick_CTRL register. The SysTick_CTRL_COUNTFLAG bit in the SysTick_CTRL register indicates if the SysTick timer has reached zero since the last time this register was read and is cleared automatically after being read. This bit can be used if an application uses polling instead of interrupt to monitor for SysTick timer events

15.2.1 Control and Configuration Registers

Register Name	Register Description	Address
SysTick_CTRL	SYSTICK Control and Status Register	0xE000E010
SysTick_LOAD	SYSTICK Reload Value Register	0xE000E014
SysTick_VAL	SYSTICK Current Value Register	0xE000E018
SysTick_CALIB	SYSTICK Calibration Register	0xE000E01C

IMPORTANT: The SysTick timer is a standard component provided with the ARM Cortex-M3 processor. The registers for this peripheral are defined in *core_cm3.h* and augmented by defines for the bit fields, bit settings and subregisters in *sk4_hw.h*. These registers are only accessible through the ARM Cortex-M3's private peripheral bus (see Section 7.2.2, "Memory Bus Structures" on page 76), and are not accessible by the CFX DSP.

15.2.1.1 SysTick_CTRL Settings

Bit Field	Field Name	Description
16	COUNTFLAG	Returns 1 if the timer counted to 0 since the last time this bit was read. Cleared when read and when the SysTick current count register is modified.
2	CLKSOURCE	Select between using the SysTick reference clock (STCLK) and the ARM Cortex-M3 core clock (CM3_CLK) as the source for the SysTick timer.
1	TICKINT	Enable or disable the SysTick interrupt handler
0	ENABLE	Enable or disable the SysTick

Field Name	Value Symbol	Value Description	Hex Value
COUNTFLAG	SYSTICK_COUNTFLAG_NOT_ZERO	SYSTICK counter has not reached zero since last read	0x0*
	SYSTICK_COUNTFLAG_ZERO	SYSTICK counter has reached zero since last read	0x1
CLKSOURCE	SYSTICK_CLKSOURCE_EXTREF_CLK	Use external reference clock (STCLK)	0x0*
	SYSTICK_CLKSOURCE_CORE_CLK	Use the core clock	0x1

Field Name	Value Symbol	Value Description	Hex Value
TICKINT	SYSTICK_TICKINT_DISABLE	Disable interrupt generation when SYSTICK timer reaches 0	0x0*
	SYSTICK_TICKINT_ENABLE	Enable interrupt generation when SYSTICK timer reaches 0	0x1
ENABLE	SYSTICK_DISABLE	Disable SYSTICK timer	0x0*
	SYSTICK_ENABLE	Enable SYSTICK Timer	0x1

15.2.1.2 SysTick_LOAD Settings

Bit Field	Field Name	Description
23:0	RELOAD	Counter reload value for the SYSTICK timer when it reaches 0

15.2.1.3 SysTick_VAL Settings

Bit Field	Field Name	Description
23:0	CURRENT	Current value of the SYSTICK counter value. Write to clear counter.

15.2.1.4 SysTick_CALIB Settings

Bits	Field	Value	Description
31	NOREF	0	Indicates if a reference clock is available
30	SKEW	0	Indicates if calibration value is exactly 10 ms or not
23:0	TENMS	0x2710	SYSTICK counter calibration value for 10 ms. A value of 0 means the calibration value is not available

Field Name	Value Symbol	Value Description	Hex Value
NOREF	SYSTICK_NOREF	No external reference clock available	0x1
	SYSTICK_REF	External reference clock available	0x0*
SKEW	SYSTICK_SKEW	Calibration value is not exactly 10 ms	0x1
	SYSTICK_NOSKEW	Calibration value is exactly 10 ms	0x0*

15.3 Error-Correction Coding Blocks

The Ezairo 7100 system provides the ARM Cortex-M3 processor with hardware accelerators that support two different error correction coding standards to improve the bit-error rates of wireless transmissions.

- The Hamming and Interleaving block produces and decodes sets of Hamming encoded data blocks that can optionally be interleaved. Each Hamming block encodes a block of 16 data bits into 21 encoded bits by adding 5 parity bits. This block can be decoded later by the Hamming decoder, which can correct one-bit errors and detect two-bit errors in each block. Six of these data blocks are collected into a larger 126-bit data block (packed into a 128-bit block with two user-defined bits). These six blocks can be interleaved to support the correction of short error bursts that affect up to six adjacent bits of a transmission by spreading the errors across the underlying blocks.

For more information about Hamming and Interleaving, see Section 15.3.1, “Hamming and Interleaving Encoder” and Section 15.3.2, “Hamming and Interleaving Decoder”.

- The Reed-Solomon block produces and decodes sets of Reed-Solomon encoded data blocks. Each data block consists of 11 four-bit input data symbols, and 4 four-bit parity symbols. The Reed-Solomon decoder can detect and correct errors in any one symbol for each 15-symbol block. For more information about Reed-Solomon encoding and decoding, see Section 15.3.3, “Reed-Solomon Encoder” and Section 15.3.4, “Reed-Solomon Decoder” respectively.

The error-correction coding blocks are supported by the ECB_STATUS interrupt. This interrupt is used to indicate that:

- Errors have occurred when processing data using the Hamming and Interleaving block.
- Errors have occurred when processing data using the Reed-Solomon block.
- An overall encoding sequence that uses the Reed-Solomon block completed.

To configure the interrupt sources that can trigger an ECB status interrupt, enable the desired interrupts using the C_ECB_INT_ENABLE register. To read the status of these interrupt sources, or to clear the cause of a pending ECB status interrupt, use the C_ECB_INT_STATUS register.

15.3.1 Hamming and Interleaving Encoder

The Hamming encoder block implements error correction encoding on 96-bits of data using a Hamming (21,16) encoding scheme. Following Hamming encoding, the output blocks can be interleaved to space out bit errors between the blocks. To enable the Hamming and interleaving encoder, set the ECB_CTRL_HMI_ENC_EBL bit from the C_ECB_CTRL register.

Each of the six Hamming (21, 16) encoders produces five parity bits for each 16-bit data word provided. The entire 96-bit block of data is encoded in parallel. For a 16-bit data input, the parity is generated using the following equations where d15 is the most significant data bit, d0 is the least significant data bit, and p* are the parity bits.

$$p0 = d0 \oplus d1 \oplus d3 \oplus d4 \oplus d6 \oplus d8 \oplus d10 \oplus d11 \oplus d13 \oplus d15$$

$$p1 = d0 \oplus d2 \oplus d3 \oplus d5 \oplus d6 \oplus d9 \oplus d10 \oplus d12 \oplus d13$$

$$p2 = d1 \oplus d2 \oplus d3 \oplus d7 \oplus d8 \oplus d9 \oplus d10 \oplus d14 \oplus d15$$

$$p3 = d4 \oplus d5 \oplus d6 \oplus d7 \oplus d8 \oplus d9 \oplus d10$$

$$p4 = d11 \oplus d12 \oplus d13 \oplus d14 \oplus d15$$

The five parity bits will be appended to the end of the 16-bit data word to create a new 21-bit word.

The input to the Hamming encoder is 128-bits and is arranged as shown in Figure 27, spread over four 32-bit registers. The data to be protected by Hamming encoding is provided through the C_ECB_HMI_IN3, C_ECB_HMI_IN2, and C_ECB_HMI_IN1 registers. The input parity bits and extra bits to round to a 128-bit block are provided in C_ECB_HMI_IN0. If parity bits are generated by the encoder, they replace the parity bits provided through this register. The extra bits are available for application-defined use in both 126-bit and 128-bit modes (as described below), and are not supported by any underlying parity checking.

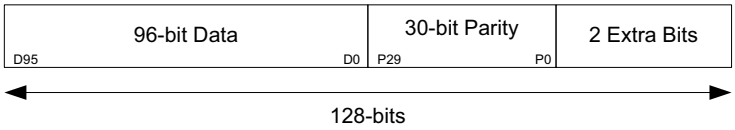


Figure 27. Hamming Encoder Input Arrangement

The 96-bit input data consists of six 16-bit data blocks. The 30-bit parity data consists of the corresponding six 5-bit parity blocks. When providing data to the Hamming encoder (either directly or through the DMA), the input can be configured to only require the data portion of the input (i.e., the C_ECB_HMI_IN3, C_ECB_HMI_IN2, and C_ECB_HMI_IN1 registers) by using the HMI_DMA_3REG bit setting for the ECB_CTRL_HMI_DMA_MODE bit from the C_ECB_CTRL register.

NOTE: The parity calculation of the Hamming encoder can be bypassed using the ARM Cortex-M3 processor to directly define the data to be stored in the parity bits. If the Hamming encoder parity generation is enabled through the ECB_CTRL_HMI_PARITY_EBL bit field in the C_ECB_CTRL register, the 30-bit parity section shown in Figure 27 is not utilized; otherwise, these parity bits are used in place of calculated parity bits for the Hamming and Interleaving block encoder. In both modes, interleaving of data using the encoder is possible.

If the ECB_CTRL_HMI_DMA_MODE bit is using the HMI_DMA_4REG setting, all four registers must be provided for the output signal to be considered ready.

NOTE: Use of the four-register mode is only required if the user application is only using the interleaver (and not the Hamming encoder), or if the two user-defined bits must be updated for each Hamming encoded block.

The format at the output of the Hamming encoder is given in Figure 28.

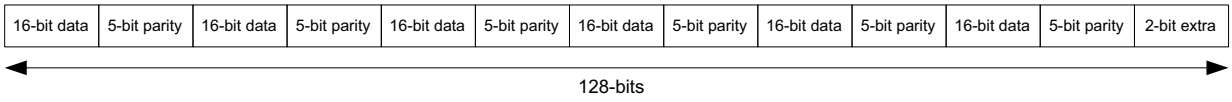


Figure 28. Hamming Encoder Output Arrangement

Following the Hamming encoder, the interleave encoder adds burst error protection by spreading out the effect of a burst error among several Hamming blocks. The interleaving can be enabled or disabled using the ECB_CTRL_HMI_INTERLEAVE_EBL bit from the C_ECB_CTRL register. If interleaving is disabled, the output of the Hamming and Interleaving encoder block is the output of the Hamming encoder (shown in Figure 28).

The interleave encoder supports two modes, which are set by the ECB_CTRL_HMI_INTERLEAVE_MODE field of the C_ECB_CTRL register. In the 126-bit mode, the last two least significant bits are directly driven to the output, and the remaining 126 are interleaved with a degree of 6. In the 128-bit mode, all of the 128-bits are interleaved with a degree of 8.

The 126-bit interleaving interleaves the data from each Hamming encoded (21, 16) block, breaking the input into sections as shown in Figure 29.

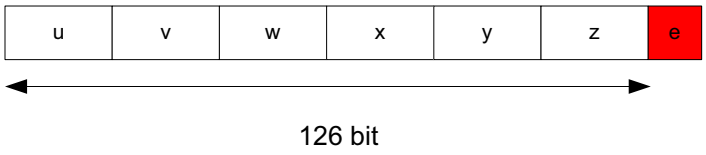


Figure 29. 126-Bit Interleaving Input Data Grouping

The interleaving in 126-bit mode results in the following output arrangement:

```
u20,v20,w20,x20,y20,z20,u19,v19,w19,x19,z19...u0,v0,w0,x0,y0,z0,e1,e0
```

The extra two LSBs are simply added to the interleaved vector. This results in an interleaving of degree 6, separating each bit of a block with five bits from other blocks.

The 128-bit interleaving is similar to 126 interleaving, but includes the two least significant bits. The 128 bits are divided into 8 blocks of 16 bits, as shown in Figure 30.

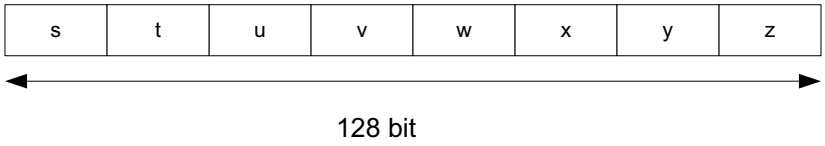


Figure 30. 128-Bit Interleaving Input Data Grouping

The interleaving in 128-bit mode results in the following output arrangement:

```
s15,t15,u15,v15,w15,x15,y15,z15...s0,t0,u0,v0,w0,x0,y0,z0
```

This results in an interleaving of degree 8, separating each bit of a block with seven bits of other blocks.

IMPORTANT: If using Hamming encoding, the 126-bit interleave mode optimally separates the data bits in each Hamming (21, 16) block. Because of this, use of 126-bit interleave mode over 128-bit interleave mode is strongly recommended when using Hamming encoding.

If any of the active input registers are overwritten before a previous output has been read, an `HMI_ENC_IN_ERR` interrupt is generated. Similarly if any of the output content is read while the input registers are only partially updated, an `HMI_ENC_OUT_ERR` interrupt is generated. To enable these interrupts, set the `ECB_INT_ENABLE_HMI_ENC_IN_ERR` and `ECB_INT_ENABLE_HMI_ENC_OUT_ERR` bits in the `C_ECB_INT_ENABLE` register.

IMPORTANT: If the accelerator is disabled after writing some, but not all, of the input registers, all of the input registers must be updated again when the encoder is re-enabled to avoid an error interrupt and to set the output ready signal.

15.3.2 Hamming and Interleaving Decoder

The interleaving decoder takes input interleaved data and de-interleaves it into blocks that can be used by the Hamming decoder. The Hamming decoder block then checks the data, and performs an error correction check on the encoded data using the Hamming (21,16) encoding scheme. To enable the Hamming and interleaving decoder, set the ECB_CTRL_IHM_DEC_EBL bit from the C_ECB_CTRL register.

The interleave decoder reassembles the received interleaved data before performing error detection and correction. De-interleaving can be done on all 128 bits or 126 bits. If the ECB_CTRL_IHM_INTERLEAVE_EBL bit field in the C_ECB_CTRL register is set, then the data input is de-interleaved and transferred to the Hamming decoder. Otherwise, the input data is directly transferred to the Hamming decoder. The interleave decoder supports two modes of operation through the ECB_CTRL_IHM_INTERLEAVE_MODE bit field from the C_ECB_CTRL register:

- In the 126-bit mode, the two LSBs are directly fed through to the output and the remaining 126 are de-interleaved at the 6th degree. This restores the 126-bit interleaving input data grouping shown in Figure 29.
- In the 128-bit mode all of the input content is de-interleaved at the 8th degree. This restores the 128-bit interleaving input data grouping shown in Figure 30.

Following the interleave decoder, the Hamming decoder block uses Hamming error detection and correction on the six Hamming (21, 16) blocks that form the 126-bits of encoded data provided to the Hamming and interleaving decoder. It re-calculates the 5 parity bits for each 16-bit data block, and compares these generated parity bits with the transmitted parity bits.

For each block, the Hamming decoder can be configured in one of two modes. In these modes, the Hamming decoder can:

- Detect, but *not* correct, up to two errors in the 16-bit data and 5 parity bits. When configured in this mode, the output of the error detection circuit is undefined in the presence of more than two bit errors in each 21-bit encoded data block.
- Detect and correct up to one error in the 16-bit data and 5 parity bits. The output of the error correction circuit is undefined in the presence of more than one bit error over each 21-bit encoded data block.

The Hamming decoder input format is shown in Figure 28 on page 282. If the Hamming decoder should not correct errors, clear the ECB_CTRL_IHM_COR_EBL bit from the C_ECB_CTRL register. In this case, the input data is reformatted and sent out as in Figure 27 on page 282 without any alteration to the data section. If the error correction is enabled, the Hamming decoder tries to detect errors, and will correct up to one error in each encoded block.

Output data from the Hamming decoder (or the interleaver if the Hamming decoder is disabled), is stored to the C_ECB_IHM_OUT1, C_ECB_IHM_OUT2 and C_ECB_IHM_OUT3 registers. The C_ECB_IHM_OUT0 register additionally outputs the 30 parity and 2 extra bits. If a user application only uses the data portion of the output from the Interleave and Hamming decoder, the output can be configured to only provide the data portion of the decoded signal by using the IHM_DMA_3REG bit setting for the ECB_CTRL_IHM_DMA_MODE bit from the C_ECB_CTRL register. If using the IHM_DMA_4REG setting instead, all four registers are provided and must be read to avoid output errors.

If any of the active input registers are overwritten before outputs of the previous block are read, an IHM_DEC_IN_ERR interrupt is generated. Similarly, if any of the active output registers are read while the input registers are only partially updated, an IHM_DEC_OUT_ERR interrupt is generated. To enable these interrupts, set the ECB_INT_ENABLE_HMI_DEC_IN_ERR and ECB_INT_ENABLE_HMI_DEC_OUT_ERR bits in the C_ECB_INT_ENABLE register.

The decoder captures the error detection and correction status in the C_ECB_STATUS register. The block error status is captured when the output is ready. It holds two status bits for each Hamming decoder block. If an error is detected but not corrected in the block, the corresponding ECB_STATUS_IHM_EDSR_BLK* status bit is set. If an error is detected and corrected in the block, the corresponding ECB_STATUS_IHM_EDCSR_BLK* status bit is set. When this status bit is set, the bit index that was corrected is captured in the appropriate ECB_IHM_ECP_BLK* bit field in the C_ECB_IHM_ECP register. If there is any correction in the data portion of the block, the pointer value will be between 0 and 15. If there is any correction in the parity portion of the block, the pointer value will be between 16 and 20.

NOTE: If the Hamming block cannot correct an error, the pointers are kept at zero. Therefore the pointer values cannot exceed 20.

IMPORTANT: If the accelerator is disabled after writing some, but not all, of the input registers, all of the input registers must be updated again when the encoder is re-enabled to avoid an error interrupt and to set the output ready signal.

15.3.3 Reed-Solomon Encoder

The Reed-Solomon encoder implements a [15,11] Reed-Solomon code, which creates encoded data blocks consisting of 15 four-bit symbols from 11 data symbols and 4 code symbols. This data block contains 60 bits, of which 44 bits are data and 16 bits are code.

The encoder defines the code symbols using the input data passed through a set of four generator polynomials. Each polynomial is a separate Galois field polynomial that calculates one code symbol when the 11 data symbols are shifted through the polynomial calculator. As shown in the timing diagram of the Reed-Solomon encoder core (Figure 31), it receives one input symbol per clock cycle and provides one output symbol on the following clock cycle. The effective delay through the encoder core itself is one ARM Cortex-M3 core clock cycle. A new block of raw data is accepted only after the previous block has been encoded and read by the ARM Cortex-M3 processor.

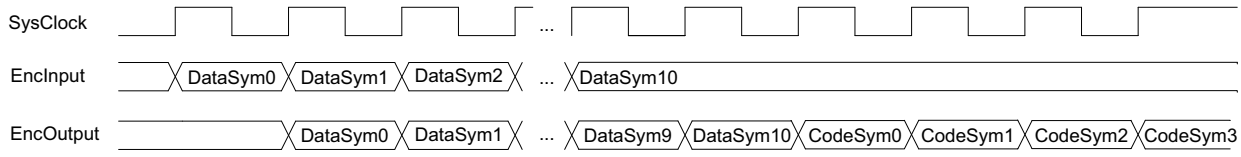


Figure 31. Timing Diagram of Reed-Solomon Core Encoder

The Reed-Solomon encoder block has a programmable countdown-counter, which can be set or read from the ECB_RS_BLK_CNT_RS_ENC_BLK_CNT bit field in the C_ECB_RS_BLK_CNT register. It sets the number of Reed-Solomon blocks encoded in a block set. When this counter is zero, the Reed-Solomon encoder is disabled. The countdown considers a block complete only when all of the 15 encoded data symbols are sent to the output interface. When it reaches zero, an ECB_STATUS interrupt is generated indicating an RS_ENC_BLK_END event has occurred. If operating in DMA mode, the Reed-Solomon encoder input interface does not generate any new requests until the block counter is updated with a non-zero value.

IMPORTANT: To enable the Reed-Solomon encoder to continuously encode a stream of data blocks, set the block count to 0xFF. If configured in this way, the Reed-Solomon block counter is not decremented when blocks are encoded.

Input and output interfaces are included with the Reed-Solomon encoder to support smooth data transfers between the ARM Cortex-M3 processor and the encoder.

The input interface to the encoder uses the C_ECB_RS_ENC_IN register. The input interface utilizes a circular pointer advancing four bits at a time to split the input register into symbols. The input interface block consists of the least significant 1, 2, 4 or 8 four-bit symbols (4, 8, 16 or 32 bits) from the input interface register. To select how many symbols are used as inputs to the Reed-Solomon encoder for each write to the C_ECB_RS_ENC_IN register, configure the ECB_CTRL_RS_ENC_IN_PTR_MODE bit field of the C_ECB_CTRL register. The register content is transferred into the encoder one symbol at a time, using the interface pointer. This pointer can be configured to operate in increment (RS_ENC_IN_INC) or decrement (RS_ENC_IN_DEC) mode using the ECB_CTRL_RS_ENC_IN_PTR_DIR bit from the C_ECB_CTRL register.

- In increment mode, transfers are from the symbol in the least significant position to the symbol in the most significant position.
- In decrement mode, transfers are from the symbol in the most significant position to the symbol in the least significant position.

Figure 32 shows the order in which the pointer selects symbols to be used by the encoder from the input interface for each input pointer mode.

The current pointer value can be read from the ECB_RS_PTR_RS_ENC_IN_PTR bit field in the C_ECB_RS_PTR register at any time, but cannot be overwritten.

	32-bit, Inc	16-bit, Inc	8-bit, Inc	4-bit	32-bit, Dec	16-bit, Dec	8-bit, Dec
Bit 28	Symbol 7				Symbol 0		
	Symbol 6				Symbol 1		
	Symbol 5				Symbol 2		
	Symbol 4				Symbol 3		
	Symbol 3	Symbol 3			Symbol 4	Symbol 0	
	Symbol 2	Symbol 2			Symbol 5	Symbol 1	
	Symbol 1	Symbol 1	Symbol 1		Symbol 6	Symbol 2	Symbol 0
Bit 0	Symbol 0	Symbol 0	Symbol 0	Symbol 0	Symbol 7	Symbol 3	Symbol 1

Figure 32. Input/Output Pointer Modes

When the input interface pointer is at the zero address and no update on the C_ECB_RS_ENC_IN register is detected, an RS_ENC_IN_RDY interrupt is generated, indicating that the register is empty and ready to be refilled. The input interface will not transfer any new symbols to the RS encoder core until the interface register is refilled. If the input interface register is overwritten before all encoded symbols are read out, an ECB_STATUS interrupt is generated that indicates an RS_ENC_IN_ERR event has occurred.

The output interface from the encoder uses the C_ECB_RS_ENC_OUT register. The output interface also uses a circular pointer advancing four bits at a time to split the output register into symbols in a separately configurable way from the input. Similar to the input interface pointer, the output interface pointer:

- Can be configured for 4-, 8-, 16-, or 32-bit (1, 2, 4, or 8 symbol) mode using the ECB_CTRL_RS_ENC_OUT_PTR_MODE bit field of the C_ECB_CTRL register

- Can be configured for increment or decrement mode using the ECB_CTRL_RS_ENC_OUT_PTR_DIR bit from the C_ECB_CTRL register
- Can be read (but not written) using the ECB_RS_PTR_RS_ENC_OUT_PTR bit field from the C_ECB_RS_PTR register
- Figure 32 shows the order in which the pointer places symbols to be generated by the encoder in the output interface for all pointer modes

The output register C_ECB_RS_ENC_OUT is considered to be full when the next output pointer value will be zero. This generates the RS_ENC_OUT_RDY interrupt, indicating that the output register is ready to be read. If the output register is read before the pointer wraps back to zero, the error interrupt will not be set to allow for partial output interface reads. A partial read can be necessary to complete the transfer of the last symbols of a set of encoded data blocks that do not form a multiple of the word size.

CAUTION: Partial reads must only be utilized when the block counter is zero and left over symbols are in the output register. If the output register is read when the encoder is still processing data, a symbol can be missed due to a race condition.

NOTE: The RS encoder does not have any internal data buffer. Therefore, to read any new symbol, it has to make sure that the output interface is ready to capture a new symbol. The RS encoder core will provide an acknowledgement to the input interface indicating that the new symbol is being read. This acknowledgement increments the input pointer. The RS encoder also provides a similar signal to the output interface so that the new symbol is captured and the output pointer is incremented.

To reset the Reed-Solomon encoder's internal state machines, set the ECB_CMD_RS_ENC_CLR bit field from the C_ECB_CMD register. The encoder can be reset even when disabled.

15.3.3.1 Using the Reed-Solomon Encoder with Interrupts

To use the Reed-Solomon encoder with interrupts, use the NVIC to enable the RS_ENC_IN_RDY and RS_ENC_OUT_RDY interrupts.

Once the encoder input and output pointers are configured, do the following to encode one set of blocks:

1. Reset the Reed-Solomon encoder
2. Enable the encoder by setting the block count to the desired number of blocks.

When interrupts are raised, perform the appropriate action:

- When an RS_ENC_IN_RDY interrupt is raised, write one word to the C_ECB_RS_ENC_IN register. The number of symbols written should match the input pointer mode because additional symbols will be ignored.
- When an RS_ENC_OUT_RDY interrupt is raised, first read the C_ECB_INT_STATUS register to check the ECB_INT_STATUS_RS_ENC_BLK_END status bit, which indicates whether the data available in the C_ECB_RS_ENC_OUT register are the last symbols of the block.
  - If the ECB_INT_STATUS_RS_ENC_BLK_END status bit is set, clear it, then read the last set of symbols from the C_ECB_RS_ENC_OUT register.

    To verify how many symbols are valid in the data read from the C_ECB_RS_ENC_OUT register, read the ECB_RS_PTR_RS_ENC_OUT_PTR bit field from the C_ECB_RS_PTR register, which indicates where the next symbol would have been written.

- If the RS_ENC_BLK_END status bit is not set, read one word from the C_ECB_RS_ENC_OUT register. This register contains the number of symbols specified by the output pointer mode.

For information on configuring the NVIC, see Section 15.1, “Nested Vectored Interrupt Controller (NVIC)”.

15.3.3.2 Using the Reed-Solomon Encoder with the DMA Controller

To use the Reed-Solomon encoder with the DMA controller for input or output, use the NVIC to disable the appropriate RS_ENC_IN_RDY or RS_ENC_OUT_RDY interrupts. Instead, use the NVIC to enable the ECB_STATUS interrupt, and use the C_ECB_INT_ENABLE register to enable the RS_ENC_IN_ERR and RS_ENC_BLK_END interrupt triggers for the ECB_STATUS interrupt.

To encode one set of blocks, configure the transfers input pointer, output pointer, internal state, and block count as required for interrupts (described in Section 15.3.3.1, “Using the Reed-Solomon Encoder with Interrupts”). Then configure a DMA channel to handle the input or output as desired, ensuring that:

- If a DMA channel is used to control Reed-Solomon encoder inputs:
  - The C_ECB_RS_ENC_IN register address is set as the destination address
  - The Reed-Solomon encoder is selected as the destination for the DMA channel

- If a DMA channel is used to control Reed-Solomon encoder outputs:
  - The C_ECB_RS_ENC_OUT register address is set as the source address
  - The Reed-Solomon encoder is selected as the source for the DMA channel

- The word size for the DMA transfer matches the input and/or output pointer configuration.
- The transfer length is configured to match the number of words that must be transferred to complete the block set

Once the DMA channel is configured, wait for the ECB_STATUS interrupt to indicate the RS_ENC_BLK_END event, which signifies that the DMA transfer was completed and that the remaining encoded symbols are available in the C_ECB_RS_ENC_OUT register. Clear the ECB_INT_STATUS_RS_ENC_BLK_END status bit, then read the last set of symbols from the C_ECB_RS_ENC_OUT register.

To verify how many symbols are valid in the data read from the C_ECB_RS_ENC_OUT register, read the ECB_RS_PTR_RS_ENC_OUT_PTR bit field from the C_ECB_RS_PTR register, which indicates where the next symbol would have been written.

If an ECB_STATUS interrupt indicates that an RS_ENC_IN_ERR event has occurred, the ARM Cortex-M3 processor must manually handle this event.

For more information about the DMA, see Section 15.5, “Direct Memory Access (DMA) Controller”.

15.3.4 Reed-Solomon Decoder

The Reed-Solomon decoder implements a [15,11] Reed-Solomon code, which creates encoded data blocks consisting of 15 four-bit symbols from 11 data symbols and 4 code symbols. This data block contains 60 bits, of which 44 bits are data and 16 bits are code. The decoder block performs forward error correction, using data that has been previously encoded (including by a block such as the Reed-Solomon encoder described in Section 15.3.3, “Reed-Solomon Encoder”).

This Reed-Solomon decoder can detect and correct up to two symbols that contain errors per block of encoded data. The decoding algorithm operation output is undefined in the presence of more than two symbol errors. This means that it can flag severely corrupted data as correctable, even though this algorithm can only correct up to two symbol errors.

The Reed-Solomon decoder block has a programmable countdown-counter that can be set or read from the ECB_RS_BLK_CNT_RS_DEC_BLK_CNT bit field in the C_ECB_RS_BLK_CNT register. This counter sets the number of Reed-Solomon blocks decoded in a block set. When this counter is zero, the Reed-Solomon decoder is disabled. The countdown only considers a block complete when all of the 11 data symbols are sent to the output interface. When it reaches zero, an ECB_STATUS interrupt is generated indicating an RS_DEC_BLK_END event has occurred. If operating in DMA mode, the Reed-Solomon decoder input interface will not generate any new requests until the block counter is updated with a non-zero value.

IMPORTANT: To enable the Reed-Solomon decoder to continuously encode a stream of data blocks, set the block count to 0xFF. If configured in this way, the Reed-Solomon block counter is not decremented when blocks are encoded.

NOTE: If the Reed-Solomon decoder's counter is cleared while processing is ongoing, the decoder completes the current block's processing and transfers the output data to the output interface before being deactivated.

The Reed-Solomon decoder contains an internal buffer that holds one encoded Reed-Solomon block. The decoder starts the error detection and correction operation automatically once the block of data to be decoded has been loaded to this decoder. The Reed-Solomon decoder has a variable latency as listed in Table 61. The latency is measured from the time that the last symbol to be decoded is copied to the buffer, to the time when the error correction is finished. Therefore the latency information does not take into account the transfer of the symbols between the decoder core and the input and output interfaces.

Table 61. Latencies in Reed-Solomon Decoder

Number of Symbols with Errors	Latency (system clock cycle)
0	7
1	64
2	79
Uncorrectable	65

Input and output interfaces are included with the Reed-Solomon decoder to support smooth data transfers between the ARM Cortex-M3 processor and the decoder.

The input interface from the encoder uses the C_ECB_RS_DEC_IN register. The input interface also uses a circular pointer advancing four bits at a time to split the input register into symbols. Similar to the Reed-Solomon encoder input interface pointer, the decoder's input interface pointer:

- Can be configured for 4-, 8-, 16-, or 32-bit (1, 2, 4, or 8 symbol) mode using the ECB_CTRL_RS_DEC_IN_PTR_MODE bit field of the C_ECB_CTRL register
- Can be configured for increment or decrement mode using the ECB_CTRL_RS_DEC_IN_PTR_DIR bit from the C_ECB_CTRL register
- Can be read (but not written) using the ECB_RS_PTR_RS_DEC_IN_PTR bit field from the C_ECB_RS_PTR register

- Figure 32 shows the order in which the pointer selects symbols to be used by the decoder from the input interface for each input pointer mode.

Similarly, the output interface from the encoder uses the C_ECB_RS_DEC_OUT register. The output interface also uses a circular pointer advancing 4-bits at a time to split the output register into symbols in a separately configurable way from the input. Similar to the input interface pointer, the output interface pointer:

- Can be configured for 4-, 8-, 16-, or 32-bit (1, 2, 4, or 8 symbol) mode using the ECB_CTRL_RS_DEC_OUT_PTR_MODE bit field of the C_ECB_CTRL register
- Can be configured for increment or decrement mode using the ECB_CTRL_RS_DEC_OUT_PTR_DIR bit from the C_ECB_CTRL register
- Can be read (but not written) using the ECB_RS_PTR_RS_DEC_OUT_PTR bit field from the C_ECB_RS_PTR register
- Figure 32 shows the order in which the pointer places symbols to be generated by the decoder in the output interface for all pointer modes

The input and output interface blocks for the Reed-Solomon decoder cannot operate simultaneously. The input interface is active when starting up, and the output interface is not active until the internal buffer is filled. When the buffer is full, the input interface idles and waits until the error correction is completed. When the error correction is finished, the output interface is active until it has emptied the buffer.

When the input interface pointer is at the zero address and more data is needed to fill the internal buffer, an RS_DEC_IN_RDY interrupt is generated, indicating that the register is empty and ready to be refilled. The input interface does not transfer any new symbols to the RS encoder core until the interface register is refilled. If the input interface register is overwritten before all encoded symbols are copied to the internal buffer, an RS_DEC_IN_ERR interrupt is generated.

The output register C_ECB_RS_DEC_OUT is considered to be full when the next output pointer value is zero. This generates the RS_ENC_OUT_RDY interrupt, indicating that the output register is ready to be read. If the output register is read before the pointer wraps back to zero, the error interrupt will not be set to allow for partial output interface reads. A partial read can be necessary to complete the transfer of the last symbols of a set of encoded data blocks that do not form a multiple of the word size.

CAUTION: Partial reads must only be utilized when the block counter is zero and left over symbols are in the output register. If the output register is read when the encoder is still processing data, a symbol can be missed due to a race condition.

The C_ECB_STATUS register contains two bit fields that are used to report the number of errors that have been corrected by the Reed-Solomon decoder. These bit fields are:

- ECB_STATUS_RS_DEC_BLK_ERR, which indicates the number of symbols that were corrected in the most recently completed block
- ECB_STATUS_RS_DEC_ACC_ERR, which indicates the number of symbols that have been corrected since the decoder was most recently reset, or the block counter was last set.

If the Reed-Solomon decoder detects that a symbol error cannot be corrected, the decoder generates an RS_DEC_MISS_ERROR interrupt.

To reset the Reed-Solomon decoder's internal state machines, set the ECB_CMD_RS_DEC_CLR bit field from the C_ECB_CMD register. The decoder can be reset even when disabled.

15.3.4.1 Using the Reed-Solomon Decoder with Interrupts

To use the Reed-Solomon decoder with interrupts, use the NVIC to enable the RS_DEC_IN_RDY and RS_DEC_OUT_RDY interrupts.

Once the decoder input and output pointers are configured, do the following to decode one set of blocks:

1. Reset the Reed-Solomon decoder
2. Enable the decoder by setting the block count to the desired number of blocks.

When interrupts are raised, perform the appropriate action:

- When an RS_DEC_IN_RDY interrupt is raised, write one word to the C_ECB_RS_DEC_IN register. The number of symbols written should match the input pointer mode (additional symbols will be ignored).
- When an RS_DEC_OUT_RDY interrupt is raised, first read the C_ECB_INT_STATUS register to check the ECB_INT_STATUS_RS_DEC_BLK_END status bit, which indicates whether the data available in the C_ECB_RS_DEC_OUT register are the last symbols of the block.
  - If the ECB_INT_STATUS_RS_DEC_BLK_END status bit is set, clear it, then read the last set of symbols from the C_ECB_RS_DEC_OUT register.

    To verify how many symbols are valid in the data read from the C_ECB_RS_DEC_OUT register, read the ECB_RS_PTR_RS_DEC_OUT_PTR bit field from the C_ECB_RS_PTR register, which will indicate where the next symbol would have been written.

  - If the ECB_INT_STATUS_RS_DEC_BLK_END status bit is not set, read one word from the C_ECB_RS_DEC_OUT register. This register will contain the number of symbols specified by the output pointer mode.

  For every RS_DEC_OUT_RDY interrupt handled, the user application can read the C_ECB_STATUS register. This register supports providing an application with information about how many errors were detected in the last encoded data block, and how many errors have been detected since the beginning of the decoding process.

For information on configuring the NVIC, see Section 15.1, “Nested Vectored Interrupt Controller (NVIC)”.

15.3.4.2 Using the Reed-Solomon Decoder with the DMA Controller

To use the Reed-Solomon decoder with the DMA controller for input or output, use the NVIC to disable the appropriate RS_DEC_IN_RDY or RS_DEC_OUT_RDY interrupts. Instead, use the NVIC to enable the ECB_STATUS interrupt, and use the C_ECB_INT_ENABLE register to enable the RS_DEC_IN_ERR, RS_DEC_BLK_END, and RS_DEC_MISS_ERR interrupt triggers for the ECB_STATUS interrupt.

To decode one set of blocks, configure the transfers input pointer, output pointer, internal state and block count as required for interrupts (described in Section 15.3.4.1, “Using the Reed-Solomon Decoder with Interrupts”). Then configure a DMA channel to handle the input or output as desired, ensuring that:

- If a DMA channel is used to control Reed-Solomon decoder inputs:
  - The C_ECB_RS_DEC_IN register address is set as the destination address
  - The Reed-Solomon decoder is selected as the destination for the DMA channel

- If a DMA channel is used to control Reed-Solomon decoder outputs:
  - The C_ECB_RS_DEC_OUT register address is set as the source address
  - The Reed-Solomon decoder is selected as the source for the DMA channel

- The word size for the DMA transfer matches the input and/or output pointer configuration.
- The transfer length is configured to match the number of words that must be transferred to complete the block set

Once the DMA channel is configured, wait for the ECB_STATUS interrupt:

- The RS_DEC_BLK_END event indicates that the DMA transfer was completed and that the remaining decoded symbols are available in the C_ECB_RS_DEC_OUT register.

  To handle this event, clear the ECB_INT_STATUS_RS_DEC_BLK_END status bit, then read the last set of symbols from the C_ECB_RS_DEC_OUT register. To verify how many symbols are valid in the data read from the C_ECB_RS_DEC_OUT register, read the ECB_RS_PTR_RS_DEC_OUT_PTR bit field from the C_ECB_RS_PTR register, which indicates where the next symbol would have been written.

- The RS_DEC_IN_ERR and RS_DEC_MISS_ERR events indicate that an error has occurred that cannot be directly handled by the DMA or the Reed-Solomon decoder. The ARM Cortex-M3 processor must handle these errors manually.

For more information about the DMA, see Section 15.5, “Direct Memory Access (DMA) Controller”.

15.3.5 Control and Configuration Registers

Register Name	Register Description	Address
ECB_INT_STATUS C_ECB_INT_STATUS	ECB interrupt status register	0x40000000 P:0x3000
ECB_INT_ENABLE C_ECB_INT_ENABLE	ECB interrupt mask register	0x40000004 P:0x3001
ECB_CTRL C_ECB_CTRL	ECB control register	0x40000008 P:0x3002
ECB_RS_BLK_CNT C_ECB_RS_BLK_CNT	ECB Reed-Solomon block counter values	0x4000000C P:0x3003
ECB_CMD C_ECB_CMD	ECB commands	0x40000010 P:0x3004
ECB_RS_ENC_IN C_ECB_RS_ENC_IN	ECB Reed-Solomon encoder input data	0x40000014 P:0x3005
ECB_RS_ENC_OUT C_ECB_RS_ENC_OUT	ECB Reed-Solomon encoder output data	0x40000018 P:0x3006
ECB_RS_DEC_IN C_ECB_RS_DEC_IN	ECB Reed-Solomon decoder input data	0x4000001C P:0x3007
ECB_RS_DEC_OUT C_ECB_RS_DEC_OUT	ECB Reed-Solomon decoder output data	0x40000020 P:0x3008
ECB_RS_PTR C_ECB_RS_PTR	ECB Reed-Solomon pointer values	0x40000024 P:0x3009

Register Name	Register Description	Address
ECB_HMI_IN0 C_ECB_HMI_IN0	ECB Hamming and Interleave encoder input data 0	0x40000028 P:0x300A
ECB_HMI_IN1 C_ECB_HMI_IN1	ECB Hamming and Interleave encoder input data 1	0x4000002C P:0x300B
ECB_HMI_IN2 C_ECB_HMI_IN2	ECB Hamming and Interleave encoder input data 2	0x40000030 P:0x300C
ECB_HMI_IN3 C_ECB_HMI_IN3	ECB Hamming and Interleave encoder input data 3	0x40000034 P:0x300D
ECB_HMI_OUT0 C_ECB_HMI_OUT0	ECB Hamming and Interleave encoder output data 0	0x40000038 P:0x300E
ECB_HMI_OUT1 C_ECB_HMI_OUT1	ECB Hamming and Interleave encoder output data 1	0x4000003C P:0x300F
ECB_HMI_OUT2 C_ECB_HMI_OUT2	ECB Hamming and Interleave encoder output data 2	0x40000040 P:0x3010
ECB_HMI_OUT3 C_ECB_HMI_OUT3	ECB Hamming and Interleave encoder output data 3	0x40000044 P:0x3011
ECB_IHM_IN0 C_ECB_IHM_IN0	ECB Interleave and Hamming Decoder Input Data 0	0x40000048 P:0x3012
ECB_IHM_IN1 C_ECB_IHM_IN1	ECB Interleave and Hamming Decoder Input Data 1	0x4000004C P:0x3013
ECB_IHM_IN2 C_ECB_IHM_IN2	ECB Interleave and Hamming Decoder Input Data 2	0x40000050 P:0x3014
ECB_IHM_IN3 C_ECB_IHM_IN3	ECB Interleave and Hamming Decoder Input Data 3	0x40000054 P:0x3015
ECB_IHM_OUT0 C_ECB_IHM_OUT0	ECB Interleave and Hamming Decoder Output Data 0	0x40000058 P:0x3016
ECB_IHM_OUT1 C_ECB_IHM_OUT1	ECB Interleave and Hamming Decoder Output Data 1	0x4000005C P:0x3017
ECB_IHM_OUT2 C_ECB_IHM_OUT2	ECB Interleave and Hamming Decoder Output Data 2	0x40000060 P:0x3018
ECB_IHM_OUT3 C_ECB_IHM_OUT3	ECB Interleave and Hamming Decoder Output Data 3	0x40000064 P:0x3019
ECB_STATUS C_ECB_STATUS	ECB Error Status Register	0x40000068 P:0x301A
ECB_IHM_ECP C_ECB_IHM_ECP	ECB Hamming Decoder Error Correction Pointer	0x4000006C P:0x301B

15.3.5.1 ECB_INT_STATUS, C_ECB_INT_STATUS Settings

Bit Field	Field Name	Description
25	IHM_DEC_OUT_ERR	The Interleave and Hamming decoder output interface error interrupt status
24	IHM_DEC_IN_ERR	The Interleave and Hamming decoder input interface error interrupt status
17	HMI_ENC_OUT_ERR	The Hamming and Interleave encoder output interface error interrupt status
16	HMI_ENC_IN_ERR	The Hamming and Interleave encoder input interface error interrupt status
10	RS_DEC_MISS_ERR	The Reed-Solomon decoder missed error interrupt status
9	RS_DEC_BLK_END	The Reed-Solomon decoder block end interrupt status
8	RS_DEC_IN_ERR	The Reed-Solomon decoder output interface error interrupt status
1	RS_ENC_BLK_END	The Reed-Solomon encoder block end interrupt status
0	RS_ENC_IN_ERR	The Reed-Solomon encoder input interface error interrupt status

Field Name	Value Symbol	Value Description	Hex Value
IHM_DEC_OUT_ERR	INT_ACK_IHM_DEC_OUT_ERR	Acknowledge the interrupt	0x1
	INT_IDLE_IHM_DEC_OUT_ERR	This source has not set an interrupt	0x0*
	INT_PENDING_IHM_DEC_OUT_ERR	The Interleave and Hamming decoder output register was read before its input registers were updated.	0x1
IHM_DEC_IN_ERR	INT_ACK_IHM_DEC_IN_ERR	Acknowledge the interrupt	0x1
	INT_IDLE_IHM_DEC_IN_ERR	This source has not set an interrupt	0x0*
	INT_PENDING_IHM_DEC_IN_ERR	The Interleave and Hamming decoder input register was overwritten before its output registers were read.	0x1
HMI_ENC_OUT_ERR	INT_ACK_HMI_ENC_OUT_ERR	Acknowledge the interrupt	0x1
	INT_IDLE_HMI_ENC_OUT_ERR	This source has not set an interrupt	0x0*
	INT_PENDING_HMI_ENC_OUT_ERR	The Hamming and Interleave encoder output register was read before its input registers were updated.	0x1
HMI_ENC_IN_ERR	INT_ACK_HMI_ENC_IN_ERR	Acknowledge the interrupt	0x1
	INT_IDLE_HMI_ENC_IN_ERR	This source has not set an interrupt	0x0*
	INT_PENDING_HMI_ENC_IN_ERR	The Hamming and Interleave encoder input register was overwritten before its output registers were read.	0x1
RS_DEC_MISS_ERR	INT_ACK_RS_DEC_MISS_ERR	Acknowledge the interrupt	0x1
	INT_IDLE_RS_DEC_MISS_ERR	This source has not set an interrupt	0x0*
	INT_PENDING_RS_DEC_MISS_ERR	The RS decoder couldn't fix an error in the block	0x1
RS_DEC_BLK_END	INT_ACK_RS_DEC_BLK_END	Acknowledge the interrupt	0x1
	INT_IDLE_RS_DEC_BLK_END	This source has not set an interrupt	0x0*
	INT_PENDING_RS_DEC_BLK_END	The RS decoder block counter counted down to zero	0x1

Field Name	Value Symbol	Value Description	Hex Value
RS_DEC_IN_ERR	INT_ACK_RS_DEC_IN_ERR	Acknowledge the interrupt	0x1
	INT_IDLE_RS_DEC_IN_ERR	This source has not set an interrupt	0x0*
	INT_PENDING_RS_DEC_IN_ERR	The RS decoder input register was overwritten before the output registers were read.	0x1
RS_ENC_BLK_END	INT_ACK_RS_ENC_BLK_END	Acknowledge the interrupt	0x1
	INT_IDLE_RS_ENC_BLK_END	This source has not set an interrupt	0x0*
	INT_PENDING_RS_ENC_BLK_END	The RS encoder block counter counted down to zero	0x1
RS_ENC_IN_ERR	INT_ACK_RS_ENC_IN_ERR	Acknowledge the interrupt	0x1
	INT_IDLE_RS_ENC_IN_ERR	This source has not set an interrupt	0x0*
	INT_PENDING_RS_ENC_IN_ERR	The RS encoder input register was overwritten before the output registers were read.	0x1

15.3.5.2 ECB_INT_ENABLE, C_ECB_INT_ENABLE Settings

Bit Field	Field Name	Description
25	IHM_DEC_OUT_ERR	The Interleave and Hamming decoder output interface error interrupt mask
24	IHM_DEC_IN_ERR	The Interleave and Hamming decoder input interface error interrupt mask
17	HMI_ENC_OUT_ERR	The Hamming and Interleave encoder output interface error interrupt mask
16	HMI_ENC_IN_ERR	The Hamming and Interleave encoder input interface error interrupt mask
10	RS_DEC_MISS_ERR	The Reed-Solomon decoder missed error interrupt mask
9	RS_DEC_BLK_END	The Reed-Solomon decoder block end interrupt mask
8	RS_DEC_IN_ERR	The Reed-Solomon decoder output interface error interrupt mask
1	RS_ENC_BLK_END	The Reed-Solomon encoder block end interrupt mask
0	RS_ENC_IN_ERR	The Reed-Solomon Encoder input interface error interrupt mask

Field Name	Value Symbol	Value Description	Hex Value
IHM_DEC_OUT_ERR	INT_DIS_IHM_DEC_OUT_ERR	This source cannot set an interrupt	0x0*
	INT_EBL_IHM_DEC_OUT_ERR	This source can set the ECB_COM interrupt line	0x1
IHM_DEC_IN_ERR	INT_DIS_IHM_DEC_IN_ERR	This source cannot set an interrupt	0x0*
	INT_EBL_IHM_DEC_IN_ERR	This source can set the ECB_COM interrupt line	0x1
HMI_ENC_OUT_ERR	INT_DIS_HMI_ENC_OUT_ERR	This source cannot set an interrupt	0x0*
	INT_EBL_HMI_ENC_OUT_ERR	This source can set the ECB_COM interrupt line	0x1
HMI_ENC_IN_ERR	INT_DIS_HMI_ENC_IN_ERR	This source cannot set an interrupt	0x0*
	INT_EBL_HMI_ENC_IN_ERR	This source can set the ECB_COM interrupt line	0x1
RS_DEC_MISS_ERR	INT_DIS_RS_DEC_MISS_ERR	This source cannot set an interrupt	0x0*
	INT_EBL_RS_DEC_MISS_ERR	This source can set the ECB_COM interrupt line	0x1

Field Name	Value Symbol	Value Description	Hex Value
RS_DEC_BLK_END	INT_DIS_RS_DEC_BLK_END	This source cannot set an interrupt	0x0*
	INT_EBL_RS_DEC_BLK_END	This source can set the ECB_COM interrupt line	0x1
RS_DEC_IN_ERR	INT_DIS_RS_DEC_IN_ERR	This source cannot set an interrupt	0x0*
	INT_EBL_RS_DEC_IN_ERR	This source can set the ECB_COM interrupt line	0x1
RS_ENC_BLK_END	INT_DIS_RS_ENC_BLK_END	This source cannot set an interrupt	0x0*
	INT_EBL_RS_ENC_BLK_END	This source can set the ECB_COM interrupt line	0x1
RS_ENC_IN_ERR	INT_DIS_RS_ENC_IN_ERR	This source cannot set an interrupt	0x0*
	INT_EBL_RS_ENC_IN_ERR	This source can set the ECB_COM interrupt line	0x1

15.3.5.3 ECB_CTRL, C_ECB_CTRL Settings

Bit Field	Field Name	Description
28	IHM_DEC_EBL	Enable/disable the Interleave and Hamming decoder
27	IHM_COR_EBL	Hamming Decoder error correction enable
26	IHM_INTERLEAVE_EBL	Enable/disable the de-interleaver
25	IHM_INTERLEAVE_MODE	De-Interleaver mode
24	IHM_DMA_MODE	The Interleave and Hamming Decoder DMA mode
20	HMI_ENC_EBL	Hamming encoder enable
19	HMI_PARITY_EBL	Hamming encoder parity generator enable
18	HMI_INTERLEAVE_EBL	Enable/disable the interleaver
17	HMI_INTERLEAVE_MODE	Interleaver mode
16	HMI_DMA_MODE	The Hamming and Interleave Encoder DMA mode
13	RS_DEC_IN_PTR_DIR	Reed-Solomon Decoder Input Register Pointer Update Direction
12	RS_DEC_OUT_PTR_DIR	Reed-Solomon Decoder Output Register Pointer Update Direction
11:10	RS_DEC_IN_PTR_MODE	Reed-Solomon Decoder Input Register Pointer Mode
9:8	RS_DEC_OUT_PTR_MODE	Reed-Solomon Decoder Output Register Pointer Mode
5	RS_ENC_IN_PTR_DIR	Reed-Solomon Encoder Input Register Pointer Update Direction
4	RS_ENC_OUT_PTR_DIR	Reed-Solomon Encoder Output Register Pointer Update Direction
3:2	RS_ENC_IN_PTR_MODE	Reed-Solomon Encoder Input Register Pointer Mode
1:0	RS_ENC_OUT_PTR_MODE	Reed-Solomon Encoder Output Register Pointer Mode

Field Name	Value Symbol	Value Description	Hex Value
IHM_DEC_EBL	IHM_DISABLE	Disable the Interleave and Hamming decoder	0x0*
	IHM_ENABLE	Enable the Interleave and Hamming decoder	0x1

Field Name	Value Symbol	Value Description	Hex Value
IHM_COR_EBL	IHM_COR_DISABLE	The decoder does not correct the errors detected	0x0
	IHM_COR_ENABLE	The decoder corrects the errors detected	0x1*
IHM_INTERLEAVE_EBL	IHM_INTERLEAVE_DISABLE	The de-interleaver transfers the accelerator input as is	0x0
	IHM_INTERLEAVE_ENABLE	The de-interleaver is enabled, the accelerator input is de-interleaved	0x1*
IHM_INTERLEAVE_MODE	IHM_INTERLEAVE_128B	The de-interleaver operates in 128 bit mode	0x0*
	IHM_INTERLEAVE_126B	The de-interleaver operates in 126 bit mode	0x1
IHM_DMA_MODE	IHM_DMA_3REG	The interrupt is cleared when the OUT1, OUT2, and OUT3 registers are read	0x0*
	IHM_DMA_4REG	The interrupt is cleared only when all output registers are read	0x1
HMI_ENC_EBL	HMI_DISABLE	Disable the Hamming and Interleave encoder	0x0*
	HMI_ENABLE	Enable the Hamming and Interleave encoder	0x1
HMI_PARITY_EBL	HMI_PARITY_DISABLE	The encoder does not overwrite the parity bits	0x0
	HMI_PARITY_ENABLE	The encoder inserts the Hamming parity bits	0x1*
HMI_INTERLEAVE_EBL	HMI_INTERLEAVE_DISABLE	The interleaver transfers the encoder output as is	0x0
	HMI_INTERLEAVE_ENABLE	The interleaver is enabled, the encoder output is interleaved	0x1*
HMI_INTERLEAVE_MODE	HMI_INTERLEAVE_128B	The interleaver operates in 128 bit mode	0x0*
	HMI_INTERLEAVE_126B	The interleaver operates in 126 bit mode	0x1
HMI_DMA_MODE	HMI_DMA_3REG	The interrupt is cleared when the IN1, IN2 and IN3 registers are updated	0x0*
	HMI_DMA_4REG	The interrupt is cleared only when all input registers are updated	0x1
RS_DEC_IN_PTR_DIR	RS_DEC_IN_INC	Decoder input pointer increments by 1 RS symbol	0x0*
	RS_DEC_IN_DEC	Decoder input pointer decrements by 1 RS symbol	0x1
RS_DEC_OUT_PTR_DIR	RS_DEC_OUT_INC	Decoder Output pointer increments by 1 RS symbol	0x0*
	RS_DEC_OUT_DEC	Decoder Output pointer decrements by 1 RS symbol	0x1

Field Name	Value Symbol	Value Description	Hex Value
RS_DEC_IN_PTR_MODE	RS_DEC_IN_32BIT	Input data uses 32-bits per word	0x0*
	RS_DEC_IN_16BIT	Input data uses 16-bits per word (LSB aligned)	0x1
	RS_DEC_IN_8BIT	Input data uses 8-bits per word (LSB aligned)	0x2
	RS_DEC_IN_4BIT	Input data uses 4-bits per word (LSB aligned)	0x3
RS_DEC_OUT_PTR_MODE	RS_DEC_OUT_32BIT	Output data uses 32-bits per word	0x0*
	RS_DEC_OUT_16BIT	Output data uses 16-bits per word (LSB aligned)	0x1
	RS_DEC_OUT_8BIT	Output data uses 8-bits per word (LSB aligned)	0x2
	RS_DEC_OUT_4BIT	Output data uses 4-bits per word (LSB aligned)	0x3
RS_ENC_IN_PTR_DIR	RS_ENC_IN_INC	Encoder input pointer increments by 1 RS symbol	0x0*
	RS_ENC_IN_DEC	Encoder input pointer decrements by 1 RS symbol	0x1
RS_ENC_OUT_PTR_DIR	RS_ENC_OUT_INC	Encoder output pointer increments by 1 RS symbol	0x0*
	RS_ENC_OUT_DEC	Encoder output pointer decrements by 1 RS symbol	0x1
RS_ENC_IN_PTR_MODE	RS_ENC_IN_32BIT	Input data uses 32-bits per word	0x0*
	RS_ENC_IN_16BIT	Input data uses 16-bits per word (LSB aligned)	0x1
	RS_ENC_IN_8BIT	Input data uses 8-bits per word (LSB aligned)	0x2
	RS_ENC_IN_4BIT	Input data uses 4-bits per word (LSB aligned)	0x3
RS_ENC_OUT_PTR_MODE	RS_ENC_OUT_32BIT	Output data uses 32-bits per word	0x0*
	RS_ENC_OUT_16BIT	Output data uses 16-bits per word (LSB aligned)	0x1
	RS_ENC_OUT_8BIT	Output data uses 8-bits per word (LSB aligned)	0x2
	RS_ENC_OUT_4BIT	Output data uses 4-bits per word (LSB aligned)	0x3

15.3.5.4 ECB_RS_BLK_CNT, C_ECB_RS_BLK_CNT Settings

Bit Field	Field Name	Description
15:8	RS_DEC_BLK_CNT	RS Decoder block size to count down from (255 is continuous run)
7:0	RS_ENC_BLK_CNT	RS Encoder block size to count down from (255 is continuous run)

15.3.5.5 ECB_CMD, C_ECB_CMD Settings

Bit Field	Field Name	Description
8	RS_DEC_CLR	Reed-Solomon Decoder Clear
0	RS_ENC_CLR	Reed-Solomon Encoder Clear

Field Name	Value Symbol	Value Description	Hex Value
RS_DEC_CLR	RS_DEC_CLEAR	Reset the RS decoder state	0x1
RS_ENC_CLR	RS_ENC_CLEAR	Reset the RS encoder state	0x1

15.3.5.6 ECB_RS_ENC_IN, C_ECB_RS_ENC_IN Settings

Bit Field	Field Name	Description
31:0	RS_ENC_IN	The RS encoder input register

15.3.5.7 ECB_RS_ENC_OUT, C_ECB_RS_ENC_OUT Settings

Bit Field	Field Name	Description
31:0	RS_ENC_OUT	The RS encoder output register

15.3.5.8 ECB_RS_DEC_IN, C_ECB_RS_DEC_IN Settings

Bit Field	Field Name	Description
31:0	RS_DEC_IN	The RS decoder input register

15.3.5.9 ECB_RS_DEC_OUT, C_ECB_RS_DEC_OUT Settings

Bit Field	Field Name	Description
31:0	RS_DEC_OUT	The RS decoder output register

15.3.5.10 ECB_RS_PTR, C_ECB_RS_PTR Settings

Bit Field	Field Name	Description
13:11	RS_DEC_IN_PTR	The RS decoder input pointer value
10:8	RS_DEC_OUT_PTR	The RS decoder output pointer value
5:3	RS_ENC_IN_PTR	The RS encoder input pointer value
2:0	RS_ENC_OUT_PTR	The RS encoder output pointer value

15.3.5.11 ECB_HMI_IN0, C_ECB_HMI_IN0 Settings

Bit Field	Field Name	Description
31:0	HMI_IN0	HMI input data word 0

15.3.5.12 ECB_HMI_IN1, C_ECB_HMI_IN1 Settings

Bit Field	Field Name	Description
31:0	HMI_IN1	HMI input data word 1

15.3.5.13 ECB_HMI_IN2, C_ECB_HMI_IN2 Settings

Bit Field	Field Name	Description
31:0	HMI_IN2	HMI input data word 2

15.3.5.14 ECB_HMI_IN3, C_ECB_HMI_IN3 Settings

Bit Field	Field Name	Description
31:0	HMI_IN3	HMI input data word 3

15.3.5.15 ECB_HMI_OUT0, C_ECB_HMI_OUT0 Settings

Bit Field	Field Name	Description
31:0	HMI_OUT0	HMI output data word 0

15.3.5.16 ECB_HMI_OUT1, C_ECB_HMI_OUT1 Settings

Bit Field	Field Name	Description
31:0	HMI_OUT1	HMI output data word 1

15.3.5.17 ECB_HMI_OUT2, C_ECB_HMI_OUT2 Settings

Bit Field	Field Name	Description
31:0	HMI_OUT2	HMI output data word 2

15.3.5.18 ECB_HMI_OUT3, C_ECB_HMI_OUT3 Settings

Bit Field	Field Name	Description
31:0	HMI_OUT3	HMI output data word 3

15.3.5.19 ECB_IHM_IN0, C_ECB_IHM_IN0 Settings

Bit Field	Field Name	Description
31:0	IHM_IN0	IHM input data word 0

15.3.5.20 ECB_IHM_IN1, C_ECB_IHM_IN1 Settings

Bit Field	Field Name	Description
31:0	IHM_IN1	IHM input data word 1

15.3.5.21 ECB_IHM_IN2, C_ECB_IHM_IN2 Settings

Bit Field	Field Name	Description
31:0	IHM_IN2	IHM input data word 2

15.3.5.22 ECB_IHM_IN3, C_ECB_IHM_IN3 Settings

Bit Field	Field Name	Description
31:0	IHM_IN3	IHM input data word 3

15.3.5.23 ECB_IHM_OUT0, C_ECB_IHM_OUT0 Settings

Bit Field	Field Name	Description
31:0	IHM_OUT0	IHM output data word 0

15.3.5.24 ECB_IHM_OUT1, C_ECB_IHM_OUT1 Settings

Bit Field	Field Name	Description
31:0	IHM_OUT1	IHM output data word 1

15.3.5.25 ECB_IHM_OUT2, C_ECB_IHM_OUT2 Settings

Bit Field	Field Name	Description
31:0	IHM_OUT2	IHM output data word 2

15.3.5.26 ECB_IHM_OUT3, C_ECB_IHM_OUT3 Settings

Bit Field	Field Name	Description
31:0	IHM_OUT3	IHM output data word 3

15.3.5.27 ECB_STATUS, C_ECB_STATUS Settings

Bit Field	Field Name	Description
19	IHM_EDSR_BLK5	Hamming block 5 error detection but no correction status
18	IHM_EDCSR_BLK5	Hamming block 5 error detection and correction status
17	IHM_EDSR_BLK4	Hamming block 4 error detection but no correction status
16	IHM_EDCSR_BLK4	Hamming block 4 error detection and correction status
15	IHM_EDSR_BLK3	Hamming block 3 error detection but no correction status
14	IHM_EDCSR_BLK3	Hamming block 3 error detection and correction status
13	IHM_EDSR_BLK2	Hamming block 2 error detection but no correction status
12	IHM_EDCSR_BLK2	Hamming block 2 error detection and correction status
11	IHM_EDSR_BLK1	Hamming block 1 error detection but no correction status
10	IHM_EDCSR_BLK1	Hamming block 1 error detection and correction status

Bit Field	Field Name	Description
9	IHM_EDSR_BLK0	Hamming block 0 error detection but no correction status
8	IHM_EDCSR_BLK0	Hamming block 0 error detection and correction status
7:6	RS_DEC_BLK_ERR	The RS decoder error count in the last block
5:0	RS_DEC_ACC_ERR	The RS decoder accumulated error count

Field Name	Value Symbol	Value Description	Hex Value
IHM_EDSR_BLK5	IHM_BLK5_CLEAN	The Hamming decoder couldn't detect any errors in the block	0x0*
	IHM_BLK5_ERR_DETECTED	The Hamming decoder detected errors in the block	0x1
IHM_EDCSR_BLK5	IHM_BLK5_ERR_NONE	The Hamming decoder couldn't detect any errors in the block	0x0*
	IHM_BLK5_ERR_CORRECTED	The Hamming decoder detected an error in the block and corrected it	0x1
IHM_EDSR_BLK4	IHM_BLK4_CLEAN	The Hamming decoder couldn't detect any errors in the block	0x0*
	IHM_BLK4_ERR_DETECTED	The Hamming decoder detected errors in the block	0x1
IHM_EDCSR_BLK4	IHM_BLK4_ERR_NONE	The Hamming decoder couldn't detect any errors in the block	0x0*
	IHM_BLK4_ERR_CORRECTED	The Hamming decoder detected an error in the block and corrected it	0x1
IHM_EDSR_BLK3	IHM_BLK3_CLEAN	The Hamming decoder couldn't detect any errors in the block	0x0*
	IHM_BLK3_ERR_DETECTED	The Hamming decoder detected errors in the block	0x1
IHM_EDCSR_BLK3	IHM_BLK3_ERR_NONE	The Hamming decoder couldn't detect any errors in the block	0x0*
	IHM_BLK3_ERR_CORRECTED	The Hamming decoder detected an error in the block and corrected it	0x1
IHM_EDSR_BLK2	IHM_BLK2_CLEAN	The Hamming decoder couldn't detect any errors in the block	0x0*
	IHM_BLK2_ERR_DETECTED	The Hamming decoder detected errors in the block	0x1
IHM_EDCSR_BLK2	IHM_BLK2_ERR_NONE	The Hamming decoder couldn't detect any errors in the block	0x0*
	IHM_BLK2_ERR_CORRECTED	The Hamming decoder detected an error in the block and corrected it	0x1
IHM_EDSR_BLK1	IHM_BLK1_CLEAN	The Hamming decoder couldn't detect any errors in the block	0x0*
	IHM_BLK1_ERR_DETECTED	The Hamming decoder detected errors in the block	0x1
IHM_EDCSR_BLK1	IHM_BLK1_ERR_NONE	The Hamming decoder couldn't detect any errors in the block	0x0*
	IHM_BLK1_ERR_CORRECTED	The Hamming decoder detected an error in the block and corrected it	0x1

Field Name	Value Symbol	Value Description	Hex Value
IHM_EDSR_BLK0	IHM_BLK0_CLEAN	The Hamming decoder couldn't detect any errors in the block	0x0*
	IHM_BLK0_ERR_DETECTED	The Hamming decoder detected errors in the block	0x1
IHM_EDCSR_BLK0	IHM_BLK0_ERR_NONE	The Hamming decoder couldn't detect any errors in the block	0x0*
	IHM_BLK0_ERR_CORRECTED	The Hamming decoder detected an error in the block and corrected it	0x1

15.3.5.28 ECB_IHM_ECP, C_ECB_IHM_ECP Settings

Bit Field	Field Name	Description
29:25	BLK5	The Hamming decoder block 5 error correction pointer
24:20	BLK4	The Hamming decoder block 4 error correction pointer
19:15	BLK3	The Hamming decoder block 3 error correction pointer
14:10	BLK2	The Hamming decoder block 2 error correction pointer
9:5	BLK1	The Hamming decoder block 1 error correction pointer
4:0	BLK0	The Hamming decoder block 0 error correction pointer

15.4 Audio Codec Blocks

The Ezairo 7100 system provides four separate hardware accelerators that are used to encode and decode two different audio compression CODECs: G.722 and CVSD. The hardware accelerators are implemented independently of each other, and can be used simultaneously.

- The G.722 accelerators provide an encoder and decoder for a subband audio compression compliant to the ITU-T G.722 standard. This standard provides a wideband audio coding with a bandwidth of 50 Hz to 7 kHz compressed into a 64 kbps data stream. Further information about this CODEC is available from the International Telecommunications Union, described in the recommendation document available at *http://www.itu.int/rec/T-REC-G.722/en*
- The CVSD accelerators provide an encoder and decoder for a continuously variable slope delta modulation voice CODEC. These accelerators provide a 64 kbps implementation that is compliant to the Bluetooth CVSD standard.

The audio CODEC blocks are supported by the ACB_STATUS interrupt. This interrupt is used to indicate that errors have occurred when inputting data to any of the blocks. This interrupt can also be used to indicate when data can be provided to the G.722 decoder through the concealment inputs. To configure the interrupt sources that can trigger an ACB_STATUS interrupt, enable the desired events using the C_ACB_INT_ENABLE register. To read the status of these event sources, or to clear the cause of a pending ACB_STATUS interrupt, use the C_ACB_INT_STATUS register.

15.4.1 G.722 Encoder

The G.722 CODEC encoder accelerator consists of two main sections: the encoder quadrature mirror filter (QMF) FilterBank, and the subband adaptive pulse code modulation (SB-ADPCM) encoder. To enable simple communications between the ARM Cortex-M3 processor and the accelerator, it also includes input and output interfaces. The block diagram of the encoder, excluding the input and output interfaces, is shown in Figure 33, which is copied from document ITU-T G.722, *7 kHz audio-coding within 64 kbit/s*.

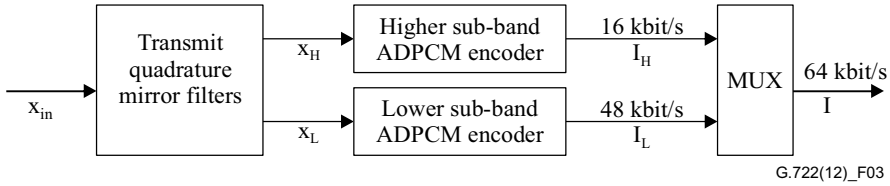


Figure 33. G.722 Encoder Block Diagram

To enable the G.722 encoder, set the `ACB_CTRL_G722_ENC_EBL` bit from the `C_ACB_CTRL` register.

15.4.1.1 G.722 Encoder QMF Filterbank

The G.722 encoder implements the QMF filterbank dictated by the G.722 standard. This filterbank is a symmetric, perfect reconstruction FIR filterbank with 24 coefficients (12 of which are unique coefficients). The standard dictates that the filterbank processes data at a 16 kHz sample rate. The magnitude response of the QMF filterbank is given in Figure 34.

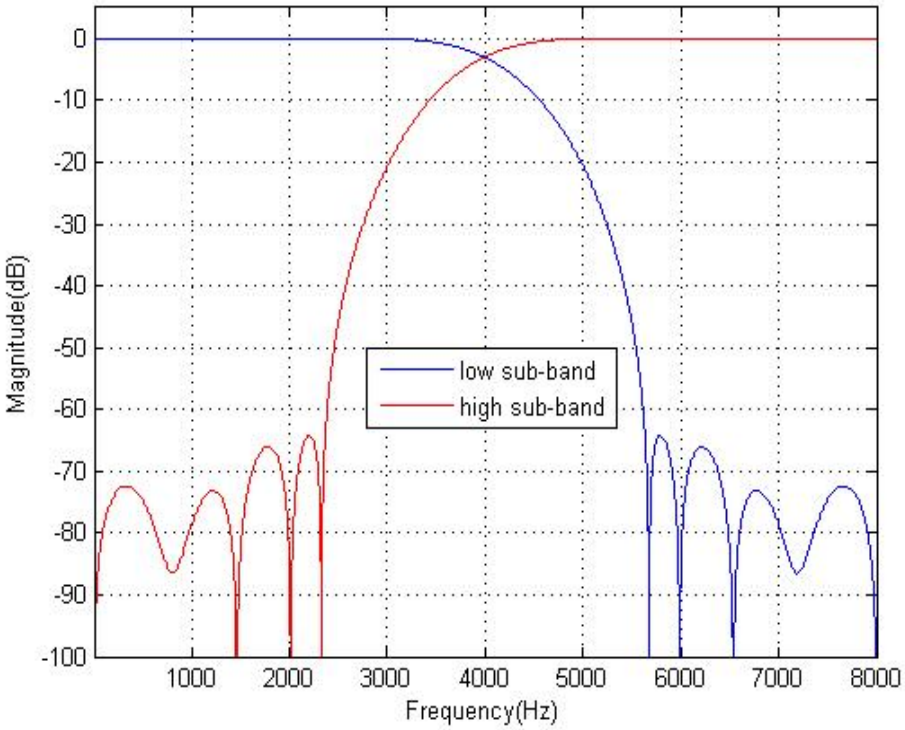


Figure 34. G.722 QMF Filterbank Magnitude Response

NOTE: Because of the 23 taps in the filter, the encoder filterbank has a constant group delay of 11.5 input samples (0.72 ms for 16 kHz sampling).

The output sampling rate for the QMF filterbank is decimated by a factor of 2 for each subband. That is, the G.722 QMF filterbank requires two input samples to create one subband output sample in the high subband, and one subband output sample in the low subband. The input and output data from the QMF filterbank uses a data word length of 16 bits.

If a user application wants to implement a non-standard QMF filterbank, the QMF filterbank can be bypassed by setting the ACB_CTRL_G722_ENC_FB_BYPASS bit of the C_ACB_CTRL register. If the QMF filterbank is bypassed, the input audio must be pre-filtered before being provided to the G.722 encoder accelerator. The decimated pre-filtered input data is provided to the input interface with:

- The decimated high subband sample provided to the upper half of the input interface
- The decimated low subband sample provided to the lower half of the input interface

The internal state of the QMF filterbank must be cleared prior to processing a new data stream since this state, set by previous data streams, can introduce artifacts to the G.722 output. To clear the internal state of the G.722 QMF filterbank, set the ACB_CMD_G722_ENC_FB_CLR bit from the C_ACB_CMD register. The G.722 QMF filterbank registers are cleared one cycle after this register bit is set.

15.4.1.2 G.722 SB-ADPCM Encoders

The G.722 subband adaptive pulse-code modulation (SB-ADPCM) encoder follows the QMF filterbank. A block diagram showing the lower SB-ADPCM encoder is provided in Figure 35, and a block diagram of the higher SB-ADPCM encoder is provided in Figure 36. They are copied from document ITU-T G.722, *7 kHz audio-coding within 64 kbit/s*.

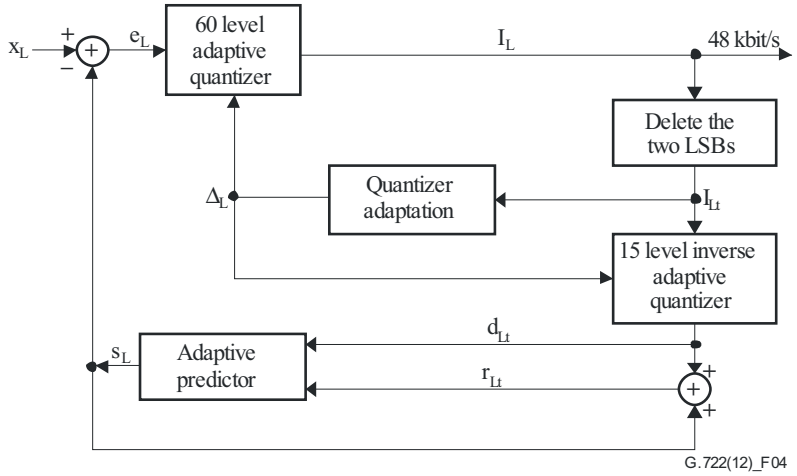


Figure 35. Lower SB-ADPCM Encoder

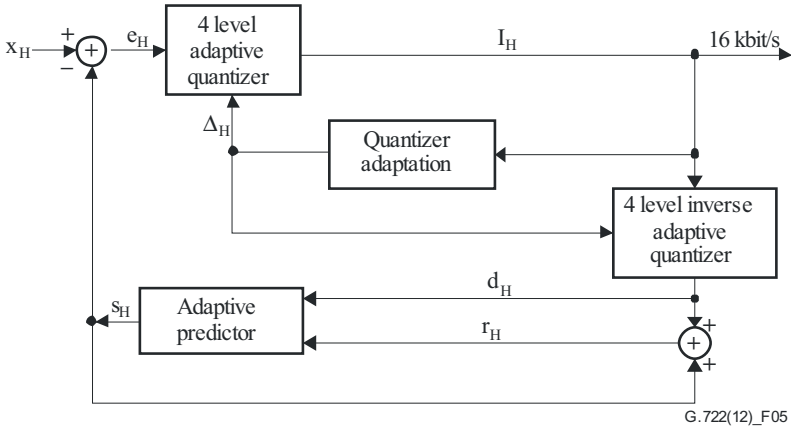


Figure 36. Higher SB-ADPCM Encoder

The lower SB-ADPCM encoder generates a 6-bit output from its 16-bit input, and the higher SB-ADPCM encoder generates a 2-bit output from its 16-bit input. These bits are combined at the output from the G.722 encoder accelerator in following format:

$I_{H1}I_{H2}I_{L1}I_{L2}I_{L3}I_{L4}I_{L5}I_{L6}$

In this octet format, the I_{H1} and I_{L1} are MSBs of the higher and lower subband encoder outputs respectively.

NOTE: The details of the sub-blocks of the G.722 SB-ADPCM encoder are not repeated here (refer to the G.722 standard). The encoder does not allow any auxiliary channel insertion at the output as the only mode supported is mode1, where all 6-bits coming out of the lower band encoder are utilized.

IMPORTANT: The G.722 implementation in Ezairo 7100 does not support message insertion in between the transmitted audio data.

The internal state of the SB-ADPCM encoder must be cleared prior to processing a new data stream since this state, set by previous data streams, can introduce artifacts to the G.722 output. To clear the internal state of the G.722 SB-ADPCM encoder, set the `ACB_CMD_G722_ENC_SBBND_CLR` bit from the `C_ACB_CMD` register. The G.722 SB-ADPCM encoder registers will be cleared one cycle after this register bit is set.

15.4.1.3 G.722 Encoder Input and Output Interfaces

The input and output interfaces provided for the G.722 encoder accelerator coordinate the flow of samples through the accelerator. A block diagram highlighting these interfaces and how they relate to the G.722 QMF filterbank and SB-ADPCM encoders is provided in Figure 37.

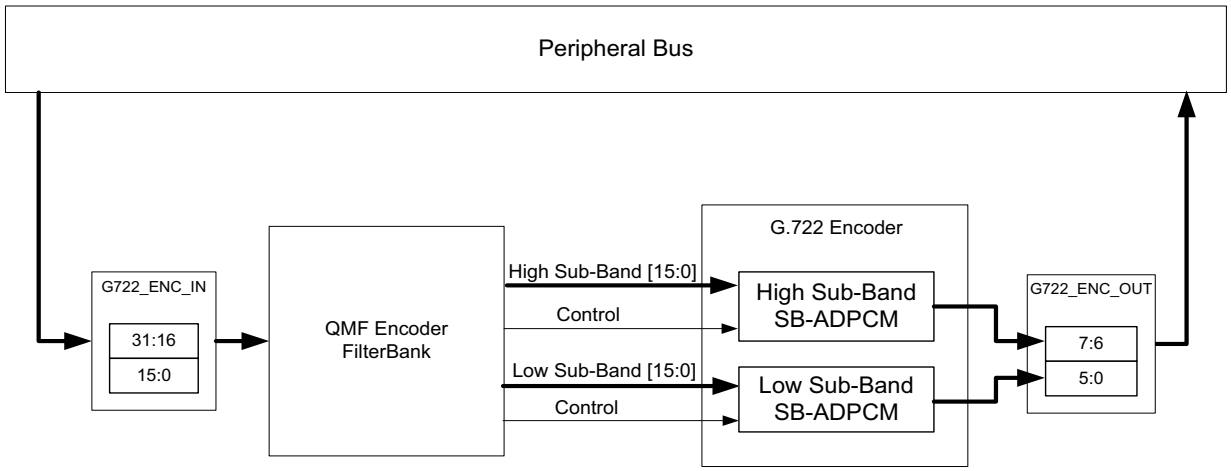


Figure 37. G.722 Encoder with Input and Output Interfaces

The G.722 standard requires input data to be formatted in an m1p14 format, where the sign bit is repeated to convert the 15-bit data to a 16-bit sample. If input data to the G.722 encoder is formatted in a standard frac16 (m1p15) format, the G.722 encoder can be configured for 16-bit input mode which quantizes the LSB of the input data to convert the 16-bit input data to the required 15-bit data. To select between supporting 15-bit or 16-bit input data, configure the `ACB_CTRL_G722_ENC_IN_FORMAT` bit from the `C_ACB_CTRL` register.

The input interface uses a 32-bit register, `C_ACB_G722_ENC_IN`, to supply the QMF filterbank with two 16-bit audio samples (or a high subband and low subband input if the QMF filterbank is bypassed). The QMF filterbank and SB-ADPCM encoder remain idle until both of the 16-bit sample fields in the input interface register have been written. When data is provided to the G.722 encoder, the input interface transfers the low sample first, then the high sample from the G.722 input register to the QMF filterbank. A `G722_ENC_IN_RDY` interrupt is triggered when the G.722 encoder is enabled and idle. This interrupt indicates that the input interface is ready to accept more data.

The output interface uses an 8-bit register, `C_ACB_G722_ENC_OUT`, to store the compressed data from the two SB-ADPCM encoders. When this output is updated, a `G722_ENC_OUT_RDY` interrupt is triggered. After triggering this interrupt, the output interface waits until the output register is read before triggering another `G722_ENC_IN_RDY` interrupt indicating that the G.722 encoder is idle and ready for more data.

If the input interface registers are overwritten while the G.722 encoder accelerator is sill processing the previous samples, a `G722_ENC_IN_ERR` interrupt is generated. In this situation, the accelerator continues its operation and disregards the new data written to the input registers. To enable this interrupt, set the `ACB_INT_ENABLE_G722_ENC_IN_ERR` bit in the `C_ACB_INT_ENABLE` register.

15.4.2 G.722 Decoder

The G.722 CODEC decoder accelerator consists of two main sections: the decoder subband adaptive pulse code modulation (SB-ADPCM) decoder and quadrature mirror filter (QMF) filterbank. To enable simple communications between the ARM Cortex-M3 processor and the accelerator, it also includes input and output interfaces. The block diagram of the decoder excluding the input and output interfaces is shown in Figure 38, which is copied from document ITU-T G.722, *7 kHz audio-coding within 64 kbit/s*.

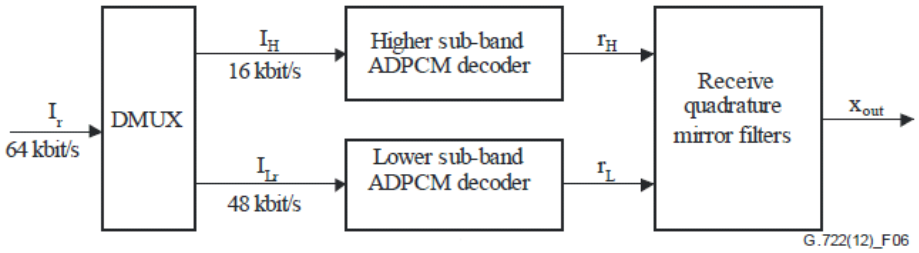


Figure 38. G.722 Codec Decoder Block Diagram

To enable the G.722 decoder, set the `ACB_CTRL_G722_DEC_EBL` bit from the `C_ACB_CTRL` register.

15.4.2.1 G.722 SB-ADPCM Decoders

The subband decoder receives the 8-bit compressed data and splits it into lower and higher channels as:

$$I_H = I_{r1}I_{r2}$$

$$I_L = I_{r3}I_{r4}I_{r5}I_{r6}I_{r7}I_{r8}$$

In this notation, the I_{r1} stands for the MSB and I_{r8} stands for the LSB of the input data.

The 2-bit higher subband compressed data is channeled into the higher subband ADPCM decoder. A block diagram of this decoder is provided in Figure 39. The 6-bit lower subband compressed data is channeled into the lower subband ADPCM decoder. A block diagram of this decoder is given in Figure 40. The figures are copied from document ITU-T G.722, *7 kHz audio-coding within 64 kbit/s.*

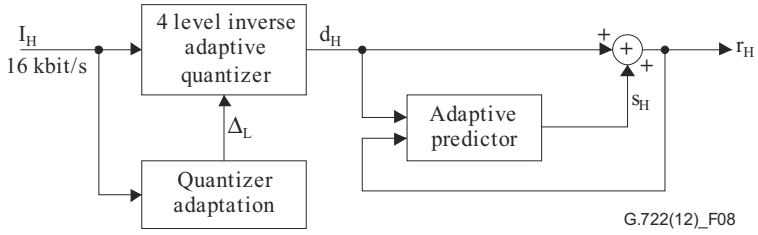


Figure 39. G.722 Higher Subband Decoder Block Diagram

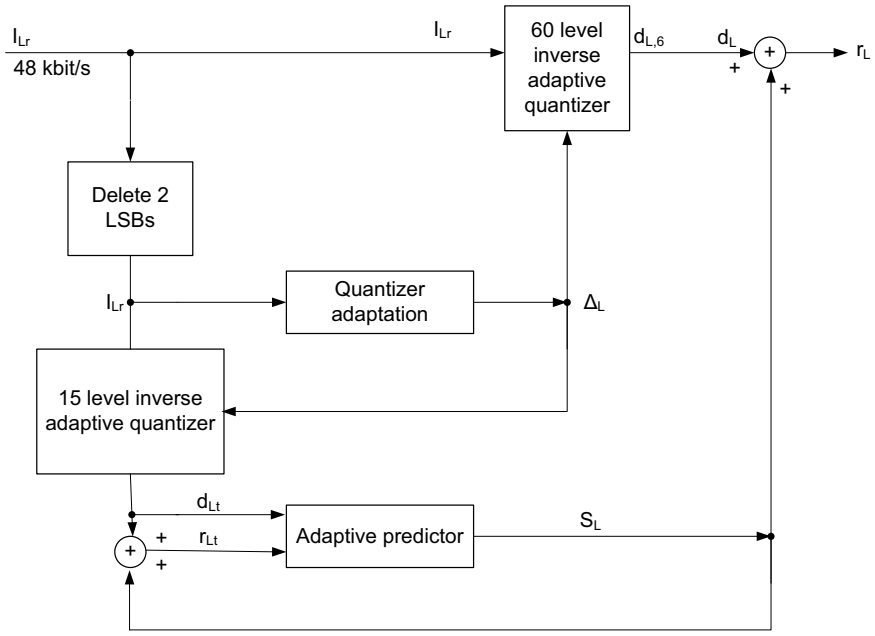


Figure 40. G.722 Lower Subband Decoder Block Diagram

This decoder output from each of the SB-ADPCM decoders is 16-bit data in m1p14 format with the sign bit repeated, sampled at half the output sample rate from the decoder. These output signals are output to the decoder QMF filterbank, which is used to combine the output signals.

NOTE: The details of the sub-blocks of the G.722 SB-ADPCM decoder are not repeated here (refer to the G.722 standard). The decoder does not support auxiliary channel information that has been inserted into the low subband data as the only mode supported is mode1, where all six bits supplied to the lower subband decoder are utilized in creating the output.

IMPORTANT: The G.722 implementation in Ezairo 7100 does not support messages inserted into the transmitted audio data.

The internal state of the SB-ADPCM decoder must be cleared prior to processing a new data stream since this state, set by previous data streams, can introduce artifacts to the G.722 output. To clear the internal state of the G.722 SB-ADPCM decoder, set the `ACB_CMD_G722_DEC_SBBND_CLR` bit from the `C_ACB_CMD` register. The G.722 SB-ADPCM decoder registers are cleared one cycle after this register bit is set.

15.4.2.2 G.722 Decoder QMF Filterbank

The G.722 decoder accelerator uses a QMF filterbank that is equivalent to the filterbank used by the G.722 encoder, as required by the G.722 specification. For information about the filterbank itself, see Section 15.4.1.1, “G.722 Encoder QMF Filterbank” on page 304.

The filterbank receives two 16-bit subband samples in m1p14 format, with the sign bit repeated to extend the sample to 16 bits. This G.722 specification requires that this data is provided at a 8 kHz sample rate and is interpolated

by a factor of two before being passed through the filterbank. The filterbank uses this input data to produce 16-bit output samples in the m1p14 format (with an extra sign bit) at the required 16 kHz sampling rate.

For a user application to implement a non-standard QMF filterbank, it can bypass the QMF filterbank by setting the ACB_CTRL_G722_DEC_FB_BYPASS bit of the C_ACB_CTRL register. If the QMF filterbank is bypassed, the output audio data is provided prior to interpolation with the high subband sample being provided in the upper half of the output interface register, and the low subband sample being provided in the lower half of the output interface register.

The internal state of the QMF filterbank must be cleared prior to processing a new data stream since this state, set by previous data streams, can introduce artifacts to the G.722 output. To clear the internal state of the G.722 QMF filterbank, set the ACB_CMD_G722_DEC_FB_CLR bit from the C_ACB_CMD register. The G.722 QMF filterbank registers are cleared one cycle after this register bit is set.

15.4.2.3 G.722 Decoder Concealment Support

The G.722 decoder accelerator provides support for an application to implement the packet loss concealment algorithms described in Appendix IV of the G.722 standard. The concealment algorithm register accesses supported by the accelerator, as required by the standard, include the following:

- The C_ACB_G722_DEC_FBIN register provides read and write access to the high and low intermediate data between the G.722 SB-ADPCM decoders and the decoder QMF filterbank.
- The ACB_CMD_G722_DEC_LSB_CNCL_CLR bit from the C_ACB_CMD register can be used to clear the DLTi registers from the lower subband decoder.
- The PLT1, PLT2, RLT1, DETL, NBL, SL, and SZL registers from the low subband decoder can be overwritten using the C_ACB_G722_DEC_PLT, C_ACB_G722_DEC_RLT, C_ACB_G722_DEC_SX, and C_ACB_G722_DEC_LSB_QUANT registers.
- The NBH and DETH registers from the high subband decoder can be overwritten using the C_ACB_G722_DEC_HSB_QUANT register.

To enable concealment mode, set the ACB_CTRL_G722_DEC_CONCL_EBL bit from the C_ACB_CTRL register. When operating in concealment mode, the accelerator produces a G722_DEC_CONCEAL_RDY interrupt when subband data is available to be overwritten according to the needs of the user application. The user application can read the current intermediate data from the C_ACB_G722_DEC_FBIN register and can replace this data before the G.722 decoder completes processing using the QMF filterbank. While held, the internal state registers from the SB-ADPCM decoders listed above can be updated by the use application.

IMPORTANT: The G.722 decoder accelerator does not directly implement any packet loss concealment algorithm.

15.4.2.4 G.722 Decoder Input and Output Interfaces

The input and output interfaces provided for the G.722 decoder accelerator coordinate the flow of samples through the accelerator. A block diagram highlighting these interfaces and how they relate to the G.722 SB-ADPCM decoders and QMF filterbank is provided in Figure 41.

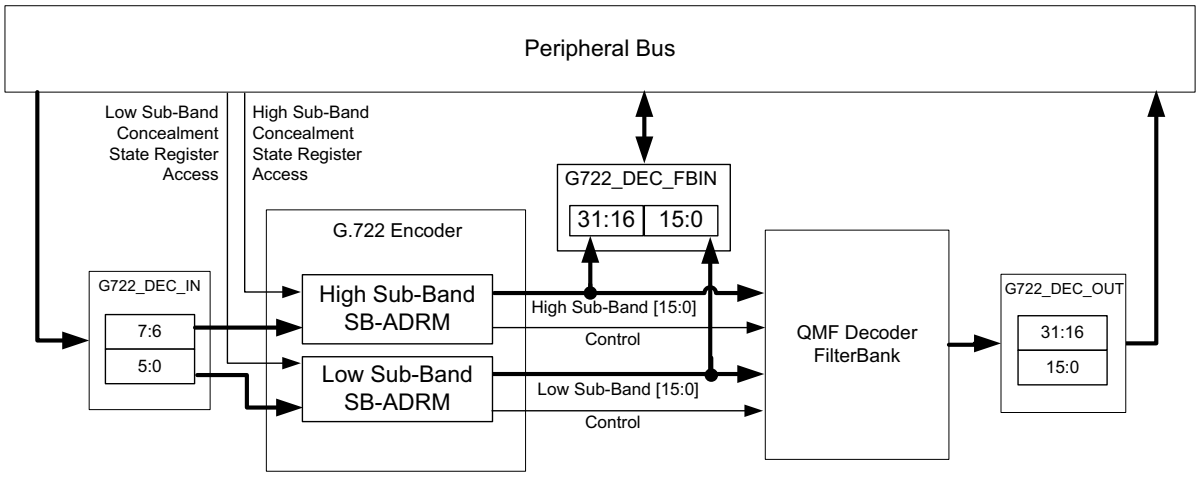


Figure 41. G.722 Decoder and ARM Cortex-M3 Processor Interface block diagram

The input interface to the G.722 decoder uses an 8-bit register, C_ACB_G722_DEC_IN, to supply the SB-ADPCM decoders with high and low subband input data. The SB-ADPCM decoders and QMF filterbank both remain idle until the input interface register has been written. A G722_DEC_IN_RDY interrupt is triggered when the G.722 decoder is enabled and idle. This interrupt indicates that the input interface is ready to accept more data.

The output interface uses a 32-bit register, C_ACB_G722_DEC_OUT, to store the decompressed data from the QMF filterbank (or a high subband and low subband output if the QMF filterbank is bypassed). When both output samples are available, the G722_DEC_OUT_RDY interrupt is triggered. After triggering this interrupt, the output interface waits until the output register is read before triggering another G722_ENC_IN_RDY interrupt indicating that the G.722 encoder is idle and ready for more data.

The G.722 standard requires output data to be formatted in a m1p14 format, where the sign bit is repeated to convert the 15-bit data to a 16-bit sample. The output data from the G.722 encoder can also be formatted in a standard frac16 (m1p15) format. To select between supporting 15-bit or 16-bit output data, configure the ACB_CTRL_G722_DEC_IN_FORMAT bit from the C_ACB_CTRL register.

If the input interface register is overwritten while the G.722 decoder accelerator is still processing the previous samples, a G722_DEC_IN_ERR interrupt is generated. In this situation, the accelerator continues its operation and disregards the new data written to the input register. To enable this interrupt, set the ACB_INT_ENABLE_G722_DEC_IN_ERR bit in the C_ACB_INT_ENABLE register.

15.4.3 CVSD Encoder

The CVSD CODEC encoder accelerator consists of two main sections: a pre-filter, and the CVSD encoder itself. To enable simple communications between the ARM Cortex-M3 processor and the accelerator, it also includes input and output interfaces.

To enable the CVSD encoder, set the ACB_CTRL_CVSD_ENC_EBL bit from the C_ACB_CTRL register.

15.4.3.1 CVSD Encoder Pre-Filter

The CVSD encoder pre-filter consists of three identical half-band low pass filters. The sampling rate at the input of the 2nd and 3rd low pass filters is doubled with respect to the previous filter. Therefore the effective output sample rate of the CVSD pre-filter is four times the pre-filters input sample rate. A block diagram showing the pre-filter is provided in Figure 42.

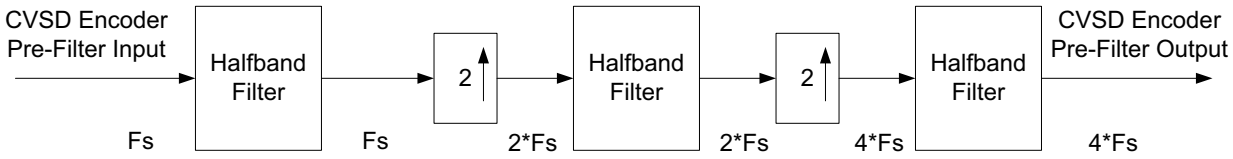


Figure 42. CVSD Encoder Pre-Filter Block Diagram

The magnitude response of each half-band filter is given in Figure 43. As a half-band filter, it has a -3 dB point at Fs/4 and applies a minimum 44 dB stop band attenuation after 1.3*(Fs/4) frequency. The filter sampling frequency controls the frequency of the samples at the input of the pre-filter block.

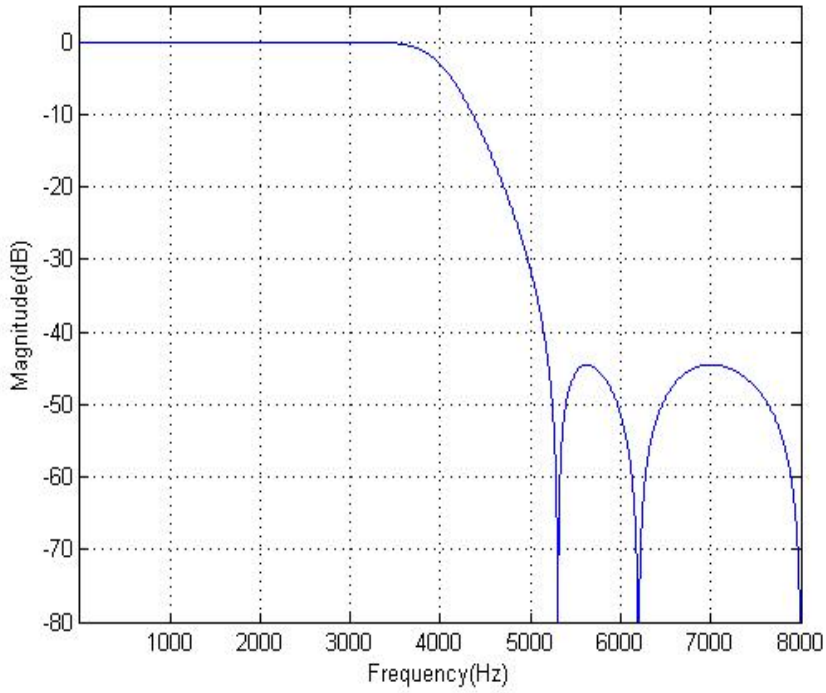


Figure 43. Magnitude Response of the Pre-Filter Half-Band Filter at a 16 kHz Sampling Rate

The cascaded half-band filters and interpolation results in the full pre-filter magnitude response plotted in Figure 44. This response curve shows the response for a 64 kHz output sample rate, which is equivalent to a 16 kHz input sample rate. As the full pre-filter magnitude response highlights, the bandwidth of the signal is limited to ~Fs/4,

where Fs is the sampling frequency at the input of the accelerator. Even though the sampling rate at the output of the pre-filter is increased by a factor of four, the input signal bandwidth is reduced by half.

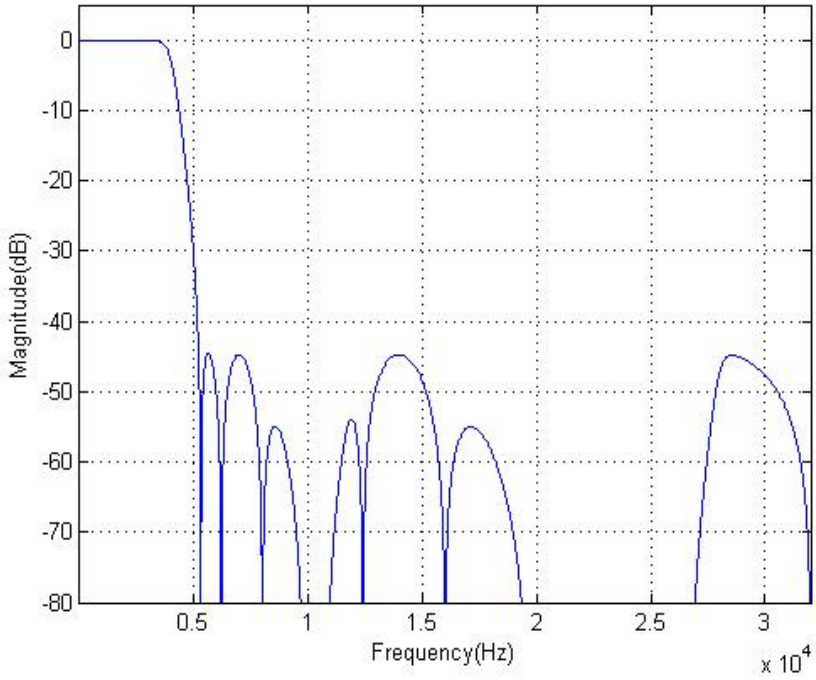


Figure 44. Magnitude Response of the Pre-Filter at a 64 kHz Output Sampling Rate

The group delay response of the pre-filter at the passband of the filter is plotted in Figure 45. The plot is generated for a 16 kHz input sample rate. The group delay is defined in terms of samples at the output sampling rate of the accelerator. For a 16 kHz input sample rate, the peak group delay of the total filter is $32.2 / 64000 = 0.5\text{ms}$. If the input sampling rate is different from 16 kHz, the passband of the filter as well as the group delay scales accordingly. For example, at a 20 kHz input sampling rate, the passband is up to 5 kHz and the peak group delay (at 5 kHz) is 0.4 ms. In addition to this mathematical delay, it takes 16 system clock cycles for the pre-filter block to generate four 16-bit interpolated samples out of one 16-bit input sample. The delay between each interpolated sample is 4 SYSCLK cycles.

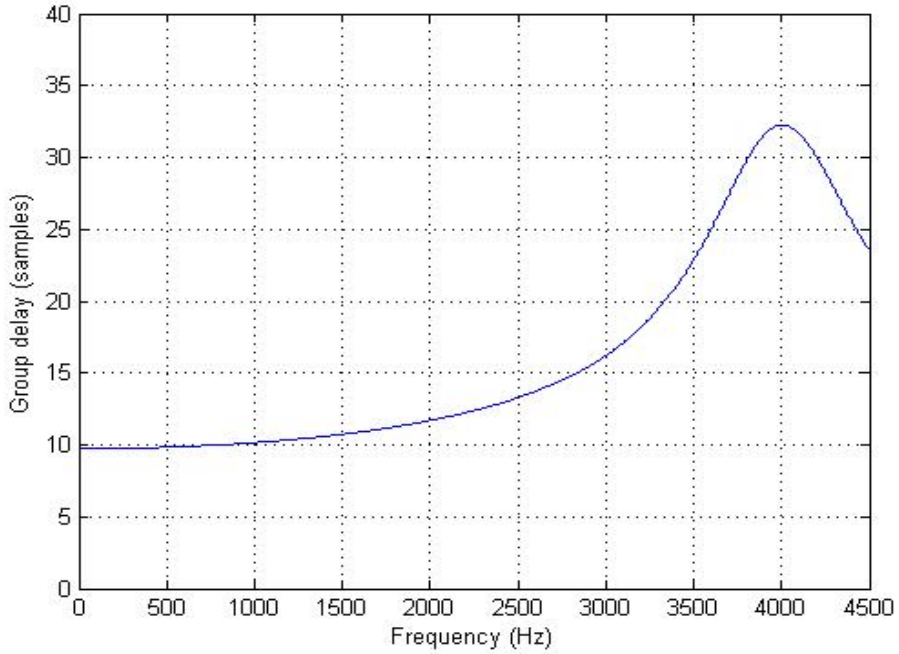


Figure 45. Pre-Filter Group Delay in Its Passband (16 kHz Input Sample Rate)

To reset or clear the internal state of the CVSD pre-filter, set the `ACB_CMD_CVSD_ENC_PREFILT_CLR` bit from the `C_ACB_CMD` register. The CVSD pre-filter must be idle to properly clear the state. As a result, if this register bit is set while the accelerator is active, it is not applied until the accelerator is finished with its processing. If a clear is pending, the `ACB_CMD_CVSD_ENC_PREFILT_CLR` bit is set when read.

15.4.3.2 CVSD Encoder

The CVSD encoder operates on the output samples from the pre-filter. The block diagram of the encoder is given in Figure 46. It consists of two major sections: the accumulator and the step size controller. The output of the accumulator is compared with the input signal. If the input sample is larger, the encoder emits a 1 bit and adds the step size to the accumulator. If the input sample is smaller, the encoder emits a 0 bit and subtracts the step size from the accumulator. The outputs are collected as the encoder output, which is also fed back to the accumulator and step size controller.

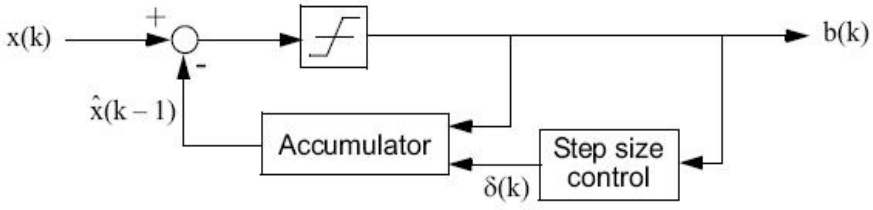


Figure 46. CVSD Encoder Block Diagram

IMPORTANT: The CVSD encoder can be configured to comply with the Bluetooth CVSD standard. See the standard's required configuration for 16-bit data in Table 62.

A block diagram for the accumulator portion of the CVSD encoder is provided in Figure 47. The accumulator multiplies the encoder output and the step size. Since the b(k) consists of a sign bit only, effectively the accumulator adds or subtracts the step size from the previous accumulator output. There is a decay factor h applied to the accumulator output. This factor decays the principal integrator and lessens the effect of transmission errors on decoded outputs.

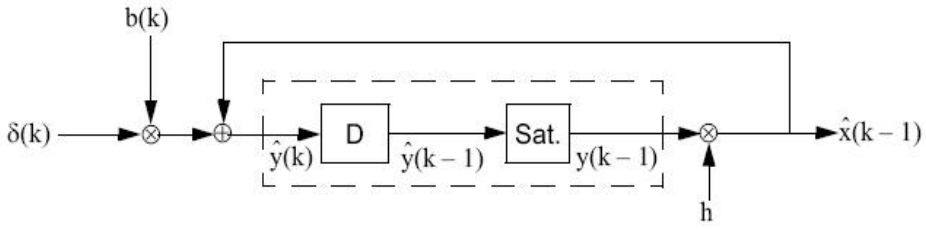


Figure 47. CVSD Encoder Accumulator

A block diagram for the step size controller is provided in Figure 48. Normally the lower feedback loop in the block diagram is active. That means the step size value is continuously decayed by a factor of beta until the minimum step size is reached. This decay factor (β) is also called the syllabic time constant. It is picked to accommodate the speech pitch changes. It also provides leakage to recover from the transmission errors. The step size controller continuously monitors the encoder output. If a sub-set (J bits) in the last output bits (K) are equal, the step size controller interferes with the decay of the step size, and increases the step size amount in steps of the minimum size, until no more J bits are consecutively being equal to each other, or the step size reaches its maximum allowed value. This mechanism prevents slope overload, which can occur when steps are too small to track the original waveform accurately.

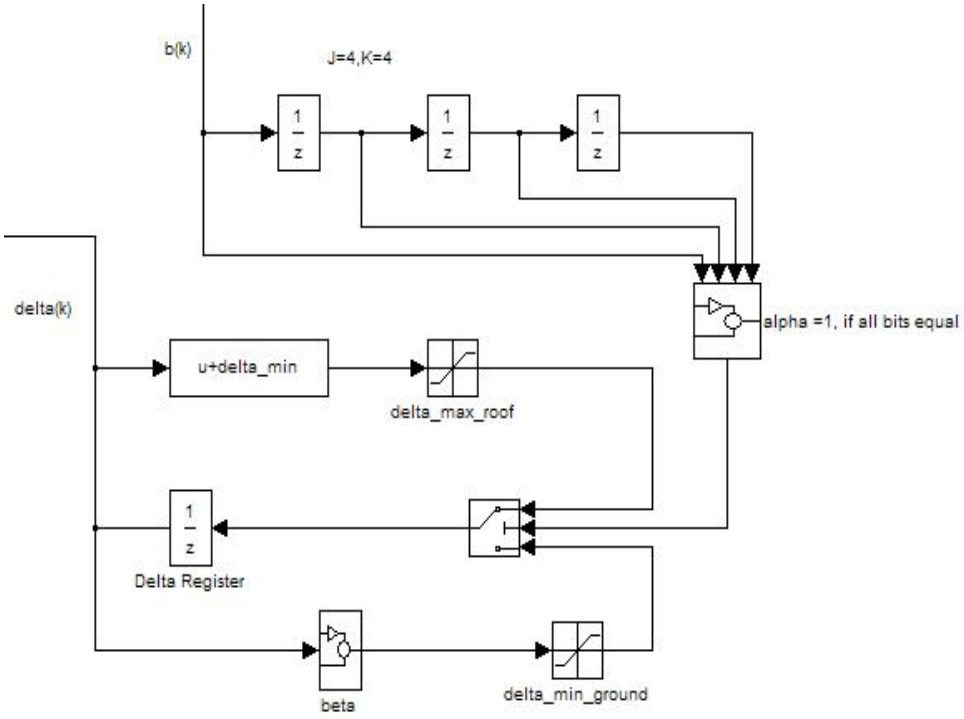


Figure 48. CVSD Encoder Step Size Controller Block Diagram

The Bluetooth standard sets strict values for all parameters used for the CVSD encoder (and decoder). The standard's required configuration for 16-bit data is provided in Table 62. This configuration is optimized for 64 kHz operation of the CVSD with a 4 kHz band limited input signal.

Table 62. Bluetooth CVSD Parameters

Parameter	Value
h	$1 - \frac{1}{32}$
β	$1 - \frac{1}{1024}$
J	4
K	4
δ_{min}	10
δ_{max}	1280
y_{min}	-2^{15} or $-2^{15} + 1$
y_{max}	$2^{15} - 1$

To extend the functionality to different sampling rates, some programmability is introduced in β, h and J values.

- β is the decay factor and is based on a bit shift by a number of bits. The CVSD encoder supports the configuration of 8 to 11 bit shifts, configurable by using the `ACB_CTRL_CVSD_ENC_BETA` bit field from the `C_ACB_CTRL` register. The Bluetooth specification requires β to be defined to provide a 10-bit shift.
- h is defined in a similar fashion to β, and can be configured for a 4 to 7 bit shift using the `ACB_CTRL_CVSD_ENC_H` bit field from the `C_ACB_CTRL` register. The Bluetooth specification requires h to be defined to provide a 5-bit shift.
- J is the number of consecutive CVSD encoder inputs to check for zeros before applying a decay to the step size used for the encoder output. This can be configured for 3, 4 or 5 consecutive encoder outputs. The Bluetooth standard requires J to be set to 4.

The CVSD encoder takes a single system clock cycle to generate its 1-bit output from the 16-bit input signal. The resulting sample rate is 64 kbps, assuming a 16 kHz input signal to the pre-filter.

To reset or clear the internal state of the CVSD encoder, set the `ACB_CMD_CVSD_ENC_CORE_CLR` bit from the `C_ACB_CMD` register. The CVSD encoder must be idle to properly clear the state. As a result, if this register bit is set while the accelerator is active, it will not be applied until the accelerator is finished with its processing. If a clear is pending, the `ACB_CMD_CVSD_ENC_CORE_CLR` bit will be set when read.

15.4.3.3 CVSD Encoder Input and Output Interfaces

The input and output interfaces provided for the CVSD encoder accelerator coordinate the flow of samples through the accelerator. A block diagram highlighting these interfaces and how they relate to the CVSD pre-filter and encoder is provided in Figure 49.

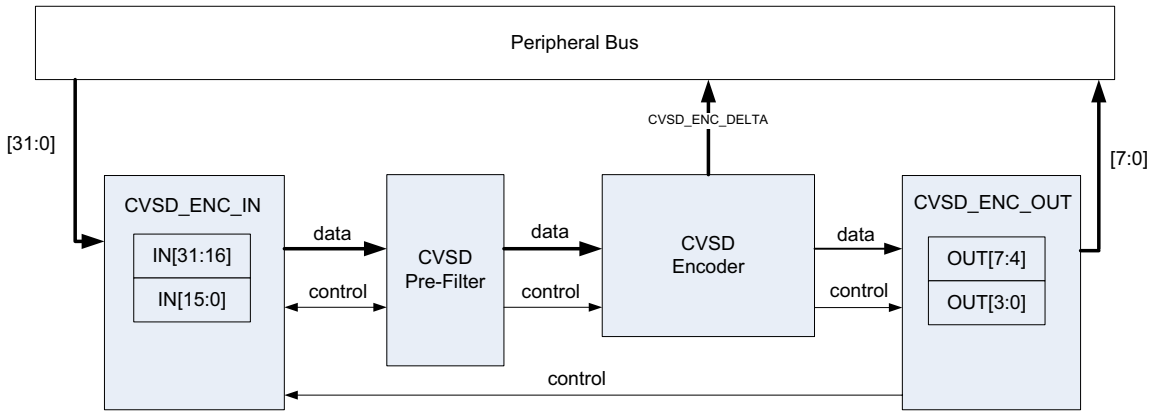


Figure 49. CVSD Encoder with Input and Output Interfaces

The input interface uses a 32-bit register, C_ACB_CVSD_ENC_IN, to supply the pre-filter with two 16-bit audio samples. The pre-filter and encoder remain idle until both of the 16-bit sample fields in the input interface register have been updated. A CVSD_ENC_IN_RDY interrupt is triggered when the CVSD encoder is enabled and idle. This interrupt indicates that the input interface is ready to accept more data.

The output interface uses an 8-bit register, C_ACB_CVSD_ENC_OUT, to store the single bit compressed data from the encoder. When a full set of eight compressed bits are captured, the CVSD_ENC_OUT_RDY interrupt is triggered. After triggering this interrupt, the output interface waits until the output register is read before triggering another CVSD_ENC_IN_RDY interrupt indicating that the CVSD encoder is idle and ready for more data.

When data is provided to the CVSD encoder, the input interface transfers the low sample from the CVSD input to the pre-filter. Once the pre-filter has completed processing of the lower sample, the input interface, the upper sample is transferred to the pre-filter. For each sample generated by the pre-filter, the CVSD encoder provides one bit which is captured into the output interface register starting from either the MSB or the LSB of the output register (i.e., filled from bit 7 to 0 if configured for MSB first or bits 0 to 7 if configured for LSB first), as specified using the ACB_CTRL_CVSD_ENC_OUT_PTR_DIR bit from the C_ACB_CTRL register.

If the input interface register is overwritten before the CVSD encoder is ready to accept more input information, a CVSD_ENC_IN_ERR interrupt is generated. If no corrective action is taken when this error occurs, some samples will not be processed. To enable this interrupt, set the ACB_INT_ENABLE_CVSD_ENC_IN_ERR bit in the C_ACB_INT_ENABLE register.

In addition to the input and output interfaces, the CVSD encoder provides information on the current step size (delta) used by the step size controller through the C_ACB_CVSD_DELTA register.

15.4.4 CVSD Decoder

The CVSD CODEC decoder accelerator consists of two main sections: the CVSD decoder itself and the post-filter. To enable simple communications between the ARM Cortex-M3 processor and the accelerator it also includes input and output interfaces.

To enable the CVSD decoder, set the ACB_CTRL_CVSD_DEC_EBL bit from the C_ACB_CTRL register.

15.4.4.1 CVSD Decoder

The CVSD decoder takes in a 1-bit sample and generates a 16-bit audio sample out of this input sample. A block diagram of the decoder is shown in Figure 50. It consists of two major sections: the accumulator and step size controller. The single bit decoder input b(k) is fed into the accumulator as well as the step size controller. The accumulator combines this information with the step size and sends the new accumulated sample out.

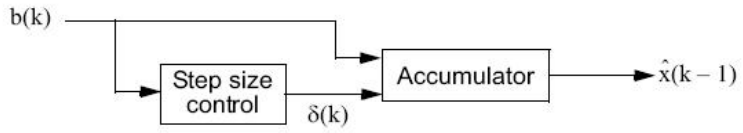


Figure 50. CVSD Decoder Block Diagram

IMPORTANT: The CVSD decoder can be configured to comply with the Bluetooth CVSD standard. For more information, see https://www.bluetooth.org/en-us/specification/adopted-specifications.

The decoder is implemented and must be configured as a mirror image of the encoder to reconstruct the audio sample with reasonable fidelity. Because of this, the accumulator and step size controller structures are implemented in the exactly the same way as the CVSD encoder's accumulator and step size controller (see Figure 47 on page 315 and Figure 48 on page 315). All configuration options described for the CVSD encoder also apply to the CVSD decoder.

The CVSD decoder can be configured using the `C_ACB_CTRL` register, setting:

- b using the `ACB_CTRL_CVSD_DEC_BETA` bit field
- h using the `ACB_CTRL_CVSD_DEC_H` bit field
- J using the `ACB_CTRL_CVSD_DEC_J` bit field

It takes a single system clock cycle for the decoder to finish its processing.

To reset or clear the internal state of the CVSD decoder, set the `ACB_CMD_CVSD_DEC_CORE_CLR` bit from the `C_ACB_CMD` register. The CVSD decoder must be idle to properly clear the state. As a result, if this register bit is set while the accelerator is active, it will not be applied until the accelerator is finished with its processing. If a clear is pending, `ACB_CMD_CVSD_DEC_CORE_CLR` bit is set when read.

15.4.4.2 CVSD Decoder Post-Filter

The CVSD decoder post-filter consists of three identical stages of half-band low pass IIR filters. Each low pass filter is identical to the half-band filter stages used to implement the CVSD encoder pre-filter (see Section 15.4.3.1, "CVSD Encoder Pre-Filter"). The sampling rate at the input of the post-filter is four times higher than at the output of the post-filter. The block diagram of the post-filter is given in Figure 51.

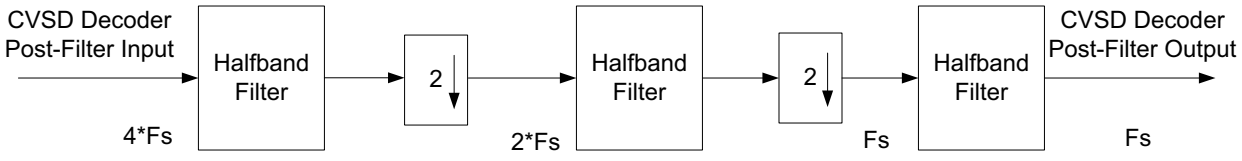


Figure 51. CVSD Decoder Post-Filter Block Diagram

The magnitude and group delay responses of the post-filter structure is exactly equal to the responses of the CVSD encoder pre-filter (see Figure 44 on page 313 and Figure 45 on page 314). It takes 16 system clock cycles for the post-filter to generate a single decimated output sample from four input samples generated by the CVSD decoder.

To reset or clear the internal state of the CVSD post-filter, set the ACB_CMD_CVSD_DEC_POSTFILT_CLR bit from the C_ACB_CMD register. The CVSD post-filter must be idle to properly clear the state. As a result, if this register bit is set while the accelerator is active, it is not applied until the accelerator is finished with its processing. If a clear is pending, ACB_CMD_CVSD_DEC_POSTFILT_CLR bit is set when read.

15.4.4.3 CVSD Decoder Input and Output Interfaces

The input and output interfaces provided for the CVSD decoder accelerator coordinate the flow of samples through the accelerator. A block diagram highlighting these interfaces and how they relate to the CVSD decoder and post-filter is provided in Figure 52.

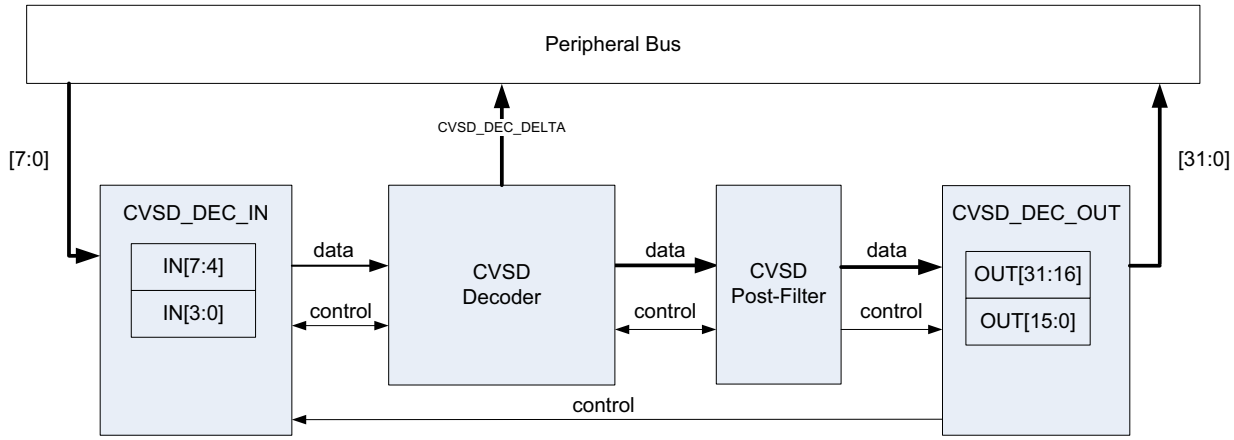


Figure 52. CVSD Decoder with Input and Output Interface

The input interface uses an 8-bit register, C_ACB_CVSD_DEC_IN, to supply data to the CVSD decoder. The decoder and post-filter remain idle until this register has been updated. A CVSD_DEC_IN_RDY interrupt is triggered when the CVSD decoder is enabled and idle. This interrupt indicates that the input interface is ready to accept more data.

The output interface uses a 32-bit register, C_ACB_CVSD_DEC_OUT, to store the two decimated output samples the CVSD post-filter generates from the input samples. When both samples have been updated, the CVSD_DEC_OUT_RDY interrupt is triggered. After triggering this interrupt the output interface waits until the output register is read before triggering another CVSD_DEC_IN_RDY interrupt indicating that the CVSD decoder is idle and ready for more data.

When data is provided to the CVSD decoder, the input interface transfers the data starting with either the MSB or the LSB of the input register (i.e., read from bit 7 to 0 if configured for MSB first or bits 0 to 7 if configured for LSB first), as specified using the ACB_CTRL_CVSD_DEC_IN_PTR_DIR bit from the C_ACB_CTRL register. The data output from the CVSD post-filter updates first the lower sample of the output interface, then updates the upper sample of the output interface.

If the input interface register is overwritten before the CVSD decoder is ready to accept more input information, a CVSD_DEC_IN_ERR interrupt is generated. If no corrective action is taken when this error occurs, processing of some samples will be skipped. To enable this interrupt, set the ACB_INT_ENABLE_CVSD_DEC_IN_ERR bit in the C_ACB_INT_ENABLE register.

15.4.5 Control and Configuration Registers

Register Name	Register Description	Address
ACB_INT_STATUS C_ACB_INT_STATUS	ACB interrupt Status Register	0x40000100 P:0x3040
ACB_INT_ENABLE C_ACB_INT_ENABLE	ACB interrupt Mask Register	0x40000104 P:0x3041
ACB_CTRL C_ACB_CTRL	ACB Control Registers	0x40000108 P:0x3042
ACB_CMD C_ACB_CMD	ACB Commands	0x4000010C P:0x3043
ACB_CVSD_ENC_IN C_ACB_CVSD_ENC_IN	ACB CVSD encoder input data register	0x40000110 P:0x3044
ACB_CVSD_ENC_OUT C_ACB_CVSD_ENC_OUT	ACB CVSD encoder output data register	0x40000114 P:0x3045
ACB_CVSD_DEC_IN C_ACB_CVSD_DEC_IN	ACB CVSD decoder input data register	0x40000118 P:0x3046
ACB_CVSD_DEC_OUT C_ACB_CVSD_DEC_OUT	ACB CVSD decoder output data register	0x4000011C P:0x3047
ACB_CVSD_DELTA C_ACB_CVSD_DELTA	ACB CVSD delta registers	0x40000120 P:0x3048
ACB_G722_ENC_IN C_ACB_G722_ENC_IN	ACB G.722 encoder input data register	0x40000124 P:0x3049
ACB_G722_ENC_OUT C_ACB_G722_ENC_OUT	ACB G.722 encoder output data register	0x40000128 P:0x304A
ACB_G722_DEC_IN C_ACB_G722_DEC_IN	ACB G.722 decoder input data register	0x4000012C P:0x304B
ACB_G722_DEC_OUT C_ACB_G722_DEC_OUT	ACB G.722 decoder output data register	0x40000130 P:0x304C
ACB_G722_DEC_FBIN C_ACB_G722_DEC_FBIN	ACB G.722 decoder filterbank input registers	0x40000134 P:0x304D
ACB_G722_DEC_PLT C_ACB_G722_DEC_PLT	ACB G.722 decoder lower subband decoder PLT registers	0x40000138 P:0x304E
ACB_G722_DEC_RLT C_ACB_G722_DEC_RLT	ACB G.722 decoder lower subband decoder RLT register	0x4000013C P:0x304F
ACB_G722_DEC_SX C_ACB_G722_DEC_SX	ACB G.722 decoder lower subband decoder SL and SLZ pipeline registers	0x40000140 P:0x3050
ACB_G722_DEC_LSB_QUANT C_ACB_G722_DEC_LSB_QUANT	ACB G.722 decoder lower subband quantizer registers	0x40000144 P:0x3051

Register Name	Register Description	Address
ACB_G722_DEC_HSB_QUANT C_ACB_G722_DEC_HSB_QUANT	ACB G.722 decoder higher subband quantizer registers	0x40000148 P:0x3052

15.4.5.1 ACB_INT_STATUS, C_ACB_INT_STATUS Settings

Bit Field	Field Name	Description
24	CVSD_DEC_IN_ERR	The CVSD decoder input interface error interrupt status
16	CVSD_ENC_IN_ERR	The CVSD encoder input interface error interrupt status
9	G722_DEC_CONCEAL_RDY	The G.722 decoder concealment flag interrupt status
8	G722_DEC_IN_ERR	The G.722 decoder input interface error interrupt status
0	G722_ENC_IN_ERR	The G.722 encoder input interface error interrupt status

Field Name	Value Symbol	Value Description	Hex Value
CVSD_DEC_IN_ERR	INT_ACK_CVSD_DEC_IN_ERR	Acknowledge the interrupt	0x1
	INT_IDLE_CVSD_DEC_IN_ERR	This source has not set an interrupt	0x0*
	INT_PENDING_CVSD_DEC_IN_ERR	The CVSD decoder input interface register was overwritten before the accelerator finished its processing	0x1
CVSD_ENC_IN_ERR	INT_ACK_CVSD_ENC_IN_ERR	Acknowledge the interrupt	0x1
	INT_IDLE_CVSD_ENC_IN_ERR	This source has not set an interrupt	0x0*
	INT_PENDING_CVSD_ENC_IN_ERR	The CVSD encoder input interface register was overwritten before the accelerator finished its processing	0x1
G722_DEC_CONCEAL_RDY	INT_ACK_G722_DEC_CONCEAL_RDY	Acknowledge the interrupt	0x1
	INT_IDLE_G722_DEC_CONCEAL_RDY	This source has not set an interrupt	0x0*
	INT_PENDING_G722_DEC_CONCEAL_RDY	The G.722 decoder finished the decoding and is waiting for the subband signal update from the ARM Cortex-M3 processor to continue with filterbank	0x1
G722_DEC_IN_ERR	INT_ACK_G722_DEC_IN_ERR	Acknowledge the interrupt	0x1
	INT_IDLE_G722_DEC_IN_ERR	This source has not set an interrupt	0x0*
	INT_PENDING_G722_DEC_IN_ERR	The G.722 decoder input interface register was overwritten before the accelerator finished its processing	0x1
G722_ENC_IN_ERR	INT_ACK_G722_ENC_IN_ERR	Acknowledge the interrupt	0x1
	INT_IDLE_G722_ENC_IN_ERR	This source has not set an interrupt	0x0*
	INT_PENDING_G722_ENC_IN_ERR	The G.722 encoder input interface register was overwritten before the accelerator finished its processing	0x1

15.4.5.2 ACB_INT_ENABLE, C_ACB_INT_ENABLE Settings

Bit Field	Field Name	Description
24	CVSD_DEC_IN_ERR	The CVSD decoder input interface error interrupt mask
16	CVSD_ENC_IN_ERR	The CVSD encoder input interface error interrupt mask
9	G722_DEC_CONCEAL_RDY	The G.722 decoder concealment flag interrupt mask
8	G722_DEC_IN_ERR	The G.722 decoder input interface error interrupt mask
0	G722_ENC_IN_ERR	The G.722 encoder input interface error interrupt mask

Field Name	Value Symbol	Value Description	Hex Value
CVSD_DEC_IN_ERR	INT_DIS_CVSD_DEC_IN_ERR	This source cannot set an interrupt	0x0*
	INT_EBL_CVSD_DEC_IN_ERR	This source can set the ACB_COM interrupt line	0x1
CVSD_ENC_IN_ERR	INT_DIS_CVSD_ENC_IN_ERR	This source cannot set an interrupt	0x0*
	INT_EBL_CVSD_ENC_IN_ERR	This source can set the ACB_COM interrupt line	0x1
G722_DEC_CONCEAL_RDY	INT_DIS_G722_DEC_CONCEAL_RDY	This source cannot set an interrupt	0x0*
	INT_EBL_G722_DEC_CONCEAL_RDY	This source can set the ACB_COM interrupt line	0x1
G722_DEC_IN_ERR	INT_DIS_G722_DEC_IN_ERR	This source cannot set an interrupt	0x0*
	INT_EBL_G722_DEC_IN_ERR	This source can set the ACB_COM interrupt line	0x1
G722_ENC_IN_ERR	INT_DIS_G722_ENC_IN_ERR	This source cannot set an interrupt	0x0*
	INT_EBL_G722_ENC_IN_ERR	This source can set the ACB_COM interrupt line	0x1

15.4.5.3 ACB_CTRL, C_ACB_CTRL Settings

Bit Field	Field Name	Description
31	CVSD_DEC_IN_PTR_DIR	CVSD decoder input register pointer update direction
30:29	CVSD_DEC_H	CVSD decoder h parameter
28:27	CVSD_DEC_BETA	CVSD decoder beta parameter
26:25	CVSD_DEC_J	CVSD decoder J parameter
24	CVSD_DEC_EBL	Enable/disable the CVSD Decoder
23	CVSD_ENC_OUT_PTR_DIR	CVSD encoder output register pointer update direction
22:21	CVSD_ENC_H	CVSD encoder h parameter
20:19	CVSD_ENC_BETA	CVSD encoder beta parameter
18:17	CVSD_ENC_J	CVSD encoder J parameter
16	CVSD_ENC_EBL	Enable/disable the CVSD encoder
11	G722_DEC_OUT_FORMAT	G.722 decoder output format
10	G722_DEC_FB_BYPASS	G.722 decoder filterbank bypass control
9	G722_DEC_CONCL_EBL	G.722 decoder concealment mode control

Bit Field	Field Name	Description
8	G722_DEC_EBL	Enable/disable the G.722 decoder
2	G722_ENC_IN_FORMAT	G.722 encoder input data format
1	G722_ENC_FB_BYPASS	G.722 encoder filterbank bypass control
0	G722_ENC_EBL	Enable/disable the G.722 encoder

Field Name	Value Symbol	Value Description	Hex Value
CVSD_DEC_IN_PTR_DIR	CVSD_DEC_IN_INC	Decoder input pointer increments by 1 bit	0x0*
	CVSD_DEC_IN_DEC	Decoder input pointer decrements by 1 bit	0x1
CVSD_DEC_H	CVSD_DEC_H_4	Accumulator decay factor (h) is set to 1-(1/16)	0x0
	CVSD_DEC_H_5	Accumulator decay factor (h) set to 1-(1/32). Bluetooth standard	0x1*
	CVSD_DEC_H_6	Accumulator decay factor (h) set to 1-(1/64)	0x2
	CVSD_DEC_H_7	Accumulator decay factor (h) set to 1-(1/128)	0x3
CVSD_DEC_BETA	CVSD_DEC_BETA_8	Step size decay factor (beta) is set to 1-(1/256)	0x0
	CVSD_DEC_BETA_9	Step size decay factor (beta) is set to 1-(1/512)	0x1
	CVSD_DEC_BETA_10	Step size decay factor (beta) is set to 1-(1/1024). Bluetooth standard	0x2*
	CVSD_DEC_BETA_11	Step size decay factor (beta) is set to 1-(1/2048)	0x3
CVSD_DEC_J	CVSD_DEC_J_3	Alpha is set if 3 consecutive samples are equal	0x0
	CVSD_DEC_J_4	Alpha is set if 4 consecutive samples are equal. Bluetooth standard	0x1*
	CVSD_DEC_J_5	Alpha is set if 5 consecutive samples are equal	0x2
CVSD_DEC_EBL	CVSD_DEC_DISABLE	Disable the CVSD decoder	0x0*
	CVSD_DEC_ENABLE	Enable the CVSD decoder	0x1
CVSD_ENC_OUT_PTR_DIR	CVSD_ENC_OUT_INC	Encoder output pointer increments by 1 bit	0x0*
	CVSD_ENC_OUT_DEC	Encoder output pointer decrements by 1 bit	0x1
CVSD_ENC_H	CVSD_ENC_H_4	Accumulator decay factor (h) is set to 1-(1/16)	0x0
	CVSD_ENC_H_5	Accumulator decay factor (h) set to 1-(1/32). Bluetooth standard	0x1*
	CVSD_ENC_H_6	Accumulator decay factor (h) set to 1-(1/64)	0x2
	CVSD_ENC_H_7	Accumulator decay factor (h) set to 1-(1/128)	0x3

Field Name	Value Symbol	Value Description	Hex Value
CVSD_ENC_BETA	CVSD_ENC_BETA_8	Step size decay factor (beta) is set to 1-(1/256)	0x0
	CVSD_ENC_BETA_9	Step size decay factor (beta) is set to 1-(1/512)	0x1
	CVSD_ENC_BETA_10	Step size decay factor (beta) is set to 1-(1/1024). Bluetooth standard	0x2*
	CVSD_ENC_BETA_11	Step size decay factor (beta) is set to 1-(1/2048)	0x3
CVSD_ENC_J	CVSD_ENC_J_3	Alpha is set if 3 consecutive samples are equal	0x0
	CVSD_ENC_J_4	Alpha is set if 4 consecutive samples are equal. Bluetooth standard	0x1*
	CVSD_ENC_J_5	Alpha is set if 5 consecutive samples are equal	0x2
CVSD_ENC_EBL	CVSD_ENC_DISABLE	Disable the CVSD encoder	0x0*
	CVSD_ENC_ENABLE	Enable the CVSD encoder	0x1
G722_DEC_OUT_FORMAT	G722_DEC_OUT_16BIT	The m1p14 output data is aligned to m1p15 format (zero padded)	0x0*
	G722_DEC_OUT_15BIT	The m1p14 output data is aligned to m2p14 format (sign extended)	0x1
G722_DEC_FB_BYPASS	G722_DEC_FB_ENABLE	The G.722 decoder filterbank is active.	0x0*
	G722_DEC_FB_BYPASS	The G.722 decoder filterbank is bypassed; its input is passed to its output as is.	0x1
G722_DEC_CONCL_EBL	G722_CONCL_DISABLE	The G.722 decoder is in normal operation mode.	0x0*
	G722_CONCL_ENABLE	The G.722 decoder is in concealment mode. It waits for ARM Cortex-M3 processor to update the filterbank inputs after post-processing the subband decoder outputs.	0x1
G722_DEC_EBL	G722_DEC_DISABLE	Disable the G.722 decoder	0x0*
	G722_DEC_ENABLE	Enable the G.722 decoder	0x1
G722_ENC_IN_FORMAT	G722_ENC_IN_16BIT	The input data is in m1p15 format. It is internally quantized to m2p14 format (standard)	0x0*
	G722_ENC_IN_15BIT	The input data is in m2p14 format. It is internally used as is	0x1
G722_ENC_FB_BYPASS	G722_ENC_FB_ENABLE	The G.722 encoder filterbank is active.	0x0*
	G722_ENC_FB_BYPASS	The G.722 encoder filterbank is bypassed; its input is passed to its output as is.	0x1
G722_ENC_EBL	G722_ENC_DISABLE	Disable the G.722 encoder	0x0*
	G722_ENC_ENABLE	Enable the G.722 encoder	0x1

15.4.5.4 ACB_CMD, C_ACB_CMD Settings

Bit Field	Field Name	Description
25	CVSD_DEC_CORE_CLR	CVSD decoder core section clear
24	CVSD_DEC_POSTFILT_CLR	CVSD decoder post-filter section clear
17	CVSD_ENC_CORE_CLR	CVSD encoder core section clear
16	CVSD_ENC_PREFILT_CLR	CVSD encoder pre-filter section clear
10	G722_DEC_SBBND_CLR	G.722 subband decoders section clear
9	G722_DEC_FB_CLR	G.722 decoder filterbank section clear
8	G722_DEC_LSB_CNCL_CLR	G.722 decoder lower subband concealment clear
1	G722_ENC_SBBND_CLR	G.722 subband encoders section clear
0	G722_ENC_FB_CLR	G.722 encoder filterbank section clear

Field Name	Value Symbol	Value Description	Hex Value
CVSD_DEC_CORE_CLR	CVSD_DEC_CORE_CLEAR	All registers will be set to their reset value (when the accelerator is idle)	0x1
CVSD_DEC_POSTFILT_CLR	CVSD_DEC_POSTFILT_CLEAR	All registers will be set to their reset value (when the accelerator is idle)	0x1
CVSD_ENC_CORE_CLR	CVSD_ENC_CORE_CLEAR	All registers will be set to their reset value (when the accelerator is idle)	0x1
CVSD_ENC_PREFILT_CLR	CVSD_ENC_PREFILT_CLEAR	All registers will be set to their reset value (when the accelerator is idle)	0x1
G722_DEC_SBBND_CLR	G722_DEC_SBBND_CLEAR	All registers will be set to their reset value (when the accelerator is idle)	0x1
G722_DEC_FB_CLR	G722_DEC_FB_CLEAR	All registers will be set to their reset value (when the accelerator is idle)	0x1
G722_DEC_LSB_CNCL_CLR	G722_DEC_LSB_CNCL_CLEAR	Decoder lower subband register is cleared to their reset values	0x1
G722_ENC_SBBND_CLR	G722_ENC_SBBND_CLEAR	All registers will be set to their reset value (when the accelerator is idle)	0x1
G722_ENC_FB_CLR	G722_ENC_FB_CLEAR	All registers will be set to their reset value (when the accelerator is idle)	0x1

15.4.5.5 ACB_CVSD_ENC_IN, C_ACB_CVSD_ENC_IN Settings

Bit Field	Field Name	Description
31:16	CVSD_ENC_IN1	Audio sample 1
15:0	CVSD_ENC_IN0	Audio sample 0

15.4.5.6 ACB_CVSD_ENC_OUT, C_ACB_CVSD_ENC_OUT Settings

Bit Field	Field Name	Description
7:4	CVSD_ENC_OUT1	Compressed CVSD encoder output 1
3:0	CVSD_ENC_OUT0	Compressed CVSD encoder output 0

15.4.5.7 ACB_CVSD_DEC_IN, C_ACB_CVSD_DEC_IN Settings

Bit Field	Field Name	Description
7:4	CVSD_DEC_IN1	Compressed sample 1
3:0	CVSD_DEC_IN0	Compressed sample 0

15.4.5.8 ACB_CVSD_DEC_OUT, C_ACB_CVSD_DEC_OUT Settings

Bit Field	Field Name	Description
31:16	CVSD_DEC_OUT1	Decompressed audio sample 1
15:0	CVSD_DEC_OUT0	Decompressed audio sample 0

15.4.5.9 ACB_CVSD_DELTA, C_ACB_CVSD_DELTA Settings

Bit Field	Field Name	Description
31:16	CVSD_ENC_DELTA	The step size register inside the encoder
15:0	CVSD_DEC_DELTA	The step size register inside the decoder

15.4.5.10 ACB_G722_ENC_IN, C_ACB_G722_ENC_IN Settings

Bit Field	Field Name	Description
31:16	G722_ENC_IN1	Audio sample 1
15:0	G722_ENC_IN0	Audio sample 0

15.4.5.11 ACB_G722_ENC_OUT, C_ACB_G722_ENC_OUT Settings

Bit Field	Field Name	Description
7:0	G722_ENC_OUT	Compressed G.722 encoder output

15.4.5.12 ACB_G722_DEC_IN, C_ACB_G722_DEC_IN Settings

Bit Field	Field Name	Description
7:0	G722_DEC_IN	G.722 decoder input to decompress

15.4.5.13 ACB_G722_DEC_OUT, C_ACB_G722_DEC_OUT Settings

Bit Field	Field Name	Description
31:16	G722_DEC_OUT1	Audio sample 1
15:0	G722_DEC_OUT0	Audio sample 0

15.4.5.14 ACB_G722_DEC_FBIN, C_ACB_G722_DEC_FBIN Settings

Bit Field	Field Name	Description
31:16	G722_DEC_RH	Higher subband decoder output to the QMF
15:0	G722_DEC_RL	Lower subband decoder output to the QMF

15.4.5.15 ACB_G722_DEC_PLT, C_ACB_G722_DEC_PLT Settings

Bit Field	Field Name	Description
31	G722_DEC_LSB_PLT2	Decoder lower subband PLT2 sign register
15	G722_DEC_LSB_PLT1	Decoder lower subband PLT1 sign register

15.4.5.16 ACB_G722_DEC_RLT, C_ACB_G722_DEC_RLT Settings

Bit Field	Field Name	Description
15:0	G722_DEC_LSB_RLT1	Decoder lower subband RLT1 register

15.4.5.17 ACB_G722_DEC_SX, C_ACB_G722_DEC_SX Settings

Bit Field	Field Name	Description
31:16	G722_DEC_LSB_SZL	Decoder lower subband SZL pipeline register
15:0	G722_DEC_LSB_SL	Decoder lower subband SL pipeline register

15.4.5.18 ACB_G722_DEC_LSB_QUANT, C_ACB_G722_DEC_LSB_QUANT Settings

Bit Field	Field Name	Description
30:18	G722_DEC_LSB_DETL	Decoder lower subband DETL register
14:0	G722_DEC_LSB_NBL	Decoder lower subband NBL register

15.4.5.19 ACB_G722_DEC_HSB_QUANT, C_ACB_G722_DEC_HSB_QUANT Settings

Bit Field	Field Name	Description
30:18	G722_DEC_HSB_DETH	Decoder higher subband DETH register
14:0	G722_DEC_HSB_NBH	Decoder higher subband NBH register

15.5 Direct Memory Access (DMA) Controller

15.5.1 Introduction

The direct memory access (DMA) controller module allows background transfers between peripherals and memory without core intervention. This allows the system core to be used for other computational needs while allowing high-speed sustained transfers to and from the peripherals. The DMA is connected to the ARM Cortex-M3 core, the ARM Cortex-M3 processor's peripherals and interfaces, and the ARM Cortex-M3 processor's data memory via four independent channels, as shown in Figure 53.

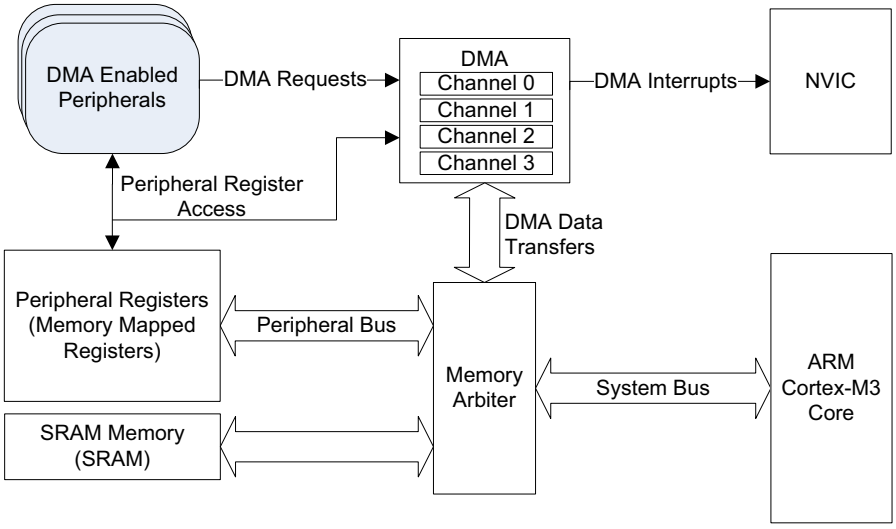


Figure 53. DMA Overview

15.5.2 DMA Channel Configuration

The DMA has four independently configurable channels. To enable or disable a DMA channel, configure the `DMA_CH*_CTRL0_ENABLE` bit from the `C_DMA_CH*_CTRL0` register. If the DMA channel is in the midst of an operation when it is disabled, the operation is aborted. All pending bus requests are subsequently aborted when the channel is disabled.

Other DMA channel configurations include:

Source Address Configuration

The `C_DMA_CH*_SRC_BASE_ADDR` register defines the base address for source data for a DMA channel.

The `C_DMA_CH*_NEXT_SRC_ADDR` register contains a calculated value that indicates the address where the next word will be read from. This pointer is updated with each word transferred, based on the base address, current transfer count (provided as `C_DMA_CH*_WORD_CNT`), and other source address configurations. Since the base source address is not modified during a transfer, the increment setting only affects the calculation of the next address.

NOTE: When the DMA channel is disabled, the next address is always the base address.

Each transfer can be configured to use a static location for the source data, or to increment the source data pointer after every word transferred. Configure this mode using the `DMA_CH*_CTRL0_SRC_ADDR_INC` bit from the `C_DMA_CH*_CTRL0` register. Transfers that use a peripheral as the source typically have source incrementing disabled. Transfers that use memory as the source typically have source incrementing enabled.

If the address is incremented, the increment can be configured to be positive or negative and to use a step size of between one and four 32-bit words by setting the

DMA_CH*_CTRL0_SRC_ADDR_STEP_MODE bit and the DMA_CH*_CTRL0_SRC_ADDR_STEP_SIZE bit field from the C_DMA_CH*_CTRL0 register.

Destination Address Configuration

The C_DMA_CH*_DST_BASE_ADDR register defines the base address for destination data for a DMA channel.

The C_DMA_CH*_NEXT_DST_ADDR register contains a calculated value that indicates the address where the next word will be written. This pointer is updated with each word transferred, based on the base address, current transfer count (provided as C_DMA_CH*_WORD_CNT), and other destination address configurations. Since the base destination address is not modified during a transfer, the increment setting only affects the calculation of the next address.

NOTE: When the DMA channel is disabled, the next address is always the base address.

Each transfer can be configured to use a static location for the destination data or to increment the destination data pointer after every word transferred. Configure this mode using the DMA_CH*_CTRL0_DST_ADDR_INC bit from the C_DMA_CH*_CTRL0 register.

If the address is incremented, the increment can be configured to be positive or negative, and to use a step size of between one and four 32-bit words by setting the DMA_CH*_CTRL0_DST_ADDR_STEP_MODE bit and the DMA_CH*_CTRL0_DST_ADDR_STEP_SIZE bit field from the C_DMA_CH*_CTRL0 register.

Source and Destination Data

The source data and destination data can be configured to be interpreted as packed into a variety of different word sizes and endian ordering. For more information, see Section 15.5.3, “Word Size, Data Packing and Transfer Length” on page 330.

Transfer Length and Intermediate Transfer Counter Configuration

The DMA supports variable transfer lengths up to 2^{16} packed words (32 bits per word). The length of a transfer can be set using the DMA_CH*_CTRL1_TRANSFER_LENGTH bit field from the C_DMA_CH*_CTRL1 register, where a setting of 0 indicates a 2^{16}-word transfer.

The transfer can also be configured to indicate when a portion of the transfer has been completed by setting the DMA_CH*_CTRL1_COUNTER_INT_VALUE bit field from the C_DMA_CH*_CTRL1 register. This counter value can be set for any portion of the transfer length from one word to one word less than the overall transfer length. The counter interrupt is never triggered if the counter interrupt value is set to 0, the transfer length, or any value that exceeds the transfer length.

Each transfer can be configured to run once (linear mode) and complete, or to run repeatedly (circular mode), producing a circular buffer. When operating in circular mode, the DMA transfer repeats until explicitly stopped by the user application. Configure this mode using the DMA_CH*_CTRL0_ADDR_MODE bit from the C_DMA_CH*_CTRL0 register.

For more information about transfer length, see Section 15.5.3.3, “Transfer Length” on page 333. For more information about interrupts indicating the transfer length and counter configuration, see Section 15.5.5, “DMA Interrupt Configuration” on page 337.

Transfer Types and Peripherals Used

Each transfer is defined as one of peripheral-to-memory, memory-to-peripheral, memory-to-memory, or peripheral-to-peripheral. The transfer mode is selected using the DMA_CH*_CTRL0_TRANSFER_TYPE bit field from the C_DMA_CH*_CTRL0 register.

For transfer types that use a source peripheral, select the desired source peripheral using the DMA_CH*_CTRL0_SRC_SELECT bit field from the C_DMA_CH*_CTRL0 register.

For transfer types that use a destination peripheral, select the desired destination peripheral using the DMA_CH*_CTRL0_DST_SELECT bit field from the C_DMA_CH*_CTRL0 register.

For more information about transfer type behavior and the peripherals that can be used as sources and destinations for DMA transfers, see Section 15.5.4, “DMA Transfer Types” on page 333.

Interrupt Configuration

The interrupts used to coordinate with and control a DMA transfer can be defined using the DMA_CH*_CTRL0_*_INT_ENABLE bits from the C_DMA_CH*_CTRL0 register. For more information, see Section 15.5.5, “DMA Interrupt Configuration” on page 337.

Transfer Priority

The relative priority of this DMA channel’s transfer. For more information, see Section 15.5.6, “Channel Priority”.

IMPORTANT: The DMA uses the current counter value to calculate the next source address and next destination address. The base source address and base destination address are not changed. The starting length is not changed either. Only the counter value is modified during a transfer. This allows the DMA configuration to be reused (either explicitly through firmware or in circular mode) without rewriting the configuration register for multiple transfers.

The DMA controller contains a summary status register (C_DMA_STATUS) indicating the completion status of each DMA channel (idle or complete), and the channel’s interrupt status. The firmware can use this to quickly assess the status of all DMA channels.

15.5.3 Word Size, Data Packing and Transfer Length

15.5.3.1 Word Size

Each DMA channel can have different word sizes set for the source and destination, as shown in Table 63. The word sizes can be configured as 4-bit, 8-bit, 16-bit, or 32-bit. The native memory word size is 32-bit. Therefore, smaller word sizes can be used to optimize memory utilization when retrieving or storing data.

Table 63. DMA Word Sizes

Encoded Word Size	Word Size
0x3	4 bits
0x0	8 bits
0x1	16 bits
0x2	32 bits

15.5.3.2 Data Packing

Packing and unpacking help to reduce bus utilization. These operations are performed automatically by the DMA controller when the source and destination have different word sizes. Specifically:

- When the source word size is smaller than the destination word size, packing is used to consolidate data from multiple source word transfers.
- When the destination word size is smaller than the source word size, unpacking is used to split data into multiple destination word transfers.

For example, when reading 8-bit bytes from the I^2C interface and storing them to data memory, four 8-bit data words from the I^2C interface are packed before they are written to memory. In this example, it is most efficient to make the destination word size 32-bit to minimize bus utilization and maximize memory use efficiency.

When both the source and destination have the same word size, no packing or unpacking of data is performed.

NOTE: When reading from or writing to memory, the DMA might utilize only a portion of the memory data if the selected word sizes differ.

The packing and unpacking behavior is illustrated in Figure 54. The figure provides a mapping of how data is transferred between source and destination for:

- All source and destination word size combinations
- Both big and little endian transfers

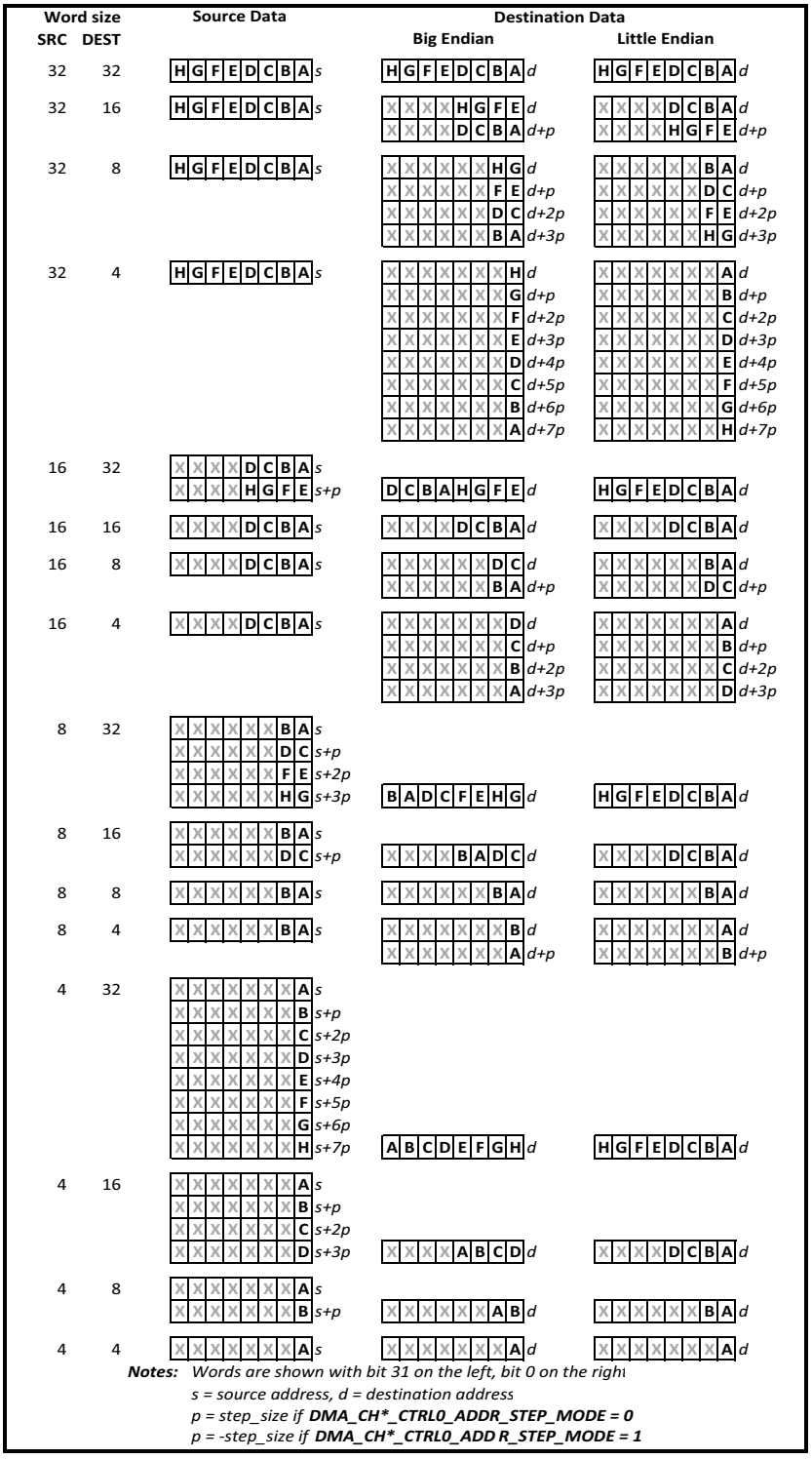


Figure 54. DMA Packing and Unpacking

For memory-to-peripheral transfers, the counter in the DMA channel always operates on the peripheral's word size. For peripheral-to-memory, peripheral-to-peripheral and memory-to-memory transfers, the counter in the DMA channel always operates on the source's word size. The next address for reading or writing can be determined from:

- The configured word size
- Current counter
- The step size and direction
- The base address

This can result in an actual read or write if no previously loaded data is available, or use of the temporary register if sufficient data has already been loaded.

When data is stored to memory, it is always right aligned. When the destination word size is larger than the source word size, the data is stored starting from the LSB. The upper bits are always written as zero (padded mode). If the configured destination word size is smaller than the configured source word size for a transfer to memory, the data is aligned at the LSB and truncated (truncate mode).

In packing or unpacking data, the endianness of the data is important because the order in which data bytes should be extracted from a word is different between little and big endian. The DMA supports processing both little endian and big endian data, through the `DMA_CH*_CTRL0_BYTE_ORDER` bit from the `DMA_CH*_CTRL0` register.

IMPORTANT: To provide a nibble ordering that is consistent with the byte endianess, the ordering of nibbles must match the byte ordering. This ensures that transfers which use data packing of 4-bit words—for either the source or destination—order data consistently.

NOTE: When packing data into memory for a transfer length that is not a multiple of the destination word size, the final data memory word is automatically zero padded, before being written to data memory at the end of the transfer.

15.5.3.3 Transfer Length

The transfer length and DMA counter depend on the transfer type, source word size, and destination word size.

- For Peripheral-to-Memory, Peripheral-to-Peripheral and Memory-to-Memory transfers, the transfer length equals the number of words of the specified word size at the source.
- For Memory-to-Peripheral transfers, the transfer length equals the number of words of the specified word size at the destination.

15.5.4 DMA Transfer Types

The DMA supports several types of data transfers between the ARM Cortex-M3 processor's memory and the interfaces and peripherals connected to the peripheral bus, including:

- Memory-to-Memory transfers
- Memory-to-Peripheral transfers
- Peripheral-to-Memory transfers
- Peripheral-to-Peripheral transfers

The interfaces and peripherals mapped onto the peripheral bus that are valid sources of data for a DMA data transfer are listed in Table 64.

Table 64. Valid Sources of DMA Data

Source	Encoded Bits
I^2C	0x0
SPI	0x1
PCM	0x2
Hamming and Interleaving Encoder	0x3
Hamming and Interleaving Decoder	0x4
Reed-Solomon Encoder	0x5
Reed-Solomon Decoder	0x6
CVSD Encoder	0x7
CVSD Decoder	0x8
G.722 Encoder	0x9
G.722 Decoder	0xA

Each source interface or peripheral, when configured for DMA operation, asserts its DMA request signal when data can be read from the interface. This signal is cleared automatically when a data value is read from the interface using the peripheral bus.

The interfaces and peripherals mapped onto the peripheral bus that are valid destinations for data from a DMA data transfer are listed in Table 65.

Table 65. Valid Destinations of DMA Data

Destination	Encoded Bits
I2C	0x0
SPI	0x1
PCM	0x2
Hamming and Interleaving Encoder	0x3
Hamming and Interleaving Decoder	0x4
Reed-Solomon Encoder	0x5
Reed-Solomon Decoder	0x6
CVSD Encoder	0x7
CVSD Decoder	0x8
G.722 Encoder	0x9
G.722 Decoder	0xA

Each destination interface or peripheral, when configured for DMA operation, asserts their DMA request signal when data can be written to the peripheral bus. This signal is cleared automatically when a data value is written to that interface using the peripheral bus.

CAUTION: When the DMA interface is used to control transfers that use an interface or peripheral, all accesses to that interface or peripheral's data registers using the peripheral bus will clear the DMA request signals. If a DMA request signal is cleared due to a CFX DSP or ARM Cortex-M3 processor access, the underlying DMA transfer will be corrupted.

IMPORTANT: Due to the structure of the SPI interface, when initializing an SPI data transfer using a DMA channel, a user application must pre-load the SPI data register (SPI_DATA) with the first data word of the transfer. For more information about the SPI interface, see Section 16.4, "Serial Peripheral Interface (SPI)" on page 395.

NOTE: To ease understanding, throughout the remainder of this section, all interfaces and peripherals used in DMA transfers are called *peripherals* due to their memory-mapping onto the peripheral bus.

15.5.4.1 Memory-to-Memory (MM)

The DMA memory-to-memory mode is used to transfer data from one location in the ARM Cortex-M3 processor's data memory to another location in the same memory. This relieves the processor from using cycles for a simple copy operation. The operation sequence is as follows (assuming 32-bit operation):

1. The DMA requests access to the ARM Cortex-M3 processor's data memory via the ARM Cortex-M3 processor's data memory arbiter.
2. The DMA is granted access to the ARM Cortex-M3 processor's data memory.
3. The DMA reads the word at the next source address and stores the word in a temporary register.
4. The DMA releases the ARM Cortex-M3 processor's data memory.
5. The DMA requests access to the ARM Cortex-M3 processor's data memory via the ARM Cortex-M3 processor's data memory arbiter.
6. The DMA is granted access to the ARM Cortex-M3 processor's data memory.
7. The DMA writes the word at the next destination address.
8. The DMA releases the ARM Cortex-M3 processor's data memory.
9. The DMA channel's counter is incremented.
10. If the transfer length is not reached, the DMA waits one cycle before starting the next transfer. If the transfer length is reached and the DMA is in linear mode, the DMA channel switches to the complete state. Circular mode is not applicable for Memory-to-Memory transfers.
11. Any DMA interrupts that are triggered by this transfer are generated.

IMPORTANT: The maximum transfer rate for memory-to-memory transfers is one transfer every two SYSCLK cycles.

15.5.4.2 Memory-to-Peripheral

The DMA memory-to-peripheral mode is used to transfer data from the ARM Cortex-M3 processor's data memory to a peripheral on the peripheral bus. If the peripheral is configured to operate in DMA mode, the operation sequence is as follows (assuming 32-bit operation):

1. The DMA channel receives a DMA request from the peripheral.
2. The DMA requests access to the ARM Cortex-M3 processor's data memory via the ARM Cortex-M3 processor's data memory arbiter.
3. The DMA is granted access to the ARM Cortex-M3 processor's data memory.
4. The DMA reads the word at the next source address, storing it to a temporary register.
5. The DMA releases the ARM Cortex-M3 processor's data memory.
6. Through the peripheral bus bridge, the DMA requests access to the peripheral bus.
7. The DMA is granted access to the peripheral bus.
8. The DMA writes the word at the next destination address.
9. The DMA channel acknowledges the peripheral DMA request (implied by the previous write operation).

10. The DMA releases the peripheral bus.
11. The DMA channel's counter is incremented.
12. If the transfer length is not reached, the DMA channel waits for the next peripheral DMA request. If the transfer length is reached and the DMA is in linear mode, the DMA channel switches to the complete state. If the DMA is in circular mode, the DMA resets the counter to 0 and remains enabled.
13. Any DMA interrupts that are triggered by this transfer are generated.

15.5.4.3 Peripheral-to-Memory (PM)

The DMA peripheral-to-memory mode is used to transfer data from a peripheral to the ARM Cortex-M3 processor's data memory. If the peripheral is configured to operate in DMA mode, the operation sequence is as follows (assuming 32-bit operation):

1. The peripheral generates a new data word.
2. The peripheral asserts a signal on a DMA request line (indicates that the buffer is full).
3. The DMA channel receives a DMA request from the peripheral.
4. Through the peripheral bus bridge, the DMA requests access to the peripheral bus.
5. The DMA is granted access to the peripheral bus.
6. The DMA reads the word at the next source address and stores the word to a temporary register.
7. The DMA releases the peripheral bus.
8. The DMA channel acknowledges the peripheral DMA request (implied by the previous read operation).
9. The DMA requests access to the ARM Cortex-M3 processor's data memory via the ARM Cortex-M3 processor's data memory arbiter.
10. The DMA is granted access to the ARM Cortex-M3 processor's data memory.
11. The DMA writes the word at the next destination address.
12. The DMA releases the ARM Cortex-M3 processor's data memory.
13. The DMA channel's counter is incremented.
14. If the transfer length is not reached, the DMA waits for the next peripheral DMA request. If the transfer length is reached and the DMA is in linear mode, the DMA channel switches to the complete state. If the DMA is in circular mode, the DMA resets the counter to 0 and remains enabled.
15. Any DMA interrupts that are triggered by this transfer are generated.

15.5.4.4 Peripheral-to-Peripheral (PP)

The DMA peripheral-to-peripheral mode is used to transfer data from a peripheral to another peripheral. If both peripherals are configured to operate in DMA mode, the operation sequence is as follows (assuming 32-bit operation):

1. The DMA channel receives a DMA request from both the source and destination peripherals in any order. For the source peripheral, the following operation is required to generate a request:
   a. The source peripheral generates a new data word.
   b. The source peripheral asserts a signal on a DMA request line (indicates that the buffer is full).
   c. The DMA channel receives a DMA request from the source peripheral.

   The destination peripheral generates a request when it can receive an additional data word.

2. Through the peripheral bus bridge, the DMA requests access to the peripheral bus.
3. The DMA is granted access to the peripheral bus.
4. The DMA reads the word at the next source address and stores the word to a temporary register.
5. The DMA releases the peripheral bus.
6. The DMA channel acknowledges the source peripheral DMA request (implied by the previous read operation).
7. Through the peripheral bus bridge, the DMA requests access to the peripheral bus.
8. The DMA is granted access to the peripheral bus.

9. The DMA writes the word at the next destination address.
10. The DMA channel acknowledges the destination peripheral DMA request (implied by the previous write operation).
11. The DMA releases the peripheral bus.
12. The DMA channel's counter is incremented.
13. If the transfer length is not reached, the DMA channel waits for the next peripheral DMA request. If the transfer length is reached and the DMA is in linear mode, the DMA channel switches to the complete state. If the DMA is in circular mode, the DMA resets the counter to 0 and remains enabled.
14. Any DMA interrupts that should be triggered by this transfer are generated.

IMPORTANT: To ensure that DMA operations are atomic, the DMA channel operation does not begin until DMA requests are received from both peripherals.

15.5.5 DMA Interrupt Configuration

The DMA has a separate interrupt for each DMA channel. Each DMA channel can be configured using its DMA_CH*_CTRL0 register to assert an interrupt for several independent conditions:

- When a transfer has started—set the DMA_CH*_CTRL0_START_INT_ENABLE bit to enable this interrupt.
- When a transfer has reached the counter interrupt value—set the DMA_CH*_CTRL1_COUNTER_INT_VALUE bit field in the DMA_CH*_CTRL1 register to configure this interrupt, and set the DMA_CH*_CTRL0_COUNTER_INT_ENABLE bit to enable it.
- When a transfer is complete—set the DMA_CH*_CTRL0_COMPLETE_INT_ENABLE bit to enable it.
- When an error has occurred—set the DMA_CH*_CTRL0_ERROR_INT_ENABLE bit to enable it.
- When the DMA channel is disabled—set the DMA_CH*_CTRL0_DISABLE_INT_ENABLE bit to enable it.

The DMA channel status register (C_DMA_STATUS) indicates which interrupts have triggered. When an interrupt is generated, the application is responsible for reading the status register for that channel to determine which interrupts have been triggered. For all interrupts, the application's interrupt handlers are responsible for clearing the status and thus clearing the pending interrupt.

NOTE: A start interrupt is generated each time a transfer starts (including when a circular transfer restarts). Similarly, a complete interrupt is generated each time a transfer is completed. A disable interrupt is triggered whenever the channel is disabled for any reason (including when a linear transfer is completed).

When using a circular DMA transfer, the counter interrupt and complete interrupt can be used in tandem to create a two-page buffer for continuous data transfers. When the first page has transferred, the counter interrupt triggers. When the second page has transferred, the complete interrupt triggers. In this configuration:

For a memory-to-peripheral transfer

The counter interrupt indicates that data in the first page of the source buffer has transferred, and can be replaced while the second page is transferred. Similarly, the complete interrupt indicates that the second page can be replaced while the replacement first page is transferred.

For peripheral-to-memory transfers

The counter interrupt indicates when the first page of the buffer can be processed. The complete interrupt indicates when the second page of the buffer can be processed.

15.5.6 Channel Priority

Only one transfer using the DMA channels can be actively serviced at a time. To coordinate DMA channels, the DMA contains a channel arbiter that is responsible for determining which DMA channel is active. The relative priority of a channel can be set using the `DMA_CH*_CTRL0_CHANNEL_PRIORITY` bit field from the `DMA_CH*_CTRL0` register.

When a DMA channel receives DMA requests from the peripherals it is configured to use, it indicates to the arbiter that it is ready to transfer one word. Each DMA channel request is handled automatically. Several situations where multiple requests might be pending are:

- Two or more DMA channels receive a DMA request during the same clock cycle
- Two or more DMA requests come in during the processing of a third DMA request
- A DMA channel is enabled with multiple requests already pending

When choosing which DMA channel to activate, the DMA arbiter applies the channel's priority settings as follows:

- When multiple channels are ready, the channel with the lowest priority setting is activated.
- If more than one channel is ready to be activated and they share the same priority setting, the lowest numbered channel is activated.

When a single channel is ready, the arbiter grants access and the channel begins to service that request immediately.

IMPORTANT: A lower priority DMA channel might never be served if a higher priority DMA channel is generating requests too fast. This type of situation must be avoided by application design.

15.5.7 Data Memory Usage by the DMA and ARM Cortex-M3 Processor

The DMA has direct access to the ARM Cortex-M3 processor data memory. This access is limited by a system bus arbiter to avoid memory access conflicts between the DMA and the CFX and ARM Cortex-M3 processors. Memory conflicts between the CFX and ARM Cortex-M3 processors are handled separately, as described in Section 7.3.2, "P Memory Space" on page 80.

When the DMA is granted access to memory, it performs a single operation (read or write) and releases the ARM Cortex-M3 processor data memory. This ensures that the DMA never blocks access to data memory from the processors for more than a single memory operation.

The data memory arbiter implements a round-robin arbitration scheme. When both the DMA and the ARM Cortex-M3 processor or CFX DSP request access, access grants alternate between the DMA and the processors. Assuming that the ARM Cortex-M3 processor is performing only normal (non-bit-banded) memory transfers, the worst-case access time to the ARM Cortex-M3 processor data memory for DMA transfers can be established with high certainty. Typically, if the DMA and the processors are both always requesting data memory access, they can each utilize the memory 50% of the time.

NOTE: If the ARM Cortex-M3 processor is utilizing bit-banding operations, the DMA's data memory access might be delayed by an additional cycle, because the memory controller uses additional cycles for these operations.

15.5.8 DMA Registers

Register Name	Register Description	Address
DMA_CH0_CTRL0 C_DMA_CH0_CTRL0	DMA channel 0 control and configuration	0x40000200 P:0x3080
DMA_CH0_SRC_BASE_ADDR C_DMA_CH0_SRC_BASE_ADDR	DMA channel 0 source base address	0x40000204 P:0x3081
DMA_CH0_DEST_BASE_ADDR C_DMA_CH0_DEST_BASE_ADDR	DMA channel 0 destination base address	0x40000208 P:0x3082
DMA_CH0_CTRL1 C_DMA_CH0_CTRL1	DMA channel 0 transfer control	0x4000020C P:0x3083
DMA_CH0_NEXT_SRC_ADDR C_DMA_CH0_NEXT_SRC_ADDR	DMA channel 0 next source address	0x40000210 P:0x3084
DMA_CH0_NEXT_DEST_ADDR C_DMA_CH0_NEXT_DEST_ADDR	DMA channel 0 next destination address	0x40000214 P:0x3085
DMA_CH0_WORD_CNT C_DMA_CH0_WORD_CNT	DMA channel 0 word count	0x40000218 P:0x3086
DMA_CH1_CTRL0 C_DMA_CH1_CTRL0	DMA channel 1 control and configuration	0x40000220 P:0x3088
DMA_CH1_SRC_BASE_ADDR C_DMA_CH1_SRC_BASE_ADDR	DMA channel 1 source base address	0x40000224 P:0x3089
DMA_CH1_DEST_BASE_ADDR C_DMA_CH1_DEST_BASE_ADDR	DMA channel 1 destination base address	0x40000228 P:0x308A
DMA_CH1_CTRL1 C_DMA_CH1_CTRL1	DMA channel 1 transfer control	0x4000022C P:0x308B
DMA_CH1_NEXT_SRC_ADDR C_DMA_CH1_NEXT_SRC_ADDR	DMA channel 1 next source address	0x40000230 P:0x308C
DMA_CH1_NEXT_DEST_ADDR C_DMA_CH1_NEXT_DEST_ADDR	DMA channel 1 next destination address	0x40000234 P:0x308D
DMA_CH1_WORD_CNT C_DMA_CH1_WORD_CNT	DMA channel 1 word count	0x40000238 P:0x308E
DMA_CH2_CTRL0 C_DMA_CH2_CTRL0	DMA channel 2 control and configuration	0x40000240 P:0x3090
DMA_CH2_SRC_BASE_ADDR C_DMA_CH2_SRC_BASE_ADDR	DMA channel 2 source base address	0x40000244 P:0x3091
DMA_CH2_DEST_BASE_ADDR C_DMA_CH2_DEST_BASE_ADDR	DMA channel 2 destination base address	0x40000248 P:0x3092
DMA_CH2_CTRL1 C_DMA_CH2_CTRL1	DMA channel 2 transfer control	0x4000024C P:0x3093

Register Name	Register Description	Address
DMA_CH2_NEXT_SRC_ADDR C_DMA_CH2_NEXT_SRC_ADDR	DMA channel 2 next source address	0x40000250 P:0x3094
DMA_CH2_NEXT_DEST_ADDR C_DMA_CH2_NEXT_DEST_ADDR	DMA channel 2 next destination address	0x40000254 P:0x3095
DMA_CH2_WORD_CNT C_DMA_CH2_WORD_CNT	DMA channel 2 word count	0x40000258 P:0x3096
DMA_CH3_CTRL0 C_DMA_CH3_CTRL0	DMA channel 3 control and configuration	0x40000260 P:0x3098
DMA_CH3_SRC_BASE_ADDR C_DMA_CH3_SRC_BASE_ADDR	DMA channel 3 source base address	0x40000264 P:0x3099
DMA_CH3_DEST_BASE_ADDR C_DMA_CH3_DEST_BASE_ADDR	DMA channel 3 destination base address	0x40000268 P:0x309A
DMA_CH3_CTRL1 C_DMA_CH3_CTRL1	DMA channel 3 transfer control	0x4000026C P:0x309B
DMA_CH3_NEXT_SRC_ADDR C_DMA_CH3_NEXT_SRC_ADDR	DMA channel 3 next source address	0x40000270 P:0x309C
DMA_CH3_NEXT_DEST_ADDR C_DMA_CH3_NEXT_DEST_ADDR	DMA channel 3 next destination address	0x40000274 P:0x309D
DMA_CH3_WORD_CNT C_DMA_CH3_WORD_CNT	DMA channel 3 word count	0x40000278 P:0x309E
DMA_STATUS C_DMA_STATUS	DMA status	0x40000290 P:0x30A4

15.5.8.1 DMA_CHx_CTRLy, C_DMA_CHx_CTRLy Settings

The following bit fields and field names apply equally to all registers with the names DMA_CH**x**_CTRL**y**, C_DMA_CH**x**_CTRL**y** where **x** is 0 to 3 and **y** is 0 or 1.

Bit Field	Field Name	Description
31:30	DEST_ADDR_STEP_SIZE	Select the destination address step size
29:28	SRC_ADDR_STEP_SIZE	Select the source address step size
27	DEST_ADDR_STEP_MODE	Configure the destination address to either increment or decrement
26	SRC_ADDR_STEP_MODE	Configure the source address to either increment or decrement
25	BYTE_ORDER	Select the byte ordering for the DMA channel
24	DISABLE_INT_ENABLE	Raise an interrupt when the DMA channel is disabled
23	ERROR_INT_ENABLE	Raise an interrupt when a state machine error occurs during a DMA transfer
22	COMPLETE_INT_ENABLE	Raise an interrupt when the DMA transfer completes
21	COUNTER_INT_ENABLE	Raise an interrupt when the DMA transfer reaches the counter value
20	START_INT_ENABLE	Raise an interrupt when the DMA transfer starts
19:18	DEST_WORD_SIZE	Select the destination word size

Bit Field	Field Name	Description
17:16	SRC_WORD_SIZE	Select the source word size
15:12	DEST_SELECT	Select the destination peripheral
11:8	SRC_SELECT	Select the source peripheral
7:6	CHANNEL_PRIORITY	Select the priority level for this channel
5:4	TRANSFER_TYPE	Select the type of transfer
3	DEST_ADDR_INC	Configure whether the destination address should increment
2	SRC_ADDR_INC	Configure whether the source address should increment
1	ADDR_MODE	Select the addressing mode for this channel
0	ENABLE	Enable DMA channel 0

Field Name	Value Symbol	Value Description	Hex Value
DEST_ADDR_STEP_SIZE	DMA_CH**x**_DEST_ADDR_STEP_SIZE_1	Set the step size of DMA channel **x** to 1	0x0*
	DMA_CH**x**_DEST_ADDR_STEP_SIZE_2	Set the step size of DMA channel **x** to 2	0x1
	DMA_CH**x**_DEST_ADDR_STEP_SIZE_3	Set the step size of DMA channel **x** to 3	0x2
	DMA_CH**x**_DEST_ADDR_STEP_SIZE_4	Set the step size of DMA channel **x** to 4	0x3
SRC_ADDR_STEP_SIZE	DMA_CH**x**_SRC_ADDR_STEP_SIZE_1	Set the step size of DMA channel **x** to 1	0x0*
	DMA_CH**x**_SRC_ADDR_STEP_SIZE_2	Set the step size of DMA channel **x** to 2	0x1
	DMA_CH**x**_SRC_ADDR_STEP_SIZE_3	Set the step size of DMA channel **x** to 3	0x2
	DMA_CH**x**_SRC_ADDR_STEP_SIZE_4	Set the step size of DMA channel **x** to 4	0x3
DEST_ADDR_STEP_MODE	DMA_CH**x**_DEST_ADDR_POS	Increment the destination address used by DMA channel **x**	0x0*
	DMA_CH**x**_DEST_ADDR_NEG	Decrement the destination address used by DMA channel **x**	0x1
SRC_ADDR_STEP_MODE	DMA_CH**x**_SRC_ADDR_POS	Increment the source address used by DMA channel **x**	0x0*
	DMA_CH**x**_SRC_ADDR_NEG	Decrement the source address used by DMA channel **x**	0x1
BYTE_ORDER	DMA_CH**x**_LITTLE_ENDIAN	DMA channel **x** uses little endian mode	0x0*
	DMA_CH**x**_BIG_ENDIAN	DMA channel **x** uses big endian mode	0x1
DISABLE_INT_ENABLE	DMA_CH**x**_DISABLE_INT_DISABLE	Disable channel disable indicator interrupts for DMA channel **x**	0x0*
	DMA_CH**x**_DISABLE_INT_ENABLE	Enable channel disable indicator interrupts for DMA channel **x**	0x1
ERROR_INT_ENABLE	DMA_CH**x**_ERROR_INT_DISABLE	Disable error indicator interrupts for DMA channel **x**	0x0*
	DMA_CH**x**_ERROR_INT_ENABLE	Enable error indicator interrupts for DMA channel **x**	0x1
COMPLETE_INT_ENABLE	DMA_CH**x**_COMPLETE_INT_DISABLE	Disable completion interrupts for DMA channel **x**	0x0*
	DMA_CH**x**_COMPLETE_INT_ENABLE	Enable completion interrupts for DMA channel **x**	0x1

Field Name	Value Symbol	Value Description	Hex Value
COUNTER_INT_ENABLE	DMA_CH**x**_COUNTER_INT_DISABLE	Disable counter interrupts for DMA channel **x**	0x0*
	DMA_CH**x**_COUNTER_INT_ENABLE	Enable counter interrupts for DMA channel **x**	0x1
START_INT_ENABLE	DMA_CH**x**_START_INT_DISABLE	Disable start interrupts for DMA channel **x**	0x0*
	DMA_CH**x**_START_INT_ENABLE	Enable start interrupts for DMA channel **x**	0x1
DEST_WORD_SIZE	DMA_CH**x**_DEST_WORD_SIZE_8	Destination data uses 8-bit words	0x0*
	DMA_CH**x**_DEST_WORD_SIZE_16	Destination data uses 16-bit words	0x1
	DMA_CH**x**_DEST_WORD_SIZE_32	Destination data uses 32-bit words	0x2
	DMA_CH**x**_DEST_WORD_SIZE_4	Destination data uses 4-bit words	0x3
SRC_WORD_SIZE	DMA_CH**x**_SRC_WORD_SIZE_8	Source data uses 8-bit words	0x0*
	DMA_CH**x**_SRC_WORD_SIZE_16	Source data uses 16-bit words	0x1
	DMA_CH**x**_SRC_WORD_SIZE_32	Source data uses 32-bit words	0x2
	DMA_CH**x**_SRC_WORD_SIZE_4	Source data uses 4-bit words	0x3
DEST_SELECT	DMA_CH**x**_DEST_I2C	Data writes are triggered by the I^2C request line	0x0*
	DMA_CH**x**_DEST_SPI	Data writes are triggered by the SPI request line	0x1
	DMA_CH**x**_DEST_PCM	Data writes are triggered by the PCM request line	0x2
	DMA_CH**x**_DEST_HMI_ENC	Data writes are triggered by the Hamming encoder request line	0x3
	DMA_CH**x**_DEST_HMI_DEC	Data writes are triggered by the Hamming decoder request line	0x4
	DMA_CH**x**_DEST_RS_ENC	Data writes are triggered by the Reed-Solomon encoder request line	0x5
	DMA_CH**x**_DEST_RS_DEC	Data writes are triggered by the Reed-Solomon decoder request line	0x6
	DMA_CH**x**_DEST_CVSD_ENC	Data writes are triggered by the CVSD encoder request line	0x7
	DMA_CH**x**_DEST_CVSD_DEC	Data writes are triggered by the CVSD decoder request line	0x8
	DMA_CH**x**_DEST_G722_ENC	Data writes are triggered by the G.722 encoder request line	0x9
	DMA_CH**x**_DEST_G722_DEC	Data writes are triggered by the G.722 decoder request line	0xA

Field Name	Value Symbol	Value Description	Hex Value
SRC_SELECT	DMA_CH**x**_SRC_I2C	Data reads are triggered by the I^2C request line	0x0*
	DMA_CH**x**_SRC_SPI	Data reads are triggered by the SPI request line	0x1
	DMA_CH**x**_SRC_PCM	Data reads are triggered by the PCM request line	0x2
	DMA_CH**x**_SRC_HMI_ENC	Data reads are triggered by the Hamming encoder request line	0x3
	DMA_CH**x**_SRC_HMI_DEC	Data reads are triggered by the Hamming decoder request line	0x4
	DMA_CH**x**_SRC_RS_ENC	Data reads are triggered by the Reed-Solomon encoder request line	0x5
	DMA_CH**x**_SRC_RS_DEC	Data reads are triggered by the Reed-Solomon decoder request line	0x6
	DMA_CH**x**_SRC_CVSD_ENC	Data reads are triggered by the CVSD encoder request line	0x7
	DMA_CH**x**_SRC_CVSD_DEC	Data reads are triggered by the CVSD decoder request line	0x8
	DMA_CH**x**_SRC_G722_ENC	Data reads are triggered by the G.722 encoder request line	0x9
	DMA_CH**x**_SRC_G722_DEC	Data reads are triggered by the G.722 decoder request line	0xA
CHANNEL_PRIORITY	DMA_CH**x**_PRIORITY_0	Set the priority of DMA channel **x** to 0 (Highest)	0x0*
	DMA_CH**x**_PRIORITY_1	Set the priority of DMA channel **x** to 1	0x1
	DMA_CH**x**_PRIORITY_2	Set the priority of DMA channel **x** to 2	0x2
	DMA_CH**x**_PRIORITY_3	Set the priority of DMA channel **x** to 3 (Lowest)	0x3
TRANSFER_TYPE	DMA_CH**x**_TRANSFER_M_TO_M	DMA channel **x** will provide a memory-to-memory data transfer	0x0*
	DMA_CH**x**_TRANSFER_M_TO_P	DMA channel **x** will provide a memory-to-peripheral data transfer	0x1
	DMA_CH**x**_TRANSFER_P_TO_M	DMA channel **x** will provide a peripheral-to-memory data transfer	0x2
	DMA_CH**x**_TRANSFER_P_TO_P	DMA channel **x** will provide a peripheral-to-peripheral data transfer	0x3
DEST_ADDR_INC	DMA_CH**x**_DEST_ADDR_STATIC	Do not increment the destination address used by DMA channel **x**	0x0*
	DMA_CH**x**_DEST_ADDR_INC	Increment the destination address used by DMA channel **x**	0x1
SRC_ADDR_INC	DMA_CH**x**_SRC_ADDR_STATIC	Do not increment the source address used by DMA channel **x**	0x0*
	DMA_CH**x**_SRC_ADDR_INC	Increment the source address used by DMA channel **x**	0x1
ADDR_MODE	DMA_CH**x**_ADDR_CIRC	Use circular addressing for DMA channel **x**	0x0*
	DMA_CH**x**_ADDR_LIN	Use linear addressing for DMA channel **x**	0x1

Field Name	Value Symbol	Value Description	Hex Value
ENABLE	DMA_CH**x**_DISABLE	Disable DMA channel **x**	0x0*
	DMA_CH**x**_ENABLE	Enable DMA channel **x**	0x1

15.5.8.2 DMA_CH*_SRC_BASE_ADDR, C_DMA_CH*_SRC_BASE_ADDR Settings

DMA_CH0_SRC_BASE_ADDR, C_DMA_CH0_SRC_BASE_ADDR to DMA_CH3_SRC_BASE_ADDR, C_DMA_CH3_SRC_BASE_ADDR all have the same bit fields.

Bit Field	Field Name	Description
31:0	DMA_CH*_SRC_BASE_ADDR	Base address for the source of data transferred using DMA channel *

15.5.8.3 DMA_CH*_DEST_BASE_ADDR, C_DMA_CH*_DEST_BASE_ADDR Settings

DMA_CH0_DEST_BASE_ADDR, C_DMA_CH0_DEST_BASE_ADDR to DMA_CH3_DEST_BASE_ADDR, C_DMA_CH3_DEST_BASE_ADDR all have the same bit fields.

Bit Field	Field Name	Description
31:0	DMA_CH*_DEST_BASE_ADDR	Base address for the destination of data transferred using DMA channel *

15.5.8.4 DMA_CH*_NEXT_SRC_ADDR, C_DMA_CH*_NEXT_SRC_ADDR Settings

DMA_CH0_NEXT_SRC_ADDR, C_DMA_CH0_NEXT_SRC_ADDR to DMA_CH3_NEXT_SRC_ADDR, C_DMA_CH3_NEXT_SRC_ADDR all have the same bit fields.

Bit Field	Field Name	Description
31:0	DMA_CH*_NEXT_SRC_ADDR	Address of the next data to be transferred using DMA channel *

15.5.8.5 DMA_CH*_NEXT_DEST_ADDR, C_DMA_CH*_NEXT_DEST_ADDR Settings

DMA_CH0_NEXT_DEST_ADDR, C_DMA_CH0_NEXT_DEST_ADDR to DMA_CH3_NEXT_DEST_ADDR, C_DMA_CH3_NEXT_DEST_ADDR all have the same bit fields.

Bit Field	Field Name	Description
31:0	DMA_CH*_NEXT_DEST_ADDR	Address where the next data to be transferred using DMA channel * will be stored

15.5.8.6 DMA_CH*_WORD_CNT, C_DMA_CH*_WORD_CNT Settings

DMA_CH0_WORD_CNT, C_DMA_CH0_WORD_CNT to DMA_CH3_WORD_CNT, C_DMA_CH3_WORD_CNT all have the same bit fields.

Bit Field	Field Name	Description
15:0	DMA_CH*_WORD_CNT	The number of words that have been transferred using DMA channel * during the current transfer

15.5.8.7 DMA_STATUS, C_DMA_STATUS Settings

Bit Field	Field Name	Description
31	CH3_ERROR_INT_STATUS	Indicate if a state machine error interrupt has occurred on DMA channel 3
30	CH3_COMPLETE_INT_STATUS	Indicate if a complete interrupt has occurred on DMA channel 3
29	CH3_COUNTER_INT_STATUS	Indicate if a counter interrupt has occurred on DMA channel 3
28	CH3_START_INT_STATUS	Indicate if a start interrupt has occurred on DMA channel 3
27	CH3_DISABLE_INT_STATUS	Indicate if a channel disable interrupt has occurred on DMA channel 3
26:24	CH3_STATE	DMA channel 3 state
23	CH2_ERROR_INT_STATUS	Indicate if a state machine error interrupt has occurred on DMA channel 2
22	CH2_COMPLETE_INT_STATUS	Indicate if a complete interrupt has occurred on DMA channel 2
21	CH2_COUNTER_INT_STATUS	Indicate if a counter interrupt has occurred on DMA channel 2
20	CH2_START_INT_STATUS	Indicate if a start interrupt has occurred on DMA channel 2
19	CH2_DISABLE_INT_STATUS	Indicate if a channel disable interrupt has occurred on DMA channel 2
18:16	CH2_STATE	DMA channel 2 state
15	CH1_ERROR_INT_STATUS	Indicate if a state machine error interrupt has occurred on DMA channel 1
14	CH1_COMPLETE_INT_STATUS	Indicate if a complete interrupt has occurred on DMA channel 1
13	CH1_COUNTER_INT_STATUS	Indicate if a counter interrupt has occurred on DMA channel 1
12	CH1_START_INT_STATUS	Indicate if a start interrupt has occurred on DMA channel 1
11	CH1_DISABLE_INT_STATUS	Indicate if a channel disable interrupt has occurred on DMA channel 1
10:8	CH1_STATE	DMA channel 1 state
7	CH0_ERROR_INT_STATUS	Indicate if a state machine error interrupt has occurred on DMA channel 0
6	CH0_COMPLETE_INT_STATUS	Indicate if a complete interrupt has occurred on DMA channel 0
5	CH0_COUNTER_INT_STATUS	Indicate if a counter interrupt has occurred on DMA channel 0
4	CH0_START_INT_STATUS	Indicate if a start interrupt has occurred on DMA channel 0
3	CH0_DISABLE_INT_STATUS	Indicate if a channel disable interrupt has occurred on DMA channel 0
2:0	CH0_STATE	DMA channel 0 state

Field Name	Value Symbol	Value Description	Hex Value
CH3_ERROR_INT_STATUS	DMA_CH3_ERROR_INT	Indicate that a state machine error interrupt has occurred	0x1
CH3_COMPLETE_INT_STATUS	DMA_CH3_COMPLETE_INT	Indicate that a complete interrupt has occurred	0x1
CH3_COUNTER_INT_STATUS	DMA_CH3_COUNTER_INT	Indicate that a counter interrupt has occurred	0x1
CH3_START_INT_STATUS	DMA_CH3_START_INT	Indicate that a start interrupt has occurred	0x1
CH3_DISABLE_INT_STATUS	DMA_CH3_DISABLE_INT	Indicate that a channel disable interrupt has occurred	0x1
CH3_STATE	DMA_CH3_IDLE	Indicate that DMA channel 3 is idle	0x0*
CH2_ERROR_INT_STATUS	DMA_CH2_ERROR_INT	Indicate that a state machine error interrupt has occurred	0x1

Field Name	Value Symbol	Value Description	Hex Value
CH2_COMPLETE_INT_STATUS	DMA_CH2_COMPLETE_INT	Indicate that a complete interrupt has occurred	0x1
CH2_COUNTER_INT_STATUS	DMA_CH2_COUNTER_INT	Indicate that a counter interrupt has occurred	0x1
CH2_START_INT_STATUS	DMA_CH2_START_INT	Indicate that a start interrupt has occurred	0x1
CH2_DISABLE_INT_STATUS	DMA_CH2_DISABLE_INT	Indicate that a channel disable interrupt has occurred	0x1
CH2_STATE	DMA_CH2_IDLE	Indicate that DMA channel 2 is idle	0x0*
CH1_ERROR_INT_STATUS	DMA_CH1_ERROR_INT	Indicate that a state machine error interrupt has occurred	0x1
CH1_COMPLETE_INT_STATUS	DMA_CH1_COMPLETE_INT	Indicate that a complete interrupt has occurred	0x1
CH1_COUNTER_INT_STATUS	DMA_CH1_COUNTER_INT	Indicate that a counter interrupt has occurred	0x1
CH1_START_INT_STATUS	DMA_CH1_START_INT	Indicate that a start interrupt has occurred	0x1
CH1_DISABLE_INT_STATUS	DMA_CH1_DISABLE_INT	Indicate that a channel disable interrupt has occurred	0x1
CH1_STATE	DMA_CH1_IDLE	Indicate that DMA channel 1 is idle	0x0*
CH0_ERROR_INT_STATUS	DMA_CH0_ERROR_INT	Indicate that a state machine error interrupt has occurred	0x1
CH0_COMPLETE_INT_STATUS	DMA_CH0_COMPLETE_INT	Indicate that a complete interrupt has occurred	0x1
CH0_COUNTER_INT_STATUS	DMA_CH0_COUNTER_INT	Indicate that a counter interrupt has occurred	0x1
CH0_START_INT_STATUS	DMA_CH0_START_INT	Indicate that a start interrupt has occurred	0x1
CH0_DISABLE_INT_STATUS	DMA_CH0_DISABLE_INT	Indicate that a channel disable interrupt has occurred	0x1
CH0_STATE	DMA_CH0_IDLE	Indicate that DMA channel 0 is idle	0x0*

15.6 Cyclic Redundancy Check (CRC) Generator

The ARM Cortex-M3 core peripherals provide two standard cyclic redundancy check (CRC) algorithms that can be used to ensure data integrity for wireless communications. These CRC implementations ensure data integrity because they guarantee that all single-bit errors, two-bit errors, burst errors (i.e., multiple bit errors in a row), and any error containing an odd number of bits are caught. As calculating a CRC is a common but moderately expensive software calculation, this hardware CRC generator is included as a peripheral to the ARM Cortex-M3 processor.

The ARM Cortex-M3 core peripherals includes a CRC generator that supports both the CRC-CCITT and CRC-32 (IEEE 802.3) algorithms. The parameters associated with the CRC-CCITT algorithm implementation are provided in Table 66.

Table 66. CRC-CCITT Algorithm Parameters

CRC Parameter	Parameter Value
Order	16
Polynomial	$x^{16} + x^{12} + x^5 + 1$
Polynomial (hex)	0x1021
Initial Value (hex)	0xFFFF
Final XOR Value (hex)	0x0000

No data manipulation is required for the output CRC generated for the standard CRC-CCITT algorithm (i.e., no data byte reversal, reversal of the final result or other finalization).

The parameters associated with the CRC-32 algorithm implementation are provided in Table 67.

Table 67. CRC-32 Algorithm Parameters

CRC Parameter	Parameter Value
Order	32
Polynomial	$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$
Polynomial (hex)	0x4C11DB7
Initial Value (hex)	0xFFFFFFFF
Final XOR Value (hex)	0xFFFFFFFF

The output CRC generated for the standard CRC-32 algorithm requires data byte reversal and reversal of the final result.

The ARM Cortex-M3 processor's CRC generator supports non-standard variants of the CRC-CCITT and CRC-32 standard implementation.

- To use non-standard CRC ordering of data with each data byte, set the `CRC_CTRL_BIT_ORDER` bit from the `C_CRC_CTRL` register.
- To use non-standard CRC ordering of the final result, set the `CRC_CTRL_FINAL_CRC_REVERSE` bit from the `C_CRC_CTRL` register.
- To use non-standard CRC XOR of the final result, set the `CRC_CTRL_FINAL_CRC_XOR` bit from the `C_CRC_CTRL` register. If configured for a non-standard XOR, this uses a final XOR value of 0xFFFF for CRC-CCITT and 0x00000000 for CRC-32.

To use the CRC generator:

1. Select the type of endian byte ordering and the desired CRC algorithm in the `C_CRC_CTRL` register.
2. Write an initial value of 0xFFFF or 0xFFFFFFFF to the `CRC_DATA` register.
3. To the CRC input registers, write any data that must be included in the final CRC read from the generator; 1-bit, 8-bit, 16-bit, 24-bit and 32-bit data values are supported. Input data to the CRC generator can be interpreted as either little-endian or big-endian data. To configure the endianness of input data to the CRC algorithms, configure the `CRC_CTRL_BYTE_ORDER` bit from the `C_CRC_CTRL` register.
4. At any time, you can read the `CRC_DATA` register to obtain the CRC for the data that has been added using the CRC input registers since the last time the register was initialized.

Select which CRC is used by setting the CRC_CTRL_CRC_TYPE bit field from C_CRC_CTRL to CRC_CCITT or CRC_32.

15.6.1 CRC Registers

Register Name	Register Description	Address
CRC_CTRL C_CRC_CTRL	CRC Generator Control	0x40000A00 P:0x3280
CRC_VALUE C_CRC_VALUE	CRC Generator Value	0x40000A04 P:0x3281
CRC_ADD_1 C_CRC_ADD_1	CRC Generator - Add 1 Bit	0x40000A08 P:0x3282
CRC_ADD_8 C_CRC_ADD_8	CRC Generator - Add 1 Byte	0x40000A0C P:0x3283
CRC_ADD_16 C_CRC_ADD_16	CRC Generator - Add 1 Half-word	0x40000A10 P:0x3284
CRC_ADD_24 C_CRC_ADD_24	CRC Generator - Add 3 Bytes	0x40000A14 P:0x3285
CRC_ADD_32 C_CRC_ADD_32	CRC Generator - Add 1 Word	0x40000A18 P:0x3286

15.6.1.1 CRC_CTRL, C_CRC_CTRL

Bit Field	Field Name	Description
4	FINAL_CRC_XOR	Selects the final CRC XOR mode
3	FINAL_CRC_REVERSE	Selects the final CRC reversal mode
2	BIT_ORDER	Selects the bit order for bytes added to the CRC
1	CRC_TYPE	Selects the CRC type
0	BYTE_ORDER	Selects the endianness for bytes added to the CRC

Field Name	Value Symbol	Value Description	Hex Value
FINAL_CRC_XOR	CRC_FINAL_XOR_STANDARD	Final CRC XOR is done according to the standard (CRC-CCITT: no XOR; CRC-32: XOR with 0xFFFFFFFF)	0x0*
	CRC_FINAL_XOR_NON_STANDARD	Final CRC XOR is opposite to the standard	0x1
FINAL_CRC_REVERSE	CRC_FINAL_REVERSE_STANDARD	Final CRC reversal is done according to the standard (CRC-CCITT: normal; CRC-32 reversed)	0x0*
	CRC_FINAL_REVERSE_NON_STANDARD	Final CRC reversal is done in opposite of the standard	0x1

Field Name	Value Symbol	Value Description	Hex Value
BIT_ORDER	CRC_BIT_ORDER_STANDARD	Bit order is as defined by the standard (CRC-CCITT: normal; CRC-32 reversed)	0x0*
	CRC_BIT_ORDER_NON_STANDARD	Bit order is opposite of the standard	0x1
CRC_TYPE	CRC_CCITT	CRC-CCITT algorithm selected	0x0*
	CRC_32	CRC-32 (IEEE 802.3) algorithm selected	0x1
BYTE_ORDER	CRC_BIG_ENDIAN	Bytes are added to the CRC in big-endian order	0x0*
	CRC_LITTLE_ENDIAN	Bytes are added to the CRC in little-endian order	0x1

15.6.1.2 CRC_VALUE, C_CRC_VALUE

Bit Field	Field Name	Description
31:0	CURRENT_CRC	CRC generator value: Write 0xFFFFFFF (32) or 0xFFFF (CCITT) to initialize the CRC; read provides the current CRC value.

Field Name	Value Symbol	Value Description	Hex Value
CURRENT_CRC	CRC_CCITT_INIT_VALUE	Initial value for the CRC CCITT calculation	0xFFFF*
	CRC_32_INIT_VALUE	Initial value for the CRC 32 calculation	0xFFFFFFFF

15.6.1.3 CRC_ADD_1, C_CRC_ADD_1

Bit Field	Field Name	Description
0	CRC_ADD_1_BIT	Add 1 bit to the CRC calculation

15.6.1.4 CRC_ADD_8, C_CRC_ADD_8

Bit Field	Field Name	Description
7:0	CRC_ADD_8	Add 1 byte (8 bits) to the CRC calculation

15.6.1.5 CRC_ADD_16, C_CRC_ADD_16

Bit Field	Field Name	Description
15:0	CRC_ADD_16	Add 1 half-word (16 bits) to the CRC calculation

15.6.1.6 CRC_ADD_24, C_CRC_ADD_24

Bit Field	Field Name	Description
23:0	CRC_ADD_24_BITS	Add 3 bytes (24 bits) to the CRC calculation

15.6.1.7 CRC_ADD_32, C_CRC_ADD_32

Bit Field	Field Name	Description
31:0	CRC_ADD_32	Add 1 word (32 bits) to the CRC calculation

CHAPTER 16
CFX External Interfaces

16.1 DIGITAL I/O (DIO)

The Ezairo 7100 system contains 30 digital I/O pads that can be configured:

- To support the external interfaces, output clocks, and other I/Os from the CFX DSP and ARM Cortex-M3 processor subsystems
- As general-purpose I/Os controllable from one of these cores

DIOs support all digital inputs or output functionality that are not supported directly by a dedicated I/O. For more information about the functional configuration of DIO pads, see Section 16.1.1, “Functional Configuration”.

Dedicated I/Os are supplied for the following pads which are not part of the DIO pad set:

- The reset pad (NRESET)—for more information, see Section 19.3, “Resets” on page 478.
- The external clock I/O pad (EXTCLK)—for more information, see Section 18.2.6, “External Clock (EXTCLK)” on page 448.
- The SCL and SDA pads for the debug ports and CFX general-purpose I^2C interface—for more information, see Section 13.1, “Inter-IC Interface (I2C) Bus®” on page 188.

The DIO pads support a variety of physical configuration parameters that might be required for proper interfacing with external components, including:

- Pull-up resistors
- Low-pass input filtering
- Output drive strength

A complete description of the physical configuration of DIOs can be found in Section 16.1.2, “Physical Configuration”.

The DIOs are divided into three VDDO supplied sections of 10 pads each, grouped as DIO0-DIO9, DIO10-DIO19, and DIO20-DIO29, and supplied by VDDO1, VDDO2, and VDDO3 respectively. For more information about power supply configuration for DIOs and dedicated I/O pads, see Section 19.1.1.3, “Digital Output Supply Voltages (VDDO1, VDDO2, and VDDO3)” on page 468.

16.1.1 Functional Configuration

The DIO pads can be configured using the `DIO_CFG_IO_MODE` bit field from the `D_DIO_CFG_*` registers:

- For a variety of digital output modes
- For a general-purpose input mode, with the input function configured by the `D_`**`interface`**`_SRC` registers
- As an LSAD input

Table 68 contains a listing of the functional modes a DIO can be configured for. Table 69 contains a list of the input sources that a given DIO can be assigned to supply.

LSAD multiplexing uses a special case functional configuration mapping. The six LSAD inputs are mapped in order to the first six DIO pads of each VDDO supplied section. For each LSAD input, only one of the three DIO pads can be configured as the source for the LSAD. For example, LSAD 0 can be sourced from DIO 0, DIO 10 or DIO 20.

NOTE: If a DIO is configured for LSAD mode, the pad is disconnected from both the digital input sensor and digital output driver. For DIOs that do not have an associated LSAD, the pad is completely disconnected. For DIOs with an associated LSAD, the pad is connected to the LSAD analog input.

CAUTION: While a DIO can be configured to be both an output and an input, it is the user application's responsibility to ensure that the DIO is not driving an output to a pad that is also being driven externally to Ezairo 7100. If a DIO pad signal has two drivers, the physical values and inputs that are read from this pad are considered undefined.

Table 68. DIO Multiplexed Functionality

Mode	Setting	Description
0x00 0x01	DIO_MODE_GPIO_IN_0 DIO_MODE_GPIO_IN_1	GPIO input mode; bit 0 is the input data value (both DIO_MODE_GPIO_IN_0 and DIO_MODE_GPIO_IN_1 configure the DIO for GPIO input mode)
0x02 0x03	DIO_MODE_GPIO_OUT_0 DIO_MODE_GPIO_OUT_1	GPIO output mode; bit 0 is the output data value (GPIO_OUT_0 provides an output low and GPIO_OUT_1 provides an output high)
0x04	DIO_MODE_SYNC_MASTER	Multi-chip synchronization master output
0x05	DIO_MODE_UCLK	User clock (UCLK) output
0x06	DIO_MODE_SLOWCLK	Slow clock (SLOWCLK) output
0x07	DIO_MODE_SYSCLK	System clock (SYSCLK) output
0x08	DIO_MODE_PCM_SERO	PCM interface serial output signal
0x09	DIO_MODE_PCM_FRAME	PCM interface frame output (PCM master mode only)
0x0A	DIO_MODE_SPI_SERO	SPI serial output
0x0B	DIO_MODE_SPI_CS	SPI chip select output
0x0C	DIO_MODE_SPI_CLK	SPI clock output
0x0D	DIO_MODE_UART_TX	UART interface transmit signal
0x0E	DIO_MODE_SDMCLK	Sigma-delta modulator clock (SDMCLK) output
0x0F	DIO_MODE_SDA	Debug and general-purpose I^2C data output (open collector)
0x10 0x11	DIO_MODE_C_GPIO_IN_0 DIO_MODE_C_GPIO_IN_1	ARM Cortex-M3 core GPIO input mode; bit 0 is the input data value (both C_GPIO_IN_0 and C_GPIO_IN_1 configure the DIO for ARM Cortex-M3 core controlled GPIO input mode)
0x12 0x13	DIO_MODE_C_GPIO_OUT_0 DIO_MODE_C_GPIO_OUT_1	ARM Cortex-M3 core GPIO output mode; bit 0 is the output data value (C_GPIO_OUT_0 provides an output low and C_GPIO_OUT_1 provides an output high)
0x14	DIO_MODE_C_SDA	ARM Cortex-M3 core I^2C data output (open collector)
0x15	DIO_MODE_C_SCL	ARM Cortex-M3 core I^2C clock output (open collector)
0x16	DIO_MODE_DMIC_OUT0	Digital microphone output channel 0
0x17	DIO_MODE_DMIC_OUT1	Digital microphone output channel 1
0x18	DIO_MODE_C_PCM_SERO	ARM Cortex-M3 core PCM interface serial output signal
0x19	DIO_MODE_C_PCM_FRAME	ARM Cortex-M3 core PCM interface frame output (PCM master mode only)
0x1A	DIO_MODE_C_SPI_SERO	ARM Cortex-M3 core SPI serial output

Table 68. DIO Multiplexed Functionality

Mode	Setting	Description
0x1B	DIO_MODE_C_SPI_CS	ARM Cortex-M3 core SPI chip select output (SPI master mode only)
0x1C	DIO_MODE_C_SPI_CLK	ARM Cortex-M3 core SPI clock output (SPI master mode only)
0x1D	DIO_MODE_SCL	Debug and general-purpose I^2C clock output (open collector)
0x1E	DIO_MODE_INPUT	Digital input only
0x1F	DIO_MODE_LSAD	LSAD input; digital input disabled and forced low

Table 69. DIO Input Functionality

Configuration Register	Signal	Description
D_PCM_SRC	PCM_SERI	PCM interface serial input signal
	PCM_FRAME	PCM interface frame input (PCM slave mode only)
	PCM_CLK	PCM interface clock input
D_SPI_SRC	SPI_SERI	SPI serial input signal
D_UART_SRC	UART_RX	UART interface receive signal
D_SYNC_SLAVE_SRC	SYNC_SLAVE	Multi-chip synchronization slave input
D_DAC_SCALE_SRC	DAC_SCALE	DAC output scaling control input
D_C_PCM_SRC	C_PCM_SERI	ARM Cortex-M3 core PCM interface serial input signal
	C_PCM_FRAME	ARM Cortex-M3 core PCM interface frame input (PCM slave mode only)
	C_PCM_CLK	ARM Cortex-M3 core PCM interface clock input
D_C_SPI_SRC	C_SPI_SERI	ARM Cortex-M3 core SPI serial input signal
	C_SPI_CS	ARM Cortex-M3 core SPI chip select input (SPI slave mode only)
	C_SPI_CLK	ARM Cortex-M3 core SPI clock input (SPI slave mode only)
D_C_I2C_SRC	C_I2C_SCL	ARM Cortex-M3 core I^2C clock input (open collector)
	C_I2C_SDA	ARM Cortex-M3 core I^2C data input (open collector)
D_DMIC_SRC	DMIC0	Digital microphone input channel 0
	DMIC1	Digital microphone input channel 1
	DMIC2	Digital microphone input channel 2
	DMIC3	Digital microphone input channel 3
D_PLL_SRC	REF_CLK	PLL reference clock input

16.1.2 Physical Configuration

The Ezairo 7100 system includes physical configuration parameters for each DIO. These parameters are used to configure the pads as:

- Inputs with or without a pull-up resistor and with or without an input low-pass filter
- The drive strength of outputs

In addition to the configurable physical parameters, all DIO pads contain Schmitt triggers to filter out noise observed at the inputs.

The physical configuration for a DIO pad is set using the appropriate D_DIO_CFG_* register. The configuration bits provided by these registers are:

- The DIO_CFG_PUS bit connects or disconnects the strong (1kohm) pull-up resistor
- The DIO_CFG_PUW bit connects or disconnects the weak (250kohm) pull-up resistor
- The DIO_CFG_LPF bit enables a low-pass filter that can be used to clean up the DIO's received input signal
- The DIO_CFG_DRIVE bit allows you to select the drive strength for the DIO interface lines

IMPORTANT: For optimal noise performance, the low-pass filters provided for DIO inputs should be enabled for input signals received at less than 2 MHz. For signals that use a frequency that exceeds 2 MHz, the DIO low-pass filters should be disabled.

16.1.3 Control and Configuration Registers

Register Name	Register Description	Address
D_DIO_CFG_0	Digital I/O configuration	P:0xF280
D_DIO_CFG_1	Digital I/O configuration	P:0xF281
D_DIO_CFG_2	Digital I/O configuration	P:0xF282
D_DIO_CFG_3	Digital I/O configuration	P:0xF283
D_DIO_CFG_4	Digital I/O configuration	P:0xF284
D_DIO_CFG_5	Digital I/O configuration	P:0xF285
D_DIO_CFG_6	Digital I/O configuration	P:0xF286
D_DIO_CFG_7	Digital I/O configuration	P:0xF287
D_DIO_CFG_8	Digital I/O configuration	P:0xF288
D_DIO_CFG_9	Digital I/O configuration	P:0xF289
D_DIO_CFG_10	Digital I/O configuration	P:0xF28A
D_DIO_CFG_11	Digital I/O configuration	P:0xF28B
D_DIO_CFG_12	Digital I/O configuration	P:0xF28C
D_DIO_CFG_13	Digital I/O configuration	P:0xF28D
D_DIO_CFG_14	Digital I/O configuration	P:0xF28E
D_DIO_CFG_15	Digital I/O configuration	P:0xF28F
D_DIO_CFG_16	Digital I/O configuration	P:0xF290
D_DIO_CFG_17	Digital I/O configuration	P:0xF291
D_DIO_CFG_18	Digital I/O configuration	P:0xF292
D_DIO_CFG_19	Digital I/O configuration	P:0xF293
D_DIO_CFG_20	Digital I/O configuration	P:0xF294
D_DIO_CFG_21	Digital I/O configuration	P:0xF295
D_DIO_CFG_22	Digital I/O configuration	P:0xF296
D_DIO_CFG_23	Digital I/O configuration	P:0xF297
D_DIO_CFG_24	Digital I/O configuration	P:0xF298

Register Name	Register Description	Address
D_DIO_CFG_25	Digital I/O configuration	P:0xF299
D_DIO_CFG_26	Digital I/O configuration	P:0xF29A
D_DIO_CFG_27	Digital I/O configuration	P:0xF29B
D_DIO_CFG_28	Digital I/O configuration	P:0xF29C
D_DIO_CFG_29	Digital I/O configuration	P:0xF29D
D_PCM_SRC	PCM input selection	P:0xF2B2
D_SPI_SRC	SPI interface input selection	P:0xF2B3
D_UART_SRC	UART interface input selection	P:0xF2B4
D_SYNC_SLAVE_SRC	Slave sync interface input selection	P:0xF2B5
D_DAC_SCALE_SRC	DA scale interface input selection	P:0xF2B6
D_C_PCM_SRC	C_PCM input selection	P:0xF2B7
D_C_SPI_SRC	C_SPI input selection	P:0xF2B8
D_C_I2C_SRC	C_I2C input selection	P:0xF2B9
D_DMIC_SRC	DMIC input selection	P:0xF2BA
D_PLL_SRC	PLL input selection	P:0xF2BB

16.1.3.1 D_DIO_CFG_* Settings

The following bit fields and field names apply equally to registers D_DIO_CFG_0 to D_DIO_CFG_29.

Bit Field	Field Name	Description
11	DRIVE	Drive strength configuration
10	LPF	Low Pass Filter enable
9	PUW	Weak pull-up resistor enable/disable
8	PUS	Strong pull-up resistor enable/disable
4:0	IO_MODE	I/O mode selection

Field Name	Value Symbol	Value Description	Hex Value
DRIVE	DIO_LOW_DRIVE	Low drive strength	0x0*
	DIO_HIGH_DRIVE	High drive strength	0x1
LPF	DIO_LPF_DISABLE	Disable low pass filter	0x0*
	DIO_LPF_ENABLE	Enable low pass filter	0x1
PUW	DIO_WEAK_PU_DISABLE	Disable the weak pull-up resistor	0x0
	DIO_WEAK_PU_ENABLE	Enable the weak pull-up resistor	0x1*
PUS	DIO_STRONG_PU_DISABLE	Disable the strong pull-up resistor	0x0*
	DIO_STRONG_PU_ENABLE	Enable the strong pull-up resistor	0x1

Field Name	Value Symbol	Value Description	Hex Value
IO_MODE	DIO_MODE_GPIO_IN_0	Input (GPIO input mode, data value 0)	0x0
	DIO_MODE_GPIO_IN_1	Input (GPIO input mode, data value 1)	0x1
	DIO_MODE_GPIO_OUT_0	Output low (GPIO output mode, data value 0)	0x2
	DIO_MODE_GPIO_OUT_1	Output high (GPIO output mode, data value 1)	0x3
	DIO_MODE_SYNC_MASTER	Output master multi-chip synchronization interface signal	0x4
	DIO_MODE_UCLK	Output UCLK (user clock) signal	0x5
	DIO_MODE_SLOWCLK	Output SLOWCLK (slow clock) signal	0x6
	DIO_MODE_SYSCLK	Output SYSCLK (system clock) signal	0x7
	DIO_MODE_PCM_SERO	Output PCM_SERO interface signal	0x8
	DIO_MODE_PCM_FRAME	Output PCM_FRAME interface signal	0x9
	DIO_MODE_SPI_SERO	Output SPI_SERO interface signal	0xA
	DIO_MODE_SPI_CS	Output SPI_CS interface signal	0xB
	DIO_MODE_SPI_CLK	Output SPI_CLK interface signal	0xC
	DIO_MODE_UART_TX	Output UART_TX interface signal	0xD
	DIO_MODE_SDMCLK	Output SDMCLK signal	0xE
	DIO_MODE_SDA	Output SDA interface signal (open collector)	0xF
	DIO_MODE_C_GPIO_IN_0	Input (ARM Cortex-M3 core GPIO input mode, data value 0)	0x10
IO_MODE (continued)	DIO_MODE_C_GPIO_IN_1	Input (ARM Cortex-M3 core GPIO input mode, data value 1)	0x11
	DIO_MODE_C_GPIO_OUT_0	Output low (ARM Cortex-M3 core GPIO output mode, data value 0)	0x12
	DIO_MODE_C_GPIO_OUT_1	Output high (ARM Cortex-M3 core GPIO output mode, data value 1)	0x13
	DIO_MODE_C_SDA	Output C_SDA interface signal (open collector)	0x14
	DIO_MODE_C_SCL	Output C_SCL interface signal (open collector)	0x15
	DIO_MODE_DMIC_OUT0	Output DMIC_OUT0	0x16
	DIO_MODE_DMIC_OUT1	Output DMIC_OUT1	0x17
	DIO_MODE_C_PCM_SERO	Output C_PCM_SERO interface signal	0x18
	DIO_MODE_C_PCM_FRAME	Output C_PCM_FRAME interface signal	0x19
	DIO_MODE_C_SPI_SERO	Output C_SPI_SERO interface signal	0x1A
	DIO_MODE_C_SPI_CS	Output C_SPI_CS interface signal	0x1B
	DIO_MODE_C_SPI_CLK	Output C_SPI_CLK interface signal	0x1C
	DIO_MODE_SCL	Output SCL interface signal (open collector)	0x1D
	DIO_MODE_INPUT	Digital input enabled	0x1E
	DIO_MODE_LSAD	LSAD input selected. Signal to core is forced low. Digital input disabled.	0x1F

16.1.3.2 D_PCM_SRC Settings

Bit Field	Field Name	Description
20:16	SERI	PCM_SERI input selection

Bit Field	Field Name	Description
12:8	FRAME	PCM_FRAME input selection
4:0	CLK	PCM_CLK input selection

Field Name	Value Symbol	Value Description	Hex Value
SERI	PCM_SERI_SRC_DIO_0	Select DIO[0] as source	0x0
	PCM_SERI_SRC_DIO_1	Select DIO[1] as source	0x1
	PCM_SERI_SRC_DIO_2	Select DIO[2] as source	0x2
	PCM_SERI_SRC_DIO_3	Select DIO[3] as source	0x3
	PCM_SERI_SRC_DIO_4	Select DIO[4] as source	0x4
	PCM_SERI_SRC_DIO_5	Select DIO[5] as source	0x5
	PCM_SERI_SRC_DIO_6	Select DIO[6] as source	0x6
	PCM_SERI_SRC_DIO_7	Select DIO[7] as source	0x7
	PCM_SERI_SRC_DIO_8	Select DIO[8] as source	0x8
	PCM_SERI_SRC_DIO_9	Select DIO[9] as source	0x9
	PCM_SERI_SRC_DIO_10	Select DIO[10] as source	0xA
	PCM_SERI_SRC_DIO_11	Select DIO[11] as source	0xB
	PCM_SERI_SRC_DIO_12	Select DIO[12] as source	0xC
	PCM_SERI_SRC_DIO_13	Select DIO[13] as source	0xD
	PCM_SERI_SRC_DIO_14	Select DIO[14] as source	0xE
	PCM_SERI_SRC_DIO_15	Select DIO[15] as source	0xF
	PCM_SERI_SRC_DIO_16	Select DIO[16] as source	0x10
	PCM_SERI_SRC_DIO_17	Select DIO[17] as source	0x11
	PCM_SERI_SRC_DIO_18	Select DIO[18] as source	0x12
	PCM_SERI_SRC_DIO_19	Select DIO[19] as source	0x13
	PCM_SERI_SRC_DIO_20	Select DIO[20] as source	0x14
	PCM_SERI_SRC_DIO_21	Select DIO[21] as source	0x15
	PCM_SERI_SRC_DIO_22	Select DIO[22] as source	0x16
	PCM_SERI_SRC_DIO_23	Select DIO[23] as source	0x17
	PCM_SERI_SRC_DIO_24	Select DIO[24] as source	0x18
	PCM_SERI_SRC_DIO_25	Select DIO[25] as source	0x19
	PCM_SERI_SRC_DIO_26	Select DIO[26] as source	0x1A
	PCM_SERI_SRC_DIO_27	Select DIO[27] as source	0x1B
	PCM_SERI_SRC_DIO_28	Select DIO[28] as source	0x1C
	PCM_SERI_SRC_DIO_29	Select DIO[29] as source	0x1D
	PCM_SERI_SRC_CONST_LOW	Select constant low as source	0x1E
	PCM_SERI_SRC_CONST_HIGH	Select constant high as source	0x1F*

Field Name	Value Symbol	Value Description	Hex Value
FRAME	PCM_FRAME_SRC_DIO_0	Select DIO[0] as source	0x0
	PCM_FRAME_SRC_DIO_1	Select DIO[1] as source	0x1
	PCM_FRAME_SRC_DIO_2	Select DIO[2] as source	0x2
	PCM_FRAME_SRC_DIO_3	Select DIO[3] as source	0x3
	PCM_FRAME_SRC_DIO_4	Select DIO[4] as source	0x4
	PCM_FRAME_SRC_DIO_5	Select DIO[5] as source	0x5
	PCM_FRAME_SRC_DIO_6	Select DIO[6] as source	0x6
	PCM_FRAME_SRC_DIO_7	Select DIO[7] as source	0x7
	PCM_FRAME_SRC_DIO_8	Select DIO[8] as source	0x8
	PCM_FRAME_SRC_DIO_9	Select DIO[9] as source	0x9
	PCM_FRAME_SRC_DIO_10	Select DIO[10] as source	0xA
	PCM_FRAME_SRC_DIO_11	Select DIO[11] as source	0xB
	PCM_FRAME_SRC_DIO_12	Select DIO[12] as source	0xC
	PCM_FRAME_SRC_DIO_13	Select DIO[13] as source	0xD
	PCM_FRAME_SRC_DIO_14	Select DIO[14] as source	0xE
	PCM_FRAME_SRC_DIO_15	Select DIO[15] as source	0xF
	PCM_FRAME_SRC_DIO_16	Select DIO[16] as source	0x10
	PCM_FRAME_SRC_DIO_17	Select DIO[17] as source	0x11
	PCM_FRAME_SRC_DIO_18	Select DIO[18] as source	0x12
	PCM_FRAME_SRC_DIO_19	Select DIO[19] as source	0x13
	PCM_FRAME_SRC_DIO_20	Select DIO[20] as source	0x14
	PCM_FRAME_SRC_DIO_21	Select DIO[21] as source	0x15
	PCM_FRAME_SRC_DIO_22	Select DIO[22] as source	0x16
	PCM_FRAME_SRC_DIO_23	Select DIO[23] as source	0x17
	PCM_FRAME_SRC_DIO_24	Select DIO[24] as source	0x18
	PCM_FRAME_SRC_DIO_25	Select DIO[25] as source	0x19
	PCM_FRAME_SRC_DIO_26	Select DIO[26] as source	0x1A
	PCM_FRAME_SRC_DIO_27	Select DIO[27] as source	0x1B
	PCM_FRAME_SRC_DIO_28	Select DIO[28] as source	0x1C
	PCM_FRAME_SRC_DIO_29	Select DIO[29] as source	0x1D
	PCM_FRAME_SRC_CONST_LOW	Select constant low as source	0x1E
	PCM_FRAME_SRC_CONST_HIGH	Select constant high as source	0x1F*

Field Name	Value Symbol	Value Description	Hex Value
CLK	PCM_CLK_SRC_DIO_0	Select DIO[0] as source	0x0
	PCM_CLK_SRC_DIO_1	Select DIO[1] as source	0x1
	PCM_CLK_SRC_DIO_2	Select DIO[2] as source	0x2
	PCM_CLK_SRC_DIO_3	Select DIO[3] as source	0x3
	PCM_CLK_SRC_DIO_4	Select DIO[4] as source	0x4
	PCM_CLK_SRC_DIO_5	Select DIO[5] as source	0x5
	PCM_CLK_SRC_DIO_6	Select DIO[6] as source	0x6
	PCM_CLK_SRC_DIO_7	Select DIO[7] as source	0x7
	PCM_CLK_SRC_DIO_8	Select DIO[8] as source	0x8
	PCM_CLK_SRC_DIO_9	Select DIO[9] as source	0x9
	PCM_CLK_SRC_DIO_10	Select DIO[10] as source	0xA
	PCM_CLK_SRC_DIO_11	Select DIO[11] as source	0xB
	PCM_CLK_SRC_DIO_12	Select DIO[12] as source	0xC
	PCM_CLK_SRC_DIO_13	Select DIO[13] as source	0xD
	PCM_CLK_SRC_DIO_14	Select DIO[14] as source	0xE
	PCM_CLK_SRC_DIO_15	Select DIO[15] as source	0xF
	PCM_CLK_SRC_DIO_16	Select DIO[16] as source	0x10
	PCM_CLK_SRC_DIO_17	Select DIO[17] as source	0x11
	PCM_CLK_SRC_DIO_18	Select DIO[18] as source	0x12
	PCM_CLK_SRC_DIO_19	Select DIO[19] as source	0x13
	PCM_CLK_SRC_DIO_20	Select DIO[20] as source	0x14
	PCM_CLK_SRC_DIO_21	Select DIO[21] as source	0x15
	PCM_CLK_SRC_DIO_22	Select DIO[22] as source	0x16
	PCM_CLK_SRC_DIO_23	Select DIO[23] as source	0x17
	PCM_CLK_SRC_DIO_24	Select DIO[24] as source	0x18
	PCM_CLK_SRC_DIO_25	Select DIO[25] as source	0x19
	PCM_CLK_SRC_DIO_26	Select DIO[26] as source	0x1A
	PCM_CLK_SRC_DIO_27	Select DIO[27] as source	0x1B
	PCM_CLK_SRC_DIO_28	Select DIO[28] as source	0x1C
	PCM_CLK_SRC_DIO_29	Select DIO[29] as source	0x1D
	PCM_CLK_SRC_CONST_LOW	Select constant low as source	0x1E
	PCM_CLK_SRC_CONST_HIGH	Select constant high as source	0x1F*

16.1.3.3 D_SPI_SRC Settings

Bit Field	Field Name	Description
4:0	SERI	SPI_SERI input selection

Field Name	Value Symbol	Value Description	Hex Value
SERI	SPI_SERI_SRC_DIO_0	Select DIO[0] as source	0x0
	SPI_SERI_SRC_DIO_1	Select DIO[1] as source	0x1
	SPI_SERI_SRC_DIO_2	Select DIO[2] as source	0x2
	SPI_SERI_SRC_DIO_3	Select DIO[3] as source	0x3
	SPI_SERI_SRC_DIO_4	Select DIO[4] as source	0x4
	SPI_SERI_SRC_DIO_5	Select DIO[5] as source	0x5
	SPI_SERI_SRC_DIO_6	Select DIO[6] as source	0x6
	SPI_SERI_SRC_DIO_7	Select DIO[7] as source	0x7
	SPI_SERI_SRC_DIO_8	Select DIO[8] as source	0x8
	SPI_SERI_SRC_DIO_9	Select DIO[9] as source	0x9
	SPI_SERI_SRC_DIO_10	Select DIO[10] as source	0xA
	SPI_SERI_SRC_DIO_11	Select DIO[11] as source	0xB
	SPI_SERI_SRC_DIO_12	Select DIO[12] as source	0xC
	SPI_SERI_SRC_DIO_13	Select DIO[13] as source	0xD
	SPI_SERI_SRC_DIO_14	Select DIO[14] as source	0xE
	SPI_SERI_SRC_DIO_15	Select DIO[15] as source	0xF
	SPI_SERI_SRC_DIO_16	Select DIO[16] as source	0x10
	SPI_SERI_SRC_DIO_17	Select DIO[17] as source	0x11
	SPI_SERI_SRC_DIO_18	Select DIO[18] as source	0x12
	SPI_SERI_SRC_DIO_19	Select DIO[19] as source	0x13
	SPI_SERI_SRC_DIO_20	Select DIO[20] as source	0x14
	SPI_SERI_SRC_DIO_21	Select DIO[21] as source	0x15
	SPI_SERI_SRC_DIO_22	Select DIO[22] as source	0x16
	SPI_SERI_SRC_DIO_23	Select DIO[23] as source	0x17
	SPI_SERI_SRC_DIO_24	Select DIO[24] as source	0x18
	SPI_SERI_SRC_DIO_25	Select DIO[25] as source	0x19
	SPI_SERI_SRC_DIO_26	Select DIO[26] as source	0x1A
	SPI_SERI_SRC_DIO_27	Select DIO[27] as source	0x1B
	SPI_SERI_SRC_DIO_28	Select DIO[28] as source	0x1C
	SPI_SERI_SRC_DIO_29	Select DIO[29] as source	0x1D
	SPI_SERI_SRC_CONST_LOW	Select constant low as source	0x1E
	SPI_SERI_SRC_CONST_HIGH	Select constant high as source	0x1F*

16.1.3.4 D_UART_SRC Settings

Bit Field	Field Name	Description
4:0	RX	UART_RX input selection

Field Name	Value Symbol	Value Description	Hex Value
RX	UART_RX_SRC_DIO_0	Select DIO[0] as source	0x0
	UART_RX_SRC_DIO_1	Select DIO[1] as source	0x1
	UART_RX_SRC_DIO_2	Select DIO[2] as source	0x2
	UART_RX_SRC_DIO_3	Select DIO[3] as source	0x3
	UART_RX_SRC_DIO_4	Select DIO[4] as source	0x4
	UART_RX_SRC_DIO_5	Select DIO[5] as source	0x5
	UART_RX_SRC_DIO_6	Select DIO[6] as source	0x6
	UART_RX_SRC_DIO_7	Select DIO[7] as source	0x7
	UART_RX_SRC_DIO_8	Select DIO[8] as source	0x8
	UART_RX_SRC_DIO_9	Select DIO[9] as source	0x9
	UART_RX_SRC_DIO_10	Select DIO[10] as source	0xA
	UART_RX_SRC_DIO_11	Select DIO[11] as source	0xB
	UART_RX_SRC_DIO_12	Select DIO[12] as source	0xC
	UART_RX_SRC_DIO_13	Select DIO[13] as source	0xD
	UART_RX_SRC_DIO_14	Select DIO[14] as source	0xE
	UART_RX_SRC_DIO_15	Select DIO[15] as source	0xF
	UART_RX_SRC_DIO_16	Select DIO[16] as source	0x10
	UART_RX_SRC_DIO_17	Select DIO[17] as source	0x11
	UART_RX_SRC_DIO_18	Select DIO[18] as source	0x12
	UART_RX_SRC_DIO_19	Select DIO[19] as source	0x13
	UART_RX_SRC_DIO_20	Select DIO[20] as source	0x14
	UART_RX_SRC_DIO_21	Select DIO[21] as source	0x15
	UART_RX_SRC_DIO_22	Select DIO[22] as source	0x16
	UART_RX_SRC_DIO_23	Select DIO[23] as source	0x17
	UART_RX_SRC_DIO_24	Select DIO[24] as source	0x18
	UART_RX_SRC_DIO_25	Select DIO[25] as source	0x19
	UART_RX_SRC_DIO_26	Select DIO[26] as source	0x1A
	UART_RX_SRC_DIO_27	Select DIO[27] as source	0x1B
	UART_RX_SRC_DIO_28	Select DIO[28] as source	0x1C
	UART_RX_SRC_DIO_29	Select DIO[29] as source	0x1D
	UART_RX_SRC_CONST_LOW	Select constant low as source	0x1E
	UART_RX_SRC_CONST_HIGH	Select constant high as source	0x1F*

16.1.3.5 D_SYNC_SLAVE_SRC Settings

Bit Field	Field Name	Description
4:0	SYNC	SYNC_SLAVE input selection

Field Name	Value Symbol	Value Description	Hex Value
SYNC	SYNC_SRC_DIO_0	Select DIO[0] as source	0x0
	SYNC_SRC_DIO_1	Select DIO[1] as source	0x1
	SYNC_SRC_DIO_2	Select DIO[2] as source	0x2
	SYNC_SRC_DIO_3	Select DIO[3] as source	0x3
	SYNC_SRC_DIO_4	Select DIO[4] as source	0x4
	SYNC_SRC_DIO_5	Select DIO[5] as source	0x5
	SYNC_SRC_DIO_6	Select DIO[6] as source	0x6
	SYNC_SRC_DIO_7	Select DIO[7] as source	0x7
	SYNC_SRC_DIO_8	Select DIO[8] as source	0x8
	SYNC_SRC_DIO_9	Select DIO[9] as source	0x9
	SYNC_SRC_DIO_10	Select DIO[10] as source	0xA
	SYNC_SRC_DIO_11	Select DIO[11] as source	0xB
	SYNC_SRC_DIO_12	Select DIO[12] as source	0xC
	SYNC_SRC_DIO_13	Select DIO[13] as source	0xD
	SYNC_SRC_DIO_14	Select DIO[14] as source	0xE
	SYNC_SRC_DIO_15	Select DIO[15] as source	0xF
	SYNC_SRC_DIO_16	Select DIO[16] as source	0x10
	SYNC_SRC_DIO_17	Select DIO[17] as source	0x11
	SYNC_SRC_DIO_18	Select DIO[18] as source	0x12
	SYNC_SRC_DIO_19	Select DIO[19] as source	0x13
	SYNC_SRC_DIO_20	Select DIO[20] as source	0x14
	SYNC_SRC_DIO_21	Select DIO[21] as source	0x15
	SYNC_SRC_DIO_22	Select DIO[22] as source	0x16
	SYNC_SRC_DIO_23	Select DIO[23] as source	0x17
	SYNC_SRC_DIO_24	Select DIO[24] as source	0x18
	SYNC_SRC_DIO_25	Select DIO[25] as source	0x19
	SYNC_SRC_DIO_26	Select DIO[26] as source	0x1A
	SYNC_SRC_DIO_27	Select DIO[27] as source	0x1B
	SYNC_SRC_DIO_28	Select DIO[28] as source	0x1C
	SYNC_SRC_DIO_29	Select DIO[29] as source	0x1D
	SYNC_SRC_CONST_LOW	Select constant low as source	0x1E
	SYNC_SRC_CONST_HIGH	Select constant high as source	0x1F*

16.1.3.6 D_DAC_SCALE_SRC Settings

Bit Field	Field Name	Description
4:0	DAC_SCALE	DAC_SCALE input selection

Field Name	Value Symbol	Value Description	Hex Value
DAC_SCALE	DAC_SCALE_SRC_DIO_0	Select DIO[0] as source	0x0
	DAC_SCALE_SRC_DIO_1	Select DIO[1] as source	0x1
	DAC_SCALE_SRC_DIO_2	Select DIO[2] as source	0x2
	DAC_SCALE_SRC_DIO_3	Select DIO[3] as source	0x3
	DAC_SCALE_SRC_DIO_4	Select DIO[4] as source	0x4
	DAC_SCALE_SRC_DIO_5	Select DIO[5] as source	0x5
	DAC_SCALE_SRC_DIO_6	Select DIO[6] as source	0x6
	DAC_SCALE_SRC_DIO_7	Select DIO[7] as source	0x7
	DAC_SCALE_SRC_DIO_8	Select DIO[8] as source	0x8
	DAC_SCALE_SRC_DIO_9	Select DIO[9] as source	0x9
	DAC_SCALE_SRC_DIO_10	Select DIO[10] as source	0xA
	DAC_SCALE_SRC_DIO_11	Select DIO[11] as source	0xB
	DAC_SCALE_SRC_DIO_12	Select DIO[12] as source	0xC
	DAC_SCALE_SRC_DIO_13	Select DIO[13] as source	0xD
	DAC_SCALE_SRC_DIO_14	Select DIO[14] as source	0xE
	DAC_SCALE_SRC_DIO_15	Select DIO[15] as source	0xF
	DAC_SCALE_SRC_DIO_16	Select DIO[16] as source	0x10
	DAC_SCALE_SRC_DIO_17	Select DIO[17] as source	0x11
	DAC_SCALE_SRC_DIO_18	Select DIO[18] as source	0x12
	DAC_SCALE_SRC_DIO_19	Select DIO[19] as source	0x13
	DAC_SCALE_SRC_DIO_20	Select DIO[20] as source	0x14
	DAC_SCALE_SRC_DIO_21	Select DIO[21] as source	0x15
	DAC_SCALE_SRC_DIO_22	Select DIO[22] as source	0x16
	DAC_SCALE_SRC_DIO_23	Select DIO[23] as source	0x17
	DAC_SCALE_SRC_DIO_24	Select DIO[24] as source	0x18
	DAC_SCALE_SRC_DIO_25	Select DIO[25] as source	0x19
	DAC_SCALE_SRC_DIO_26	Select DIO[26] as source	0x1A
	DAC_SCALE_SRC_DIO_27	Select DIO[27] as source	0x1B
	DAC_SCALE_SRC_DIO_28	Select DIO[28] as source	0x1C
	DAC_SCALE_SRC_DIO_29	Select DIO[29] as source	0x1D
	DAC_SCALE_SRC_CONST_LOW	Select constant low as source	0x1E
	DAC_SCALE_SRC_CONST_HIGH	Select constant high as source	0x1F*

16.1.3.7 D_C_PCM_SRC Settings

Bit Field	Field Name	Description
20:16	SERI	C_PCM_SERI input selection
12:8	FRAME	C_PCM_FRAME input selection
4:0	CLK	C_PCM_CLK input selection

Field Name	Value Symbol	Value Description	Hex Value
SERI	C_PCM_SERI_SRC_DIO_0	Select DIO[0] as source	0x0
	C_PCM_SERI_SRC_DIO_1	Select DIO[1] as source	0x1
	C_PCM_SERI_SRC_DIO_2	Select DIO[2] as source	0x2
	C_PCM_SERI_SRC_DIO_3	Select DIO[3] as source	0x3
	C_PCM_SERI_SRC_DIO_4	Select DIO[4] as source	0x4
	C_PCM_SERI_SRC_DIO_5	Select DIO[5] as source	0x5
	C_PCM_SERI_SRC_DIO_6	Select DIO[6] as source	0x6
	C_PCM_SERI_SRC_DIO_7	Select DIO[7] as source	0x7
	C_PCM_SERI_SRC_DIO_8	Select DIO[8] as source	0x8
	C_PCM_SERI_SRC_DIO_9	Select DIO[9] as source	0x9
	C_PCM_SERI_SRC_DIO_10	Select DIO[10] as source	0xA
	C_PCM_SERI_SRC_DIO_11	Select DIO[11] as source	0xB
	C_PCM_SERI_SRC_DIO_12	Select DIO[12] as source	0xC
	C_PCM_SERI_SRC_DIO_13	Select DIO[13] as source	0xD
	C_PCM_SERI_SRC_DIO_14	Select DIO[14] as source	0xE
	C_PCM_SERI_SRC_DIO_15	Select DIO[15] as source	0xF
	C_PCM_SERI_SRC_DIO_16	Select DIO[16] as source	0x10
	C_PCM_SERI_SRC_DIO_17	Select DIO[17] as source	0x11
	C_PCM_SERI_SRC_DIO_18	Select DIO[18] as source	0x12
	C_PCM_SERI_SRC_DIO_19	Select DIO[19] as source	0x13
	C_PCM_SERI_SRC_DIO_20	Select DIO[20] as source	0x14
	C_PCM_SERI_SRC_DIO_21	Select DIO[21] as source	0x15
	C_PCM_SERI_SRC_DIO_22	Select DIO[22] as source	0x16
	C_PCM_SERI_SRC_DIO_23	Select DIO[23] as source	0x17
	C_PCM_SERI_SRC_DIO_24	Select DIO[24] as source	0x18
	C_PCM_SERI_SRC_DIO_25	Select DIO[25] as source	0x19
	C_PCM_SERI_SRC_DIO_26	Select DIO[26] as source	0x1A
	C_PCM_SERI_SRC_DIO_27	Select DIO[27] as source	0x1B
	C_PCM_SERI_SRC_DIO_28	Select DIO[28] as source	0x1C
	C_PCM_SERI_SRC_DIO_29	Select DIO[29] as source	0x1D
	C_PCM_SERI_SRC_CONST_LOW	Select constant low as source	0x1E
	C_PCM_SERI_SRC_CONST_HIGH	Select constant high as source	0x1F*

Field Name	Value Symbol	Value Description	Hex Value
FRAME	C_PCM_FRAME_SRC_DIO_0	Select DIO[0] as source	0x0
	C_PCM_FRAME_SRC_DIO_1	Select DIO[1] as source	0x1
	C_PCM_FRAME_SRC_DIO_2	Select DIO[2] as source	0x2
	C_PCM_FRAME_SRC_DIO_3	Select DIO[3] as source	0x3
	C_PCM_FRAME_SRC_DIO_4	Select DIO[4] as source	0x4
	C_PCM_FRAME_SRC_DIO_5	Select DIO[5] as source	0x5
	C_PCM_FRAME_SRC_DIO_6	Select DIO[6] as source	0x6
	C_PCM_FRAME_SRC_DIO_7	Select DIO[7] as source	0x7
	C_PCM_FRAME_SRC_DIO_8	Select DIO[8] as source	0x8
	C_PCM_FRAME_SRC_DIO_9	Select DIO[9] as source	0x9
	C_PCM_FRAME_SRC_DIO_10	Select DIO[10] as source	0xA
	C_PCM_FRAME_SRC_DIO_11	Select DIO[11] as source	0xB
	C_PCM_FRAME_SRC_DIO_12	Select DIO[12] as source	0xC
	C_PCM_FRAME_SRC_DIO_13	Select DIO[13] as source	0xD
	C_PCM_FRAME_SRC_DIO_14	Select DIO[14] as source	0xE
	C_PCM_FRAME_SRC_DIO_15	Select DIO[15] as source	0xF
	C_PCM_FRAME_SRC_DIO_16	Select DIO[16] as source	0x10
	C_PCM_FRAME_SRC_DIO_17	Select DIO[17] as source	0x11
	C_PCM_FRAME_SRC_DIO_18	Select DIO[18] as source	0x12
	C_PCM_FRAME_SRC_DIO_19	Select DIO[19] as source	0x13
	C_PCM_FRAME_SRC_DIO_20	Select DIO[20] as source	0x14
	C_PCM_FRAME_SRC_DIO_21	Select DIO[21] as source	0x15
	C_PCM_FRAME_SRC_DIO_22	Select DIO[22] as source	0x16
	C_PCM_FRAME_SRC_DIO_23	Select DIO[23] as source	0x17
	C_PCM_FRAME_SRC_DIO_24	Select DIO[24] as source	0x18
	C_PCM_FRAME_SRC_DIO_25	Select DIO[25] as source	0x19
	C_PCM_FRAME_SRC_DIO_26	Select DIO[26] as source	0x1A
	C_PCM_FRAME_SRC_DIO_27	Select DIO[27] as source	0x1B
	C_PCM_FRAME_SRC_DIO_28	Select DIO[28] as source	0x1C
	C_PCM_FRAME_SRC_DIO_29	Select DIO[29] as source	0x1D
	C_PCM_FRAME_SRC_CONST_LOW	Select constant low as source	0x1E
	C_PCM_FRAME_SRC_CONST_HIGH	Select constant high as source	0x1F*

Field Name	Value Symbol	Value Description	Hex Value
CLK	C_PCM_CLK_SRC_DIO_0	Select DIO[0] as source	0x0
	C_PCM_CLK_SRC_DIO_1	Select DIO[1] as source	0x1
	C_PCM_CLK_SRC_DIO_2	Select DIO[2] as source	0x2
	C_PCM_CLK_SRC_DIO_3	Select DIO[3] as source	0x3
	C_PCM_CLK_SRC_DIO_4	Select DIO[4] as source	0x4
	C_PCM_CLK_SRC_DIO_5	Select DIO[5] as source	0x5
	C_PCM_CLK_SRC_DIO_6	Select DIO[6] as source	0x6
	C_PCM_CLK_SRC_DIO_7	Select DIO[7] as source	0x7
	C_PCM_CLK_SRC_DIO_8	Select DIO[8] as source	0x8
	C_PCM_CLK_SRC_DIO_9	Select DIO[9] as source	0x9
	C_PCM_CLK_SRC_DIO_10	Select DIO[10] as source	0xA
	C_PCM_CLK_SRC_DIO_11	Select DIO[11] as source	0xB
	C_PCM_CLK_SRC_DIO_12	Select DIO[12] as source	0xC
	C_PCM_CLK_SRC_DIO_13	Select DIO[13] as source	0xD
	C_PCM_CLK_SRC_DIO_14	Select DIO[14] as source	0xE
	C_PCM_CLK_SRC_DIO_15	Select DIO[15] as source	0xF
	C_PCM_CLK_SRC_DIO_16	Select DIO[16] as source	0x10
	C_PCM_CLK_SRC_DIO_17	Select DIO[17] as source	0x11
	C_PCM_CLK_SRC_DIO_18	Select DIO[18] as source	0x12
	C_PCM_CLK_SRC_DIO_19	Select DIO[19] as source	0x13
	C_PCM_CLK_SRC_DIO_20	Select DIO[20] as source	0x14
	C_PCM_CLK_SRC_DIO_21	Select DIO[21] as source	0x15
	C_PCM_CLK_SRC_DIO_22	Select DIO[22] as source	0x16
	C_PCM_CLK_SRC_DIO_23	Select DIO[23] as source	0x17
	C_PCM_CLK_SRC_DIO_24	Select DIO[24] as source	0x18
	C_PCM_CLK_SRC_DIO_25	Select DIO[25] as source	0x19
	C_PCM_CLK_SRC_DIO_26	Select DIO[26] as source	0x1A
	C_PCM_CLK_SRC_DIO_27	Select DIO[27] as source	0x1B
	C_PCM_CLK_SRC_DIO_28	Select DIO[28] as source	0x1C
	C_PCM_CLK_SRC_DIO_29	Select DIO[29] as source	0x1D
	C_PCM_CLK_SRC_CONST_LOW	Select constant low as source	0x1E
	C_PCM_CLK_SRC_CONST_HIGH	Select constant high as source	0x1F*

16.1.3.8 D_C_SPI_SRC Settings

Bit Field	Field Name	Description
20:16	SERI	C_SPI_SERI input selection
12:8	CS	C_SPI_CS input selection
4:0	CLK	C_SPI_CLK input selection

Field Name	Value Symbol	Value Description	Hex Value
SERI	C_SPI_SERI_SRC_DIO_0	Select DIO[0] as source	0x0
	C_SPI_SERI_SRC_DIO_1	Select DIO[1] as source	0x1
	C_SPI_SERI_SRC_DIO_2	Select DIO[2] as source	0x2
	C_SPI_SERI_SRC_DIO_3	Select DIO[3] as source	0x3
	C_SPI_SERI_SRC_DIO_4	Select DIO[4] as source	0x4
	C_SPI_SERI_SRC_DIO_5	Select DIO[5] as source	0x5
	C_SPI_SERI_SRC_DIO_6	Select DIO[6] as source	0x6
	C_SPI_SERI_SRC_DIO_7	Select DIO[7] as source	0x7
	C_SPI_SERI_SRC_DIO_8	Select DIO[8] as source	0x8
	C_SPI_SERI_SRC_DIO_9	Select DIO[9] as source	0x9
	C_SPI_SERI_SRC_DIO_10	Select DIO[10] as source	0xA
	C_SPI_SERI_SRC_DIO_11	Select DIO[11] as source	0xB
	C_SPI_SERI_SRC_DIO_12	Select DIO[12] as source	0xC
	C_SPI_SERI_SRC_DIO_13	Select DIO[13] as source	0xD
	C_SPI_SERI_SRC_DIO_14	Select DIO[14] as source	0xE
	C_SPI_SERI_SRC_DIO_15	Select DIO[15] as source	0xF
	C_SPI_SERI_SRC_DIO_16	Select DIO[16] as source	0x10
	C_SPI_SERI_SRC_DIO_17	Select DIO[17] as source	0x11
	C_SPI_SERI_SRC_DIO_18	Select DIO[18] as source	0x12
	C_SPI_SERI_SRC_DIO_19	Select DIO[19] as source	0x13
	C_SPI_SERI_SRC_DIO_20	Select DIO[20] as source	0x14
	C_SPI_SERI_SRC_DIO_21	Select DIO[21] as source	0x15
	C_SPI_SERI_SRC_DIO_22	Select DIO[22] as source	0x16
	C_SPI_SERI_SRC_DIO_23	Select DIO[23] as source	0x17
	C_SPI_SERI_SRC_DIO_24	Select DIO[24] as source	0x18
	C_SPI_SERI_SRC_DIO_25	Select DIO[25] as source	0x19
	C_SPI_SERI_SRC_DIO_26	Select DIO[26] as source	0x1A
	C_SPI_SERI_SRC_DIO_27	Select DIO[27] as source	0x1B
	C_SPI_SERI_SRC_DIO_28	Select DIO[28] as source	0x1C
	C_SPI_SERI_SRC_DIO_29	Select DIO[29] as source	0x1D
	C_SPI_SERI_SRC_CONST_LOW	Select constant low as source	0x1E
	C_SPI_SERI_SRC_CONST_HIGH	Select constant high as source	0x1F*

Field Name	Value Symbol	Value Description	Hex Value
CS	C_SPI_CS_SRC_DIO_0	Select DIO[0] as source	0x0
	C_SPI_CS_SRC_DIO_1	Select DIO[1] as source	0x1
	C_SPI_CS_SRC_DIO_2	Select DIO[2] as source	0x2
	C_SPI_CS_SRC_DIO_3	Select DIO[3] as source	0x3
	C_SPI_CS_SRC_DIO_4	Select DIO[4] as source	0x4
	C_SPI_CS_SRC_DIO_5	Select DIO[5] as source	0x5
	C_SPI_CS_SRC_DIO_6	Select DIO[6] as source	0x6
	C_SPI_CS_SRC_DIO_7	Select DIO[7] as source	0x7
	C_SPI_CS_SRC_DIO_8	Select DIO[8] as source	0x8
	C_SPI_CS_SRC_DIO_9	Select DIO[9] as source	0x9
	C_SPI_CS_SRC_DIO_10	Select DIO[10] as source	0xA
	C_SPI_CS_SRC_DIO_11	Select DIO[11] as source	0xB
	C_SPI_CS_SRC_DIO_12	Select DIO[12] as source	0xC
	C_SPI_CS_SRC_DIO_13	Select DIO[13] as source	0xD
	C_SPI_CS_SRC_DIO_14	Select DIO[14] as source	0xE
	C_SPI_CS_SRC_DIO_15	Select DIO[15] as source	0xF
	C_SPI_CS_SRC_DIO_16	Select DIO[16] as source	0x10
	C_SPI_CS_SRC_DIO_17	Select DIO[17] as source	0x11
	C_SPI_CS_SRC_DIO_18	Select DIO[18] as source	0x12
	C_SPI_CS_SRC_DIO_19	Select DIO[19] as source	0x13
	C_SPI_CS_SRC_DIO_20	Select DIO[20] as source	0x14
	C_SPI_CS_SRC_DIO_21	Select DIO[21] as source	0x15
	C_SPI_CS_SRC_DIO_22	Select DIO[22] as source	0x16
	C_SPI_CS_SRC_DIO_23	Select DIO[23] as source	0x17
	C_SPI_CS_SRC_DIO_24	Select DIO[24] as source	0x18
	C_SPI_CS_SRC_DIO_25	Select DIO[25] as source	0x19
	C_SPI_CS_SRC_DIO_26	Select DIO[26] as source	0x1A
	C_SPI_CS_SRC_DIO_27	Select DIO[27] as source	0x1B
	C_SPI_CS_SRC_DIO_28	Select DIO[28] as source	0x1C
	C_SPI_CS_SRC_DIO_29	Select DIO[29] as source	0x1D
	C_SPI_CS_SRC_CONST_LOW	Select constant low as source	0x1E
	C_SPI_CS_SRC_CONST_HIGH	Select constant high as source	0x1F*

Field Name	Value Symbol	Value Description	Hex Value
CLK	C_SPI_CLK_SRC_DIO_0	Select DIO[0] as source	0x0
	C_SPI_CLK_SRC_DIO_1	Select DIO[1] as source	0x1
	C_SPI_CLK_SRC_DIO_2	Select DIO[2] as source	0x2
	C_SPI_CLK_SRC_DIO_3	Select DIO[3] as source	0x3
	C_SPI_CLK_SRC_DIO_4	Select DIO[4] as source	0x4
	C_SPI_CLK_SRC_DIO_5	Select DIO[5] as source	0x5
	C_SPI_CLK_SRC_DIO_6	Select DIO[6] as source	0x6
	C_SPI_CLK_SRC_DIO_7	Select DIO[7] as source	0x7
	C_SPI_CLK_SRC_DIO_8	Select DIO[8] as source	0x8
	C_SPI_CLK_SRC_DIO_9	Select DIO[9] as source	0x9
	C_SPI_CLK_SRC_DIO_10	Select DIO[10] as source	0xA
	C_SPI_CLK_SRC_DIO_11	Select DIO[11] as source	0xB
	C_SPI_CLK_SRC_DIO_12	Select DIO[12] as source	0xC
	C_SPI_CLK_SRC_DIO_13	Select DIO[13] as source	0xD
	C_SPI_CLK_SRC_DIO_14	Select DIO[14] as source	0xE
	C_SPI_CLK_SRC_DIO_15	Select DIO[15] as source	0xF
	C_SPI_CLK_SRC_DIO_16	Select DIO[16] as source	0x10
	C_SPI_CLK_SRC_DIO_17	Select DIO[17] as source	0x11
	C_SPI_CLK_SRC_DIO_18	Select DIO[18] as source	0x12
	C_SPI_CLK_SRC_DIO_19	Select DIO[19] as source	0x13
	C_SPI_CLK_SRC_DIO_20	Select DIO[20] as source	0x14
	C_SPI_CLK_SRC_DIO_21	Select DIO[21] as source	0x15
	C_SPI_CLK_SRC_DIO_22	Select DIO[22] as source	0x16
	C_SPI_CLK_SRC_DIO_23	Select DIO[23] as source	0x17
	C_SPI_CLK_SRC_DIO_24	Select DIO[24] as source	0x18
	C_SPI_CLK_SRC_DIO_25	Select DIO[25] as source	0x19
	C_SPI_CLK_SRC_DIO_26	Select DIO[26] as source	0x1A
	C_SPI_CLK_SRC_DIO_27	Select DIO[27] as source	0x1B
	C_SPI_CLK_SRC_DIO_28	Select DIO[28] as source	0x1C
	C_SPI_CLK_SRC_DIO_29	Select DIO[29] as source	0x1D
	C_SPI_CLK_SRC_CONST_LOW	Select constant low as source	0x1E
	C_SPI_CLK_SRC_CONST_HIGH	Select constant high as source	0x1F*

16.1.3.9 D_C_I2C_SRC Settings

Bit Field	Field Name	Description
12:8	SDA	C_SDA input selection
4:0	SCL	C_SCL input selection

Field Name	Value Symbol	Value Description	Hex Value
SDA	C_SDA_SRC_DIO_0	Select DIO[0] as source	0x0
	C_SDA_SRC_DIO_1	Select DIO[1] as source	0x1
	C_SDA_SRC_DIO_2	Select DIO[2] as source	0x2
	C_SDA_SRC_DIO_3	Select DIO[3] as source	0x3
	C_SDA_SRC_DIO_4	Select DIO[4] as source	0x4
	C_SDA_SRC_DIO_5	Select DIO[5] as source	0x5
	C_SDA_SRC_DIO_6	Select DIO[6] as source	0x6
	C_SDA_SRC_DIO_7	Select DIO[7] as source	0x7
	C_SDA_SRC_DIO_8	Select DIO[8] as source	0x8
	C_SDA_SRC_DIO_9	Select DIO[9] as source	0x9
	C_SDA_SRC_DIO_10	Select DIO[10] as source	0xA
	C_SDA_SRC_DIO_11	Select DIO[11] as source	0xB
	C_SDA_SRC_DIO_12	Select DIO[12] as source	0xC
	C_SDA_SRC_DIO_13	Select DIO[13] as source	0xD
	C_SDA_SRC_DIO_14	Select DIO[14] as source	0xE
	C_SDA_SRC_DIO_15	Select DIO[15] as source	0xF
	C_SDA_SRC_DIO_16	Select DIO[16] as source	0x10
	C_SDA_SRC_DIO_17	Select DIO[17] as source	0x11
	C_SDA_SRC_DIO_18	Select DIO[18] as source	0x12
	C_SDA_SRC_DIO_19	Select DIO[19] as source	0x13
	C_SDA_SRC_DIO_20	Select DIO[20] as source	0x14
	C_SDA_SRC_DIO_21	Select DIO[21] as source	0x15
	C_SDA_SRC_DIO_22	Select DIO[22] as source	0x16
	C_SDA_SRC_DIO_23	Select DIO[23] as source	0x17
	C_SDA_SRC_DIO_24	Select DIO[24] as source	0x18
	C_SDA_SRC_DIO_25	Select DIO[25] as source	0x19
	C_SDA_SRC_DIO_26	Select DIO[26] as source	0x1A
	C_SDA_SRC_DIO_27	Select DIO[27] as source	0x1B
	C_SDA_SRC_DIO_28	Select DIO[28] as source	0x1C
	C_SDA_SRC_DIO_29	Select DIO[29] as source	0x1D
	C_SDA_SRC_CONST_LOW	Select constant low as source	0x1E
	C_SDA_SRC_CONST_HIGH	Select constant high as source	0x1F*

Field Name	Value Symbol	Value Description	Hex Value
SCL	C_SCL_SRC_DIO_0	Select DIO[0] as source	0x0
	C_SCL_SRC_DIO_1	Select DIO[1] as source	0x1
	C_SCL_SRC_DIO_2	Select DIO[2] as source	0x2
	C_SCL_SRC_DIO_3	Select DIO[3] as source	0x3
	C_SCL_SRC_DIO_4	Select DIO[4] as source	0x4
	C_SCL_SRC_DIO_5	Select DIO[5] as source	0x5
	C_SCL_SRC_DIO_6	Select DIO[6] as source	0x6
	C_SCL_SRC_DIO_7	Select DIO[7] as source	0x7
	C_SCL_SRC_DIO_8	Select DIO[8] as source	0x8
	C_SCL_SRC_DIO_9	Select DIO[9] as source	0x9
	C_SCL_SRC_DIO_10	Select DIO[10] as source	0xA
	C_SCL_SRC_DIO_11	Select DIO[11] as source	0xB
	C_SCL_SRC_DIO_12	Select DIO[12] as source	0xC
	C_SCL_SRC_DIO_13	Select DIO[13] as source	0xD
	C_SCL_SRC_DIO_14	Select DIO[14] as source	0xE
	C_SCL_SRC_DIO_15	Select DIO[15] as source	0xF
	C_SCL_SRC_DIO_16	Select DIO[16] as source	0x10
	C_SCL_SRC_DIO_17	Select DIO[17] as source	0x11
	C_SCL_SRC_DIO_18	Select DIO[18] as source	0x12
	C_SCL_SRC_DIO_19	Select DIO[19] as source	0x13
	C_SCL_SRC_DIO_20	Select DIO[20] as source	0x14
	C_SCL_SRC_DIO_21	Select DIO[21] as source	0x15
	C_SCL_SRC_DIO_22	Select DIO[22] as source	0x16
	C_SCL_SRC_DIO_23	Select DIO[23] as source	0x17
	C_SCL_SRC_DIO_24	Select DIO[24] as source	0x18
	C_SCL_SRC_DIO_25	Select DIO[25] as source	0x19
	C_SCL_SRC_DIO_26	Select DIO[26] as source	0x1A
	C_SCL_SRC_DIO_27	Select DIO[27] as source	0x1B
	C_SCL_SRC_DIO_28	Select DIO[28] as source	0x1C
	C_SCL_SRC_DIO_29	Select DIO[29] as source	0x1D
	C_SCL_SRC_CONST_LOW	Select constant low as source	0x1E
	C_SCL_SRC_CONST_HIGH	Select constant high as source	0x1F*

16.1.3.10 D_DMIC_SRC Settings

Bit Field	Field Name	Description
28:24	DMIC3	DMIC3 input selection
20:16	DMIC2	DMIC2 input selection

Bit Field	Field Name	Description
12:8	DMIC1	DMIC1 input selection
4:0	DMIC0	DMIC0 input selection

Field Name	Value Symbol	Value Description	Hex Value
DMIC3	DMIC3_SRC_DIO_0	Select DIO[0] as source	0x0
	DMIC3_SRC_DIO_1	Select DIO[1] as source	0x1
	DMIC3_SRC_DIO_2	Select DIO[2] as source	0x2
	DMIC3_SRC_DIO_3	Select DIO[3] as source	0x3
	DMIC3_SRC_DIO_4	Select DIO[4] as source	0x4
	DMIC3_SRC_DIO_5	Select DIO[5] as source	0x5
	DMIC3_SRC_DIO_6	Select DIO[6] as source	0x6
	DMIC3_SRC_DIO_7	Select DIO[7] as source	0x7
	DMIC3_SRC_DIO_8	Select DIO[8] as source	0x8
	DMIC3_SRC_DIO_9	Select DIO[9] as source	0x9
	DMIC3_SRC_DIO_10	Select DIO[10] as source	0xA
	DMIC3_SRC_DIO_11	Select DIO[11] as source	0xB
	DMIC3_SRC_DIO_12	Select DIO[12] as source	0xC
	DMIC3_SRC_DIO_13	Select DIO[13] as source	0xD
	DMIC3_SRC_DIO_14	Select DIO[14] as source	0xE
	DMIC3_SRC_DIO_15	Select DIO[15] as source	0xF
	DMIC3_SRC_DIO_16	Select DIO[16] as source	0x10
	DMIC3_SRC_DIO_17	Select DIO[17] as source	0x11
	DMIC3_SRC_DIO_18	Select DIO[18] as source	0x12
	DMIC3_SRC_DIO_19	Select DIO[19] as source	0x13
	DMIC3_SRC_DIO_20	Select DIO[20] as source	0x14
	DMIC3_SRC_DIO_21	Select DIO[21] as source	0x15
	DMIC3_SRC_DIO_22	Select DIO[22] as source	0x16
	DMIC3_SRC_DIO_23	Select DIO[23] as source	0x17
	DMIC3_SRC_DIO_24	Select DIO[24] as source	0x18
	DMIC3_SRC_DIO_25	Select DIO[25] as source	0x19
	DMIC3_SRC_DIO_26	Select DIO[26] as source	0x1A
	DMIC3_SRC_DIO_27	Select DIO[27] as source	0x1B
	DMIC3_SRC_DIO_28	Select DIO[28] as source	0x1C
	DMIC3_SRC_DIO_29	Select DIO[29] as source	0x1D
	DMIC3_SRC_CONST_LOW	Select constant low as source	0x1E
	DMIC3_SRC_CONST_HIGH	Select constant high as source	0x1F*

Field Name	Value Symbol	Value Description	Hex Value
DMIC2	DMIC2_SRC_DIO_0	Select DIO[0] as source	0x0
	DMIC2_SRC_DIO_1	Select DIO[1] as source	0x1
	DMIC2_SRC_DIO_2	Select DIO[2] as source	0x2
	DMIC2_SRC_DIO_3	Select DIO[3] as source	0x3
	DMIC2_SRC_DIO_4	Select DIO[4] as source	0x4
	DMIC2_SRC_DIO_5	Select DIO[5] as source	0x5
	DMIC2_SRC_DIO_6	Select DIO[6] as source	0x6
	DMIC2_SRC_DIO_7	Select DIO[7] as source	0x7
	DMIC2_SRC_DIO_8	Select DIO[8] as source	0x8
	DMIC2_SRC_DIO_9	Select DIO[9] as source	0x9
	DMIC2_SRC_DIO_10	Select DIO[10] as source	0xA
	DMIC2_SRC_DIO_11	Select DIO[11] as source	0xB
	DMIC2_SRC_DIO_12	Select DIO[12] as source	0xC
	DMIC2_SRC_DIO_13	Select DIO[13] as source	0xD
	DMIC2_SRC_DIO_14	Select DIO[14] as source	0xE
	DMIC2_SRC_DIO_15	Select DIO[15] as source	0xF
	DMIC2_SRC_DIO_16	Select DIO[16] as source	0x10
	DMIC2_SRC_DIO_17	Select DIO[17] as source	0x11
	DMIC2_SRC_DIO_18	Select DIO[18] as source	0x12
	DMIC2_SRC_DIO_19	Select DIO[19] as source	0x13
	DMIC2_SRC_DIO_20	Select DIO[20] as source	0x14
	DMIC2_SRC_DIO_21	Select DIO[21] as source	0x15
	DMIC2_SRC_DIO_22	Select DIO[22] as source	0x16
	DMIC2_SRC_DIO_23	Select DIO[23] as source	0x17
	DMIC2_SRC_DIO_24	Select DIO[24] as source	0x18
	DMIC2_SRC_DIO_25	Select DIO[25] as source	0x19
	DMIC2_SRC_DIO_26	Select DIO[26] as source	0x1A
	DMIC2_SRC_DIO_27	Select DIO[27] as source	0x1B
	DMIC2_SRC_DIO_28	Select DIO[28] as source	0x1C
	DMIC2_SRC_DIO_29	Select DIO[29] as source	0x1D
	DMIC2_SRC_CONST_LOW	Select constant low as source	0x1E
	DMIC2_SRC_CONST_HIGH	Select constant high as source	0x1F*

Field Name	Value Symbol	Value Description	Hex Value
DMIC1	DMIC1_SRC_DIO_0	Select DIO[0] as source	0x0
	DMIC1_SRC_DIO_1	Select DIO[1] as source	0x1
	DMIC1_SRC_DIO_2	Select DIO[2] as source	0x2
	DMIC1_SRC_DIO_3	Select DIO[3] as source	0x3
	DMIC1_SRC_DIO_4	Select DIO[4] as source	0x4
	DMIC1_SRC_DIO_5	Select DIO[5] as source	0x5
	DMIC1_SRC_DIO_6	Select DIO[6] as source	0x6
	DMIC1_SRC_DIO_7	Select DIO[7] as source	0x7
	DMIC1_SRC_DIO_8	Select DIO[8] as source	0x8
	DMIC1_SRC_DIO_9	Select DIO[9] as source	0x9
	DMIC1_SRC_DIO_10	Select DIO[10] as source	0xA
	DMIC1_SRC_DIO_11	Select DIO[11] as source	0xB
	DMIC1_SRC_DIO_12	Select DIO[12] as source	0xC
	DMIC1_SRC_DIO_13	Select DIO[13] as source	0xD
	DMIC1_SRC_DIO_14	Select DIO[14] as source	0xE
	DMIC1_SRC_DIO_15	Select DIO[15] as source	0xF
	DMIC1_SRC_DIO_16	Select DIO[16] as source	0x10
	DMIC1_SRC_DIO_17	Select DIO[17] as source	0x11
	DMIC1_SRC_DIO_18	Select DIO[18] as source	0x12
	DMIC1_SRC_DIO_19	Select DIO[19] as source	0x13
	DMIC1_SRC_DIO_20	Select DIO[20] as source	0x14
	DMIC1_SRC_DIO_21	Select DIO[21] as source	0x15
	DMIC1_SRC_DIO_22	Select DIO[22] as source	0x16
	DMIC1_SRC_DIO_23	Select DIO[23] as source	0x17
	DMIC1_SRC_DIO_24	Select DIO[24] as source	0x18
	DMIC1_SRC_DIO_25	Select DIO[25] as source	0x19
	DMIC1_SRC_DIO_26	Select DIO[26] as source	0x1A
	DMIC1_SRC_DIO_27	Select DIO[27] as source	0x1B
	DMIC1_SRC_DIO_28	Select DIO[28] as source	0x1C
	DMIC1_SRC_DIO_29	Select DIO[29] as source	0x1D
	DMIC1_SRC_CONST_LOW	Select constant low as source	0x1E
	DMIC1_SRC_CONST_HIGH	Select constant high as source	0x1F*

Field Name	Value Symbol	Value Description	Hex Value
DMIC0	DMIC0_SRC_DIO_0	Select DIO[0] as source	0x0
	DMIC0_SRC_DIO_1	Select DIO[1] as source	0x1
	DMIC0_SRC_DIO_2	Select DIO[2] as source	0x2
	DMIC0_SRC_DIO_3	Select DIO[3] as source	0x3
	DMIC0_SRC_DIO_4	Select DIO[4] as source	0x4
	DMIC0_SRC_DIO_5	Select DIO[5] as source	0x5
	DMIC0_SRC_DIO_6	Select DIO[6] as source	0x6
	DMIC0_SRC_DIO_7	Select DIO[7] as source	0x7
	DMIC0_SRC_DIO_8	Select DIO[8] as source	0x8
	DMIC0_SRC_DIO_9	Select DIO[9] as source	0x9
	DMIC0_SRC_DIO_10	Select DIO[10] as source	0xA
	DMIC0_SRC_DIO_11	Select DIO[11] as source	0xB
	DMIC0_SRC_DIO_12	Select DIO[12] as source	0xC
	DMIC0_SRC_DIO_13	Select DIO[13] as source	0xD
	DMIC0_SRC_DIO_14	Select DIO[14] as source	0xE
	DMIC0_SRC_DIO_15	Select DIO[15] as source	0xF
	DMIC0_SRC_DIO_16	Select DIO[16] as source	0x10
	DMIC0_SRC_DIO_17	Select DIO[17] as source	0x11
	DMIC0_SRC_DIO_18	Select DIO[18] as source	0x12
	DMIC0_SRC_DIO_19	Select DIO[19] as source	0x13
	DMIC0_SRC_DIO_20	Select DIO[20] as source	0x14
	DMIC0_SRC_DIO_21	Select DIO[21] as source	0x15
	DMIC0_SRC_DIO_22	Select DIO[22] as source	0x16
	DMIC0_SRC_DIO_23	Select DIO[23] as source	0x17
	DMIC0_SRC_DIO_24	Select DIO[24] as source	0x18
	DMIC0_SRC_DIO_25	Select DIO[25] as source	0x19
	DMIC0_SRC_DIO_26	Select DIO[26] as source	0x1A
	DMIC0_SRC_DIO_27	Select DIO[27] as source	0x1B
	DMIC0_SRC_DIO_28	Select DIO[28] as source	0x1C
	DMIC0_SRC_DIO_29	Select DIO[29] as source	0x1D
	DMIC0_SRC_CONST_LOW	Select constant low as source	0x1E
	DMIC0_SRC_CONST_HIGH	Select constant high as source	0x1F*

16.1.3.11 D_PLL_SRC Settings

Bit Field	Field Name	Description
4:0	REF_CLK	PLL reference clock input selection

Field Name	Value Symbol	Value Description	Hex Value
REF_CLK	PLL_REF_CLK_SRC_DIO_0	Select DIO[0] as source	0x0
	PLL_REF_CLK_SRC_DIO_1	Select DIO[1] as source	0x1
	PLL_REF_CLK_SRC_DIO_2	Select DIO[2] as source	0x2
	PLL_REF_CLK_SRC_DIO_3	Select DIO[3] as source	0x3
	PLL_REF_CLK_SRC_DIO_4	Select DIO[4] as source	0x4
	PLL_REF_CLK_SRC_DIO_5	Select DIO[5] as source	0x5
	PLL_REF_CLK_SRC_DIO_6	Select DIO[6] as source	0x6
	PLL_REF_CLK_SRC_DIO_7	Select DIO[7] as source	0x7
	PLL_REF_CLK_SRC_DIO_8	Select DIO[8] as source	0x8
	PLL_REF_CLK_SRC_DIO_9	Select DIO[9] as source	0x9
	PLL_REF_CLK_SRC_DIO_10	Select DIO[10] as source	0xA
	PLL_REF_CLK_SRC_DIO_11	Select DIO[11] as source	0xB
	PLL_REF_CLK_SRC_DIO_12	Select DIO[12] as source	0xC
	PLL_REF_CLK_SRC_DIO_13	Select DIO[13] as source	0xD
	PLL_REF_CLK_SRC_DIO_14	Select DIO[14] as source	0xE
	PLL_REF_CLK_SRC_DIO_15	Select DIO[15] as source	0xF
	PLL_REF_CLK_SRC_DIO_16	Select DIO[16] as source	0x10
	PLL_REF_CLK_SRC_DIO_17	Select DIO[17] as source	0x11
	PLL_REF_CLK_SRC_DIO_18	Select DIO[18] as source	0x12
	PLL_REF_CLK_SRC_DIO_19	Select DIO[19] as source	0x13
	PLL_REF_CLK_SRC_DIO_20	Select DIO[20] as source	0x14
	PLL_REF_CLK_SRC_DIO_21	Select DIO[21] as source	0x15
	PLL_REF_CLK_SRC_DIO_22	Select DIO[22] as source	0x16
	PLL_REF_CLK_SRC_DIO_23	Select DIO[23] as source	0x17
	PLL_REF_CLK_SRC_DIO_24	Select DIO[24] as source	0x18
	PLL_REF_CLK_SRC_DIO_25	Select DIO[25] as source	0x19
	PLL_REF_CLK_SRC_DIO_26	Select DIO[26] as source	0x1A
	PLL_REF_CLK_SRC_DIO_27	Select DIO[27] as source	0x1B
	PLL_REF_CLK_SRC_DIO_28	Select DIO[28] as source	0x1C
	PLL_REF_CLK_SRC_DIO_29	Select DIO[29] as source	0x1D
	PLL_REF_CLK_SRC_CONST_LOW	Select constant low as source	0x1E*
	PLL_REF_CLK_SRC_EXT_CLK	EXT_CLK as source	0x1F

16.2 General-Purpose I/O (GPIO) Interface

Ezairo 7100 can configure any of the DIO pads as CFX software-controlled general-purpose digital I/O (GPIO) pads. The function of these GPIO pads is defined by a user application, which can use them for general input or output.

The D_DIO_MODE register indicates which DIO pads have been configured for GPIO functionality; bits in this register are set (for example, DIO0_IS_GPIO) if they are configured as GPIOs and cleared otherwise (for example,

DIO0_IS_NOT_GPIO). For any pads that are defined as a GPIO, the D_DIO_DIR register can be written to set the input/output direction for these pads.

The value observed at the digital input pads can be read from the D_DIO_DATA register. This value is read as 0 for all pads configured as LSAD inputs because the digital input is not enabled in this mode. For all other modes, the physical value of the pad is directly measured. For each VDDO* domain, the D_DIO_DATA_VDDO_* register provides the DIO data for the DIO pads assigned to that VDDO* domain.

NOTE: If a DIO is configured for a GPIO mode, bit 0 of the D_DIO_CFG_* register for a DIO can be used to detect or set the data value for the GPIO (hence the DIO_MODE_GPIO_IN_0, DIO_MODE_GPIO_IN_1 and DIO_MODE_GPIO_OUT_0, DIO_MODE_GPIO_OUT_1 bit setting pairs).

16.2.1 GPIO Interrupt

The GPIO interface provides an associated configurable interrupt which, when enabled, signals the occurrence of an event or a condition on a specified GPIO pad. See Section 14.1, “Interrupt Controller” on page 218 for information regarding interrupt configuration and handling.

The GPIO interrupt supports triggering an interrupt from one of four input sources, each with their own GPIO event. Each of the sources can be configured using one of the D_CFX_DIO_INT_CFG_* registers. For each of the four sources:

- To select the GPIO pad to use as a trigger for the interrupt, set the INT_CFG_SRC bit field.
- To select the event to use as a trigger for the interrupt, use the INT_CFG bit field. A list of the possible triggering events (including a description of each event) is listed in Table 70.

Table 70. GPIO Interrupt Events

Event	Event Description
None	An interrupt is never generated.
High level	An interrupt is generated whenever a logical 1 is detected on the GPIO pin. Interrupts continue to be generated while this condition remains true.
Low level	An interrupt is generated whenever a logical 0 is detected on the GPIO pin. Interrupts continue to be generated while this condition remains true.
Rising Edge	An interrupt is generated when a transition from a logical 0 to a logical 1 is detected.
Falling Edge	An interrupt is generated when a transition from a logical 1 to a logical 0 is detected.
Transition	An interrupt is generated when any transition between a logical 0 and a logical 1 is detected.

IMPORTANT: When reconfiguring a GPIO interrupt source while one or more GPIO interrupt sources are active, first configure the input selection, while keeping the event type as disabled (using DIO_EVENT_NONE for the CFX core’s GPIO interrupts, or C_DIO_EVENT_NONE for the ARM Cortex-M3 core’s GPIO interrupts), to avoid an accidental interrupt trigger due to the transition between input selections. After two SYSCLK cycles, the event type can be set without any risk of an unexpected interrupt.

When a CFX GPIO interrupt has triggered, the D_CFX_DIO_INT_STATUS register indicates which source or set of sources triggered the GPIO interrupt.

The ARM Cortex-M3 core GPIO interfaces also supports a GPIO interrupt configurable in the same way as the CFX GPIO interrupt, using the D_CM3_DIO_INT_CFG_* registers or their mirror in the C_DIO_INT_CFG_* registers. This interrupt can be configured for all of the same modes and the same or different sources as the CFX GPIO interrupt. For more information, see Section 17.2.1, “GPIO Interrupts” on page 406.

16.2.2 Control and Configuration Registers

Register Name	Register Description	Address
D_DIO_DATA	DIOs data access pseudo register	P:0xF2A0
D_DIO_DIR	GPIOs data direction pseudo register	P:0xF2A1
D_DIO_MODE	DIOs mode pseudo register	P:0xF2A2
D_DIO_DATA_VDDO_0	DIOs on VDDO[2:0] data access pseudo register	P:0xF2A4
D_DIO_DATA_VDDO_1	DIOs on VDDO[2:0] data access pseudo register	P:0xF2A5
D_DIO_DATA_VDDO_2	DIOs on VDDO[2:0] data access pseudo register	P:0xF2A6
D_CFX_DIO_INT_CFG_0	CFX DIO interrupt configuration	P:0xF2A8
D_CFX_DIO_INT_CFG_1	CFX DIO interrupt configuration	P:0xF2A9
D_CFX_DIO_INT_CFG_2	CFX DIO interrupt configuration	P:0xF2AA
D_CFX_DIO_INT_CFG_3	CFX DIO interrupt configuration	P:0xF2AB
D_CFX_DIO_INT_STATUS	CFX DIO interrupt status	P:0xF2B0

16.2.2.1 D_DIO_DATA Settings

Bit Field	Field Name	Description
29:0	DIO	DIO [29:0] read data (pads which are in LSAD mode return 0)
29:0	GPIO	GPIO [29:0] write data (updates output data of DIO [29:0] only for pads with IO_MODE 0b000XX)

Field Name	Value Symbol	Value Description	Hex Value
DIO	DIO0_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO1_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO2_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO3_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO4_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO5_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO6_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO7_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO8_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO9_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO10_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO11_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO12_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO13_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO14_LOW	DIO pad is low or pad is in LSAD mode	0x0*

Field Name	Value Symbol	Value Description	Hex Value
DIO (continued)	DIO15_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO16_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO17_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO18_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO19_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO20_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO21_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO22_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO23_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO24_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO25_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO26_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO27_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO28_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO29_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	DIO0_HIGH	DIO pad is high	0x1
	DIO1_HIGH	DIO pad is high	0x2
	DIO2_HIGH	DIO pad is high	0x4
	DIO3_HIGH	DIO pad is high	0x8
	DIO4_HIGH	DIO pad is high	0x10
	DIO5_HIGH	DIO pad is high	0x20
	DIO6_HIGH	DIO pad is high	0x40
	DIO7_HIGH	DIO pad is high	0x80
	DIO8_HIGH	DIO pad is high	0x100
	DIO9_HIGH	DIO pad is high	0x200
	DIO10_HIGH	DIO pad is high	0x400
	DIO11_HIGH	DIO pad is high	0x800
	DIO12_HIGH	DIO pad is high	0x1000
	DIO13_HIGH	DIO pad is high	0x2000
	DIO14_HIGH	DIO pad is high	0x4000
	DIO15_HIGH	DIO pad is high	0x8000
	DIO16_HIGH	DIO pad is high	0x10000
	DIO17_HIGH	DIO pad is high	0x20000
	DIO18_HIGH	DIO pad is high	0x40000
	DIO19_HIGH	DIO pad is high	0x80000
	DIO20_HIGH	DIO pad is high	0x100000
	DIO21_HIGH	DIO pad is high	0x200000
	DIO22_HIGH	DIO pad is high	0x400000

Field Name	Value Symbol	Value Description	Hex Value
DIO (continued)	DIO23_HIGH	DIO pad is high	0x800000
	DIO24_HIGH	DIO pad is high	0x1000000
	DIO25_HIGH	DIO pad is high	0x2000000
	DIO26_HIGH	DIO pad is high	0x4000000
	DIO27_HIGH	DIO pad is high	0x8000000
	DIO28_HIGH	DIO pad is high	0x10000000
	DIO29_HIGH	DIO pad is high	0x20000000
GPIO	GPIO0_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO1_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO2_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO3_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO4_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO5_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO6_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO7_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO8_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO9_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO10_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO11_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO12_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO13_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO14_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO15_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO16_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO17_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO18_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO19_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO20_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO21_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO22_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO23_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO24_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO25_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO26_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO27_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO28_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO29_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	GPIO0_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x1

Field Name	Value Symbol	Value Description	Hex Value
GPIO (continued)	GPIO1_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x2
	GPIO2_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x4
	GPIO3_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x8
	GPIO4_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x10
	GPIO5_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x20
	GPIO6_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x40
	GPIO7_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x80
	GPIO8_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x100
	GPIO9_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x200
	GPIO10_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x400
	GPIO11_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x800
	GPIO12_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x1000
	GPIO13_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x2000
	GPIO14_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x4000
	GPIO15_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x8000
	GPIO16_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x10000
	GPIO17_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x20000
	GPIO18_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x40000
	GPIO19_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x80000
	GPIO20_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x100000
	GPIO21_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x200000
	GPIO22_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x400000
	GPIO23_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x800000
	GPIO24_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x1000000
	GPIO25_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x2000000
	GPIO26_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x4000000
	GPIO27_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x8000000
	GPIO28_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x10000000
	GPIO29_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x20000000

16.2.2.2 D_DIO_DIR Settings

Bit Field	Field Name	Description
29:0	DIO	Get DIO direction
29:0	GPIO	Set DIO GPIO direction (only if IO_MODE is 0b000XX)

Field Name	Value Symbol	Value Description	Hex Value
DIO	DIO0_INPUT	DIO is an input	0x0*
	DIO1_INPUT	DIO is an input	0x0*
	DIO2_INPUT	DIO is an input	0x0*
	DIO3_INPUT	DIO is an input	0x0*
	DIO4_INPUT	DIO is an input	0x0*
	DIO5_INPUT	DIO is an input	0x0*
	DIO6_INPUT	DIO is an input	0x0*
	DIO7_INPUT	DIO is an input	0x0*
	DIO8_INPUT	DIO is an input	0x0*
	DIO9_INPUT	DIO is an input	0x0*
	DIO10_INPUT	DIO is an input	0x0*
	DIO11_INPUT	DIO is an input	0x0*
	DIO12_INPUT	DIO is an input	0x0*
	DIO13_INPUT	DIO is an input	0x0*
	DIO14_INPUT	DIO is an input	0x0*
	DIO15_INPUT	DIO is an input	0x0*
	DIO16_INPUT	DIO is an input	0x0*
	DIO17_INPUT	DIO is an input	0x0*
	DIO18_INPUT	DIO is an input	0x0*
	DIO19_INPUT	DIO is an input	0x0*
	DIO20_INPUT	DIO is an input	0x0*
	DIO21_INPUT	DIO is an input	0x0*
	DIO22_INPUT	DIO is an input	0x0*
	DIO23_INPUT	DIO is an input	0x0*
	DIO24_INPUT	DIO is an input	0x0*
	DIO25_INPUT	DIO is an input	0x0*
	DIO26_INPUT	DIO is an input	0x0*
	DIO27_INPUT	DIO is an input	0x0*
	DIO28_INPUT	DIO is an input	0x0*
	DIO29_INPUT	DIO is an input	0x0*
	DIO0_OUTPUT	DIO is an output	0x1
	DIO1_OUTPUT	DIO is an output	0x2
	DIO2_OUTPUT	DIO is an output	0x4
	DIO3_OUTPUT	DIO is an output	0x8
	DIO4_OUTPUT	DIO is an output	0x10
	DIO5_OUTPUT	DIO is an output	0x20
	DIO6_OUTPUT	DIO is an output	0x40

Field Name	Value Symbol	Value Description	Hex Value
DIO (continued)	DIO7_OUTPUT	DIO is an output	0x80
	DIO8_OUTPUT	DIO is an output	0x100
	DIO9_OUTPUT	DIO is an output	0x200
	DIO10_OUTPUT	DIO is an output	0x400
	DIO11_OUTPUT	DIO is an output	0x800
	DIO12_OUTPUT	DIO is an output	0x1000
	DIO13_OUTPUT	DIO is an output	0x2000
	DIO14_OUTPUT	DIO is an output	0x4000
	DIO15_OUTPUT	DIO is an output	0x8000
	DIO16_OUTPUT	DIO is an output	0x10000
	DIO17_OUTPUT	DIO is an output	0x20000
	DIO18_OUTPUT	DIO is an output	0x40000
	DIO19_OUTPUT	DIO is an output	0x80000
	DIO20_OUTPUT	DIO is an output	0x100000
	DIO21_OUTPUT	DIO is an output	0x200000
	DIO22_OUTPUT	DIO is an output	0x400000
	DIO23_OUTPUT	DIO is an output	0x800000
	DIO24_OUTPUT	DIO is an output	0x1000000
	DIO25_OUTPUT	DIO is an output	0x2000000
	DIO26_OUTPUT	DIO is an output	0x4000000
	DIO27_OUTPUT	DIO is an output	0x8000000
	DIO28_OUTPUT	DIO is an output	0x10000000
	DIO29_OUTPUT	DIO is an output	0x20000000
GPIO	GPIO0_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO1_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO2_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO3_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO4_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO5_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO6_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO7_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO8_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO9_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO10_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO11_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO12_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO13_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO14_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0

Field Name	Value Symbol	Value Description	Hex Value
GPIO (continued)	GPIO15_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO16_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO17_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO18_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO19_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO20_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO21_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO22_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO23_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO24_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO25_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO26_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO27_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO28_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO29_INPUT	Set DIO to input if IO_MODE is 0b000XX	0x0
	GPIO0_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x1
	GPIO1_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x2
	GPIO2_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x4
	GPIO3_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x8
	GPIO4_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x10
	GPIO5_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x20
	GPIO6_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x40
	GPIO7_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x80
	GPIO8_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x100
	GPIO9_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x200
	GPIO10_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x400
	GPIO11_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x800
	GPIO12_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x1000
	GPIO13_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x2000
	GPIO14_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x4000
	GPIO15_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x8000
	GPIO16_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x10000
	GPIO17_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x20000
	GPIO18_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x40000
	GPIO19_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x80000
	GPIO20_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x100000
	GPIO21_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x200000

Field Name	Value Symbol	Value Description	Hex Value
GPIO (continued)	GPIO22_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x400000
	GPIO23_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x800000
	GPIO24_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x1000000
	GPIO25_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x2000000
	GPIO26_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x4000000
	GPIO27_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x8000000
	GPIO28_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x10000000
	GPIO29_OUTPUT	Set DIO to output if IO_MODE is 0b000XX	0x20000000

16.2.2.3 D_DIO_MODE Settings

Bit Field	Field Name	Description
29:0	GPIO	DIO mode

Field Name	Value Symbol	Value Description	Hex Value
GPIO	DIO0_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO1_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO2_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO3_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO4_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO5_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO6_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO7_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO8_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO9_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO10_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO11_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO12_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO13_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO14_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO15_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO16_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO17_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO18_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO19_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO20_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO21_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*

Field Name	Value Symbol	Value Description	Hex Value
GPIO (continued)	DIO22_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO23_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO24_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO25_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO26_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO27_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO28_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO29_IS_NOT_GPIO	This DIO is not configured as a CFX controlled GPIO	0x0*
	DIO0_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x1
	DIO1_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x2
	DIO2_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x4
	DIO3_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x8
	DIO4_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x10
	DIO5_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x20
	DIO6_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x40
	DIO7_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x80
	DIO8_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x100
	DIO9_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x200
	DIO10_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x400
	DIO11_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x800
	DIO12_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x1000
	DIO13_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x2000
	DIO14_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x4000
	DIO15_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x8000
	DIO16_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x10000
	DIO17_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x20000
	DIO18_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x40000
	DIO19_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x80000
	DIO20_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x100000
	DIO21_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x200000
	DIO22_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x400000
	DIO23_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x800000
	DIO24_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x1000000
	DIO25_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x2000000
	DIO26_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x4000000
	DIO27_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x8000000
	DIO28_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x10000000
	DIO29_IS_GPIO	This DIO is configured as a CFX controlled GPIO	0x20000000

16.2.2.4 D_DIO_DATA_VDDO_* Settings

The following bit fields and field names apply equally to registers D_DIO_DATA_VDDO_0 to D_DIO_DATA_VDDO_2.

Bit Field	Field Name	Description
9:0	DIO	DIO read data (pads which are in LSAD mode, return 0)
9:0	GPIO	GPIO write data (updates output data of DIO only for pads with IO_MODE 0b000XX)

Field Name	Value Symbol	Value Description	Hex Value
DIO	VDDO_DIO0_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	VDDO_DIO1_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	VDDO_DIO2_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	VDDO_DIO3_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	VDDO_DIO4_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	VDDO_DIO5_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	VDDO_DIO6_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	VDDO_DIO7_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	VDDO_DIO8_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	VDDO_DIO9_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	VDDO_DIO0_HIGH	DIO pad is high	0x1
	VDDO_DIO1_HIGH	DIO pad is high	0x2
	VDDO_DIO2_HIGH	DIO pad is high	0x4
	VDDO_DIO3_HIGH	DIO pad is high	0x8
	VDDO_DIO4_HIGH	DIO pad is high	0x10
	VDDO_DIO5_HIGH	DIO pad is high	0x20
	VDDO_DIO6_HIGH	DIO pad is high	0x40
	VDDO_DIO7_HIGH	DIO pad is high	0x80
	VDDO_DIO8_HIGH	DIO pad is high	0x100
	VDDO_DIO9_HIGH	DIO pad is high	0x200

Field Name	Value Symbol	Value Description	Hex Value
GPIO	VDDO_GPIO0_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	VDDO_GPIO1_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	VDDO_GPIO2_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	VDDO_GPIO3_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	VDDO_GPIO4_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	VDDO_GPIO5_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	VDDO_GPIO6_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	VDDO_GPIO7_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	VDDO_GPIO8_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	VDDO_GPIO9_LOW	Set the DIO pad to low if IO_MODE is 0b000XX	0x0
	VDDO_GPIO0_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x1
	VDDO_GPIO1_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x2
	VDDO_GPIO2_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x4
	VDDO_GPIO3_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x8
	VDDO_GPIO4_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x10
	VDDO_GPIO5_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x20
	VDDO_GPIO6_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x40
	VDDO_GPIO7_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x80
	VDDO_GPIO8_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x100
	VDDO_GPIO9_HIGH	Set the DIO pad to high if IO_MODE is 0b000XX	0x200

16.2.2.5 D_CFX_DIO_INT_CFG_*

The following bit fields and field names apply equally to registers D_CFX_DIO_INT_CFG_0 to D_CFX_DIO_INT_CFG_3.

Bit Field	Field Name	Description
7:5	EVENT	Interrupt event configuration
4:0	SRC	Interrupt input selection

Field Name	Value Symbol	Value Description	Hex Value
EVENT	CFX_DIO_EVENT_NONE	Interrupt not triggered	0x0*
	CFX_DIO_EVENT_HIGH_LEVEL	Interrupt triggered on high state	0x1
	CFX_DIO_EVENT_LOW_LEVEL	Interrupt triggered on low state	0x2
	CFX_DIO_EVENT_RISING_EDGE	Interrupt triggered on rising edge	0x3
	CFX_DIO_EVENT_FALLING_EDGE	Interrupt triggered on falling edge	0x4
	CFX_DIO_EVENT_TRANSITION	Interrupt triggered on any edge	0x5

Field Name	Value Symbol	Value Description	Hex Value
SRC	CFX_DIO_SRC_DIO_0	Select DIO[0] as source	0x0*
	CFX_DIO_SRC_DIO_1	Select DIO[1] as source	0x1
	CFX_DIO_SRC_DIO_2	Select DIO[2] as source	0x2
	CFX_DIO_SRC_DIO_3	Select DIO[3] as source	0x3
	CFX_DIO_SRC_DIO_4	Select DIO[4] as source	0x4
	CFX_DIO_SRC_DIO_5	Select DIO[5] as source	0x5
	CFX_DIO_SRC_DIO_6	Select DIO[6] as source	0x6
	CFX_DIO_SRC_DIO_7	Select DIO[7] as source	0x7
	CFX_DIO_SRC_DIO_8	Select DIO[8] as source	0x8
	CFX_DIO_SRC_DIO_9	Select DIO[9] as source	0x9
	CFX_DIO_SRC_DIO_10	Select DIO[10] as source	0xA
	CFX_DIO_SRC_DIO_11	Select DIO[11] as source	0xB
	CFX_DIO_SRC_DIO_12	Select DIO[12] as source	0xC
	CFX_DIO_SRC_DIO_13	Select DIO[13] as source	0xD
	CFX_DIO_SRC_DIO_14	Select DIO[14] as source	0xE
	CFX_DIO_SRC_DIO_15	Select DIO[15] as source	0xF
	CFX_DIO_SRC_DIO_16	Select DIO[16] as source	0x10
	CFX_DIO_SRC_DIO_17	Select DIO[17] as source	0x11
	CFX_DIO_SRC_DIO_18	Select DIO[18] as source	0x12
	CFX_DIO_SRC_DIO_19	Select DIO[19] as source	0x13
	CFX_DIO_SRC_DIO_20	Select DIO[20] as source	0x14
	CFX_DIO_SRC_DIO_21	Select DIO[21] as source	0x15
	CFX_DIO_SRC_DIO_22	Select DIO[22] as source	0x16
	CFX_DIO_SRC_DIO_23	Select DIO[23] as source	0x17
	CFX_DIO_SRC_DIO_24	Select DIO[24] as source	0x18
	CFX_DIO_SRC_DIO_25	Select DIO[25] as source	0x19
	CFX_DIO_SRC_DIO_26	Select DIO[26] as source	0x1A
	CFX_DIO_SRC_DIO_27	Select DIO[27] as source	0x1B
	CFX_DIO_SRC_DIO_28	Select DIO[28] as source	0x1C
	CFX_DIO_SRC_DIO_29	Select DIO[29] as source	0x1D
	CFX_DIO_SRC_SCL	Select SCL as source	0x1E
	CFX_DIO_SRC_SDA	Select SDA as source	0x1F

16.2.2.6 D_CFX_DIO_INT_STATUS Settings

Bit Field	Field Name	Description
3	CFX_INT3_STATUS	CFX interrupt 3 status (cleared when read)
2	CFX_INT2_STATUS	CFX interrupt 2 status (cleared when read)

Bit Field	Field Name	Description
1	CFX_INT1_STATUS	CFX interrupt 1 status (cleared when read)
0	CFX_INT0_STATUS	CFX interrupt 0 status (cleared when read)

Field Name	Value Symbol	Value Description	Hex Value
CFX_INT3_STATUS	CFX_DIO_INT3_FALSE	Interrupt 3 not triggered	0x0*
	CFX_DIO_INT3_TRUE	Interrupt 3 triggered	0x1
CFX_INT2_STATUS	CFX_DIO_INT2_FALSE	Interrupt 2 not triggered	0x0*
	CFX_DIO_INT2_TRUE	Interrupt 2 triggered	0x1
CFX_INT1_STATUS	CFX_DIO_INT1_FALSE	Interrupt 1 not triggered	0x0*
	CFX_DIO_INT1_TRUE	Interrupt 1 triggered	0x1
CFX_INT0_STATUS	CFX_DIO_INT0_FALSE	Interrupt 0 not triggered	0x0*
	CFX_DIO_INT0_TRUE	Interrupt 0 triggered	0x1

16.3 Low-Speed Analog-to-Digital (LSAD) Converter Interface

The Low-Speed Analog-to-Digital (LSAD) converters provide a low-frequency 10-bit A/D conversion of six external signals and two internal signals. The external signals are multiplexed onto the six DIO pads at the base of each VDDO-supplied (VDDO1 to VDDO3) DIO block, and are numbered as channels 0 to 5. For more information about DIO configuration for LSADs, see Section 16.1, “Digital I/O (DIO)”.

The purpose of the LSAD converter is to sample input voltages that typically change slowly—for example, the voltage associated with a potentiometer-based volume control.

The converted output from the most recent conversion of each external signal can be found in the A_LSAD_CHx_DATA registers. The internal signals of the LSAD are used to monitor the battery voltage (VBAT) and LSAD reference voltage (GND relative to VREG). The converted output from the most recent conversion of these internal signals can be found in the VBAT (A_LSAD_VBAT_DATA) and GND (A_LSAD_GND_DATA) registers respectively.

The input source for each of the general-purpose LSADs can be selected using the LSAD_SRC_CFG_CH* bit fields from the A_LSAD_SRC_CFG register. Each of these bit fields can either specify the DIO that the specified LSAD will use, or specify a system voltage that the application will monitor using the specified LSAD. The system voltages that can be monitored by the LSAD interface are:

- VDDC (LSAD 5)
- VDDM (LSAD 4)
- VDBL (LSAD 3)
- VDDA (LSAD 2)
- VREG (LSAD 1)
- DAC0 (LSAD 0)

For more information about the power supplies measured here, see Section 19.1, “Power Supply” on page 466.

The LSAD signals are sampled sequentially at a sampling rate defined by SLOWCLK and prescaled using the LSAD_CTRL_FREQ bit field in the A_LSAD_CTRL register. The overall sampling frequency is defined by:

$$f_{LSAD} = \frac{f_{SLOWCLK}}{20 \times (LSAD_CTRL_FREQ + 1)}$$

NOTE: If the slow clock is operating at 1.28 MHz, the default setting of the LSAD_CTRL_FREQ bit field results in an overall sampling rate of 12.8 kHz and a sampling rate of 1.6 kHz for each individual LSAD input.

Each LSAD can be configured to run in one of two modes by setting or clearing the appropriate LSAD_CTRL_xxx_MODE bits in the A_LSAD_CTRL register. In both modes, the LSAD measures an input with respect to either VREF or the bandgap (VREG) directly, converting the LSAD measurement to a nominal input range between 0 V and 2 V (assuming a 1.0 V reference; for a 0.97 V reference the range is 0 V to 1.94 V) using a converter that measures signals in the range of -1 V to 3 V (assuming a 1.0 V reference; for a 0.97 V reference the range is -0.97 V to 2.91 V)

In trimmer mode The output data value from the LSAD uses a 9-bit digitally saturated unsigned result scaled between 0x000 and 0x1FF to represent a signal in the 0 V to 2 V range.

In audio mode The output data value from the LSAD uses a 10-bit unsaturated signed result scaled between 0x300 and 0x0FF to represent a signal in the 0 V to 2 V range. Results in the range 0x200 to 0x1FF are possible in this mode and correspond to voltages in the larger -1 V to 3 V range; however, the DIO pads might skew the measurements outside the nominal range.

16.3.1 Control and Configuration Registers

Register Name	Register Description	Address
A_LSAD_CH0_DATA	LSAD conversion result for CH0	P:0xF180
A_LSAD_CH1_DATA	LSAD conversion result for CH1	P:0xF181
A_LSAD_CH2_DATA	LSAD conversion result for CH2	P:0xF182
A_LSAD_CH3_DATA	LSAD conversion result for CH3	P:0xF183
A_LSAD_CH4_DATA	LSAD conversion result for CH4	P:0xF184
A_LSAD_CH5_DATA	LSAD conversion result for CH5	P:0xF185
A_LSAD_VBAT_DATA	LSAD conversion result for VBAT	P:0xF186
A_LSAD_GND_DATA	LSAD conversion result for GND	P:0xF187
A_LSAD_CTRL	Configuration registers Low Speed Analog to Digital Convertor	P:0xF188
A_LSAD_SRC_CFG	LSAD source multiplexer control	P:0xF189

16.3.1.1 A_LSAD_CTRL Settings

Bit Field	Field Name	Description
11	GND_MODE	Defines if result for LSAD_GND_OUT is signed or unsigned
10	VBAT_MODE	Defines if result for LSAD_VBAT_OUT is signed or unsigned
9	CH5_MODE	Defines if result for LSAD_CH5_OUT is signed or unsigned
8	CH4_MODE	Defines if result for LSAD_CH4_OUT is signed or unsigned
7	CH3_MODE	Defines if result for LSAD_CH3_OUT is signed or unsigned
6	CH2_MODE	Defines if result for LSAD_CH2_OUT is signed or unsigned

Bit Field	Field Name	Description
5	CH1_MODE	Defines if result for LSAD_CH1_OUT is signed or unsigned
4	CH0_MODE	Defines if result for LSAD_CH0_OUT is signed or unsigned
3:0	FREQ	Defines the sampling frequency of the LSAD channels

Field Name	Value Symbol	Value Description	Hex Value
GND_MODE	LSAD_CTRL_GND_MODE_TRIMMER	Unsigned	0x0*
	LSAD_CTRL_GND_MODE_AUDIO_OUT	Signed	0x1
VBAT_MODE	LSAD_CTRL_VBAT_MODE_TRIMMER	Unsigned	0x0*
	LSAD_CTRL_VBAT_MODE_AUDIO_OUT	Signed	0x1
CH5_MODE	LSAD_CTRL_CH5_MODE_TRIMMER	Unsigned	0x0*
	LSAD_CTRL_CH5_MODE_AUDIO_OUT	Signed	0x1
CH4_MODE	LSAD_CTRL_CH4_MODE_TRIMMER	Unsigned	0x0*
	LSAD_CTRL_CH4_MODE_AUDIO_OUT	Signed	0x1
CH3_MODE	LSAD_CTRL_CH3_MODE_TRIMMER	Unsigned	0x0*
	LSAD_CTRL_CH3_MODE_AUDIO_OUT	Signed	0x1
CH2_MODE	LSAD_CTRL_CH2_MODE_TRIMMER	Unsigned	0x0*
	LSAD_CTRL_CH2_MODE_AUDIO_OUT	Signed	0x1
CH1_MODE	LSAD_CTRL_CH1_MODE_TRIMMER	Unsigned	0x0*
	LSAD_CTRL_CH1_MODE_AUDIO_OUT	Signed	0x1
CH0_MODE	LSAD_CTRL_CH0_MODE_TRIMMER	Unsigned	0x0*
	LSAD_CTRL_CH0_MODE_AUDIO_OUT	Signed	0x1
FREQ	LSAD_CTRL_DISABLE	LSAD Disabled	0x0
	LSAD_CTRL_PRESCALE_40	Sample rate is SLOWCLK/40	0x1
	LSAD_CTRL_PRESCALE_60	Sample rate is SLOWCLK/60	0x2
	LSAD_CTRL_PRESCALE_80	Sample rate is SLOWCLK/80	0x3
	LSAD_CTRL_PRESCALE_100	Sample rate is SLOWCLK/100	0x4*
	LSAD_CTRL_PRESCALE_120	Sample rate is SLOWCLK/120	0x5
	LSAD_CTRL_PRESCALE_140	Sample rate is SLOWCLK/140	0x6
	LSAD_CTRL_PRESCALE_160	Sample rate is SLOWCLK/160	0x7
	LSAD_CTRL_PRESCALE_180	Sample rate is SLOWCLK/180	0x8
	LSAD_CTRL_PRESCALE_200	Sample rate is SLOWCLK/200	0x9
	LSAD_CTRL_PRESCALE_220	Sample rate is SLOWCLK/220	0xA
	LSAD_CTRL_PRESCALE_240	Sample rate is SLOWCLK/240	0xB
	LSAD_CTRL_PRESCALE_260	Sample rate is SLOWCLK/260	0xC
	LSAD_CTRL_PRESCALE_280	Sample rate is SLOWCLK/280	0xD
	LSAD_CTRL_PRESCALE_300	Sample rate is SLOWCLK/300	0xE
	LSAD_CTRL_PRESCALE_320	Sample rate is SLOWCLK/320	0xF

16.3.1.2 A_LSAD_SRC_CFG Settings

Bit Field	Field Name	Description
22	CH5_REF	Select channel 5 reference
21:20	CH5	Select channel 5 source
18	CH4_REF	Select channel 4 reference
17:16	CH4	Select channel 4 source
14	CH3_REF	Select channel 3 reference
13:12	CH3	Select channel 3 source
10	CH2_REF	Select channel 2 reference
9:8	CH2	Select channel 2 source
6	CH1_REF	Select channel 1 reference
5:4	CH1	Select channel 1 source
2	CH0_REF	Select channel 0 reference
1:0	CH0	Select channel 0 source

Field Name	Value Symbol	Value Description	Hex Value
CH5_REF	LSAD_CH5_REF_VREG	Use VREG as reference	0x0*
	LSAD_CH5_REF_VREF	Use VREF as reference	0x1
CH5	LSAD_CH5_DIO5	Select DIO(5) pad as LSAD CH5 input	0x0
	LSAD_CH5_DIO15	Select DIO(15) pad as LSAD CH5 input	0x1
	LSAD_CH5_DIO25	Select DIO(25) pad as LSAD CH5 input	0x2
	LSAD_CH5_VDDC	Select VDDC as LSAD CH5 input	0x3*
CH4_REF	LSAD_CH4_REF_VREG	Use VREG as reference	0x0*
	LSAD_CH4_REF_VREF	Use VREF as reference	0x1
CH4	LSAD_CH4_DIO4	Select DIO(4) pad as LSAD CH4 input	0x0
	LSAD_CH4_DIO14	Select DIO(14) pad as LSAD CH4 input	0x1
	LSAD_CH4_DIO24	Select DIO(24) pad as LSAD CH4 input	0x2
	LSAD_CH4_VDDM	Select VDDM as LSAD CH4 input	0x3*
CH3_REF	LSAD_CH3_REF_VREG	Use VREG as reference	0x0*
	LSAD_CH3_REF_VREF	Use VREF as reference	0x1
CH3	LSAD_CH3_DIO3	Select DIO(3) pad as LSAD CH3 input	0x0
	LSAD_CH3_DIO13	Select DIO(13) pad as LSAD CH3 input	0x1
	LSAD_CH3_DIO23	Select DIO(23) pad as LSAD CH3 input	0x2
	LSAD_CH3_VDBL	Select VDBL as LSAD CH3 input	0x3*
CH2_REF	LSAD_CH2_REF_VREG	Use VREG as reference	0x0*
	LSAD_CH2_REF_VREF	Use VREF as reference	0x1

Field Name	Value Symbol	Value Description	Hex Value
CH2	LSAD_CH2_DIO2	Select DIO(2) pad as LSAD CH2 input	0x0
	LSAD_CH2_DIO12	Select DIO(12) pad as LSAD CH2 input	0x1
	LSAD_CH2_DIO22	Select DIO(22) pad as LSAD CH2 input	0x2
	LSAD_CH2_VDDA	Select VDDA as LSAD CH2 input	0x3*
CH1_REF	LSAD_CH1_REF_VREG	Use VREG as reference	0x0*
	LSAD_CH1_REF_VREF	Use VREF as reference	0x1
CH1	LSAD_CH1_DIO1	Select DIO(1) pad as LSAD CH1 input	0x0
	LSAD_CH1_DIO11	Select DIO(11) pad as LSAD CH1 input	0x1
	LSAD_CH1_DIO21	Select DIO(21) pad as LSAD CH1 input	0x2
	LSAD_CH1_VREG	Select VREG as LSAD CH1 input	0x3*
CH0_REF	LSAD_CH0_REF_VREG	Use VREG as reference	0x0*
	LSAD_CH0_REF_VREF	Use VREF as reference	0x1
CH0	LSAD_CH0_DIO0	Select DIO(0) pad as LSAD CH0 input	0x0
	LSAD_CH0_DIO10	Select DIO(10) pad as LSAD CH0 input	0x1
	LSAD_CH0_DIO20	Select DIO(20) pad as LSAD CH0 input	0x2
	LSAD_CH0_DAC0	Select analog DAC0 as LSAD CH0 input	0x3*

16.4 SERIAL PERIPHERAL INTERFACE (SPI)

The Serial Peripheral Interface (SPI) of the Ezairo 7100 system is designed to communicate with external components with a particular focus on EEPROM and related non-volatile memory (NVM) devices. Setting the SPI_CFG_ENABLE bit in the D_SPI_CFG control register enables the SPI interface.

The EEPROM library from the firmware is designed to assist with using the SPI interface with an external EEPROM. See the *Firmware Reference Manual for Ezairo 7100* for more details about this library.

The SPI interface implementation operates in master mode only and all data is transmitted or received starting with the MSB of the word received. To configure the interface to transmit or receive between 1 and 32 bits of data for each word, set the SPI_CTRL_N_BITS bit field in the D_SPI_CTRL register (where in each transaction bits = SPI_CTRL_N_BITS+1).

To select the direction of data transmission, use the SPI_CTRL_RW field from the D_SPI_CTRL register. When transmitting data over the SPI interface, setting the SPI_CTRL_START bit in the D_SPI_CTRL register starts a transaction. This same bit (which is also known as the SPI_CTRL_BUSY bit) can also be read to indicate whether the SPI interface is currently processing a transaction.

If the SPI_CFG_AUTO bit in the D_SPI_CFG register is set, the SPI interface operates in auto mode. In this mode, reading or writing the data through the D_SPI_DATA register sets the SPI_CTRL_START bit, initiating the next transaction on the SPI interface.

IMPORTANT: The SPI_CTRL_BUSY bit is cleared for one cycle between transfers in auto mode. If an application is polling this bit to determine when a transfer completes, ensure that at least one cycle has elapsed after starting the transfer by writing to or reading from the D_SPI_DATA register before polling for the completion of a transfer.

NOTE: When the interface is disabled, any data currently being transmitted is allowed to complete before the interface shuts down.

Data to be transmitted and data that has been received are both stored in the D_SPI_DATA transmit and receive register. The contents of the D_SPI_DATA register are also mirrored into the D_SPI_DATA_M register and other mirrored registers (as identified by the D_SPI_DATA_M_ register name prefix) that provide partial transaction results. Reading from these mirrored registers does not trigger an SPI transaction when running in auto mode.

The interface operates using a divided form of SYSCLK, operating at a power-of-2 division from this clock as defined by the SPI_CFG_PRESCALE bit field in the D_SPI_CFG register. The interface can also be configured to react to either clock polarity by clearing or setting the SPI_CFG_CLKPOL bit in the D_SPI_CFG register. If this bit is set, the serial output changes on the rising edge and the serial input is sampled on the falling edge. If this bit is cleared, the serial output changes on the falling edge and the serial input is sampled on the rising edge.

16.4.1 SPI Interrupt

The SPI interface uses an associated interrupt which, when enabled, signals the completion of a transaction. See Section 14.1, “Interrupt Controller” on page 218 for information regarding interrupt configuration and handling.

When an interrupt is received while a user application is receiving data from the SPI interface, the new data received can be read from the D_SPI_DATA register. Alternately, if the application is transmitting data using the SPI interface, the D_SPI_DATA register can now be loaded with the next word of data to be transmitted.

16.4.2 Control and Configuration Registers

Register Name	Register Description	Address
D_SPI_CFG	Configuration of the SPI interface	P:0xF1C0
D_SPI_CTRL	Control SPI interface transactions	P:0xF1C1
D_SPI_DATA	SPI transmit and receive data	P:0xF1C2
D_SPI_DATA_M	SPI shadow data register	P:0xF1C3
D_SPI_DATA_M_16LSB	Shadow read data register 16 LSB of SPI_DATA	P:0xF1C4
D_SPI_DATA_M_16MSB	Shadow read data register 16 MSB of SPI_DATA	P:0xF1C5
D_SPI_DATA_M_8LSB	Shadow read data register 8 LSB of SPI_DATA	P:0xF1C6
D_SPI_DATA_M_8_15BITS	Shadow read data register bits 8 to 15 of SPI_DATA	P:0xF1C7
D_SPI_DATA_M_16_23BITS	Shadow read data register bits 16 to 23 of SPI_DATA	P:0xF1C8
D_SPI_DATA_M_8MSB	Shadow read data registers 8 MSB of SPI_DATA	P:0xF1C9

16.4.2.1 D_SPI_CFG Settings

Bit Field	Field Name	Description
5	CLKPOL	Select SPI interface clock polarity
4	AUTO	Enable/disable automatic read/write mode
3	ENABLE	Enable/disable the SPI interface
2:0	PRESCALE	Divider for the SPI interface

Field Name	Value Symbol	Value Description	Hex Value
CLKPOL	SPI_CLKPOL_NORMAL	Data sampled on the rising edge	0x0*
	SPI_CLKPOL_INVERSE	Data sampled on the falling edge	0x1
AUTO	SPI_PORT_AUTO_DISABLE	Disable automatic read/write mode	0x0*
	SPI_PORT_AUTO_ENABLE	Enable automatic read/write mode	0x1
ENABLE	SPI_PORT_DISABLE	Disable the SPI interface	0x0*
	SPI_PORT_ENABLE	Enable the SPI interface	0x1
PRESCALE	SPI_CLK_PRESCALE_2	SPI clock prescaled by 2	0x0*
	SPI_CLK_PRESCALE_4	SPI clock prescaled by 4	0x1
	SPI_CLK_PRESCALE_8	SPI clock prescaled by 8	0x2
	SPI_CLK_PRESCALE_16	SPI clock prescaled by 16	0x3
	SPI_CLK_PRESCALE_32	SPI clock prescaled by 32	0x4
	SPI_CLK_PRESCALE_64	SPI clock prescaled by 64	0x5
	SPI_CLK_PRESCALE_128	SPI clock prescaled by 128	0x6
	SPI_CLK_PRESCALE_256	SPI clock prescaled by 256	0x7

16.4.2.2 D_SPI_CTRL Settings

Bit Field	Field Name	Description
7	BUSY	SPI state
7	START	Start SPI transaction
6	RW	Control read/write data from SPI
5	CS_N	SPI chip select pin data
4:0	N_BITS	Number of bits to send or receive (MSB sent first)

Field Name	Value Symbol	Value Description	Hex Value
START	SPI_STOP	Stop the current SPI transaction	0x0*
	SPI_START	Start the next SPI transaction	0x1
RW	SPI_WRITE	Configure the port for writing	0x0*
	SPI_READ	Configure the port for reading	0x1
CS_N	SPI_CS_LOW	Set the SPI CS signal low	0x0*
	SPI_CS_HIGH	Set the SPI CS signal high	0x1

16.5 Universal Asynchronous Receiver-Transmitter (UART) Interface

The general-purpose UART interface is designed to support RS-232 communications with the Ezairo 7100 system. Setting the UART_CFG_ENABLE bit in the D_UART_CFG control register enables the UART interface.

The UART interface operates in full-duplex mode using a standard data format of 1 start bit, 8 data bits and 1 stop bit. All data bytes being sent or received are interpreted as starting with the LSB. Figure 55 shows the waveform for a UART transmit or receive transaction.

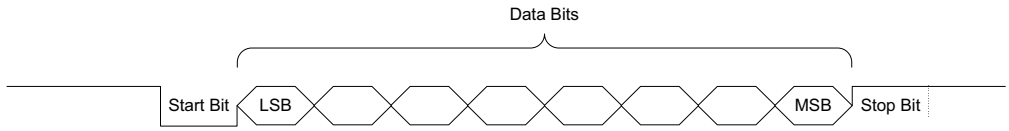


Figure 55. UART Transaction Waveform

Data to be transmitted and data that has been received are both stored in the D_UART_DATA transmit and receive register.

NOTE: When the interface is disabled, any data currently being transmitted is allowed to complete before the interface shuts down.

The baud rate (specified in bits per second) for the UART interface is defined in terms of SYSCLK and several configuration parameters as seen in the following equation for baud rate calculation:

$$\text{baud rate} = \frac{(\text{UART_CFG_PRESCALE} \cdot \text{SYSCLK})}{(2^{18} \times (1 + 11 \times \text{UART_CFG_PRESCALE_ENABLE}))}$$

Starting from SYSCLK, a constant division by 2^{18} (or $0.2048 \times [1.28 \times 10^{6}]$) is applied. Next, a pre-scaling division by 12 can be added by setting the UART_CFG_PRESCALE_ENABLE bit in the D_UART_CFG register. Finally, the UART_CFG_PRESCALE bit field from the same register provides a 16-bit multiplication of the resulting value to obtain the final baud rate. Together these settings allow the selection of a baud rate between 0.4 baud and 320 kbaud for a SYSCLK of 1.28 MHz.

For example, to obtain a standard 115200 baud rate with a SYSCLK of 1.28 MHz, clear the UART_CFG_PRESCALE_ENABLE bit and set the UART_CFG_PRESCALE bit field to 23593.

16.5.1 UART Interrupts

The UART interface uses an interrupt that controls transmission and reception of UART data. The UART interrupt is multiplexed with HEAR interrupt 5. See Section 14.1, “Interrupt Controller” on page 218 for information regarding interrupt configuration and handling.

If a user application is receiving data from the UART interface, the UART interrupt and UART_DATA_READY_UART_RX_READY bit field from the D_UART_DATA_READY register signal that a data byte was successfully received and written to the D_UART_DATA register.

If a user application is transmitting data from the UART interface, the UART interrupt and UART_DATA_READY_UART_TX_READY bit field from the D_UART_DATA_READY register signal that transmission of the data value in the D_UART_DATA register has started and that the application can now load the next byte to be transmitted.

16.5.2 Control and Configuration Registers

Register Name	Register Description	Address
D_UART_CFG	UART configuration registers	P:0xF2C0

Register Name	Register Description	Address
D_UART_DATA	UART data register	P:0xF2C1
D_UART_DATA_READY	UART data ready register	P:0xF2C2

16.5.2.1 D_UART_CFG Settings

Bit Field	Field Name	Description
23:8	PRESCALE	Pre-scaling multiplier in baud rate calculation
4	PRESCALE_ENABLE	Enable/disable a fixed prescaler by 12
0	ENABLE	Enable/disable the UART interface

Field Name	Value Symbol	Value Description	Hex Value
PRESCALE_ENABLE	UART_PRESCALE_DISABLE	Disable pre-scaling division by 12	0x0*
	UART_PRESCALE_ENABLE	Enable pre-scaling division by 12	0x1
ENABLE	UART_DISABLE	Disable the UART interface	0x0*
	UART_ENABLE	Enable the UART interface	0x1

16.5.2.2 D_UART_DATA Settings

Bit Field	Field Name	Description
7:0	UART_DATA	Data to transmit or receive through UART interface

16.5.2.3 D_UART_DATA_READY Settings

Bit Field	Field Name	Description
1	UART_RX_READY	Status of the UART receive buffer
0	UART_TX_READY	Status of the UART transmit buffer

Field Name	Value Symbol	Value Description	Hex Value
UART_RX_READY	UART_RX_NOT_READY	Received data not available	0x0*
	UART_RX_READY	Received data available	0x1
UART_TX_READY	UART_TX_NOT_READY	New transmit data cannot be supplied	0x0*
	UART_TX_READY	New transmit data can be supplied	0x1

CHAPTER 17
ARM Cortex-M3 Processor External Interfaces

17.1 I²C Interface

The ARM Cortex-M3 processor external interfaces include an I²C interface supporting both master and slave mode transfers. This interface implements a subset of the Philips I²C interface as described in the *Philips I²C Bus Specification* document.

The ARM Cortex-M3 processor's I²C interface generally shares a design implementation with the CFX's general-purpose I²C interface. Differences include:

- This ARM Cortex-M3 processor's I²C interface has its own dedicated I²C bus defined by the DIO block (not shared with the debug ports)
- Some register settings are mapped differently
- The underlying clock used to sample the interface lines for this interface can be disabled in all system states.

For information about:

- I²C buses, see Section 13.1, "Inter-IC Interface (I2C) Bus®" on page 188
- Configuring the DIOs to support the ARM Cortex-M3 processor's I²C bus, see Section 16.1, "Digital I/O (DIO)" on page 351
- The general-purpose I²C implementation this interface shares, see Section 13.4, "General Purpose I2C Interface" on page 203.

IMPORTANT: I²C interfaces are defined using open-collector pads for the I²C bus. Because these pads do not drive a high signal for the bus signals—relying on the bus's pull-up resistors for proper operation—user applications that use this interface must use either the internal pull-up resistors implemented for the DIO pads, or a set of external pull-up resistors on the DIOs assigned to be the I²C bus lines.

17.1.1 ARM Cortex-M3 Processor I²C interface Register Mapping

Differences between the mapping of the registers from the general-purpose I²C interface and the mapping of registers to the ARM Cortex-M3 processor's I²C interface include:

- Instead of supporting masking of the I²C sample clock only when it is connected to the standby oscillator, the ARM Cortex-M3 processor's I²C interface supports masking this interface clock (and hence disabling the I²C interface) in all system clock modes. To enable the I²C interface, set the `I2C_CTRL0_I2C_SAMPLE_CLK_ENABLE` bit from the `C_I2C_CTRL0` register.

  All other configurations defined for the `D_I2C_CFG` register are defined identically for the ARM Cortex-M3 processor's I²C interface through the `C_I2C_CTRL0` register.

- The write-only bits from the `D_I2C_CTRL` register, defined in Table 35 on page 203, are implemented in the `C_I2C_CTRL1` register. The event and state bits from the `D_I2C_CTRL` register that are defined in this same table are implemented in the `C_I2C_STATUS` register.

The `C_I2C_DATA`, `C_I2C_DATA_M` and `C_I2C_ADDR_START` registers are implemented identically to the `D_I2C_DATA`, `D_I2C_DATA_M` and `D_I2C_ADDR_START` registers.

17.1.2 I²C Registers

Register Name	Register Description	Address
I2C_CTRL0 C_I2C_CTRL0	I²C Interface Configuration and Control	0x40000900 P:0x3240
I2C_CTRL1 C_I2C_CTRL1	I²C Interface Status and Control	0x40000904 P:0x3241
I2C_DATA C_I2C_DATA	I²C Interface Data	0x40000910 P:0x3244
I2C_DATA_M C_I2C_DATA_M	I²C Interface Data (Mirror)	0x40000914 P:0x3245
I2C_ADDR_START C_I2C_ADDR_START	I²C Master Address and Start	0x40000918 P:0x3246
I2C_STATUS C_I2C_STATUS	I²C Status	0x4000091C P:0x3247

17.1.2.1 I2C_CTRL0, C_I2C_CTRL0 Settings

Bit Field	Field Name	Description
23:16	MASTER_SPEED_PRESCALER	Prescaler used to divide SYSCLK to the correct SCL frequency when operating in I²C interface master mode. SCL is prescaled by (MASTER_SPEED_PRESCALER + 1) * 3
14:8	SLAVE_ADDRESS	Set the I²C slave address for this interface
4	CONTROLLER	Select whether the data transfer will be controlled by the ARM Cortex-M3 processor or the DMA for I2C
3	STOP_INT_ENABLE	Configure whether stop interrupts will be generated by the I²C interface
2	AUTO_ACK_ENABLE	Select whether an acknowledgement is automatically generated or not
1	I2C_SAMPLE_CLK_ENABLE	Enable/disable the I²C sample clock (mandatory to enable the I2C)
0	SLAVE_ENABLE	Select whether the I²C interface will be enabled for slave mode or not

Field Name	Value Symbol	Value Description	Hex Value
MASTER_SPEED_PRESCALER	C_I2C_MASTER_SPEED_3	Prescale SCL from SYSCLK by 3	0x0*
	C_I2C_MASTER_SPEED_6	Prescale SCL from SYSCLK by 6	0x1
	C_I2C_MASTER_SPEED_9	Prescale SCL from SYSCLK by 9	0x2
	C_I2C_MASTER_SPEED_12	Prescale SCL from SYSCLK by 12	0x3
	C_I2C_MASTER_SPEED_15	Prescale SCL from SYSCLK by 15	0x4
	C_I2C_MASTER_SPEED_18	Prescale SCL from SYSCLK by 18	0x5
	C_I2C_MASTER_SPEED_21	Prescale SCL from SYSCLK by 21	0x6
	C_I2C_MASTER_SPEED_24	Prescale SCL from SYSCLK by 24	0x7
	C_I2C_MASTER_SPEED_27	Prescale SCL from SYSCLK by 27	0x8
	C_I2C_MASTER_SPEED_30	Prescale SCL from SYSCLK by 30	0x9
	C_I2C_MASTER_SPEED_33	Prescale SCL from SYSCLK by 33	0xA
	C_I2C_MASTER_SPEED_36	Prescale SCL from SYSCLK by 36	0xB
	C_I2C_MASTER_SPEED_39	Prescale SCL from SYSCLK by 39	0xC
	C_I2C_MASTER_SPEED_42	Prescale SCL from SYSCLK by 42	0xD
	C_I2C_MASTER_SPEED_45	Prescale SCL from SYSCLK by 45	0xE
	C_I2C_MASTER_SPEED_48	Prescale SCL from SYSCLK by 48	0xF
	C_I2C_MASTER_SPEED_51	Prescale SCL from SYSCLK by 51	0x10
CONTROLLER	C_I2C_CONTROLLER_CM3	The ARM Cortex-M3 processor controls data transfers using I2C	0x0*
	C_I2C_CONTROLLER_DMA	The DMA controls data transfers using I2C	0x1
STOP_INT_ENABLE	C_I2C_STOP_INT_DISABLE	Stop interrupts will not be generated	0x0*
	C_I2C_STOP_INT_ENABLE	A stop interrupt will be generated when a stop condition occurs for an active transaction	0x1
AUTO_ACK_ENABLE	C_I2C_AUTO_ACK_DISABLE	Require manual acknowledgement of all I^2C interface transfers	0x0*
	C_I2C_AUTO_ACK_ENABLE	Use automatic acknowledgement for I^2C interface transfers	0x1
I2C_SAMPLE_CLK_ENABLE	C_I2C_SAMPLE_CLK_DISABLE	Disable the I^2C sample clock (I^2C is disabled)	0x0*
	C_I2C_SAMPLE_CLK_ENABLE	Enable the I^2C sample clock (I^2C is enabled)	0x1
SLAVE_ENABLE	C_I2C_SLAVE_DISABLE	Disable I^2C interface slave mode operation	0x0*
	C_I2C_SLAVE_ENABLE	Enable I^2C interface slave mode operation	0x1

17.1.2.2 I2C_CTRL1, C_I2C_CTRL1 Settings

Bit Field	Field Name	Description
5	RESET	Reset the I^{2}C interface
4	LAST_DATA	Indicate that the current data is the last byte of a data transfer
3	STOP	Issue a stop condition the I^{2}C interface bus
1	NACK	Issue a not acknowledge on the I^{2}C interface bus
0	ACK	Issue an acknowledge on the I^{2}C interface bus

Field Name	Value Symbol	Value Description	Hex Value
RESET	C_I2C_RESET	Reset the I^{2}C interface	0x1
LAST_DATA	C_I2C_LAST_DATA	Indicate that the current data is the last byte of a data transfer	0x1
STOP	C_I2C_STOP	Issue a stop condition the I^{2}C interface bus	0x1
NACK	C_I2C_NACK	Issue a not acknowledge on the I^{2}C interface bus	0x1
ACK	C_I2C_ACK	Issue an acknowledge on the I^{2}C interface bus	0x1

17.1.2.3 I2C_DATA, C_I2C_DATA Settings

Bit Field	Field Name	Description
7:0	I2C_DATA	Single byte buffer for data transmitted and received over the I^{2}C interface

17.1.2.4 I2C_DATA_M, C_I2C_DATA_M Settings

Bit Field	Field Name	Description
7:0	I2C_DATA_M	Mirror of the single byte buffer for data transmitted and received over the I^{2}C interface

17.1.2.5 I2C_ADDR_START, C_I2C_ADDR_START Settings

Bit Field	Field Name	Description
7:1	ADDRESS	I^{2}C address to use for the transaction
0	READ_WRITE	Select whether a read or a write transaction is started

Field Name	Value Symbol	Value Description	Hex Value
READ_WRITE	C_I2C_START_WRITE	Start an I^{2}C write transaction	0x0
	C_I2C_START_READ	Start an I^{2}C read transaction	0x1

17.1.2.6 I2C_STATUS, C_I2C_STATUS Settings

Bit Field	Field Name	Description
15	ERROR_S	Error status bit (sticky) - This status bit is automatically reset when the I2C_STATUS register is read.
14	BUS_ERROR_S	Bus error status bit (sticky) - This status bit is automatically reset when the I2C_STATUS register is read.
13	START_PENDING	Master frame start pending status bit
12	MASTER_MODE	Master mode status bit
11	DMA_REQ	Indicate if the I^2C interface is currently requesting DMA data
10	STOP_DETECT	Indicate if an I^2C stop bit has been detected
9	DATA_EVENT	Indicate that the I^2C interface either needs data to transmit or has received data
8	ERROR	Indicate if the I^2C interface has detected a bus error (automatically cleared when a stop condition is detected)
7	BUS_ERROR	Indicate if the I^2C interface has detected a bus error (automatically cleared when a stop condition is detected)
6	BUFFER_FULL	Indicate if the I^2C data buffer is full
5	CLK_STRETCH	Indicate if the I^2C interface is holding the clock signal
4	BUS_FREE	Indicate if the I^2C interface bus is free
3	ADDR_DATA	Indicate if the I^2C data register holds an address or data byte
2	READ_WRITE	Indicate whether the I^2C bus transfer is a read or a write
1	GEN_CALL	Indicate whether the I^2C bus transfer is using the general call address or another address
0	ACK_STATUS	Indicate whether an acknowledge or a not acknowledge has been received

Field Name	Value Symbol	Value Description	Hex Value
ERROR_S	C_I2C_NO_ERROR_S	I^2C interface is not and has not been in an error state	0x0*
	C_I2C_ERROR_S	I^2C interface is or has been in an error state	0x1
BUS_ERROR_S	C_I2C_NO_BUS_ERROR_S	I^2C interface is not and has not been in the bus error state	0x0*
	C_I2C_BUS_ERROR_S	I^2C interface is or has been in the bus error state	0x1
START_PENDING	C_I2C_START_NOT_PENDING	No pending master start frame	0x0*
	C_I2C_START_PENDING	A master frame is pending to start (bit is set when the C_I2C_ADDR_START register is written)	0x1
MASTER_MODE	C_I2C_MASTER_INACTIVE	I^2C interface is not operating in master mode	0x0*
	C_I2C_MASTER_ACTIVE	I^2C interface is operating in master mode	0x1

Field Name	Value Symbol	Value Description	Hex Value
DMA_REQ	C_I2C_NO_DMA_REQUEST	The I²C interface is not requesting a DMA action	0x0*
	C_I2C_DMA_REQUEST	The I²C interface is requesting a DMA action	0x1
STOP_DETECT	C_I2C_NO_STOP_DETECTED	No stop condition has been detected on the I²C bus	0x0*
	C_I2C_STOP_DETECTED	A stop condition has been detected on the I²C bus	0x1
DATA_EVENT	C_I2C_NON_DATA_EVENT	No I²C data is needed or available	0x0*
	C_I2C_DATA_EVENT	I²C data is needed or is available	0x1
ERROR	C_I2C_NO_ERROR	I²C interface is not in an error state	0x0*
	C_I2C_ERROR	I²C interface is in an error state	0x1
BUS_ERROR	C_I2C_NO_BUS_ERROR	I²C interface is not in the bus error state	0x0*
	C_I2C_BUS_ERROR	I²C interface is in the bus error state	0x1
BUFFER_FULL	C_I2C_BUFFER_EMPTY	The I²C interface buffer is empty	0x0*
	C_I2C_BUFFER_FULL	The I²C interface buffer is full	0x1
CLK_STRETCH	C_I2C_CLK_NOT_STRETCHED	The I²C clock line is not being stretched	0x0*
	C_I2C_CLK_STRETCHED	The I²C SCL line is being held low	0x1
BUS_FREE	C_I2C_BUS_BUSY	One or both of the I²C bus lines is currently 0	0x0
	C_I2C_BUS_FREE	Both I²C bus lines are currently free	0x1*
ADDR_DATA	C_I2C_DATA_IS_DATA	The I²C data register holds data	0x0*
	C_I2C_DATA_IS_ADDR	The I²C data register holds an address	0x1
READ_WRITE	C_I2C_IS_WRITE	The current I²C transfer is a write	0x0*
	C_I2C_IS_READ	The current I²C transfer is a read	0x1
GEN_CALL	C_I2C_ADDR_OTHER	The address used for the current I²C transfer is not the general call address	0x0*
	C_I2C_ADDR_GEN_CALL	The address used for the current I²C transfer is the general call address	0x1
ACK_STATUS	C_I2C_HAS_ACK	Indicate that the last I²C byte was acknowledged	0x0*
	C_I2C_HAS_NACK	Indicate that the last I²C byte was not acknowledged	0x1

17.2 GPIO Interfaces

Ezairo 7100 can configure any of the DIO pads as ARM Cortex-M3 processor software-controlled general-purpose digital I/O (GPIO) pads. The function of these GPIO pads is defined by a user application, which can use them for general input or output.

The GPIO interface is provided with several registers to identify which DIO pads have been configured for ARM Cortex-M3 processor controlled GPIO functionality (C_DIO_MODE), their I/O direction (C_DIO_DIR) and value

(C_DIO_DATA). The C_DIO_MODE, C_DIO_DIR and C_DIO_DATA registers are implemented identically to the D_DIO_MODE, D_DIO_DIR and D_DIO_DATA registers, as described in Section 16.2, “General-Purpose I/O (GPIO) Interface” on page 376.

IMPORTANT: When the DIO’s ARM Cortex-M3 processor-controlled GPIO functionality is selected, the configured pins are placed under ARM Cortex-M3 processor software control. When an alternate DIO functional configuration is selected (including any DIO functionality used for other ARM Cortex-M3 processor external interfaces), the pins are directly under that selected system component’s control.

17.2.1 GPIO Interrupts

The GPIO interface provides an associated configurable interrupt which, when enabled, signals the occurrence of an event or a condition on a specified GPIO pad. See Section 15.1, “Nested Vectored Interrupt Controller (NVIC)” on page 239 for information regarding interrupt configuration and handling.

The GPIO interrupt can be configured using the C_DIO_INT_CFG_* registers, and the status of this interrupt can be found in the C_DIO_INT_STATUS register. These registers are implemented identically to the D_DIO_CFX_INT_CFG_* and D_DIO_CFX_INT_STATUS registers, and are mirrored by the D_DIO_CM3_INT_CFG_* and D_DIO_CM3_INT_STATUS registers. For more information, see Section 16.2.1, “GPIO Interrupt” on page 377.

17.2.2 GPIO Interface Registers

Register Name	Register Description	Address
DIO_DATA C_DIO_DATA	DIOs data access pseudo register	0x40000500 P:0x3140
DIO_DIR C_DIO_DIR	GPIOs data direction pseudo register	0x40000504 P:0x3141
DIO_MODE C_DIO_MODE	DIOs mode pseudo register	0x40000508 P:0x3142
DIO_INT_CFG_0 C_DIO_INT_CFG_0	ARM Cortex-M3 core DIO interrupt 0 configuration	0x4000050C P:0x3143
DIO_INT_CFG_1 C_DIO_INT_CFG_1	ARM Cortex-M3 core DIO interrupt 1 configuration	0x40000510 P:0x3144
DIO_INT_CFG_2 C_DIO_INT_CFG_2	ARM Cortex-M3 core DIO interrupt 2 configuration	0x40000514 P:0x3145
DIO_INT_CFG_3 C_DIO_INT_CFG_3	ARM Cortex-M3 core DIO interrupt 3 configuration	0x40000518 P:0x3146
DIO_INT_STATUS C_DIO_INT_STATUS	ARM Cortex-M3 core DIO interrupt status	0x4000051C P:0x3147
D_CM3_DIO_INT_CFG_0	ARM Cortex-M3 core DIO interrupt configuration	P:0xF2AC
D_CM3_DIO_INT_CFG_1	ARM Cortex-M3 core DIO interrupt configuration	P:0xF2AD
D_CM3_DIO_INT_CFG_2	ARM Cortex-M3 core DIO interrupt configuration	P:0xF2AE
D_CM3_DIO_INT_CFG_3	ARM Cortex-M3 core DIO interrupt configuration	P:0xF2AF
D_CM3_DIO_INT_STATUS	ARM Cortex-M3 core DIO interrupt status	P:0xF2B1

17.2.2.1 DIO_DATA, C_DIO_DATA Settings

Bit Field	Field Name	Description
29:0	C_DIO	DIO [29:0] read data (pads configured for LSAD mode return 0)
29:0	C_GPIO	GPIO [29:0] write data (updates output data of DIO [29:0] only for pads with IO_MODE 0b100XX)

Field Name	Value Symbol	Value Description	Hex Value
C_DIO	C_DIO0_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO1_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO2_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO3_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO4_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO5_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO6_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO7_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO8_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO9_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO10_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO11_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO12_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO13_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO14_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO15_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO16_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO17_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO18_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO19_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO20_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO21_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO22_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO23_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO24_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO25_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO26_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO27_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO28_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO29_LOW	DIO pad is low or pad is in LSAD mode	0x0*
	C_DIO0_HIGH	DIO pad is high	0x1
	C_DIO1_HIGH	DIO pad is high	0x2
	C_DIO2_HIGH	DIO pad is high	0x4
	C_DIO3_HIGH	DIO pad is high	0x8
	C_DIO4_HIGH	DIO pad is high	0x10
	C_DIO5_HIGH	DIO pad is high	0x20
	C_DIO6_HIGH	DIO pad is high	0x40

Field Name	Value Symbol	Value Description	Hex Value
C_DIO (continued)	C_DIO7_HIGH	DIO pad is high	0x80
	C_DIO8_HIGH	DIO pad is high	0x100
	C_DIO9_HIGH	DIO pad is high	0x200
	C_DIO10_HIGH	DIO pad is high	0x400
	C_DIO11_HIGH	DIO pad is high	0x800
	C_DIO12_HIGH	DIO pad is high	0x1000
	C_DIO13_HIGH	DIO pad is high	0x2000
	C_DIO14_HIGH	DIO pad is high	0x4000
	C_DIO15_HIGH	DIO pad is high	0x8000
	C_DIO16_HIGH	DIO pad is high	0x10000
	C_DIO17_HIGH	DIO pad is high	0x20000
	C_DIO18_HIGH	DIO pad is high	0x40000
	C_DIO19_HIGH	DIO pad is high	0x80000
	C_DIO20_HIGH	DIO pad is high	0x100000
	C_DIO21_HIGH	DIO pad is high	0x200000
	C_DIO22_HIGH	DIO pad is high	0x400000
	C_DIO23_HIGH	DIO pad is high	0x800000
	C_DIO24_HIGH	DIO pad is high	0x1000000
	C_DIO25_HIGH	DIO pad is high	0x2000000
	C_DIO26_HIGH	DIO pad is high	0x4000000
	C_DIO27_HIGH	DIO pad is high	0x8000000
	C_DIO28_HIGH	DIO pad is high	0x10000000
	C_DIO29_HIGH	DIO pad is high	0x20000000
C_GPIO	C_GPIO0_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO1_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO2_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO3_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO4_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO5_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO6_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO7_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO8_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO9_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO10_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO11_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO12_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO13_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO14_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0

Field Name	Value Symbol	Value Description	Hex Value
C_GPIO (continued)	C_GPIO15_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO16_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO17_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO18_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO19_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO20_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO21_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO22_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO23_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO24_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO25_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO26_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO27_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO28_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO29_LOW	Set the DIO pad to low if IO_MODE is 0b100XX	0x0
	C_GPIO0_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x1
	C_GPIO1_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x2
	C_GPIO2_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x4
	C_GPIO3_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x8
	C_GPIO4_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x10
	C_GPIO5_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x20
	C_GPIO6_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x40
	C_GPIO7_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x80
	C_GPIO8_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x100
	C_GPIO9_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x200
	C_GPIO10_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x400
	C_GPIO11_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x800
	C_GPIO12_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x1000
	C_GPIO13_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x2000
	C_GPIO14_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x4000
	C_GPIO15_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x8000
	C_GPIO16_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x10000
	C_GPIO17_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x20000
	C_GPIO18_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x40000
	C_GPIO19_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x80000
	C_GPIO20_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x100000
	C_GPIO21_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x200000
	C_GPIO22_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x400000

Field Name	Value Symbol	Value Description	Hex Value
C_GPIO (continued)	C_GPIO23_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x800000
	C_GPIO24_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x1000000
	C_GPIO25_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x2000000
	C_GPIO26_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x4000000
	C_GPIO27_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x8000000
	C_GPIO28_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x10000000
	C_GPIO29_HIGH	Set the DIO pad to high if IO_MODE is 0b100XX	0x20000000

17.2.2.2 DIO_DIR, C_DIO_DIR Settings

Bit Field	Field Name	Description
29:0	C_DIO	Get DIO direction
29:0	C_GPIO	Set DIO GPIO direction (only if IO_MODE is 0b100XX)

Field Name	Value Symbol	Value Description	Hex Value
C_DIO	C_DIO0_INPUT	DIO is an input	0x0*
	C_DIO1_INPUT	DIO is an input	0x0*
	C_DIO2_INPUT	DIO is an input	0x0*
	C_DIO3_INPUT	DIO is an input	0x0*
	C_DIO4_INPUT	DIO is an input	0x0*
	C_DIO5_INPUT	DIO is an input	0x0*
	C_DIO6_INPUT	DIO is an input	0x0*
	C_DIO7_INPUT	DIO is an input	0x0*
	C_DIO8_INPUT	DIO is an input	0x0*
	C_DIO9_INPUT	DIO is an input	0x0*
	C_DIO10_INPUT	DIO is an input	0x0*
	C_DIO11_INPUT	DIO is an input	0x0*
	C_DIO12_INPUT	DIO is an input	0x0*
	C_DIO13_INPUT	DIO is an input	0x0*
	C_DIO14_INPUT	DIO is an input	0x0*
	C_DIO15_INPUT	DIO is an input	0x0*
	C_DIO16_INPUT	DIO is an input	0x0*
	C_DIO17_INPUT	DIO is an input	0x0*
	C_DIO18_INPUT	DIO is an input	0x0*
	C_DIO19_INPUT	DIO is an input	0x0*
	C_DIO20_INPUT	DIO is an input	0x0*
	C_DIO21_INPUT	DIO is an input	0x0*

Field Name	Value Symbol	Value Description	Hex Value
C_DIO (continued)	C_DIO22_INPUT	DIO is an input	0x0*
	C_DIO23_INPUT	DIO is an input	0x0*
	C_DIO24_INPUT	DIO is an input	0x0*
	C_DIO25_INPUT	DIO is an input	0x0*
	C_DIO26_INPUT	DIO is an input	0x0*
	C_DIO27_INPUT	DIO is an input	0x0*
	C_DIO28_INPUT	DIO is an input	0x0*
	C_DIO29_INPUT	DIO is an input	0x0*
	C_DIO0_OUTPUT	DIO is an output	0x1
	C_DIO1_OUTPUT	DIO is an output	0x2
	C_DIO2_OUTPUT	DIO is an output	0x4
	C_DIO3_OUTPUT	DIO is an output	0x8
	C_DIO4_OUTPUT	DIO is an output	0x10
	C_DIO5_OUTPUT	DIO is an output	0x20
	C_DIO6_OUTPUT	DIO is an output	0x40
	C_DIO7_OUTPUT	DIO is an output	0x80
	C_DIO8_OUTPUT	DIO is an output	0x100
	C_DIO9_OUTPUT	DIO is an output	0x200
	C_DIO10_OUTPUT	DIO is an output	0x400
	C_DIO11_OUTPUT	DIO is an output	0x800
	C_DIO12_OUTPUT	DIO is an output	0x1000
	C_DIO13_OUTPUT	DIO is an output	0x2000
	C_DIO14_OUTPUT	DIO is an output	0x4000
	C_DIO15_OUTPUT	DIO is an output	0x8000
	C_DIO16_OUTPUT	DIO is an output	0x10000
	C_DIO17_OUTPUT	DIO is an output	0x20000
	C_DIO18_OUTPUT	DIO is an output	0x40000
	C_DIO19_OUTPUT	DIO is an output	0x80000
	C_DIO20_OUTPUT	DIO is an output	0x100000
	C_DIO21_OUTPUT	DIO is an output	0x200000
	C_DIO22_OUTPUT	DIO is an output	0x400000
	C_DIO23_OUTPUT	DIO is an output	0x800000
	C_DIO24_OUTPUT	DIO is an output	0x1000000
	C_DIO25_OUTPUT	DIO is an output	0x2000000
	C_DIO26_OUTPUT	DIO is an output	0x4000000
	C_DIO27_OUTPUT	DIO is an output	0x8000000
	C_DIO28_OUTPUT	DIO is an output	0x10000000
	C_DIO29_OUTPUT	DIO is an output	0x20000000

Field Name	Value Symbol	Value Description	Hex Value
C_GPIO	C_GPIO0_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO1_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO2_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO3_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO4_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO5_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO6_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO7_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO8_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO9_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO10_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO11_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO12_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO13_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO14_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO15_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO16_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO17_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO18_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO19_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO20_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO21_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO22_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO23_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO24_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO25_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO26_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO27_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO28_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO29_INPUT	Set DIO to input if IO_MODE is 0b100XX	0x0
	C_GPIO0_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x1
	C_GPIO1_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x2
	C_GPIO2_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x4
	C_GPIO3_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x8
	C_GPIO4_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x10
	C_GPIO5_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x20
	C_GPIO6_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x40
	C_GPIO7_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x80

Field Name	Value Symbol	Value Description	Hex Value
C_GPIO (continued)	C_GPIO8_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x100
	C_GPIO9_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x200
	C_GPIO10_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x400
	C_GPIO11_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x800
	C_GPIO12_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x1000
	C_GPIO13_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x2000
	C_GPIO14_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x4000
	C_GPIO15_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x8000
	C_GPIO16_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x10000
	C_GPIO17_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x20000
	C_GPIO18_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x40000
	C_GPIO19_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x80000
	C_GPIO20_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x100000
	C_GPIO21_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x200000
	C_GPIO22_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x400000
	C_GPIO23_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x800000
	C_GPIO24_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x1000000
	C_GPIO25_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x2000000
	C_GPIO26_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x4000000
	C_GPIO27_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x8000000
	C_GPIO28_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x10000000
	C_GPIO29_OUTPUT	Set DIO to output if IO_MODE is 0b100XX	0x20000000

17.2.2.3 DIO_MODE, C_DIO_MODE Settings

Bit Field	Field Name	Description
29:0	C_GPIO	DIO mode

Field Name	Value Symbol	Value Description	Hex Value
C_GPIO	C_DIO0_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*
	C_DIO1_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*
	C_DIO2_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*
	C_DIO3_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*

Field Name	Value Symbol	Value Description	Hex Value
C_GPIO (continued)	C_DIO4_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*
	C_DIO5_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*
	C_DIO6_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*
	C_DIO7_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*
	C_DIO8_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*
	C_DIO9_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*
	C_DIO10_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*
	C_DIO11_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*
	C_DIO12_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*
	C_DIO13_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*
	C_DIO14_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*
	C_DIO15_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*
	C_DIO16_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*
	C_DIO17_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*
	C_DIO18_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*
	C_DIO19_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*
	C_DIO20_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*
	C_DIO21_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*
	C_DIO22_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*
	C_DIO23_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*
	C_DIO24_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*
	C_DIO25_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*

Field Name	Value Symbol	Value Description	Hex Value
C_GPIO (continued)	C_DIO26_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*
	C_DIO27_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*
	C_DIO28_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*
	C_DIO29_IS_NOT_C_GPIO	This DIO is not configured as an ARM Cortex-M3 processor controlled GPIO	0x0*
	C_DIO0_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x1
	C_DIO1_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x2
	C_DIO2_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x4
	C_DIO3_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x8
	C_DIO4_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x10
	C_DIO5_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x20
	C_DIO6_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x40
	C_DIO7_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x80
	C_DIO8_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x100
	C_DIO9_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x200
	C_DIO10_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x400
	C_DIO11_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x800
	C_DIO12_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x1000
	C_DIO13_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x2000
	C_DIO14_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x4000
	C_DIO15_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x8000
	C_DIO16_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x10000
	C_DIO17_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x20000

Field Name	Value Symbol	Value Description	Hex Value
C_GPIO (continued)	C_DIO18_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x40000
	C_DIO19_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x80000
	C_DIO20_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x100000
	C_DIO21_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x200000
	C_DIO22_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x400000
	C_DIO23_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x800000
	C_DIO24_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x1000000
	C_DIO25_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x2000000
	C_DIO26_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x4000000
	C_DIO27_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x8000000
	C_DIO28_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x10000000
	C_DIO29_IS_C_GPIO	This DIO is configured as an ARM Cortex-M3 processor controlled GPIO	0x20000000

17.2.2.4 DIO_INT_CFG_*, C_DIO_INT_CFG_*, D_CM3_DIO_INT_CFG_* Settings

The following bit fields and field names apply equally to registers DIO_INT_CFG_0 to DIO_INT_CFG_3 (C_DIO_INT_CFG_0 to C_DIO_INT_CFG_3), and their shadow registers D_CM3_DIO_INT_CFG_0 to D_CM3_DIO_INT_CFG_3.

Bit Field	Field Name	Description
7:5	CM3_INT_EVENT	ARM Cortex-M3 core interrupt event configuration
4:0	CM3_INT_SRC	ARM Cortex-M3 core interrupt input selection

Field Name	Value Symbol	Value Description	Hex Value
CM3_INT_EVENT	C_DIO_EVENT_NONE	Interrupt not triggered	0x0*
	C_DIO_EVENT_HIGH_LEVEL	Interrupt triggered on high state	0x1
	C_DIO_EVENT_LOW_LEVEL	Interrupt triggered on low state	0x2
	C_DIO_EVENT_RISING_EDGE	Interrupt triggered on rising edge	0x3
	C_DIO_EVENT_FALLING_EDGE	Interrupt triggered on falling edge	0x4
	C_DIO_EVENT_TRANSITION	Interrupt triggered on any edge	0x5

Field Name	Value Symbol	Value Description	Hex Value
CM3_INT_SRC	C_DIO_SRC_DIO_0	Select DIO[0] as source	0x0*
	C_DIO_SRC_DIO_1	Select DIO[1] as source	0x1
	C_DIO_SRC_DIO_2	Select DIO[2] as source	0x2
	C_DIO_SRC_DIO_3	Select DIO[3] as source	0x3
	C_DIO_SRC_DIO_4	Select DIO[4] as source	0x4
	C_DIO_SRC_DIO_5	Select DIO[5] as source	0x5
	C_DIO_SRC_DIO_6	Select DIO[6] as source	0x6
	C_DIO_SRC_DIO_7	Select DIO[7] as source	0x7
	C_DIO_SRC_DIO_8	Select DIO[8] as source	0x8
	C_DIO_SRC_DIO_9	Select DIO[9] as source	0x9
	C_DIO_SRC_DIO_10	Select DIO[10] as source	0xA
	C_DIO_SRC_DIO_11	Select DIO[11] as source	0xB
	C_DIO_SRC_DIO_12	Select DIO[12] as source	0xC
	C_DIO_SRC_DIO_13	Select DIO[13] as source	0xD
	C_DIO_SRC_DIO_14	Select DIO[14] as source	0xE
	C_DIO_SRC_DIO_15	Select DIO[15] as source	0xF
	C_DIO_SRC_DIO_16	Select DIO[16] as source	0x10
	C_DIO_SRC_DIO_17	Select DIO[17] as source	0x11
	C_DIO_SRC_DIO_18	Select DIO[18] as source	0x12
	C_DIO_SRC_DIO_19	Select DIO[19] as source	0x13
	C_DIO_SRC_DIO_20	Select DIO[20] as source	0x14
	C_DIO_SRC_DIO_21	Select DIO[21] as source	0x15
	C_DIO_SRC_DIO_22	Select DIO[22] as source	0x16
	C_DIO_SRC_DIO_23	Select DIO[23] as source	0x17
	C_DIO_SRC_DIO_24	Select DIO[24] as source	0x18
	C_DIO_SRC_DIO_25	Select DIO[25] as source	0x19
	C_DIO_SRC_DIO_26	Select DIO[26] as source	0x1A
	C_DIO_SRC_DIO_27	Select DIO[27] as source	0x1B
	C_DIO_SRC_DIO_28	Select DIO[28] as source	0x1C
	C_DIO_SRC_DIO_29	Select DIO[29] as source	0x1D
CM3_INT_SRC (continued)	C_DIO_SRC_SCL	Select SCL as source	0x1E
	C_DIO_SRC_SDA	Select SDA as source	0x1F

17.2.2.5 DIO_INT_STATUS, C_DIO_INT_STATUS and D_CM3_DIO_INT_STATUS Settings

The following bit fields and field names apply equally to register DIO_INT_STATUS (C_DIO_INT_STATUS) and its mirror register D_CM3_DIO_INT_STATUS.

Bit Field	Field Name	Description
3	CM3_INT3_STATUS	ARM Cortex-M3 core interrupt 3 status (cleared when read)
2	CM3_INT2_STATUS	ARM Cortex-M3 core interrupt 2 status (cleared when read)
1	CM3_INT1_STATUS	ARM Cortex-M3 core interrupt 1 status (cleared when read)
0	CM3_INT0_STATUS	ARM Cortex-M3 core interrupt 0 status (cleared when read)

Field Name	Value Symbol	Value Description	Hex Value
CM3_INT3_STATUS	C_DIO_INT3_FALSE	Interrupt 3 not triggered	0x0*
	C_DIO_INT3_TRUE	Interrupt 3 triggered	0x1
CM3_INT2_STATUS	C_DIO_INT2_FALSE	Interrupt 2 not triggered	0x0*
	C_DIO_INT2_TRUE	Interrupt 2 triggered	0x1
CM3_INT1_STATUS	C_DIO_INT1_FALSE	Interrupt 1 not triggered	0x0*
	C_DIO_INT1_TRUE	Interrupt 1 triggered	0x1
CM3_INT0_STATUS	C_DIO_INT0_FALSE	Interrupt 0 not triggered	0x0*
	C_DIO_INT0_TRUE	Interrupt 0 triggered	0x1

17.3 PCM Interface

17.3.1 PCM Interface Introduction

The ARM Cortex-M3 processor has access to a highly configurable pulse code modulation (PCM) interface that can be used to stream control, configuration or signal data into and out of the microcontroller. For more information about configuring the multiplexed DIO functionality, see Section 16.1, “Digital I/O (DIO)” on page 351.

This interface makes use of four external signals in communications. These signals are:

- An input interface clock signal (C_PCM_CLK)
- A bidirectional frame signal (C_PCM_FRAME)
- A serial output data signal (C_PCM_SERO)
- A serial input data signal (C_PCM_SERI)

One of the advantages of this interface is the high communication speed that allows the interface to operate at any frequency up to and including the system clock frequency.

This interface can be configured to operate in either full-duplex mode or half-duplex mode, either transmitting or receiving, because of the separate serial data lines.

The PCM interface uses the PCM_CTRL_ENABLE bit in the C_PCM_CTRL register to enable and disable the PCM interface entirely. When using the PCM interface in any mode, enable this bit and ensure that the proper DIO multiplexing has been selected.

The PCM interface can be configured as a either a master device (controlling the frame signal, and potentially providing the interface PCM_CLK through an internally routed DIO function), or as a slave device (receiving the frame signal and usually not providing the PCM_CLK). To select between these configurations, configure the PCM_CTRL_SLAVE bit as appropriate in the C_PCM_CTRL register.

Either the ARM Cortex-M3 processor or the DMA can control the data transferred by the PCM interface. Select the controller by configuring the PCM_CTRL_CONTROLLER bit in the C_PCM_CTRL register.

IMPORTANT: The PCM interface configuration and internal status registers are tightly synchronized to the input PCM clock. As a result, the PCM interface is only reset and the internal configuration of the PCM interface is only updated on a PCM clock edge.

17.3.2 PCM Signal Configuration

The ARM Cortex-M3 processor's PCM interface uses a wide number of configuration options to help to define a set of signals that the system can interpret. These configuration options are comparable to the PCM signal configuration provided for the shared PCM/I2S interface, described in Section 12.2, "PCM Signal Configuration" on page 173.

For the PCM frame configuration, the following configurations are equivalent:

Sub-frames The PCM_CTRL_FRAME_SUBFRAMES bit from the C_PCM_CTRL register configures whether sub-frames are used. This configuration is equivalent to the PCM_CFG_SUBFRAME bit from the D_PCM_CFG register.

Length The PCM_CTRL_FRAME_LENGTH bit field from the C_PCM_CTRL register configures how many words are expected in each frame. This configuration is equivalent to the PCM_CFG_MULTIWORD_CTRL bit field from the D_PCM_CFG register.

Width The PCM_CTRL_FRAME_WIDTH bit from the C_PCM_CTRL register configures whether a short-width or long-width frame signal is used. This configuration is equivalent to the PCM_CFG_FRAME_WIDTH bit from the D_PCM_CFG register.

Alignment The PCM_CTRL_FRAME_ALIGN bit from the C_PCM_CTRL register configures whether the frame is aligned with the first bit of the frame or the last bit of the previous frame. This configuration is equivalent to the PCM_CFG_FRAME_ALIGN bit from the D_PCM_CFG register.

For PCM clock configuration, the following configuration is equivalent:

Polarity The PCM_EXT_CFG_CM3_PCM_CLK_POL bit from the D_PCM_EXT_CFG register configures whether the PCM interface registers data from the frame and data pins on the rising edge or falling edge of the PCM clock. This configuration is equivalent to the PCM_CFG_CFX_PCM_CLK_POL bit from the D_PCM_CFG register.

For PCM data configuration, the following configurations are equivalent:

Word Size The PCM_CTRL_WORD_SIZE bit field from the C_PCM_CTRL register configures the size of a word transmitted using the ARM Cortex-M3 processor's PCM interface. This configuration is equivalent to the PCM_CFG_WORD_SIZE bit field from the D_PCM_CFG register, except that the word size is limited to multiples of a byte.

Bit Order The PCM_CTRL_BIT_ORDER bit from the C_PCM_CTRL register configures whether the data is transmitted starting with the MSB or starting with the LSB. This configuration is equivalent to the PCM_CFG_BIT_ORDER bit from the D_PCM_CFG register.

TX Data The PCM_CTRL_TX_ALIGN bit from the C_PCM_CTRL register configures whether the data is loaded from the most significant portion of the data register or from the least significant portion of the data register (both configurations are equivalent when using 32-bit data words). This configuration is equivalent to the PCM_CFG_FIELD_TX_DATA bit from the D_PCM_CFG register.

The ARM Cortex-M3 processor's PCM interface can be configured for I2S in the same way that the shared PCM/ I^2S interface is. Section 12.4, "I2S Configuration and Usage" on page 179 contains further information about the required signal configuration and other interface configurations required to make this interface compatible with I^2S, including an example timing diagram.

IMPORTANT: If the PCM interface is not idle when changing any configuration settings, changing the PCM configuration during a transaction results in undefined behavior on the associated PCM signal lines.

17.3.3 PCM Interrupt Configuration

The PCM interface uses three associated interrupts that control transmission and reception of PCM data, and report errors that occurred while transmitting or receiving PCM data. See Section 15.1, "Nested Vectored Interrupt Controller (NVIC)" on page 239 for information regarding interrupt configuration and handling.

If a user application is transmitting data from the PCM interface, the PCM_TX interrupt signals:

- The start of the data value transmission in the C_PCM_TX_DATA register
- That the application can now load the next data word (of the specified size) to be transmitted

If a user application is receiving data from the PCM interface, the PCM_RX interrupt signals that a data word of the specified size was successfully received and written to the C_PCM_RX_DATA register.

The PCM transmit and receive interrupts are generated for every word of data transmitted in a valid PCM frame regardless of the length of a PCM frame.

When the DMA is used to control data transfers over the PCM interface, the INT_PCM_ERROR interrupt signals that either:

- An overrun occurred when the DMA could not read the received data from the C_PCM_RX_DATA register to the DMA buffer before this data was overwritten
- An underrun occurred when the DMA could not write the C_PCM_TX_DATA register between loads of the internal PCM transmission registers.

17.3.4 PCM Registers

Register Name	Register Description	Address
PCM_CTRL C_PCM_CTRL	PCM Control	0x40000800 P:0x3200
PCM_TX_DATA C_PCM_TX_DATA	PCM Transmit Data	0x40000804 P:0x3201
PCM_RX_DATA C_PCM_RX_DATA	PCM Receive Data	0x40000808 P:0x3202

Register Name	Register Description	Address
PCM_STATUS C_PCM_STATUS	PCM Status	0x4000080C P:0x3203

17.3.4.1 PCM_CTRL, C_PCM_CTRL Settings

Bit Field	Field Name	Description
12	BIT_ORDER	Select whether the data will be transmitted starting with the MSB or LSB
11	TX_ALIGN	Select what bits to use for transmit data
10:9	WORD_SIZE	Select the number of bits per PCM word
8	FRAME_ALIGN	Align the PCM frame signal to the first/last bit
7	FRAME_WIDTH	Use a long/short PCM frame signal
6:4	FRAME_LENGTH	Select the number of words per PCM frame
3	SUBFRAME	Select between a frame signal with every word and a frame
2	CONTROLLER	Select whether the data transfer will be controlled by the ARM Cortex-M3 processor or the DMA for PCM
1	ENABLE	Enable/disable the PCM interface
0	SLAVE	Use the PCM interface as a master/slave

Field Name	Value Symbol	Value Description	Hex Value
BIT_ORDER	C_PCM_BIT_ORDER_MSB_FIRST	PCM data is ordered from MSB to LSB	0x0*
	C_PCM_BIT_ORDER_LSB_FIRST	PCM data is ordered from LSB to MSB	0x1
TX_ALIGN	C_PCM_TX_ALIGN_MSB	Use MSBs of transmit buffer when word size is less that 32 bits	0x0*
	C_PCM_TX_ALIGN_LSB	Use LSBs of transmit buffer when word size is less that 32 bits	0x1
WORD_SIZE	C_PCM_WORD_SIZE_8	Use 8-bit words	0x0
	C_PCM_WORD_SIZE_16	Use 16-bit words	0x1
	C_PCM_WORD_SIZE_24	Use 24-bit words	0x2
	C_PCM_WORD_SIZE_32	Use 32-bit words	0x3*
FRAME_ALIGN	C_PCM_FRAME_ALIGN_LAST	Align the PCM frame signal to the last bit of the frame	0x0*
	C_PCM_FRAME_ALIGN_FIRST	Align the PCM frame signal to the first bit of the frame	0x1
FRAME_WIDTH	C_PCM_FRAME_WIDTH_SHORT	The PCM frame is high for one PCM clock period	0x0*
	C_PCM_FRAME_WIDTH_LONG	The PCM frame is high for half of the frame length	0x1

Field Name	Value Symbol	Value Description	Hex Value
FRAME_LENGTH	C_PCM_MULTIWORD_2	PCM frames contain 2 words	0x0*
	C_PCM_MULTIWORD_4	PCM frames contain 4 words	0x1
	C_PCM_MULTIWORD_6	PCM frames contain 6 words	0x2
	C_PCM_MULTIWORD_8	PCM frames contain 8 words	0x3
	C_PCM_MULTIWORD_10	PCM frames contain 10 words	0x4
	C_PCM_MULTIWORD_12	PCM frames contain 12 words	0x5
	C_PCM_MULTIWORD_14	PCM frames contain 14 words	0x6
	C_PCM_MULTIWORD_16	PCM frames contain 16 words	0x7
SUBFRAME	C_PCM_SUBFRAME_DISABLE	Generate a frame signal every frame	0x0*
	C_PCM_SUBFRAME_ENABLE	Generate a frame signal every word	0x1
CONTROLLER	C_PCM_CONTROLLER_CM3	The ARM Cortex-M3 processor controls PCM data transfers	0x0*
	C_PCM_CONTROLLER_DMA	The DMA controls PCM data transfers	0x1
ENABLE	C_PCM_DISABLE	Disable the PCM interface	0x0*
	C_PCM_ENABLE	Enable the PCM interface	0x1
SLAVE	C_PCM_SELECT_MASTER	Use the PCM interface in master mode	0x0
	C_PCM_SELECT_SLAVE	Use the PCM interface in slave mode	0x1*

17.3.4.2 PCM_TX_DATA, C_PCM_TX_DATA Settings

Bit Field	Field Name	Description
31:0	PCM_TX_DATA	Data to transmit over the PCM interface

17.3.4.3 PCM_RX_DATA, C_PCM_RX_DATA

Bit Field	Field Name	Description
31:0	PCM_RX_DATA	Data received from the PCM interface

17.3.4.4 PCM_STATUS, C_PCM_STATUS Settings

Bit Field	Field Name	Description
3	TRANSMIT_STATUS	Indicate that PCM data has been sent
2	RECEIVE_STATUS	Indicate that PCM data has been received
1	OVERRUN_STATUS	Indicate that an overrun has occurred when receiving data on the PCM interface
0	UNDERRUN_STATUS	Indicate that an underrun has occurred when transmitting data on the PCM interface

Field Name	Value Symbol	Value Description	Hex Value
TRANSMIT_STATUS	C_PCM_TRANSMIT_FALSE	PCM data transmit flag not set	0x0*
	C_PCM_TRANSMIT_TRUE	PCM transmit data sent	0x1
	C_PCM_TRANSMIT_CLEAR	Clear the PCM transmit status bit	0x1
RECEIVE_STATUS	C_PCM_RECEIVE_FALSE	PCM data receive flag not set	0x0*
	C_PCM_RECEIVE_TRUE	PCM data received	0x1
	C_PCM_RECEIVE_CLEAR	Clear the PCM receive status bit	0x1
OVERRUN_STATUS	C_PCM_OVERRUN_FALSE	No PCM input overrun detected	0x0*
	C_PCM_OVERRUN_TRUE	PCM input overrun detected	0x1
	C_PCM_OVERRUN_CLEAR	Clear the PCM overrun bit	0x1
UNDERRUN_STATUS	C_PCM_UNDERRUN_FALSE	No PCM input underrun detected	0x0*
	C_PCM_UNDERRUN_TRUE	PCM input underrun detected	0x1
	C_PCM_UNDERRUN_CLEAR	Clear the PCM underrun bit	0x1

17.3.5 I²S Configuration and Usage

The I²S protocol is designed to allow a device to share data digitally between devices. To use the PCM interface in the Ezairo 7100 microcontroller for I²S communications, the required signal and data management settings for the PCM interface are listed in Table 71. As a side effect of the configuration requiring that the device operate in PCM slave mode, the frame signal must be used as an input for I²S communications.

Table 71. Required Configuration Settings for I²S Configuration

Configuration Bit or Bit Field	Setting
PCM_WORD_SIZE	PCM_WORD_SIZE_16
PCM_MULTIWORD_CTRL	PCM_MULTIWORD_2
PCM_FRAME_WIDTH	PCM_FRAME_WIDTH_LONG
PCM_FRAME_ALIGN	PCM_FRAME_ALIGN_LAST_BIT
PCM_BIT_ORDER	PCM_BIT_ORDER_MSB_FIRST
PCM_SUBFRAME	PCM_SUBFRAME_DISABLE
PCM_SLAVE	PCM_SLAVE
CM3_PCM_CLK_POL	CM3_PCM_SAMPLE_RISING_EDGE[1]

1. The PCM interface produces a data stream that complies with the I²S standard with either sampling edge. However, to comply completely with the standard, the interface must be configured to sample data on the rising edge of the clock.

When operating in this configuration, the behavior of the PCM interface is slightly different than in any other configuration. As shown in the I²S signal timing diagram (Figure 56) the microcontroller allows bit 15 to be repeatedly transmitted until a rising or falling edge on the frame signal is detected. When configured for slave mode with a word size of 16 and a long frame width, the PCM interface synchronizes communications with both the rising edge and falling edge of the frame signal (instead of only synchronizing with the rising edge as it does in all other configurations). In this way, the interface is compatible with I²S signals that use a width greater than 16 bits of data.

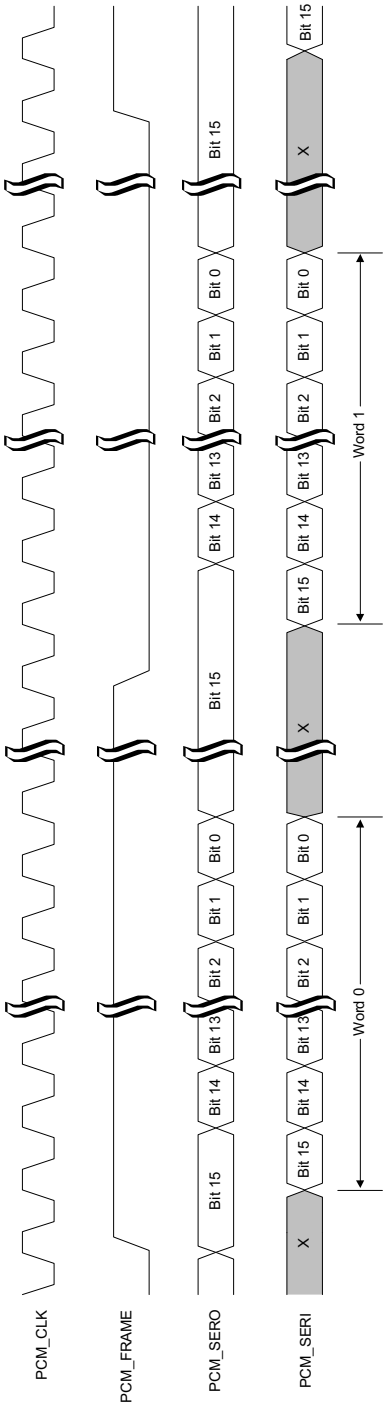


Figure 56. Signal Timing Diagram: I²S Configuration

This I^2S implementation is compatible with PCM interface implementations for data that is exactly 16 bits in length and where the frame signal has a duty cycle of exactly 50%.

17.4 SPI Interface

The ARM Cortex-M3 processor external interface includes a Serial Peripheral Interface (SPI) that allows the system to communicate with external components such as wireless radios, display controllers and non-volatile memories (NVM).

17.4.1 SPI Interface Configuration

The SPI interface can be enabled or disabled using the SPI_CTRL0_ENABLE bit in the C_SPI_CTRL0 register. For more information about configuring the multiplexed DIO functionality, see Section 16.1, “Digital I/O (DIO)” on page 351.

Data transfers using the SPI interfaces can be controlled directly by the ARM Cortex-M3 processor or indirectly using the DMA. To select the controller for the SPI interface, use the SPI_CTRL0_CONTROLLER bit in the C_SPI_CTRL0 register.

17.4.2 SPI Interface Data Transfers

The SPI interfaces can be configured to operate as an SPI master device or an SPI slave device by configuring the SPI_CTRL0_SLAVE bit in the C_SPI_CTRL0 register. The differences in the SPI pad configurations between master and slave mode are as follows:

C_SPI_CLK

In master mode, the C_SPI_CLK signal is supplied by the Ezairo 7100 microcontroller using SYSCLK prescaled by a power of 2, which is configured using the SPI_CTRL0_PRESCALE bit field from the C_SPI_CTRL0 register.

In slave mode, the C_SPI_CLK signal is sourced from a remote SPI master device. For proper operation, the frequency of the SPI_CLK inputs must abide by the following relation:

$$\mathrm{FREQ}_{\mathrm{SYSCLK}} \geq 4 \times \mathrm{FREQ}_{\mathrm{SPI_CLK}}$$

For both master and slave mode, the SPI_CTRL0_CLK_POLARITY bit in the C_SPI_CTRL0 register is used to control when data changes and is sampled. An SPI interface using normal polarity will update output signals on the rising edge of C_SPI_CLK, and sample input signals on the falling edge of C_SPI_CLK. If the polarity is inverted, output signals change on the falling edge of C_SPI_CLK and input signals are sampled on the rising edge.

C_SPI_CS

In master mode, the C_SPI_CS pad is an output controlled by the SPI_CTRL1_CS bit from the C_SPI_CTRL1 register. The signal from this pad is generally routed to the chip select input of a slave device.

In slave mode, the C_SPI_CS pad is an input sourced from a remote SPI master device.

NOTE: For an SPI device, the chip select input is generally interpreted as an active-low signal. As such, if the signal on the C_SPI_CS pad is high, the SPI slave will ignore all communications using the interface.

C_SPI_SERI This pad is a serial input signal that is used to receive data when the following conditions are met:

- C_SPI_CLK and C_SPI_CS indicate that data should be transferred.
- The SPI_CTRL1_RW_CMD bit field from the C_SPI_CTRL1 register is configured for an SPI read or read-write (full-duplex) transfer.

C_SPI_SERO This pad is a serial output signal that is used to transmit data when the following conditions are met:

- C_SPI_CLK and C_SPI_CS indicate that data should be transferred.
- The SPI_CTRL1_RW_CMD bit field from the C_SPI_CTRL1 register is configured for an SPI write or read-write (full-duplex) transfer.

The SPI interfaces support a configurable word size for each transfer of between 1 and 32 bits of data per word, as configured using the SPI_CTRL1_WORD_SIZE bit field in the C_SPI_CTRL1 register. All data transactions using an SPI interface start with the MSB of the word received and transfer (SPI_CTRL1_WORD_SIZE + 1) bits of data per word.

The SPI_CTRL1_START_BUSY bit in the C_SPI_CTRL1 register indicates if the SPI interface is currently transferring data. When operating in master mode, this same bit can be used to start an SPI transfer.

NOTE: When the interface is disabled, any data currently being transmitted is allowed to complete before the interface shuts down. If the interface is busy, the SPI_CTRL* registers cannot be modified until the interface is idle.

If the SPI_CTRL0_MODE_SELECT bit in the C_SPI_CTRL0 register is set, the SPI interface operates in auto mode to limit the overhead between SPI transfers. This mode works in conjunction with the SPI interface data registers. These registers are:

C_SPI_TX_DATA Shift register containing the data to transmit using the SPI interface.

C_SPI_RX_DATA Shift register containing the most recent data word received from the SPI interface.

When operating as an SPI master in auto mode, these registers can be used to efficiently continue a transfer. In this configuration, if writing to the SPI interface, writing the C_SPI_TX_DATA register initiates another write transfer. Similarly, if reading from the SPI interface, reading the C_SPI_RX_DATA register initiates another read transfer.

CAUTION: The SPI_CTRL1_START_BUSY bit is cleared for one cycle between transfers in auto mode. If polling this bit to determine when a transfer completes, ensure that at least one cycle has elapsed after starting the transfer before polling for the completion of a transfer.

In manual mode or when operating as an SPI slave, reading or writing either of these registers has no side effects.

IMPORTANT: When transmitting data as an SPI slave, the SPI interface must load the C_SPI_TX_DATA register between SPI_CLK edges that update the SPI output signals. If no delay occurs between the words of the SPI transfer, new data must be loaded in one SPI_CLK cycle.

17.4.3 SPI Interrupts

The SPI interfaces use three interrupts that signal the completion of a transfer over the SPI interface. See Section 15.1, “Nested Vectored Interrupt Controller (NVIC)” on page 239 for information regarding interrupt configuration and handling.

If a user application is transmitting data from the SPI interface, the SPI_TX interrupt signals:

- The start of the data value transmission in the C_SPI_TX_DATA register
- That the application can now load the next data word (of the specified size) to be transmitted

If a user application is receiving data from the SPI interface, the SPI_RX interrupt signals that a data word of the specified size was successfully received and written to the C_SPI_RX_DATA register.

When transferring a sequence of data using an SPI interface, after an interrupt. the next transfer can be initiated by using either the SPI_DATA register (auto mode only) or the SPI_CTRL1_START_BUSY bit of the C_SPI_CTRL1 register. For more information, see Section 17.4.2, “SPI Interface Data Transfers”.

If using the DMA to control data transfers over an SPI interface, use the SPI_ERROR interrupt to indicate when an error has occurred.

- An overrun occurs when the DMA cannot read the received data from the SPI_DATA register to the DMA buffer before this data is overwritten. To monitor for overrun events, set the SPI_CTRL0_OVERRUN_INT_ENABLE bit in the C_SPI_CTRL0 register. If an overrun occurs and this event monitor is enabled, the SPI_STATUS_OVERRUN_STATUS bit in the appropriate C_SPI_STATUS register will be set.
- An underrun occurs when the DMA cannot write the SPI_DATA register before a second data transfer started using this data. To monitor for underrun events, set the SPI_CTRL0_UNDERRUN_INT_ENABLE bit in the C_SPI_CTRL0 register. If an underrun occurs and this event monitor is enabled, the SPI_STATUS_UNDERRUN_STATUS bit in the appropriate C_SPI_STATUS register will be set.

17.4.4 SPI Registers

Register Name	Register Description	Address
SPI_CTRL0 C_SPI_CTRL0	SPI Control and Configuration Register	0x40000700 P:0x31C0
SPI_CTRL1 C_SPI_CTRL1	SPI Transaction Control Register	0x40000704 P:0x31C1
SPI_TX_DATA C_SPI_TX_DATA	SPI Transmit Data	0x40000708 P:0x31C2
SPI_RX_DATA C_SPI_RX_DATA	SPI Received Data	0x4000070C P:0x31C3
SPI_STATUS C_SPI_STATUS	SPI Status	0x40000710 P:0x31C4

17.4.4.1 SPI_CTRL0, C_SPI_CTRL0 Settings

Bit Field	Field Name	Description
10	OVERRUN_INT_ENABLE	Enable/disable SPI overrun interrupts
9	UNDERRUN_INT_ENABLE	Enable/disable SPI underrun interrupts
8	CONTROLLER	Select whether the data transfer will be controlled by the ARM Cortex-M3 processor or the DMA for SPI
7	SLAVE	Use the SPI interface as master or slave
5	CLK_POLARITY	Select the polarity of the SPI clock
4	MODE_SELECT	Select between manual and auto transaction handling modes for SPI master transactions
3	ENABLE	Enable/disable the SPI interface
2:0	PRESCALE	Prescale the SPI interface clock for master transfers

Field Name	Value Symbol	Value Description	Hex Value
OVERRUN_INT_ENABLE	C_SPI_OVERRUN_INT_DISABLE	No interrupts are raised when an overrun occurs on the SPI interface	0x0*
	C_SPI_OVERRUN_INT_ENABLE	An interrupt is raised when an overrun occurs on the SPI interface	0x1
UNDERRUN_INT_ENABLE	C_SPI_UNDERRUN_INT_DISABLE	No interrupts are raised when an underrun occurs on the SPI interface	0x0*
	C_SPI_UNDERRUN_INT_ENABLE	An interrupt is raised when an underrun occurs on the SPI interface	0x1
CONTROLLER	C_SPI_CONTROLLER_CM3	The ARM Cortex-M3 processor controls data transfers using SPI	0x0*
	C_SPI_CONTROLLER_DMA	The DMA controls data transfers using SPI	0x1
SLAVE	C_SPI_SELECT_MASTER	Use the SPI interface in master mode	0x0*
	C_SPI_SELECT_SLAVE	Use the SPI interface in slave mode	0x1
CLK_POLARITY	C_SPI_CLK_POLARITY_NORMAL	SERO changes on the rising edge of SPI clock; SERI changes on the falling edge of SPI clock	0x0*
	C_SPI_CLK_POLARITY_INVERSE	SERO changes on the falling edge of SPI clock; SERI changes on the rising edge of SPI clock	0x1
MODE_SELECT	C_SPI_MODE_SELECT_MANUAL	Master transfers using the SPI interface do not automatically continue	0x0*
	C_SPI_MODE_SELECT_AUTO	Automatically continue master transfers using the SPI interface	0x1
ENABLE	C_SPI_DISABLE	Disable the SPI interface	0x0*
	C_SPI_ENABLE	Enable the SPI interface	0x1

Field Name	Value Symbol	Value Description	Hex Value
PRESCALE	C_SPI_PRESCALE_2	Prescale the SPI interface clock by 2	0x0*
	C_SPI_PRESCALE_4	Prescale the SPI interface clock by 4	0x1
	C_SPI_PRESCALE_8	Prescale the SPI interface clock by 8	0x2
	C_SPI_PRESCALE_16	Prescale the SPI interface clock by 16	0x3
	C_SPI_PRESCALE_32	Prescale the SPI interface clock by 32	0x4
	C_SPI_PRESCALE_64	Prescale the SPI interface clock by 64	0x5
	C_SPI_PRESCALE_128	Prescale the SPI interface clock by 128	0x6
	C_SPI_PRESCALE_256	Prescale the SPI interface clock by 256	0x7

17.4.4.2 SPI_CTRL1, C_SPI_CTRL1 Settings

Bit Field	Field Name	Description
8	START_BUSY	Start an SPI data transfer and indicate if a transfer is in progress
7:6	RW_CMD	Issue a read command or write command to the SPI interface
5	CS	Set the chip-select line for SPI (master mode); read the chip-select line for SPI (slave mode)
4:0	WORD_SIZE	Select the word size used by the SPI interface (word size = SPI_CTRL1_WORD_SIZE + 1)

Field Name	Value Symbol	Value Description	Hex Value
START_BUSY	C_SPI_IDLE	Stop a transfer or indicate that the SPI interface is idle	0x0*
	C_SPI_START	Start a transfer on the SPI interface (master mode only)	0x1
	C_SPI_BUSY	Indicate that the SPI interface is currently transferring data	0x1
RW_CMD	C_SPI_NOP	No operation	0x0*
	C_SPI_WRITE_DATA	Write data using the SPI interface	0x1
	C_SPI_READ_DATA	Read data using the SPI interface	0x2
	C_SPI_RW_DATA	Read and write data using the SPI interface	0x3
CS	C_SPI_CS_0	Set the SPI CS signal low	0x0
	C_SPI_CS_1	Set the SPI CS signal high	0x1*
WORD_SIZE	C_SPI_WORD_SIZE_1	SPI transfers use 1-bit words	0x0*
	C_SPI_WORD_SIZE_4	SPI transfers use 4-bit words	0x3
	C_SPI_WORD_SIZE_8	SPI transfers use 8-bit words	0x7
	C_SPI_WORD_SIZE_16	SPI transfers use 16-bit words	0xF
	C_SPI_WORD_SIZE_24	SPI transfers use 24-bit words	0x17
	C_SPI_WORD_SIZE_32	SPI transfers use 32-bit words	0x1F

17.4.4.3 SPI_TX_DATA, C_SPI_TX_DATA Settings

Bit Field	Field Name	Description
31:0	SPI_DATA_OUT	Single word buffer for data to be transmitted over the SPI interface

17.4.4.4 SPI_RX_DATA, C_SPI_RX_DATA Settings

Bit Field	Field Name	Description
31:0	SPI_DATA_IN	Single word buffer for data that has been received over the SPI interface

17.4.4.5 SPI_STATUS, C_SPI_STATUS Settings

Bit Field	Field Name	Description
3	TRANSMIT_STATUS	Indicate that the transmission of the data is completed
2	RECEIVE_STATUS	Indicate that new data has been received
1	OVERRUN_STATUS	Indicate that an overrun has occurred when receiving data on the SPI interface
0	UNDERRUN_STATUS	Indicate that an underrun has occurred when transmitting data on the SPI interface

Field Name	Value Symbol	Value Description	Hex Value
TRANSMIT_STATUS	C_SPI_TRANSMIT_FALSE	SPI data transmit flag not set	0x0*
	C_SPI_TRANSMIT_TRUE	SPI transmit data sent	0x1
	C_SPI_TRANSMIT_CLEAR	Clear the SPI transmit status bit	0x1
RECEIVE_STATUS	C_SPI_RECEIVE_FALSE	SPI data receive flag not set	0x0*
	C_SPI_RECEIVE_TRUE	SPI data received	0x1
	C_SPI_RECEIVE_CLEAR	Clear the SPI receive status bit	0x1
OVERRUN_STATUS	C_SPI_OVERRUN_FALSE	No SPI input overrun detected	0x0*
	C_SPI_OVERRUN_TRUE	SPI input overrun detected	0x1
	C_SPI_OVERRUN_CLEAR	Clear the SPI overrun bit	0x1
UNDERRUN_STATUS	C_SPI_UNDERRUN_FALSE	No SPI input underrun detected	0x0*
	C_SPI_UNDERRUN_TRUE	SPI input underrun detected	0x1
	C_SPI_UNDERRUN_CLEAR	Clear the SPI underrun bit	0x1

CHAPTER 18
Clock Components

18.1 Clock Generation

All clocks and clock domains in the Ezairo 7100 system are derived from the system clock (SYSCLK). Because of its importance to the functioning of Ezairo 7100 and the execution of applications, this clock signal can be generated in multiple ways for maximum flexibility. Choose one of the following sources:

- The internal VCO-based primary oscillator (discussed in Section 18.1.1, “Primary Oscillator”)
- The internal standby oscillator (discussed in Section 18.1.2, “Standby Oscillator” on page 440)
- The external clock pad (discussed in Section 18.2.6, “External Clock (EXTCLK)” on page 448)

18.1.1 Primary Oscillator

The primary oscillator on Ezairo 7100 is a phase-locked loop (PLL) that can be configured to provide a clock signal for use as the system clock (`SYSCLK`) within the Ezairo 7100 system. This oscillator can operate in two modes:

- An oscillator only mode (VCO) discussed in Section 18.1.1.1, “Voltage-Controlled Oscillator” on page 433
- A PLL mode that uses feedback control to synchronize to a reference clock

If another clock source is selected to clock the system, this oscillator can be disabled by clearing the `OSC_CTRL_1_ENABLE` bit field from the `A_OSC_CTRL_1` control register.

CAUTION: The primary oscillator must not be disabled while it is configured as the source for `SYSCLK` because this will place the Ezairo 7100 system into an unclocked state. For more information about configuring the source for SYSCLK, see Section 18.2.1, “System Clock (SYSCLK)”.

The primary oscillator includes a voltage controlled oscillator, a phase detector and loop filter, and a number of detection and clock monitoring circuits. Figure 57 provides a top-level overview of the underlying blocks that form the PLL.

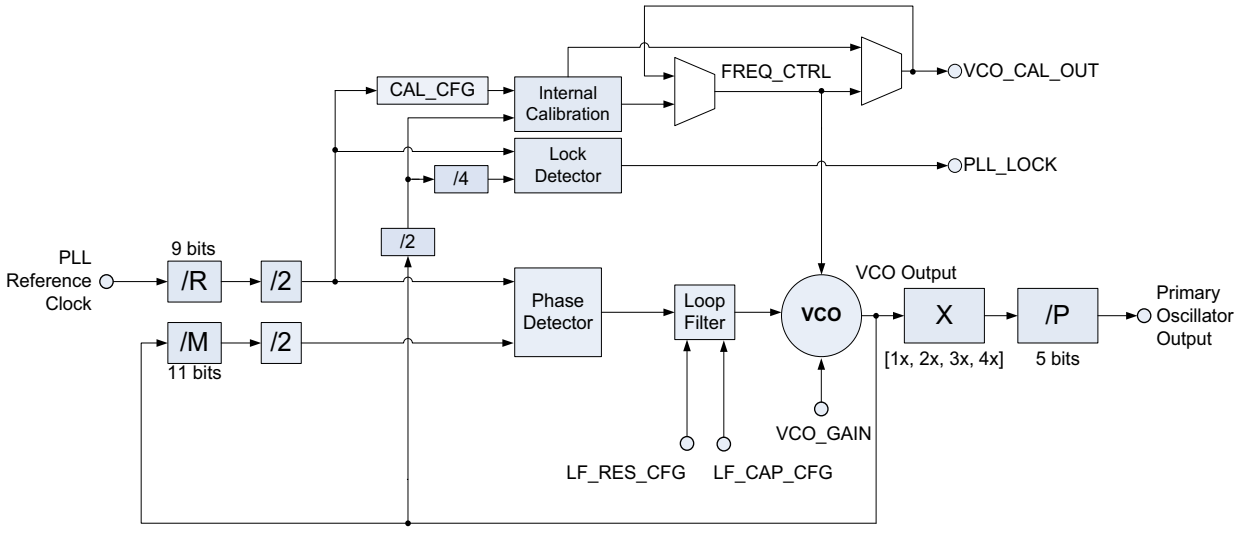


Figure 57. Block Diagram of the Primary Oscillator

To guarantee a well-behaved startup, the oscillator defaults to oscillator-only operation (without feedback control) as described in Section 18.1.1.1, “Voltage-Controlled Oscillator”. If the system is required to synchronize to an external reference clock, the oscillator can be configured to operate in PLL mode where the phase of the oscillator’s output clock is synchronized with the phase of the reference clock. This mode is described in Section 18.1.1.2, “Phase-Locked Loop”.

To select between these modes, set or clear the `PLL_CTRL_0_PLL_EN` bit from the `A_PLL_CTRL_0` register. In oscillator-only mode, the output frequency is defined completely by the VCO configuration. In PLL mode, the oscillator calibrates its output using the PLL configuration and reference clock to perform feedback control of the signal.

18.1.1.1 Voltage-Controlled Oscillator

Using the primary oscillator in oscillator-only mode is sufficient for most applications; as a result, it is the default and most commonly used source for clocking an Ezairo 7100 chip. In this mode, only the voltage-controlled oscillator (VCO) portion of the primary oscillator is enabled.

The base frequency of the primary oscillator is set using the `OSC_CTRL_0_FREQ_CTRL` bit field from the `A_OSC_CTRL_0` register. This base frequency is multiplied by 1, 2, 3 or 4 using the `OSC_CTRL_1_MULT` bit field from the `A_OSC_CTRL_1` register.

NOTE: The output frequency of the VCO is non-monotonic across transitions in the `OSC_CTRL_0_FREQ_CTRL` bit field where the eight LSBs switch from 0xFF to 0x00. At these

transitions, the output frequency with LSBs set to 0x00 are always lower than the output frequency set when the LSBs are 0xFF.

IMPORTANT: When multiplying the base frequency of the primary oscillator by three using the OSC_CTRL_1_MULT bit field from the A_OSC_CTRL_1 register, the resulting clock output does not have a 50% duty cycle. For all other multiplier values (including multiplication by 1), the clock output has an approximately 50% duty cycle.

If SYSCLK uses a multiplier of three, the non-50% duty cycle nature of SYSCLK means that the effective frequency must be treated as $(4/3) \times f_{SYSCLK}$. This effective frequency is subject to the limitations on the maximum SYSCLK frequency, and shall be used when determining the VDDC supply voltage required.

If any clock derived from SYSCLK must have a 50% duty cycle and SYSCLK uses a multiplier of three, those clocks must be divided by a multiple of three.

The output frequency of the VCO can be divided by up to 32 using the VCO post divisor. This divisor is set using the PLL_CTRL_2_POST_DIVIDER bit field from A_PLL_CTRL_2 register.

The frequency of the primary oscillator can be dynamically throttled to accommodate an application's changing processing needs in an efficient way, without introducing audio artifacts. The primary oscillator's output frequency can be throttled using:

- Multiplier-based clock throttling that adjusts the multiplier applied to the VCO output without affecting the clocks used by the analog input and output components (adjusts the OSC_CTRL_1_MULT bit field from A_OSC_CTRL_1)
- Frequency-based clock throttling that adjusts the base frequency of the VCO output; simultaneously, it adjusts the clocks used by the analog input and output components to accommodate this change (i.e., adjusts the OSC_CTRL_0_FREQ_CTRL bit field from the A_OSC_CTRL_0 register)

For more information about throttling the primary oscillator frequency using these features, see Section 18.3, "Clock Throttling".

18.1.1.1.1 VCO Temperature Compensation

The VCO supports temperature compensation for better frequency stability across the operating temperatures supported by the Ezairo 7100 device. By default, the VCO provides a frequency change of less than ±2% across the entire operating temperature range. To support a tighter frequency range, enable temperature compensation by setting the OSC_CTRL_2_TEMP_CTRL_EBL bit from the D_OSC_CTRL_2 register.

Whenever the VCO frequency is updated using the OSC_CTRL_0_FREQ_CTRL bit field, update the VCO temperature compensation by:

1. Writing the VCO frequency configuration to OSC_CTRL_0_FREQ_CTRL
2. Reading the nominal calibration settings (OSC_CTRL_3_POS_TEMP_OUT and OSC_CTRL_3_NEG_TEMP_OUT) from the A_OSC_CTRL_3 register
3. Incrementing the value read from OSC_CTRL_3_POS_TEMP_OUT and decrementing the value read from OSC_CTRL_3_NEG_TEMP_OUT if the VCO frequency decreases as the temperature increases. Decrementing the value read from OSC_CTRL_3_POS_TEMP_OUT and incrementing the value read from OSC_CTRL_3_NEG_TEMP_OUT if the VCO frequency increases as the temperature decreases.

4. Writing the updated OSC_CTRL_3_POS_TEMP_OUT value to the OSC_CTRL_2_POS_TEMP_CTRL bit field, and the updated OSC_CTRL_3_NEG_TEMP_OUT value to the OSC_CTRL_2_NEG_TEMP_CTRL bit field. Both bit fields are from the A_OSC_CTRL_2 register.

18.1.1.2 Phase-Locked Loop

The phase-locked loop (PLL) can be used to synchronize the system clock (and all of the clocks derived from SYSCLK) with an external reference clock. Before configuring the PLL, configure the oscillator for free-run mode using PLL_CTRL_0_FREE_RUN bit from the A_PLL_CTRL_0 register. In this mode, the output frequency is defined by the VCO only (as described in Section 18.1.1.1, “Voltage-Controlled Oscillator”). When the PLL is locked, the frequency of the PLL is defined by:

Reference Clock Frequency
: The frequency of the reference clock input

Reference Clock Divisor (R)
: Division factor used to divide the input reference clock to the PLL phase detector and loop filter. The reference clock divisor is set using the PLL_CTRL_1_REF_DIVIDER bit field from A_PLL_CTRL_1 register.

VCO Clock Divisor (M)
: Division factor used to divide the VCO output clock before it is provided at the input to the PLL phase detector and loop filter. The VCO clock divisor is set using the PLL_CTRL_1_VCO_DIVIDER bit field from A_PLL_CTRL_1 register.

VCO Multiplier Setting (X)
: Multiplier for the VCO output clock. Set using the OSC_CTRL_1_MULT bit field from the A_OSC_CTRL_1 register, as discussed in Section 18.1.1.1, “Voltage-Controlled Oscillator”.

VCO Post Divisor (P)
: Division factor used to divide the final output from the primary oscillator (after multiplication of the VCO output). The post divisor is set using the PLL_CTRL_2_POST_DIVIDER bit field from A_PLL_CTRL_2 register.

The output frequency of the PLL is defined as:

$$f_{PLL} = f_{REF} \times \left(\frac{(M + 1) \cdot (X + 1)}{(R + 1) \cdot (P + 1)} \right)$$

For example, the following configuration produces a 5.12 MHz output clock:

- The reference clock is a 512 kHz clock
- The reference clock is divided by 1 (R set to 0)
- The VCO output is divided by 5 before being supplied to the PLL (M is set to 4)
- The VCO output is multiplied by 2 (X is set to 1)
- The VCO output is divided by 1 (P is set to 0)

18.1.1.2.1 PLL Configuration

To use the PLL to synchronize the internal clock to a reference clock, a user application should:

1. Enable the VCO (set the OSC_CTRL_1_ENABLE bit in the A_OSC_CTRL_1 register)
2. Enable the PLL in free running mode using internal calibration (set the PLL_CTRL_0_PLL_EN and PLL_CTRL_0_FREE_RUN bits, and clear the PLL_CTRL_0_EXT_CAL_EN bit in the A_PLL_CTRL_0 register)
3. Configure any PLL performance parameters required for the application's use case (as described in Section 18.1.1.2.4, "PLL Performance Configuration" on page 437).
4. Set the reference clock divisor (R), VCO clock divisor (M).
5. Set the VCO post divisor (P).
6. Clear the PLL free-run bit to indicate that the PLL should attempt to lock to the reference signal.
7. Wait for the PLL to lock; a lock can be detected by either polling the PLL_CTRL_7_PLL_LOCK bit in the A_PLL_CTRL_7 register or using the PLL lock detection interrupt feature (see below).
8. Set the VCO output multiplier (X).

18.1.1.2.2 PLL Lock Monitoring

The lock status of the PLL is monitored using a lock detection circuit. This circuit checks the PLL alignment of the VCO with the reference clock every eight VCO output clock cycles. Compare it with the divided PLL reference clock to ensure at least nine bits of accuracy. This circuit is configured using the A_PLL_CTRL_7 register, which includes the following:

- PLL_CTRL_7_PLL_LOCK is a status bit that indicates if the PLL is currently locked.
- PLL_CTRL_7_LOCK_DET_FLAG is a status bit indicating if the PLL has been locked since this flag was last cleared. To clear this flag, set the PLL_CTRL_7_LOCK_DET_FLAG_CLEAR bit.
- PLL_CTRL_7_UNLOCK_DET_FLAG is a status bit indicating if the PLL has been unlocked since this flag was last cleared. To clear this flag, set the PLL_CTRL_7_UNLOCK_DET_FLAG_CLEAR bit.
- PLL_CTRL_7_CAL_CNT_ERROR is a status bit that indicates if an error occurred with the PLL's digital calibration counter. For more information, see Section 18.1.1.2.4, "PLL Performance Configuration".

The PLL lock detector can trigger a clock interrupt (D_VECT_CLK). The condition or conditions detected by the lock detector that are used to trigger an interrupt are set using the PLL_CTRL_7_PLL_LOCK_INT_SEL bit field from the A_PLL_CTRL_7 register. Options include:

- No PLL lock detection interrupts
- The PLL has locked
- The PLL has unlocked
- The PLL state has changed from unlocked to locked, or locked to unlocked

Because the clock interrupt has multiple possible sources (including PLL lock status, PLL re-synchronization and the PLL reference clock status), an application might have to determine the source of a clock interrupt. If a clock interrupt is triggered, the PLL_CTRL_7_PLL_LOCK_INT_STATUS bit from the D_PLL_REF_CLK_DET_CTRL register indicates if this interrupt was triggered by the lock detection circuit. Other status bits can be used to poll the current locked state or to query for any transitions.

IMPORTANT: It can be challenging to select the optimal divisors and PLL performance parameters for your system. To simplify this, we provide the SignaKlara™ Development Utility (SKDU), which supports the selection of the optimal configuration of the PLL. For more information, contact ON Semiconductor Customer Support.

18.1.1.2.3 PLL Reference Clock Configuration

The PLL sources its reference clock from either the EXTCLK input or a DIO, as configured in the DIO block. For more information, see Section 16.1, "Digital I/O (DIO)" on page 351.

The PLL supports clock detection for the reference clock source. To enable clock detection for the reference clock, set the PLL_REF_CLK_DET_CTRL_CLK_DET_ENABLE bit from the D_PLL_REF_CLK_DET_CTRL register. The clock detect circuit verifies that the reference clock has been both low and high within each period. This verification period is defined by dividing SLOWCLK by the PLL_REF_CLK_DET_CTRL_CLK_DET_DIV bit field of the D_PLL_REF_CLK_DET_CTRL register. When enabled, the clock detection circuit provides the PLL_REF_CLK_DET_CTRL_CLK_DET_STATUS bit of the D_PLL_REF_CLK_DET_CTRL register as a status bit indicating if a reference clock (high and low states) was observed since the reference clock state was last polled.

The PLL reference clock detector can trigger a clock interrupt (D_VECT_CLK). The condition or conditions detected by the clock detector that are used to trigger an interrupt are set using the PLL_REF_CLK_DET_CTRL_CLK_DET_INT_SEL bit field from the D_PLL_REF_CLK_DET_CTRL register. Options include:

- No clock detector interrupts
- A clock signal was activated
- A clock signal was deactivated
- The status of the clock changed from active to inactive, or inactive to active

If a clock interrupt is triggered, the PLL_REF_CLK_DET_CTRL_CLK_DET_INT_STATUS bit from the D_PLL_REF_CLK_DET_CTRL register indicates if this interrupt was triggered by the reference clock detection circuit.

18.1.1.2.4 PLL Performance Configuration

The PLL supports both internal and external VCO calibration. Select between these modes by setting the PLL_CTRL_0_EXT_CAL_EN bit from the A_PLL_CTRL_0 register:

- When the PLL is operating in external calibration mode, the VCO frequency setting is provided to the PLL through the PLL_CTRL_6_VCO_CAL_OUT bit field from the A_PLL_CTRL_6 register.
- When the PLL is operating in internal calibration mode, the VCO frequency setting is provided by the PLL through the PLL_CTRL_6_VCO_CAL_OUT bit field from the A_PLL_CTRL_6 register.

The PLL uses this calibration setting in place of the OSC_CTRL_0_FREQ_CTRL bit field.

NOTE: When the PLL is using external calibration, the PLL lock detection circuit and reference clock monitors can still be used to monitor for error states in the PLL.

The internal PLL calibration block uses a 16-bit counter when calibrating the VCO. This counter is incremented by the VCO output clock (divided by 2). The output of the counter is checked after a configurable number of divided reference clock cycles (configured as a power of 2 between 1 and 32, using the PLL_CTRL_3_CAL_CFG bit field of the A_PLL_CTRL_3 register).

The time required to calibrate the VCO output is directly proportional to the number of clock configuration bits that the internal calibration routine needs to calibrate. The time required to calibrate the PLL, if calibrating *b* bits is:

$$t_{CAL} = R \times \frac{1}{f_{REF}} \times 2^{PLL_CTRL_3_CAL_CFG} \times (b + 3)$$

In general, it is safe to skip calibrating the MSB—clearing the MSB to zero—as configured by using the PLL_CTRL_3_CAL_MSB_SKIP bit from the A_PLL_CTRL_3 register. The number of bits to calibrate can further be restricted by using the PLL_CTRL_3_CAL_NBITS_CFG bit field from the A_PLL_CTRL_3 register. This bit field defines

how many bits are calibrated (including the MSB), and should only be used to restrict the number of bits if the accuracy of the PLL output does not require particular fine tuning.

NOTE: If the calibration counter saturates because too many divided VCO output clock cycles have occurred since it was last cleared, a PLL_CTRL_7_CAL_CNT_ERROR error is triggered.

To eliminate clock jitter at the output of the primary oscillator as much as possible, the PLL provides a configurable VCO gain (which should be minimized), and configurable loop filter parameters (which should be maximized).

Higher VCO gains allow the VCO to track the reference clock well across temperature and some fluctuation in the input PLL reference clock frequency. To configure the VCO gain, set the PLL_CTRL_4_VCO_GAIN bit field from the D_PLL_CTRL_4 register. For optimal PLL performance, the VCO gain should be set as indicated in Table 72.

Table 72. VCO Gain Settings

Maximum VCO Frequency (MHz)	K_o (MHz/V)	VCO Gain Setting
2.56	0.2	0x00
5.12	0.6	0x01
10.24	1.2	0x03
15.36	1.7	0x05

The loop filters provide filtering for any VCO output clock jitter. Reducing this jitter in the feedback path to the VCO reduces the jitter in the output. The PLL_CTRL_4_LF_RES_CFG bit field and the PLL_CTRL_4_LF_CAP_CFG bit field from the D_PLL_CTRL_4 register define the resistance and capacitance used in this filter. For optimal PLL performance, the loop filter parameters should be set based on the VCO clock divisor setting and the K_o values from Table 72. The recommended parameter settings are listed in Table 73.

Table 73. PLL Loop Filter Configuration Settings

M/K_o	PLL_CTRL_4_LF_RES_CFG	PLL_CTRL_4_LF_CAP_CFG
5	0	0
6	0	2
7	0	6
8	0	10
9	0	14
10	0	17
11	0	21
12	0	25
13	0	28
16	0	31
20	1	31
23	3	31
x, where x = 26, 36, 46, ...	$\frac{x-16}{10} \times 3$	31

Table 73. PLL Loop Filter Configuration Settings

M/K_o	PLL_CTRL_4_LF_RES_CFG	PLL_CTRL_4_LF_CAP_CFG
x, where x = 30, 40, 50, ...	$\left(\frac{x-20}{10}\times 3\right)+1$	31
x, where x = 33, 43, 53, ...	$\left(\frac{x-23}{10}\times 3\right)+3$	31

The A_PLL_CTRL_5 register configures the temperature compensation factor used for the VCO. Its default value matches the ideal temperature compensation configuration, and should not be changed.

18.1.1.2.5 PLL Reference and SDMCLK Synchronization

The Ezairo 7100 system supports synchronization of the PLL with the sigma-delta modulator clock (SDMCLK) used to control the output signals to the digital drive and DMIC outputs. This is required for some systems using the PLL with a reference clock from an external DMIC source. Synchronization is achieved by aligning the rising edges of both the PLL reference and SDMCLK to align DMIC output properly. For information about configuring the SDMCLK, see Section 18.2.3, “Sigma-Delta Modulator Clock (SDMCLK)”. For information about configuring the DMIC outputs, see Section 11.5, “Digital Microphone Output” on page 170.

IMPORTANT: The PLL and SDMCLK synchronizer can only synchronize the clock sources if SDMCLK is divided by at least four from SYSCLK (i.e., the O_SDMCLK_PRESCALE bit field is set to 3 or higher).

To enable PLL and SDMCLK clock synchronization, set the CLK_SYNC_CLK_SYNC_ENABLE bit from the D_CLK_SYNC register. When the synchronizer is enabled, the PLL reference clock and SDMCLK are synchronized as soon as possible. The synchronization has a nominal precision of ±0.5 SYSCLK cycles. If SDMCLK is divided by at least 6 from SYSCLK, the synchronization tolerance is configurable between ±1.5 SYSCLK cycles or ±2.5 SYSCLK cycles using the CLK_SYNC_CLK_SYNC_TOLERANCE bit from the D_CLK_SYNC register. If SDMCLK is divided from SYSCLK by 4 or 5, the tolerance is always ±1.5 SYSCLK cycles.

CAUTION: The PLL and SDMCLK synchronization can result in output audio artifacts on the digital output drivers due to the discontinuity in SDMCLK.

The clock synchronizer can be used to trigger a clock interrupt (D_VECT_CLK) if a resynchronization occurs. To enable this interrupt trigger, set the CLK_SYNC_CLK_RESYNC_INT_ENABLE bit from the D_CLK_SYNC register.

The clock synchronizer has two status bits mapped into the D_CLK_SYNC register. They are:

CLK_SYNC_CLK_SYNC_STATUS
: This bit indicates if the PLL reference clock and SDMCLK are currently in sync.

CLK_SYNC_CLK_RESYNC_STATUS
: This bit indicates if the PLL reference clock and SDMCLK have resynchronized since this status bit was last read. This bit can be read when a clock interrupt is triggered to indicate if the interrupt was triggered by the clock synchronization circuit.

18.1.2 Standby Oscillator

The Ezairo 7100 system includes a second oscillator to allow for low frequency operation in standby mode. When operating in this mode, the power consumption is greatly reduced; however, the system is still executing instructions and can be woken up to regular operation mode. When the standby clock is selected as the source for the system clock (Section 18.2.1, “System Clock (SYSCLK)” on page 445), the standby oscillator provides a 20 kHz clock signal.

18.1.3 Control and Configuration Registers

Register Name	Register Description	Address
D_CLK_SYNC	SDMCLK to PLL reference clock synchronization configuration and status register	P:0xF08B
A_PLL_CTRL_0	PLL mode control register	P:0xF105
A_PLL_CTRL_1	Reference clock divider and VCO divider configuration	P:0xF106
A_PLL_CTRL_2	PLL post divider settings	P:0xF107
A_PLL_CTRL_3	PLL digital calibration settings	P:0xF108
A_PLL_CTRL_4	PLL loop filter and VCO gain settings	P:0xF109
A_PLL_CTRL_5	Temperature trimming settings of the PLL transimpedance amplifier	P:0xF10A
A_PLL_CTRL_6	VCO frequency calibration code output	P:0xF10B
A_PLL_CTRL_7	PLL lock signal, digital calibration counter error signal and PLL lock sticky bit flags	P:0xF10C
A_PLL_REF_CLK_DET_CTRL	PLL reference clock detector configuration and status	P:0xF10F

18.1.3.1 D_CLK_SYNC Settings

Bit Field	Field Name	Description
5	CLK_RESYNC_STATUS	Clock resynchronization detection status (cleared when read)
4	CLK_SYNC_STATUS	Clock synchronization status
2	CLK_RESYNC_INT_ENABLE	Clock resynchronization interrupt enable
1	CLK_SYNC_TOLERANCE	Clock synchronizer tolerance (ignored for SDMCLK_PRESCALE < 5)
0	CLK_SYNC_ENABLE	Clock synchronizer enable (ignored for SDMCLK_PRESCALE < 3)

Field Name	Value Symbol	Value Description	Hex Value
CLK_RESYNC_STATUS	CLK_RESYNC_FALSE	No clock resynchronization occurred since last read	0x0*
	CLK_RESYNC_TRUE	One or multiple clock resynchronizations occurred since last read	0x1
CLK_SYNC_STATUS	CLK_NOT_SYNCHRONIZED	SDM clock has not been synchronized	0x0*
	CLK_SYNCHRONIZED	SDM clock has been synchronized	0x1
CLK_RESYNC_INT_ENABLE	CLK_RESYNC_INT_DISABLE	Clock resynchronization interrupt disabled	0x0*
	CLK_RESYNC_INT_ENABLE	Clock resynchronization interrupt enabled	0x1

Field Name	Value Symbol	Value Description	Hex Value
CLK_SYNC_TOLERANCE	CLK_SYNC_TOLERANCE_LOW	Clock synchronization detector tolerance is +/-1.5 sys_clk cycles	0x0*
	CLK_SYNC_TOLERANCE_HIGH	Clock synchronization detector tolerance is +/-2.5 sys_clk cycles	0x1
CLK_SYNC_ENABLE	CLK_SYNC_DISABLE	Clock synchronization disabled	0x0*
	CLK_SYNC_ENABLE	Clock synchronization enabled	0x1

18.1.3.2 A_PLL_CTRL_0 Settings

Bit Field	Field Name	Description
2	EXT_CAL_EN	External calibration mode
1	FREE_RUN	PLL free running mode
0	PLL_EN	PLL enable control

Field Name	Value Symbol	Value Description	Hex Value
EXT_CAL_EN	EXTERNAL_CALIBRATION_MODE	VCO frequency controlled by the OSC_CTRL_0_FREQ_CTRL bit field from A_OSC_CTRL_0	0x1
	INTERNAL_CALIBRATION_MODE	VCO internal calibration mode	0x0*
FREE_RUN	PLL_FREE_RUN	PLL runs on VCO without feedback loop	0x1*
	PLL_TO_LOCK	PLL feedback loop activated	0x0
PLL_EN	PLL_ENABLE	PLL enabled	0x1
	PLL_DISABLE	PLL disabled	0x0*

18.1.3.3 A_PLL_CTRL_1

Bit Field	Field Name	Description
20:12	REF_DIVIDER	Input reference clock divider control - R
10:0	VCO_DIVIDER	VCO clock divider control - M

Field Name	Value Symbol	Value Description	Hex Value
REF_DIVIDER	PLL_REF_DIVIDER_MAX	Input reference clock divided by 512	0x1FF
	PLL_REF_DIVIDER_MIN	Input reference clock divided by 1	0x0*
VCO_DIVIDER	PLL_VCO_DIVIDER_MAX	VCO clock divided by 2048	0x7FF
	PLL_VCO_DIVIDER_MIN	VCO clock divided by 1	0x0*

18.1.3.4 A_PLL_CTRL_2 Settings

Bit Field	Field Name	Description
4:0	POST_DIVIDER	Output clock divider P

Field Name	Value Symbol	Value Description	Hex Value
POST_DIVIDER	PLL_POST_DIVIDER_MAX	32	0x1F
	PLL_POST_DIVIDER_MIN	1	0x0*

18.1.3.5 A_PLL_CTRL_3 Settings

Bit Field	Field Name	Description
5:4	CAL_NBITS_CFG	Defines the number of bits to calibrate
3	CAL_MSB_SKIP	Sets the MSB of the VCO calibration at a low level and skips its calibration
2:0	CAL_CFG	Defines the number of reference clock cycles used on PLL digital calibration

Field Name	Value Symbol	Value Description	Hex Value
CAL_NBITS_CFG	CAL_NBITS_16	Calibrate all 16 bits	0x3
	CAL_NBITS_12	Calibrate 12 bits	0x2*
	CAL_NBITS_10	Calibrate 10 bits	0x1
	CAL_NBITS_8	Calibrate 8 bits	0x0
CAL_MSB_SKIP	CAL_MSB_DIS	No MSB calibration	0x1
	CAL_MSB_EN	MSB calibration	0x0*
CAL_CFG	CAL_REF_DIV_32	Divide the reference clock (after R division) by 32	0x5*
	CAL_REF_DIV_16	Divide the reference clock (after R division) by 16	0x4
	CAL_REF_DIV_8	Divide the reference clock (after R division) by 8	0x3
	CAL_REF_DIV_4	Divide the reference clock (after R division) by 4	0x2
	CAL_REF_DIV_2	Divide the reference clock (after R division) by 2	0x1
	CAL_REF_DIV_1	Divide the reference clock (after R division) by 1	0x0

18.1.3.6 A_PLL_CTRL_4 Settings

Bit Field	Field Name	Description
20:16	VCO_GAIN	VCO gain control
12:8	LF_CAP_CFG	Configuration of the loop filter capacitor
4:0	LF_RES_CFG	Configuration of the loop filter resistor

Field Name	Value Symbol	Value Description	Hex Value
VCO_GAIN	VCO_GAIN_MAX	VCO gain is 32 uA at 1 V	0x1F
	VCO_GAIN_25M	VCO gain is 10 uA at 1 V	0x9*
	VCO_GAIN_MIN	VCO gain is 1 uA at 1 V	0x0

18.1.3.7 A_PLL_CTRL_5 Settings

Bit Field	Field Name	Description
3:0	V2I_TEMP_CFG	Temperature compensation trimming for the PLL transimpedance amplifier term

Field Name	Value Symbol	Value Description	Hex Value
V2I_TEMP_CFG	PLL_V2I_TEMP_COEF_MAX	80pA/degree C * Current Gain	0xF
	PLL_V2I_TEMP_COEF_DFLT	-10pA/degree C * Current Gain	0x8*
	PLL_V2I_TEMP_COEF_MIN	-100pA/degree C * Current Gain	0x0

18.1.3.8 A_PLL_CTRL_6 Settings

Bit Field	Field Name	Description
15:0	VCO_CAL_OUT	VCO frequency calibration code output

18.1.3.9 A_PLL_CTRL_7 Settings

Bit Field	Field Name	Description
6	PLL_LOCK_INT_STATUS	PLL lock interrupt status (cleared when read)
5:4	PLL_LOCK_INT_SEL	PLL lock interrupt configuration
3	UNLOCK_DET_FLAG	Sticky flag that detects whenever the PLL had unlocked
3	UNLOCK_DET_FLAG_CLEAR	Reset the sticky unlock detection flag
2	LOCK_DET_FLAG	Sticky flag that detects whenever the PLL had locked
2	LOCK_DET_FLAG_CLEAR	Reset the sticky lock detection flag
1	CAL_CNT_ERROR	Indicate an error on the digital calibration counter: saturation or low frequency value
0	PLL_LOCK	PLL signal information of correct functionality

Field Name	Value Symbol	Value Description	Hex Value
PLL_LOCK_INT_STATUS	PLL_LOCK_INT_FALSE	PLL lock interrupt not triggered	0x0*
	PLL_LOCK_INT_TRUE	PLL lock interrupt triggered	0x1
PLL_LOCK_INT_SEL	PLL_LOCK_INT_DISABLE	PLL lock interrupt disabled	0x0*
	PLL_LOCK_INT_LOCKED	If PLL is locked, an interrupt is created	0x1
	PLL_LOCK_INT_UNLOCKED	If PLL is unlocked, an interrupt is created	0x2
	PLL_LOCK_INT_ACTIVITY_CHANGE	Any PLL lock status change will create an interrupt	0x3
UNLOCK_DET_FLAG	UNLOCK_DETECTED	PLL has been unlocked at least once	0x1*
	UNLOCK_NOT_DETECTED	PLL has remained locked since this flag was last reset	0x0
UNLOCK_DET_FLAG_CLEAR	UNLOCK_FLAG_RESET	Reset the sticky unlock detection flag	0x1

Field Name	Value Symbol	Value Description	Hex Value
LOCK_DET_FLAG	LOCK_DETECTED	PLL has been locked at least once	0x1
	LOCK_NOT_DETECTED	PLL has remained unlocked since this flag was last reset	0x0*
LOCK_DET_FLAG_CLEAR	LOCK_FLAG_RESET	Reset the sticky lock detection flag	0x1
CAL_CNT_ERROR	CAL_COUNTER_ERROR	Counter error due to a saturation or VCO low frequency value	0x1
	CAL_COUNTER_OK	No error occurred	0x0*
PLL_LOCK	PLL_LOCKED	PLL is locked	0x1
	PLL_NOT_LOCKED	PLL is not locked	0x0*

18.1.3.10 A_PLL_REF_CLK_DET_CTRL Settings

Bit Field	Field Name	Description
7	CLK_DET_INT_STATUS	Clock detector interrupt status (cleared when read)
6:5	CLK_DET_INT_SEL	Clock detector interrupt configuration
3:2	CLK_DET_DIV	Clock detector configuration - Not used when running on standby clock
1	CLK_DET_STATUS	Clock detector status
0	CLK_DET_ENABLE	Clock detector enable/disable

Field Name	Value Symbol	Value Description	Hex Value
CLK_DET_INT_STATUS	CLK_DET_INT_FALSE	Clock detector interrupt not triggered	0x0*
	CLK_DET_INT_TRUE	Clock detector interrupt triggered	0x1
CLK_DET_INT_SEL	CLK_DET_INT_DISABLE	Clock detector interrupt disabled	0x0*
	CLK_DET_INT_ACTIVATED	If the clock source becomes active, an interrupt is created	0x1
	CLK_DET_INT_DEACTIVATED	If the clock source becomes inactive, an interrupt is created	0x2
	CLK_DET_INT_ACTIVITY_CHANGE	Any clock source activity change will create an interrupt	0x3
CLK_DET_DIV	CLK_DET_SLOW_CLK_DIV32	REF_CLK detector runs on slow_clk divided by 32	0x0*
	CLK_DET_SLOW_CLK_DIV64	REF_CLK detector runs on slow_clk divided by 64	0x1
	CLK_DET_SLOW_CLK_DIV96	REF_CLK detector runs on slow_clk divided by 96	0x2
	CLK_DET_SLOW_CLK_DIV128	REF_CLK detector runs on slow_clk divided by 128	0x3
CLK_DET_STATUS	CLK_NOT_ACTIVE	No clock detected	0x0*
	CLK_ACTIVE	Clock detected	0x1
CLK_DET_ENABLE	CLK_DET_DISABLE	No clock detected	0x0*
	CLK_DET_ENABLE	Clock detected	0x1

18.2 Clock Distribution

As noted in Section 18.1, “Clock Generation” on page 432, the system clock (SYSCLK) is used as the source of all the other internal clocks and clock domains. The connections of the clock distribution network that provides SYSCLK in its original or divided forms can be seen in Figure 58.

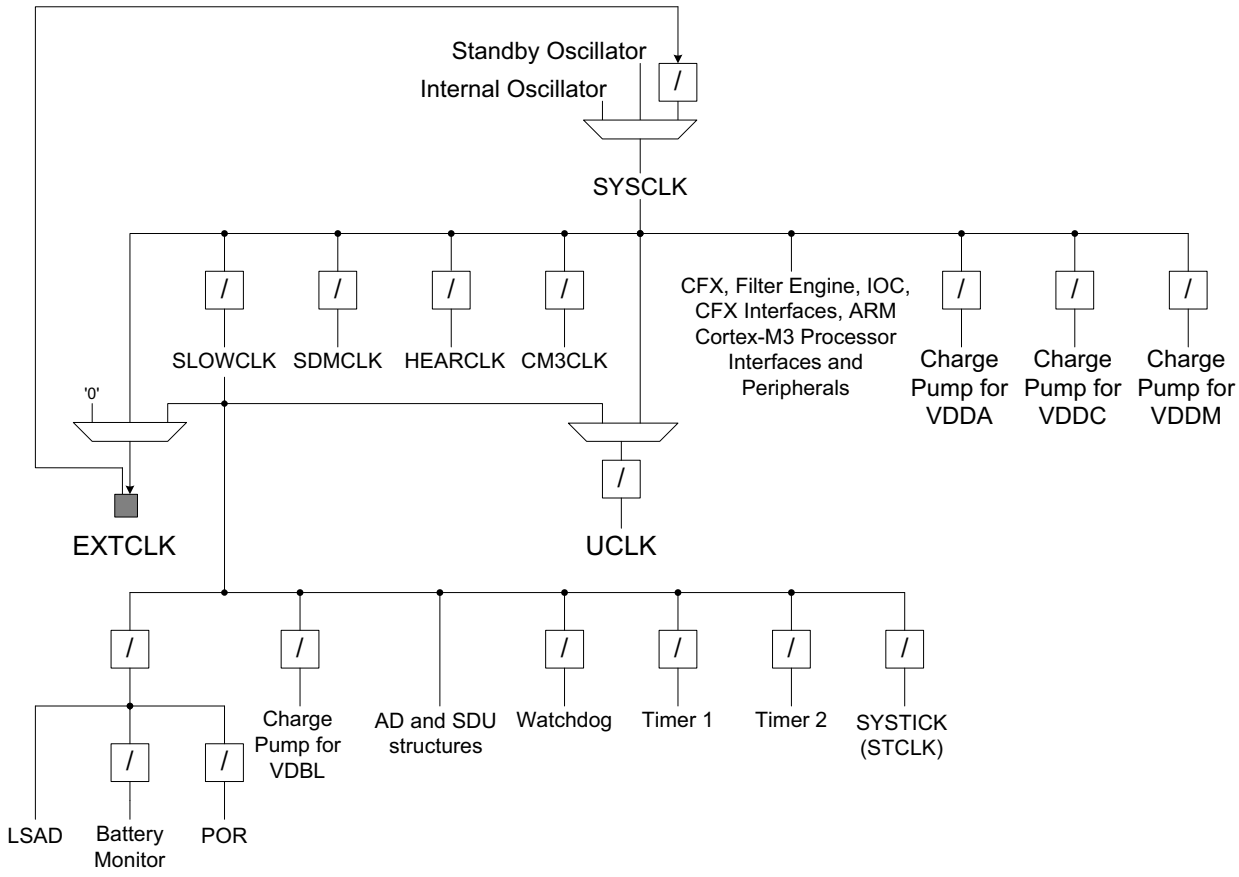


Figure 58. System Clock Distribution

Where a clock divider is used for only one component of the Ezairo 7100 device and has no side effects that change the behavior of other system components, further information about that clock divider can be found in the components description elsewhere in this manual. A description is provided, in the following subsections, for all internal clocks and clock domains that can affect more than one component of the system.

18.2.1 System Clock (SYSCLK)

The system clock is the primary clock for the system and all other clocks in the Ezairo 7100 system are derived from this clock.

By default, this clock uses the clock generated by the primary oscillator; however you can also configure it to run off of the standby oscillator or the external clock. To select the standby oscillator as the source for SYSCLK, set the `CLK_CFG_0_SYSCLK_SEL` bit from the `D_CLK_CFG_0` register. To configure the external clock as the source for SYSCLK, use the `CLK_CFG_0_EXTCLK_SEL` bit field from the `D_CLK_CFG_0` register (which is described further in

Section 18.2.6, “External Clock (EXTCLK)” on page 448).

NOTE: Configuring the external clock as input takes precedence over configuring SYSCLK to run off of the standby oscillator.

SYSCLK can be output through the EXTCLK pad (see Section 18.2.6, “External Clock (EXTCLK)”) or through one or more digital I/O pads using the DIO components. For more information about the DIO configuration, see Section 16.1, “Digital I/O (DIO)” on page 351.

IMPORTANT: For all system clock frequencies, the VDDC digital supply voltage must be set appropriately to ensure that the system’s digital logic and memories operate correctly. For more information about setting this supply voltage, see Section 19.1.2.2, “Internal Digital Supply Voltages (VDDC and VDDM)” on page 468.

18.2.2 Slow Clock (SLOWCLK)

The slow clock is used by the analog input and timer related components of the system. This includes:

- All components of the input stages
- The interpolation filters of the output stages
- The general-purpose, watchdog, and SYSTICK timers
- Various power components of the system

This clock derives from SYSCLK and uses a 6-bit integer divisor to scale SYSCLK to the correct slow clock frequency. Set the value of this divisor by using the `CLK_CFG1_SLOWCLK_PRESCALE` bit field in the `D_CLK_CFG_1` register.

IMPORTANT: System behaviors have only been characterized for SLOWCLK frequencies between 100 kHz and 2.56 MHz. Use of SLOWCLK frequencies outside of this range might result in unexpected or incorrect behavior, and are not supported.

SLOWCLK can be output through the EXTCLK pad (see Section 18.2.6, “External Clock (EXTCLK)” on page 448) or through one or more digital I/O pads using the DIO components. For more information about the DIO configuration, see Section 16.1, “Digital I/O (DIO)” on page 351.

If SYSCLK is sourced from the primary oscillator and divided (`CLK_CFG1_SLOWCLK_PRESCALE` is not set to `SLOWCLK_PRESCALE_1`), the `CLK_MULT_THROTTLING_CFG_SLOWCLK_EBL` bit in the `D_CLK_MULT_THROTTLING_CFG` register can be set to derive the slow clock directly from the primary oscillator’s VCO instead of from SYSCLK. This configuration bypasses the frequency multiplier of the primary oscillator, ensuring that the slow clock is unaffected by changes to this multiplier. For more information about multiplier-based clock throttling, see Section 18.3.1, “Multiplier-Based Clock Throttling” on page 449.

The `CLK_CFG_1_SLOWCLK_PRESCALE` bit field from the `D_CLK_CFG_1` register can be updated using the `D_SLOWCLK_THROTTLING` register. This register is used as part of the clock throttling feature, which can be used to avoid artifacts in the audio stream. For information about frequency-based clock throttling, see Section 18.3.2, “Frequency-Based Clock Throttling” on page 450.

18.2.3 Sigma-Delta Modulator Clock (SDMCLK)

The sigma-delta modulator clock is used by the sigma-delta modulators of the audio output channels.

This clock derives from SYSCLK and uses a 6-bit integer divisor to scale SYSCLK to the correct sigma-delta modulator clock frequency. Set the value of this divisor by using the O_SDMCLK_PRESCALE bit field in the A_O_SDMCLK register.

SDMCLK can be output through one or more digital I/O pads using the DIO components. For more information about the DIO configuration, see Section 16.1, “Digital I/O (DIO)” on page 351.

If SYSCLK is sourced from the primary oscillator and divided (O_SDMCLK_PRESCALE is not set to SDMCLK_PRESCALE_1), the CLK_MULT_THROTTLING_CFG_SDMCLK_EBL bit in the D_CLK_MULT_THROTTLING_CFG register can be set to derive the sigma-delta modulator clock directly from the primary oscillator’s VCO instead of from SYSCLK. This configuration bypasses the frequency multiplier of the primary oscillator, ensuring that the sigma-delta modulator clock is unaffected by changes to this multiplier. For more information about multiplier-based clock throttling, see Section 18.3.1, “Multiplier-Based Clock Throttling” on page 449.

The PRESCALE bit field in the A_O_SDMCLK register can be updated using the A_O_SDMCLK_THROTTLING register. This register is used as part of the clock throttling feature, which can be used to avoid artifacts in the audio stream. For information about frequency-based clock throttling, see Section 18.3.2, “Frequency-Based Clock Throttling” on page 450.

18.2.4 HEAR Accelerator Clock (HEARCLK)

The accelerator clock is only used as the clock for the HEAR accelerator. Normally this clock operates at the same frequency as SYSCLK. However, to spread out the dynamic power consumption of the system or to avoid conflicts in accessing shared memory and microcode memory, you might want to use a different clock frequency for the HEAR.

This clock derives from SYSCLK through scaling by a fractional divisor. Setting the CLK_CFG_0_HEARCLK_PRESCALE bit field from the D_CLK_CFG_0 register configures the clock frequency of this clock according to the following equation:

$$f_{HEARCLK} = \frac{f_{SYSCLK} \times 8}{(8 + CLK_CFG0_HEARCLK_PRESCALE)}$$

The double access mode setting configures the system to share system clock cycles between the CFX processor and the HEAR through the accelerator clock if the accelerator clock uses a prescaler. The system can use this mode to guarantee that no memory conflicts will occur between the CFX and HEAR when accessing the microcode memory or shared memory bus. To configure the double access mode setting, set the CLK_CFG_1_DBL_ACCESS_MODE bit in the D_CLK_CFG_1 register.

In double access mode, if the CLK_CFG_0_HEARCLK_PRESCALE bit field is not set to 0 (i.e., the HEAR clock is running slower than SYSCLK), the CFX processor operates at a clock frequency defined by the following equation:

$$f_{CFX\ Core} = \frac{f_{SYSCLK} \times CLK_CFG0_HEARCLK_PRESCALE}{(8 + CLK_CFG0_HEARCLK_PRESCALE)}$$

In this way, the HEAR accelerator clock frequency remains the same and the execution on the CFX processor is limited to those system clock cycles when the accelerator clock is not running.

An example use case for double access mode might use an accelerator clock prescaler of 8. This results in both the CFX DSP and HEAR Accelerator each operating at $\frac{f_{SYSCLK}}{2}$. If you then doubled SYSCLK frequency and applied an additional division by 2 to all derived clocks (including the slow clock), you would end up with the original number of clock cycles for both of the cores without the possibility of memory conflicts occurring between the cores.

18.2.5 ARM Cortex-M3 Processor Clock (CM3CLK)

The ARM Cortex-M3 processor clock is only used as the clock for the ARM Cortex-M3 core. Normally this clock operates at the same frequency as SYSCLK. However, to spread out the dynamic power consumption of the system or to avoid conflicts when accessing FIFO memory, or the ARM Cortex-M3 processor's memory that is shared between the CFX and the ARM Cortex-M3 core, you might want to use a different clock frequency for the ARM Cortex-M3 processor.

This clock derives from SYSCLK and uses a 4-bit integer divisor to scale SYSCLK to the correct coprocessor execution frequency. Set the value of this divisor by using the `CLK_CFG_0_CM3CLK_PRESCALE` bit field in the `D_CLK_CFG_0` register.

18.2.6 External Clock (EXTCLK)

The external clock in the Ezairo 7100 system can be used as both an input and an output clock source. To select the operating configuration for the external clock, set the `CLK_CFG_0_EXTCLK_SEL` bit field of the `D_CLK_CFG_0` register.

When configured as an input source, `EXTCLK` is passed through a prescaling integer clock divider before being used as the source for `SYSCLK`. To set the divisor used for this clock division, set the `CLK_CFG_0_EXTCLK_PRESCALE` bit field in the `D_CLK_CFG_0` register.

When EXTCLK is disabled or is configured as an input source, the EXTCLK pad can be configured to use an optional pull-up resistor. To configure this resistor, use the `CLK_CFG_0_EXTCLK_PULL_EN` bit in the `D_CLK_CFG_0` register.

When configured as an output source, the external clock output pin can be driven by one of:

- SYSCLK (see Section 18.2.1, "System Clock (SYSCLK)")
- SLOWCLK (see Section 18.2.2, "Slow Clock (SLOWCLK)")
- SDMCLK (see Section 18.2.3, "Sigma-Delta Modulator Clock (SDMCLK)")

The external clock output is not scaled by any factor in addition to those scaling factors that already apply to the source clock.

18.2.7 User Clock (UCLK)

The user clock is an output clock that you can use as a clock source for the PCM interfaces or for any external components. This clock is not used internally by the Ezairo 7100 system, so its usage can be entirely dependent on the outside needs of the larger system containing Ezairo 7100.

Either `SYSCLK` or `SDMCLK` can be selected as the source for `UCLK`, by setting the `CLK_CFG_1_UCLK_SRC` bit from the `D_CLK_CFG_1` register. `UCLK` is derived from this clock through a 12-bit integer division by the `CLK_CFG_1_UCLK_PRESCALE` bit field in the `D_CLK_CFG_1` register.

If SYSCLK is sourced from the primary oscillator and UCLK is divided from SYSCLK (CLK_CFG_1_UCLK_PRESCALE is not set to UCLK_PRESCALE_1 and CLK_CFG_1_UCLK_SRC is set to UCLK_SRC_SYSCLK), the CLK_MULT_THROTTLING_CFG_UCLK_EBL bit in the D_CLK_MULT_THROTTLING_CFG register can be set to derive the user clock directly from the primary oscillator's VCO instead of from SYSCLK. This configuration bypasses the frequency multiplier of the primary oscillator, ensuring that UCLK is unaffected by changes to this multiplier. For more information about multiplier-based clock throttling, see Section 18.3.1, "Multiplier-Based Clock Throttling".

IMPORTANT: For frequency-based clock throttling, the UCLK frequency will change with its source clock frequency. To prevent changes to UCLK when using the frequency-based clock throttling SDMCLK must be used as the source for UCLK.

18.3 Clock Throttling

When the primary oscillator is selected as the source for SYSCLK, the frequency of SYSCLK can be throttled to accommodate changes to the processing needs of the algorithms used by the user application. Clock throttling is specifically designed to allow the dynamic frequency scaling of the system clock without audible artifacts.

When using clock throttling, the frequency of SYSCLK and all clocks derived from SYSCLK are increased or reduced without affecting SLOWCLK or SDMCLK. Additionally, multiplier-based clock throttling of UCLK can be used to prevent the effects of this frequency change on UCLK when this clock is not sourced from SDMCLK.

IMPORTANT: Clock throttling is intended to support the changing needs of an algorithm, by providing additional or fewer clock cycles (as appropriate) between sample blocks. While it is theoretically possible to switch the clock repeatedly at rates approaching the block sample rate, periodic switching of the clock to a frequency within the audio bank may result in unintended audio artifacts.

18.3.1 Multiplier-Based Clock Throttling

Multiplier-based clock throttling uses the VCO multiplier to adjust the SYSCLK frequency. When multiplier-based clock throttling is enabled for a clock, that clock is divided from the base VCO clock frequency (i.e., those clocks are unaffected by the setting of the OSC_CTRL_1_MULT bit field from A_OSC_CTRL_1).

IMPORTANT: When using multiplier-based clock throttling, scaling of the SYSCLK frequency is limited to multiples of the base VCO frequency (the PLL post-divisor cannot be used to divide the output frequency). Despite this limited frequency scaling range, multiplier-based clock throttling is recommended wherever possible as it is guaranteed to not create any artifacts in the audio path.

To connect a clock to the base VCO output, multiplier-based clock throttling requires:

- The clock to be divided by at least a factor of two
- The bit corresponding to the specified clock must be set in D_CLK_MULT_THROTTLING_CFG

Multiplier-based clock throttling can be used for SLOWCLK, SDMCLK, and UCLK as described below:

- If the frequency of SLOWCLK is unchanged when scaling SYSCLK using multiplier-based clock throttling, this mechanism guarantees that no audio artifacts will occur in the input stage. To enable multiplier-based clock throttling of SLOWCLK, set the CLK_MULT_THROTTLING_CFG_SLOWCLK_EBL bit from

D_CLK_MULT_THROTTLING_CFG and configure SLOWCLK to use any prescaler other than SLOWCLK_PRESCALE_1.

- If the frequency of SDMCLK is unchanged when scaling SYSCLK using multiplier-based clock throttling, this mechanism guarantees that no audio artifacts will occur in the output stage. To enable multiplier-based clock throttling of SDMCLK, set the CLK_MULT_THROTTLING_CFG_SDMCLK_EBL bit from D_CLK_MULT_THROTTLING_CFG and configure SDMCLK to use any prescaler other than SDMCLK_PRESCALE_1.
- If the frequency of UCLK is unchanged when scaling SYSCLK using multiplier-based clock throttling, this mechanism guarantees that the UCLK output will be unaffected. To enable multiplier-based clock throttling of UCLK, set the CLK_MULT_THROTTLING_CFG_UCLK_EBL bit from D_CLK_MULT_THROTTLING_CFG and configure UCLK to use any prescaler other than UCLK_PRESCALE_1.

18.3.2 Frequency-Based Clock Throttling

Frequency-based clock throttling adjusts the base frequency of the VCO and updates the SLOWCLK and SDMCLK prescaler in the same cycle. Because all three clocks are updated at the same time, the only artifacts produced are those caused be the settling time of the VCO.

IMPORTANT: The error introduced due to the settling time of the VCO will always be less than one SYSCLK clock cycle, which limits any artifacts that might occur due to a frequency change. For frequency-based throttling that double or halve the VCO frequency, artifacts observed should not be audible. For larger frequency-based clock throttling steps an audible artifact is possible, and the user might wish to use the advanced frequency-based clock throttling options described in Section 18.3.2.1, "Advanced Frequency-Based Clock Throttling Options" to limit or eliminate these artifacts.

When initiating a frequency-based clock throttling switch, a user application must:

1. Set the A_OSC_CTRL_THROTTLING register with the updated frequency setting that should be copied to the A_OSC_CTRL_0 register when applying clock throttling.
2. Set the D_SLOWCLK_THROTTLING register with the updated value to use for the CLK_CFG_1_SLOWCLK_PRESCALE bit field from the D_CLK_CFG_1 register after clock throttling. The updated SLOWCLK prescaler should ensure that the frequency of SLOWCLK before and after the clock throttling event is unchanged.
3. Set the A_O_SDMCLK_THROTTLING register with the updated value to use for the O_SDMCLK_PRESCALE bit field from the A_O_SDMCLK register after clock throttling. The updated SDMCLK prescaler should ensure that the frequency of SDMCLK before and after the clock throttling event is unchanged.
4. Write CLK_THROTTLE_APPLY to the CLK_THROTTLING_CTRL_APPLY bit field from the D_CLK_THROTTLING_CTRL register.

An application can then poll the CLK_THROTTLING_CTRL_STATUS bit field from the D_CLK_THROTTLING_CTRL register, which indicates if the frequency-based clock throttling has completed or is still pending.

NOTE: The main error that will be observed when applying frequency-based clock throttling is the error in the updated VCO output frequency. To limit this effect, the initial frequency and frequency switched to must both be calibrated configurations of the VCO and both configurations must use

the same multiplier value stored to the OSC_CTRL_1_MULT bit field from A_OSC_CTRL_1 register.

IMPORTANT: For optimal performance, frequency-based clock throttling should be limited to steps of no more than 400% of the lower frequency. For example, frequency switches from 1.28 MHz to 5.12 MHz or from 10.24 MHz to 2.56 MHz would fit this recommendation, however clock frequency changes from 1.28 MHz to 10.24 MHz should transition through an intermediate frequency first.

18.3.2.1 Advanced Frequency-Based Clock Throttling Options

When switching the frequency of the system clocks using frequency-based clock throttling, the primary cause of audio artifacts relates to the error in the VCO output frequency. This output frequency will generally settle in less than one clock period, however in some cases, the error in the first clock period may be enough to produce a detectable audio artifact due to an imbalance in the charge observed on the H-bridge of the digital drive outputs. If your system is using frequency-based clock throttling and audio artifacts are detected, two advanced clock throttling options are provided.

Frequency Correction

To correct for frequency errors in the first cycles following the application of a clock throttling setting, the frequency configuration applied to the A_OSC_CTRL_0 register can be updated up to three times in consecutive clock cycles to correct for the error in each clock cycle due to the switch in frequency.

In addition to the normal frequency-based clock throttling steps, the A_OSC_CTRL_THROTTLING_2 and A_OSC_CTRL_THROTTLING_3 registers are set with the intermediate values to be copied to the A_OSC_CTRL_0 register.

When clock throttling is applied by writing CLK_THROTTLE_APPLY_2 to the CLK_THROTTLING_CTRL_APPLY bit field from the D_CLK_THROTTLING_CTRL register, the value from A_OSC_CTRL_THROTTLING_2 is used in the first clock cycle, and the value in A_OSC_CTRL_THROTTLING is used thereafter.

When clock throttling is applied by writing CLK_THROTTLE_APPLY_3 to the CLK_THROTTLING_CTRL_APPLY bit field from the D_CLK_THROTTLING_CTRL register, the value from A_OSC_CTRL_THROTTLING_3 is used in the first clock cycle, the value from A_OSC_CTRL_THROTTLING_2 is used in the second clock cycle, and the value in A_OSC_CTRL_THROTTLING is used thereafter.

The values that should be used for each frequency correction step when using frequency-based clock throttling are defined based on the step between the initial and final frequencies. The values, listed in Table 74, define the offset relative to the final calibrated setting that should be used for each corrected step. For example, if throttling the clock from 5.12 MHz to 1.28 MHz the A_OSC_CTRL_THROTTLING register should be set to the final calibrated clock setting for 1.28 MHz. If this setting was defined as x, the A_OSC_CTRL_THROTTLING_2 register should be set to $(x + 16)$ and A_OSC_CTRL_THROTTLING_3 register should be set to $(x – 32)$.

Table 74. Correction Factors for Frequency Correction

Initial Frequency (MHz)	Final Frequency (MHz)					
	1.28		2.56		5.12	
	First Step	Second Step	First Step	Second Step	First Step	Second Step
1.28	-	-	175	-32	319	-32
2.56	-16	16	-	-	319	-32
5.12	-32	16	-32	16	-	-

Output State Control

If the O_SDM_CTRL_OD_DELAY bit from the A_O_SDM_CTRL register is set, the output on the negative terminal of the receiver is delayed by one cycle when compared with the positive terminal. This produces an additional state at the output to the sigma-delta modulator where the receiver pads of the H-bridge are outputting the same value. By setting the CLK_THROTTLING_CFG_CLK_THROTTLING_ON_OD_ZERO bit from the D_CLK_THROTTLING_CFG register, the application of the clock throttling values will wait until the H-bridge will be in this state before switching the divisors and VCO frequency. Because the H-bridge is in a balanced state during the first clock cycle, any error observed at the VCO frequency does not charge either side of the H-bridge which avoids the potential audio artifacts.

NOTE: When using this mode, it may take some time before a pending clock throttling is applied as the clocking block has to wait for the SLOWCLK, SDMCLK and output drivers to be ready to update in the same cycle. This issue can be exasperated by large amplitude outputs or by having both output drivers enabled.

For more information about configuring the sigma-delta modulators, see Section 11.3, “Sigma-Delta Modulator” on page 165.

18.4 Control and Configuration Registers

Register Name	Register Description	Address
D_CLK_CFG_0	ARM Cortex-M3 core, external and HEAR clock prescale registers	P:0xF080
D_CLK_CFG_1	UCLK and SLOWCLK prescale registers	P:0xF081
D_CLK_CFG_CPA	Prescale registers for analog charge pump	P:0xF084
D_CLK_CFG_CPD	Prescale register for the digital interface charge pump	P:0xF085
D_CLK_CFG_CPC	Prescale registers for digital core charge pump	P:0xF086
D_CLK_CFG_CPM	Prescale registers for memories charge pump	P:0xF087
D_SLOWCLK_THROTTLING	Slow clock throttling configuration register	P:0xF088
D_CLK_THROTTLING_CTRL	VCO frequency-based clock throttling control register	P:0xF089
D_CLK_THROTTLING_CFG	VCO frequency-based clock throttling configuration register	P:0xF08C
D_CLK_MULT_THROTTLING_CFG	VCO multiplier-based clock throttling configuration register	P:0xF08A
A_OSC_CTRL_0	Primary oscillator frequency setting	P:0xF100
A_OSC_CTRL_1	Enable settings and multiplier settings of the primary oscillator	P:0xF101

Register Name	Register Description	Address
A_OSC_CTRL_2	Temperature compensation for the primary oscillator	P:0xF102
A_OSC_CTRL_3	Temperature compensation output values of the primary oscillator	P:0xF103
A_OSC_CTRL_THROTTLING	System clock throttling configuration register	P:0xF104
A_OSC_CTRL_THROTTLING_2	System clock throttling configuration register 2	P:0xF118
A_OSC_CTRL_THROTTLING_3	System clock throttling configuration register 3	P:0xF119
A_O_SDMCLK	SDM clock configuration register	P:0xF151
A_O_SDMCLK_THROTTLING	SDM clock throttling configuration register	P:0xF154

18.4.1 D_CLK_CFG_0 Settings

Bit Field	Field Name	Description
24	EXTCLK_CLK_SEL	Controls whether SLOWCLK or SDMCLK can be output on EXT_CLK
23:20	CM3CLK_PRESCALE	Prescale value for the CM3CLK clock (1 to 16 in steps of 1)
19	EXTCLK_PULL_EN	Controls the pad pull-up
18	SYSCLK_SEL	Controls the functionality of SYSCLK
17:16	EXTCLK_SEL	Controls the functionality of the EXT_CLK pin
15:12	EXTCLK_PRESCALE	Prescale value for the input clock from pad EXT_CLK (1 to 16 in steps of 1)
11	EXTCLK_DRIVE	EXT_CLK drive control
9:0	HEARCLK_PRESCALE	Prescale value for the HEAR clock (1 to 128.875 in steps of 0.125)

Field Name	Value Symbol	Value Description	Hex Value
EXTCLK_CLK_SEL	EXTCLK_OUT_SRC_SLOWCLK	SLOWCLK can be output on EXT_CLK	0x0*
	EXTCLK_OUT_SRC_SDMCLK	SDMCLK can be output on EXT_CLK	0x1

Field Name	Value Symbol	Value Description	Hex Value
CM3CLK_PRESCALE	CM3CLK_PRESCALE_1	Divide by 1	0x0*
	CM3CLK_PRESCALE_2	Divide by 2	0x1
	CM3CLK_PRESCALE_3	Divide by 3	0x2
	CM3CLK_PRESCALE_4	Divide by 4	0x3
	CM3CLK_PRESCALE_5	Divide by 5	0x4
	CM3CLK_PRESCALE_6	Divide by 6	0x5
	CM3CLK_PRESCALE_7	Divide by 7	0x6
	CM3CLK_PRESCALE_8	Divide by 8	0x7
	CM3CLK_PRESCALE_9	Divide by 9	0x8
	CM3CLK_PRESCALE_10	Divide by 10	0x9
	CM3CLK_PRESCALE_11	Divide by 11	0xA
	CM3CLK_PRESCALE_12	Divide by 12	0xB
	CM3CLK_PRESCALE_13	Divide by 13	0xC
	CM3CLK_PRESCALE_14	Divide by 14	0xD
	CM3CLK_PRESCALE_15	Divide by 15	0xE
	CM3CLK_PRESCALE_16	Divide by 16	0xF
EXTCLK_PULL_EN	EXTCLK_PU_DISABLED	EXTCLK pull-up is disabled	0x0
	EXTCLK_PU_ENABLED	EXTCLK pull-up is enabled when EXTCLK is in input mode	0x1*
SYSCLK_SEL	SYSCLK_SEL_DEFAULT	Select the default primary oscillator as the internal source for SYSCLK	0x0*
	SYSCLK_SEL_STANDBY	Select the standby oscillator as the internal source for SYSCLK	0x1
EXTCLK_SEL	EXTCLK_DISABLED	EXTCLK is disabled	0x0*
	EXTCLK_OUT_SYSCLK	EXTCLK is SYSCLK	0x1
	EXTCLK_OUT_SLOWCLK	EXTCLK is SLOWCLK	0x2
	EXTCLK_IN_ENABLE	EXTCLK is an input	0x3

Field Name	Value Symbol	Value Description	Hex Value
EXTCLK_PRESCALE	EXTCLK_PRESCALE_1	Divide by 1	0x0*
	EXTCLK_PRESCALE_2	Divide by 2	0x1
	EXTCLK_PRESCALE_3	Divide by 3	0x2
	EXTCLK_PRESCALE_4	Divide by 4	0x3
	EXTCLK_PRESCALE_5	Divide by 5	0x4
	EXTCLK_PRESCALE_6	Divide by 6	0x5
	EXTCLK_PRESCALE_7	Divide by 7	0x6
	EXTCLK_PRESCALE_8	Divide by 8	0x7
	EXTCLK_PRESCALE_9	Divide by 9	0x8
	EXTCLK_PRESCALE_10	Divide by 10	0x9
	EXTCLK_PRESCALE_11	Divide by 11	0xA
	EXTCLK_PRESCALE_12	Divide by 12	0xB
	EXTCLK_PRESCALE_13	Divide by 13	0xC
	EXTCLK_PRESCALE_14	Divide by 14	0xD
	EXTCLK_PRESCALE_15	Divide by 15	0xE
	EXTCLK_PRESCALE_16	Divide by 16	0xF
EXTCLK_DRIVE	EXTCLK_LOW_DRIVE	Low drive strength	0x0*
	EXTCLK_HIGH_DRIVE	High drive strength	0x1
HEARCLK_PRESCALE	HEARCLK_PRESCALE_1_000	Divide by 1	0x0*
	HEARCLK_PRESCALE_1_125	Divide by 1.125	0x1
	HEARCLK_PRESCALE_1_250	Divide by 1.25	0x2
	HEARCLK_PRESCALE_1_375	Divide by 1.375	0x3
	HEARCLK_PRESCALE_1_500	Divide by 1.5	0x4
	HEARCLK_PRESCALE_1_625	Divide by 1.625	0x5
	HEARCLK_PRESCALE_1_750	Divide by 1.75	0x6
	HEARCLK_PRESCALE_1_875	Divide by 1.875	0x7
	HEARCLK_PRESCALE_2_000	Divide by 2	0x8
	HEARCLK_PRESCALE_3_000	Divide by 3	0x10
	HEARCLK_PRESCALE_4_000	Divide by 4	0x18
	HEARCLK_PRESCALE_8_000	Divide by 8	0x38
	HEARCLK_PRESCALE_16_000	Divide by 16	0x78
	HEARCLK_PRESCALE_24_000	Divide by 24	0xB8
	HEARCLK_PRESCALE_32_000	Divide by 32	0xF8
	HEARCLK_PRESCALE_64_000	Divide by 64	0x1F8
	HEARCLK_PRESCALE_128_000	Divide by 128	0x3F8

18.4.2 D_CLK_CFG_1 Settings

Bit Field	Field Name	Description
22	SLOWCLK_SRC	Indicates the user clock source (SYSCLK or SDMCLK)
21	UCLK_SRC	Indicates the user clock source (SYSCLK or SDMCLK)
20	DBL_ACCESS_MODE	Creates a virtual double access for the shared memories
19:8	UCLK_PRESCALE	Prescale value for the user clock (1 to 4096 in steps of 1)
5:0	SLOWCLK_PRESCALE	Prescale value for the slow clock (1 to 64 in steps of 1)

Field Name	Value Symbol	Value Description	Hex Value
SLOWCLK_SRC	SLOWCLK_SRC_SYSCLK	SLOWCLK is SYSCLK	0x0*
	SLOWCLK_SRC_SDMCLK	SLOWCLK is SDMCLK	0x1
UCLK_SRC	UCLK_SRC_SYSCLK	UCLK is SYSCLK	0x0*
	UCLK_SRC_SDMCLK	UCLK is SDMCLK	0x1
DBL_ACCESS_MODE	DBL_ACCESS_MODE_DISABLE	Shared memory access is normal	0x0*
	DBL_ACCESS_MODE_ENABLE	Shared memory access uses a virtual double access mode	0x1
UCLK_PRESCALE	UCLK_PRESCALE_1	Divide by 1	0x0*
	UCLK_PRESCALE_2	Divide by 2	0x1
	UCLK_PRESCALE_3	Divide by 3	0x2
	UCLK_PRESCALE_4	Divide by 4	0x3
	UCLK_PRESCALE_5	Divide by 5	0x4
	UCLK_PRESCALE_6	Divide by 6	0x5
	UCLK_PRESCALE_7	Divide by 7	0x6
	UCLK_PRESCALE_8	Divide by 8	0x7
	UCLK_PRESCALE_9	Divide by 9	0x8
	UCLK_PRESCALE_10	Divide by 10	0x9
	UCLK_PRESCALE_16	Divide by 16	0xF
	UCLK_PRESCALE_32	Divide by 32	0x1F
	UCLK_PRESCALE_64	Divide by 64	0x3F
	UCLK_PRESCALE_128	Divide by 128	0x7F
	UCLK_PRESCALE_256	Divide by 256	0xFF
	UCLK_PRESCALE_512	Divide by 512	0x1FF
	UCLK_PRESCALE_1024	Divide by 1024	0x3FF
	UCLK_PRESCALE_2048	Divide by 2048	0x7FF
	UCLK_PRESCALE_4096	Divide by 4096	0xFFF

Field Name	Value Symbol	Value Description	Hex Value
SLOWCLK_PRESCALE	SLOWCLK_PRESCALE_1	Divide by 1	0x0*
	SLOWCLK_PRESCALE_2	Divide by 2	0x1
	SLOWCLK_PRESCALE_3	Divide by 3	0x2
	SLOWCLK_PRESCALE_4	Divide by 4	0x3
	SLOWCLK_PRESCALE_5	Divide by 5	0x4
	SLOWCLK_PRESCALE_6	Divide by 6	0x5
	SLOWCLK_PRESCALE_7	Divide by 7	0x6
	SLOWCLK_PRESCALE_8	Divide by 8	0x7
	SLOWCLK_PRESCALE_9	Divide by 9	0x8
	SLOWCLK_PRESCALE_10	Divide by 10	0x9
	SLOWCLK_PRESCALE_11	Divide by 11	0xA
	SLOWCLK_PRESCALE_12	Divide by 12	0xB
	SLOWCLK_PRESCALE_13	Divide by 13	0xC
	SLOWCLK_PRESCALE_14	Divide by 14	0xD
	SLOWCLK_PRESCALE_15	Divide by 15	0xE
	SLOWCLK_PRESCALE_16	Divide by 16	0xF
	SLOWCLK_PRESCALE_17	Divide by 17	0x10
	SLOWCLK_PRESCALE_18	Divide by 18	0x11
	SLOWCLK_PRESCALE_19	Divide by 19	0x12
	SLOWCLK_PRESCALE_20	Divide by 20	0x13
	SLOWCLK_PRESCALE_21	Divide by 21	0x14
	SLOWCLK_PRESCALE_22	Divide by 22	0x15
	SLOWCLK_PRESCALE_23	Divide by 23	0x16
	SLOWCLK_PRESCALE_24	Divide by 24	0x17
	SLOWCLK_PRESCALE_25	Divide by 25	0x18
	SLOWCLK_PRESCALE_26	Divide by 26	0x19
	SLOWCLK_PRESCALE_27	Divide by 27	0x1A
	SLOWCLK_PRESCALE_28	Divide by 28	0x1B
	SLOWCLK_PRESCALE_29	Divide by 29	0x1C
	SLOWCLK_PRESCALE_30	Divide by 30	0x1D
	SLOWCLK_PRESCALE_31	Divide by 31	0x1E
	SLOWCLK_PRESCALE_32	Divide by 32	0x1F
	SLOWCLK_PRESCALE_33	Divide by 33	0x20
	SLOWCLK_PRESCALE_34	Divide by 34	0x21
	SLOWCLK_PRESCALE_35	Divide by 35	0x22
	SLOWCLK_PRESCALE_36	Divide by 36	0x23
	SLOWCLK_PRESCALE_37	Divide by 37	0x24
	SLOWCLK_PRESCALE_38	Divide by 38	0x25

Field Name	Value Symbol	Value Description	Hex Value
SLOWCLK_PRESCALE (continued)	SLOWCLK_PRESCALE_39	Divide by 39	0x26
	SLOWCLK_PRESCALE_40	Divide by 40	0x27
	SLOWCLK_PRESCALE_41	Divide by 41	0x28
	SLOWCLK_PRESCALE_42	Divide by 42	0x29
	SLOWCLK_PRESCALE_43	Divide by 43	0x2A
	SLOWCLK_PRESCALE_44	Divide by 44	0x2B
	SLOWCLK_PRESCALE_45	Divide by 45	0x2C
	SLOWCLK_PRESCALE_46	Divide by 46	0x2D
	SLOWCLK_PRESCALE_47	Divide by 47	0x2E
	SLOWCLK_PRESCALE_48	Divide by 48	0x2F
	SLOWCLK_PRESCALE_49	Divide by 49	0x30
	SLOWCLK_PRESCALE_50	Divide by 50	0x31
	SLOWCLK_PRESCALE_51	Divide by 51	0x32
	SLOWCLK_PRESCALE_52	Divide by 52	0x33
	SLOWCLK_PRESCALE_53	Divide by 53	0x34
	SLOWCLK_PRESCALE_54	Divide by 54	0x35
	SLOWCLK_PRESCALE_55	Divide by 55	0x36
	SLOWCLK_PRESCALE_56	Divide by 56	0x37
	SLOWCLK_PRESCALE_57	Divide by 57	0x38
	SLOWCLK_PRESCALE_58	Divide by 58	0x39
	SLOWCLK_PRESCALE_59	Divide by 59	0x3A
	SLOWCLK_PRESCALE_60	Divide by 60	0x3B
	SLOWCLK_PRESCALE_61	Divide by 61	0x3C
	SLOWCLK_PRESCALE_62	Divide by 62	0x3D
	SLOWCLK_PRESCALE_63	Divide by 63	0x3E
	SLOWCLK_PRESCALE_64	Divide by 64	0x3F

18.4.3 D_CLK_CFG_CPA Settings

Bit Field	Field Name	Description
5:0	CP_VDDA_CLK_PRESCALE	Prescale the clock for the VDDA charge pump (1 to 64 in steps of 1)

18.4.4 D_CLK_CFG_CPD Settings

Bit Field	Field Name	Description
5:0	CP_VDBL_CLK_PRESCALE	Prescale the clock for the VDBL charge pump (1 to 64 in steps of 1)

18.4.5 D_CLK_CFG_CPC Settings

Bit Field	Field Name	Description
5:0	CP_VDDC_CLK_PRESCALE	Prescale the clock for the VDDC charge pump (1 to 64 in steps of 1)

18.4.6 D_CLK_CFG_CPM Settings

Bit Field	Field Name	Description
5:0	CP_VDDM_CLK_PRESCALE	Prescale the clock for the VDDM charge pump (1 to 64 in steps of 1)

18.4.7 D_SLOWCLK_THROTTLING Settings

Bit Field	Field Name	Description
5:0	PRESCALE	Future clock throttling value for SLOWCLK_PRESCALE

18.4.8 D_CLK_THROTTLING_CTRL Settings

Bit Field	Field Name	Description
1:0	APPLY	Clock throttling apply command
1:0	STATUS	Clock throttling apply status

Field Name	Value Symbol	Value Description	Hex Value
APPLY	CLK_THROTTLE_ABORT	Abort clock throttling if it is pending	0x0
	CLK_THROTTLE_APPLY	Apply new clock throttling values (as soon as the next counter value of the slow clock divider is zero)	0x1
	CLK_THROTTLE_APPLY_2	Apply two new clock throttling values (as soon as the next counter value of the slow clock divider is zero)	0x2
	CLK_THROTTLE_APPLY_3	Apply three new clock throttling values (as soon as the next counter value of the slow clock divider is zero)	0x3
STATUS	CLK_THROTTLE_DISABLED	Clock throttling inactive	0x0*
	CLK_THROTTLE_PENDING	New clock throttling values are pending	0x1
	CLK_THROTTLE_PENDING_2	Two new clock throttling values are pending	0x2
	CLK_THROTTLE_PENDING_3	Three clock throttling values are pending	0x3

18.4.9 D_CLK_THROTTLING_CFG Settings

Bit Field	Field Name	Description
0	CLK_THROTTLING_ON_OD_ZERO	Enable VCO frequency-based clock throttling only on OD zero output

Field Name	Value Symbol	Value Description	Hex Value
CLK_THROTTLING_ON_OD_ZERO	CLK_THROTTLING_ON_OD_ZERO_DISABLE	Always allow VCO frequency-based clock throttling	0x0*
	CLK_THROTTLING_ON_OD_ZERO_ENABLE	Only allow VCO frequency-based clock throttling when OD_DELAY is disabled, or when both OD0 and OD1 are either disabled or output a zero	0x1

18.4.10 D_CLK_MULT_THROTTLING_CFG Settings

Bit Field	Field Name	Description
2	UCLK_EBL	User clock VCO multiplier based clock throttling enable
1	DACLK_EBL	OSRDAC clock VCO multiplier based clock throttling enable
0	SLOWCLK_EBL	Slow clock VCO multiplier based clock throttling enable

Field Name	Value Symbol	Value Description	Hex Value
UCLK_EBL	VCO_MULT_THROTTLE_UCLK_DISABLE	User clock is not divided by the VCO multiplier value	0x0*
	VCO_MULT_THROTTLE_UCLK_ENABLE	User clock is divided by the VCO multiplier value	0x1
DACLK_EBL	VCO_MULT_THROTTLE_DACLK_DISABLE	OSRDAC clock is not divided by the VCO multiplier value	0x0*
	VCO_MULT_THROTTLE_DACLK_ENABLE	OSRDAC clock is divided by the VCO multiplier value	0x1
SLOWCLK_EBL	VCO_MULT_THROTTLE_SLOWCLK_DISABLE	Slow clock is not divided by the VCO multiplier value	0x0*
	VCO_MULT_THROTTLE_SLOWCLK_ENABLE	Slow clock is divided by the VCO multiplier value	0x1

18.4.11 A_OSC_CTRL_0 Settings

Bit Field	Field Name	Description
15:0	FREQ_CTRL	Oscillator frequency programming control. If 0x000 is written to bits [15:4], this field will keep its previous value.

Field Name	Value Symbol	Value Description	Hex Value
FREQ_CTRL	OSC_FREQ_MAX	Maximum frequency	0xFFFF
	OSC_FREQ_2M56	Nominal oscillator frequency of 2.56 MHz	0x400*
	OSC_FREQ_MIN	Minimum frequency	0x10

18.4.12 A_OSC_CTRL_1 Settings

Bit Field	Field Name	Description
5:4	MULT	Oscillator clock multiplier control
0	ENABLE	Oscillator enable control

Field Name	Value Symbol	Value Description	Hex Value
MULT	OSC_MULTIPLY_BY_4	Multiply the oscillator frequency by 4	0x3
	OSC_MULTIPLY_BY_3	Multiply the oscillator frequency by 3	0x2
	OSC_MULTIPLY_BY_2	Multiply the oscillator frequency by 2	0x1
	OSC_MULTIPLY_BY_1	Multiply the oscillator frequency by 1	0x0*
ENABLE	OSC_DISABLE	Disable the oscillator	0x0
	OSC_ENABLE	Enable the oscillator	0x1*

18.4.13 A_OSC_CTRL_2 Settings

Bit Field	Field Name	Description
21:16	POS_TEMP_CTRL	Oscillator PTAT compensation control
13:8	NEG_TEMP_CTRL	Oscillator CTAT compensation control
0	TEMP_CTRL_EBL	Oscillator external temperature compensation control enable

Field Name	Value Symbol	Value Description	Hex Value
POS_TEMP_CTRL	POS_TEMP_HIGHEST_CURRENT	Highest current	0x3F
	POS_TEMP_MID_CURRENT	Reset value	0x20*
	POS_TEMP_LOWEST_CURRENT	Lowest current	0x1
NEG_TEMP_CTRL	NEG_TEMP_HIGHEST_CURRENT	Highest current	0x3F
	NEG_TEMP_MID_CURRENT	Reset value	0x20*
	NEG_TEMP_LOWEST_CURRENT	Lowest current	0x1
TEMP_CTRL_EBL	OSC_TEMP_CTRL_DISABLE	External temperature compensation disabled	0x0*
	OSC_TEMP_CTRL_ENABLE	External temperature compensation enabled	0x1

18.4.14 A_OSC_CTRL_3 Settings

Bit Field	Field Name	Description
13:8	POS_TEMP_OUT	Oscillator PTAT output value
5:0	NEG_TEMP_OUT	Oscillator CTAT output value

18.4.15 A_OSC_CTRL_THROTTLING Settings

Bit Field	Field Name	Description
15:0	FREQ_CTRL_THROTTLING	Future clock throttling value for OSC_CTRL_0_FREQ_CTRL. If 0x000 is written to bits [15:4], OSC_CTRL_0_FREQ_CTRL will keep its previous value.

18.4.16 A_OSC_CTRL_THROTTLING_2 Settings

Bit Field	Field Name	Description
15:0	FREQ_CTRL_THROTTLING	Future clock throttling value for OSC_CTRL_0_FREQ_CTRL. If 0x000 is written to bits [15:4], OSC_CTRL_0_FREQ_CTRL will keep its previous value.

18.4.17 A_OSC_CTRL_THROTTLING_3 Settings

Bit Field	Field Name	Description
15:0	FREQ_CTRL_THROTTLING	Future clock throttling value for OSC_CTRL_0_FREQ_CTRL. If 0x000 is written to bits [15:4], OSC_CTRL_0_FREQ_CTRL will keep its previous value.

18.4.18 A_O_SDMCLK Settings

Bit Field	Field Name	Description
5:0	PRESCALE	Prescaler for the SDMCLK to achieve the desired oversampling ratio. SDMCLK=(SLOWCLK_PRESCALE+1)*(SFCR+8)*8/ (SDMCLK_PRESCALE+1); SDMCLK=SYS_CLK/(SDMCLK_PRESCALE+1)

Field Name	Value Symbol	Value Description	Hex Value
PRESCALE	SDMCLK_PRESCALE_1	Divide by 1	0x0*
	SDMCLK_PRESCALE_2	Divide by 2	0x1
	SDMCLK_PRESCALE_3	Divide by 3	0x2
	SDMCLK_PRESCALE_4	Divide by 4	0x3
	SDMCLK_PRESCALE_5	Divide by 5	0x4
	SDMCLK_PRESCALE_6	Divide by 6	0x5
	SDMCLK_PRESCALE_7	Divide by 7	0x6
	SDMCLK_PRESCALE_8	Divide by 8	0x7
	SDMCLK_PRESCALE_9	Divide by 9	0x8
	SDMCLK_PRESCALE_10	Divide by 10	0x9
	SDMCLK_PRESCALE_11	Divide by 11	0xA
	SDMCLK_PRESCALE_12	Divide by 12	0xB
	SDMCLK_PRESCALE_13	Divide by 13	0xC
	SDMCLK_PRESCALE_14	Divide by 14	0xD
	SDMCLK_PRESCALE_15	Divide by 15	0xE
	SDMCLK_PRESCALE_16	Divide by 16	0xF
	SDMCLK_PRESCALE_17	Divide by 17	0x10
	SDMCLK_PRESCALE_18	Divide by 18	0x11
	SDMCLK_PRESCALE_19	Divide by 19	0x12
	SDMCLK_PRESCALE_20	Divide by 20	0x13
	SDMCLK_PRESCALE_21	Divide by 21	0x14
	SDMCLK_PRESCALE_22	Divide by 22	0x15
	SDMCLK_PRESCALE_23	Divide by 23	0x16
	SDMCLK_PRESCALE_24	Divide by 24	0x17

Field Name	Value Symbol	Value Description	Hex Value
PRESCALE (continued)	SDMCLK_PRESCALE_25	Divide by 25	0x18
	SDMCLK_PRESCALE_26	Divide by 26	0x19
	SDMCLK_PRESCALE_27	Divide by 27	0x1A
	SDMCLK_PRESCALE_28	Divide by 28	0x1B
	SDMCLK_PRESCALE_29	Divide by 29	0x1C
	SDMCLK_PRESCALE_30	Divide by 30	0x1D
	SDMCLK_PRESCALE_31	Divide by 31	0x1E
	SDMCLK_PRESCALE_32	Divide by 32	0x1F
	SDMCLK_PRESCALE_33	Divide by 33	0x20
	SDMCLK_PRESCALE_34	Divide by 34	0x21
	SDMCLK_PRESCALE_35	Divide by 35	0x22
	SDMCLK_PRESCALE_36	Divide by 36	0x23
	SDMCLK_PRESCALE_37	Divide by 37	0x24
	SDMCLK_PRESCALE_38	Divide by 38	0x25
	SDMCLK_PRESCALE_39	Divide by 39	0x26
	SDMCLK_PRESCALE_40	Divide by 40	0x27
	SDMCLK_PRESCALE_41	Divide by 41	0x28
	SDMCLK_PRESCALE_42	Divide by 42	0x29
	SDMCLK_PRESCALE_43	Divide by 43	0x2A
	SDMCLK_PRESCALE_44	Divide by 44	0x2B
	SDMCLK_PRESCALE_45	Divide by 45	0x2C
	SDMCLK_PRESCALE_46	Divide by 46	0x2D
	SDMCLK_PRESCALE_47	Divide by 47	0x2E
	SDMCLK_PRESCALE_48	Divide by 48	0x2F
	SDMCLK_PRESCALE_49	Divide by 49	0x30
	SDMCLK_PRESCALE_50	Divide by 50	0x31
	SDMCLK_PRESCALE_51	Divide by 51	0x32
	SDMCLK_PRESCALE_52	Divide by 52	0x33
	SDMCLK_PRESCALE_53	Divide by 53	0x34
	SDMCLK_PRESCALE_54	Divide by 54	0x35
	SDMCLK_PRESCALE_55	Divide by 55	0x36
	SDMCLK_PRESCALE_56	Divide by 56	0x37
	SDMCLK_PRESCALE_57	Divide by 57	0x38
	SDMCLK_PRESCALE_58	Divide by 58	0x39
	SDMCLK_PRESCALE_59	Divide by 59	0x3A
	SDMCLK_PRESCALE_60	Divide by 60	0x3B
	SDMCLK_PRESCALE_61	Divide by 61	0x3C
	SDMCLK_PRESCALE_62	Divide by 62	0x3D

Field Name	Value Symbol	Value Description	Hex Value
PRESCALE (continued)	SDMCLK_PRESCALE_63	Divide by 63	0x3E
	SDMCLK_PRESCALE_64	Divide by 64	0x3F

18.4.19 A_O_SDMCLK_THROTTLING

Bit Field	Field Name	Description
5:0	PRESCALE	Future clock throttling value for O_SDMCLK

CHAPTER 19
Power Components

19.1 Power Supply

The power supply is a critical component of the Ezairo 7100 system, as system performance, particularly audio performance, is greatly affected by the supplied power.

The power supply is powered by a battery supply voltage (VBAT), which is used as the supply for a number of power sources, including:

- A regulated, low-noise voltage reference source for the analog components (VREG)
- Two configurable regulated supplies for digital components (VDDC, VDDM)
- An on-chip charge pump for the analog system components (VDDA)
- An on-chip charge pump for the digital interfaces and external components (VDBL)

The components that make up the power supply can be divided into three types of supply voltages:

1. Power supply input voltages, described further in Section 19.1.1, “Power Supply Inputs”
2. Internal power supply voltages, described further in Section 19.1.2, “Internal Power Supply Voltages”
3. Power supply output voltages, described further in Section 19.1.3, “Power Supply Outputs”

Figure 59 shows a block diagram of the main components of the power supply.

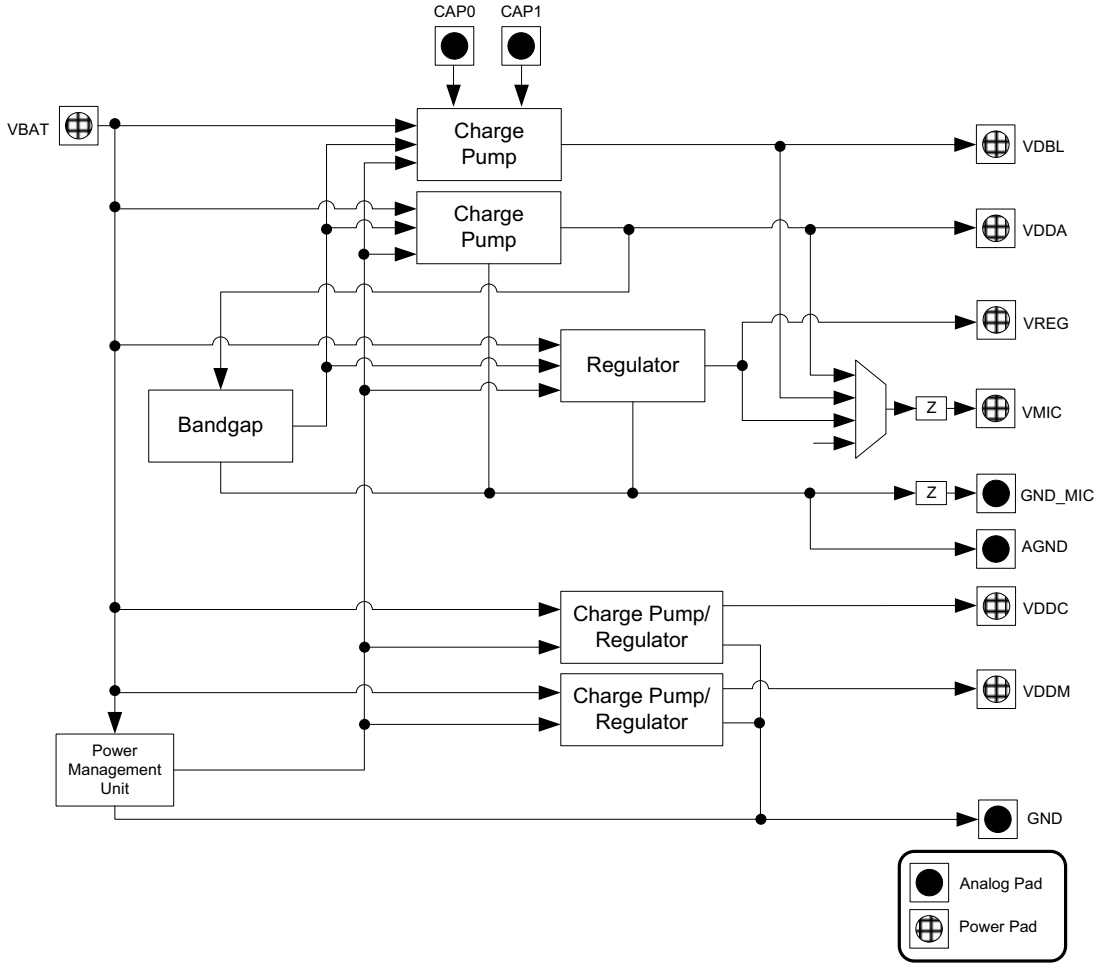


Figure 59. Top-Level Power Supply Block Diagram

19.1.1 Power Supply Inputs

19.1.1.1 Battery Supply Voltage (VBAT)

The primary voltage supplied to an Ezairo 7100 chip is known as the battery supply voltage because in a typical application, this voltage is supplied by a battery.

VBAT directly supplies each of the regulators and charge-pumps included in the Ezairo 7100 chip. This supply voltage is monitored by the power management unit and the battery monitor to ensure safe system operation, and to signal the current voltage level to user applications. See Section 19.2, “Power Management Unit” on page 476, and Section 19.4, “Battery Monitor” on page 479 for more information.

19.1.1.2 Output Receiver Supply Voltage (VBATOD)

To allow maximum flexibility, the direct-digital output drivers from the output stage are powered from the VBATOD pad. Common configurations include connecting this pad externally to VBAT or VDBL based on your system needs.

The sigma-delta modulator in the output stage is a digital system component powered by VDDC (see Section 19.1.2.2, “Internal Digital Supply Voltages (VDDC and VDDM)”).

19.1.1.3 Digital Output Supply Voltages (VDDO1, VDDO2, and VDDO3)

The digital output supply voltages are attached to the digital I/O pads on an Ezairo 7100 chip. This includes:

- All DIO pads, as described in Section 16.1, “Digital I/O (DIO)” on page 351
- The SCL and SDA pads for the debug ports and CFX general-purpose I^2C interface. For more information, see Section 13.1, “Inter-IC Interface (I2C) Bus®” on page 188.
- The EXTCLK I/O pad, described in Section 18.2.6, “External Clock (EXTCLK)” on page 448
- The input pad, described in Section 19.3, “Resets” on page 478

The internal system digital logic is attached to the pads through internal level translators. They shift the voltage level from VDDC to the correct VDDO voltage for digital outputs, and translate input digital signals from the correct VDDO voltage to VDDC for digital inputs. Table 75 provides a mapping of the digital pads to VDDO supply voltages.

Table 75. Digital Supply Mapping

Supply Voltage	Pads
VDDO1	DIO0 to DIO9
VDDO2	DIO10 to DIO19
VDDO3	DIO20 to DIO29
	SCL and SDA, supporting the debug ports and CFX general-purpose I^2C interfaces
	EXTCLK
	NRESET

The VDDO inputs will generally be connected externally to VBAT or VDBL based on the desired communications voltages of digital communications using the associated digital I/O pads. All three VDDO voltages can be supplied from different sources or the same source, as required for communications with external system components.

19.1.2 Internal Power Supply Voltages

19.1.2.1 Internal Band Gap Reference Voltage (VREF)

The Ezairo 7100 chip includes a band gap reference voltage that is stabilized over temperature and process variations. This reference voltage is used in the generation of all of the regulated voltages in the Ezairo 7100 chip and can be used to provide a calibrated 0.97 V reference signal to all components using the reference voltage.

To trim the level of the band gap voltage, use the `BG_LEVEL` bit field from the `A_BG_CTRL` register. Temperature compensation circuitry for the band gap can be configured using the `BG_CTRL_BG_TEMP_COEF_TRIM` bit field from this register; however, use of the BG_TEMP_COEF_0 setting (indicating no additional adjustments are necessary) is recommended.

19.1.2.2 Internal Digital Supply Voltages (VDDC and VDDM)

The Ezairo 7100 chip includes a pair of internally regulated digital supply voltages:

- VDDC is the core digital voltage that is used for most of the digital components of the system.
- VDDM is the memory digital voltage that is used for memories and memory-mapped elements of the Ezairo 7100 system. This voltage must always be greater than or equal to the VDDC voltage for proper

operation. Additionally, if any of the VDDO* voltages exceed 2.1 V, VDDM must be trimmed to a minimum of 1.1 V to ensure the proper level translation of input signals.

IMPORTANT: The VDDC voltage is calibrated to 0.82 V and the VDDM voltage is calibrated to 0.90 V by default, supporting SYSCLK frequencies up to 10.24 MHz. For SYSLCK frequencies below 5.12 MHz and above 10.24 MHz, alternate target voltages for VDDC and VDDM are recommended. For appropriate VDDC and VDDM voltages for a specific configuration, see the Ezairo 7100 data sheet.

Each of the digital supply voltages is generated using a high-efficiency fractional charge pump in parallel with a linear regulator. To configure these fractional charge pumps to be enabled or disabled, use the ENABLE bit from the A_CP_VDDC_CTRL and A_CP_VDDM_CTRL registers.

The VDDC and VDDM fractional charge pumps use internal capacitances that are periodically refreshed to maintain the supply. The refresh frequency for the VDDC and VDDM charge pumps are divided from SYSCLK (see Section 18.2.1, “System Clock (SYSCLK)” on page 445) using the CLK_CFG_CPC_CP_VDDC_CLK_PRESCALE bit field from the D_CLK_CFG_CPC register, and the CLK_CFG_CPM_CP_VDDM_CLK_PRESCALE bit field from the D_CLK_CFG_CPM register, respectively. These clocks must be configured for an update frequency between 1 MHz and 10 MHz, with no restrictions on the duty cycle of the source clock.

For optimal performance, the VDDC and VDDM fractional charge pumps can be configured to use different fractional ratios when converting the supply voltage by using the CP_MODE bit fields from the A_CP_VDDC_CTRL and A_CP_VDDM_CTRL registers. For configuration details, contact ON Semiconductor Customer Support.

IMPORTANT: For more information about calibration and configuration of the internal digital supply voltages, refer to the Calibration Library chapter of the *Firmware Reference Manual for Ezairo 7100*.

19.1.2.3 Regulated Supply Voltage (VREG)

The Ezairo 7100 chip supplies a regulated reference supply voltage at the VREG pad to allow for additional external filtering of the regulated voltages within the system.

19.1.2.4 Analog Supply Voltage (VDDA)

The analog supply voltage makes use of the internal charge pump to generate a configurable regulated supply voltage that is used for all of the analog audio components. In particular this is used for the input stage of the system. This signal is typically configured as a 2.0 V supply voltage.

The regulated, doubled analog supply voltage is accessible at the VDDA pad for capacitive filtering.

The VDDA charge pump can be enabled or disabled using the CP_VDDA_CTRL_EBL bit field from the A_CP_VDDA_CTRL register. The state of the charge pump used to create VDDA can be reset to its power-on reset state using the CP_VDDA_CTRL_NRESET bit field of this same register. The nominal voltage supplied by VDDA can be calibrated using the CP_VDDA_CTRL_VTRIM bit field of the A_CP_VDDA_CTRL register.

The VDDA charge pump uses an internal capacitance that is periodically refreshed to maintain the supply. The refresh frequency for the charge pump is divided from SYSCLK (see Section 18.2.1, “System Clock (SYSCLK)” on page 445) using the CLK_CFG_CPD_CP_VDDA_CLK_PRESCALE bit field from the D_CLK_CFG_CPD register. This clock must be configured for an update frequency between 1 MHz and 10 MHz, with no restrictions on the duty cycle of the source clock.

19.1.3 Power Supply Outputs

19.1.3.1 Regulated Doubled Supply Voltage (VDBL)

The regulated doubled supply voltage makes use of the internal charge pump to generate a configurable regulated supply voltage that is used for communications with external components. It can also be used to power external devices that interface with the Ezairo 7100 chip. This signal is typically configured as a 2.0 V supply voltage, but can be configured to supply between 1.8 V and 2.1 V as needed.

The regulated doubled supply voltage is available at the VDBL output pad. This supply voltage meets the regulated voltage specifications from the Ezairo 7100 data sheet when the system is supplied with a VBAT supply voltage of at least 1.05 V.

The VDBL charge pump can be enabled or disabled using the `CP_VDBL_CTRL_EBL` bit field from the `A_CP_VDBL_CTRL` register. The state of the charge pump used to create VDBL can be reset to its power-on reset state using the `CP_VDBL_CTRL_NRESET` bit field of this same register.

The nominal voltage supplied through VDBL can be trimmed using the `CP_VDBL_CTRL_VTRIM` bit field of the `A_CP_VDBL_CTRL` register. It is also possible to trim the current that can be provided by VDBL, and hence the maximum power output from VDBL, to either supply a maximum of 4 mA (`VDBL_PTRIM_4MA`) or a maximum of 15 mA (`VDBL_PTRIM_16MA`) using the `CP_VDBL_CTRL_PTRIM` bit field from this register.

The current consumed by the VDBL charge pump itself can be trimmed by using the `CP_VDBL_CTRL_ITRIM` bit field from the `A_CP_VDBL_CTRL` register; however, we recommend using the default supply setting of 36 uA for optimal VDBL performance.

The VDBL charge pump uses an external capacitor that is periodically refreshed to maintain the supply. The refresh frequency for the charge pump is divided from SLOWCLK (see Section 18.2.2, “Slow Clock (SLOWCLK)” on page 446) using the `CLK_CFG_CPD_CP_VDBL_CLK_PRESCALE` bit field from the `D_CLK_CFG_CPD` register. If the VDBL charge pump uses the 4 mA output current trim setting, the refresh frequency should be set for a 100 kHz refresh rate. If the application uses the 15 mA output current trim setting, the refresh frequency should be set for a 180 kHz refresh rate. In almost all cases, the divided clock used to refresh VDBL should have a 50% duty cycle.

19.1.3.2 Microphone Supply Voltage (VMIC) and Microphone Ground Reference (GND_MIC)

The Ezairo 7100 chip provides VMIC as a configurable, noise-robust output supply voltage that can be used to supply external microphones. This supply voltage is paired with an associated noise-robust ground pad (GND_MIC) that is intended to be used to connect to the ground side of any external microphone powered from VMIC.

Ezairo 7100 can be configured using the `A_VMIC_SW_CTRL` register to connect the VMIC pad to any of the following sources:

- HiZ (effectively disabling VMIC)
- VREG, see Section 19.1.2.3, “Regulated Supply Voltage (VREG)”
- VDDA, see Section 19.1.2.4, “Analog Supply Voltage (VDDA)”
- VDBL, see Section 19.1.3.1, “Regulated Doubled Supply Voltage (VDBL)”

19.1.4 Control and Configuration Registers

Register Name	Register Description	Address
A_VREG_CTRL	Configuration registers of Voltage Regulator	P:0xF110
A_BG_CTRL	Configuration registers of the bandgap	P:0xF111
A_VMIC_SW_CTRL	VMIC switch control	P:0xF113
A_CP_VDDA_CTRL	Analog charge pump configuration registers	P:0xF120
A_CP_VDBL_CTRL	Interface charge pump configuration registers	P:0xF121
A_CP_VDDC_TRIM	Logic supply trimming register	P:0xF122
A_CP_VDDC_CTRL	Logic supply control register	P:0xF123
A_CP_VDDM_TRIM	Memory supply trimming register	P:0xF124
A_CP_VDDM_CTRL	Memory supply control register	P:0xF125

19.1.4.1 A_VREG_CTRL Settings

Bit Field	Field Name	Description
1	ENABLE	VREG enable control
0	NRESET	VREG regulator reset control

Field Name	Value Symbol	Value Description	Hex Value
ENABLE	VREG_HIZ	Regulator in HiZ mode for external reference	0x0
	VREG_EN	Regulator outputs 1 V	0x1*
NRESET	VREG_RESET	VREG shorted to VBAT	0x0*
	VREG_NO_RESET	Regulator outputs 1 V	0x1

19.1.4.2 A_BG_CTRL Settings

Bit Field	Field Name	Description
12:8	BG_TEMP_COEF_TRIM	Temperature coefficient trimming control
4:0	BG_LEVEL	Reference voltage trimming control

Field Name	Value Symbol	Value Description	Hex Value
BG_TEMP_COEF_TRIM	BG_TEMP_COEF_NEG_205	Temperature dependency -205 ppm/C	0x1F
	BG_TEMP_COEF_NEG_195	Temperature dependency -195 ppm/C	0x1E
	BG_TEMP_COEF_0	Temperature dependency 0 ppm/C	0x10*
	BG_TEMP_COEF_155	Temperature dependency 155 ppm/C	0x1
	BG_TEMP_COEF_165	Temperature dependency 165 ppm/C	0x0

Field Name	Value Symbol	Value Description	Hex Value
BG_LEVEL	BG_TRIM_1_045V	1.045 V	0x1F
	BG_TRIM_1_04V	1.04 V	0x1E
	BG_TRIM_1_035V	1.035 V	0x1D
	BG_TRIM_1_03V	1.03 V	0x1C
	BG_TRIM_1_025V	1.025 V	0x1B
	BG_TRIM_1_02V	1.02 V	0x1A
	BG_TRIM_1_015V	1.015 V	0x19
	BG_TRIM_1_01V	1.01 V	0x18
	BG_TRIM_1_005V	1.005 V	0x17
	BG_TRIM_1_00V	1.00 V	0x16
	BG_TRIM_0_995V	0.995 V	0x15
	BG_TRIM_0_99V	0.99 V	0x14
	BG_TRIM_0_985V	0.985 V	0x13
	BG_TRIM_0_98V	0.98 V	0x12
	BG_TRIM_0_975V	0.975 V	0x11
	BG_TRIM_0_97V	0.97 V	0x10*
	BG_TRIM_0_965V	0.965 V	0xF
	BG_TRIM_0_96V	0.96 V	0xE
	BG_TRIM_0_955V	0.955 V	0xD
	BG_TRIM_0_95V	0.95 V	0xC
	BG_TRIM_0_945V	0.945 V	0xB
	BG_TRIM_0_94V	0.94 V	0xA
	BG_TRIM_0_935V	0.935 V	0x9
	BG_TRIM_0_93V	0.93 V	0x8
	BG_TRIM_0_925V	0.925 V	0x7
	BG_TRIM_0_92V	0.92 V	0x6
	BG_TRIM_0_915V	0.915 V	0x5
	BG_TRIM_0_91V	0.91 V	0x4
	BG_TRIM_0_905V	0.905 V	0x3
	BG_TRIM_0_90V	0.90 V	0x2
	BG_TRIM_0_895V	0.895 V	0x1
	BG_TRIM_0_89V	0.89 V	0x0

19.1.4.3 A_VMIC_SW_CTRL Settings

Bit Field	Field Name	Description
1:0	VMIC_CONTROL	Selection of the state of VMIC

Field Name	Value Symbol	Value Description	Hex Value
VMIC_CONTROL	VMIC_SW_OFF	All switches are off. VMIC output is HiZ	0x0*
	VMIC_SW_VREG	VMIC output is VREG	0x1
	VMIC_SW_VDDA	VMIC output is VDDA	0x2
	VMIC_SW_VDBL	VMIC output is VDBL	0x3

19.1.4.4 A_CP_VDDA_CTRL Settings

Bit Field	Field Name	Description
17	NRESET	Reset the analog charge pump analog
16	EBL	Enable/disable the analog charge pump
5:0	TRIM	CP output voltage trimming configuration at a step of 10 mV

Field Name	Value Symbol	Value Description	Hex Value
NRESET	VDDA_RESET	Reset the charge pump	0x0*
	VDDA_NO_RESET	No charge pump reset	0x1
EBL	VDDA_DISABLE	Disable the charge pump	0x0*
	VDDA_ENABLE	Enable the charge pump	0x1
TRIM	VDDA_TRIM_1_52V	Charge pump outputs 1.52 V	0x0
	VDDA_TRIM_2_0V	Charge pump outputs 2.00 V	0x32*
	VDDA_TRIM_2_13V	Charge pump outputs 2.13 V	0x3F

19.1.4.5 A_CP_VDBL_CTRL Settings

Bit Field	Field Name	Description
17	NRESET	Resets interface charge pump
16	EBL	Enables interface charge pump
13:12	PTRIM	Output power trimming
10:8	ITRIM	(0b000=12 uA, 0b111=36 uA in 3 uA steps)
5:0	VTRIM	CP output voltage trimming configuration at a step of 10 mV

Field Name	Value Symbol	Value Description	Hex Value
NRESET	VDBL_RESET	Reset the charge pump	0x0*
	VDBL_NO_RESET	No charge pump reset	0x1
EBL	VDBL_DISABLE	Disable the charge pump	0x0*
	VDBL_ENABLE	Enable the charge pump	0x1
PTRIM	VDBL_PTRIM_4MA	Charge pump max current to 4 mA	0x0*
	VDBL_PTRIM_16MA	Charge pump max current to 15 mA	0x3

Field Name	Value Symbol	Value Description	Hex Value
ITRIM	VDBL_TRIM_12UA	12 uA	0x0
	VDBL_TRIM_36UA	36 uA	0x7*
VTRIM	VDBL_TRIM_1_52V	Charge pump outputs 1.52 V	0x0
	VDBL_TRIM_1_8V	Charge pump outputs 1.80 V	0x1E
	VDBL_TRIM_2_0V	Charge pump outputs 2.00 V	0x32*
	VDBL_TRIM_2_13V	Charge pump outputs 2.13 V	0x3F

19.1.4.6 A_CP_VDDC_TRIM

Bit Field	Field Name	Description
15:8	CP_TRIM	Charge pump output voltage trimming voltage in 2.5 mV steps
7:0	LDO_TRIM	Linear regulator trimming voltage in 2.5 mV steps

Field Name	Value Symbol	Value Description	Hex Value
CP_TRIM	VDDC_VOLTAGE_0P70	0.70 V (uncalibrated)	0x0
	VDDC_VOLTAGE_1P00	1.00 V (uncalibrated)	0x78*
	VDDC_VOLTAGE_1P34	1.34 V (uncalibrated)	0xFF
LDO_TRIM	VDDC_LDO_0P70	0.70 V (uncalibrated)	0x0
	VDDC_LDO_1P00	1.00 V (uncalibrated)	0x78*
	VDDC_LDO_1P34	1.34 V (uncalibrated)	0xFF

19.1.4.7 A_CP_VDDC_CTRL

Bit Field	Field Name	Description
6:4	CP_MODE	Charge pump mode
1	VLOW_EN	Enable VLOW detection as reset source (at 90% of LDO voltage)
0	ENABLE	Charge pump enable control

Field Name	Value Symbol	Value Description	Hex Value
CP_MODE	VDDC_RATIO_4_5	4/5 ratio mode	0x0*
	VDDC_RATIO_3_4	3/4 ratio mode	0x1
	VDDC_RATIO_2_3	2/3 ratio mode	0x2
	VDDC_RATIO_1_2	1/2 ratio mode	0x3
	VDDC_RATIO_5_4	5/4 ratio mode	0x4
	VDDC_RATIO_4_3	4/3 ratio mode	0x5
	VDDC_RATIO_3_2	3/2 ratio mode	0x6
	VDDC_RATIO_2_1	2/1 ratio mode	0x7
VLOW_EN	VDDC_VLOW_DISABLE	Ignore VLOW detection	0x0*
	VDDC_VLOW_ENABLE	Enable VLOW detection	0x1

Field Name	Value Symbol	Value Description	Hex Value
ENABLE	VDDC_DISABLE	Disable the charge pump	0x0*
	VDDC_ENABLE	Enable the charge pump	0x1

19.1.4.8 A_CP_VDDM_TRIM

Bit Field	Field Name	Description
15:8	CP_TRIM	Charge pump output voltage trimming voltage in 2.5 mV steps
7:0	LDO_TRIM	Linear regulator trimming voltage in 2.5 mV steps

Field Name	Value Symbol	Value Description	Hex Value
CP_TRIM	VDDM_VOLTAGE_0P70	0.70 V (uncalibrated)	0x0
	VDDM_VOLTAGE_1P00	1.00 V (uncalibrated)	0x78*
	VDDM_VOLTAGE_1P34	1.34 V (uncalibrated)	0xFF
LDO_TRIM	VDDM_LDO_0P70	0.70 V (uncalibrated)	0x0
	VDDM_LDO_1P00	1.00 V (uncalibrated)	0x78*
	VDDM_LDO_1P34	1.34 V (uncalibrated)	0xFF

19.1.4.9 A_CP_VDDM_CTRL

Bit Field	Field Name	Description
6:4	CP_MODE	Charge pump mode
1	VLOW_EN	Enable V_{low} detection as reset source (at 90% of LDO voltage)
0	ENABLE	Charge pump enable control

Field Name	Value Symbol	Value Description	Hex Value
CP_MODE	VDDM_RATIO_4_5	4/5 ratio mode	0x0*
	VDDM_RATIO_3_4	3/4 ratio mode	0x1
	VDDM_RATIO_2_3	2/3 ratio mode	0x2
	VDDM_RATIO_1_2	1/2 ratio mode	0x3
	VDDM_RATIO_5_4	5/4 ratio mode	0x4
	VDDM_RATIO_4_3	4/3 ratio mode	0x5
	VDDM_RATIO_3_2	3/2 ratio mode	0x6
	VDDM_RATIO_2_1	2/1 ratio mode	0x7
VLOW_EN	VDDM_VLOW_DISABLE	Ignore VLOW detection	0x0*
	VDDM_VLOW_ENABLE	Enable VLOW detection	0x1
ENABLE	VDDM_DISABLE	Disable the charge pump	0x0*
	VDDM_ENABLE	Enable the charge pump	0x1

19.2 Power Management Unit

The power management unit consists primarily of two components:

- A power supervisory circuit to ensure safe system operation under any battery condition
- A power-on reset (POR) component that ensures startup behavior and configuration

These components work together to ensure proper system behavior under all battery conditions. They also ensure that the system startup sequence behaves correctly as voltage fluctuations related to system startup might otherwise result in an indeterminate system state. Neither component is user configurable. Further information about each of these components is provided in the following sections.

19.2.1 Power Supervisory Unit

The power supervisory unit monitors the battery supply voltage (VBAT) and optionally, the internal digital supply voltages (VDDC, VDDM), safely shutting down the system without user intervention when the supply voltages are below the thresholds required for valid system operation. This unit is used to ensure that the system operates correctly and is not damaged by the power supply fluctuations that are encountered when a battery is inserted or removed from a hearing aid.

When power is first supplied to the system, the power supervisory circuitry starts to monitor the battery supply voltage as it crosses the low VBAT threshold. If the voltage continues to rise past the high VBAT voltage threshold, and stays above the high VBAT threshold for an appropriate period of time, the system releases the reset signal and uses the power-on-reset module (see Section 19.2.2, “Power-On-Reset (POR) Sequence”) to ensure proper system startup behavior. This startup sequence is shown in Figure 60.

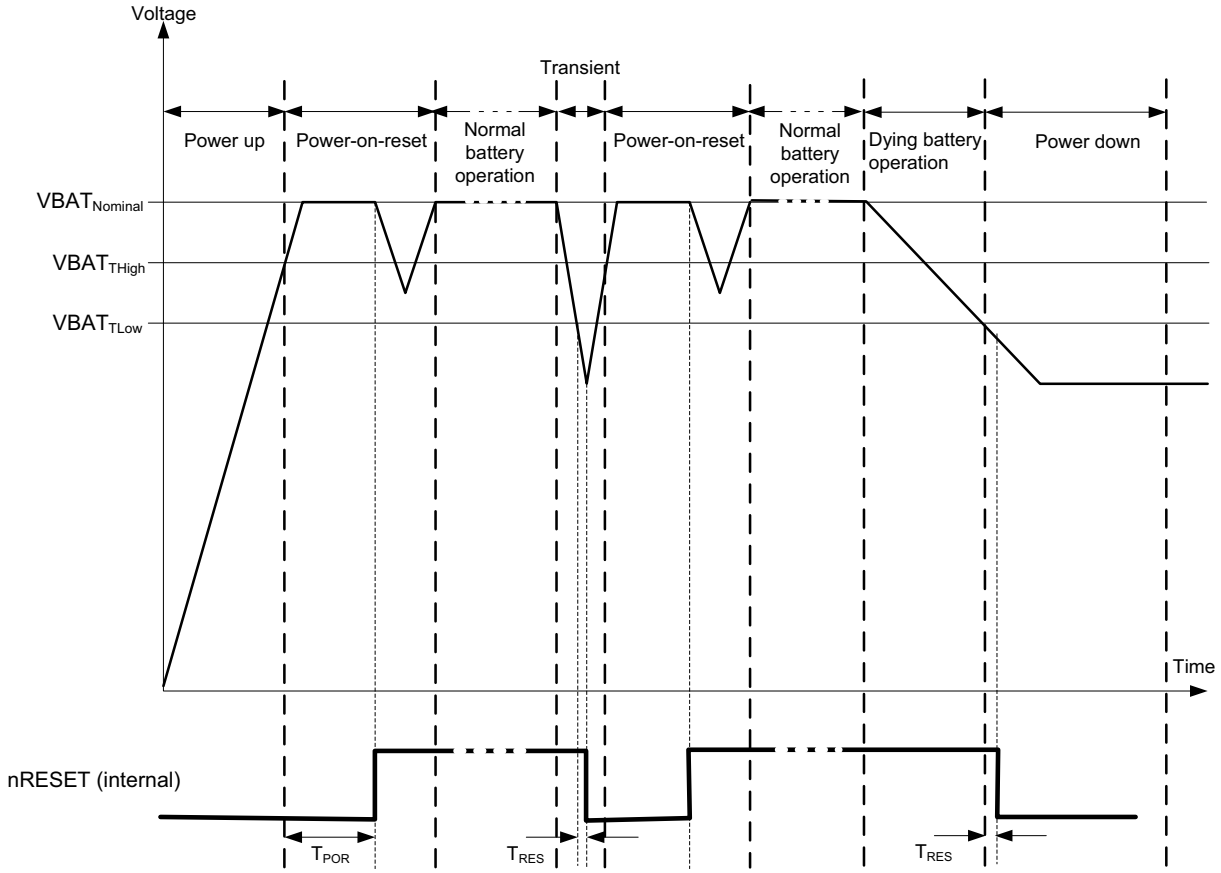


Figure 60. Power Supervisory Behavior

If VBAT falls below the low voltage threshold due to a voltage transient or dying battery, the system is returned to a reset state. The system is held in a reset state until the voltage recovers, and VBAT is above the high voltage threshold for the POR delay. These possible transitions are also shown in Figure 60.

The relevant threshold voltages and nominal POR delay are supplied in Table 76.

Table 76. Power Supervisory Parameters

Parameter	Level
Nominal POR Delay	12 ms
VBAT High Threshold	0.90 V ± 50 mV
VBAT Low Threshold	0.88 V ± 50 mV

NOTE: The voltage difference between the high and low VBAT voltage thresholds is used to prevent repeated system resets due to voltage oscillation around the low voltage threshold.

Monitoring of VDDC and VDDM is enabled by setting the `VLOW_EN` bit from the `A_CP_VDDC_CTRL` and `A_CP_VDDM_CTRL` registers respectively. When enabled, this monitor triggers a power supervisor reset when the

detected VDDC or VDDM voltage falls below 90% of the expected output (as determined by the VDD* LDO configuration). For more information about VDDC and VDDM, see Section 19.1.2.2, “Internal Digital Supply Voltages (VDDC and VDDM)” on page 468.

19.2.2 Power-On-Reset (POR) Sequence

The Ezairo 7100 device uses a power-on-reset (POR) sequence to ensure proper system behavior during startup, and to ensure proper system configuration after startup.

At the start of the POR sequence, audio output is disabled and all configuration and control registers are asynchronously reset to their default values (as specified in this manual). At the start of the POR sequence, the core registers for all four of the system’s cores are cleared and the contents of all RAM is unspecified.

The POR sequence consists of two components: voltage supply stabilization and boot ROM initialization. During the voltage supply stabilization component, the internal VDDC and VDDM regulators are enabled and allowed to stabilize.

The boot ROM initialization component is then run, initializing and enabling the VDDA and VDBL charge pumps, amongst other components. Following a stabilization delay (nominally 140 ms), the POR sequence is complete, and the device attempts to load data from an EEPROM using SPI on DIO0 to DIO3. For more information about:

- The boot ROM, see the Boot ROM section of the *Firmware Reference Manual for Ezairo 7100*
- The CFX SPI, see Section 16.4, “Serial Peripheral Interface (SPI)” on page 395
- DIO configuration, see Section 16.1, “Digital I/O (DIO)” on page 351

19.3 Resets

The Ezairo 7100 system contains a variety of reset sources that can be used to reset the whole Ezairo 7100 system, or a set of system components.

Exiting from any full system reset triggers the POR sequence, as described in Section 19.2.2, “Power-On-Reset (POR) Sequence”. Full system resets can be triggered by:

- The power supervisory
- The watchdog timer (see Section 14.3, “Watchdog Timer” on page 230)
- The dedicated nRESET pad

Partial resets supported by the Ezairo 7100 system include:

- A hard reset of a core, which resets the core registers and potentially the registers for its peripherals and interfaces. These resets can be triggered:
  - For the HEAR and its peripherals (including the FIFOs) by setting the `SYSTEM_CTRL_HEAR_RESET` bit from the `D_SYSTEM_CTRL` register
  - For just the ARM Cortex-M3 processor by setting the `SYSTEM_CTRL_2_CM3_RESET` bit from the `D_SYSTEM_CTRL_2` register
  - For the ARM Cortex-M3 processor, its interfaces, and its peripherals by setting the `SYSTEM_CTRL_2_CM3_SUBSYSTEM_RESET` bit from the `D_SYSTEM_CTRL_2` register
  - For the Filter Engine by setting the `FENG_CMDS_DECODER_RESET` and `FENG_CMDS_AUDIO_RESET` bits from the `D_FENG_CMDS` register

- A soft reset of a core, which resets only the core registers for a core. These resets can be triggered:

- For the CFX and its peripherals by triggering a CFX reset interrupt (CFX interrupt vector 0)
- For the ARM Cortex-M3 processor by triggering a reset interrupt in the NVIC (NVIC interrupt vector 1)
- For the ARM Cortex-M3 processor by requesting a reset through the ARM Cortex-M3 processor's debug port, debug monitor, or using the VECTRESET bit from the AIRCR register, as described in Section 15.1.8, "Application Interrupt and Reset Control Register" on page 247

19.4 BATTERY MONITOR

The Ezairo 7100 device includes a battery monitor that can be used to signal a user application that the VBAT supply voltage has fallen below a specified voltage threshold.

To specify the value of this threshold voltage, set the value of the A_SUP_THRSH_DATA register. This register allows you to specify any voltage threshold between 0 V and 2 V in 7.8 mV steps. The default voltage threshold is 1.0 V, corresponding to a setting of 0x80 for the A_SUP_THRSH_DATA register.

If the supplied VBAT voltage stays below the threshold voltage, the battery monitor increments at an interval determined by the speed of the LSAD interface (and hence this interval is indirectly dependent on SLOWCLK as well). Under default system settings, the update interval is 80 ms and the exact battery monitor update frequency can be calculated using:

$$f_{\text{Battery Monitor}} = \frac{1}{1024} \times \left[\frac{f_{\text{SLOWCLK}}}{20 \times (\text{LSAD_CTRL_FREQ} + 1)} \right]$$

CAUTION: In soft power down mode with the analog components disabled, the battery monitor is also disabled because it depends on the LSAD components being active.

The count of the number of times that the supply voltage has dropped below the specified voltage threshold is provided through the A_SUP_COUNT_DATA register. This read-only register provides the current battery monitor count.

IMPORTANT: A_SUP_COUNT_DATA is reset when it is read.

19.4.1 Control and Configuration Registers

Register Name	Register Description	Address
A_SUP_THRSH_DATA	Threshold value for battery monitor	P:0xF190
A_SUP_COUNT_DATA	Battery voltage below SUP_THRESHOLD counter	P:0xF191

19.4.1.1 A_SUP_THRSH_DATA Settings

Bit Field	Field Name	Description
7:0	SUP_THRSH_DATA	Battery monitor (LSAD_VBAT) low-voltage detection threshold

Field Name	Value Symbol	Value Description	Hex Value
SUP_THRSH_DATA	SUP_THRESH_1P996V	Threshold is 1.996 V	0xFF
	SUP_THRESH_1P000V	Threshold is 1.0 V	0x80*
	SUP_THRESH_0P000V	Threshold is 0 V	0x0

19.4.1.2 A_SUP_COUNT_DATA Settings

Bit Field	Field Name	Description
7:0	SUP_COUNT_DATA	Number of times the battery voltage has fallen below the battery monitor voltage threshold (reading the register resets the counter)

19.5 Soft Power Down Mode Tools

The Ezairo 7100 device provides an ultra-low power mode where the cores and other digital components of the system are still active but are being clocked from a SYSCLK that is using the standby clock and the analog circuitry is disabled. Since the CFX is still active, an event on the GPIO interface or another system element can trigger a reactivation of the system from soft power down mode.

To reduce the power consumed during soft power down mode, the analog components of the Ezairo 7100 device can be disconnected from their voltage supplies. To disable power to these components, set the ANALOG_PWR_CTRL_ANALOG_DISABLE bit in the ANALOG_PWR_CTRL register.

As the digital interfaces are still operative in soft power down mode, disconnect system elements such as the IOC from the input and output stages when in soft power down mode to prevent unexpected behavior upon restoration of the normal clock. The I^2C debug port interface bus is disconnected by default when in soft power down mode, unless the I2C_CFG_CONNECT_IN_STANDBY bit from the D_I2C_CFG register is set (for more information about configuring the I^2C debug ports and their bus, see Chapter 13, “Debug Ports and CFX General-Purpose I2C” on page 188). Other interfaces that are not explicitly disabled remain active, although they cannot operate at their typical communication frequencies since all interface clocks are scaled down proportionally with SYSCLK.

Use the following sequence of tasks to enter soft power down mode:

1. Switch the source for SYSCLK to the standby clock. For more information about changing the source of SYSCLK, see Section 18.2.1, “System Clock (SYSCLK)” on page 445.
2. Disable the analog blocks from the input and output stages by setting the ANALOG_PWR_CTRL_ANALOG_DISABLE bit in the ANALOG_PWR_CTRL control register as described above.

IMPORTANT: You must follow the above order because the power supplied to the primary oscillator is disabled along with the rest of the analog circuitry and cannot be disabled unless the system is operating using the standby clock. For similar reasons, when re-enabling the system in response to an event, enable the analog circuitry before restoring the normal SYSCLK.

19.5.1 Control and Configuration Registers

Register Name	Register Description	Address
A_ANALOG_PWR_CTRL	Analog power down control	P:0xF112

19.5.1.1 A_ANALOG_PWR_CTRL Settings

Bit Field	Field Name	Description
0	ANALOG_DISABLE	Analog power down control

Field Name	Value Symbol	Value Description	Hex Value
ANALOG_DISABLE	ANALOG_DISABLE	All analog blocks except the PMU are disabled	0x1
	ANALOG_ENABLE	Analog blocks are enabled	0x0*

APPENDIX A
Control and Configuration Registers

This appendix lists all the registers that are available. Refer to the appropriate section for information about the control and configuration registers for a block. The sections are:

- Section A.1, “Error Correction Coding (ECC) Configuration and Control” on page 484
- Section A.2, “Audio Codec Block (ACB) Configuration and Control” on page 488
- Section A.3, “DMA Controller Configuration and Control” on page 491
- Section A.4, “External Interrupt Configuration and Control” on page 498
- Section A.5, “DIO Configuration and Control” on page 499
- Section A.6, “SPI Interface Configuration and Control” on page 500
- Section A.7, “PCM Interface Configuration and Control” on page 501
- Section A.8, “I2C Interface Configuration and Control” on page 502
- Section A.9, “CRC Generator Control” on page 504
- Section A.10, “SYSTICK Timer” on page 505
- Section A.11, “Nested Vector Interrupt Controller” on page 506
- Section A.12, “Debug Controller” on page 521
- Section A.13, “System Control” on page 522
- Section A.14, “Watchdog Timer Configuration” on page 523
- Section A.15, “General Purpose Timers 1 and 2” on page 524
- Section A.16, “HEAR Function Chain Commands” on page 525
- Section A.17, “HEAR Interrupt Multiplexing” on page 526
- Section A.18, “CFX/ARM Cortex-M3 Processor Control and Configuration” on page 527
- Section A.19, “Memory Configuration” on page 528
- Section A.20, “External Breakpoint Configuration” on page 530
- Section A.21, “Hardware CRC” on page 531
- Section A.22, “Chip Version Information (Read-only)” on page 532
- Section A.23, “Debug Port Configuration” on page 533
- Section A.24, “Extended Configuration” on page 534
- Section A.25, “Interrupt Configuration” on page 535
- Section A.26, “Clock Configuration” on page 544
- Section A.27, “IOC Input Configuration” on page 546
- Section A.28, “IOC FIFO Configuration” on page 547
- Section A.29, “IOC Output Configuration” on page 549
- Section A.30, “A/D and D/A Converter Access” on page 550
- Section A.31, “Oscillator Control” on page 551
- Section A.32, “Analog” on page 553
- Section A.33, “Charge Pump” on page 554
- Section A.34, “A/D Converter Control” on page 555
- Section A.35, “Output Stage Control” on page 559
- Section A.36, “Audio Mux Configuration” on page 561
- Section A.37, “DMIC Output Control” on page 562
- Section A.38, “Low-speed A/D” on page 563
- Section A.39, “Low-voltage Detection” on page 565
- Section A.40, “Low-voltage Count” on page 566
- Section A.41, “SPI Interface” on page 567
- Section A.42, “I2C Interface” on page 568
- Section A.43, “PCM Interface” on page 570
- Section A.44, “DIO Interface” on page 571
- Section A.45, “UART Interface” on page 578

- Section A.46, “Filter Engine Control and Configuration” on page 579
- Section A.47, “Filter Engine Register File” on page 587
- Section A.48, “Signal Detection Unit” on page 590
- Section A.49, “HEAR FIFO A0 Configuration and Control” on page 591
- Section A.50, “HEAR FIFO A1 Configuration and Control” on page 594
- Section A.51, “HEAR FIFO B0 Configuration and Control” on page 597
- Section A.52, “HEAR FIFO B1 Configuration and Control” on page 600
- Section A.53, “CFX FIFO Interrupt Selection” on page 603
- Section A.54, “HEAR FIFO Interrupt Selection” on page 604
- Section A.55, “ARM Cortex-M3 Core FIFO Interrupt Selection” on page 605
- Section A.56, “HEAR Interrupt Configuration and Control” on page 606
- Section A.57, “Block Floating Point Configuration and Control” on page 610

A.1 Error Correction Coding (ECC) Configuration and Control

Address	Register Name	Register Write	Register Read	Default	Description
0x40000000	ECB_INT_STATUS	(25) IHM_DEC_OUT_ERR	(25) IHM_DEC_OUT_ERR	0x0	The Interleave and Hamming decoder output interface error interrupt status
P:0x3000	C_ECB_INT_STATUS	(24) IHM_DEC_IN_ERR	(24) IHM_DEC_IN_ERR	0x0	The Interleave and Hamming decoder input interface error interrupt status
		(17) HMI_ENC_OUT_ERR	(17) HMI_ENC_OUT_ERR	0x0	The Hamming and Interleave encoder output interface error interrupt status
		(16) HMI_ENC_IN_ERR	(16) HMI_ENC_IN_ERR	0x0	The Hamming and Interleave encoder input interface error interrupt status
		(10) RS_DEC_MISS_ERR	(10) RS_DEC_MISS_ERR	0x0	The Reed Solomon decoder missed error interrupt status
		(9) RS_DEC_BLK_END	(9) RS_DEC_BLK_END	0x0	The Reed Solomon decoder block end interrupt status
		(8) RS_DEC_IN_ERR	(8) RS_DEC_IN_ERR	0x0	The Reed Solomon decoder output interface error interrupt status
		(1) RS_ENC_BLK_END	(1) RS_ENC_BLK_END	0x0	The Reed Solomon encoder block end interrupt status
		(0) RS_ENC_IN_ERR	(0) RS_ENC_IN_ERR	0x0	The Reed Solomon encoder input interface error interrupt status
0x40000004	ECB_INT_ENABLE	(25) IHM_DEC_OUT_ERR	(25) IHM_DEC_OUT_ERR	0x0	The Interleave and Hamming decoder output interface error interrupt mask
P:0x3001	C_ECB_INT_ENABLE	(24) IHM_DEC_IN_ERR	(24) IHM_DEC_IN_ERR	0x0	The Interleave and Hamming decoder input interface error interrupt mask
		(17) HMI_ENC_OUT_ERR	(17) HMI_ENC_OUT_ERR	0x0	The Hamming and Interleave encoder output interface error interrupt mask
		(16) HMI_ENC_IN_ERR	(16) HMI_ENC_IN_ERR	0x0	The Hamming and Interleave encoder input interface error interrupt mask
		(10) RS_DEC_MISS_ERR	(10) RS_DEC_MISS_ERR	0x0	The Reed Solomon decoder missed error interrupt mask
		(9) RS_DEC_BLK_END	(9) RS_DEC_BLK_END	0x0	The Reed Solomon decoder block end interrupt mask
		(8) RS_DEC_IN_ERR	(8) RS_DEC_IN_ERR	0x0	The Reed Solomon decoder output interface error interrupt mask
		(1) RS_ENC_BLK_END	(1) RS_ENC_BLK_END	0x0	The Reed Solomon encoder block end interrupt mask
		(0) RS_ENC_IN_ERR	(0) RS_ENC_IN_ERR	0x0	The Reed Solomon encoder input interface error interrupt mask
0x40000008	ECB_CTRL	(28) IHM_DEC_EBL	(28) IHM_DEC_EBL	0x0	Enable/disable the Interleave and Hamming decoder

Address	Register Name	Register Write	Register Read	Default	Description
P:0x3002	C_ECB_CTRL	(27) IHM_COR_EBL	(27) IHM_COR_EBL	0x1	Hamming Decoder error correction enable
		(26) IHM_INTERLEAVE_EBL	(26) IHM_INTERLEAVE_EBL	0x1	Enable/disable the de-interleaver
		(25) IHM_INTERLEAVE_MODE	(25) IHM_INTERLEAVE_MODE	0x0	De-Interleaver mode
		(24) IHM_DMA_MODE	(24) IHM_DMA_MODE	0x0	The Interleave and Hamming decoder DMA mode
		(20) HMI_ENC_EBL	(20) HMI_ENC_EBL	0x0	Hamming encoder enable
		(19) HMI_PARITY_EBL	(19) HMI_PARITY_EBL	0x1	Hamming encoder parity generator enable
		(18) HMI_INTERLEAVE_EBL	(18) HMI_INTERLEAVE_EBL	0x1	Enable/disable the interleaver
		(17) HMI_INTERLEAVE_MODE	(17) HMI_INTERLEAVE_MODE	0x0	Interleaver mode
		(16) HMI_DMA_MODE	(16) HMI_DMA_MODE	0x0	The Hamming and Interleave encoder DMA mode
		(13) RS_DEC_IN_PTR_DIR	(13) RS_DEC_IN_PTR_DIR	0x0	Reed Solomon decoder input register pointer update direction
		(12) RS_DEC_OUT_PTR_DIR	(12) RS_DEC_OUT_PTR_DIR	0x0	Reed Solomon decoder output register pointer update direction
		(11:10) RS_DEC_IN_PTR_MODE	(11:10) RS_DEC_IN_PTR_MODE	0x0	Reed Solomon decoder input register pointer mode
		(9:8) RS_DEC_OUT_PTR_MODE	(9:8) RS_DEC_OUT_PTR_MODE	0x0	Reed Solomon decoder output register pointer mode
		(5) RS_ENC_IN_PTR_DIR	(5) RS_ENC_IN_PTR_DIR	0x0	Reed Solomon encoder input register pointer update direction
		(4) RS_ENC_OUT_PTR_DIR	(4) RS_ENC_OUT_PTR_DIR	0x0	Reed Solomon encoder output register pointer update direction
		(3:2) RS_ENC_IN_PTR_MODE	(3:2) RS_ENC_IN_PTR_MODE	0x0	Reed Solomon encoder input register pointer mode
		(1:0) RS_ENC_OUT_PTR_MODE	(1:0) RS_ENC_OUT_PTR_MODE	0x0	Reed Solomon encoder output register pointer mode
0x4000000C	ECB_RS_BLK_CNT	(15:8) RS_DEC_BLK_CNT	(15:8) RS_DEC_BLK_CNT	0x0	Reed Solomon decoder block size to count down from (255 is continuous run)
P:0x3003	C_ECB_RS_BLK_CNT	(7:0) RS_ENC_BLK_CNT	(7:0) RS_ENC_BLK_CNT	0x0	Reed Solomon encoder block size to count down from (255 is continuous run)
0x40000010	ECB_CMD	(8) RS_DEC_CLR	-	N/A	Reed Solomon decoder clear
P:0x3004	C_ECB_CMD	(0) RS_ENC_CLR	-	N/A	Reed Solomon encoder clear
0x40000014	ECB_RS_ENC_IN	(31:0) RS_ENC_IN	(31:0) RS_ENC_IN	0x0	The RS encoder input register
P:0x3005	C_ECB_RS_ENC_IN				
0x40000018	ECB_RS_ENC_OUT	(31:0) RS_ENC_OUT	(31:0) RS_ENC_OUT	0x0	The RS encoder output register
P:0x3006	C_ECB_RS_ENC_OUT				
0x4000001C	ECB_RS_DEC_IN	(31:0) RS_DEC_IN	(31:0) RS_DEC_IN	0x0	The RS decoder input register
P:0x3007	C_ECB_RS_DEC_IN				
0x40000020	ECB_RS_DEC_OUT	(31:0) RS_DEC_OUT	(31:0) RS_DEC_OUT	0x0	The RS decoder output register
P:0x3008	C_ECB_RS_DEC_OUT				
0x40000024	ECB_RS_PTR	(13:11) RS_DEC_IN_PTR	(13:11) RS_DEC_IN_PTR	0x0	The RS decoder input pointer value

Address	Register Name	Register Write	Register Read	Default	Description
P:0x3009	C_ECB_RS_PTR	(10:8) RS_DEC_OUT_PTR	(10:8) RS_DEC_OUT_PTR	0x0	The RS decoder output pointer value
		(5:3) RS_ENC_IN_PTR	(5:3) RS_ENC_IN_PTR	0x0	The RS encoder input pointer value
		(2:0) RS_ENC_OUT_PTR	(2:0) RS_ENC_OUT_PTR	0x0	The RS encoder output pointer value
0x40000028 P:0x300A	ECB_HMI_IN0 C_ECB_HMI_IN0	(31:0) HMI_IN0	(31:0) HMI_IN0	0x0	HMI input data word 0
0x4000002C P:0x300B	ECB_HMI_IN1 C_ECB_HMI_IN1	(31:0) HMI_IN1	(31:0) HMI_IN1	0x0	HMI input data word 1
0x40000030 P:0x300C	ECB_HMI_IN2 C_ECB_HMI_IN2	(31:0) HMI_IN2	(31:0) HMI_IN2	0x0	HMI input data word 2
0x40000034 P:0x300D	ECB_HMI_IN3 C_ECB_HMI_IN3	(31:0) HMI_IN3	(31:0) HMI_IN3	0x0	HMI input data word 3
0x40000038 P:0x300E	ECB_HMI_OUT0 C_ECB_HMI_OUT0	-	(31:0) HMI_OUT0	0x0	HMI output data word 0
0x4000003C P:0x300F	ECB_HMI_OUT1 C_ECB_HMI_OUT1	-	(31:0) HMI_OUT1	0x0	HMI output data word 1
0x40000040 P:0x3010	ECB_HMI_OUT2 C_ECB_HMI_OUT2	-	(31:0) HMI_OUT2	0x0	HMI output data word 2
0x40000044 P:0x3011	ECB_HMI_OUT3 C_ECB_HMI_OUT3	-	(31:0) HMI_OUT3	0x0	HMI output data word 3
0x40000048 P:0x3012	ECB_IHM_IN0 C_ECB_IHM_IN0	(31:0) IHM_IN0	(31:0) IHM_IN0	0x0	IHM input data word 0
0x4000004C P:0x3013	ECB_IHM_IN1 C_ECB_IHM_IN1	(31:0) IHM_IN1	(31:0) IHM_IN1	0x0	IHM input data word 1
0x40000050 P:0x3014	ECB_IHM_IN2 C_ECB_IHM_IN2	(31:0) IHM_IN2	(31:0) IHM_IN2	0x0	IHM input data word 2
0x40000054 P:0x3015	ECB_IHM_IN3 C_ECB_IHM_IN3	(31:0) IHM_IN3	(31:0) IHM_IN3	0x0	IHM input data word 3
0x40000058 P:0x3016	ECB_IHM_OUT0 C_ECB_IHM_OUT0	-	(31:0) IHM_OUT0	0x0	IHM output data word 0
0x4000005C P:0x3017	ECB_IHM_OUT1 C_ECB_IHM_OUT1	-	(31:0) IHM_OUT1	0x0	IHM output data word 1
0x40000060 P:0x3018	ECB_IHM_OUT2 C_ECB_IHM_OUT2	-	(31:0) IHM_OUT2	0x0	IHM output data word 2
0x40000064 P:0x3019	ECB_IHM_OUT3 C_ECB_IHM_OUT3	-	(31:0) IHM_OUT3	0x0	IHM output data word 3

Address	Register Name	Register Write	Register Read	Default	Description
0x40000068	ECB_STATUS	-	(19) IHM_EDSR_BLK5	0x0	Hamming block 5 error detection but no correction status
P:0x301A	C_ECB_STATUS	-	(18) IHM_EDCSR_BLK5	0x0	Hamming block 5 error detection and correction status
		-	(17) IHM_EDSR_BLK4	0x0	Hamming block 4 error detection but no correction status
		-	(16) IHM_EDCSR_BLK4	0x0	Hamming block 4 error detection and correction status
		-	(15) IHM_EDSR_BLK3	0x0	Hamming block 3 error detection but no correction status
		-	(14) IHM_EDCSR_BLK3	0x0	Hamming block 3 error detection and correction status
		-	(13) IHM_EDSR_BLK2	0x0	Hamming block 2 error detection but no correction status
		-	(12) IHM_EDCSR_BLK2	0x0	Hamming block 2 error detection and correction status
		-	(11) IHM_EDSR_BLK1	0x0	Hamming block 1 error detection but no correction status
		-	(10) IHM_EDCSR_BLK1	0x0	Hamming block 1 error detection and correction status
		-	(9) IHM_EDSR_BLK0	0x0	Hamming block 0 error detection but no correction status
		-	(8) IHM_EDCSR_BLK0	0x0	Hamming block 0 error detection and correction status
		-	(7:6) RS_DEC_BLK_ERR	0x0	The RS decoder error count in the last block
		-	(5:0) RS_DEC_ACC_ERR	0x0	The RS decoder accumulated error count
0x4000006C	ECB_IHM_ECP	-	(29:25) BLK5	0x0	The Hamming decoder block 5 error correction pointer
P:0x301B	C_ECB_IHM_ECP	-	(24:20) BLK4	0x0	The Hamming decoder block 4 error correction pointer
		-	(19:15) BLK3	0x0	The Hamming decoder block 3 error correction pointer
		-	(14:10) BLK2	0x0	The Hamming decoder block 2 error correction pointer
		-	(9:5) BLK1	0x0	The Hamming decoder block 1 error correction pointer
		-	(4:0) BLK0	0x0	The Hamming decoder block 0 error correction pointer

A.2 AUDIO CODEC BLOCK (ACB) CONFIGURATION AND CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
0x40000100	ACB_INT_STATUS	(24) CVSD_DEC_IN_ERR	(24) CVSD_DEC_IN_ERR	0x0	The CVSD decoder input interface error interrupt status
P:0x3040	C_ACB_INT_STATUS	(16) CVSD_ENC_IN_ERR	(16) CVSD_ENC_IN_ERR	0x0	The CVSD encoder input interface error interrupt status
		(9) G722_DEC_CONCEAL_RDY	(9) G722_DEC_CONCEAL_RDY	0x0	The G.722 decoder concealment flag interrupt status
		(8) G722_DEC_IN_ERR	(8) G722_DEC_IN_ERR	0x0	The G.722 decoder input interface error interrupt status
		(0) G722_ENC_IN_ERR	(0) G722_ENC_IN_ERR	0x0	The G.722 encoder input interface error interrupt status
0x40000104	ACB_INT_ENABLE	(24) CVSD_DEC_IN_ERR	(24) CVSD_DEC_IN_ERR	0x0	The CVSD decoder input interface error interrupt mask
P:0x3041	C_ACB_INT_ENABLE	(16) CVSD_ENC_IN_ERR	(16) CVSD_ENC_IN_ERR	0x0	The CVSD encoder input interface error interrupt mask
		(9) G722_DEC_CONCEAL_RDY	(9) G722_DEC_CONCEAL_RDY	0x0	The G.722 decoder concealment flag interrupt mask
		(8) G722_DEC_IN_ERR	(8) G722_DEC_IN_ERR	0x0	The G.722 decoder input interface error interrupt mask
		(0) G722_ENC_IN_ERR	(0) G722_ENC_IN_ERR	0x0	The G.722 encoder input interface error interrupt mask
0x40000108	ACB_CTRL	(31) CVSD_DEC_IN_PTR_DIR	(31) CVSD_DEC_IN_PTR_DIR	0x0	CVSD decoder input register pointer update direction
P:0x3042	C_ACB_CTRL	(30:29) CVSD_DEC_H	(30:29) CVSD_DEC_H	0x1	CVSD decoder h parameter
		(28:27) CVSD_DEC_BETA	(28:27) CVSD_DEC_BETA	0x2	CVSD decoder beta parameter
		(26:25) CVSD_DEC_J	(26:25) CVSD_DEC_J	0x1	CVSD decoder J parameter
		(24) CVSD_DEC_EBL	(24) CVSD_DEC_EBL	0x0	Enable/disable the CVSD decoder
		(23) CVSD_ENC_OUT_PTR_DIR	(23) CVSD_ENC_OUT_PTR_DIR	0x0	CVSD encoder output register pointer update direction
		(22:21) CVSD_ENC_H	(22:21) CVSD_ENC_H	0x1	CVSD encoder h parameter
		(20:19) CVSD_ENC_BETA	(20:19) CVSD_ENC_BETA	0x2	CVSD encoder beta parameter
		(18:17) CVSD_ENC_J	(18:17) CVSD_ENC_J	0x1	CVSD encoder J parameter
		(16) CVSD_ENC_EBL	(16) CVSD_ENC_EBL	0x0	Enable/disable the CVSD encoder
		(11) G722_DEC_OUT_FORMAT	(11) G722_DEC_OUT_FORMAT	0x0	G.722 decoder output format
		(10) G722_DEC_FB_BYPASS	(10) G722_DEC_FB_BYPASS	0x0	G.722 decoder filterbank bypass control
		(9) G722_DEC_CONCL_EBL	(9) G722_DEC_CONCL_EBL	0x0	G.722 decoder concealment mode control
		(8) G722_DEC_EBL	(8) G722_DEC_EBL	0x0	Enable/disable the G.722 decoder
		(2) G722_ENC_IN_FORMAT	(2) G722_ENC_IN_FORMAT	0x0	G.722 encoder input data format
		(1) G722_ENC_FB_BYPASS	(1) G722_ENC_FB_BYPASS	0x0	G.722 encoder filterbank bypass control
		(0) G722_ENC_EBL	(0) G722_ENC_EBL	0x0	Enable/disable the G.722 encoder
0x4000010C	ACB_CMD	(25) CVSD_DEC_CORE_CLR	-	N/A	CVSD decoder core section clear

Address	Register Name	Register Write	Register Read	Default	Description
P:0x3043	C_ACB_CMD	(24) CVSD_DEC_POSTFILT_CLR	-	N/A	CVSD decoder post-filter section clear
		(17) CVSD_ENC_CORE_CLR	-	N/A	CVSD encoder core section clear
		(16) CVSD_ENC_PREFILT_CLR	-	N/A	CVSD encoder pre-filter section clear
		(10) G722_DEC_SBBND_CLR	-	N/A	G.722 subband decoders section clear
		(9) G722_DEC_FB_CLR	-	N/A	G.722 decoder filterbank section clear
		(8) G722_DEC_LSB_CNCL_CLR	-	N/A	G.722 decoder lower subband concealment clear
		(1) G722_ENC_SBBND_CLR	-	N/A	G.722 subband encoders section clear
		(0) G722_ENC_FB_CLR	-	N/A	G.722 encoder filterbank section clear
0x40000110	ACB_CVSD_ENC_IN	(31:16) CVSD_ENC_IN1	(31:16) CVSD_ENC_IN1	0x0	Audio sample 1
P:0x3044	C_ACB_CVSD_ENC_IN	(15:0) CVSD_ENC_IN0	(15:0) CVSD_ENC_IN0	0x0	Audio sample 0
0x40000114	ACB_CVSD_ENC_OUT	-	(7:4) CVSD_ENC_OUT1	0x0	Compressed CVSD encoder output 1
P:0x3045	C_ACB_CVSD_ENC_OUT	-	(3:0) CVSD_ENC_OUT0	0x0	Compressed CVSD encoder output 0
0x40000118	ACB_CVSD_DEC_IN	(7:4) CVSD_DEC_IN1	(7:4) CVSD_DEC_IN1	0x0	Compressed sample 1
P:0x3046	C_ACB_CVSD_DEC_IN	(3:0) CVSD_DEC_IN0	(3:0) CVSD_DEC_IN0	0x0	Compressed sample 0
0x4000011C	ACB_CVSD_DEC_OUT	-	(31:16) CVSD_DEC_OUT1	0x0	Decompressed audio sample 1
P:0x3047	C_ACB_CVSD_DEC_OUT	-	(15:0) CVSD_DEC_OUT0	0x0	Decompressed audio sample 0
0x40000120	ACB_CVSD_DELTA	-	(31:16) CVSD_ENC_DELTA	0x140	The step size register inside the encoder
P:0x3048	C_ACB_CVSD_DELTA	-	(15:0) CVSD_DEC_DELTA	0x140	The step size register inside the decoder
0x40000124	ACB_G722_ENC_IN	(31:16) G722_ENC_IN1	(31:16) G722_ENC_IN1	0x0	Audio sample 1
P:0x3049	C_ACB_G722_ENC_IN	(15:0) G722_ENC_IN0	(15:0) G722_ENC_IN0	0x0	Audio sample 0
0x40000128	ACB_G722_ENC_OUT	-	(7:0) G722_ENC_OUT	0x0	Compressed G.722 encoder output
P:0x304A	C_ACB_G722_ENC_OUT				
0x4000012C	ACB_G722_DEC_IN	(7:0) G722_DEC_IN	(7:0) G722_DEC_IN	0x0	G.722 decoder input to decompress
P:0x304B	C_ACB_G722_DEC_IN				
0x40000130	ACB_G722_DEC_OUT	-	(31:16) G722_DEC_OUT1	0x0	Audio sample 1
P:0x304C	C_ACB_G722_DEC_OUT	-	(15:0) G722_DEC_OUT0	0x0	Audio sample 0
0x40000134	ACB_G722_DEC_FBIN	(31:16) G722_DEC_RH	(31:16) G722_DEC_RH	0x0	Higher subband decoder output to the QMF
P:0x304D	C_ACB_G722_DEC_FBIN	(15:0) G722_DEC_RL	(15:0) G722_DEC_RL	0x0	Lower subband decoder output to the QMF
0x40000138	ACB_G722_DEC_PLT	(31) G722_DEC_LSB_PLT2	(31) G722_DEC_LSB_PLT2	0x0	Decoder lower subband PLT2 sign register
P:0x304E	C_ACB_G722_DEC_PLT	(15) G722_DEC_LSB_PLT1	(15) G722_DEC_LSB_PLT1	0x0	Decoder lower subband PLT1 sign register
0x4000013C	ACB_G722_DEC_RLT	(15:0) G722_DEC_LSB_RLT1	(15:0) G722_DEC_LSB_RLT1	0x0	Decoder lower subband RLT1 register
P:0x304F	C_ACB_G722_DEC_RLT				
0x40000140	ACB_G722_DEC_SX	(31:16) G722_DEC_LSB_SZL	(31:16) G722_DEC_LSB_SZL	0x0	Decoder lower subband SZL pipeline register
P:0x3050	C_ACB_G722_DEC_SX	(15:0) G722_DEC_LSB_SL	(15:0) G722_DEC_LSB_SL	0x0	Decoder lower subband SL pipeline register
0x40000144	ACB_G722_DEC_LSB_QUANT	(30:18) G722_DEC_LSB_DETL	(30:18) G722_DEC_LSB_DETL	0x8	Decoder lower subband DETL register

Address	Register Name	Register Write	Register Read	Default	Description
P:0x3051	C_ACB_G722_DEC_LSB_QUANT	(14:0) G722_DEC_LSB_NBL	(14:0) G722_DEC_LSB_NBL	0x0	Decoder lower subband NBL register
0x40000148	ACB_G722_DEC_HSB_QUANT	(30:18) G722_DEC_HSB_DETH	(30:18) G722_DEC_HSB_DETH	0x2	Decoder higher subband DETH register
P:0x3052	C_ACB_G722_DEC_HSB_QUANT	(14:0) G722_DEC_HSB_NBH	(14:0) G722_DEC_HSB_NBH	0x0	Decoder higher subband NBH register

A.3 DMA Controller Configuration and Control

Address	Register Name	Register Write	Register Read	Default	Description
0x40000200	DMA_CH0_CTRL0	(31:30) DEST_ADDR_STEP_SIZE	(31:30) DEST_ADDR_STEP_SIZE	0x0	Select the destination address step size
P:0x3080	C_DMA_CH0_CTRL0	(29:28) SRC_ADDR_STEP_SIZE	(29:28) SRC_ADDR_STEP_SIZE	0x0	Select the source address step size
		(27) DEST_ADDR_STEP_MODE	(27) DEST_ADDR_STEP_MODE	0x0	Configure the destination address to either increment or decrement
		(26) SRC_ADDR_STEP_MODE	(26) SRC_ADDR_STEP_MODE	0x0	Configure the source address to either increment or decrement
		(25) BYTE_ORDER	(25) BYTE_ORDER	0x0	Select the byte ordering for the DMA channel
		(24) DISABLE_INT_ENABLE	(24) DISABLE_INT_ENABLE	0x0	Raise an interrupt when the DMA channel is disabled
		(23) ERROR_INT_ENABLE	(23) ERROR_INT_ENABLE	0x0	Raise an interrupt when a state machine error occurs during a DMA transfer
		(22) COMPLETE_INT_ENABLE	(22) COMPLETE_INT_ENABLE	0x0	Raise an interrupt when the DMA transfer completes
		(21) COUNTER_INT_ENABLE	(21) COUNTER_INT_ENABLE	0x0	Raise an interrupt when the DMA transfer reaches the counter value
		(20) START_INT_ENABLE	(20) START_INT_ENABLE	0x0	Raise an interrupt when the DMA transfer starts
		(19:18) DEST_WORD_SIZE	(19:18) DEST_WORD_SIZE	0x0	Select the destination word size
		(17:16) SRC_WORD_SIZE	(17:16) SRC_WORD_SIZE	0x0	Select the source word size
		(15:12) DEST_SELECT	(15:12) DEST_SELECT	0x0	Select the request line for the destination
		(11:8) SRC_SELECT	(11:8) SRC_SELECT	0x0	Select the request line for the source
		(7:6) CHANNEL_PRIORITY	(7:6) CHANNEL_PRIORITY	0x0	Select the priority level for this channel
		(5:4) TRANSFER_TYPE	(5:4) TRANSFER_TYPE	0x0	Select the type of transfer implemented by DMA channel 0
		(3) DEST_ADDR_INC	(3) DEST_ADDR_INC	0x0	Configure whether the destination address should increment
		(2) SRC_ADDR_INC	(2) SRC_ADDR_INC	0x0	Configure whether the source address should increment
		(1) ADDR_MODE	(1) ADDR_MODE	0x0	Select the addressing mode for this channel
		(0) ENABLE	(0) ENABLE	0x0	Enable DMA channel 0
0x40000204	DMA_CH0_SRC_BASE_ADDR	(31:0) DMA_CH0_SRC_BASE_ADDR	(31:0) DMA_CH0_SRC_BASE_ADDR	0x0	Base address for the source of data transferred using DMA channel 0
P:0x3081	C_DMA_CH0_SRC_BASE_ADDR				
0x40000208	DMA_CH0_DEST_BASE_ADDR	(31:0) DMA_CH0_DEST_BASE_ADDR	(31:0) DMA_CH0_DEST_BASE_ADDR	0x0	Base address for the destination of data transferred using DMA channel 0
P:0x3082	C_DMA_CH0_DEST_BASE_ADDR				
0x4000020C	DMA_CH0_CTRL1	(31:16) COUNTER_INT_VALUE	(31:16) COUNTER_INT_VALUE	0x0	Trigger a counter interrupt when the DMA transfer word count reaches this value
P:0x3083	C_DMA_CH0_CTRL1	(15:0) TRANSFER_LENGTH	(15:0) TRANSFER_LENGTH	0x0	The length, in words, of each data transfer using DMA channel 0
0x40000210	DMA_CH0_NEXT_SRC_ADDR	-	(31:0) DMA_CH0_NEXT_SRC_ADDR	0x0	Address of the next data to be transferred using DMA channel 0
P:0x3084	C_DMA_CH0_NEXT_SRC_ADDR				

Address	Register Name	Register Write	Register Read	Default	Description
0x40000214 P:0x3085	DMA_CH0_NEXT_DEST_ADDR C_DMA_CH0_NEXT_DEST_ADDR	-	(31:0) DMA_CH0_NEXT_DEST_ADDR	0x0	Address where the next data to be transferred using DMA channel 0 will be stored
0x40000218 P:0x3086	DMA_CH0_WORD_CNT C_DMA_CH0_WORD_CNT	-	(15:0) DMA_CH0_WORD_CNT	0x0	The number of words that have been transferred using DMA channel 0 during the current transfer
0x40000220 P:0x3088	DMA_CH1_CTRL0 C_DMA_CH1_CTRL0	(31:30) DEST_ADDR_STEP_SIZE	(31:30) DEST_ADDR_STEP_SIZE	0x0	Select the destination address step size
		(29:28) SRC_ADDR_STEP_SIZE	(29:28) SRC_ADDR_STEP_SIZE	0x0	Select the source address step size
		(27) DEST_ADDR_STEP_MODE	(27) DEST_ADDR_STEP_MODE	0x0	Configure the destination address to either increment or decrement
		(26) SRC_ADDR_STEP_MODE	(26) SRC_ADDR_STEP_MODE	0x0	Configure the source address to either increment or decrement
		(25) BYTE_ORDER	(25) BYTE_ORDER	0x0	Select the byte ordering for the DMA channel
		(24) DISABLE_INT_ENABLE	(24) DISABLE_INT_ENABLE	0x0	Raise an interrupt when the DMA channel is disabled
		(23) ERROR_INT_ENABLE	(23) ERROR_INT_ENABLE	0x0	Raise an interrupt when a state machine error occurs during a DMA transfer
		(22) COMPLETE_INT_ENABLE	(22) COMPLETE_INT_ENABLE	0x0	Raise an interrupt when the DMA transfer completes
		(21) COUNTER_INT_ENABLE	(21) COUNTER_INT_ENABLE	0x0	Raise an interrupt when the DMA transfer reaches the counter value
		(20) START_INT_ENABLE	(20) START_INT_ENABLE	0x0	Raise an interrupt when the DMA transfer starts
		(19:18) DEST_WORD_SIZE	(19:18) DEST_WORD_SIZE	0x0	Select the destination word size
		(17:16) SRC_WORD_SIZE	(17:16) SRC_WORD_SIZE	0x0	Select the source word size
		(15:12) DEST_SELECT	(15:12) DEST_SELECT	0x0	Select the request line for the destination
		(11:8) SRC_SELECT	(11:8) SRC_SELECT	0x0	Select the request line for the source
		(7:6) CHANNEL_PRIORITY	(7:6) CHANNEL_PRIORITY	0x0	Select the priority level for this channel
		(5:4) TRANSFER_TYPE	(5:4) TRANSFER_TYPE	0x0	Select the type of transfer implemented by DMA channel 1
		(3) DEST_ADDR_INC	(3) DEST_ADDR_INC	0x0	Configure whether the destination address should increment
		(2) SRC_ADDR_INC	(2) SRC_ADDR_INC	0x0	Configure whether the source address should increment
		(1) ADDR_MODE	(1) ADDR_MODE	0x0	Select the addressing mode for this channel
		(0) ENABLE	(0) ENABLE	0x0	Enable DMA channel 1
0x40000224 P:0x3089	DMA_CH1_SRC_BASE_ADDR C_DMA_CH1_SRC_BASE_ADDR	(31:0) DMA_CH1_SRC_BASE_ADDR	(31:0) DMA_CH1_SRC_BASE_ADDR	0x0	Base address for the source of data transferred using DMA channel 1
0x40000228 P:0x308A	DMA_CH1_DEST_BASE_ADDR C_DMA_CH1_DEST_BASE_ADDR	(31:0) DMA_CH1_DEST_BASE_ADDR	(31:0) DMA_CH1_DEST_BASE_ADDR	0x0	Base address for the destination of data transferred using DMA channel 1
0x4000022C	DMA_CH1_CTRL1	(31:16) COUNTER_INT_VALUE	(31:16) COUNTER_INT_VALUE	0x0	Trigger a counter interrupt when the DMA transfer word count reaches this value

Address	Register Name	Register Write	Register Read	Default	Description
P:0x308B	C_DMA_CH1_CTRL1	(15:0) TRANSFER_LENGTH	(15:0) TRANSFER_LENGTH	0x0	The length, in words, of each data transfer using DMA channel 1
0x40000230 P:0x308C	DMA_CH1_NEXT_SRC_ADDR C_DMA_CH1_NEXT_SRC_ADDR	-	(31:0) DMA_CH1_NEXT_SRC_ADDR	0x0	Address of the next data to be transferred using DMA channel 1
0x40000234 P:0x308D	DMA_CH1_NEXT_DEST_ADDR C_DMA_CH1_NEXT_DEST_ADDR	-	(31:0) DMA_CH1_NEXT_DEST_ADDR	0x0	Address where the next data to be transferred using DMA channel 1 will be stored
0x40000238 P:0x308E	DMA_CH1_WORD_CNT C_DMA_CH1_WORD_CNT	-	(15:0) DMA_CH1_WORD_CNT	0x0	The number of words that have been transferred using DMA channel 1 during the current transfer
0x40000240 P:0x3090	DMA_CH2_CTRL0 C_DMA_CH2_CTRL0	(31:30) DEST_ADDR_STEP_SIZE	(31:30) DEST_ADDR_STEP_SIZE	0x0	Select the destination address step size
		(29:28) SRC_ADDR_STEP_SIZE	(29:28) SRC_ADDR_STEP_SIZE	0x0	Select the source address step size
		(27) DEST_ADDR_STEP_MODE	(27) DEST_ADDR_STEP_MODE	0x0	Configure the destination address to either increment or decrement
		(26) SRC_ADDR_STEP_MODE	(26) SRC_ADDR_STEP_MODE	0x0	Configure the source address to either increment or decrement
		(25) BYTE_ORDER	(25) BYTE_ORDER	0x0	Select the byte ordering for the DMA channel
		(24) DISABLE_INT_ENABLE	(24) DISABLE_INT_ENABLE	0x0	Raise an interrupt when the DMA channel is disabled
		(23) ERROR_INT_ENABLE	(23) ERROR_INT_ENABLE	0x0	Raise an interrupt when a state machine error occurs during a DMA transfer
		(22) COMPLETE_INT_ENABLE	(22) COMPLETE_INT_ENABLE	0x0	Raise an interrupt when the DMA transfer completes
		(21) COUNTER_INT_ENABLE	(21) COUNTER_INT_ENABLE	0x0	Raise an interrupt when the DMA transfer reaches the counter value
		(20) START_INT_ENABLE	(20) START_INT_ENABLE	0x0	Raise an interrupt when the DMA transfer starts
		(19:18) DEST_WORD_SIZE	(19:18) DEST_WORD_SIZE	0x0	Select the destination word size
		(17:16) SRC_WORD_SIZE	(17:16) SRC_WORD_SIZE	0x0	Select the source word size
		(15:12) DEST_SELECT	(15:12) DEST_SELECT	0x0	Select the request line for the destination
		(11:8) SRC_SELECT	(11:8) SRC_SELECT	0x0	Select the request line for the source
		(7:6) CHANNEL_PRIORITY	(7:6) CHANNEL_PRIORITY	0x0	Select the priority level for this channel
		(5:4) TRANSFER_TYPE	(5:4) TRANSFER_TYPE	0x0	Select the type of transfer implemented by DMA channel 2
		(3) DEST_ADDR_INC	(3) DEST_ADDR_INC	0x0	Configure whether the destination address should increment
		(2) SRC_ADDR_INC	(2) SRC_ADDR_INC	0x0	Configure whether the source address should increment
		(1) ADDR_MODE	(1) ADDR_MODE	0x0	Select the addressing mode for this channel
		(0) ENABLE	(0) ENABLE	0x0	Enable DMA channel 2
0x40000244 P:0x3091	DMA_CH2_SRC_BASE_ADDR C_DMA_CH2_SRC_BASE_ADDR	(31:0) DMA_CH2_SRC_BASE_ADDR	(31:0) DMA_CH2_SRC_BASE_ADDR	0x0	Base address for the source of data transferred using DMA channel 2

Address	Register Name	Register Write	Register Read	Default	Description
0x40000248 P:0x3092	DMA_CH2_DEST_BASE_ADDR C_DMA_CH2_DEST_BASE_ADDR	(31:0) DMA_CH2_DEST_BASE_ADDR	(31:0) DMA_CH2_DEST_BASE_ADDR	0x0	Base address for the destination of data transferred using DMA channel 2
0x4000024C P:0x3093	DMA_CH2_CTRL1 C_DMA_CH2_CTRL1	(31:16) COUNTER_INT_VALUE (15:0) TRANSFER_LENGTH	(31:16) COUNTER_INT_VALUE (15:0) TRANSFER_LENGTH	0x0 0x0	Trigger a counter+I553 interrupt when the DMA transfer word count reaches this value The length, in words, of each data transfer using DMA channel 2
0x40000250 P:0x3094	DMA_CH2_NEXT_SRC_ADDR C_DMA_CH2_NEXT_SRC_ADDR	-	(31:0) DMA_CH2_NEXT_SRC_ADDR	0x0	Address of the next data to be transferred using DMA channel 2
0x40000254 P:0x3095	DMA_CH2_NEXT_DEST_ADDR C_DMA_CH2_NEXT_DEST_ADDR	-	(31:0) DMA_CH2_NEXT_DEST_ADDR	0x0	Address where the next data to be transferred using DMA channel 2 will be stored
0x40000258 P:0x3096	DMA_CH2_WORD_CNT C_DMA_CH2_WORD_CNT	-	(15:0) DMA_CH2_WORD_CNT	0x0	The number of words that have been transferred using DMA channel 2 during the current transfer
0x40000260	DMA_CH3_CTRL0	(31:30) DEST_ADDR_STEP_SIZE	(31:30) DEST_ADDR_STEP_SIZE	0x0	Select the destination address step size

Address	Register Name	Register Write	Register Read	Default	Description
P:0x3098	C_DMA_CH3_CTRL0	(29:28) SRC_ADDR_STEP_SIZE	(29:28) SRC_ADDR_STEP_SIZE	0x0	Select the source address step size
		(27) DEST_ADDR_STEP_MODE	(27) DEST_ADDR_STEP_MODE	0x0	Configure the destination address to either increment or decrement
		(26) SRC_ADDR_STEP_MODE	(26) SRC_ADDR_STEP_MODE	0x0	Configure the source address to either increment or decrement
		(25) BYTE_ORDER	(25) BYTE_ORDER	0x0	Select the byte ordering for the DMA channel
		(24) DISABLE_INT_ENABLE	(24) DISABLE_INT_ENABLE	0x0	Raise an interrupt when the DMA channel is disabled
		(23) ERROR_INT_ENABLE	(23) ERROR_INT_ENABLE	0x0	Raise an interrupt when a state machine error occurs during a DMA transfer
		(22) COMPLETE_INT_ENABLE	(22) COMPLETE_INT_ENABLE	0x0	Raise an interrupt when the DMA transfer completes
		(21) COUNTER_INT_ENABLE	(21) COUNTER_INT_ENABLE	0x0	Raise an interrupt when the DMA transfer reaches the counter value
		(20) START_INT_ENABLE	(20) START_INT_ENABLE	0x0	Raise an interrupt when the DMA transfer starts
		(19:18) DEST_WORD_SIZE	(19:18) DEST_WORD_SIZE	0x0	Select the destination word size
		(17:16) SRC_WORD_SIZE	(17:16) SRC_WORD_SIZE	0x0	Select the source word size
		(15:12) DEST_SELECT	(15:12) DEST_SELECT	0x0	Select the request line for the destination
		(11:8) SRC_SELECT	(11:8) SRC_SELECT	0x0	Select the request line for the source
		(7:6) CHANNEL_PRIORITY	(7:6) CHANNEL_PRIORITY	0x0	Select the priority level for this channel
		(5:4) TRANSFER_TYPE	(5:4) TRANSFER_TYPE	0x0	Select the type of transfer implemented by DMA channel 3
		(3) DEST_ADDR_INC	(3) DEST_ADDR_INC	0x0	Configure whether the destination address should increment
		(2) SRC_ADDR_INC	(2) SRC_ADDR_INC	0x0	Configure whether the source address should increment
		(1) ADDR_MODE	(1) ADDR_MODE	0x0	Select the addressing mode for this channel
		(0) ENABLE	(0) ENABLE	0x0	Enable DMA channel 3
0x40000264 P:0x3099	DMA_CH3_SRC_BASE_ADDR C_DMA_CH3_SRC_BASE_ADDR	(31:0) DMA_CH3_SRC_BASE_ADDR	(31:0) DMA_CH3_SRC_BASE_ADDR	0x0	Base address for the source of data transferred using DMA channel 3
0x40000268 P:0x309A	DMA_CH3_DEST_BASE_ADDR C_DMA_CH3_DEST_BASE_ADDR	(31:0) DMA_CH3_DEST_BASE_ADDR	(31:0) DMA_CH3_DEST_BASE_ADDR	0x0	Base address for the destination of data transferred using DMA channel 3
0x4000026C	DMA_CH3_CTRL1	(31:16) COUNTER_INT_VALUE	(31:16) COUNTER_INT_VALUE	0x0	Trigger a counter interrupt when the DMA transfer word count reaches this value
P:0x309B	C_DMA_CH3_CTRL1	(15:0) TRANSFER_LENGTH	(15:0) TRANSFER_LENGTH	0x0	The length, in words, of each data transfer using DMA channel 3
0x40000270 P:0x309C	DMA_CH3_NEXT_SRC_ADDR C_DMA_CH3_NEXT_SRC_ADDR	-	(31:0) DMA_CH3_NEXT_SRC_ADDR	0x0	Address of the next data to be transferred using DMA channel 3
0x40000274 P:0x309D	DMA_CH3_NEXT_DEST_ADDR C_DMA_CH3_NEXT_DEST_ADDR	-	(31:0) DMA_CH3_NEXT_DEST_ADDR	0x0	Address where the next data to be transferred using DMA channel 3 will be stored

Address	Register Name	Register Write	Register Read	Default	Description
0x40000278 P:0x309E	DMA_CH3_WORD_CNT C_DMA_CH3_WORD_CNT	-	(15:0) DMA_CH3_WORD_CNT	0x0	The number of words that have been transferred using DMA channel 3 during the current transfer
0x40000290	DMA_STATUS	(31) CH3_ERROR_INT_STATUS	(31) CH3_ERROR_INT_STATUS	0x0	Indicate if a state machine error interrupt has occurred on DMA channel 3

Address	Register Name	Register Write	Register Read	Default	Description
P:0x30A4	C_DMA_STATUS	(30) CH3_COMPLETE_INT_STATUS	(30) CH3_COMPLETE_INT_STATUS	0x0	Indicate if a complete interrupt has occurred on DMA channel 3
		(29) CH3_COUNTER_INT_STATUS	(29) CH3_COUNTER_INT_STATUS	0x0	Indicate if a counter interrupt has occurred on DMA channel 3
		(28) CH3_START_INT_STATUS	(28) CH3_START_INT_STATUS	0x0	Indicate if a start interrupt has occurred on DMA channel 3
		(27) CH3_DISABLE_INT_STATUS	(27) CH3_DISABLE_INT_STATUS	0x0	Indicate if a channel disable interrupt has occurred on DMA channel 3
		-	(26:24) CH3_STATE	0x0	DMA channel 3 state
		(23) CH2_ERROR_INT_STATUS	(23) CH2_ERROR_INT_STATUS	0x0	Indicate if a state machine error interrupt has occurred on DMA channel 2
		(22) CH2_COMPLETE_INT_STATUS	(22) CH2_COMPLETE_INT_STATUS	0x0	Indicate if a complete interrupt has occurred on DMA channel 2
		(21) CH2_COUNTER_INT_STATUS	(21) CH2_COUNTER_INT_STATUS	0x0	Indicate if a counter interrupt has occurred on DMA channel 2
		(20) CH2_START_INT_STATUS	(20) CH2_START_INT_STATUS	0x0	Indicate if a start interrupt has occurred on DMA channel 2
		(19) CH2_DISABLE_INT_STATUS	(19) CH2_DISABLE_INT_STATUS	0x0	Indicate if a channel disable interrupt has occurred on DMA channel 2
		-	(18:16) CH2_STATE	0x0	DMA channel 2 state
		(15) CH1_ERROR_INT_STATUS	(15) CH1_ERROR_INT_STATUS	0x0	Indicate if a state machine error interrupt has occurred on DMA channel 1
		(14) CH1_COMPLETE_INT_STATUS	(14) CH1_COMPLETE_INT_STATUS	0x0	Indicate if a complete interrupt has occurred on DMA channel 1
		(13) CH1_COUNTER_INT_STATUS	(13) CH1_COUNTER_INT_STATUS	0x0	Indicate if a counter interrupt has occurred on DMA channel 1
		(12) CH1_START_INT_STATUS	(12) CH1_START_INT_STATUS	0x0	Indicate if a start interrupt has occurred on DMA channel 1
		(11) CH1_DISABLE_INT_STATUS	(11) CH1_DISABLE_INT_STATUS	0x0	Indicate if a channel disable interrupt has occurred on DMA channel 1
		-	(10:8) CH1_STATE	0x0	DMA channel 1 state
		(7) CH0_ERROR_INT_STATUS	(7) CH0_ERROR_INT_STATUS	0x0	Indicate if a state machine error interrupt has occurred on DMA channel 0
		(6) CH0_COMPLETE_INT_STATUS	(6) CH0_COMPLETE_INT_STATUS	0x0	Indicate if a complete interrupt has occurred on DMA channel 0
		(5) CH0_COUNTER_INT_STATUS	(5) CH0_COUNTER_INT_STATUS	0x0	Indicate if a counter interrupt has occurred on DMA channel 0
		(4) CH0_START_INT_STATUS	(4) CH0_START_INT_STATUS	0x0	Indicate if a start interrupt has occurred on DMA channel 0
		(3) CH0_DISABLE_INT_STATUS	(3) CH0_DISABLE_INT_STATUS	0x0	Indicate if a channel disable interrupt has occurred on DMA channel 0
		-	(2:0) CH0_STATE	0x0	DMA channel 0 state

A.4 EXTERNAL INTERRUPT CONFIGURATION AND CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
0x40000300	CFX_INT_CMD	(3) CFX_CMD_3	-	N/A	Trigger an ARM Cortex-M3 core command 3 interrupt on the CFX
P:0x30C0	C_CFX_INT_CMD	(2) CFX_CMD_2	-	N/A	Trigger an ARM Cortex-M3 core command 2 interrupt on the CFX
		(1) CFX_CMD_1	-	N/A	Trigger an ARM Cortex-M3 core command 1 interrupt on the CFX
		(0) CFX_CMD_0	-	N/A	Trigger an ARM Cortex-M3 core command 0 interrupt on the CFX

A.5 DIO CONFIGURATION AND CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
0x40000500	DIO_DATA	-	(29:0) C_DIO	0x0	DIO [29:0] read data (pads configured for LSAD mode return 0)
P:0x3140	C_DIO_DATA	(29:0) C_GPIO	-	N/A	GPIO [29:0] write data (updates output data of DIO [29:0] only for pads with IO_MODE 0b100xX)
0x40000504	DIO_DIR	-	(29:0) C_DIO	0x0	Get DIO direction
P:0x3141	C_DIO_DIR	(29:0) C_GPIO	-	N/A	Set DIO GPIO direction (only if IO_MODE is 0b100xX)
0x40000508	DIO_MODE	-	(29:0) C_GPIO	0x0	DIO mode
P:0x3142	C_DIO_MODE				
0x4000050C	DIO_INT_CFG_0	(7:5) CM3_INT_EVENT	(7:5) CM3_INT_EVENT	0x0	ARM Cortex-M3 core interrupt event configuration
P:0x3143	C_DIO_INT_CFG_0	(4:0) CM3_INT_SRC	(4:0) CM3_INT_SRC	0x0	ARM Cortex-M3 core interrupt input selection
0x40000510	DIO_INT_CFG_1	(7:5) CM3_INT_EVENT	(7:5) CM3_INT_EVENT	0x0	ARM Cortex-M3 core interrupt event configuration
P:0x3144	C_DIO_INT_CFG_1	(4:0) CM3_INT_SRC	(4:0) CM3_INT_SRC	0x0	ARM Cortex-M3 core interrupt input selection
0x40000514	DIO_INT_CFG_2	(7:5) CM3_INT_EVENT	(7:5) CM3_INT_EVENT	0x0	ARM Cortex-M3 core interrupt event configuration
P:0x3145	C_DIO_INT_CFG_2	(4:0) CM3_INT_SRC	(4:0) CM3_INT_SRC	0x0	ARM Cortex-M3 core interrupt input selection
0x40000518	DIO_INT_CFG_3	(7:5) CM3_INT_EVENT	(7:5) CM3_INT_EVENT	0x0	ARM Cortex-M3 core interrupt event configuration
P:0x3146	C_DIO_INT_CFG_3	(4:0) CM3_INT_SRC	(4:0) CM3_INT_SRC	0x0	ARM Cortex-M3 core interrupt input selection
0x4000051C	DIO_INT_STATUS	-	(3) CM3_INT3_STATUS	0x0	ARM Cortex-M3 core interrupt 3 status (cleared when read)
P:0x3147	C_DIO_INT_STATUS	-	(2) CM3_INT2_STATUS	0x0	ARM Cortex-M3 core interrupt 2 status (cleared when read)
		-	(1) CM3_INT1_STATUS	0x0	ARM Cortex-M3 core interrupt 1 status (cleared when read)
		-	(0) CM3_INT0_STATUS	0x0	ARM Cortex-M3 core interrupt 0 status (cleared when read)

A.6 SPI Interface Configuration and Control

Address	Register Name	Register Write	Register Read	Default	Description
0x40000700	SPI_CTRL0	(10) OVERRUN_INT_ENABLE	(10) OVERRUN_INT_ENABLE	0x0	Enable/disable SPI overrun interrupts
P:0x31C0	C_SPI_CTRL0	(9) UNDERRUN_INT_ENABLE	(9) UNDERRUN_INT_ENABLE	0x0	Enable/disable SPI underrun interrupts
		(8) CONTROLLER	(8) CONTROLLER	0x0	Select whether the data transfer will be controlled by the ARM Cortex-M3 core or the DMA for SPI
		(7) SLAVE	(7) SLAVE	0x0	Use the SPI interface as master or slave
		(5) CLK_POLARITY	(5) CLK_POLARITY	0x0	Select the polarity of the SPI clock
		(4) MODE_SELECT	(4) MODE_SELECT	0x0	Select between manual and auto transaction handling modes for SPI master transactions
		(3) ENABLE	(3) ENABLE	0x0	Enable/disable the SPI interface
		(2:0) PRESCALE	(2:0) PRESCALE	0x0	Prescale the SPI interface clock for master transfers
0x40000704	SPI_CTRL1	(8) START_BUSY	(8) START_BUSY	0x0	Start an SPI data transfer and indicate if a transfer is in progress
P:0x31C1	C_SPI_CTRL1	(7:6) RW_CMD	(7:6) RW_CMD	0x0	Issue a read command or write command to the SPI interface
		(5) CS	(5) CS	0x1	Set the chip-select line for SPI (master mode), read the chip-select line for SPI (slave mode)
		(4:0) WORD_SIZE	(4:0) WORD_SIZE	0x0	Select the word size used by the SPI interface (word size = SPI_CTRL1_WORD_SIZE + 1)
0x40000708	SPI_TX_DATA	(31:0) SPI_TX_DATA	(31:0) SPI_TX_DATA	0x0	Single word buffer for data to be transmitted over the SPI interface
P:0x31C2	C_SPI_TX_DATA				
0x4000070C	SPI_RX_DATA	-	(31:0) SPI_RX_DATA	0x0	Single word buffer for data that has been received over the SPI interface
P:0x31C3	C_SPI_RX_DATA				
0x40000710	SPI_STATUS	(3) TRANSMIT_STATUS	(3) TRANSMIT_STATUS	0x0	Indicate that the transmission of the data is completed
P:0x31C4	C_SPI_STATUS	(2) RECEIVE_STATUS	(2) RECEIVE_STATUS	0x0	Indicate that new data has been received
		(1) OVERRUN_STATUS	(1) OVERRUN_STATUS	0x0	Indicate that an overrun has occurred when receiving data on the SPI interface
		(0) UNDERRUN_STATUS	(0) UNDERRUN_STATUS	0x0	Indicate that an underrun has occurred when transmitting data on the SPI interface

A.7 PCM Interface Configuration and Control

Address	Register Name	Register Write	Register Read	Default	Description
0x40000800	PCM_CTRL	(12) BIT_ORDER	(12) BIT_ORDER	0x0	Select whether the data will be transmitted starting with the MSB or LSB
P:0x3200	C_PCM_CTRL	(11) TX_ALIGN	(11) TX_ALIGN	0x0	Select what bits to use for transmit data
		(10:9) WORD_SIZE	(10:9) WORD_SIZE	0x3	Select the number of bits per PCM word
		(8) FRAME_ALIGN	(8) FRAME_ALIGN	0x0	Align the PCM frame signal to the first/last bit
		(7) FRAME_WIDTH	(7) FRAME_WIDTH	0x0	Use a long/short PCM frame signal
		(6:4) FRAME_LENGTH	(6:4) FRAME_LENGTH	0x0	Select the number of words per PCM frame
		(3) FRAME_SUBFRAMES	(3) FRAME_SUBFRAMES	0x0	Select between a frame signal with every word and a frame
		(2) CONTROLLER	(2) CONTROLLER	0x0	Select whether the data transfer will be controlled by the ARM Cortex-M3 core or the DMA for PCM
		(1) ENABLE	(1) ENABLE	0x0	Enable/disable the PCM interface
		(0) SLAVE	(0) SLAVE	0x1	Use the PCM interface as a master/slave
0x40000804	PCM_TX_DATA	(31:0) PCM_TX_DATA	(31:0) PCM_TX_DATA	0x0	Data to transmit over the PCM interface
P:0x3201	C_PCM_TX_DATA				
0x40000808	PCM_RX_DATA	-	(31:0) PCM_RX_DATA	0x0	Data received from the PCM interface
P:0x3202	C_PCM_RX_DATA				
0x4000080C	PCM_STATUS	(3) TRANSMIT_STATUS	(3) TRANSMIT_STATUS	0x0	Indicate that PCM data has been sent
P:0x3203	C_PCM_STATUS	(2) RECEIVE_STATUS	(2) RECEIVE_STATUS	0x0	Indicate that PCM data has been received
		(1) OVERRUN_STATUS	(1) OVERRUN_STATUS	0x0	Indicate that an overrun has occurred when receiving data on the PCM interface
		(0) UNDERRUN_STATUS	(0) UNDERRUN_STATUS	0x0	Indicate that an underrun has occurred when transmitting data on the PCM interface

A.8 I2C INTERFACE CONFIGURATION AND CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
0x40000900	I2C_CTRL0	(23:16) MASTER_SPEED_PRESCALER	(23:16) MASTER_SPEED_PRESCALER	0x0	Prescaler used to divide SLOWCLK to the correct SCL frequency when operating in I^{2}C interface master mode. SCL is prescaled by (MASTER_SPEED_PRESCALER + 1) * 3
P:0x3240	C_I2C_CTRL0	(14:8) SLAVE_ADDRESS	(14:8) SLAVE_ADDRESS	0x10	Set the I^{2}C slave address for this chip
		(4) CONTROLLER	(4) CONTROLLER	0x0	Select whether the data transfer will be controlled by the ARM Cortex-M3 processor or the DMA for I^{2}C
		(3) STOP_INT_ENABLE	(3) STOP_INT_ENABLE	0x0	Configure whether stop interrupts will be generated by the I^{2}C interface
		(2) AUTO_ACK_ENABLE	(2) AUTO_ACK_ENABLE	0x0	Select whether acknowledgement is automatically generated or not
		(1) I2C_SAMPLE_CLK_ENABLE	(1) I2C_SAMPLE_CLK_ENABLE	0x0	Enable/disable the I^{2}C sample clock (mandatory to enable the I^{2}C)
		(0) SLAVE_ENABLE	(0) SLAVE_ENABLE	0x0	Select whether the I^{2}C interface will be enabled for slave mode or not
0x40000904	I2C_CTRL1	(5) RESET	-	N/A	Reset the I^{2}C interface
P:0x3241	C_I2C_CTRL1	(4) LAST_DATA	-	N/A	Indicate that the current data is the last byte of a data transfer
		(3) STOP	-	N/A	Issue a stop condition the I^{2}C interface bus
		(1) NACK	-	N/A	Issue a not acknowledge on the I^{2}C interface bus
		(0) ACK	-	N/A	Issue an acknowledge on the I^{2}C interface bus
0x40000910	I2C_DATA	(7:0) I2C_DATA	(7:0) I2C_DATA	0x0	Single byte buffer for data transmitted and received over the I^{2}C interface
P:0x3244	C_I2C_DATA				
0x40000914	I2C_DATA_M	(7:0) I2C_DATA_M	(7:0) I2C_DATA_M	0x0	Mirror of the single byte buffer for data transmitted and received over the I^{2}C interface
P:0x3245	C_I2C_DATA_M				
0x40000918	I2C_ADDR_START	(7:1) ADDRESS	-	N/A	I^{2}C address to use for the transaction
P:0x3246	C_I2C_ADDR_START	(0) READ_WRITE	-	N/A	Select whether a read or a write transaction is started
0x4000091C	I2C_STATUS	-	(15) ERROR_S	0x0	Error status bit (sticky) - This status bit is automatically reset when the I2C_STATUS register is read.

Address	Register Name	Register Write	Register Read	Default	Description
P:0x3247	C_I2C_STATUS	-	(14) BUS_ERROR_S	0x0	Bus error status bit (sticky) - This status bit is automatically reset when the I2C_STATUS register is read.
		-	(13) START_PENDING	0x0	Master frame start pending status bit
		-	(12) MASTER_MODE	0x0	Master mode status bit
		-	(11) DMA_REQ	0x0	Indicate if the I^2C interface is currently requesting DMA data
		-	(10) STOP_DETECT	0x0	Indicate if an I^2C stop bit has been detected
		-	(9) DATA_EVENT	0x0	Indicate that I^2C interface either needs data to transmit or has received data
		-	(8) ERROR	0x0	Indicate if the I^2C interface has detected a bus error (automatically cleared when a stop condition is detected)
		-	(7) BUS_ERROR	0x0	Indicate if the I^2C interface has detected a bus error (automatically cleared when a stop condition is detected)
		-	(6) BUFFER_FULL	0x0	Indicate if the I^2C data buffer is full
		-	(5) CLK_STRETCH	0x0	Indicate if the I^2C interface is holding the clock signal
		-	(4) BUS_FREE	0x1	Indicate if the I^2C interface bus is free
		-	(3) ADDR_DATA	0x0	Indicate if the I^2C data register holds an address or data byte
		-	(2) READ_WRITE	0x0	Indicate whether the I^2C bus transfer is a read or a write
		-	(1) GEN_CALL	0x0	Indicate whether the I^2C bus transfer is using the general call address or another address
		-	(0) ACK_STATUS	0x0	Indicate whether an acknowledge or a not acknowledge has been received

A.9 CRC GENERATOR CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
0x40000A00	CRC_CTRL	(4) FINAL_CRC_XOR	(4) FINAL_CRC_XOR	0x0	Selects the final CRC XOR mode
P:0x3280	C_CRC_CTRL	(3) FINAL_CRC_REVERSE	(3) FINAL_CRC_REVERSE	0x0	Selects the final CRC reversal mode
		(2) BIT_ORDER	(2) BIT_ORDER	0x0	Selects the bit order for bytes added to the CRC
		(1) CRC_TYPE	(1) CRC_TYPE	0x0	Selects the CRC type
		(0) BYTE_ORDER	(0) BYTE_ORDER	0x0	Selects the endianess for bytes added to the CRC
0x40000A04	CRC_VALUE	(31:0) CURRENT_CRC	(31:0) CURRENT_CRC	0xFFFF	CRC generator value: Write 0xFFFFFFF (32) or 0xFFFF (CCITT) to initialize the CRC; read provides the current CRC value.
P:0x3281	C_CRC_VALUE				
0x40000A08	CRC_ADD_1	(0) CRC_ADD_1_BIT	-	N/A	Add 1 bit to the CRC calculation
P:0x3282	C_CRC_ADD_1				
0x40000A0C	CRC_ADD_8	(7:0) CRC_ADD_8	-	N/A	Add 1 byte (8 bits) to the CRC calculation
P:0x3283	C_CRC_ADD_8				
0x40000A10	CRC_ADD_16	(15:0) CRC_ADD_16	-	N/A	Add 1 half-word (16 bits) to the CRC calculation
P:0x3284	C_CRC_ADD_16				
0x40000A14	CRC_ADD_24	(23:0) CRC_ADD_24_BITS	-	N/A	Add 3 bytes (24 bits) to the CRC calculation
P:0x3285	C_CRC_ADD_24				
0x40000A18	CRC_ADD_32	(31:0) CRC_ADD_32	-	N/A	Add 1 word (32 bits) to the CRC calculation
P:0x3286	C_CRC_ADD_32				

A.10 SYSTICK TIMER

Address	Register Name	Register Write	Register Read	Default	Description
0xE000E010	SysTick_CTRL	-	(16) COUNTFLAG	0x0	Reads as 1 if the SYSTICK counter has reached 0 since the last time the timer has reached 0. Clears to 0 on read.
		(2) CLKSOURCE	(2) CLKSOURCE	0x0	SYSTICK timer clock source
		(1) TICKINT	(1) TICKINT	0x0	SYSTICK timer interrupt enable
		(0) ENABLE	(0) ENABLE	0x0	SYSTICK timer enable
0xE000E014	SysTick_LOAD	(23:0) RELOAD	(23:0) RELOAD	0x0	Counter reload value for the SYSTICK timer when it reaches 0
0xE000E018	SysTick_VAL	(23:0) CURRENT	(23:0) CURRENT	0x0	Current value of the SYSTICK counter value. Write to clear counter.
0xE000E01C	SysTick_CALIB	-	(31) NOREF	0x0	Indicates if a reference clock is available
		-	(30) SKEW	0x0	Indicates if a calibration value is exactly 10 ms or not
		-	(23:0) TENMS	0x2710	SYSTICK counter calibration value for 10 ms. A value of 0 means the calibration value is not available

A.11 NESTED VECTOR INTERRUPT CONTROLLER

Address	Register Name	Register Write	Register Read	Default	Description
0xE000E004	SCnSCB_ICTR	-	(4:0) INTLINESNUM	0x1	Number of interrupt inputs in steps of 32

Address	Register Name	Register Write	Register Read	Default	Description
0xE000E100	NVIC_ISER0	(31) CVSD_DEC_OUT_RDY	(31) CVSD_DEC_OUT_RDY	0x0	CVSD CODEC decoder output interface interrupt set enable
		(30) CVSD_DEC_IN_RDY	(30) CVSD_DEC_IN_RDY	0x0	CVSD CODEC decoder input interface interrupt set enable
		(29) CVSD_ENC_OUT_RDY	(29) CVSD_ENC_OUT_RDY	0x0	CVSD CODEC encoder output interface interrupt set enable
		(28) CVSD_ENC_IN_RDY	(28) CVSD_ENC_IN_RDY	0x0	CVSD CODEC encoder input interface interrupt set enable
		(27) I2C	(27) I2C	0x0	I^2C interrupt set enable
		(26) PCM_ERROR	(26) PCM_ERROR	0x0	PCM error interrupt set enable
		(25) PCM_TX	(25) PCM_TX	0x0	PCM transmit buffer empty interrupt set enable
		(24) PCM_RX	(24) PCM_RX	0x0	PCM receive buffer full interrupt set enable
		(23) SPI_ERROR	(23) SPI_ERROR	0x0	SPI error interrupt set enable
		(22) SPI_TX	(22) SPI_TX	0x0	SPI transmit buffer empty interrupt set enable
		(21) SPI_RX	(21) SPI_RX	0x0	SPI receive buffer full interrupt set enable
		(20) GPIO	(20) GPIO	0x0	GPIO interrupt set enable
		(19) DMA3	(19) DMA3	0x0	DMA channel 3 interrupt set enable
		(18) DMA2	(18) DMA2	0x0	DMA channel 2 interrupt set enable
		(17) DMA1	(17) DMA1	0x0	DMA channel 1 interrupt set enable
		(16) DMA0	(16) DMA0	0x0	DMA channel 0 interrupt set enable
		(15) CFX_7	(15) CFX_7	0x0	CFX event interrupt 7 set enable
		(14) CFX_6	(14) CFX_6	0x0	CFX event interrupt 6 set enable
		(13) CFX_5	(13) CFX_5	0x0	CFX event interrupt 5 set enable
		(12) CFX_4	(12) CFX_4	0x0	CFX event interrupt 4 set enable
		(11) CFX_3	(11) CFX_3	0x0	CFX event interrupt 3 set enable
		(10) CFX_2	(10) CFX_2	0x0	CFX event interrupt 2 set enable
		(9) CFX_1	(9) CFX_1	0x0	CFX event interrupt 1 set enable
		(8) CFX_0	(8) CFX_0	0x0	CFX event interrupt 0 set enable
		(7) FIFO_7	(7) FIFO_7	0x0	FIFO interrupt 7 set enable
		(6) FIFO_6	(6) FIFO_6	0x0	FIFO interrupt 6 set enable
		(5) FIFO_5	(5) FIFO_5	0x0	FIFO interrupt 5 set enable
		(4) FIFO_4	(4) FIFO_4	0x0	FIFO interrupt 4 set enable
		(3) FIFO_3	(3) FIFO_3	0x0	FIFO interrupt 3 set enable
		(2) FIFO_2	(2) FIFO_2	0x0	FIFO interrupt 2 set enable
		(1) FIFO_1	(1) FIFO_1	0x0	FIFO interrupt 1 set enable
		(0) FIFO_0	(0) FIFO_0	0x1	FIFO interrupt 0 set enable

Address	Register Name	Register Write	Register Read	Default	Description
0xE000E104	NVIC_ISER1	(9) ACB_COM	(9) ACB_COM	0x0	ACB combined interrupt set enable
		(8) ECB_COM	(8) ECB_COM	0x0	ECB combined interrupt set enable
		(7) RS_DEC_OUT_RDY	(7) RS_DEC_OUT_RDY	0x0	RS decoder output registers interrupt set enable
		(6) RS_DEC_IN_RDY	(6) RS_DEC_IN_RDY	0x0	RS decoder input registers interrupt set enable
		(5) RS_ENC_OUT_RDY	(5) RS_ENC_OUT_RDY	0x0	RS encoder output registers interrupt set enable
		(4) RS_ENC_IN_RDY	(4) RS_ENC_IN_RDY	0x0	RS encoder input registers interrupt set enable
		(3) G722_DEC_OUT_RDY	(3) G722_DEC_OUT_RDY	0x0	G.722 CODEC decoder output interface registers interrupt set enable
		(2) G722_DEC_IN_RDY	(2) G722_DEC_IN_RDY	0x0	G.722 CODEC decoder input interface registers interrupt set enable
		(1) G722_ENC_OUT_RDY	(1) G722_ENC_OUT_RDY	0x0	G.722 CODEC encoder output interface registers interrupt set enable
		(0) G722_ENC_IN_RDY	(0) G722_ENC_IN_RDY	0x0	G.722 CODEC encoder input interface registers interrupt set enable

Address	Register Name	Register Write	Register Read	Default	Description
0xE000E180	NVIC_ICER0	(31) CVSD_DEC_OUT_RDY	(31) CVSD_DEC_OUT_RDY	0x0	CVSD CODEC decoder output interface interrupt clear enable
		(30) CVSD_DEC_IN_RDY	(30) CVSD_DEC_IN_RDY	0x0	CVSD CODEC decoder input interface interrupt clear enable
		(29) CVSD_ENC_OUT_RDY	(29) CVSD_ENC_OUT_RDY	0x0	CVSD CODEC encoder output interface interrupt clear enable
		(28) CVSD_ENC_IN_RDY	(28) CVSD_ENC_IN_RDY	0x0	CVSD CODEC encoder input interface interrupt clear enable
		(27) I2C	(27) I2C	0x0	I^2C interrupt clear enable
		(26) PCM_ERROR	(26) PCM_ERROR	0x0	PCM error interrupt clear enable
		(25) PCM_TX	(25) PCM_TX	0x0	PCM transmit buffer empty interrupt clear enable
		(24) PCM_RX	(24) PCM_RX	0x0	PCM receive buffer full interrupt clear enable
		(23) SPI_ERROR	(23) SPI_ERROR	0x0	SPI error interrupt clear enable
		(22) SPI_TX	(22) SPI_TX	0x0	SPI transmit buffer empty interrupt clear enable
		(21) SPI_RX	(21) SPI_RX	0x0	SPI receive buffer full interrupt clear enable
		(20) GPIO	(20) GPIO	0x0	GPIO interrupt clear enable
		(19) DMA3	(19) DMA3	0x0	DMA channel 3 interrupt clear enable
		(18) DMA2	(18) DMA2	0x0	DMA channel 2 interrupt clear enable
		(17) DMA1	(17) DMA1	0x0	DMA channel 1 interrupt clear enable
		(16) DMA0	(16) DMA0	0x0	DMA channel 0 interrupt clear enable
		(15) CFX_7	(15) CFX_7	0x0	CFX event interrupt 7 clear enable
		(14) CFX_6	(14) CFX_6	0x0	CFX event interrupt 6 clear enable
		(13) CFX_5	(13) CFX_5	0x0	CFX event interrupt 5 clear enable
		(12) CFX_4	(12) CFX_4	0x0	CFX event interrupt 4 clear enable
		(11) CFX_3	(11) CFX_3	0x0	CFX event interrupt 3 clear enable
		(10) CFX_2	(10) CFX_2	0x0	CFX event interrupt 2 clear enable
		(9) CFX_1	(9) CFX_1	0x0	CFX event interrupt 1 clear enable
		(8) CFX_0	(8) CFX_0	0x0	CFX event interrupt 0 clear enable
		(7) FIFO_7	(7) FIFO_7	0x0	FIFO interrupt 7 clear enable
		(6) FIFO_6	(6) FIFO_6	0x0	FIFO interrupt 6 clear enable
		(5) FIFO_5	(5) FIFO_5	0x0	FIFO interrupt 5 clear enable
		(4) FIFO_4	(4) FIFO_4	0x0	FIFO interrupt 4 clear enable
		(3) FIFO_3	(3) FIFO_3	0x0	FIFO interrupt 3 clear enable
		(2) FIFO_2	(2) FIFO_2	0x0	FIFO interrupt 2 clear enable
		(1) FIFO_1	(1) FIFO_1	0x0	FIFO interrupt 1 clear enable
		(0) FIFO_0	(0) FIFO_0	0x1	FIFO interrupt 0 clear enable

Address	Register Name	Register Write	Register Read	Default	Description
0xE000E184	NVIC_ICER1	(9) ACB_COM	(9) ACB_COM	0x0	ACB combined interrupt clear enable
		(8) ECB_COM	(8) ECB_COM	0x0	ECB combined interrupt clear enable
		(7) RS_DEC_OUT_RDY	(7) RS_DEC_OUT_RDY	0x0	RS decoder output registers interrupt clear enable
		(6) RS_DEC_IN_RDY	(6) RS_DEC_IN_RDY	0x0	RS decoder input registers interrupt clear enable
		(5) RS_ENC_OUT_RDY	(5) RS_ENC_OUT_RDY	0x0	RS encoder output registers interrupt clear enable
		(4) RS_ENC_IN_RDY	(4) RS_ENC_IN_RDY	0x0	RS encoder input registers interrupt clear enable
		(3) G722_DEC_OUT_RDY	(3) G722_DEC_OUT_RDY	0x0	G.722 CODEC decoder output interface registers interrupt clear enable
		(2) G722_DEC_IN_RDY	(2) G722_DEC_IN_RDY	0x0	G.722 CODEC decoder input interface registers interrupt clear enable
		(1) G722_ENC_OUT_RDY	(1) G722_ENC_OUT_RDY	0x0	G.722 CODEC encoder output interface registers interrupt clear enable
		(0) G722_ENC_IN_RDY	(0) G722_ENC_IN_RDY	0x0	G.722 CODEC encoder input interface registers interrupt clear enable

Address	Register Name	Register Write	Register Read	Default	Description
0xE000E200	NVIC_ISPR0	(31) CVSD_DEC_OUT_RDY	(31) CVSD_DEC_OUT_RDY	0x0	CVSD CODEC decoder output interface interrupt set pending
		(30) CVSD_DEC_IN_RDY	(30) CVSD_DEC_IN_RDY	0x0	CVSD CODEC decoder input interface interrupt set pending
		(29) CVSD_ENC_OUT_RDY	(29) CVSD_ENC_OUT_RDY	0x0	CVSD CODEC encoder output interface interrupt set pending
		(28) CVSD_ENC_IN_RDY	(28) CVSD_ENC_IN_RDY	0x0	CVSD CODEC encoder input interface interrupt set pending
		(27) I2C	(27) I2C	0x0	I^2C interrupt set pending
		(26) PCM_ERROR	(26) PCM_ERROR	0x0	PCM error interrupt set pending
		(25) PCM_TX	(25) PCM_TX	0x0	PCM transmit buffer empty interrupt set pending
		(24) PCM_RX	(24) PCM_RX	0x0	PCM receive buffer full interrupt set pending
		(23) SPI_ERROR	(23) SPI_ERROR	0x0	SPI error interrupt set pending
		(22) SPI_TX	(22) SPI_TX	0x0	SPI transmit buffer empty interrupt set pending
		(21) SPI_RX	(21) SPI_RX	0x0	SPI receive buffer full interrupt set pending
		(20) GPIO	(20) GPIO	0x0	GPIO interrupt set pending
		(19) DMA3	(19) DMA3	0x0	DMA channel 3 interrupt set pending
		(18) DMA2	(18) DMA2	0x0	DMA channel 2 interrupt set pending
		(17) DMA1	(17) DMA1	0x0	DMA channel 1 interrupt set pending
		(16) DMA0	(16) DMA0	0x0	DMA channel 0 interrupt set pending
		(15) CFX_7	(15) CFX_7	0x0	CFX event interrupt 7 set pending
		(14) CFX_6	(14) CFX_6	0x0	CFX event interrupt 6 set pending
		(13) CFX_5	(13) CFX_5	0x0	CFX event interrupt 5 set pending
		(12) CFX_4	(12) CFX_4	0x0	CFX event interrupt 4 set pending
		(11) CFX_3	(11) CFX_3	0x0	CFX event interrupt 3 set pending
		(10) CFX_2	(10) CFX_2	0x0	CFX event interrupt 2 set pending
		(9) CFX_1	(9) CFX_1	0x0	CFX event interrupt 1 set pending
		(8) CFX_0	(8) CFX_0	0x0	CFX event interrupt 0 set pending
		(7) FIFO_7	(7) FIFO_7	0x0	FIFO interrupt 7 set pending
		(6) FIFO_6	(6) FIFO_6	0x0	FIFO interrupt 6 set pending
		(5) FIFO_5	(5) FIFO_5	0x0	FIFO interrupt 5 set pending
		(4) FIFO_4	(4) FIFO_4	0x0	FIFO interrupt 4 set pending
		(3) FIFO_3	(3) FIFO_3	0x0	FIFO interrupt 3 set pending
		(2) FIFO_2	(2) FIFO_2	0x0	FIFO interrupt 2 set pending
		(1) FIFO_1	(1) FIFO_1	0x0	FIFO interrupt 1 set pending
		(0) FIFO_0	(0) FIFO_0	0x1	FIFO interrupt 0 set pending

Address	Register Name	Register Write	Register Read	Default	Description
0xE000E204	NVIC_ISPR1	(9) ACB_COM	(9) ACB_COM	0x0	ACB combined interrupt set pending
		(8) ECB_COM	(8) ECB_COM	0x0	ECB combined interrupt set pending
		(7) RS_DEC_OUT_RDY	(7) RS_DEC_OUT_RDY	0x0	RS decoder output registers interrupt set pending
		(6) RS_DEC_IN_RDY	(6) RS_DEC_IN_RDY	0x0	RS decoder input registers interrupt set pending
		(5) RS_ENC_OUT_RDY	(5) RS_ENC_OUT_RDY	0x0	RS encoder output registers interrupt set pending
		(4) RS_ENC_IN_RDY	(4) RS_ENC_IN_RDY	0x0	RS encoder input registers interrupt set pending
		(3) G722_DEC_OUT_RDY	(3) G722_DEC_OUT_RDY	0x0	G.722 CODEC decoder output interface registers interrupt set pending
		(2) G722_DEC_IN_RDY	(2) G722_DEC_IN_RDY	0x0	G.722 CODEC decoder input interface registers interrupt set pending
		(1) G722_ENC_OUT_RDY	(1) G722_ENC_OUT_RDY	0x0	G.722 CODEC encoder output interface registers interrupt set pending
		(0) G722_ENC_IN_RDY	(0) G722_ENC_IN_RDY	0x0	G.722 CODEC encoder input interface registers interrupt set pending

Address	Register Name	Register Write	Register Read	Default	Description
0xE000E280	NVIC_ICPR0	(31) CVSD_DEC_OUT_RDY	(31) CVSD_DEC_OUT_RDY	0x0	CVSD CODEC decoder output interface interrupt clear pending
		(30) CVSD_DEC_IN_RDY	(30) CVSD_DEC_IN_RDY	0x0	CVSD CODEC decoder input interface interrupt clear pending
		(29) CVSD_ENC_OUT_RDY	(29) CVSD_ENC_OUT_RDY	0x0	CVSD CODEC encoder output interface interrupt clear pending
		(28) CVSD_ENC_IN_RDY	(28) CVSD_ENC_IN_RDY	0x0	CVSD CODEC encoder input interface interrupt clear pending
		(27) I2C	(27) I2C	0x0	I^2C interrupt clear pending
		(26) PCM_ERROR	(26) PCM_ERROR	0x0	PCM error interrupt clear pending
		(25) PCM_TX	(25) PCM_TX	0x0	PCM transmit buffer empty interrupt clear pending
		(24) PCM_RX	(24) PCM_RX	0x0	PCM receive buffer full interrupt clear pending
		(23) SPI_ERROR	(23) SPI_ERROR	0x0	SPI error interrupt clear pending
		(22) SPI_TX	(22) SPI_TX	0x0	SPI transmit buffer empty interrupt clear pending
		(21) SPI_RX	(21) SPI_RX	0x0	SPI receive buffer full interrupt clear pending
		(20) GPIO	(20) GPIO	0x0	GPIO interrupt clear pending
		(19) DMA3	(19) DMA3	0x0	DMA channel 3 interrupt clear pending
		(18) DMA2	(18) DMA2	0x0	DMA channel 2 interrupt clear pending
		(17) DMA1	(17) DMA1	0x0	DMA channel 1 interrupt clear pending
		(16) DMA0	(16) DMA0	0x0	DMA channel 0 interrupt clear pending
		(15) CFX_7	(15) CFX_7	0x0	CFX event interrupt 7 clear pending
		(14) CFX_6	(14) CFX_6	0x0	CFX event interrupt 6 clear pending
		(13) CFX_5	(13) CFX_5	0x0	CFX event interrupt 5 clear pending
		(12) CFX_4	(12) CFX_4	0x0	CFX event interrupt 4 clear pending
		(11) CFX_3	(11) CFX_3	0x0	CFX event interrupt 3 clear pending
		(10) CFX_2	(10) CFX_2	0x0	CFX event interrupt 2 clear pending
		(9) CFX_1	(9) CFX_1	0x0	CFX event interrupt 1 clear pending
		(8) CFX_0	(8) CFX_0	0x0	CFX event interrupt 0 clear pending
		(7) FIFO_7	(7) FIFO_7	0x0	FIFO interrupt 7 clear pending
		(6) FIFO_6	(6) FIFO_6	0x0	FIFO interrupt 6 clear pending
		(5) FIFO_5	(5) FIFO_5	0x0	FIFO interrupt 5 clear pending
		(4) FIFO_4	(4) FIFO_4	0x0	FIFO interrupt 4 clear pending
		(3) FIFO_3	(3) FIFO_3	0x0	FIFO interrupt 3 clear pending
		(2) FIFO_2	(2) FIFO_2	0x0	FIFO interrupt 2 clear pending
		(1) FIFO_1	(1) FIFO_1	0x0	FIFO interrupt 1 clear pending
		(0) FIFO_0	(0) FIFO_0	0x1	FIFO interrupt 0 clear pending

Address	Register Name	Register Write	Register Read	Default	Description
0xE000E284	NVIC_ICPR1	(9) ACB_COM	(9) ACB_COM	0x0	ACB combined interrupt clear pending
		(8) ECB_COM	(8) ECB_COM	0x0	ECB combined interrupt clear pending
		(7) RS_DEC_OUT_RDY	(7) RS_DEC_OUT_RDY	0x0	RS decoder output registers interrupt clear pending
		(6) RS_DEC_IN_RDY	(6) RS_DEC_IN_RDY	0x0	RS decoder input registers interrupt clear pending
		(5) RS_ENC_OUT_RDY	(5) RS_ENC_OUT_RDY	0x0	RS encoder output registers interrupt clear pending
		(4) RS_ENC_IN_RDY	(4) RS_ENC_IN_RDY	0x0	RS encoder input registers interrupt clear pending
		(3) G722_DEC_OUT_RDY	(3) G722_DEC_OUT_RDY	0x0	G.722 CODEC decoder output interface registers interrupt clear pending
		(2) G722_DEC_IN_RDY	(2) G722_DEC_IN_RDY	0x0	G.722 CODEC decoder input interface registers interrupt clear pending
		(1) G722_ENC_OUT_RDY	(1) G722_ENC_OUT_RDY	0x0	G.722 CODEC encoder output interface registers interrupt clear pending
		(0) G722_ENC_IN_RDY	(0) G722_ENC_IN_RDY	0x0	G.722 CODEC encoder input interface registers interrupt clear pending

Address	Register Name	Register Write	Register Read	Default	Description
0xE000E300	NVIC_IABR0	-	(31) CVSD_DEC_OUT_RDY	0x0	CVSD CODEC decoder output interface interrupt active status
		-	(30) CVSD_DEC_IN_RDY	0x0	CVSD CODEC decoder input interface interrupt active status
		-	(29) CVSD_ENC_OUT_RDY	0x0	CVSD CODEC encoder output interface interrupt active status
		-	(28) CVSD_ENC_IN_RDY	0x0	CVSD CODEC encoder input interface interrupt active status
		-	(27) I2C	0x0	I^2C interrupt active status
		-	(26) PCM_ERROR	0x0	PCM error interrupt active status
		-	(25) PCM_TX	0x0	PCM transmit buffer empty interrupt active status
		-	(24) PCM_RX	0x0	PCM receive buffer full interrupt active status
		-	(23) SPI_ERROR	0x0	SPI error interrupt active status
		-	(22) SPI_TX	0x0	SPI transmit buffer empty interrupt active status
		-	(21) SPI_RX	0x0	SPI receive buffer full interrupt active status
		-	(20) GPIO	0x0	GPIO interrupt active status
		-	(19) DMA3	0x0	DMA channel 3 interrupt active status
		-	(18) DMA2	0x0	DMA channel 2 interrupt active status
		-	(17) DMA1	0x0	DMA channel 1 interrupt active status
		-	(16) DMA0	0x0	DMA channel 0 interrupt active status
		-	(15) CFX_7	0x0	CFX event interrupt 7 active status
		-	(14) CFX_6	0x0	CFX event interrupt 6 active status
		-	(13) CFX_5	0x0	CFX event interrupt 5 active status
		-	(12) CFX_4	0x0	CFX event interrupt 4 active status
		-	(11) CFX_3	0x0	CFX event interrupt 3 active status
		-	(10) CFX_2	0x0	CFX event interrupt 2 active status
		-	(9) CFX_1	0x0	CFX event interrupt 1 active status
		-	(8) CFX_0	0x0	CFX event interrupt 0 active status
		-	(7) FIFO_7	0x0	FIFO interrupt 7 active status
		-	(6) FIFO_6	0x0	FIFO interrupt 6 active status
		(5) FIFO_5	(5) FIFO_5	0x0	FIFO interrupt 5 active status
		(4) FIFO_4	(4) FIFO_4	0x0	FIFO interrupt 4 active status
		(3) FIFO_3	(3) FIFO_3	0x0	FIFO interrupt 3 active status
		(2) FIFO_2	(2) FIFO_2	0x0	FIFO interrupt 2 active status
		-	(1) FIFO_1	0x0	FIFO interrupt 1 active status
		-	(0) FIFO_0	0x1	FIFO interrupt 0 active status

Address	Register Name	Register Write	Register Read	Default	Description
0xE000E304	NVIC_IABR1	(9) ACB_COM	(9) ACB_COM	0x0	ACB combined interrupt active status
		(8) ECB_COM	(8) ECB_COM	0x0	ECB combined interrupt active status
		(7) RS_DEC_OUT_RDY	(7) RS_DEC_OUT_RDY	0x0	RS decoder output registers interrupt active status
		(6) RS_DEC_IN_RDY	(6) RS_DEC_IN_RDY	0x0	RS decoder input registers interrupt active status
		(5) RS_ENC_OUT_RDY	(5) RS_ENC_OUT_RDY	0x0	RS encoder output registers interrupt active status
		-	(4) RS_ENC_IN_RDY	0x0	RS encoder input registers interrupt active status
		-	(3) G722_DEC_OUT_RDY	0x0	G.722 CODEC decoder output interface registers interrupt active status
		-	(2) G722_DEC_IN_RDY	0x0	G.722 CODEC decoder input interface registers interrupt active status
		-	(1) G722_ENC_OUT_RDY	0x0	G.722 CODEC encoder output interface registers interrupt active status
		-	(0) G722_ENC_IN_RDY	0x0	G.722 CODEC encoder input interface registers interrupt active status
0xE000E400	NVIC_IP0	(31:24) FIFO_3_PRIORITY	(31:24) FIFO_3_PRIORITY	0x0	Configure the FIFO interrupt 3 priority
		(23:16) FIFO_2_PRIORITY	(23:16) FIFO_2_PRIORITY	0x0	Configure the FIFO interrupt 2 priority
		(15:8) FIFO_1_PRIORITY	(15:8) FIFO_1_PRIORITY	0x0	Configure the FIFO interrupt 1 priority
		(7:0) FIFO_0_PRIORITY	(7:0) FIFO_0_PRIORITY	0x0	Configure the FIFO interrupt 0 priority
0xE000E404	NVIC_IP1	(31:24) FIFO_7_PRIORITY	(31:24) FIFO_7_PRIORITY	0x0	Configure the FIFO interrupt 7 priority
		(23:16) FIFO_6_PRIORITY	(23:16) FIFO_6_PRIORITY	0x0	Configure the FIFO interrupt 6 priority
		(15:8) FIFO_5_PRIORITY	(15:8) FIFO_5_PRIORITY	0x0	Configure the FIFO interrupt 5 priority
		(7:0) FIFO_4_PRIORITY	(7:0) FIFO_4_PRIORITY	0x0	Configure the FIFO interrupt 4 priority
0xE000E408	NVIC_IP2	(31:24) CFX_3_PRIORITY	(31:24) CFX_3_PRIORITY	0x0	Configure the CFX event interrupt 3 priority
		(23:16) CFX_2_PRIORITY	(23:16) CFX_2_PRIORITY	0x0	Configure the CFX event interrupt 2 priority
		(15:8) CFX_1_PRIORITY	(15:8) CFX_1_PRIORITY	0x0	Configure the CFX event interrupt 1 priority
		(7:0) CFX_0_PRIORITY	(7:0) CFX_0_PRIORITY	0x0	Configure the CFX event interrupt 0 priority
0xE000E40C	NVIC_IP3	(31:24) CFX_7_PRIORITY	(31:24) CFX_7_PRIORITY	0x0	Configure the CFX event interrupt 7 priority
		(23:16) CFX_6_PRIORITY	(23:16) CFX_6_PRIORITY	0x0	Configure the CFX event interrupt 6 priority
		(15:8) CFX_5_PRIORITY	(15:8) CFX_5_PRIORITY	0x0	Configure the CFX event interrupt 5 priority
		(7:0) CFX_4_PRIORITY	(7:0) CFX_4_PRIORITY	0x0	Configure the CFX event interrupt 4 priority
0xE000E410	NVIC_IP4	(31:24) DMA3_PRIORITY	(31:24) DMA3_PRIORITY	0x0	Configure the DMA channel 3 interrupt priority
		(23:16) DMA2_PRIORITY	(23:16) DMA2_PRIORITY	0x0	Configure the DMA channel 2 interrupt priority
		(15:8) DMA1_PRIORITY	(15:8) DMA1_PRIORITY	0x0	Configure the DMA channel 1 interrupt priority
		(7:0) DMA0_PRIORITY	(7:0) DMA0_PRIORITY	0x0	Configure the DMA channel 0 interrupt priority

Address	Register Name	Register Write	Register Read	Default	Description
0xE000E414	NVIC_IP5	(31:24) SPI_ERROR_PRIORITY	(31:24) SPI_ERROR_PRIORITY	0x0	Configure the SPI error interrupt priority
		(23:16) SPI_TX_PRIORITY	(23:16) SPI_TX_PRIORITY	0x0	Configure the SPI TX interrupt priority
		(15:8) SPI_RX_PRIORITY	(15:8) SPI_RX_PRIORITY	0x0	Configure the SPI RX interrupt priority
		(7:0) GPIO_PRIORITY	(7:0) GPIO_PRIORITY	0x0	Configure the GPIO interrupt priority
0xE000E418	NVIC_IP6	(31:24) I2C_PRIORITY	(31:24) I2C_PRIORITY	0x0	Configure the I^2C interrupt priority
		(23:16) PCM_ERROR_PRIORITY	(23:16) PCM_ERROR_PRIORITY	0x0	Configure the PCM error interrupt priority
		(15:8) PCM_TX_PRIORITY	(15:8) PCM_TX_PRIORITY	0x0	Configure the PCM TX interrupt priority
		(7:0) PCM_RX_PRIORITY	(7:0) PCM_RX_PRIORITY	0x0	Configure the PCM RX interrupt priority
0xE000E41C	NVIC_IP7	(31:24) CVSD_DEC_OUT_RDY_PRIORITY	(31:24) CVSD_DEC_OUT_RDY_PRIORITY	0x0	Configure the CVSD decoder out ready interrupt priority
		(23:16) CVSD_DEC_IN_RDY_PRIORITY	(23:16) CVSD_DEC_IN_RDY_PRIORITY	0x0	Configure the CVSD decoder in ready interrupt priority
		(15:8) CVSD_ENC_OUT_RDY_PRIORITY	(15:8) CVSD_ENC_OUT_RDY_PRIORITY	0x0	Configure the CVSD encoder out ready interrupt priority
		(7:0) CVSD_ENC_IN_RDY_PRIORITY	(7:0) CVSD_ENC_IN_RDY_PRIORITY	0x0	Configure the CVSD encoder in ready interrupt priority
0xE000E420	NVIC_IP8	(31:24) G722_DEC_OUT_RDY_PRIORITY	(31:24) G722_DEC_OUT_RDY_PRIORITY	0x0	Configure the G.722 decoder out ready interrupt priority
		(23:16) G722_DEC_IN_RDY_PRIORITY	(23:16) G722_DEC_IN_RDY_PRIORITY	0x0	Configure the G.722 decoder in ready interrupt priority
		(15:8) G722_ENC_OUT_RDY_PRIORITY	(15:8) G722_ENC_OUT_RDY_PRIORITY	0x0	Configure the G.722 encoder out ready interrupt priority
		(7:0) G722_ENC_IN_RDY_PRIORITY	(7:0) G722_ENC_IN_RDY_PRIORITY	0x0	Configure the G.722 encoder in ready interrupt priority
0xE000E424	NVIC_IP9	(31:24) RS_DEC_OUT_RDY_PRIORITY	(31:24) RS_DEC_OUT_RDY_PRIORITY	0x0	Configure the Reed-Solomon decoder out ready interrupt priority
		(23:16) RS_DEC_IN_RDY_PRIORITY	(23:16) RS_DEC_IN_RDY_PRIORITY	0x0	Configure the Reed-Solomon decoder in ready interrupt priority
		(15:8) RS_ENC_OUT_RDY_PRIORITY	(15:8) RS_ENC_OUT_RDY_PRIORITY	0x0	Configure the Reed-Solomon encoder out ready interrupt priority
		(7:0) RS_ENC_IN_RDY_PRIORITY	(7:0) RS_ENC_IN_RDY_PRIORITY	0x0	Configure the Reed-Solomon encoder in ready interrupt priority
0xE000E428	NVIC_IP10	(15:8) ACB_COM_PRIORITY	(15:8) ACB_COM_PRIORITY	0x0	Configure the usage fault priority
		(7:0) ECB_COM_PRIORITY	(7:0) ECB_COM_PRIORITY	0x0	Configure the ECB combined interrupt priority
0xE000ED00	SCB_CPUID	-	(31:24) IMPLEMENTER	0x41	Implementer code
		-	(23:20) VARIANT	0x2	Implementation variant
		-	(19:16) ARCHITECTURE	0xF	Architecture number
		-	(15:4) PARTNO	0xC23	Part number
		-	(3:0) REVISION	0x1	Revision code

Address	Register Name	Register Write	Register Read	Default	Description
0xE000ED04	SCB_ICSR	(31) NMIPENDSET	(31) NMIPENDSET	0x0	Indicates the NMI exception is pending
		(28) PENDSVSET	(28) PENDSVSET	0x0	Write 1 to set pending status for PendSV exception. Read indicates PendSV pending status
		(27) PENDSVCLR	-	N/A	Write 1 to clear PendSV exception pending status
		(26) PENDSTSET	(26) PENDSTSET	0x0	Write 1 to set pending status for SYSTICK exception. Read value indicates SYSTICK pending status
		(25) PENDSTCLR	-	N/A	Write 1 to clear SYSTICK exception pending status
		-	(23) ISRPREEMPT	0x0	Indicates that a pending interrupt is going to be active in the next step
		-	(22) ISRPENDING	0x0	Indicates that an external interrupt is pending
		-	(21:12) VECTPENDING	0x0	Pending external interrupt number
		-	(11) RETTOBASE	0x0	Set to 1 when an exception handler is being run.
		-	(9:0) VECTACTIVE	0x0	Number of current running interrupt service routine
0xE000ED08	SCB_VTOR	(31:7) TBLOFF	(31:7) TBLOFF	0x0	Table offset value in code or RAM region. Must be multiple of 128.
0xE000ED0C	SCB_AIRCR	(31:16) VECTKEY	(31:16) VECTKEY	0xFA05	Access key for writing this register. Must be set to 0x05FA to write the other register fields.
		-	(15) ENDIANESS	0x0	Indicates endianness for data.
		(10:8) PRIGROUP	(10:8) PRIGROUP	0x7	Priority group setting. Controls how many bits in the priority register are used for pre-empty priority vs. sub-priority.
		(2) SYSRESETREQ	-	N/A	Requests a chip-level system reset
		(1) VECTCLRACTIVE	-	N/A	Clears all active exception state information
		(0) VECTRESET	-	N/A	Reset the ARM Cortex-M3 processor
0xE000ED10	SCB_SCR	(4) SEVONPEND	(4) SEVONPEND	0x0	Set to 1 to cause the WFE to wake up if a new interrupt is pended
		(2) SLEEPDEEP	(2) SLEEPDEEP	0x0	Enable SLEEPDEEP output signal when entering sleep mode
		(1) SLEEPONEXIT	(1) SLEEPONEXIT	0x0	Enable sleep on exit feature when returning from handler to thread mode
0xE000ED14	SCB_CCR	(9) STKALIGN	(9) STKALIGN	0x0	Force stack to be double-word aligned
		(8) BFHFNMIGN	(8) BFHFNMIGN	0x0	Ignore data bus faults in hard fault and NMI exceptions
		(4) DIV_0_TRP	(4) DIV_0_TRP	0x0	Trap on divide by zero
		(3) UNALIGN_TRP	(3) UNALIGN_TRP	0x0	Trap on unaligned data access
		(1) USERSETMPEND	(1) USERSETMPEND	0x0	Allow user code to write to software trigger interrupt register
		(0) NONBASETHRDENA	(0) NONBASETHRDENA	0x0	Non base thread mode enable

Address	Register Name	Register Write	Register Read	Default	Description
0xE000ED18	SCB_SHP0	(23:16) NVIC_USAGE_FAULT_PRIORITY	(23:16) NVIC_USAGE_FAULT_PRIORITY	0x0	Configure the usage fault priority
		(15:8) NVIC_BUS_FAULT_PRIORITY	(15:8) NVIC_BUS_FAULT_PRIORITY	0x0	Configure the bus fault priority
		(7:0) NVIC_MEM_FAULT_PRIORITY	(7:0) NVIC_MEM_FAULT_PRIORITY	0x0	Configure the memory fault priority
0xE000ED1C	SCB_SHP1	(31:24) NVIC_SVC_PRIORITY	(31:24) NVIC_SVC_PRIORITY	0x0	Configure the SVC priority
0xE000ED20	SCB_SHP2	(31:24) NVIC_SYSTICK_PRIORITY	(31:24) NVIC_SYSTICK_PRIORITY	0x0	Configure the SysTick interrupt priority
		(23:16) NVIC_PENDSV_PRIORITY	(23:16) NVIC_PENDSV_PRIORITY	0x0	Configure the PendSV priority
		(7:0) NVIC_MONITOR_PRIORITY	(7:0) NVIC_MONITOR_PRIORITY	0x0	Configure the monitor priority
0xE000ED24	SCB_SHCSR	(18) USGFAULTENA	(18) USGFAULTENA	0x0	Usage fault handler enable
		(17) BUSFAULTENA	(17) BUSFAULTENA	0x0	Bus fault handler enable
		(16) MEMFAULTENA	(16) MEMFAULTENA	0x0	Memory management fault handler enable
		(15) SVCALLPENDED	(15) SVCALLPENDED	0x0	SVCall is pending or was started and replaced by a higher priority exception
		(14) BUSFAULTPENDED	(14) BUSFAULTPENDED	0x0	Bus fault is pending or was started and replaced by a higher priority exception
		(13) MEMFAULTPENDED	(13) MEMFAULTPENDED	0x0	Memory management fault is pending or was started and replaced by a higher priority exception
		(12) USGFAULTPENDED	(12) USGFAULTPENDED	0x0	Usage fault is pending or was started and replaced by a higher priority exception
		(11) SYSTICKACT	(11) SYSTICKACT	0x0	SYSTICK exception handler is active
		(10) PENDSVACT	(10) PENDSVACT	0x0	PendSV exception handler is active
		(8) MONITORACT	(8) MONITORACT	0x0	Debug monitor exception handler is active
		(7) SVCALLACT	(7) SVCALLACT	0x0	SVCall exception handler is active
		(3) USGFAULTACT	(3) USGFAULTACT	0x0	Usage fault exception handler is active
		(1) BUSFAULTACT	(1) BUSFAULTACT	0x0	Bus fault exception handler is active
		(0) MEMFAULTACT	(0) MEMFAULTACT	0x0	Memory management fault exception handler is active

Address	Register Name	Register Write	Register Read	Default	Description
0xE000ED28	SCB_CFSR	(25) DIVBYZERO	(25) DIVBYZERO	0x0	Indicates divide by zero will take place
		(24) UNALIGNED	(24) UNALIGNED	0x0	Indicates unaligned access will take place
		(19) NOCP	(19) NOCP	0x0	Attempt to execute a coprocessor instruction
		(18) INVPC	(18) INVPC	0x0	Attempt to do exception with bad value in EXC_RETURN number
		(17) INVSTATE	(17) INVSTATE	0x0	Attempt to switch to invalid (e.g. ARM) state
		(16) UNDEFINSTR	(16) UNDEFINSTR	0x0	Attempt to execute an undefined instruction
		-	(15) BFARVALID	0x0	Indicates if bus fault address register is valid
		(12) STKERR	(12) STKERR	0x0	Indicates stacking error
		(11) UNSTKERR	(11) UNSTKERR	0x0	Indicates unstacking error
		(10) IMPRECISERR	(10) IMPRECISERR	0x0	Indicates imprecise data access violation
		(9) PRECISERR	(9) PRECISERR	0x0	Indicates precise data access violation
		(8) IBUSERR	(8) IBUSERR	0x0	Indicates instruction access violation
		-	(7) MMARVALID	0x0	Indicates if memory management fault address register is valid
		(4) MSTKERR	(4) MSTKERR	0x0	Indicates stacking error
		(3) MUNSTKERR	(3) MUNSTKERR	0x0	Indicates unstacking error
		(1) DACCVIOL	(1) DACCVIOL	0x0	Indicates data access violation
		(0) IACCVIOL	(0) IACCVIOL	0x0	Indicates instruction access violation
0xE000ED2C	SCB_HFSR	(31) DEBUGEVT	(31) DEBUGEVT	0x0	Indicates hard fault is triggered by debug event
		(30) FORCED	(30) FORCED	0x0	Indicates hard fault is taken because of a lower priority (e.g., bus, memory management or usage) fault
		(1) VECTBL	(1) VECTBL	0x0	Indicates hard fault is taken due to failed vector fetch
0xE000ED30	SCB_DFSR	(4) EXTERNAL	(4) EXTERNAL	0x0	Indicates external debug request signal asserted
		(3) VCATCH	(3) VCATCH	0x0	Indicates vector fetch occurred
		(2) DWTTRAP	(2) DWTTRAP	0x0	Indicates DWT match occurred
		(1) BKPT	(1) BKPT	0x0	Indicates BKPT instruction executed
		(0) HALTED	(0) HALTED	0x0	Indicates halt requested by NVIC
0xE000ED34	SCB_MMFAR	-	(31:0) NVIC_MMAR	0x0	Memory monitor address register
0xE000ED38	SCB_BFAR	-	(31:0) NVIC_BFAR	0x0	Bus fault address register
0xE000EF00	NVIC_STIR	(7:0) INTID	-	N/A	The interrupt number to trigger

A.12 DEBUG CONTROLLER

Address	Register Name	Register Write	Register Read	Default	Description
0xE000EDF0	CoreDebug_DHCSR	(31:16) DBGKEY	-	N/A	Debug key must be written to this field in order to write the rest of the register
		-	(25) S_RESET_ST	0x0	Core has been reset or is being rest. Bit is cleared on read.
		-	(24) S_RETIRE_ST	0x0	Status bit indicating an instruction completed since this register was last read
		-	(19) S_LOCKUP	0x0	Indicates if core is in lockup state
		-	(18) S_SLEEP	0x0	Indicates if core is in sleep mode
		-	(17) S_HALT	0x0	Indicates is core is halted
		-	(16) S_REGRDY	0x0	Indicates register read/write operation is completed
		(5) C_SNAPSTALL	(5) C_SNAPSTALL	0x0	Set to break a stalled memory access
		(3) C_MASKINTS	(3) C_MASKINTS	0x0	Mask interrupts while stepping
		(2) C_STEP	(2) C_STEP	0x0	Single step the processor
		(1) C_HALT	(1) C_HALT	0x0	Halt the processor
		(0) C_DEBUGEN	(0) C_DEBUGEN	0x1	Enable halt mode debugging
0xE000EDF4	CoreDebug_DCRSR	(16) REGWnR	-	N/A	Indicates direction of register transfer
		(4:0) REGSEL	-	N/A	Indicates register to be accessed
0xE000EDF8	CoreDebug_DCRDR	(31:0) DEBUG_REGDATA	(31:0) DEBUG_REGDATA	0x0	Register read/write data for debugging
0xE000EDFC	CoreDebug_DEMCR	(24) TRCENA	(24) TRCENA	0x0	Trace system enable
		(19) MON_REQ	(19) MON_REQ	0x0	Indicates that the debug monitor is caused by a manual pending request rather than a hardware event
		(18) MON_STEP	(18) MON_STEP	0x0	Single step the processor
		(17) MON_PEND	(17) MON_PEND	0x0	Pend the monitor exception request
		(16) MON_EN	(16) MON_EN	0x0	Enable the debug monitor exception
		(10) VC_HARDERR	(10) VC_HARDERR	0x0	Debug trap on hard faults
		(9) VC_INTERR	(9) VC_INTERR	0x0	Debug trap on interrupt service errors
		(8) VC_BUSERR	(8) VC_BUSERR	0x0	Debug trap on bus faults
		(7) VC_STATERR	(7) VC_STATERR	0x0	Debug trap on usage fault state errors
		(6) VC_CHKERR	(6) VC_CHKERR	0x0	Debug trap on fault-enabled checking errors (e.g. unaligned access, divide by zero, etc.)
		(5) VC_NOCPERR	(5) VC_NOCPERR	0x0	Debug trap on usage fault no coprocessor errors
		(4) VC_MMERR	(4) VC_MMERR	0x0	Debug trap on memory management fault
		(0) VC_CORERESET	(0) VC_CORERESET	0x0	Debug trap on core reset

A.13 SYSTEM CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF000	D_SYSTEM_CTRL	(7) PRIORITY_HEAR	-	N/A	Write a 1 to give HEAR priority
		-	(6) PRIORITY_STATUS	0x0	CFX/HEAR priority status
		(6) PRIORITY_CFX	-	N/A	Write a 1 to give CFX priority
		(5) HEAR_PAUSE	-	N/A	Write a 1 to pause HEAR
		-	(4) HEAR_RUNNING	0x0	HEAR running status
		(4) HEAR_RESUME	-	N/A	Write a 1 to run HEAR
		(3) HEAR_RESET	-	N/A	Write a 1 to reset HEAR
		-	(2) TIMER_2_BUSY	0x0	Check status of timer 2
		(2) TIMER_2_START	-	N/A	Write a 1 to start timer 2
		-	(1) TIMER_1_BUSY	0x0	Check status of timer 1
		(1) TIMER_1_START	-	N/A	Write a 1 to start timer 1
		(0) WATCHDOG_REFRESH	-	N/A	Write a 1 to reset the watchdog

A.14 WATCHDOG TIMER CONFIGURATION

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF001	D_WATCHDOG_CFG	(3:0) TIMEOUT	(3:0) TIMEOUT	0xB	Watchdog timeout period. Values 0xC to 0xF result in the same timeout period as the value 0xB.

A.15 General Purpose Timers 1 and 2

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF002	D_TIMER_1_CFG	(16) START_STOP	(16) START_STOP	0x0	State of the timer
		(15) MODE	(15) MODE	0x0	Mode of operation of the timer
		(14:12) PRESCALE	(14:12) PRESCALE	0x0	Prescale value of the timer
		(11:0) DELAY	(11:0) DELAY	0x0	Expiration delay of the timer
P:0xF003	D_TIMER_2_CFG	(16) START_STOP	(16) START_STOP	0x0	State of the timer
		(15) MODE	(15) MODE	0x0	Mode of operation of the timer
		(14:12) PRESCALE	(14:12) PRESCALE	0x0	Prescale value of the timer
		(11:0) DELAY	(11:0) DELAY	0x0	Expiration delay of the timer

A.16 HEAR Function Chain Commands

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF004	D_HEAR_CMDS_0_TO_7	(7) HEAR_CMD_7	-	N/A	Write a 1 to issue HEAR function chain 7
		(6) HEAR_CMD_6	-	N/A	Write a 1 to issue HEAR function chain 6
		(5) HEAR_CMD_5	-	N/A	Write a 1 to issue HEAR function chain 5
		(4) HEAR_CMD_4	-	N/A	Write a 1 to issue HEAR function chain 4
		(3) HEAR_CMD_3	-	N/A	Write a 1 to issue HEAR function chain 3
		(2) HEAR_CMD_2	-	N/A	Write a 1 to issue HEAR function chain 2
		(1) HEAR_CMD_1	-	N/A	Write a 1 to issue HEAR function chain 1
		(0) HEAR_CMD_0	-	N/A	Write a 1 to issue HEAR function chain 0
P:0xF005	D_HEAR_CMDS_8_TO_15	(7) HEAR_CMD_15	-	N/A	Write a 1 to issue HEAR function chain 15
		(6) HEAR_CMD_14	-	N/A	Write a 1 to issue HEAR function chain 14
		(5) HEAR_CMD_13	-	N/A	Write a 1 to issue HEAR function chain 13
		(4) HEAR_CMD_12	-	N/A	Write a 1 to issue HEAR function chain 12
		(3) HEAR_CMD_11	-	N/A	Write a 1 to issue HEAR function chain 11
		(2) HEAR_CMD_10	-	N/A	Write a 1 to issue HEAR function chain 10
		(1) HEAR_CMD_9	-	N/A	Write a 1 to issue HEAR function chain 9
		(0) HEAR_CMD_8	-	N/A	Write a 1 to issue HEAR function chain 8

A.17 HEAR INTERRUPT MULTIPLEXING

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF009	D_HEAR_INT_SELECT	(2) HEAR_5_UART	(2) HEAR_5_UART	0x0	HEAR 5/UART interrupt selection
		(1) HEAR_7_SDU	(1) HEAR_7_SDU	0x0	HEAR 7/signal detection unit interrupt selection
		(0) HEAR_6_FENG	(0) HEAR_6_FENG	0x0	HEAR 6/Filter Engine interrupt selection

A.18 CFX/ARM CORTEX-M3 PROCESSOR CONTROL AND CONFIGURATION

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF00A	D_SYSTEM_CTRL_2	(7) PRIORITY_CM3	-	N/A	Write a 1 to give the ARM Cortex-M3 core priority
		-	(6) PRIORITY_STATUS	0x0	CFX/ARM Cortex-M3 core priority status
		(6) PRIORITY_CFX	-	N/A	Write a 1 to give CFX priority
		(5) CM3_PAUSE	-	N/A	Write a 1 to pause the ARM Cortex-M3 core
		-	(4) CM3_RUNNING	0x0	ARM Cortex-M3 core running status
		(4) CM3_RESUME	-	N/A	Write a 1 to run the ARM Cortex-M3 core
		(3) CM3_RESET	-	N/A	Write a 1 to reset the ARM Cortex-M3 core
		(2) CM3_SUBSYSTEM_RESET	-	N/A	Write a 1 to reset the ARM Cortex-M3 core and sub-system
		(1) PRAM0_CM3	-	N/A	Write a 1 to assign PRAM0 to the ARM Cortex-M3 core
		-	(0) PRAM0_STATUS	0x0	PRAM0 CFX/ARM Cortex-M3 core assignment status
		(0) PRAM0_CFX	-	N/A	Write a 1 to assign PRAM0 to CFX
P:0xF00B	D_CM3_CMDS	(7) CM3_CMD_7	-	N/A	Write a 1 to issue ARM Cortex-M3 processor interrupt 7
		(6) CM3_CMD_6	-	N/A	Write a 1 to issue ARM Cortex-M3 processor interrupt 6
		(5) CM3_CMD_5	-	N/A	Write a 1 to issue ARM Cortex-M3 processor interrupt 5
		(4) CM3_CMD_4	-	N/A	Write a 1 to issue ARM Cortex-M3 processor interrupt 4
		(3) CM3_CMD_3	-	N/A	Write a 1 to issue ARM Cortex-M3 processor interrupt 3
		(2) CM3_CMD_2	-	N/A	Write a 1 to issue ARM Cortex-M3 processor interrupt 2
		(1) CM3_CMD_1	-	N/A	Write a 1 to issue ARM Cortex-M3 processor interrupt 1
		(0) CM3_CMD_0	-	N/A	Write a 1 to issue ARM Cortex-M3 processor interrupt 0
P:0xF00C	D_CM3_NMI	(0) CM3_NMI_STATE	(0) CM3_NMI_STATE	0x0	ARM Cortex-M3 processor non-maskable interrupt

A.19 MEMORY CONFIGURATION

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF010	D_MEM_CFG_0	(3) ME_FORCE_PROM1	(3) ME_FORCE_PROM1	0x0	Force ME of PROM1
		(2) ME_FORCE_PROM0	(2) ME_FORCE_PROM0	0x0	Force ME of PROM0
		(1) PROM1_PD	(1) PROM1_PD	0x0	PROM1 power-down control
		(0) PROM0_PD	(0) PROM0_PD	0x0	PROM0 power-down control
P:0xF011	D_MEM_CFG_1	(6) ME_FORCE_CFX_YRAM0	(6) ME_FORCE_CFX_YRAM0	0x0	Force ME of CFX YRAM0
		(5) ME_FORCE_CFX_XRAM2	(5) ME_FORCE_CFX_XRAM2	0x0	Force ME of CFX XRAM2
		(4) ME_FORCE_CFX_XRAM1	(4) ME_FORCE_CFX_XRAM1	0x0	Force ME of CFX XRAM1
		(3) ME_FORCE_CFX_XRAM0	(3) ME_FORCE_CFX_XRAM0	0x0	Force ME of CFX XRAM0
		(2) ME_FORCE_CFX_PRAM2	(2) ME_FORCE_CFX_PRAM2	0x0	Force ME of CFX PRAM2
		(1) ME_FORCE_CFX_PRAM1	(1) ME_FORCE_CFX_PRAM1	0x0	Force ME of CFX PRAM1
		(0) ME_FORCE_CFX_PRAM0	(0) ME_FORCE_CFX_PRAM0	0x0	Force ME of CFX PRAM0
P:0xF012	D_MEM_CFG_2	(6) ME_FORCE_HEAR_PMEM	(6) ME_FORCE_HEAR_PMEM	0x0	Force ME of HEAR PMEM
		(5) ME_FORCE_HEAR_MEMT5	(5) ME_FORCE_HEAR_MEMT5	0x0	Force ME of HEAR MEMT5
		(4) ME_FORCE_HEAR_MEMT4	(4) ME_FORCE_HEAR_MEMT4	0x0	Force ME of HEAR MEMT4
		(3) ME_FORCE_HEAR_MEMT3	(3) ME_FORCE_HEAR_MEMT3	0x0	Force ME of HEAR MEMT3
		(2) ME_FORCE_HEAR_MEMT2	(2) ME_FORCE_HEAR_MEMT2	0x0	Force ME of HEAR MEMT2
		(1) ME_FORCE_HEAR_MEMT1	(1) ME_FORCE_HEAR_MEMT1	0x0	Force ME of HEAR MEMT1
		(0) ME_FORCE_HEAR_MEMT0	(0) ME_FORCE_HEAR_MEMT0	0x0	Force ME of HEAR MEMT0
P:0xF013	D_MEM_CFG_3	(7) ME_FORCE_HEAR_D1	(7) ME_FORCE_HEAR_D1	0x0	Force ME of HEAR D1
		(6) ME_FORCE_HEAR_D0	(6) ME_FORCE_HEAR_D0	0x0	Force ME of HEAR D0
		(5) ME_FORCE_HEAR_C1	(5) ME_FORCE_HEAR_C1	0x0	Force ME of HEAR C1
		(4) ME_FORCE_HEAR_C0	(4) ME_FORCE_HEAR_C0	0x0	Force ME of HEAR C0
		(3) ME_FORCE_HEAR_B1	(3) ME_FORCE_HEAR_B1	0x0	Force ME of HEAR B1
		(2) ME_FORCE_HEAR_B0	(2) ME_FORCE_HEAR_B0	0x0	Force ME of HEAR B0
		(1) ME_FORCE_HEAR_A1	(1) ME_FORCE_HEAR_A1	0x0	Force ME of HEAR A1
		(0) ME_FORCE_HEAR_A0	(0) ME_FORCE_HEAR_A0	0x0	Force ME of HEAR A0
P:0xF014	D_MEM_CFG_4	(3) ME_FORCE_CM3_SRAM	(3) ME_FORCE_CM3_SRAM	0x0	Force ME of CM3 SRAM
		(2) ME_FORCE_CM3_PRAM2	(2) ME_FORCE_CM3_PRAM2	0x0	Force ME of CM3 PRAM2
		(1) ME_FORCE_CM3_PRAM1	(1) ME_FORCE_CM3_PRAM1	0x0	Force ME of CM3 PRAM1
		(0) ME_FORCE_CM3_PRAM0	(0) ME_FORCE_CM3_PRAM0	0x0	Force ME of CM3 PRAM0

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF015	D_MEM_CFG_5	(3) ME_FORCE_FENG_PMEM	(3) ME_FORCE_FENG_PMEM	0x0	Force ME of Filter Engine PMEM
		(2) ME_FORCE_FENG_CMEM	(2) ME_FORCE_FENG_CMEM	0x0	Force ME of Filter Engine CMEM
		(1) ME_FORCE_FENG_SMEM1	(1) ME_FORCE_FENG_SMEM1	0x0	Force ME of Filter Engine SMEM1
		(0) ME_FORCE_FENG_SMEM0	(0) ME_FORCE_FENG_SMEM0	0x0	Force ME of Filter Engine SMEM0

A.20 EXTERNAL BREAKPOINT CONFIGURATION

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF020	D_EXT_BREAK_CFG	(8:6) COMP_MODE	(8:6) COMP_MODE	0x0	Select the comparison mode
		(5:4) ADDR_COND	(5:4) ADDR_COND	0x0	Select the address condition
		(3:2) ACCESS_COND	(3:2) ACCESS_COND	0x0	Select the access or data condition
		(1:0) BUS_SELECT	(1:0) BUS_SELECT	0x0	Select the data bus to monitor for external breakpoints
P:0xF021	D_EXT_BREAK_DATA	(31:0) EXT_BREAK_DATA	(31:0) EXT_BREAK_DATA	0x0	External break point data
		(2) CM3_FIFO_CONFLICT	(2) CM3_FIFO_CONFLICT	0x0	Configured detection of FIFO memory conflicts between the ARM Cortex-M3 core or DMA and the HEAR or CFX
		(1) CFX_CM3_CONFLICT	(1) CFX_CM3_CONFLICT	0x0	Configured detection of memory conflicts between the CFX and ARM Cortex-M3 core
		(0) CFX_HEAR_CONFLICT	(0) CFX_HEAR_CONFLICT	0x0	Configured detection of memory conflicts between the CFX and HEAR

A.21 HARDWARE CRC

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF022	D_CRC_VALUE	(15:0) CRC_VALUE	(15:0) CRC_VALUE	0xFFFF	Value of CRC-CCITT for supplied data
P:0xF023	D_CRC_ADD_1	(0) CRC_ADD_1	-	N/A	Add 1-bit data to the CRC
P:0xF024	D_CRC_ADD_8	(7:0) CRC_ADD_8	-	N/A	Add 8-bit data to the CRC
P:0xF025	D_CRC_ADD_16	(15:0) CRC_ADD_16	-	N/A	Add 16-bit data to the CRC
P:0xF026	D_CRC_ADD_24	(23:0) CRC_ADD_24	-	N/A	Add 24-bit data to the CRC
P:0xF027	D_CRC_ADD_32	(31:0) CRC_ADD_32	-	N/A	Add 32-bit data to the CRC

A.22 CHIP VERSION INFORMATION (READ-ONLY)

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF030	D_CHIP_FAMILY	-	(7:0) CHIP_FAMILY	0x6	Chip family number
P:0xF031	D_CHIP_VERSION	-	(7:0) CHIP_VERSION	0x1	Chip version number
P:0xF032	D_CHIP_REVISION	-	(15:0) CHIP_REVISION	0x100	Chip revision number
P:0xF033	D_CFX_DEBUG_VERSION	-	(15:0) CFX_DEBUG_VERSION	0x201	CFX debug port version number
P:0xF034	D_CM3_DEBUG_VERSION	-	(15:0) CM3_DEBUG_VERSION	0x200	ARM Cortex-M3 core debug version number

A.23 DEBUG PORT CONFIGURATION

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF035	D_DEBUG_CFG	-	(3) DBG_REG_WRITTEN	0x0	Data register write status
		-	(2) DBG_REG_READ	0x0	Data register read status
		-	(1) AUDIO_MODE	0x0	NOAHlink streaming audio mode configuration
		(0) ACCESS_MODE	(0) ACCESS_MODE	0x1	Restricted access mode configuration
P:0xF036	D_DEBUG_DATA	(31:0) DEBUG_DATA	(31:0) DEBUG_DATA	0x0	Debug port data register

A.24 EXTENDED CONFIGURATION

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF03E	D_PCM_EXT_CFG	(1) CM3_PCM_CLK_POL	(1) CM3_PCM_CLK_POL	0x0	ARM Cortex-M3 core PCM clock polarity
		(0) CFX_PCM_CLK_POL	(0) CFX_PCM_CLK_POL	0x0	CFX PCM clock polarity

A.25 INTERRUPT CONFIGURATION

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF040	D_INT_ENABLE	(31) CM3_3	(31) CM3_3	0x0	ARM Cortex-M3 core interrupt 3 enable/disable
		(30) CM3_2	(30) CM3_2	0x0	ARM Cortex-M3 core interrupt 2 enable/disable
		(29) CM3_1	(29) CM3_1	0x0	ARM Cortex-M3 core interrupt 1 enable/disable
		(28) CM3_0	(28) CM3_0	0x0	ARM Cortex-M3 core interrupt 0 enable/disable
		(27) C_I2C	(27) C_I2C	0x0	ARM Cortex-M3 I^2C interrupt enable/disable
		(26) C_PCM	(26) C_PCM	0x0	ARM Cortex-M3 PCM interrupt enable/disable
		(25) C_SPI	(25) C_SPI	0x0	ARM Cortex-M3 SPI interrupt enable/disable
		(24) CLK	(24) CLK	0x0	CLK interrupt enable/disable
		(23) FIFO_7	(23) FIFO_7	0x0	FIFO interrupt 7 enable/disable
		(22) FIFO_6	(22) FIFO_6	0x0	FIFO interrupt 6 enable/disable
		(21) FIFO_5	(21) FIFO_5	0x0	FIFO interrupt 5 enable/disable
		(20) FIFO_4	(20) FIFO_4	0x0	FIFO interrupt 4 enable/disable
		(19) FIFO_3	(19) FIFO_3	0x0	FIFO interrupt 3 enable/disable
		(18) FIFO_2	(18) FIFO_2	0x0	FIFO interrupt 2 enable/disable
		(17) FIFO_1	(17) FIFO_1	0x0	FIFO interrupt 1 enable/disable
		(16) FIFO_0	(16) FIFO_0	0x0	FIFO interrupt 0 enable/disable
		(15) HEAR_7_SDU	(15) HEAR_7_SDU	0x0	HEAR 7 / SDU interrupt enable/disable
		(14) HEAR_6_FENG	(14) HEAR_6_FENG	0x0	HEAR 6 / Filter Engine interrupt enable/disable
		(13) HEAR_5_UART	(13) HEAR_5_UART	0x0	HEAR 5 / UART interrupt enable/disable
		(12) HEAR_4	(12) HEAR_4	0x0	HEAR 4 interrupt enable/disable
		(11) HEAR_3	(11) HEAR_3	0x0	HEAR 3 interrupt enable/disable
		(10) HEAR_2	(10) HEAR_2	0x0	HEAR 2 interrupt enable/disable
		(9) HEAR_1	(9) HEAR_1	0x0	HEAR 1 interrupt enable/disable
		(8) HEAR_0	(8) HEAR_0	0x0	HEAR 0 interrupt enable/disable
		(7) GPIO	(7) GPIO	0x0	GPIO interrupt enable/disable
		(6) PCM	(6) PCM	0x0	PCM interrupt enable/disable
		(5) I2C	(5) I2C	0x0	I^2C interrupt enable/disable
		(4) SPI	(4) SPI	0x0	SPI interrupt enable/disable
		(3) TIMER_2	(3) TIMER_2	0x0	Timer 2 interrupt enable/disable
		(2) TIMER_1	(2) TIMER_1	0x0	Timer 1 interrupt enable/disable
		(1) WATCHDOG	(1) WATCHDOG	0x0	Watchdog timer interrupt enable/disable
		(0) MASTER	(0) MASTER	0x0	Master enable/disable for all CFX interrupts

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF041	D_INT_STATUS	(31) CM3_3	(31) CM3_3	0x0	ARM Cortex-M3 core interrupt 3
		(30) CM3_2	(30) CM3_2	0x0	ARM Cortex-M3 core interrupt 2
		(29) CM3_1	(29) CM3_1	0x0	ARM Cortex-M3 core interrupt 1
		(28) CM3_0	(28) CM3_0	0x0	ARM Cortex-M3 core interrupt 0
		(27) C_I2C	(27) C_I2C	0x0	ARM Cortex-M3 I^2C interrupt
		(26) C_PCM	(26) C_PCM	0x0	ARM Cortex-M3 PCM interrupt
		(25) C_SPI	(25) C_SPI	0x0	ARM Cortex-M3 SPI interrupt
		(24) CLK	(24) CLK	0x0	CLK interrupt
		(23) FIFO_7	(23) FIFO_7	0x0	FIFO interrupt 7
		(22) FIFO_6	(22) FIFO_6	0x0	FIFO interrupt 6
		(21) FIFO_5	(21) FIFO_5	0x0	FIFO interrupt 5
		(20) FIFO_4	(20) FIFO_4	0x0	FIFO interrupt 4
		(19) FIFO_3	(19) FIFO_3	0x0	FIFO interrupt 3
		(18) FIFO_2	(18) FIFO_2	0x0	FIFO interrupt 2
		(17) FIFO_1	(17) FIFO_1	0x0	FIFO interrupt 1
		(16) FIFO_0	(16) FIFO_0	0x0	FIFO interrupt 0
		(15) HEAR_7_SDU	(15) HEAR_7_SDU	0x0	HEAR 7 / SDU interrupt
		(14) HEAR_6_FENG	(14) HEAR_6_FENG	0x0	HEAR 6 / Filter Engine interrupt
		(13) HEAR_5_UART	(13) HEAR_5_UART	0x0	HEAR 5 / UART interrupt
		(12) HEAR_4	(12) HEAR_4	0x0	HEAR 4 interrupt
		(11) HEAR_3	(11) HEAR_3	0x0	HEAR 3 interrupt
		(10) HEAR_2	(10) HEAR_2	0x0	HEAR 2 interrupt
		(9) HEAR_1	(9) HEAR_1	0x0	HEAR 1 interrupt
		(8) HEAR_0	(8) HEAR_0	0x0	HEAR 0 interrupt
		(7) GPIO	(7) GPIO	0x0	GPIO interrupt
		(6) PCM	(6) PCM	0x0	PCM interrupt
		(5) I2C	(5) I2C	0x0	I^2C interrupt
		(4) SPI	(4) SPI	0x0	SPI interrupt
		(3) TIMER_2	(3) TIMER_2	0x0	Timer 2 interrupt
		(2) TIMER_1	(2) TIMER_1	0x0	Timer 1 interrupt
		(1) WATCHDOG	(1) WATCHDOG	0x0	Watchdog timer interrupt

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF042	D_INT_PRIORITY	(31) CM3_3	(31) CM3_3	0x0	ARM Cortex-M3 core interrupt 3 priority
		(30) CM3_2	(30) CM3_2	0x0	ARM Cortex-M3 core interrupt 2 priority
		(29) CM3_1	(29) CM3_1	0x0	ARM Cortex-M3 core interrupt 1 priority
		(28) CM3_0	(28) CM3_0	0x0	ARM Cortex-M3 core interrupt 0 priority
		(27) C_I2C	(27) C_I2C	0x0	ARM Cortex-M3 core I^2C interrupt priority
		(26) C_PCM	(26) C_PCM	0x0	ARM Cortex-M3 core PCM interrupt priority
		(25) C_SPI	(25) C_SPI	0x0	ARM Cortex-M3 core SPI interrupt priority
		(24) CLK	(24) CLK	0x0	CLK interrupt priority
		(23) FIFO_7	(23) FIFO_7	0x0	FIFO interrupt 7 priority
		(22) FIFO_6	(22) FIFO_6	0x0	FIFO interrupt 6 priority
		(21) FIFO_5	(21) FIFO_5	0x0	FIFO interrupt 5 priority
		(20) FIFO_4	(20) FIFO_4	0x0	FIFO interrupt 4 priority
		(19) FIFO_3	(19) FIFO_3	0x0	FIFO interrupt 3 priority
		(18) FIFO_2	(18) FIFO_2	0x0	FIFO interrupt 2 priority
		(17) FIFO_1	(17) FIFO_1	0x0	FIFO interrupt 1 priority
		(16) FIFO_0	(16) FIFO_0	0x0	FIFO interrupt 0 priority
		(15) HEAR_7_SDU	(15) HEAR_7_SDU	0x0	HEAR 7 / SDU interrupt priority
		(14) HEAR_6_FENG	(14) HEAR_6_FENG	0x0	HEAR 6 / Filter Engine interrupt priority
		(13) HEAR_5_UART	(13) HEAR_5_UART	0x0	HEAR 5 / UART interrupt priority
		(12) HEAR_4	(12) HEAR_4	0x0	HEAR 4 interrupt priority
		(11) HEAR_3	(11) HEAR_3	0x0	HEAR 3 interrupt priority
		(10) HEAR_2	(10) HEAR_2	0x0	HEAR 2 interrupt priority
		(9) HEAR_1	(9) HEAR_1	0x0	HEAR 1 interrupt priority
		(8) HEAR_0	(8) HEAR_0	0x0	HEAR 0 interrupt priority
		(7) GPIO	(7) GPIO	0x0	GPIO interrupt priority
		(6) PCM	(6) PCM	0x0	PCM interrupt priority
		(5) I2C	(5) I2C	0x0	I^2C interrupt priority
		(4) SPI	(4) SPI	0x0	SPI interrupt priority
		(3) TIMER_2	(3) TIMER_2	0x0	Timer 2 interrupt priority
		(2) TIMER_1	(2) TIMER_1	0x0	Timer 1 interrupt priority
		(1) WATCHDOG	(1) WATCHDOG	0x0	Watchdog timer interrupt priority

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF050	D_INT_EBL_0	(7) GPIO	-	N/A	GPIO interrupt enable
		(6) PCM	-	N/A	PCM interrupt enable
		(5) I2C	-	N/A	I^2C interrupt enable
		(4) SPI	-	N/A	SPI interrupt enable
		(3) TIMER_2	-	N/A	Timer 2 interrupt enable
		(2) TIMER_1	-	N/A	Timer 1 interrupt enable
		(1) WATCHDOG	-	N/A	Watchdog timer interrupt enable
		(0) MASTER	-	N/A	Master enable for all CFX interrupts
P:0xF051	D_INT_EBL_1	(7) HEAR_7_SDU	-	N/A	HEAR 7 / SDU interrupt enable
		(6) HEAR_6_FENG	-	N/A	HEAR 6 / Filter Engine interrupt enable
		(5) HEAR_5_UART	-	N/A	HEAR 5 / UART interrupt enable
		(4) HEAR_4	-	N/A	HEAR 4 interrupt enable
		(3) HEAR_3	-	N/A	HEAR 3 interrupt enable
		(2) HEAR_2	-	N/A	HEAR 2 interrupt enable
		(1) HEAR_1	-	N/A	HEAR 1 interrupt enable
		(0) HEAR_0	-	N/A	HEAR 0 interrupt enable
P:0xF052	D_INT_EBL_2	(7) FIFO_7	-	N/A	FIFO interrupt 7 enable
		(6) FIFO_6	-	N/A	FIFO interrupt 6 enable
		(5) FIFO_5	-	N/A	FIFO interrupt 5 enable
		(4) FIFO_4	-	N/A	FIFO interrupt 4 enable
		(3) FIFO_3	-	N/A	FIFO interrupt 3 enable
		(2) FIFO_2	-	N/A	FIFO interrupt 2 enable
		(1) FIFO_1	-	N/A	FIFO interrupt 1 enable
		(0) FIFO_0	-	N/A	FIFO interrupt 0 enable
P:0xF053	D_INT_EBL_3	(7) CM3_3	-	N/A	ARM Cortex-M3 core interrupt 3 enable
		(6) CM3_2	-	N/A	ARM Cortex-M3 core interrupt 2 enable
		(5) CM3_1	-	N/A	ARM Cortex-M3 core interrupt 1 enable
		(4) CM3_0	-	N/A	ARM Cortex-M3 core interrupt 0 enable
		(3) C_I2C	-	N/A	ARM Cortex-M3 core I^2C interrupt enable
		(2) C_PCM	-	N/A	ARM Cortex-M3 core PCM interrupt enable
		(1) C_SPI	-	N/A	ARM Cortex-M3 core SPI interrupt enable
		(0) CLK	-	N/A	CLK interrupt enable

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF054	D_INT_ACK_EBL_0	(7) GPIO	-	N/A	GPIO interrupt acknowledge and enable
		(6) PCM	-	N/A	PCM interrupt acknowledge and enable
		(5) I2C	-	N/A	I^2C interrupt acknowledge and enable
		(4) SPI	-	N/A	SPI interrupt acknowledge and enable
		(3) TIMER_2	-	N/A	Timer 2 interrupt acknowledge and enable
		(2) TIMER_1	-	N/A	Timer 1 interrupt acknowledge and enable
		(1) WATCHDOG	-	N/A	Watchdog timer interrupt acknowledge and enable
		(0) MASTER	-	N/A	Master enable for all CFX interrupts
P:0xF055	D_INT_ACK_EBL_1	(7) HEAR_7_SDU	-	N/A	HEAR 7 / SDU interrupt acknowledge and enable
		(6) HEAR_6_FENG	-	N/A	HEAR 6 / Filter Engine interrupt acknowledge and enable
		(5) HEAR_5_UART	-	N/A	HEAR 5 / UART interrupt acknowledge and enable
		(4) HEAR_4	-	N/A	HEAR 4 interrupt acknowledge and enable
		(3) HEAR_3	-	N/A	HEAR 3 interrupt acknowledge and enable
		(2) HEAR_2	-	N/A	HEAR 2 interrupt acknowledge and enable
		(1) HEAR_1	-	N/A	HEAR 1 interrupt acknowledge and enable
		(0) HEAR_0	-	N/A	HEAR 0 interrupt acknowledge and enable
P:0xF056	D_INT_ACK_EBL_2	(7) FIFO_7	-	N/A	FIFO interrupt 7 acknowledge and enable
		(6) FIFO_6	-	N/A	FIFO interrupt 6 acknowledge and enable
		(5) FIFO_5	-	N/A	FIFO interrupt 5 acknowledge and enable
		(4) FIFO_4	-	N/A	FIFO interrupt 4 acknowledge and enable
		(3) FIFO_3	-	N/A	FIFO interrupt 3 acknowledge and enable
		(2) FIFO_2	-	N/A	FIFO interrupt 2 acknowledge and enable
		(1) FIFO_1	-	N/A	FIFO interrupt 1 acknowledge and enable
		(0) FIFO_0	-	N/A	FIFO interrupt 0 acknowledge and enable

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF057	D_INT_ACK_EBL_3	(7) CM3_3	-	N/A	ARM Cortex-M3 core interrupt 3 acknowledge and enable
		(6) CM3_2	-	N/A	ARM Cortex-M3 core interrupt 2 acknowledge and enable
		(5) CM3_1	-	N/A	ARM Cortex-M3 core interrupt 1 acknowledge and enable
		(4) CM3_0	-	N/A	ARM Cortex-M3 core interrupt 0 acknowledge and enable
		(3) C_I2C	-	N/A	ARM Cortex-M3 I^2C interrupt acknowledge and enable
		(2) C_PCM	-	N/A	ARM Cortex-M3 PCM interrupt acknowledge and enable
		(1) C_SPI	-	N/A	ARM Cortex-M3 SPI interrupt acknowledge and enable
		(0) CLK	-	N/A	CLK interrupt acknowledge and enable
P:0xF058	D_INT_DIS_0	(7) GPIO	-	N/A	GPIO interrupt disable
		(6) PCM	-	N/A	PCM interrupt disable
		(5) I2C	-	N/A	I^2C interrupt disable
		(4) SPI	-	N/A	SPI interrupt disable
		(3) TIMER_2	-	N/A	Timer 2 interrupt disable
		(2) TIMER_1	-	N/A	Timer 1 interrupt disable
		(1) WATCHDOG	-	N/A	Watchdog timer interrupt disable
		(0) MASTER	-	N/A	Master disable for all CFX interrupts
P:0xF059	D_INT_DIS_1	(7) HEAR_7_SDU	-	N/A	HEAR 7 / SDU interrupt disable
		(6) HEAR_6_FENG	-	N/A	HEAR 6 / Filter Engine interrupt disable
		(5) HEAR_5_UART	-	N/A	HEAR 5 / UART interrupt disable
		(4) HEAR_4	-	N/A	HEAR 4 interrupt disable
		(3) HEAR_3	-	N/A	HEAR 3 interrupt disable
		(2) HEAR_2	-	N/A	HEAR 2 interrupt disable
		(1) HEAR_1	-	N/A	HEAR 1 interrupt disable
		(0) HEAR_0	-	N/A	HEAR 0 interrupt disable

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF05A	D_INT_DIS_2	(7) FIFO_7	-	N/A	FIFO interrupt 7 disable
		(6) FIFO_6	-	N/A	FIFO interrupt 6 disable
		(5) FIFO_5	-	N/A	FIFO interrupt 5 disable
		(4) FIFO_4	-	N/A	FIFO interrupt 4 disable
		(3) FIFO_3	-	N/A	FIFO interrupt 3 disable
		(2) FIFO_2	-	N/A	FIFO interrupt 2 disable
		(1) FIFO_1	-	N/A	FIFO interrupt 1 disable
		(0) FIFO_0	-	N/A	FIFO interrupt 0 disable
P:0xF05B	D_INT_DIS_3	(7) CM3_3	-	N/A	ARM Cortex-M3 core interrupt 3 disable
		(6) CM3_2	-	N/A	ARM Cortex-M3 core interrupt 2 disable
		(5) CM3_1	-	N/A	ARM Cortex-M3 core interrupt 1 disable
		(4) CM3_0	-	N/A	ARM Cortex-M3 core interrupt 0 disable
		(3) C_I2C	-	N/A	ARM Cortex-M3 core I^2C interrupt disable
		(2) C_PCM	-	N/A	ARM Cortex-M3 core PCM interrupt disable
		(1) C_SPI	-	N/A	ARM Cortex-M3 core SPI interrupt disable
		(0) CLK	-	N/A	CLK interrupt disable
P:0xF05C	D_INT_ACK_0	(7) GPIO	-	N/A	GPIO interrupt acknowledge
		(6) PCM	-	N/A	PCM interrupt acknowledge
		(5) I2C	-	N/A	I^2C interrupt acknowledge
		(4) SPI	-	N/A	SPI interrupt acknowledge
		(3) TIMER_2	-	N/A	Timer 2 interrupt acknowledge
		(2) TIMER_1	-	N/A	Timer 1 interrupt acknowledge
		(1) WATCHDOG	-	N/A	Watchdog timer interrupt acknowledge
		(0) MASTER	-	N/A	Master enable for all CFX interrupts
P:0xF05D	D_INT_ACK_1	(7) HEAR_7_SDU	-	N/A	HEAR 7 / SDU interrupt acknowledge
		(6) HEAR_6_FENG	-	N/A	HEAR 6 / Filter Engine interrupt acknowledge
		(5) HEAR_5_UART	-	N/A	HEAR 5 / UART interrupt acknowledge
		(4) HEAR_4	-	N/A	HEAR 4 interrupt acknowledge
		(3) HEAR_3	-	N/A	HEAR 3 interrupt acknowledge
		(2) HEAR_2	-	N/A	HEAR 2 interrupt acknowledge
		(1) HEAR_1	-	N/A	HEAR 1 interrupt acknowledge
		(0) HEAR_0	-	N/A	HEAR 0 interrupt acknowledge

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF05E	D_INT_ACK_2	(7) FIFO_7	-	N/A	FIFO interrupt 7 acknowledge
		(6) FIFO_6	-	N/A	FIFO interrupt 6 acknowledge
		(5) FIFO_5	-	N/A	FIFO interrupt 5 acknowledge
		(4) FIFO_4	-	N/A	FIFO interrupt 4 acknowledge
		(3) FIFO_3	-	N/A	FIFO interrupt 3 acknowledge
		(2) FIFO_2	-	N/A	FIFO interrupt 2 acknowledge
		(1) FIFO_1	-	N/A	FIFO interrupt 1 acknowledge
		(0) FIFO_0	-	N/A	FIFO interrupt 0 acknowledge
P:0xF05F	D_INT_ACK_3	(7) CM3_3	-	N/A	ARM Cortex-M3 core interrupt 3 acknowledge
		(6) CM3_2	-	N/A	ARM Cortex-M3 core interrupt 2 acknowledge
		(5) CM3_1	-	N/A	ARM Cortex-M3 core interrupt 1 acknowledge
		(4) CM3_0	-	N/A	ARM Cortex-M3 core interrupt 0 acknowledge
		(3) C_I2C	-	N/A	ARM Cortex-M3 core I^2C interrupt acknowledge
		(2) C_PCM	-	N/A	ARM Cortex-M3 core PCM interrupt acknowledge
		(1) C_SPI	-	N/A	ARM Cortex-M3 core SPI interrupt acknowledge
		(0) CLK	-	N/A	CLK interrupt acknowledge
P:0xF060	D_INT_PRIO_0	(7) GPIO	-	N/A	GPIO interrupt priority
		(6) PCM	-	N/A	PCM interrupt priority
		(5) I2C	-	N/A	I^2C interrupt priority
		(4) SPI	-	N/A	SPI interrupt priority
		(3) TIMER_2	-	N/A	Timer 2 interrupt priority
		(2) TIMER_1	-	N/A	Timer 1 interrupt priority
		(1) WATCHDOG	-	N/A	Watchdog timer interrupt priority
P:0xF061	D_INT_PRIO_1	(7) HEAR_7_SDU	-	N/A	HEAR 7 / SDU interrupt priority
		(6) HEAR_6_FENG	-	N/A	HEAR 6 / Filter Engine interrupt priority
		(5) HEAR_5_UART	-	N/A	HEAR 5 / UART interrupt priority
		(4) HEAR_4	-	N/A	HEAR 4 interrupt priority
		(3) HEAR_3	-	N/A	HEAR 3 interrupt priority
		(2) HEAR_2	-	N/A	HEAR 2 interrupt priority
		(1) HEAR_1	-	N/A	HEAR 1 interrupt priority
		(0) HEAR_0	-	N/A	HEAR 0 interrupt priority

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF062	D_INT_PRIO_2	(7) FIFO_7	-	N/A	FIFO interrupt 7 priority
		(6) FIFO_6	-	N/A	FIFO interrupt 6 priority
		(5) FIFO_5	-	N/A	FIFO interrupt 5 priority
		(4) FIFO_4	-	N/A	FIFO interrupt 4 priority
		(3) FIFO_3	-	N/A	FIFO interrupt 3 priority
		(2) FIFO_2	-	N/A	FIFO interrupt 2 priority
		(1) FIFO_1	-	N/A	FIFO interrupt 1 priority
		(0) FIFO_0	-	N/A	FIFO interrupt 0 priority
P:0xF063	D_INT_PRIO_3	(7) CM3_3	-	N/A	ARM Cortex-M3 interrupt 3 priority
		(6) CM3_2	-	N/A	ARM Cortex-M3 core interrupt 2 priority
		(5) CM3_1	-	N/A	ARM Cortex-M3 core interrupt 1 priority
		(4) CM3_0	-	N/A	ARM Cortex-M3 core interrupt 0 priority
		(3) C_I2C	-	N/A	ARM Cortex-M3 I^2C interrupt priority
		(2) C_PCM	-	N/A	ARM Cortex-M3 PCM interrupt priority
		(1) C_SPI	-	N/A	ARM Cortex-M3 SPI interrupt priority
		(0) CLK	-	N/A	CLK interrupt priority

A.26 CLOCK CONFIGURATION

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF080	D_CLK_CFG_0	(24) EXTCLK_CLK_SEL	(24) EXTCLK_CLK_SEL	0x0	Controls whether SLOWCLK or SDMCLK can be output on EXT_CLK
		(23:20) CM3CLK_PRESCALE	(23:20) CM3CLK_PRESCALE	0x0	Prescale value for the CM3CLK clock (1 to 16 in steps of 1)
		(19) EXTCLK_PULL_EN	(19) EXTCLK_PULL_EN	0x1	Controls the pad pull-up
		(18) SYSCLK_SEL	(18) SYSCLK_SEL	0x0	Controls the functionality of SYSCLK
		(17:16) EXTCLK_SEL	(17:16) EXTCLK_SEL	0x0	Controls the functionality of the EXT_CLK pin
		(15:12) EXTCLK_PRESCALE	(15:12) EXTCLK_PRESCALE	0x0	Prescale value for the input clock from pad EXT_CLK (1 to 16 in steps of 1)
		(11) EXTCLK_DRIVE	(11) EXTCLK_DRIVE	0x0	EXT_CLK drive control
		(9:0) HEARCLK_PRESCALE	(9:0) HEARCLK_PRESCALE	0x0	Prescale value for the HEAR clock (1 to 128.875 in steps of 0.125)
P:0xF081	D_CLK_CFG_1	(22) SLOWCLK_SRC	(22) SLOWCLK_SRC	0x0	Indicates the user clock source (SYSCLK or SDMCLK)
		(21) UCLK_SRC	(21) UCLK_SRC	0x0	Indicates the user clock source (SYSCLK or SDMCLK)
		(20) DBL_ACCESS_MODE	(20) DBL_ACCESS_MODE	0x0	Creates a virtual double access for the shared memories
		(19:8) UCLK_PRESCALE	(19:8) UCLK_PRESCALE	0x0	Prescale value for the user clock (1 to 4096 in steps of 1)
		(5:0) SLOWCLK_PRESCALE	(5:0) SLOWCLK_PRESCALE	0x0	Prescale value for the slow clock (1 to 64 in steps of 1)
P:0xF084	D_CLK_CFG_CPA	(5:0) CP_VDDA_CLK_PRESCALE	(5:0) CP_VDDA_CLK_PRESCALE	0x0	Prescale the clock for the VDDA charge pump (1 to 64 in steps of 1)
P:0xF085	D_CLK_CFG_CPD	(5:0) CP_VDBL_CLK_PRESCALE	(5:0) CP_VDBL_CLK_PRESCALE	0xF	Prescale the clock for the VDBL charge pump (1 to 64 in steps of 1)
P:0xF086	D_CLK_CFG_CPC	(5:0) CP_VDDC_CLK_PRESCALE	(5:0) CP_VDDC_CLK_PRESCALE	0x0	Prescale the clock for the VDDC charge pump (1 to 64 in steps of 1)
P:0xF087	D_CLK_CFG_CPM	(5:0) CP_VDDM_CLK_PRESCALE	(5:0) CP_VDDM_CLK_PRESCALE	0x0	Prescale the clock for the VDDM charge pump (1 to 64 in steps of 1)
P:0xF088	D_SLOWCLK_THROTTLING	(5:0) PRESCALE	(5:0) PRESCALE	0x0	Future clock throttling value for SLOWCLK_PRESCALE
P:0xF089	D_CLK_THROTTLING_CTRL	(1:0) APPLY	-	N/A	Clock throttling apply command
		-	(1:0) STATUS	0x0	Clock throttling apply status
P:0xF08A	D_CLK_MULT_THROTTLING_CFG	(2) UCLK_EBL	(2) UCLK_EBL	0x0	User clock VCO multiplier based clock throttling enable
		(1) SDMCLK_EBL	(1) SDMCLK_EBL	0x0	SDM clock VCO multiplier based clock throttling enable
		(0) SLOWCLK_EBL	(0) SLOWCLK_EBL	0x0	Slow clock VCO multiplier based clock throttling enable

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF08B	D_CLK_SYNC	-	(5) CLK_RESYNC_STATUS	0x0	Clock resynchronization detection status (cleared when read)
		-	(4) CLK_SYNC_STATUS	0x0	Clock synchronization status
		(2) CLK_RESYNC_INT_ENABLE	(2) CLK_RESYNC_INT_ENABLE	0x0	Clock resynchronization interrupt enable
		(1) CLK_SYNC_TOLERANCE	(1) CLK_SYNC_TOLERANCE	0x0	Clock synchronizer tolerance (ignored for SDMCLK_PRESCALE<5)
		(0) CLK_SYNC_ENABLE	(0) CLK_SYNC_ENABLE	0x0	Clock synchronizer enable (ignored for SDMCLK_PRESCALE<3)
P:0xF08C	D_CLK_THROTTLING_CFG	(0) CLK_THROTTLING_ON_OD_ZERO	(0) CLK_THROTTLING_ON_OD_ZERO	0x0	Enable VCO frequency-based clock throttling only on OD zero output
P:0xF08D	D_MULTI_SYNC_CFG	(1:0) SLAVE_CFG	(1:0) SLAVE_CFG	0x0	Multi-chip slave configuration

A.27 IOC INPUT CONFIGURATION

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF0C0	D_IOC_INPUT_ADC_CFG	(29:24) ADC3_STORE	(29:24) ADC3_STORE	0x3F	Select where to store the ADC3 input
		(21:16) ADC2_STORE	(21:16) ADC2_STORE	0x3F	Select where to store the ADC2 input
		(13:8) ADC1_STORE	(13:8) ADC1_STORE	0x3F	Select where to store the ADC1 input
		(5:0) ADC0_STORE	(5:0) ADC0_STORE	0x3F	Select where to store the ADC0 input
P:0xF0C1	D_IOC_INPUT_PCM_CFG	(13:8) PCM1_STORE	(13:8) PCM1_STORE	0x3F	Select where to store PCM 1 input
		(5:0) PCM0_STORE	(5:0) PCM0_STORE	0x3F	Select where to store PCM 0 input

A.28 IOC FIFO CONFIGURATION

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF0C2	D_IOC_FIFO_ACCESS_CFG	(31) FIFO_B1_7_DBL_ACC	(31) FIFO_B1_7_DBL_ACC	0x0	Defines FIFO B1 7 access type
		(30) FIFO_B1_6_DBL_ACC	(30) FIFO_B1_6_DBL_ACC	0x0	Defines FIFO B1 6 access type
		(29) FIFO_B1_5_DBL_ACC	(29) FIFO_B1_5_DBL_ACC	0x0	Defines FIFO B1 5 access type
		(28) FIFO_B1_4_DBL_ACC	(28) FIFO_B1_4_DBL_ACC	0x0	Defines FIFO B1 4 access type
		(27) FIFO_B1_3_DBL_ACC	(27) FIFO_B1_3_DBL_ACC	0x0	Defines FIFO B1 3 access type
		(26) FIFO_B1_2_DBL_ACC	(26) FIFO_B1_2_DBL_ACC	0x0	Defines FIFO B1 2 access type
		(25) FIFO_B1_1_DBL_ACC	(25) FIFO_B1_1_DBL_ACC	0x0	Defines FIFO B1 1 access type
		(24) FIFO_B1_0_DBL_ACC	(24) FIFO_B1_0_DBL_ACC	0x0	Defines FIFO B1 0 access type
		(23) FIFO_B0_7_DBL_ACC	(23) FIFO_B0_7_DBL_ACC	0x0	Defines FIFO B0 7 access type
		(22) FIFO_B0_6_DBL_ACC	(22) FIFO_B0_6_DBL_ACC	0x0	Defines FIFO B0 6 access type
		(21) FIFO_B0_5_DBL_ACC	(21) FIFO_B0_5_DBL_ACC	0x0	Defines FIFO B0 5 access type
		(20) FIFO_B0_4_DBL_ACC	(20) FIFO_B0_4_DBL_ACC	0x0	Defines FIFO B0 4 access type
		(19) FIFO_B0_3_DBL_ACC	(19) FIFO_B0_3_DBL_ACC	0x0	Defines FIFO B0 3 access type
		(18) FIFO_B0_2_DBL_ACC	(18) FIFO_B0_2_DBL_ACC	0x0	Defines FIFO B0 2 access type
		(17) FIFO_B0_1_DBL_ACC	(17) FIFO_B0_1_DBL_ACC	0x0	Defines FIFO B0 1 access type
		(16) FIFO_B0_0_DBL_ACC	(16) FIFO_B0_0_DBL_ACC	0x0	Defines FIFO B0 0 access type
		(15) FIFO_A1_7_DBL_ACC	(15) FIFO_A1_7_DBL_ACC	0x0	Defines FIFO A1 7 access type
		(14) FIFO_A1_6_DBL_ACC	(14) FIFO_A1_6_DBL_ACC	0x0	Defines FIFO A1 6 access type
		(13) FIFO_A1_5_DBL_ACC	(13) FIFO_A1_5_DBL_ACC	0x0	Defines FIFO A1 5 access type
		(12) FIFO_A1_4_DBL_ACC	(12) FIFO_A1_4_DBL_ACC	0x0	Defines FIFO A1 4 access type
		(11) FIFO_A1_3_DBL_ACC	(11) FIFO_A1_3_DBL_ACC	0x0	Defines FIFO A1 3 access type
		(10) FIFO_A1_2_DBL_ACC	(10) FIFO_A1_2_DBL_ACC	0x0	Defines FIFO A1 2 access type
		(9) FIFO_A1_1_DBL_ACC	(9) FIFO_A1_1_DBL_ACC	0x0	Defines FIFO A1 1 access type
		(8) FIFO_A1_0_DBL_ACC	(8) FIFO_A1_0_DBL_ACC	0x0	Defines FIFO A1 0 access type
		(7) FIFO_A0_7_DBL_ACC	(7) FIFO_A0_7_DBL_ACC	0x0	Defines FIFO A0 7 access type
		(6) FIFO_A0_6_DBL_ACC	(6) FIFO_A0_6_DBL_ACC	0x0	Defines FIFO A0 6 access type
		(5) FIFO_A0_5_DBL_ACC	(5) FIFO_A0_5_DBL_ACC	0x0	Defines FIFO A0 5 access type
		(4) FIFO_A0_4_DBL_ACC	(4) FIFO_A0_4_DBL_ACC	0x0	Defines FIFO A0 4 access type
		(3) FIFO_A0_3_DBL_ACC	(3) FIFO_A0_3_DBL_ACC	0x0	Defines FIFO A0 3 access type
		(2) FIFO_A0_2_DBL_ACC	(2) FIFO_A0_2_DBL_ACC	0x0	Defines FIFO A0 2 access type
		(1) FIFO_A0_1_DBL_ACC	(1) FIFO_A0_1_DBL_ACC	0x0	Defines FIFO A0 1 access type
		(0) FIFO_A0_0_DBL_ACC	(0) FIFO_A0_0_DBL_ACC	0x0	Defines FIFO A0 0 access type

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF0C3	D_IOC_FIFO_SHIFT_CFG	(5) DAC1_SHIFT	(5) DAC1_SHIFT	0x0	Configure the data alignment for DAC1
		(4) DAC0_SHIFT	(4) DAC0_SHIFT	0x0	Configure the data alignment for DAC0
		(3) ADC3_SHIFT	(3) ADC3_SHIFT	0x0	Configure the data alignment for ADC3
		(2) ADC2_SHIFT	(2) ADC2_SHIFT	0x0	Configure the data alignment for ADC2
		(1) ADC1_SHIFT	(1) ADC1_SHIFT	0x0	Configure the data alignment for ADC1
		(0) ADC0_SHIFT	(0) ADC0_SHIFT	0x0	Configure the data alignment for ADC0

A.29 IOC OUTPUT CONFIGURATION

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF0C4	D_IOC_OUTPUT_CFG	(30) PCM_SYNC	(30) PCM_SYNC	0x0	Indicate that the PCM and ADC/DAC data share the same rate
		(29:24) PCM1_LOAD	(29:24) PCM1_LOAD	0x3F	Defines the origin of the audio sample for PCM channel 1
		(21:16) PCM0_LOAD	(21:16) PCM0_LOAD	0x3F	Defines the origin of the audio sample for PCM channel 0
		(13:8) DAC1_LOAD	(13:8) DAC1_LOAD	0x3F	Defines the origin of the audio sample for the DAC1 channel
		(5:0) DAC0_LOAD	(5:0) DAC0_LOAD	0x3F	Defines the origin of the audio sample for the DAC0 channel

A.30 A/D AND D/A CONVERTER ACCESS

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF0D0	D_ADC0_DATA	-	(31:0) ADC0_DATA	0x0	Latest audio sample from ADC0 (m1p23 format, sign extended to 32 bits)
P:0xF0D1	D_ADC1_DATA	-	(31:0) ADC1_DATA	0x0	Latest audio sample from ADC1 (m1p23 format, sign extended to 32 bits)
P:0xF0D2	D_ADC2_DATA	-	(31:0) ADC2_DATA	0x0	Latest audio sample from ADC2 (m1p23 format, sign extended to 32 bits)
P:0xF0D3	D_ADC3_DATA	-	(31:0) ADC3_DATA	0x0	Latest audio sample from ADC3 (m1p23 format, sign extended to 32 bits)
P:0xF0D4	D_DAC0_DATA	(23:0) DAC0_DATA	-	N/A	Input sample for output channel 0 (m1p23 format)
P:0xF0D5	D_DAC1_DATA	(23:0) DAC1_DATA	-	N/A	Input sample for output channel 1 (m1p23 format)

A.31 OSCILLATOR CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF100	A_OSC_CTRL_0	(15:0) FREQ_CTRL	(15:0) FREQ_CTRL	0x400	Oscillator frequency programming control. If 0x000 is written to bits [15:4], this field will keep its previous value.
P:0xF101	A_OSC_CTRL_1	(5:4) MULT	(5:4) MULT	0x0	Oscillator clock multiplier control
		(0) ENABLE	(0) ENABLE	0x1	Oscillator enable control
P:0xF102	A_OSC_CTRL_2	(21:16) POS_TEMP_CTRL	(21:16) POS_TEMP_CTRL	0x20	Oscillator PTAT compensation control
		(13:8) NEG_TEMP_CTRL	(13:8) NEG_TEMP_CTRL	0x20	Oscillator CTAT compensation control
		(0) TEMP_CTRL_EBL	(0) TEMP_CTRL_EBL	0x0	Oscillator external temperature compensation control enable
P:0xF103	A_OSC_CTRL_3	-	(13:8) POS_TEMP_OUT	0x0	Oscillator PTAT output value
		-	(5:0) NEG_TEMP_OUT	0x0	Oscillator CTAT output value
P:0xF104	A_OSC_CTRL_THROTTLING	(15:0) FREQ_CTRL_THROTTLING	(15:0) FREQ_CTRL_THROTTLING	0x400	Future clock throttling value for FREQ_CTRL. If 0x000 is written to bits [15:4], FREQ_CTRL_THROTTLING will keep its previous value.
P:0xF105	A_PLL_CTRL_0	(2) EXT_CAL_EN	(2) EXT_CAL_EN	0x0	External calibration mode
		(1) FREE_RUN	(1) FREE_RUN	0x1	PLL free running mode
		(0) PLL_EN	(0) PLL_EN	0x0	PLL enable control
P:0xF106	A_PLL_CTRL_1	(20:12) REF_DIVIDER	(20:12) REF_DIVIDER	0x0	Input reference clock divider control - R
		(10:0) VCO_DIVIDER	(10:0) VCO_DIVIDER	0x0	VCO clock divider control - M
P:0xF107	A_PLL_CTRL_2	(4:0) POST_DIVIDER	(4:0) POST_DIVIDER	0x0	Output clock divider - P
P:0xF108	A_PLL_CTRL_3	(5:4) CAL_NBITS_CFG	(5:4) CAL_NBITS_CFG	0x2	Defines the number of bits to calibrate
		(3) CAL_MSB_SKIP	(3) CAL_MSB_SKIP	0x0	Sets the MSB of the VCO calibration at low level and skips this bit's calibration
		(2:0) CAL_CFG	(2:0) CAL_CFG	0x5	Defines the number of reference clock cycles used on PLL digital calibration
P:0xF109	A_PLL_CTRL_4	(20:16) VCO_GAIN	(20:16) VCO_GAIN	0x9	VCO gain control
		(12:8) LF_CAP_CFG	(12:8) LF_CAP_CFG	0x1A	Configuration of the loop filter capacitor
		(4:0) LF_RES_CFG	(4:0) LF_RES_CFG	0x1	Configuration of the loop filter resistor
P:0xF10A	A_PLL_CTRL_5	(3:0) V2I_TEMP_CFG	(3:0) V2I_TEMP_CFG	0x8	Temperature compensation trimming for the PLL transimpedance amplifier term
P:0xF10B	A_PLL_CTRL_6	-	(15:0) VCO_CAL_OUT	0x8000	VCO frequency calibration code output

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF10C	A_PLL_CTRL_7	-	(6) PLL_LOCK_INT_STATUS	0x0	PLL lock interrupt status (cleared when read)
		(5:4) PLL_LOCK_INT_SEL	(5:4) PLL_LOCK_INT_SEL	0x0	PLL lock interrupt configuration
		-	(3) UNLOCK_DET_FLAG	0x1	Sticky flag that detects whenever the PLL had unlocked
		(3) UNLOCK_DET_FLAG_CLEAR	-	N/A	Reset the sticky unlock detection flag
		-	(2) LOCK_DET_FLAG	0x0	Sticky flag that detects whenever the PLL has locked
		(2) LOCK_DET_FLAG_CLEAR	-	N/A	Reset the sticky lock detection flag
		-	(1) CAL_CNT_ERROR	0x0	Indicate an error on the digital calibration counter: saturation or low frequency value
		-	(0) PLL_LOCK	0x0	PLL signal information of correct functionality
P:0xF10F	A_PLL_REF_CLK_DET_CTRL	-	(7) CLK_DET_INT_STATUS	0x0	Clock detector interrupt status (cleared when read)
		(6:5) CLK_DET_INT_SEL	(6:5) CLK_DET_INT_SEL	0x0	Clock detector interrupt configuration
		(3:2) CLK_DET_DIV	(3:2) CLK_DET_DIV	0x0	Clock detector configuration - Not used when running on standby clock
		-	(1) CLK_DET_STATUS	0x0	Clock detector status
		(0) CLK_DET_ENABLE	(0) CLK_DET_ENABLE	0x0	Clock detector enable/disable
P:0xF118	A_OSC_CTRL_THROTTLING_2	(15:0) FREQ_CTRL_THROTTLING	(15:0) FREQ_CTRL_THROTTLING	0x400	Future clock throttling value for FREQ_CTRL. If 0x000 is written to bits [15:4], FREQ_CTRL_THROTTLING will keep its previous value.
P:0xF119	A_OSC_CTRL_THROTTLING_3	(15:0) FREQ_CTRL_THROTTLING	(15:0) FREQ_CTRL_THROTTLING	0x400	Future clock throttling value for FREQ_CTRL. If 0x000 is written to bits [15:4], FREQ_CTRL_THROTTLING will keep its previous value.

A.32 ANALOG

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF110	A_VREG_CTRL	(1) ENABLE	(1) ENABLE	0x1	VREG enable control
		(0) NRESET	(0) NRESET	0x0	VREG regulator reset control
P:0xF111	A_BG_CTRL	(12:8) BG_TEMP_COEF_TRIM	(12:8) BG_TEMP_COEF_TRIM	0x10	Temperature coefficient trimming control
		(4:0) BG_LEVEL	(4:0) BG_LEVEL	0x10	Reference voltage trimming control
P:0xF112	A_ANALOG_PWR_CTRL	(0) ANALOG_DISABLE	(0) ANALOG_DISABLE	0x0	Analog power down control
P:0xF113	A_VMIC_SW_CTRL	(1:0) VMIC_CONTROL	(1:0) VMIC_CONTROL	0x0	Selection of the state of VMIC

A.33 CHARGE PUMP

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF120	A_CP_VDDA_CTRL	(17) NRESET	(17) NRESET	0X0	Reset the analog charge pump analog
		(16) EBL	(16) EBL	0X0	Enable/disable the analog charge pump
		(5:0) TRIM	(5:0) TRIM	0X32	CP output voltage trimming configuration at a step of 10mV
P:0xF121	A_CP_VDBL_CTRL	(17) NRESET	(17) NRESET	0X0	Resets interface charge pump
		(16) EBL	(16) EBL	0X0	Enables interface charge pump
		(13:12) PTRIM	(13:12) PTRIM	0X0	Output power trimming
		(10:8) ITRIM	(10:8) ITRIM	0X7	(0b000=12uA, 0b111=36uA in 3uA steps)
		(5:0) VTRIM	(5:0) VTRIM	0X32	CP output voltage trimming configuration at a step of 10mV
P:0xF122	A_CP_VDDC_TRIM	(15:8) CP_TRIM	(15:8) CP_TRIM	0X78	Charge pump output voltage trimming voltage in 2.5 mV steps
		(7:0) LDO_TRIM	(7:0) LDO_TRIM	0X78	Linear regulator trimming voltage in 2.5 mV steps
P:0xF123	A_CP_VDDC_CTRL	(6:4) CP_MODE	(6:4) CP_MODE	0X0	Charge pump mode
		(1) VLOW_EN	(1) VLOW_EN	0X0	Enable VLOW detection as reset source (at 90% of LDO voltage)
		(0) ENABLE	(0) ENABLE	0X0	Charge pump enable control
P:0xF124	A_CP_VDDM_TRIM	(15:8) CP_TRIM	(15:8) CP_TRIM	0X78	Charge pump output voltage trimming voltage in 2.5 mV steps
		(7:0) LDO_TRIM	(7:0) LDO_TRIM	0X78	Linear regulator trimming voltage in 2.5 mV steps
P:0xF125	A_CP_VDDM_CTRL	(6:4) CP_MODE	(6:4) CP_MODE	0X0	Charge pump mode
		(1) VLOW_EN	(1) VLOW_EN	0X0	Enable Vlow detection as reset source (at 90% of LDO voltage)
		(0) ENABLE	(0) ENABLE	0X0	Charge pump enable control

A.34 A/D Converter Control

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF140	A_ADC_CTRL_0	(13) LINEOUT_ENABLE	(13) LINEOUT_ENABLE	0x0	Line output control
		(12) LPF_ENABLE	(12) LPF_ENABLE	0x1	Enables low-pass filter in preamplifier
		(9:8) ADC_EXP	(9:8) ADC_EXP	0x0	Gain exponent for ADC
		(7:4) PR_GAIN	(7:4) PR_GAIN	0x0	Select the gain of the preamplifier
		(3:1) IN_SEL	(3:1) IN_SEL	0x2	Select the input applied to ADC or preamplifier
		(0) ADC_ENABLE	(0) ADC_ENABLE	0x0	Enables ADC
P:0xF141	A_ADC_CTRL_1	(13) LINEOUT_ENABLE	(13) LINEOUT_ENABLE	0x0	Line output control
		(12) LPF_ENABLE	(12) LPF_ENABLE	0x1	Enables low-pass filter in preamplifier
		(9:8) ADC_EXP	(9:8) ADC_EXP	0x0	Gain exponent for ADC
		(7:4) PR_GAIN	(7:4) PR_GAIN	0x0	Select the gain of the preamplifier
		(3:1) IN_SEL	(3:1) IN_SEL	0x2	Select the input applied to ADC or preamplifier
		(0) ADC_ENABLE	(0) ADC_ENABLE	0x0	Enables ADC
P:0xF142	A_ADC_CTRL_2	(13) LINEOUT_ENABLE	(13) LINEOUT_ENABLE	0x0	Line output control
		(12) LPF_ENABLE	(12) LPF_ENABLE	0x1	Enables low-pass filter in preamplifier
		(9:8) ADC_EXP	(9:8) ADC_EXP	0x0	Gain exponent for ADC
		(7:4) PR_GAIN	(7:4) PR_GAIN	0x0	Select the gain of the preamplifier
		(3:1) IN_SEL	(3:1) IN_SEL	0x2	Select the input applied to ADC or preamplifier
		(0) ADC_ENABLE	(0) ADC_ENABLE	0x0	Enables ADC
P:0xF143	A_ADC_CTRL_3	(13) LINEOUT_ENABLE	(13) LINEOUT_ENABLE	0x0	Line output control
		(12) LPF_ENABLE	(12) LPF_ENABLE	0x1	Enables low-pass filter in preamplifier
		(9:8) ADC_EXP	(9:8) ADC_EXP	0x0	Gain exponent for ADC
		(7:4) PR_GAIN	(7:4) PR_GAIN	0x0	Select the gain of the preamplifier
		(3:1) IN_SEL	(3:1) IN_SEL	0x2	Select the input applied to ADC or preamplifier
		(0) ADC_ENABLE	(0) ADC_ENABLE	0x0	Enables ADC
P:0xF144	A_ADC_CFG_0	(16) DC_BIAS_HIZ	(16) DC_BIAS_HIZ	0x0	Disconnect the 500 kOhm DC bias resistor
		(14:12) P_CURRENT	(14:12) P_CURRENT	0x3	Trimming of the preamplifier current
		(10:8) ADC_CURRENT	(10:8) ADC_CURRENT	0x3	Trimming the current of the ADC
		(7) OFFSET_SIGN	(7) OFFSET_SIGN	0x0	Set sign of the offset in ADC
		(6:0) OFFSET_MAGN	(6:0) OFFSET_MAGN	0x18	Set the offset magnitude in ADC

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF145	A_ADC_CFG_1	(16) DC_BIAS_HIZ	(16) DC_BIAS_HIZ	0x0	Disconnect the 500 kOhm DC bias resistor
		(14:12) P_CURRENT	(14:12) P_CURRENT	0x3	Trimming of the preamplifier current
		(10:8) ADC_CURRENT	(10:8) ADC_CURRENT	0x3	Trimming the current of the ADC
		(7) OFFSET_SIGN	(7) OFFSET_SIGN	0x0	Set sign of the offset in ADC
		(6:0) OFFSET_MAGN	(6:0) OFFSET_MAGN	0x18	Set the offset magnitude in ADC
P:0xF146	A_ADC_CFG_2	(16) DC_BIAS_HIZ	(16) DC_BIAS_HIZ	0x0	Disconnect the 500 kOhm DC bias resistor
		(14:12) P_CURRENT	(14:12) P_CURRENT	0x3	Trimming of the preamplifier current
		(10:8) ADC_CURRENT	(10:8) ADC_CURRENT	0x3	Trimming the current of the ADC
		(7) OFFSET_SIGN	(7) OFFSET_SIGN	0x0	Set sign of the offset in ADC
		(6:0) OFFSET_MAGN	(6:0) OFFSET_MAGN	0x18	Set the offset magnitude in ADC
P:0xF147	A_ADC_CFG_3	(16) DC_BIAS_HIZ	(16) DC_BIAS_HIZ	0x0	Disconnect the 500 kOhm DC bias resistor
		(14:12) P_CURRENT	(14:12) P_CURRENT	0x3	Trimming of the preamplifier current
		(10:8) ADC_CURRENT	(10:8) ADC_CURRENT	0x3	Trimming the current of the ADC
		(7) OFFSET_SIGN	(7) OFFSET_SIGN	0x0	Set sign of the offset in ADC
		(6:0) OFFSET_MAGN	(6:0) OFFSET_MAGN	0x18	Set the offset magnitude in ADC
P:0xF148	A_ADC_DEC_CTRL_0	(31) MUTE	(31) MUTE	0x0	Mute output and enable data ready signal
		(30:28) BAND_SELECT_ADC	(30:28) BAND_SELECT_ADC	0x0	Indicates which band is filtered in the decimation filter
		(26:24) DELAY_INTEGER	(26:24) DELAY_INTEGER	0x0	Integer sample delay
		(21:16) DELAY_FRACTIONAL	(21:16) DELAY_FRACTIONAL	0x0	Fractional sample delay
		(15) DECIMATOR_EBL	(15) DECIMATOR_EBL	0x0	Enables ADC decimation
		(14:12) DCRM_CUT_OFF	(14:12) DCRM_CUT_OFF	0x2	Control the DC removal filter cut-off frequency of channel
		(11:0) ADC_GF	(11:0) ADC_GF	0x665	Gain factor for ADC
P:0xF149	A_ADC_DEC_CTRL_1	(31) MUTE	(31) MUTE	0x0	Mute output and enable data ready signal
		(30:28) BAND_SELECT_ADC	(30:28) BAND_SELECT_ADC	0x0	Indicates which band is filtered in the decimation filter
		(26:24) DELAY_INTEGER	(26:24) DELAY_INTEGER	0x0	Integer sample delay
		(21:16) DELAY_FRACTIONAL	(21:16) DELAY_FRACTIONAL	0x0	Fractional sample delay
		(15) DECIMATOR_EBL	(15) DECIMATOR_EBL	0x0	Enables ADC decimation
		(14:12) DCRM_CUT_OFF	(14:12) DCRM_CUT_OFF	0x2	Control the DC removal filter cut-off frequency of channel
		(11:0) ADC_GF	(11:0) ADC_GF	0x665	Gain factor for ADC

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF14A	A_ADC_DEC_CTRL_2	(31) MUTE	(31) MUTE	0x0	Mute output and enable data ready signal
		(30:28) BAND_SELECT_ADC	(30:28) BAND_SELECT_ADC	0x0	Indicates which band is filtered in the decimation filter
		(26:24) DELAY_INTEGER	(26:24) DELAY_INTEGER	0x0	Integer sample delay
		(21:16) DELAY_FRACTIONAL	(21:16) DELAY_FRACTIONAL	0x0	Fractional sample delay
		(15) DECIMATOR_EBL	(15) DECIMATOR_EBL	0x0	Enables ADC decimation
		(14:12) DCRM_CUT_OFF	(14:12) DCRM_CUT_OFF	0x2	Control the DC removal filter cut-off frequency of channel
		(11:0) ADC_GF	(11:0) ADC_GF	0x665	Gain factor for ADC
P:0xF14B	A_ADC_DEC_CTRL_3	(31) MUTE	(31) MUTE	0x0	Mute output and enable data ready signal
		(30:28) BAND_SELECT_ADC	(30:28) BAND_SELECT_ADC	0x0	Indicates which band is filtered in the decimation filter
		(26:24) DELAY_INTEGER	(26:24) DELAY_INTEGER	0x0	Integer sample delay
		(21:16) DELAY_FRACTIONAL	(21:16) DELAY_FRACTIONAL	0x0	Fractional sample delay
		(15) DECIMATOR_EBL	(15) DECIMATOR_EBL	0x0	Enables ADC decimation
		(14:12) DCRM_CUT_OFF	(14:12) DCRM_CUT_OFF	0x2	Control the DC removal filter cut-off frequency of channel
		(11:0) ADC_GF	(11:0) ADC_GF	0x665	Gain factor for ADC
P:0xF14C	A_IDRX_CFG_0	-	(31) CHANNEL_SEL	0x1	Indicates which ADC channel is currently used by the IDRX
		(30) ADC_OFF_EN	(30) ADC_OFF_EN	0x0	Disables the unused AD to save power
		(29) LOW_GAIN_EN	(29) LOW_GAIN_EN	0x0	Prepares the unused channel with lower gain
		(28) ENABLE	(28) ENABLE	0x0	Enables DRX on channels 0 and 1 or 2 and 3
		(27:26) MAX_PG	(27:26) MAX_PG	0x0	max_pg indicates range used by the DRX
		(25:24) MIN_PG	(25:24) MIN_PG	0x0	min_pg indicates range used by the DRX
		(23:20) HIGH_LIMIT	(23:20) HIGH_LIMIT	0x7	Gain switching limit for up gain changed
		(16:12) LOW_LIMIT	(16:12) LOW_LIMIT	0xE	Gain switching limit for down gain changed
		(11:6) TIMER2	(11:6) TIMER2	0x8	Stabilization time of preamplifier (delay before the modified channel is ready)
		(5:0) TIMER1	(5:0) TIMER1	0x3	Delay time before PG switches (delay before PG changes)

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF14D	A_IDRX_CFG_1	-	(31) CHANNEL_SEL	0x1	Indicates which ADC channel is currently used by the IDRX
		(30) ADC_OFF_EN	(30) ADC_OFF_EN	0x0	Disables the unused AD to save power
		(29) LOW_GAIN_EN	(29) LOW_GAIN_EN	0x0	Prepares the unused channel with lower gain
		(28) ENABLE	(28) ENABLE	0x0	Enables DRX on channels 0 and 1 or 2 and 3
		(27:26) MAX_PG	(27:26) MAX_PG	0x0	max_pg indicates range used by the DRX
		(25:24) MIN_PG	(25:24) MIN_PG	0x0	min_pg indicates range used by the DRX
		(23:20) HIGH_LIMIT	(23:20) HIGH_LIMIT	0x7	Gain switching limit for up gain changed
		(16:12) LOW_LIMIT	(16:12) LOW_LIMIT	0xE	Gain switching limit for down gain changed
		(11:6) TIMER2	(11:6) TIMER2	0x8	Stabilization time of preamplifier (delay before the modified channel is ready)
		(5:0) TIMER1	(5:0) TIMER1	0x3	Delay time before PG switches (delay before PG changes)
P:0xF14E	A_ADC_SAMPLE_FREQ_CFG	(9) COMPBUF_EBL	(9) COMPBUF_EBL	0x0	ADC comparator buffer enable
		(8) HIGH_FREQ	(8) HIGH_FREQ	0x0	Input stage high frequency mode (use for slow clock >1.92 MHz)
		(5:0) SAMPLE_FREQ	(5:0) SAMPLE_FREQ	0x2	Sampling frequency setting

A.35 OUTPUT STAGE CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF150	A_OUTPUT_CTRL	(7) OD1_NEG_CLK_EDGE	(7) OD1_NEG_CLK_EDGE	0x0	Configure the active clock edge for RCVR1-
		(6) OD1_POS_CLK_EDGE	(6) OD1_POS_CLK_EDGE	0x0	Configure the active clock edge for RCVR1+
		(5) OD1_HIGHPOWER	(5) OD1_HIGHPOWER	0x0	Enable high power mode for RCVR+/- for channel 1
		(4) OD1_ENABLE	(4) OD1_ENABLE	0x0	Enable RCVR1
		(3) OD0_NEG_CLK_EDGE	(3) OD0_NEG_CLK_EDGE	0x0	Configure the active clock edge for RCVR0-
		(2) OD0_POS_CLK_EDGE	(2) OD0_POS_CLK_EDGE	0x0	Configure the active clock edge for RCVR0+
		(1) OD0_HIGHPOWER	(1) OD0_HIGHPOWER	0x0	Enable high power mode for RCVR+/- for channel 0
		(0) OD0_ENABLE	(0) OD0_ENABLE	0x0	Enable RCVR0
P:0xF151	A_O_SDMCLK	(5:0) PRESCALE	(5:0) PRESCALE	0x0	Prescaler for the SDMCLK to achieve the desired oversampling ratio. SDMCLK=(SLOWCLK_PRESCALE+1)*(SFCR+8)*8 /(SDMCLK_PRESCALE+1); SDMCLK=SYS_CLK/ (SDMCLK_PRESCALE+1)
P:0xF152	A_O_DAC_GF_CTRL	(29:28) DA1_CTRL	(29:28) DA1_CTRL	0x0	Gain factor control for DAC1
		(27:16) DA1_GF	(27:16) DA1_GF	0x400	Gain factor for DAC1
		(13:12) DA0_CTRL	(13:12) DA0_CTRL	0x0	Gain factor control for DAC0
		(11:0) DA0_GF	(11:0) DA0_GF	0x400	Gain factor for DAC0
P:0xF153	A_O_SDM_CTRL	(17) TRUNCATE_EN	(17) TRUNCATE_EN	0x0	Truncates data between the successive SDM stages
		(16) OD_DELAY	(16) OD_DELAY	0x1	Delay on the RCVR
		(14:8) L_OFFSET	(14:8) L_OFFSET	0x0	Larger offset added to the DAC input
		(7) DITHER_ENABLE	(7) DITHER_ENABLE	0x1	Dither enable
		(6) DCRM_ENABLE	(6) DCRM_ENABLE	0x1	DC remove enable
		(5:0) S_OFFSET	(5:0) S_OFFSET	0x0	Small offset added to the DAC input
P:0xF154	A_O_SDMCLK_THROTTLING	(5:0) PRESCALE	(5:0) PRESCALE	0x0	Future clock throttling value for O_SDMCLK

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF155	A_O_DAC_AO_CTRL	(17) DIFF_OUT_SRC	(17) DIFF_OUT_SRC	0x0	Selects the data source for the differential output
		(16) DIFF_OUT_ENABLE	(16) DIFF_OUT_ENABLE	0x0	Analog differential output between AO0 and AO1
		(15) AO1_LPF_MODE	(15) AO1_LPF_MODE	0x0	Analog output 1 reconstruction filter cut-off frequency
		(14) AO1_ENABLE	(14) AO1_ENABLE	0x0	Analog output 1 enable
		(13:12) AO1_ITRIM	(13:12) AO1_ITRIM	0x1	Analog output 1 bias current control
		(11:9) AO1_ATT	(11:9) AO1_ATT	0x0	Analog output 1 attenuation control
		(8) AO1_MUTE	(8) AO1_MUTE	0x1	Analog output 1 mute control
		(7) AO0_LPF_MODE	(7) AO0_LPF_MODE	0x0	Analog output 0 reconstruction filter cut-off frequency
		(6) AO0_ENABLE	(6) AO0_ENABLE	0x0	Analog output 0 enable
		(5:4) AO0_ITRIM	(5:4) AO0_ITRIM	0x1	Analog output 0 bias current control
		(3:1) AO0_ATT	(3:1) AO0_ATT	0x0	Analog output 0 attenuation control
		(0) AO0_MUTE	(0) AO0_MUTE	0x1	Analog output 0 mute control

A.36 AUDIO MUX CONFIGURATION

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF156	A_AUDIO_MUX_CFG	(21) DMIC_IN_CLK_SRC	(21) DMIC_IN_CLK_SRC	0x0	Selects the DMIC IN clock source
		(20) ROUND_MODE	(20) ROUND_MODE	0x0	Selects the rounding method
		(19) SDM1_ROUND	(19) SDM1_ROUND	0x0	Applies rounding at the Sigma-Delta 1 input
		(18) SDM0_ROUND	(18) SDM0_ROUND	0x0	Applies rounding at the Sigma-Delta 0 input
		(17) INT_FILTER1_ROUND	(17) INT_FILTER1_ROUND	0x0	Applies rounding at the interpolation filter 1 input
		(16) INT_FILTER0_ROUND	(16) INT_FILTER0_ROUND	0x0	Applies rounding at the interpolation filter 0 input
		(15:14) INPUT_CH3_SRC	(15:14) INPUT_CH3_SRC	0x0	Selects the source of the audio input channel 3
		(13:12) INPUT_CH2_SRC	(13:12) INPUT_CH2_SRC	0x0	Selects the source of the audio input channel 2
		(11:10) INPUT_CH1_SRC	(11:10) INPUT_CH1_SRC	0x0	Selects the source of the audio input channel 1
		(9:8) INPUT_CH0_SRC	(9:8) INPUT_CH0_SRC	0x0	Selects the source of the audio input channel 0
		(7) IOC_SRC3	(7) IOC_SRC3	0x0	Configures the IOC audio input channel 3
		(6) IOC_SRC2	(6) IOC_SRC2	0x0	Configures the IOC audio input channel 2
		(5) IOC_SRC1	(5) IOC_SRC1	0x0	Configures the IOC audio input channel 1
		(4) IOC_SRC0	(4) IOC_SRC0	0x0	Configures the IOC audio input channel 0
		(3) INT_FILTER1	(3) INT_FILTER1	0x0	Configures the interpolation filter input channel 1
		(2) INT_FILTER0	(2) INT_FILTER0	0x0	Configures the interpolation filter input channel 0
		(1) SDM1	(1) SDM1	0x0	Configures the Sigma-Delta input channel 1
		(0) SDM0	(0) SDM0	0x0	Configures the Sigma-Delta input channel 0
P:0xF157	A_BYPASS_DATA_REG_0	(23:7) BYPASS_DATA	(23:7) BYPASS_DATA	0x0	17-bit signed data for the input data bypass (sign extended to 32 bit)
P:0xF158	A_BYPASS_DATA_REG_1	(23:7) BYPASS_DATA	(23:7) BYPASS_DATA	0x0	17-bit signed data for the input data bypass (sign extended to 32 bit)
P:0xF159	A_BYPASS_DATA_REG_2	(23:7) BYPASS_DATA	(23:7) BYPASS_DATA	0x0	17-bit signed data for the input data bypass (sign extended to 32 bit)
P:0xF15A	A_BYPASS_DATA_REG_3	(23:7) BYPASS_DATA	(23:7) BYPASS_DATA	0x0	17-bit signed data for the input data bypass (sign extended to 32 bit)

A.37 DMIC OUTPUT CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF15B	A_DMIC_OUTPUT_CTRL	(5) DMIC1_HIGH_SRC	(5) DMIC1_HIGH_SRC	0x0	Selects the source of the DMIC1 high data channel
		(4) DMIC1_LOW_SRC	(4) DMIC1_LOW_SRC	0x0	Selects the source of the DMIC1 low data channel
		(3) DMIC0_HIGH_SRC	(3) DMIC0_HIGH_SRC	0x0	Selects the source of the DMIC0 high data channel
		(2) DMIC0_LOW_SRC	(2) DMIC0_LOW_SRC	0x0	Selects the source of the DMIC0 low data channel
		(1) DMIC1_ENABLE	(1) DMIC1_ENABLE	0x0	Enables/disables the DMIC1 output
		(0) DMIC0_ENABLE	(0) DMIC0_ENABLE	0x0	Enables/disables the DMIC0 output

A.38 LOW-SPEED A/D

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF180	A_LSAD_CH0_DATA	-	(31:0) LSAD_CH0_DATA	0x0	10-bit LSAD conversion result for CH0 (sign extended to 32 bits when in signed mode)
P:0xF181	A_LSAD_CH1_DATA	-	(31:0) LSAD_CH1_DATA	0x0	10-bit LSAD conversion result for CH1 (sign extended to 32 bits when in signed mode)
P:0xF182	A_LSAD_CH2_DATA	-	(31:0) LSAD_CH2_DATA	0x0	10-bit LSAD conversion result for CH2 (sign extended to 32 bits when in signed mode)
P:0xF183	A_LSAD_CH3_DATA	-	(31:0) LSAD_CH3_DATA	0x0	10-bit LSAD conversion result for CH3 (sign extended to 32 bits when in signed mode)
P:0xF184	A_LSAD_CH4_DATA	-	(31:0) LSAD_CH4_DATA	0x0	10-bit LSAD conversion result for CH4 (sign extended to 32 bits when in signed mode)
P:0xF185	A_LSAD_CH5_DATA	-	(31:0) LSAD_CH5_DATA	0x0	10-bit LSAD conversion result for CH5 (sign extended to 32 bits when in signed mode)
P:0xF186	A_LSAD_VBAT_DATA	-	(31:0) LSAD_VBAT_DATA	0x0	10-bit LSAD conversion result for VBAT (sign extended to 32 bits when in signed mode)
P:0xF187	A_LSAD_GND_DATA	-	(31:0) LSAD_GND_DATA	0x0	10-bit LSAD conversion result for GND (sign extended to 32 bits when in signed mode)
P:0xF188	A_LSAD_CTRL	(11) GND_MODE	(11) GND_MODE	0x0	Defines if result for LSAD_GND_OUT is signed or unsigned
		(10) VBAT_MODE	(10) VBAT_MODE	0x0	Defines if result for LSAD_VBAT_OUT is signed or unsigned
		(9) CH5_MODE	(9) CH5_MODE	0x0	Defines if result for LSAD_CH5_OUT is signed or unsigned
		(8) CH4_MODE	(8) CH4_MODE	0x0	Defines if result for LSAD_CH4_OUT is signed or unsigned
		(7) CH3_MODE	(7) CH3_MODE	0x0	Defines if result for LSAD_CH3_OUT is signed or unsigned
		(6) CH2_MODE	(6) CH2_MODE	0x0	Defines if result for LSAD_CH2_OUT is signed or unsigned
		(5) CH1_MODE	(5) CH1_MODE	0x0	Defines if result for LSAD_CH1_OUT is signed or unsigned
		(4) CH0_MODE	(4) CH0_MODE	0x0	Defines if result for LSAD_CH0_OUT is signed or unsigned
		(3:0) FREQ	(3:0) FREQ	0x4	Defines the sampling frequency of the LSAD channels

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF189	A_LSAD_SRC_CFG	(22) CH5_REF	(22) CH5_REF	0x0	Select channel 5 reference
		(21:20) CH5	(21:20) CH5	0x3	Select channel 5 source
		(18) CH4_REF	(18) CH4_REF	0x0	Select channel 4 reference
		(17:16) CH4	(17:16) CH4	0x3	Select channel 4 source
		(14) CH3_REF	(14) CH3_REF	0x0	Select channel 3 reference
		(13:12) CH3	(13:12) CH3	0x3	Select channel 3 source
		(10) CH2_REF	(10) CH2_REF	0x0	Select channel 2 reference
		(9:8) CH2	(9:8) CH2	0x3	Select channel 2 source
		(6) CH1_REF	(6) CH1_REF	0x0	Select channel 1 reference
		(5:4) CH1	(5:4) CH1	0x3	Select channel 1 source
		(2) CH0_REF	(2) CH0_REF	0x0	Select channel 0 reference
		(1:0) CH0	(1:0) CH0	0x3	Select channel 0 source

A.39 LOW-VOLTAGE DETECTION

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF190	A_SUP_THRSH_DATA	(7:0) SUP_THRSH_DATA	(7:0) SUP_THRSH_DATA	0x80	Battery monitor (LSAD_VBAT) low-voltage detection threshold

A.40 LOW-VOLTAGE COUNT

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF191	A_SUP_COUNT_DATA	-	(7:0) SUP_COUNT_DATA	0x0	Number of times the battery voltage has fallen below the battery monitor voltage threshold (reading the register resets the counter)

A.41 SPI INTERFACE

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF1C0	D_SPI_CFG	(5) CLKPOL	(5) CLKPOL	0x0	Select SPI interface clock polarity
		(4) AUTO	(4) AUTO	0x0	Enable/disable automatic read/write mode
		(3) ENABLE	(3) ENABLE	0x0	Enable/disable the SPI interface
		(2:0) PRESCALE	(2:0) PRESCALE	0x0	Divider for the SPI interface
P:0xF1C1	D_SPI_CTRL	-	(7) BUSY	0x0	SPI state
		(7) START	-	N/A	Start SPI transaction
		(6) RW	(6) RW	0x0	Control read/write data from SPI
		(5) CS_N	(5) CS_N	0x0	SPI chip select pin data
		(4:0) N_BITS	(4:0) N_BITS	0x0	Number of bits to send or receive (MSB sent first)
P:0xF1C2	D_SPI_DATA	(31:0) SPI_DATA	(31:0) SPI_DATA	0x0	SPI transmit and receive data
P:0xF1C3	D_SPI_DATA_M	(31:0) SPI_DATA_M	(31:0) SPI_DATA_M	0x0	SPI shadow data register
P:0xF1C4	D_SPI_DATA_M_16LSB	-	(15:0) SPI_DATA_M_16LSB	0x0	Shadow read data register 16 LSB of SPI_DATA
P:0xF1C5	D_SPI_DATA_M_16MSB	-	(15:0) SPI_DATA_M_16MSB	0x0	Shadow read data register 16 MSB of SPI_DATA
P:0xF1C6	D_SPI_DATA_M_8LSB	-	(7:0) SPI_DATA_M_8LSB	0x0	Shadow read data register 8 LSB of SPI_DATA
P:0xF1C7	D_SPI_DATA_M_8_15BITS	-	(7:0) SPI_DATA_M_8_15BITS	0x0	Shadow read data register bits 8 to 15 of SPI_DATA
P:0xF1C8	D_SPI_DATA_M_16_23BITS	-	(7:0) SPI_DATA_M_16_23BITS	0x0	Shadow read data register bits 16 to 23 of SPI_DATA
P:0xF1C9	D_SPI_DATA_M_8MSB	-	(7:0) SPI_DATA_M_8MSB	0x0	Shadow read data registers 8 MSB of SPI_DATA

A.42 I2C INTERFACE

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF200	D_I2C_CFG	(19:12) MASTER_SPEED	(19:12) MASTER_SPEED	0x0	Define the I2C_SCL speed in master mode
		(10:4) SLAVE_ID	(10:4) SLAVE_ID	0x10	Define the I^2C slave address ID
		(3) STOP_INT	(3) STOP_INT	0x0	Enable/disable interrupts on stop conditions
		(2) AUTO_ACK	(2) AUTO_ACK	0x0	Auto acknowledge mode
		(1) CONNECT_IN_STANDBY	(1) CONNECT_IN_STANDBY	0x0	Control if the I^2C lines are connected when running on the standby clock
		(0) SLAVE_EN	(0) SLAVE_EN	0x0	I^2C slave mode enable
P:0xF201	D_I2C_CTRL	-	(14) ERROR_STICKY	0x0	Error status bit (sticky)
		-	(13) BUS_ERROR_STICKY	0x0	Bus error status bit (sticky) - This status bit is automatically reset when the I2C_CTRL register is read.
		-	(12) START_PENDING	0x0	Master frame start pending status bit
		-	(11) MASTER_MODE	0x0	Master mode status bit
		-	(10) STOP_DETECTED	0x0	Indicate if an I^2C stop bit has been detected
		-	(9) DATA_EVENT	0x0	Indicate that the I^2C interface either needs data to transmit or has received data
		-	(8) ERROR	0x0	Error status bit (non-sticky)
		-	(7) BUS_ERROR	0x0	Bus error status bit (non-sticky) - This status bit is automatically reset when the I2C_CTRL register is read.
		-	(6) BUFFER_FULL	0x0	Indicate if the I^2C data buffer is full
		-	(5) CLK_STRETCH	0x0	Clock stretching flag
		(5) RESET	-	N/A	Reset interface
		-	(4) LINE_FREE	0x0	Line free flag
		(4) LAST_DATA	-	N/A	Last data in master mode
		-	(3) ADDR_DATA	0x0	Address / Data byte
		(3) STOP_CMD	-	N/A	Send stop command
		-	(2) RW	0x0	Read / Write frame
		-	(1) GEN_CALL	0x0	General call flag
		(1) NACK	-	N/A	Do not acknowledge event
		-	(0) ACK_STATUS	0x0	Acknowledge status
		(0) ACK	-	N/A	Acknowledge event
P:0xF202	D_I2C_DATA	(7:0) I2C_DATA_TX	-	N/A	Data sent using the I^2C interface
		-	(7:0) I2C_DATA_RX	0x0	Data received using the I^2C interface

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF203	D_I2C_DATA_M	(7:0) I2C_DATA_M_TX	-	N/A	Data sent via I²C interface
		-	(7:0) I2C_DATA_M_RX	0x0	Data received via I²C interface
P:0xF204	D_I2C_CFX_DEBUG_ADDR1	(6:0) DEBUG_ADDR	(6:0) DEBUG_ADDR	0x60	Defines the first CFX processor debug I²C slave address
P:0xF205	D_I2C_ERROR_CNT	-	(7:0) I2C_ERROR_CNT	0x0	Count the number of I²C errors since this register was last read
P:0xF206	D_I2C_ADDR_START	(7:1) ADDR	-	N/A	Address to send with the start condition over the I²C bus
		(0) RW	-	N/A	Direction of the initiated I²C transfer
P:0xF207	D_I2C_PIN_CFG	(3) DRIVE	(3) DRIVE	0x1	Drive control
		(2) PULLUP_DIS	(2) PULLUP_DIS	0x0	Pull-up control
		(1) PULLUP	(1) PULLUP	0x1	Pull-up value selection
		(0) FILTER_EN	(0) FILTER_EN	0x1	Low Pass Filter control
P:0xF208	D_I2C_WATCHDOG_CFG	(2:0) TIMEOUT	(2:0) TIMEOUT	0x2	Watchdog timeout configuration
P:0xF209	D_I2C_CFX_DEBUG_ADDR2	(6:0) DEBUG_ADDR	(6:0) DEBUG_ADDR	0x60	Defines the second CFX processor debug I²C slave address
P:0xF20A	D_I2C_CM3_DEBUG_ADDR	(6:0) DEBUG_ADDR	(6:0) DEBUG_ADDR	0x40	Defines the ARM Cortex-M3 processor debug I²C slave address

A.43 PCM INTERFACE

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF240	D_PCM_CFG	(21) SATURATE_TX_DATA	(21) SATURATE_TX_DATA	0x0	Data transmitted is saturated if it cannot be stored within the configured word size
		(20) FIELD_TX_DATA	(20) FIELD_TX_DATA	0x0	Indicate which bits from PCM_DATA or IOC are transmitted
		(19) SHIFT_RX_DATA	(19) SHIFT_RX_DATA	0x0	Left or right shift the data received within the 24-bit data value
		(18:15) WORD_SIZE	(18:15) WORD_SIZE	0x0	Select the number of bits per PCM word
		(14:12) MULTIWORD_CTRL	(14:12) MULTIWORD_CTRL	0x0	Configure the number of words per PCM frame
		(11) DATA_MGT	(11) DATA_MGT	0x0	Select whether the CFX or the IOC will manage PCM data
		(7) TX_INT	(7) TX_INT	0x0	Enable/disable transmit interrupts
		(6) RX_INT	(6) RX_INT	0x0	Enable/disable receive interrupts
		(5) FRAME_WIDTH	(5) FRAME_WIDTH	0x0	Select between a short (pulse) or a long PCM frame signal
		(4) FRAME_ALIGN	(4) FRAME_ALIGN	0x0	Select the relative alignment of the PCM frame and data signals
		(3) BIT_ORDER	(3) BIT_ORDER	0x0	Select a data bit ordering for data that is sent and received, selecting whether data starts with the MSB or the LSB
		(2) SUBFRAME	(2) SUBFRAME	0x0	Select either a frame signal with every word, or a frame signal with every frame
		(1) ENABLE	(1) ENABLE	0x0	Enable the PCM interface
		(0) SLAVE	(0) SLAVE	0x1	Select master or slave mode operation for the PCM interface
P:0xF241	D_PCM_DATA_0	(24:0) PCM_DATA_0	(24:0) PCM_DATA_0	0x0	Data received or to send on channel 0
P:0xF242	D_PCM_DATA_1	(24:0) PCM_DATA_1	(24:0) PCM_DATA_1	0x0	Data received or to send on channel 1
P:0xF243	D_PCM_CTRL	-	(1) TX_INT_STATUS	0x0	Indicate if a PCM transmit interrupt was generated
		-	(0) RX_INT_STATUS	0x0	Indicate if a PCM receive interrupt was generated

A.44 DIO INTERFACE

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF280	D_DIO_CFG_0	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection
P:0xF281	D_DIO_CFG_1	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection
P:0xF282	D_DIO_CFG_2	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection
P:0xF283	D_DIO_CFG_3	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection
P:0xF284	D_DIO_CFG_4	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection
P:0xF285	D_DIO_CFG_5	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF286	D_DIO_CFG_6	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection
P:0xF287	D_DIO_CFG_7	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection
P:0xF288	D_DIO_CFG_8	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection
P:0xF289	D_DIO_CFG_9	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection
P:0xF28A	D_DIO_CFG_10	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection
P:0xF28B	D_DIO_CFG_11	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection
P:0xF28C	D_DIO_CFG_12	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF28D	D_DIO_CFG_13	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection
P:0xF28E	D_DIO_CFG_14	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection
P:0xF28F	D_DIO_CFG_15	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection
P:0xF290	D_DIO_CFG_16	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection
P:0xF291	D_DIO_CFG_17	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection
P:0xF292	D_DIO_CFG_18	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection
P:0xF293	D_DIO_CFG_19	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF294	D_DIO_CFG_20	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection
P:0xF295	D_DIO_CFG_21	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection
P:0xF296	D_DIO_CFG_22	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection
P:0xF297	D_DIO_CFG_23	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection
P:0xF298	D_DIO_CFG_24	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection
P:0xF299	D_DIO_CFG_25	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection
P:0xF29A	D_DIO_CFG_26	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF29B	D_DIO_CFG_27	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection
P:0xF29C	D_DIO_CFG_28	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection
P:0xF29D	D_DIO_CFG_29	(11) DRIVE	(11) DRIVE	0x0	Drive strength configuration
		(10) LPF	(10) LPF	0x0	Low Pass Filter enable
		(9) PUW	(9) PUW	0x1	Weak pull-up resistor enable/disable
		(8) PUS	(8) PUS	0x0	Strong pull-up resistor enable/disable
		(4:0) IO_MODE	(4:0) IO_MODE	0x1F	I/O mode selection
P:0xF2A0	D_DIO_DATA	-	(29:0) DIO	0x0	DIO [29:0] read data (pads which are in LSAD mode return 0)
		(29:0) GPIO	-	N/A	GPIO [29:0] write data (updates output data of DIO [29:0] only for pads with IO_MODE 0b000xx)
P:0xF2A1	D_DIO_DIR	-	(29:0) DIO	0x0	Get DIO direction
		(29:0) GPIO	-	N/A	Set DIO GPIO direction (only if IO_MODE is 0b000xx)
P:0xF2A2	D_DIO_MODE	-	(29:0) GPIO	0x0	DIO mode
P:0xF2A4	D_DIO_DATA_VDDO_1	-	(9:0) DIO	0x0	DIO read data (pads which are in LSAD mode return 0)
		(9:0) GPIO	-	N/A	GPIO write data (updates output data of DIO only for pads with IO_MODE 0b000xx)
P:0xF2A5	D_DIO_DATA_VDDO_2	-	(9:0) DIO	0x0	DIO read data (pads which are in LSAD mode return 0)
		(9:0) GPIO	-	N/A	GPIO write data (updates output data of DIO only for pads with IO_MODE 0b000xx)
P:0xF2A6	D_DIO_DATA_VDDO_3	-	(9:0) DIO	0x0	DIO read data (pads which are in LSAD mode return 0)
		(9:0) GPIO	-	N/A	GPIO write data (updates output data of DIO only for pads with IO_MODE 0b000xx)
P:0xF2A8	D_CFX_DIO_INT_CFG_0	(7:5) EVENT	(7:5) EVENT	0x0	Interrupt event configuration
		(4:0) SRC	(4:0) SRC	0x0	Interrupt input selection

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF2A9	D_CFX_DIO_INT_CFG_1	(7:5) EVENT	(7:5) EVENT	0x0	Interrupt event configuration
		(4:0) SRC	(4:0) SRC	0x0	Interrupt input selection
P:0xF2AA	D_CFX_DIO_INT_CFG_2	(7:5) EVENT	(7:5) EVENT	0x0	Interrupt event configuration
		(4:0) SRC	(4:0) SRC	0x0	Interrupt input selection
P:0xF2AB	D_CFX_DIO_INT_CFG_3	(7:5) EVENT	(7:5) EVENT	0x0	Interrupt event configuration
		(4:0) SRC	(4:0) SRC	0x0	Interrupt input selection
P:0xF2AC	D_CM3_DIO_INT_CFG_0	(7:5) EVENT	(7:5) EVENT	0x0	ARM Cortex-M3 core interrupt event configuration
		(4:0) SRC	(4:0) SRC	0x0	ARM Cortex-M3 core interrupt input selection
P:0xF2AD	D_CM3_DIO_INT_CFG_1	(7:5) EVENT	(7:5) EVENT	0x0	ARM Cortex-M3 core interrupt event configuration
		(4:0) SRC	(4:0) SRC	0x0	ARM Cortex-M3 core interrupt input selection
P:0xF2AE	D_CM3_DIO_INT_CFG_2	(7:5) EVENT	(7:5) EVENT	0x0	ARM Cortex-M3 core interrupt event configuration
		(4:0) SRC	(4:0) SRC	0x0	ARM Cortex-M3 core interrupt input selection
P:0xF2AF	D_CM3_DIO_INT_CFG_3	(7:5) EVENT	(7:5) EVENT	0x0	ARM Cortex-M3 core interrupt event configuration
		(4:0) SRC	(4:0) SRC	0x0	ARM Cortex-M3 core interrupt input selection
P:0xF2B0	D_CFX_DIO_INT_STATUS	-	(3) CFX_INT3_STATUS	0x0	CFX interrupt 3 status (cleared when read)
		-	(2) CFX_INT2_STATUS	0x0	CFX interrupt 2 status (cleared when read)
		-	(1) CFX_INT1_STATUS	0x0	CFX interrupt 1 status (cleared when read)
		-	(0) CFX_INT0_STATUS	0x0	CFX interrupt 0 status (cleared when read)
P:0xF2B1	D_CM3_DIO_INT_STATUS	-	(3) CM3_INT3_STATUS	0x0	ARM Cortex-M3 core interrupt 3 status (cleared when read)
		-	(2) CM3_INT2_STATUS	0x0	ARM Cortex-M3 core interrupt 2 status (cleared when read)
		-	(1) CM3_INT1_STATUS	0x0	ARM Cortex-M3 core interrupt 1 status (cleared when read)
		-	(0) CM3_INT0_STATUS	0x0	ARM Cortex-M3 core interrupt 0 status (cleared when read)
P:0xF2B2	D_PCM_SRC	(20:16) SERI	(20:16) SERI	0x1F	PCM_SERI input selection
		(12:8) FRAME	(12:8) FRAME	0x1F	PCM_FRAME input selection
		(4:0) CLK	(4:0) CLK	0x1F	PCM_CLK input selection
P:0xF2B3	D_SPI_SRC	(4:0) SERI	(4:0) SERI	0x1F	SPI_SERI input selection
P:0xF2B4	D_UART_SRC	(4:0) RX	(4:0) RX	0x1F	UART_RX input selection
P:0xF2B5	D_SYNC_SLAVE_SRC	(4:0) SYNC	(4:0) SYNC	0x1F	SYNC_SLAVE input selection
P:0xF2B6	D_DAC_SCALE_SRC	(4:0) DAC_SCALE	(4:0) DAC_SCALE	0x1F	DAC_SCALE input selection
P:0xF2B7	D_C_PCM_SRC	(20:16) SERI	(20:16) SERI	0x1F	C_PCM_SERI input selection
		(12:8) FRAME	(12:8) FRAME	0x1F	C_PCM_FRAME input selection
		(4:0) CLK	(4:0) CLK	0x1F	C_PCM_CLK input selection

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF2B8	D_C_SPI_SRC	(20:16) SERI	(20:16) SERI	0x1F	C_SPI_SERI input selection
		(12:8) CS	(12:8) CS	0x1F	C_SPI_CS input selection
		(4:0) CLK	(4:0) CLK	0x1F	C_SPI_CLK input selection
P:0xF2B9	D_C_I2C_SRC	(12:8) SDA	(12:8) SDA	0x1F	C_SDA input selection
		(4:0) SCL	(4:0) SCL	0x1F	C_SCL input selection
P:0xF2BA	D_DMIC_SRC	(28:24) DMIC3	(28:24) DMIC3	0x1F	DMIC3 input selection
		(20:16) DMIC2	(20:16) DMIC2	0x1F	DMIC2 input selection
		(12:8) DMIC1	(12:8) DMIC1	0x1F	DMIC1 input selection
		(4:0) DMIC0	(4:0) DMIC0	0x1F	DMIC0 input selection
P:0xF2BB	D_PLL_SRC	(4:0) REF_CLK	(4:0) REF_CLK	0x1E	PLL reference clock input selection

A.45 UART INTERFACE

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF2C0	D_UART_CFG	(23:8) PRESCALE	(23:8) PRESCALE	0x0	Pre-scaling multiplier in baud rate calculation
		(4) PRESCALE_ENABLE	(4) PRESCALE_ENABLE	0x0	Enable/disable a fixed prescaler by 12
		(0) ENABLE	(0) ENABLE	0x0	Enable/disable the UART interface
P:0xF2C1	D_UART_DATA	(7:0) UART_DATA	(7:0) UART_DATA	0x0	Data to transmit or receive through UART interface
P:0xF2C2	D_UART_DATA_READY	-	(1) UART_RX_READY	0x0	Status of the UART receive buffer
		-	(0) UART_TX_READY	0x0	Status of the UART transmit buffer

A.46 FILTER ENGINE CONTROL AND CONFIGURATION

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF300	D_FENG_INT_STATUS	(27) OVERLOAD	(27) OVERLOAD	0x0	Processor overload interrupt status
		(26) ALU_SAT	(26) ALU_SAT	0x0	ALU saturation interrupt status
		(25) CORE_SAT	(25) CORE_SAT	0x0	Core saturation interrupt status
		(24) MEM_ERR	(24) MEM_ERR	0x0	Illegal memory access interrupt status
		(23) REG_ERR	(23) REG_ERR	0x0	Register write access collision interrupt status
		(22) CMP	(22) CMP	0x0	Compare instruction interrupt status
		(21) PGE	(21) PGE	0x0	End of PGM execution interrupt status
		(20) OUT7	(20) OUT7	0x0	New data capture at Filter Engine output 7 interrupt status
		(19) OUT6	(19) OUT6	0x0	New data capture at Filter Engine output 6 interrupt status
		(18) OUT5	(18) OUT5	0x0	New data capture at Filter Engine output 5 interrupt status
		(17) OUT4	(17) OUT4	0x0	New data capture at Filter Engine output 4 interrupt status
		(16) OUT3	(16) OUT3	0x0	New data capture at Filter Engine output 3 interrupt status
		(15) OUT2	(15) OUT2	0x0	New data capture at Filter Engine output 2 interrupt status
		(14) OUT1	(14) OUT1	0x0	New data capture at Filter Engine output 1 interrupt status
		(13) OUT0	(13) OUT0	0x0	New data capture at Filter Engine output 0 interrupt status
		(12) ISHC	(12) ISHC	0x0	Input shadow registers capture in RF interrupt status
		(11) SRIN11	(11) SRIN11	0x0	New Filter Engine input 11 and signal ready arrival interrupt status
		(10) SRIN10	(10) SRIN10	0x0	New Filter Engine input 10 and signal ready arrival interrupt status
		(9) SRIN9	(9) SRIN9	0x0	New Filter Engine input 9 and signal ready arrival interrupt status
		(8) SRIN8	(8) SRIN8	0x0	New Filter Engine input 8 and signal ready arrival interrupt status
		(7) SRIN7	(7) SRIN7	0x0	New Filter Engine input 7 and signal ready arrival interrupt status
		(6) SRIN6	(6) SRIN6	0x0	New Filter Engine input 6 and signal ready arrival interrupt status
		(5) SRIN5	(5) SRIN5	0x0	New Filter Engine input 5 and signal ready arrival interrupt status

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF300 (continued)	D_FENG_INT_STATUS (continued)	(4) SRIN4	(4) SRIN4	0x0	New Filter Engine input 4 and signal ready arrival interrupt status
		(3) SRIN3	(3) SRIN3	0x0	New Filter Engine input 3 and signal ready arrival interrupt status
		(2) SRIN2	(2) SRIN2	0x0	New Filter Engine input 2 and signal ready arrival interrupt status
		(1) SRIN1	(1) SRIN1	0x0	New Filter Engine input 1 and signal ready arrival interrupt status
		(0) SRIN0	(0) SRIN0	0x0	New Filter Engine input 0 and signal ready arrival interrupt status

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF301	D_FENG_INT_ENABLE	(27) OVERLOAD	(27) OVERLOAD	0x0	Processor overload interrupt enable
		(26) ALU_SAT	(26) ALU_SAT	0x0	ALU saturation interrupt enable
		(25) CORE_SAT	(25) CORE_SAT	0x0	Core saturation interrupt enable
		(24) MEM_ERR	(24) MEM_ERR	0x0	Illegal memory access interrupt enable
		(23) REG_ERR	(23) REG_ERR	0x0	Register write access collision interrupt enable
		(22) CMP	(22) CMP	0x0	Compare instruction interrupt enable
		(21) PGE	(21) PGE	0x0	End of PGM execution interrupt enable
		(20) OUT7	(20) OUT7	0x0	New data capture at Filter Engine output 7 interrupt enable
		(19) OUT6	(19) OUT6	0x0	New data capture at Filter Engine output 6 interrupt enable
		(18) OUT5	(18) OUT5	0x0	New data capture at Filter Engine output 5 interrupt enable
		(17) OUT4	(17) OUT4	0x0	New data capture at Filter Engine output 4 interrupt enable
		(16) OUT3	(16) OUT3	0x0	New data capture at Filter Engine output 3 interrupt enable
		(15) OUT2	(15) OUT2	0x0	New data capture at Filter Engine output 2 interrupt enable
		(14) OUT1	(14) OUT1	0x0	New data capture at Filter Engine output 1 interrupt enable
		(13) OUT0	(13) OUT0	0x0	New data capture at Filter Engine output 0 interrupt enable
		(12) ISHC	(12) ISHC	0x0	Input shadow registers captured in RF interrupt enable
		(11) SRIN11	(11) SRIN11	0x0	New Filter Engine input 11 and signal ready arrival interrupt enable
		(10) SRIN10	(10) SRIN10	0x0	New Filter Engine input 10 and signal ready arrival interrupt enable
		(9) SRIN9	(9) SRIN9	0x0	New Filter Engine input 9 and signal ready arrival interrupt enable
		(8) SRIN8	(8) SRIN8	0x0	New Filter Engine input 8 and signal ready arrival interrupt enable

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF301 (continued)	D_FENG_INT_ENABLE (continued)	(7) SRIN7	(7) SRIN7	0x0	New Filter Engine input 7 and signal ready arrival interrupt enable
		(6) SRIN6	(6) SRIN6	0x0	New Filter Engine input 6 and signal ready arrival interrupt enable
		(5) SRIN5	(5) SRIN5	0x0	New Filter Engine input 5 and signal ready arrival interrupt enable
		(4) SRIN4	(4) SRIN4	0x0	New Filter Engine input 4 and signal ready arrival interrupt enable
		(3) SRIN3	(3) SRIN3	0x0	New Filter Engine input 3 and signal ready arrival interrupt enable
		(2) SRIN2	(2) SRIN2	0x0	New Filter Engine input 2 and signal ready arrival interrupt enable
		(1) SRIN1	(1) SRIN1	0x0	New Filter Engine input 1 and signal ready arrival interrupt enable
		(0) SRIN0	(0) SRIN0	0x0	New Filter Engine input 0 and signal ready arrival interrupt enable

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF302	D_FENG_CFG	(28:24) COEFF_OFFSET	(28:24) COEFF_OFFSET	0x0	The coefficient decoder offset
		(23) OSR7_EBL	(23) OSR7_EBL	0x0	Shadow register capture mode for output 7
		(22) OSR6_EBL	(22) OSR6_EBL	0x0	Shadow register capture mode for output 6
		(21) OSR5_EBL	(21) OSR5_EBL	0x1	Shadow register capture mode for output 5
		(20) OSR4_EBL	(20) OSR4_EBL	0x1	Shadow register capture mode for output 4
		(19) OSR3_EBL	(19) OSR3_EBL	0x1	Shadow register capture mode for output 3
		(18) OSR2_EBL	(18) OSR2_EBL	0x1	Shadow register capture mode for output 2
		(17) OSR1_EBL	(17) OSR1_EBL	0x1	Shadow register capture mode for output 1
		(16) OSR0_EBL	(16) OSR0_EBL	0x1	Shadow register capture mode for output 0
		(15) FIN11_EBL	(15) FIN11_EBL	0x0	Enable/Disable sample capture on input 11
		(14) FIN10_EBL	(14) FIN10_EBL	0x0	Enable/Disable sample capture on input 10
		(13) FIN9_EBL	(13) FIN9_EBL	0x0	Enable/Disable sample capture on input 9
		(12) FIN8_EBL	(12) FIN8_EBL	0x0	Enable/Disable sample capture on input 8
		(11) FIN7_EBL	(11) FIN7_EBL	0x0	Enable/Disable sample capture on input 7
		(10) FIN6_EBL	(10) FIN6_EBL	0x0	Enable/Disable sample capture on input 6
		(9) FIN5_EBL	(9) FIN5_EBL	0x0	Enable/Disable sample capture on input 5
		(8) FIN4_EBL	(8) FIN4_EBL	0x0	Enable/Disable sample capture on input 4
		(7) FIN3_EBL	(7) FIN3_EBL	0x0	Enable/Disable sample capture on input 3
		(6) FIN2_EBL	(6) FIN2_EBL	0x0	Enable/Disable sample capture on input 2
		(5) FIN1_EBL	(5) FIN1_EBL	0x0	Enable/Disable sample capture on input 1
		(4) FIN0_EBL	(4) FIN0_EBL	0x0	Enable/Disable sample capture on input 0
		(3:0) SRI_SEL	(3:0) SRI_SEL	0xF	Sample ready input starting the Filter Engine processing
P:0xF303	D_FENG_MRCNT	(15:12) MRCNT1	(15:12) MRCNT1	0x0	Multirate Counter 1 Value
		(11:8) MRCNT0	(11:8) MRCNT0	0x0	Multirate Counter 0 Value
		(7:4) MRCNT1_DR	(7:4) MRCNT1_DR	0x0	Multirate Counter 1 Divide Ratio (0 to15)
		(3:0) MRCNT0_DR	(3:0) MRCNT0_DR	0x0	Multirate Counter 0 Divide Ratio (0 to15)
P:0xF304	D_FENG_PGM_SECTOR_EN	(31:16) CNT1	(31:16) CNT1	0x0	FENG_MRCNT_MRCNT1 counter phase controlled sectors enable
		(15:0) CNT0	(15:0) CNT0	0x0	FENG_MRCNT_MRCNT0 counter phase controlled sectors enable

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF305	D_FENG_PHM_PH0_PH1	(23:18) PHM_CNT1PH1	(23:18) PHM_CNT1PH1	0x0	FENG_MRCNT_MRCNT1 phase 1 controlled program memory sector start
		(17:12) PHM_CNT0PH1	(17:12) PHM_CNT0PH1	0x0	FENG_MRCNT_MRCNT0 phase 1 controlled program memory sector start
		(11:6) PHM_CNT1PH0	(11:6) PHM_CNT1PH0	0x0	FENG_MRCNT_MRCNT1 phase 0 controlled program memory sector start
		(5:0) PHM_CNT0PH0	(5:0) PHM_CNT0PH0	0x0	FENG_MRCNT_MRCNT0 phase 0 controlled program memory sector start
P:0xF306	D_FENG_PHM_PH2_PH3	(23:18) PHM_CNT1PH3	(23:18) PHM_CNT1PH3	0x0	FENG_MRCNT_MRCNT1 phase 3 controlled program memory sector start
		(17:12) PHM_CNT0PH3	(17:12) PHM_CNT0PH3	0x0	FENG_MRCNT_MRCNT0 phase 3 controlled program memory sector start
		(11:6) PHM_CNT1PH2	(11:6) PHM_CNT1PH2	0x0	FENG_MRCNT_MRCNT1 phase 2 controlled program memory sector start
		(5:0) PHM_CNT0PH2	(5:0) PHM_CNT0PH2	0x0	FENG_MRCNT_MRCNT0 phase 2 controlled program memory sector start
P:0xF307	D_FENG_PHM_PH4_PH5	(23:18) PHM_CNT1PH5	(23:18) PHM_CNT1PH5	0x0	FENG_MRCNT_MRCNT1 phase 5 controlled program memory sector start
		(17:12) PHM_CNT0PH5	(17:12) PHM_CNT0PH5	0x0	FENG_MRCNT_MRCNT0 phase 5 controlled program memory sector start
		(11:6) PHM_CNT1PH4	(11:6) PHM_CNT1PH4	0x0	FENG_MRCNT_MRCNT1 phase 4 controlled program memory sector start
		(5:0) PHM_CNT0PH4	(5:0) PHM_CNT0PH4	0x0	FENG_MRCNT_MRCNT0 phase 4 controlled program memory sector start
P:0xF308	D_FENG_PHM_PH6_PH7	(23:18) PHM_CNT1PH7	(23:18) PHM_CNT1PH7	0x0	FENG_MRCNT_MRCNT1 phase 7 controlled program memory sector start
		(17:12) PHM_CNT0PH7	(17:12) PHM_CNT0PH7	0x0	FENG_MRCNT_MRCNT0 phase 7 controlled program memory sector start
		(11:6) PHM_CNT1PH6	(11:6) PHM_CNT1PH6	0x0	FENG_MRCNT_MRCNT1 phase 6 controlled program memory sector start
		(5:0) PHM_CNT0PH6	(5:0) PHM_CNT0PH6	0x0	FENG_MRCNT_MRCNT0 phase 6 controlled program memory sector start
P:0xF309	D_FENG_PHM_PH8_PH9	(23:18) PHM_CNT1PH9	(23:18) PHM_CNT1PH9	0x0	FENG_MRCNT_MRCNT1 phase 9 controlled program memory sector start
		(17:12) PHM_CNT0PH9	(17:12) PHM_CNT0PH9	0x0	FENG_MRCNT_MRCNT0 phase 9 controlled program memory sector start
		(11:6) PHM_CNT1PH8	(11:6) PHM_CNT1PH8	0x0	FENG_MRCNT_MRCNT1 phase 8 controlled program memory sector start
		(5:0) PHM_CNT0PH8	(5:0) PHM_CNT0PH8	0x0	FENG_MRCNT_MRCNT0 phase 8 controlled program memory sector start

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF30A	D_FENG_PHM_PH10_PH11	(23:18) PHM_CNT1PH11	(23:18) PHM_CNT1PH11	0x0	FENG_MRCNT_MRCNT1 phase 11 controlled program memory sector start
		(17:12) PHM_CNT0PH11	(17:12) PHM_CNT0PH11	0x0	FENG_MRCNT_MRCNT0 phase 11 controlled program memory sector start
		(11:6) PHM_CNT1PH10	(11:6) PHM_CNT1PH10	0x0	FENG_MRCNT_MRCNT1 phase 10 controlled program memory sector start
		(5:0) PHM_CNT0PH10	(5:0) PHM_CNT0PH10	0x0	FENG_MRCNT_MRCNT0 phase 10 controlled program memory sector start
P:0xF30B	D_FENG_PHM_PH12_PH13	(23:18) PHM_CNT1PH13	(23:18) PHM_CNT1PH13	0x0	FENG_MRCNT_MRCNT1 phase 13 controlled program memory sector start
		(17:12) PHM_CNT0PH13	(17:12) PHM_CNT0PH13	0x0	FENG_MRCNT_MRCNT0 phase 13 controlled program memory sector start
		(11:6) PHM_CNT1PH12	(11:6) PHM_CNT1PH12	0x0	FENG_MRCNT_MRCNT1 phase 12 controlled program memory sector start
		(5:0) PHM_CNT0PH12	(5:0) PHM_CNT0PH12	0x0	FENG_MRCNT_MRCNT0 phase 12 controlled program memory sector start
P:0xF30C	D_FENG_PHM_PH14_PH15	(29:24) PH15_END	(29:24) PH15_END	0x0	FENG_MRCNT_MRCNT1 phase 15 program memory sector end
		(23:18) PHM_CNT1PH15	(23:18) PHM_CNT1PH15	0x0	FENG_MRCNT_MRCNT1 phase 15 controlled program memory sector start
		(17:12) PHM_CNT0PH15	(17:12) PHM_CNT0PH15	0x0	FENG_MRCNT_MRCNT0 phase 15 controlled program memory sector start
		(11:6) PHM_CNT1PH14	(11:6) PHM_CNT1PH14	0x0	FENG_MRCNT_MRCNT1 phase 14 controlled program memory sector start
		(5:0) PHM_CNT0PH14	(5:0) PHM_CNT0PH14	0x0	FENG_MRCNT_MRCNT0 phase 14 controlled program memory sector start
P:0xF30D	D_FENG_COEFF_OFFSET_EN_SHADOW	(31:0) COEFF_OFFSET_EN_SHADOW	(31:0) COEFF_OFFSET_EN_SHADOW	0x0	The coefficient offset enable shadow register
P:0xF30E	D_FENG_COEFF_OFFSET_EN_ACTIVE	(31:0) COEFF_OFFSET_EN_ACTIVE	(31:0) COEFF_OFFSET_EN_ACTIVE	0x0	The coefficient offset enable active register
P:0xF30F	D_FENG_STATE_SHADOW	(15:0) STATE_SHADOW	(15:0) STATE_SHADOW	0xFFFF	The conditional processing state shadow register
P:0xF310	D_FENG_CMDS	(2) EXEC	-	N/A	Filter Engine Program Execution Start
		(1) DECODER_RESET	-	N/A	Filter Engine Decoder Reset
		(0) AUDIO_RESET	-	N/A	Filter Engine Audio Reset
P:0xF311	D_FENG_PGM_ADDR	-	(5:0) DECODER	0x0	Program memory decoder address
P:0xF312	D_FENG_CTRL	(2) PAUSE	(2) PAUSE	0x0	If set, at the next execution start Filter Engine is paused
		(1) SINGLE_STEP	-	N/A	If set, the CTRL_ENABLE is set for one clock cycle
		(0) ENABLE	(0) ENABLE	0x0	If cleared, all the clocks are shut down (CFX can still access Filter Engine)

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF313	D_FENG_STATE_W_EN	(15:0) STATE_W_EN	(15:0) STATE_W_EN	0xFFFF	The conditional processing state register's write enable
P:0xF314	D_FENG_STATE_ACTIVE	(15:0) STATE_ACTIVE	(15:0) STATE_ACTIVE	0xFFFF	The conditional processing state active register

A.47 FILTER ENGINE REGISTER FILE

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF340	D_FENG_RF_EXTENDED_0	(26:0) RF_EXTENDED	(26:0) RF_EXTENDED	0x0	Data read in m5p23 format, written in m4p23 format (sign extended to 32 bits)
P:0xF341	D_FENG_RF_EXTENDED_1	(26:0) RF_EXTENDED	(26:0) RF_EXTENDED	0x0	Data read in m5p23 format, written in m4p23 format (sign extended to 32 bits)
P:0xF342	D_FENG_RF_EXTENDED_2	(26:0) RF_EXTENDED	(26:0) RF_EXTENDED	0x0	Data read in m5p23 format, written in m4p23 format (sign extended to 32 bits)
P:0xF343	D_FENG_RF_EXTENDED_3	(26:0) RF_EXTENDED	(26:0) RF_EXTENDED	0x0	Data read in m5p23 format, written in m4p23 format (sign extended to 32 bits)
P:0xF344	D_FENG_RF_EXTENDED_4	(26:0) RF_EXTENDED	(26:0) RF_EXTENDED	0x0	Data read in m5p23 format, written in m4p23 format (sign extended to 32 bits)
P:0xF345	D_FENG_RF_EXTENDED_5	(26:0) RF_EXTENDED	(26:0) RF_EXTENDED	0x0	Data read in m5p23 format, written in m4p23 format (sign extended to 32 bits)
P:0xF346	D_FENG_RF_EXTENDED_6	(26:0) RF_EXTENDED	(26:0) RF_EXTENDED	0x0	Data read in m5p23 format, written in m4p23 format (sign extended to 32 bits)
P:0xF347	D_FENG_RF_EXTENDED_7	(26:0) RF_EXTENDED	(26:0) RF_EXTENDED	0x0	Data read in m5p23 format, written in m4p23 format (sign extended to 32 bits)
P:0xF348	D_FENG_RF_EXTENDED_8	(26:0) RF_EXTENDED	(26:0) RF_EXTENDED	0x0	Data read in m5p23 format, written in m4p23 format (sign extended to 32 bits)
P:0xF349	D_FENG_RF_EXTENDED_9	(26:0) RF_EXTENDED	(26:0) RF_EXTENDED	0x0	Data read in m5p23 format, written in m4p23 format (sign extended to 32 bits)
P:0xF34A	D_FENG_RF_EXTENDED_10	(26:0) RF_EXTENDED	(26:0) RF_EXTENDED	0x0	Data read in m5p23 format, written in m4p23 format (sign extended to 32 bits)
P:0xF34B	D_FENG_RF_EXTENDED_11	(26:0) RF_EXTENDED	(26:0) RF_EXTENDED	0x0	Data read in m5p23 format, written in m4p23 format (sign extended to 32 bits)
P:0xF34C	D_FENG_RF_EXTENDED_12	(26:0) RF_EXTENDED	(26:0) RF_EXTENDED	0x0	Data read in m5p23 format, written in m4p23 format (sign extended to 32 bits)
P:0xF34D	D_FENG_RF_EXTENDED_13	(26:0) RF_EXTENDED	(26:0) RF_EXTENDED	0x0	Data read in m5p23 format, written in m4p23 format (sign extended to 32 bits)
P:0xF34E	D_FENG_RF_EXTENDED_14	(26:0) RF_EXTENDED	(26:0) RF_EXTENDED	0x0	Data read in m5p23 format, written in m4p23 format (sign extended to 32 bits)
P:0xF34F	D_FENG_RF_EXTENDED_15	(26:0) RF_EXTENDED	(26:0) RF_EXTENDED	0x0	Data read in m5p23 format, written in m4p23 format (sign extended to 32 bits)
P:0xF350	D_FENG_RF_SINGLE_0	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF351	D_FENG_RF_SINGLE_1	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF352	D_FENG_RF_SINGLE_2	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF353	D_FENG_RF_SINGLE_3	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF354	D_FENG_RF_SINGLE_4	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF355	D_FENG_RF_SINGLE_5	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF356	D_FENG_RF_SINGLE_6	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32 bits)

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF357	D_FENG_RF_SINGLE_7	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF358	D_FENG_RF_SINGLE_8	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF359	D_FENG_RF_SINGLE_9	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF35A	D_FENG_RF_SINGLE_10	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF35B	D_FENG_RF_SINGLE_11	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF35C	D_FENG_RF_SINGLE_12	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF35D	D_FENG_RF_SINGLE_13	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF35E	D_FENG_RF_SINGLE_14	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF35F	D_FENG_RF_SINGLE_15	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF360	D_FENG_RF_SINGLE_16	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF361	D_FENG_RF_SINGLE_17	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF362	D_FENG_RF_SINGLE_18	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF363	D_FENG_RF_SINGLE_19	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF364	D_FENG_RF_SINGLE_20	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF365	D_FENG_RF_SINGLE_21	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF366	D_FENG_RF_SINGLE_22	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF367	D_FENG_RF_SINGLE_23	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF368	D_FENG_RF_SINGLE_24	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF369	D_FENG_RF_SINGLE_25	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF36A	D_FENG_RF_SINGLE_26	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF36B	D_FENG_RF_SINGLE_27	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF36C	D_FENG_RF_IN_0	(23:0) RF_IN	(23:0) RF_IN	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF36D	D_FENG_RF_IN_1	(23:0) RF_IN	(23:0) RF_IN	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF36E	D_FENG_RF_IN_2	(23:0) RF_IN	(23:0) RF_IN	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF36F	D_FENG_RF_IN_3	(23:0) RF_IN	(23:0) RF_IN	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF370	D_FENG_RF_IN_4	(23:0) RF_IN	(23:0) RF_IN	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF371	D_FENG_RF_IN_5	(23:0) RF_IN	(23:0) RF_IN	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF372	D_FENG_RF_IN_6	(23:0) RF_IN	(23:0) RF_IN	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF373	D_FENG_RF_IN_7	(23:0) RF_IN	(23:0) RF_IN	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF374	D_FENG_RF_IN_8	(23:0) RF_IN	(23:0) RF_IN	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF375	D_FENG_RF_IN_9	(23:0) RF_IN	(23:0) RF_IN	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF376	D_FENG_RF_IN_10	(23:0) RF_IN	(23:0) RF_IN	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF377	D_FENG_RF_IN_11	(23:0) RF_IN	(23:0) RF_IN	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF378	D_FENG_RF_OUT_0	(23:0) RF_OUT	(23:0) RF_OUT	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF379	D_FENG_RF_OUT_1	(23:0) RF_OUT	(23:0) RF_OUT	0x0	Data in m1p23 format (sign extended to 32 bits)

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF37A	D_FENG_RF_OUT_2	(23:0) RF_OUT	(23:0) RF_OUT	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF37B	D_FENG_RF_OUT_3	(23:0) RF_OUT	(23:0) RF_OUT	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF37C	D_FENG_RF_OUT_4	(23:0) RF_OUT	(23:0) RF_OUT	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF37D	D_FENG_RF_OUT_5	(23:0) RF_OUT	(23:0) RF_OUT	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF37E	D_FENG_RF_OUT_6	(23:0) RF_OUT	(23:0) RF_OUT	0x0	Data in m1p23 format (sign extended to 32 bits)
P:0xF37F	D_FENG_RF_OUT_7	(23:0) RF_OUT	(23:0) RF_OUT	0x0	Data in m1p23 format (sign extended to 32 bits)

A.48 SIGNAL DETECTION UNIT

Address	Register Name	Register Write	Register Read	Default	Description
P:0xF380	D_SDU_OUT_0	-	(31:0) SDU_OUT	0x0	12-bit AD conversion result, sign extended to 32 bits
P:0xF381	D_SDU_OUT_1	-	(31:0) SDU_OUT	0x0	12-bit AD conversion result, sign extended to 32 bits
P:0xF382	D_SDU_OUT_2	-	(31:0) SDU_OUT	0x0	12-bit AD conversion result, sign extended to 32 bits
P:0xF383	D_SDU_OUT_3	-	(31:0) SDU_OUT	0x0	12-bit AD conversion result, sign extended to 32 bits
P:0xF384	D_SDU_OUT_4	-	(31:0) SDU_OUT	0x0	12-bit AD conversion result, sign extended to 32 bits
P:0xF385	D_SDU_OUT_5	-	(31:0) SDU_OUT	0x0	12-bit AD conversion result, sign extended to 32 bits
P:0xF386	D_SDU_OUT_6	-	(31:0) SDU_OUT	0x0	12-bit AD conversion result, sign extended to 32 bits
P:0xF387	D_SDU_OUT_7	-	(31:0) SDU_OUT	0x0	12-bit AD conversion result, sign extended to 32 bits
P:0xF388	D_SDU_CFG	(23) POLVR	(23) POLVR	0x0	Preamp differential pair bias setting
		(22) LINE_OUT_ENABLE	(22) LINE_OUT_ENABLE	0x0	Lout pad control
		(18:16) SFCR	(18:16) SFCR	0x7	Sampling frequency configuration
		(14:12) P_CURRENT	(14:12) P_CURRENT	0x3	Trimming of the preamplifier current
		(10:8) ADC_CURRENT	(10:8) ADC_CURRENT	0x3	Trimming of the ADC current
		(7:4) PR_GAIN	(7:4) PR_GAIN	0x0	Select the gain of the preamplifier
		(3:1) IN_SEL	(3:1) IN_SEL	0x7	Select input applied to preamplifier
		(0) ADC_ENABLE	(0) ADC_ENABLE	0x0	Enable/disable the ADC

A.49 HEAR FIFO A0 CONFIGURATION AND CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
X:0x7000	D_FIFO_A0_0_START	(10:5) OFFSET	(10:5) OFFSET	0x0	Start address for FIFO A0 0
X:0x7001	D_FIFO_A0_0_END	(10:5) OFFSET	(10:5) OFFSET	0x0	End address (+1) for FIFO A0 0
X:0x7002	D_FIFO_A0_0_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A0 0
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO A0 0
X:0x7003	D_FIFO_A0_0_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A0 0
X:0x7004	D_FIFO_A0_0_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO A0 0
X:0x7005	D_FIFO_A0_0_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO A0 0
X:0x7008	D_FIFO_A0_0_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO A0 0
X:0x700C	D_FIFO_A0_0_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO A0 0
X:0x700E	D_FIFO_A0_0_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A0 0
X:0x7010	D_FIFO_A0_1_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO A0 1
X:0x7011	D_FIFO_A0_1_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO A0 1
X:0x7012	D_FIFO_A0_1_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A0 1
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO A0 1
X:0x7013	D_FIFO_A0_1_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A0 1
X:0x7014	D_FIFO_A0_1_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO A0 1
X:0x7015	D_FIFO_A0_1_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO A0 1
X:0x7018	D_FIFO_A0_1_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO A0 1
X:0x701C	D_FIFO_A0_1_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO A0 1
X:0x701E	D_FIFO_A0_1_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A0 1
X:0x7020	D_FIFO_A0_2_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO A0 2
X:0x7021	D_FIFO_A0_2_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO A0 2
X:0x7022	D_FIFO_A0_2_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A0 2
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO A0 2
X:0x7023	D_FIFO_A0_2_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A0 2
X:0x7024	D_FIFO_A0_2_BASE_PTR	(10:1) RW	(10:1) RW	0x0	FIFO pointer for FIFO A0 2
X:0x7025	D_FIFO_A0_2_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO A0 2
X:0x7028	D_FIFO_A0_2_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO A0 2
X:0x702C	D_FIFO_A0_2_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO A0 2
X:0x702E	D_FIFO_A0_2_ACCESS	(23:1) FIFO_DATA	(23:1) FIFO_DATA	0x0	Smart FIFO access for FIFO A0 2
X:0x7030	D_FIFO_A0_3_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO A0 3
X:0x7031	D_FIFO_A0_3_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO A0 3

Address	Register Name	Register Write	Register Read	Default	Description
X:0x7032	D_FIFO_A0_3_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A0 3
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO A0 3
X:0x7033	D_FIFO_A0_3_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A0 3
X:0x7034	D_FIFO_A0_3_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO A0 3
X:0x7035	D_FIFO_A0_3_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO A0 3
X:0x7038	D_FIFO_A0_3_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO A0 3
X:0x703C	D_FIFO_A0_3_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO A0 3
X:0x703E	D_FIFO_A0_3_ACCESS	(23:0) RW	(23:0) RW	0x0	Smart FIFO access for FIFO A0 3
X:0x7040	D_FIFO_A0_4_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO A0 4
X:0x7041	D_FIFO_A0_4_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO A0 4
X:0x7042	D_FIFO_A0_4_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A0 4
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO A0 4
X:0x7043	D_FIFO_A0_4_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A0 4
X:0x7044	D_FIFO_A0_4_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO A0 4
X:0x7045	D_FIFO_A0_4_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO A0 4
X:0x7048	D_FIFO_A0_4_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO A0 4
X:0x704C	D_FIFO_A0_4_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO A0 4
X:0x704E	D_FIFO_A0_4_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A0 4
X:0x7050	D_FIFO_A0_5_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO A0 5
X:0x7051	D_FIFO_A0_5_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO A0 5
X:0x7052	D_FIFO_A0_5_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A0 5
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO A0 5
X:0x7053	D_FIFO_A0_5_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A0 5
X:0x7054	D_FIFO_A0_5_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO A0 5
X:0x7055	D_FIFO_A0_5_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO A0 5
X:0x7058	D_FIFO_A0_5_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO A0 5
X:0x705C	D_FIFO_A0_5_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO A0 5
X:0x705E	D_FIFO_A0_5_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A0 5
X:0x7060	D_FIFO_A0_6_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO A0 6
X:0x7061	D_FIFO_A0_6_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO A0 6
X:0x7062	D_FIFO_A0_6_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A0 6
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO A0 6
X:0x7063	D_FIFO_A0_6_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A0 6
X:0x7064	D_FIFO_A0_6_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO A0 6
X:0x7065	D_FIFO_A0_6_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO A0 6

Address	Register Name	Register Write	Register Read	Default	Description
X:0x7068	D_FIFO_A0_6_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO A0 6
X:0x706C	D_FIFO_A0_6_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO A0 6
X:0x706E	D_FIFO_A0_6_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A0 6
X:0x7070	D_FIFO_A0_7_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO A0 7
X:0x7071	D_FIFO_A0_7_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO A0 7
X:0x7072	D_FIFO_A0_7_SIZE	(11:5) FIFO_SIZE -	(11:5) FIFO_SIZE (11) FIFO_ENABLE	0x0 0x0	Size (End-Start) for FIFO A0 7 FIFO enable for FIFO A0 7
X:0x7073	D_FIFO_A0_7_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A0 7
X:0x7074	D_FIFO_A0_7_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO A0 7
X:0x7075	D_FIFO_A0_7_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO A0 7
X:0x7078	D_FIFO_A0_7_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO A0 7
X:0x707C	D_FIFO_A0_7_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO A0 7
X:0x707E	D_FIFO_A0_7_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A0 7

A.50 HEAR FIFO A1 CONFIGURATION AND CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
X:0x7080	D_FIFO_A1_0_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO A1 0
X:0x7081	D_FIFO_A1_0_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO A1 0
X:0x7082	D_FIFO_A1_0_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A1 0
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO A1 0
X:0x7083	D_FIFO_A1_0_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A1 0
X:0x7084	D_FIFO_A1_0_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO A1 0
X:0x7085	D_FIFO_A1_0_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO A1 0
X:0x7088	D_FIFO_A1_0_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO A1 0
X:0x708C	D_FIFO_A1_0_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO A1 0
X:0x708E	D_FIFO_A1_0_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A1 0
X:0x7090	D_FIFO_A1_1_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO A1 1
X:0x7091	D_FIFO_A1_1_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO A1 1
X:0x7092	D_FIFO_A1_1_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A1 1
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO A1 1
X:0x7093	D_FIFO_A1_1_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A1 1
X:0x7094	D_FIFO_A1_1_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO A1 1
X:0x7095	D_FIFO_A1_1_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO A1 1
X:0x7098	D_FIFO_A1_1_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO A1 1
X:0x709C	D_FIFO_A1_1_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO A1 1
X:0x709E	D_FIFO_A1_1_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A1 1
X:0x70A0	D_FIFO_A1_2_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO A1 2
X:0x70A1	D_FIFO_A1_2_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO A1 2
X:0x70A2	D_FIFO_A1_2_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A1 2
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO A1 2
X:0x70A3	D_FIFO_A1_2_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A1 2
X:0x70A4	D_FIFO_A1_2_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO A1 2
X:0x70A5	D_FIFO_A1_2_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO A1 2
X:0x70A8	D_FIFO_A1_2_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO A1 2
X:0x70AC	D_FIFO_A1_2_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO A1 2
X:0x70AE	D_FIFO_A1_2_ACCESS	(23:1) FIFO_DATA	(23:1) FIFO_DATA	0x0	Smart FIFO access for FIFO A1 2
X:0x70B0	D_FIFO_A1_3_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO A1 3
X:0x70B1	D_FIFO_A1_3_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO A1 3

Address	Register Name	Register Write	Register Read	Default	Description
X:0x70B2	D_FIFO_A1_3_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A1 3
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO A1 3
X:0x70B3	D_FIFO_A1_3_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A1 3
X:0x70B4	D_FIFO_A1_3_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO A1 3
X:0x70B5	D_FIFO_A1_3_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO A1 3
X:0x70B8	D_FIFO_A1_3_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO A1 3
X:0x70BC	D_FIFO_A1_3_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO A1 3
X:0x70BE	D_FIFO_A1_3_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A1 3
X:0x70C0	D_FIFO_A1_4_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO A1 4
X:0x70C1	D_FIFO_A1_4_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO A1 4
X:0x70C2	D_FIFO_A1_4_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A1 4
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO A1 4
X:0x70C3	D_FIFO_A1_4_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A1 4
X:0x70C4	D_FIFO_A1_4_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO A1 4
X:0x70C5	D_FIFO_A1_4_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO A1 4
X:0x70C8	D_FIFO_A1_4_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO A1 4
X:0x70CC	D_FIFO_A1_4_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO A1 4
X:0x70CE	D_FIFO_A1_4_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A1 4
X:0x70D0	D_FIFO_A1_5_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO A1 5
X:0x70D1	D_FIFO_A1_5_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO A1 5
X:0x70D2	D_FIFO_A1_5_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A1 5
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO A1 5
X:0x70D3	D_FIFO_A1_5_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A1 5
X:0x70D4	D_FIFO_A1_5_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO A1 5
X:0x70D5	D_FIFO_A1_5_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO A1 5
X:0x70D8	D_FIFO_A1_5_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO A1 5
X:0x70DC	D_FIFO_A1_5_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO A1 5
X:0x70DE	D_FIFO_A1_5_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A1 5
X:0x70E0	D_FIFO_A1_6_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO A1 6
X:0x70E1	D_FIFO_A1_6_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO A1 6
X:0x70E2	D_FIFO_A1_6_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A1 6
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO A1 6
X:0x70E3	D_FIFO_A1_6_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A1 6
X:0x70E4	D_FIFO_A1_6_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO A1 6
X:0x70E5	D_FIFO_A1_6_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO A1 6

Address	Register Name	Register Write	Register Read	Default	Description
X:0x70E8	D_FIFO_A1_6_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO A1 6
X:0x70EC	D_FIFO_A1_6_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO A1 6
X:0x70EE	D_FIFO_A1_6_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A1 6
X:0x70F0	D_FIFO_A1_7_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO A1 7
X:0x70F1	D_FIFO_A1_7_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO A1 7
X:0x70F2	D_FIFO_A1_7_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A1 7
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO A1 7
X:0x70F3	D_FIFO_A1_7_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A1 7
X:0x70F4	D_FIFO_A1_7_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO A1 7
X:0x70F5	D_FIFO_A1_7_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO A1 7
X:0x70F8	D_FIFO_A1_7_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO A1 7
X:0x70FC	D_FIFO_A1_7_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO A1 7
X:0x70FE	D_FIFO_A1_7_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A1 7

A.51 HEAR FIFO B0 CONFIGURATION AND CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
X:0x7100	D_FIFO_B0_0_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO B0 0
X:0x7101	D_FIFO_B0_0_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO B0 0
X:0x7102	D_FIFO_B0_0_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B0 0
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO B0 0
X:0x7103	D_FIFO_B0_0_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B0 0
X:0x7104	D_FIFO_B0_0_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO B0 0
X:0x7105	D_FIFO_B0_0_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO B0 0
X:0x7108	D_FIFO_B0_0_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO B0 0
X:0x710C	D_FIFO_B0_0_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO B0 0
X:0x710E	D_FIFO_B0_0_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B0 0
X:0x7110	D_FIFO_B0_1_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO B0 1
X:0x7111	D_FIFO_B0_1_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO B0 1
X:0x7112	D_FIFO_B0_1_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B0 1
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO B0 1
X:0x7113	D_FIFO_B0_1_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B0 1
X:0x7114	D_FIFO_B0_1_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO B0 1
X:0x7115	D_FIFO_B0_1_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO B0 1
X:0x7118	D_FIFO_B0_1_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO B0 1
X:0x711C	D_FIFO_B0_1_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO B0 1
X:0x711E	D_FIFO_B0_1_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B0 1
X:0x7120	D_FIFO_B0_2_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO B0 2
X:0x7121	D_FIFO_B0_2_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO B0 2
X:0x7122	D_FIFO_B0_2_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B0 2
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO B0 2
X:0x7123	D_FIFO_B0_2_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B0 2
X:0x7124	D_FIFO_B0_2_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO B0 2
X:0x7125	D_FIFO_B0_2_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO B0 2
X:0x7128	D_FIFO_B0_2_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO B0 2
X:0x712C	D_FIFO_B0_2_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO B0 2
X:0x712E	D_FIFO_B0_2_ACCESS	(23:1) FIFO_DATA	(23:1) FIFO_DATA	0x0	Smart FIFO access for FIFO B0 2
X:0x7130	D_FIFO_B0_3_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO B0 3
X:0x7131	D_FIFO_B0_3_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO B0 3

Address	Register Name	Register Write	Register Read	Default	Description
X:0x7132	D_FIFO_B0_3_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B0 3
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO B0 3
X:0x7133	D_FIFO_B0_3_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B0 3
X:0x7134	D_FIFO_B0_3_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO B0 3
X:0x7135	D_FIFO_B0_3_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO B0 3
X:0x7138	D_FIFO_B0_3_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO B0 3
X:0x713C	D_FIFO_B0_3_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO B0 3
X:0x713E	D_FIFO_B0_3_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B0 3
X:0x7140	D_FIFO_B0_4_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO B0 4
X:0x7141	D_FIFO_B0_4_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO B0 4
X:0x7142	D_FIFO_B0_4_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B0 4
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO B0 4
X:0x7143	D_FIFO_B0_4_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B0 4
X:0x7144	D_FIFO_B0_4_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO B0 4
X:0x7145	D_FIFO_B0_4_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO B0 4
X:0x7148	D_FIFO_B0_4_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO B0 4
X:0x714C	D_FIFO_B0_4_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO B0 4
X:0x714E	D_FIFO_B0_4_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B0 4
X:0x7150	D_FIFO_B0_5_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO B0 5
X:0x7151	D_FIFO_B0_5_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO B0 5
X:0x7152	D_FIFO_B0_5_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B0 5
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO B0 5
X:0x7153	D_FIFO_B0_5_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B0 5
X:0x7154	D_FIFO_B0_5_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO B0 5
X:0x7155	D_FIFO_B0_5_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO B0 5
X:0x7158	D_FIFO_B0_5_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO B0 5
X:0x715C	D_FIFO_B0_5_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO B0 5
X:0x715E	D_FIFO_B0_5_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B0 5
X:0x7160	D_FIFO_B0_6_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO B0 6
X:0x7161	D_FIFO_B0_6_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO B0 6
X:0x7162	D_FIFO_B0_6_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B0 6
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO B0 6
X:0x7163	D_FIFO_B0_6_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B0 6
X:0x7164	D_FIFO_B0_6_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO B0 6
X:0x7165	D_FIFO_B0_6_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO B0 6

Address	Register Name	Register Write	Register Read	Default	Description
X:0x7168	D_FIFO_B0_6_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO B0 6
X:0x716C	D_FIFO_B0_6_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO B0 6
X:0x716E	D_FIFO_B0_6_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B0 6
X:0x7170	D_FIFO_B0_7_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO B0 7
X:0x7171	D_FIFO_B0_7_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO B0 7
X:0x7172	D_FIFO_B0_7_SIZE	(11:5) FIFO_SIZE -	(11:5) FIFO_SIZE (11) FIFO_ENABLE	0x0 0x0	Size (End-Start) for FIFO B0 7 FIFO enable for FIFO B0 7
X:0x7173	D_FIFO_B0_7_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B0 7
X:0x7174	D_FIFO_B0_7_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO B0 7
X:0x7175	D_FIFO_B0_7_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO B0 7
X:0x7178	D_FIFO_B0_7_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO B0 7
X:0x717C	D_FIFO_B0_7_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO B0 7
X:0x717E	D_FIFO_B0_7_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B0 7

A.52 HEAR FIFO B1 CONFIGURATION AND CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
X:0x7180	D_FIFO_B1_0_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO B1 0
X:0x7181	D_FIFO_B1_0_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO B1 0
X:0x7182	D_FIFO_B1_0_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B1 0
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO B1 0
X:0x7183	D_FIFO_B1_0_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B1 0
X:0x7184	D_FIFO_B1_0_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO B1 0
X:0x7185	D_FIFO_B1_0_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO B1 0
X:0x7188	D_FIFO_B1_0_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO B1 0
X:0x718C	D_FIFO_B1_0_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO B1 0
X:0x718E	D_FIFO_B1_0_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B1 0
X:0x7190	D_FIFO_B1_1_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO B1 1
X:0x7191	D_FIFO_B1_1_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO B1 1
X:0x7192	D_FIFO_B1_1_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B1 1
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO B1 1
X:0x7193	D_FIFO_B1_1_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B1 1
X:0x7194	D_FIFO_B1_1_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO B1 1
X:0x7195	D_FIFO_B1_1_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO B1 1
X:0x7198	D_FIFO_B1_1_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO B1 1
X:0x719C	D_FIFO_B1_1_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO B1 1
X:0x719E	D_FIFO_B1_1_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B1 1
X:0x71A0	D_FIFO_B1_2_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO B1 2
X:0x71A1	D_FIFO_B1_2_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO B1 2
X:0x71A2	D_FIFO_B1_2_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B1 2
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO B1 2
X:0x71A3	D_FIFO_B1_2_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B1 2
X:0x71A4	D_FIFO_B1_2_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO B1 2
X:0x71A5	D_FIFO_B1_2_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO B1 2
X:0x71A8	D_FIFO_B1_2_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO B1 2
X:0x71AC	D_FIFO_B1_2_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO B1 2
X:0x71AE	D_FIFO_B1_2_ACCESS	(23:1) FIFO_DATA	(23:1) FIFO_DATA	0x0	Smart FIFO access for FIFO B1 2
X:0x71B0	D_FIFO_B1_3_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO B1 3
X:0x71B1	D_FIFO_B1_3_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO B1 3

Address	Register Name	Register Write	Register Read	Default	Description
X:0x71B2	D_FIFO_B1_3_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B1 3
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO B1 3
X:0x71B3	D_FIFO_B1_3_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B1 3
X:0x71B4	D_FIFO_B1_3_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO B1 3
X:0x71B5	D_FIFO_B1_3_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO B1 3
X:0x71B8	D_FIFO_B1_3_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO B1 3
X:0x71BC	D_FIFO_B1_3_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO B1 3
X:0x71BE	D_FIFO_B1_3_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B1 3
X:0x71C0	D_FIFO_B1_4_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO B1 4
X:0x71C1	D_FIFO_B1_4_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO B1 4
X:0x71C2	D_FIFO_B1_4_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B1 4
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO B1 4
X:0x71C3	D_FIFO_B1_4_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B1 4
X:0x71C4	D_FIFO_B1_4_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO B1 4
X:0x71C5	D_FIFO_B1_4_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO B1 4
X:0x71C8	D_FIFO_B1_4_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO B1 4
X:0x71CC	D_FIFO_B1_4_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO B1 4
X:0x71CE	D_FIFO_B1_4_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B1 4
X:0x71D0	D_FIFO_B1_5_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO B1 5
X:0x71D1	D_FIFO_B1_5_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO B1 5
X:0x71D2	D_FIFO_B1_5_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B1 5
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO B1 5
X:0x71D3	D_FIFO_B1_5_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B1 5
X:0x71D4	D_FIFO_B1_5_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO B1 5
X:0x71D5	D_FIFO_B1_5_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO B1 5
X:0x71D8	D_FIFO_B1_5_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO B1 5
X:0x71DC	D_FIFO_B1_5_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO B1 5
X:0x71DE	D_FIFO_B1_5_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B1 5
X:0x71E0	D_FIFO_B1_6_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO B1 6
X:0x71E1	D_FIFO_B1_6_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO B1 6
X:0x71E2	D_FIFO_B1_6_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B1 6
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO B1 6
X:0x71E3	D_FIFO_B1_6_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B1 6
X:0x71E4	D_FIFO_B1_6_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO B1 6
X:0x71E5	D_FIFO_B1_6_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO B1 6

Address	Register Name	Register Write	Register Read	Default	Description
X:0x71E8	D_FIFO_B1_6_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO B1 6
X:0x71EC	D_FIFO_B1_6_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO B1 6
X:0x71EE	D_FIFO_B1_6_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B1 6
X:0x71F0	D_FIFO_B1_7_START	(10:5) START_ADDR	(10:5) START_ADDR	0x0	Start address for FIFO B1 7
X:0x71F1	D_FIFO_B1_7_END	(10:5) END_ADDR	(10:5) END_ADDR	0x0	End address (+1) for FIFO B1 7
X:0x71F2	D_FIFO_B1_7_SIZE	(11:5) FIFO_SIZE	(11:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B1 7
		-	(11) FIFO_ENABLE	0x0	FIFO enable for FIFO B1 7
X:0x71F3	D_FIFO_B1_7_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B1 7
X:0x71F4	D_FIFO_B1_7_BASE_PTR	(10:1) FIFO_PTR	(10:1) FIFO_PTR	0x0	FIFO pointer for FIFO B1 7
X:0x71F5	D_FIFO_B1_7_IOBLOCK_PTR	(10:1) IOC_PTR	(10:1) IOC_PTR	0x0	IOC pointer for FIFO B1 7
X:0x71F8	D_FIFO_B1_7_BLOCK_SIZE	(9:0) BLOCK_SIZE	(9:0) BLOCK_SIZE	0x0	Block size for FIFO B1 7
X:0x71FC	D_FIFO_B1_7_BLOCK_CNT	(9:0) BLOCK_CNT	(9:0) BLOCK_CNT	0x0	Block counter for FIFO B1 7
X:0x71FE	D_FIFO_B1_7_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B1 7

A.53 CFX FIFO INTERRUPT SELECTION

Address	Register Name	Register Write	Register Read	Default	Description
X:0x7200	D_FIFO_INT_CFX_0	(4:0) SELECT_INT	(4:0) SELECT_INT	0x0	Select the source for a CFX FIFO interrupt
X:0x7201	D_FIFO_INT_CFX_1	(4:0) SELECT_INT	(4:0) SELECT_INT	0x0	Select the source for a CFX FIFO interrupt
X:0x7202	D_FIFO_INT_CFX_2	(4:0) SELECT_INT	(4:0) SELECT_INT	0x0	Select the source for a CFX FIFO interrupt
X:0x7203	D_FIFO_INT_CFX_3	(4:0) SELECT_INT	(4:0) SELECT_INT	0x0	Select the source for a CFX FIFO interrupt
X:0x7204	D_FIFO_INT_CFX_4	(4:0) SELECT_INT	(4:0) SELECT_INT	0x0	Select the source for a CFX FIFO interrupt
X:0x7205	D_FIFO_INT_CFX_5	(4:0) SELECT_INT	(4:0) SELECT_INT	0x0	Select the source for a CFX FIFO interrupt
X:0x7206	D_FIFO_INT_CFX_6	(4:0) SELECT_INT	(4:0) SELECT_INT	0x0	Select the source for a CFX FIFO interrupt
X:0x7207	D_FIFO_INT_CFX_7	(4:0) SELECT_INT	(4:0) SELECT_INT	0x0	Select the source for a CFX FIFO interrupt

A.54 HEAR FIFO INTERRUPT SELECTION

Address	Register Name	Register Write	Register Read	Default	Description
X:0x7210	D_FIFO_INT_HEAR_0	(4:0) SELECT_INT	(4:0) SELECT_INT	0x0	Select the source for a HEAR FIFO event
X:0x7211	D_FIFO_INT_HEAR_1	(4:0) SELECT_INT	(4:0) SELECT_INT	0x0	Select the source for a HEAR FIFO event
X:0x7212	D_FIFO_INT_HEAR_2	(4:0) SELECT_INT	(4:0) SELECT_INT	0x0	Select the source for a HEAR FIFO event
X:0x7213	D_FIFO_INT_HEAR_3	(4:0) SELECT_INT	(4:0) SELECT_INT	0x0	Select the source for a HEAR FIFO event
X:0x7214	D_FIFO_INT_HEAR_4	(4:0) SELECT_INT	(4:0) SELECT_INT	0x0	Select the source for a HEAR FIFO event
X:0x7215	D_FIFO_INT_HEAR_5	(4:0) SELECT_INT	(4:0) SELECT_INT	0x0	Select the source for a HEAR FIFO event
X:0x7216	D_FIFO_INT_HEAR_6	(4:0) SELECT_INT	(4:0) SELECT_INT	0x0	Select the source for a HEAR FIFO event
X:0x7217	D_FIFO_INT_HEAR_7	(4:0) SELECT_INT	(4:0) SELECT_INT	0x0	Select the source for a HEAR FIFO event

A.55 ARM CORTEX-M3 CORE FIFO INTERRUPT SELECTION

Address	Register Name	Register Write	Register Read	Default	Description
X:0x7220	D_FIFO_INT_CM3_0	(4:0) SELECT_INT	(4:0) SELECT_INT	0x0	Select the source for an ARM Cortex-M3 core FIFO interrupt
X:0x7221	D_FIFO_INT_CM3_1	(4:0) SELECT_INT	(4:0) SELECT_INT	0x0	Select the source for an ARM Cortex-M3 core FIFO interrupt
X:0x7222	D_FIFO_INT_CM3_2	(4:0) SELECT_INT	(4:0) SELECT_INT	0x0	Select the source for an ARM Cortex-M3 core FIFO interrupt
X:0x7223	D_FIFO_INT_CM3_3	(4:0) SELECT_INT	(4:0) SELECT_INT	0x0	Select the source for an ARM Cortex-M3 core FIFO interrupt
X:0x7224	D_FIFO_INT_CM3_4	(4:0) SELECT_INT	(4:0) SELECT_INT	0x0	Select the source for an ARM Cortex-M3 core FIFO interrupt
X:0x7225	D_FIFO_INT_CM3_5	(4:0) SELECT_INT	(4:0) SELECT_INT	0x0	Select the source for an ARM Cortex-M3 core FIFO interrupt
X:0x7226	D_FIFO_INT_CM3_6	(4:0) SELECT_INT	(4:0) SELECT_INT	0x0	Select the source for an ARM Cortex-M3 core FIFO interrupt
X:0x7227	D_FIFO_INT_CM3_7	(4:0) SELECT_INT	(4:0) SELECT_INT	0x0	Select the source for an ARM Cortex-M3 core FIFO interrupt

A.56 HEAR INTERRUPT CONFIGURATION AND CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
X:0x7230	D_HEAR_FC_ENABLE	(23) FIFO_7	(23) FIFO_7	0x0	Enable/disable HEAR FIFO function chain 7
		(22) FIFO_6	(22) FIFO_6	0x0	Enable/disable HEAR FIFO function chain 6
		(21) FIFO_5	(21) FIFO_5	0x0	Enable/disable HEAR FIFO function chain 5
		(20) FIFO_4	(20) FIFO_4	0x0	Enable/disable HEAR FIFO function chain 4
		(19) FIFO_3	(19) FIFO_3	0x0	Enable/disable HEAR FIFO function chain 3
		(18) FIFO_2	(18) FIFO_2	0x0	Enable/disable HEAR FIFO function chain 2
		(17) FIFO_1	(17) FIFO_1	0x0	Enable/disable HEAR FIFO function chain 1
		(16) FIFO_0	(16) FIFO_0	0x0	Enable/disable HEAR FIFO function chain 0
		(15) CMD15	(15) CMD15	0x0	Enable/disable CFX function chain 15
		(14) CMD14	(14) CMD14	0x0	Enable/disable CFX function chain 14
		(13) CMD13	(13) CMD13	0x0	Enable/disable CFX function chain 13
		(12) CMD12	(12) CMD12	0x0	Enable/disable CFX function chain 12
		(11) CMD11	(11) CMD11	0x0	Enable/disable CFX function chain 11
		(10) CMD10	(10) CMD10	0x0	Enable/disable CFX function chain 10
		(9) CMD9	(9) CMD9	0x0	Enable/disable CFX function chain 9
		(8) CMD8	(8) CMD8	0x0	Enable/disable CFX function chain 8
		(7) CMD7	(7) CMD7	0x0	Enable/disable CFX function chain 7
		(6) CMD6	(6) CMD6	0x0	Enable/disable CFX function chain 6
		(5) CMD5	(5) CMD5	0x0	Enable/disable CFX function chain 5
		(4) CMD4	(4) CMD4	0x0	Enable/disable CFX function chain 4
		(3) CMD3	(3) CMD3	0x0	Enable/disable CFX function chain 3
		(2) CMD2	(2) CMD2	0x0	Enable/disable CFX function chain 2
		(1) CMD1	(1) CMD1	0x0	Enable/disable CFX function chain 1
		(0) CMD0	(0) CMD0	0x0	Enable/disable CFX function chain 0

Address	Register Name	Register Write	Register Read	Default	Description
X:0x7231	D_HEAR_FC_PRIORITY	(23) FIFO_7	(23) FIFO_7	0x0	Prioritize the HEAR FIFO function chain 7
		(22) FIFO_6	(22) FIFO_6	0x0	Prioritize the HEAR FIFO function chain 6
		(21) FIFO_5	(21) FIFO_5	0x0	Prioritize the HEAR FIFO function chain 5
		(20) FIFO_4	(20) FIFO_4	0x0	Prioritize the HEAR FIFO function chain 4
		(19) FIFO_3	(19) FIFO_3	0x0	Prioritize the HEAR FIFO function chain 3
		(18) FIFO_2	(18) FIFO_2	0x0	Prioritize the HEAR FIFO function chain 2
		(17) FIFO_1	(17) FIFO_1	0x0	Prioritize the HEAR FIFO function chain 1
		(16) FIFO_0	(16) FIFO_0	0x0	Prioritize the HEAR FIFO function chain 0
		(15) CFX_15	(15) CFX_15	0x0	Prioritize the CFX function chain 15
		(14) CFX_14	(14) CFX_14	0x0	Prioritize the CFX function chain 14
		(13) CMD13	(13) CMD13	0x0	Prioritize the CFX function chain 13
		(12) CMD12	(12) CMD12	0x0	Prioritize the CFX function chain 12
		(11) CMD11	(11) CMD11	0x0	Prioritize the CFX function chain 11
		(10) CMD10	(10) CMD10	0x0	Prioritize the CFX function chain 10
		(9) CMD9	(9) CMD9	0x0	Prioritize the CFX function chain 9
		(8) CMD8	(8) CMD8	0x0	Prioritize the CFX function chain 8
		(7) CMD7	(7) CMD7	0x0	Prioritize the CFX function chain 7
		(6) CMD6	(6) CMD6	0x0	Prioritize the CFX function chain 6
		(5) CMD5	(5) CMD5	0x0	Prioritize the CFX function chain 5
		(4) CMD4	(4) CMD4	0x0	Prioritize the CFX function chain 4
		(3) CMD3	(3) CMD3	0x0	Prioritize the CFX function chain 3
		(2) CMD2	(2) CMD2	0x0	Prioritize the CFX function chain 2
		(1) CMD1	(1) CMD1	0x0	Prioritize the CFX function chain 1
		(0) CMD0	(0) CMD0	0x0	Prioritize the CFX function chain 0

Address	Register Name	Register Write	Register Read	Default	Description
X:0x7237	D_HEAR_FC_STATUS	-	(23) FIFO_HEAR_7_PENDING	0x0	Read the HEAR FIFO pending function chain 7 status
		(23) FIFO_HEAR_7_CLEAR	-	N/A	Clear the HEAR FIFO pending function chain 7 status
		-	(22) FIFO_HEAR_6_PENDING	0x0	Read the HEAR FIFO pending function chain 6 status
		(22) FIFO_HEAR_6_CLEAR	-	N/A	Clear the HEAR FIFO pending function chain 6 status
		-	(21) FIFO_HEAR_5_PENDING	0x0	Read the HEAR FIFO pending function chain 5 status
		(21) FIFO_HEAR_5_CLEAR	-	N/A	Clear the HEAR FIFO pending function chain 5 status
		-	(20) FIFO_HEAR_4_PENDING	0x0	Read the HEAR FIFO pending function chain 4 status
		(20) FIFO_HEAR_4_CLEAR	-	N/A	Clear the HEAR FIFO pending function chain 4 status
		-	(19) FIFO_HEAR_3_PENDING	0x0	Read the HEAR FIFO pending function chain 3 status
		(19) FIFO_HEAR_3_CLEAR	-	N/A	Clear the HEAR FIFO pending function chain 3 status
		-	(18) FIFO_HEAR_2_PENDING	0x0	Read the HEAR FIFO pending function chain 2 status

Address	Register Name	Register Write	Register Read	Default	Description
X:0x7237 (continued)	D_HEAR_FC_STATUS (continued)	(18) FIFO_HEAR_2_CLEAR	-	N/A	Clear the HEAR FIFO pending function chain 2 status
		-	(17) FIFO_HEAR_1_PENDING	0x0	Read the HEAR FIFO pending function chain 1 status
		(17) FIFO_HEAR_1_CLEAR	-	N/A	Clear the HEAR FIFO pending function chain 1 status
		-	(16) FIFO_HEAR_0_PENDING	0x0	Read the HEAR FIFO pending function chain 0 status
		(16) FIFO_HEAR_0_CLEAR	-	N/A	Clear the HEAR FIFO pending function chain 0 status
		-	(15) CFX_15_PENDING	0x0	Read the CFX pending function chain 15 status
		(15) CFX_15_CLEAR	-	N/A	Clear the CFX pending function chain 15 status
		-	(14) CFX_14_PENDING	0x0	Read the CFX pending function chain 14 status
		(14) CFX_14_CLEAR	-	N/A	Clear the CFX pending function chain 14 status
		-	(13) CFX_13_PENDING	0x0	Read the CFX pending function chain 13 status
		(13) CFX_13_CLEAR	-	N/A	Clear the CFX pending function chain 13 status
		-	(12) CFX_12_PENDING	0x0	Read the CFX pending function chain 12 status
		(12) CFX_12_CLEAR	-	N/A	Clear the CFX pending function chain 12 status
		-	(11) CFX_11_PENDING	0x0	Read the CFX pending function chain 11 status
		(11) CFX_11_CLEAR	-	N/A	Clear the CFX pending function chain 11 status
		-	(10) CFX_10_PENDING	0x0	Read the CFX pending function chain 10 status
		(10) CFX_10_CLEAR	-	N/A	Clear the CFX pending function chain 10 status
		-	(9) CFX_9_PENDING	0x0	Read the CFX pending function chain 9 status
		(9) CFX_9_CLEAR	-	N/A	Clear the CFX pending function chain 9 status
		-	(8) CFX_8_PENDING	0x0	Read the CFX pending function chain 8 status
		(8) CFX_8_CLEAR	-	N/A	Clear the CFX pending function chain 8 status
		-	(7) CFX_7_PENDING	0x0	Read the CFX pending function chain 7 status
		(7) CFX_7_CLEAR	-	N/A	Clear the CFX pending function chain 7 status
		-	(6) CFX_6_PENDING	0x0	Read the CFX pending function chain 6 status
		(6) CFX_6_CLEAR	-	N/A	Clear the CFX pending function chain 6 status
		-	(5) CFX_5_PENDING	0x0	Read the CFX pending function chain 5 status
		(5) CFX_5_CLEAR	-	N/A	Clear the CFX pending function chain 5 status
		-	(4) CFX_4_PENDING	0x0	Read the CFX pending function chain 4 status
		(4) CFX_4_CLEAR	-	N/A	Clear the CFX pending function chain 4 status
		-	(3) CFX_3_PENDING	0x0	Read the CFX pending function chain 3 status
		(3) CFX_3_CLEAR	-	N/A	Clear the CFX pending function chain 3 status
		-	(2) CFX_2_PENDING	0x0	Read the CFX pending function chain 2 status
		(2) CFX_2_CLEAR	-	N/A	Clear the CFX pending function chain 2 status

A.57 BLOCK FLOATING POINT CONFIGURATION AND CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
X:0x7260	D_HEAR_BFP	(21:16) BLOCK_EXP	(21:16) BLOCK_EXP	0x0	Stored block exponent for the data
		(15) FMD_MODE	(15) FMD_MODE	0x0	Enable/disable FMD mode
		(12:8) BLOCK_MAX_MAG	(12:8) BLOCK_MAX_MAG	0x0	Maximum magnitude for a block of data
		(5:4) SCALE_FACTOR	(5:4) SCALE_FACTOR	0x0	Select a scaling factor for BFP
		(1:0) FORMAT	(1:0) FORMAT	0x0	Select a format based on the number of bits
X:0x7261	D_HEAR_BFP_BLOCK_EXP	(5:0) HEAR_BFP_BLOCK_EXP	(5:0) HEAR_BFP_BLOCK_EXP	0x0	Mirror of the block exponent for the data

APPENDIX B
Sampling Frequency Reference

Table 77 is a lookup table that provides a summary of the possible sampling frequencies for a set of common SLOWCLK frequencies. All sampling frequency values are provided in Hz.

Table 77. Summary of Possible Sampling Frequencies

Register Setting For SAMPLE_FREQ from A_ADC_SAMPLE_FREQ_CFG	Desired SLOWCLK Frequency:			
	640kHz	1.28MHz	1.92MHz	2.56MHz
0x00	10000.00	20000.00	30000.00	40000.00
0x01	8888.89	17777.78	26666.67	35555.56
0x02	8000.00	16000.00	24000.00	32000.00
0x03	7272.73	14545.45	21818.18	29090.91
0x04	6666.67	13333.33	20000.00	26666.67
0x05	6153.85	12307.69	18461.54	24615.38
0x06	5714.29	11428.57	17142.86	22857.14
0x07	5333.33	10666.67	16000.00	21333.33
0x08	5000.00	10000.00	15000.00	20000.00
0x09	4705.88	9411.76	14117.65	18823.53
0x0A	4444.44	8888.89	13333.33	17777.78
0x0B	4210.53	8421.05	12631.58	16842.11
0x0C	4000.00	8000.00	12000.00	16000.00
0x0D	3809.52	7619.05	11428.57	15238.10
0x0E	3636.36	7272.73	10909.09	14545.45
0x0F	3478.26	6956.52	10434.78	13913.04
0x10	3333.33	6666.67	10000.00	13333.33
0x11	3200.00	6400.00	9600.00	12800.00
0x12	3076.92	6153.85	9230.77	12307.69
0x13	2962.96	5925.93	8888.89	11851.85
0x14	2857.14	5714.29	8571.43	11428.57
0x15	2758.62	5517.24	8275.86	11034.48
0x16	2666.67	5333.33	8000.00	10666.67
0x17	2580.65	5161.29	7741.94	10322.58
0x18	2500.00	5000.00	7500.00	10000.00
0x19	2424.24	4848.48	7272.73	9696.97
0x1A	2352.94	4705.88	7058.82	9411.76
0x1B	2285.71	4571.43	6857.14	9142.86
0x1C	2222.22	4444.44	6666.67	8888.89
0x1D	2162.16	4324.32	6486.49	8648.65
0x1E	2105.26	4210.53	6315.79	8421.05
0x1F	2051.28	4102.56	6153.85	8205.13
0x20	2000.00	4000.00	6000.00	8000.00
0x21	1951.22	3902.44	5853.66	7804.88

Table 77. Summary of Possible Sampling Frequencies (Continued)

Register Setting For SAMPLE_FREQ from A_ADC_SAMPLE_FREQ_CFG	Desired SLOWCLK Frequency:			
	640kHz	1.28MHz	1.92MHz	2.56MHz
0x22	1904.76	3809.52	5714.29	7619.05
0x23	1860.47	3720.93	5581.40	7441.86
0x24	1818.18	3636.36	5454.55	7272.73
0x25	1777.78	3555.56	5333.33	7111.11
0x26	1739.13	3478.26	5217.39	6956.52
0x27	1702.13	3404.26	5106.38	6808.51
0x28	1666.67	3333.33	5000.00	6666.67
0x29	1632.65	3265.31	4897.96	6530.61
0x2A	1600.00	3200.00	4800.00	6400.00
0x2B	1568.63	3137.25	4705.88	6274.51
0x2C	1538.46	3076.92	4615.38	6153.85
0x2D	1509.43	3018.87	4528.30	6037.74
0x2E	1481.48	2962.96	4444.44	5925.93
0x2F	1454.55	2909.09	4363.64	5818.18
0x30	1428.57	2857.14	4285.71	5714.29
0x31	1403.51	2807.02	4210.53	5614.04
0x32	1379.31	2758.62	4137.93	5517.24
0x33	1355.93	2711.86	4067.80	5423.73
0x34	1333.33	2666.67	4000.00	5333.33
0x35	1311.48	2622.95	3934.43	5245.90
0x36	1290.32	2580.65	3870.97	5161.29
> 0x36	1269.84	2539.68	3809.52	5079.37

APPENDIX C
Glossary

The following abbreviations and terms are used in this manual:

breakpoints, external
: These breakpoints allow the configuration of a breakpoint event based on accesses to data and the value of the written data.

breakpoints, hardware
: These breakpoints are configured as pointers to P-Memory. They are triggered when the instruction pointed to by the hardware breakpoint is issued for CFX processor execution.

breakpoints, software
: These breakpoints are included in code for the CFX processor as `breakpoint` instructions.

CFX Digital Signal Processor
: The Ezairo 7100 processing core that is used to configure the system and coordinate the flow of signal data progressing through the system

CID
: chip identifier

CMSIS
: Cortex Microcontroller Software Interface Standard

CRC
: cyclic redundancy code

DIO
: Digital Input/Output

FIFO
: A section of memory that is treated as a circular buffer where components that use this memory use pointers to handle data in a first-in, first-out manner. A *hardware FIFO* is a FIFO instance where its registers are controlled by the IOC. A *software FIFO* is a FIFO instance where its registers are controlled by an application on the CFX or microcode configuration on the HEAR.

GPIO
: general-purpose input/output

HEAR Configurable Accelerator
: The Ezairo 7100 core that is a microcode configurable signal processing engine used to execute a variety of signal processing functions; designed to work with the CFX to reduce the signal processing load on the CFX

I^2C
: Industry standardized Inter-IC Communication protocol.

I^2S
: industry standardized inter-IC sound protocol

IOC
: input/output controller

LDP
: low-delay path

LSAD
: low-speed analog-to-digital

LSB
: least significant bit

LUT
: look up table

memory block instance
: an instance of RAM or ROM

memory bus
: a communication system that transfers data between memory or memory-mapped components and the cores that operate on that data

memory space
: a mapping of memory and memory-mapped items to a memory address; synonym for address space

MSB
: most significant bit

NVM
: non-volatile memory

PCM
: pulse code modulation

PLL
: phase-locked loop

POR
: power-on reset

QMF
: quadrature mirror filter

RAM
: random-access memory

ROM
: read-only memory

SB-ADPCM
: subband adaptive pulse code modulation

SCL
: serial clock signal (part of I^2C bus)

SDA
: serial data signal (part of I^2C bus)

SDU
: signal detection unit

SPI
: serial peripheral interface

UART
: universal asynchronous receiver-transmitter

VCO
: voltage-controlled oscillator

ON Semiconductor and are registered trademarks of Semiconductor Components Industries, LLC (SCILLC). ARM, the ARM logo and Cortex are trademarks or registered trademarks of ARM Ltd. Ezairo is either a trademark or registered trademark of SCILLC. SCILLC owns the rights to a number of patents, trademarks, copyrights, trade secrets, and other intellectual property. A listing of SCILLC's product/patent coverage may be accessed at www.onsemi.com/site/pdf/Patent-Marking.pdf. SCILLC reserves the right to make changes without further notice to any products herein. SCILLC makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does SCILLC assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. "Typical" parameters which may be provided in SCILLC data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. SCILLC does not convey any license under its patent rights nor the rights of others. SCILLC products are not designed, intended, or authorized for use as components in systems intended for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the SCILLC product could create a situation where personal injury or death may occur. Should Buyer purchase or use SCILLC products for any such unintended or unauthorized application, Buyer shall indemnify and hold SCILLC and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that SCILLC was negligent regarding the design or manufacture of the part. SCILLC is an Equal Opportunity/ Affirmative Action Employer. This literature is subject to all applicable copyright laws and is not for resale in any manner.

PUBLICATION ORDERING INFORMATION

LITERATURE FULFILLMENT:
Literature Distribution Center for ON Semiconductor
P.O. Box 5163, Denver, Colorado 80217 USA
Phone: 303-675-2175 or 800-344-3860 Toll Free USA/Canada
Fax: 303-675-2176 or 800-344-3867 Toll Free USA/Canada
Email: orderlit@onsemi.com

N. American Technical Support: 800-282-9855 Toll Free USA/Canada
Europe, Middle East and Africa Technical Support: Phone: 421 33 790 2910
Japan Customer Focus Center: Phone: 81-3-5817-1050

ON Semiconductor Website: www.onsemi.com

Order Literature: http://www.onsemi.com/orderlit

For additional information, please contact your local Sales Representative

M-20786-022